题　目　吉隆灾害链的环境特征、触发条件、预警与仿真

摘　要：

2026 年 8 月 26 日吉隆口岸受冰崩—碎屑流—泥石流—山洪链式灾害冲击。本文结合环境观测、事件运动与地形资料，依次研究灾前环境异常、局部起始条件、首达时间及规定入口后的传播。

针对问题一，结合站点逐小时观测与 ERA5 格点总降水，建立**年度等权多尺度历史校准**：在 1、3、7、15 和 30 日窗口构造特征，以历史参考—校准配置比较最大异常，并用主成分分析检验联合结构。从 17 项候选中识别格点累计总降水、2 m 相对湿度、20 m 减 2 m 气温差、10 m 标量风速和格点最大小时总降水量，校准位置依次为 0.902、0.869、0.854、0.852 和 0.845；五项在逐年剔除历史年的五次比较中均保留。30 日累计降水偏高，末 24—72 h 偏低。

针对问题二，将湿细粒物与孔隙水理想化为饱和材料柱，依据固体守恒、水量平衡与剪胀关系建立**剪胀—排水—孔压耦合动力系统**，以平均有效法向应力刻画孔压反馈。二次压力剖面给出支承层起始判据：平均有效法向应力降至浮重尺度的 1/6；在 73 s 速度窗内反求初始应力比 0.17、0.30、0.50 和 0.75 的临界累计剪切，依次为 0.0188、0.520、0.981 和 1.35，并分析排水及压力剖面影响。

针对问题三，基于问题二的局部边界，构建**触发时间预测与条件预警**模型，以多分布稳健预测集合同时表达窗内首达时间与未达类别。初始应力比 0.50 时，相对选定计算窗起点，强施剪稳健 95% 预测区间为 [0.775, 0.815] s；近临界稳健 95% 预测集合由有限区间 [71.40, 72.95] s 与窗内未达类别组成；三种速度偏移分布的模拟检验中，两条件最低单侧下界分别为 0.970003 和 0.956011。近临界下假设发布延迟 1 s、行动历时 15 s，有限局部余量为 [55.40, 56.95] s。

针对问题四，以规定的已形成流入口为相对时间零点，建立**二维守恒浅流传播模型**，并以成本感知多保真高斯过程主动学习研究局部到达边界。基准情景中局部断面约 731.8 s 首次达到 0.1 m；四个完整河段情景的口岸断面在 3600 s 内未达，窗末保持运动。14 次高层模拟在两条体积切片上得到摩擦系数括区 [0.06000, 0.06375] 和 [0.07875, 0.08250]；采用**代理预估与物理核验**后发布到达状态。

关键词：泥石流灾害链；多尺度异常分析；剪胀—孔压耦合；触发时间预测与条件预警；多保真高斯过程；二维浅流仿真

目录

一、 问题重述 1

1.1 问题背景 1

1.2 问题的提出 1

二、 问题分析 1

2.1 问题分析 1

三、 模型假设 2

四、 符号说明 3

五、 模型建立与求解 3

5.1 问题一 3

5.2 问题二 12

5.3 问题三 17

5.4 问题四 22

六、 模型评价、改进与推广 26

6.1 模型的优点 26

6.2 模型的缺点 26

6.3 模型的改进与推广 26

七、 参考文献 27

八、 附录 29

一、 问题重述

1.1 问题背景

泥石流是暴雨、冰雪融水等水源激发下，在沟谷或山坡上形成的挟带泥砂、石块和巨砾的特殊洪流。题面指出，其固体碎屑物体积含量最低为 15%、最高约为 80%，破坏性远大于洪水 [1]。题目以 2026 年 8 月 26 日吉隆口岸受灾事件为背景，专家研判其为尼泊尔境内高位冰崩引发的“冰崩—碎屑流—泥石流—山洪”链式灾害。结合环境观测与地理资料，需研究灾前环境特征、触发条件、预报和灾害仿真。

1.2 问题的提出

（1）基于气象梯度、辐射、土壤与湍流等资料，分析吉隆县及周边高寒山区大气边界层结构、天气过程及极端气象事件与泥石流关联的特征，对候选因素的关联强度排序，给出关联性最高的五个具体特征因素。

（2）自行选取并补充卫星遥感、天气预报等资料，依据泥石流的能量、物质与触发条件，研究触发关键参数的临界条件，构建触发评估模型，并从时间和空间尺度讨论模型可靠性。

（3）构建吉隆泥石流预报模型，给出泥石流触发时间的 95% 置信区间，以及预警时间、区间覆盖率与空间距离范围。

（4）构建吉隆泥石流发生的仿真环境，设定明确的关键参数，给出仿真过程及可视化结果。

二、 问题分析

2.1 问题分析

本题以链式灾害为背景，涉及灾前环境、触发、预报与仿真等相互关联的任务。问题一使用吉隆综合观测站的气象梯度（MET）、辐射（RADM）、土壤（SOIL）和湍流（FLUX）逐小时记录 [2]，并补充固定 ERA5 格点的小时总降水 [3]；站点与格点均用于区域环境背景分析。

针对问题一，分析站点温湿、风、土壤水热及格点降水过程，将截止前 1、3、7、15、30 日窗口与往年同期比较。采用年度等权和留出年校准，得到各因素的多尺度历史位置，再以主成分分析考察多变量联合变化。

针对问题二，利用事件研究的坡度与反演速度、细粒材料及渗透率资料，建立局部饱和材料柱的剪切—排水—孔压模型；以平均有效应力判据确定孔压支承层起始边界，求解不同初态的剪切传递系数和累计剪切。

针对问题三，以问题二的孔压支承层起始边界为首达事件，构建速度偏移下的分段时间映射与多分布稳健预测集合；结合随机模拟检验、初态敏感性及发布和行动延迟，计算局部与理想坡面固定目标的条件余量。

针对问题四，以地形和规定入流为条件，建立水平二维固定床浅流模型，计算局部断面过程和完整河段覆盖；以参数筛选和多保真代理比较到达边界判断。

四问分析流程如图 1-1所示。

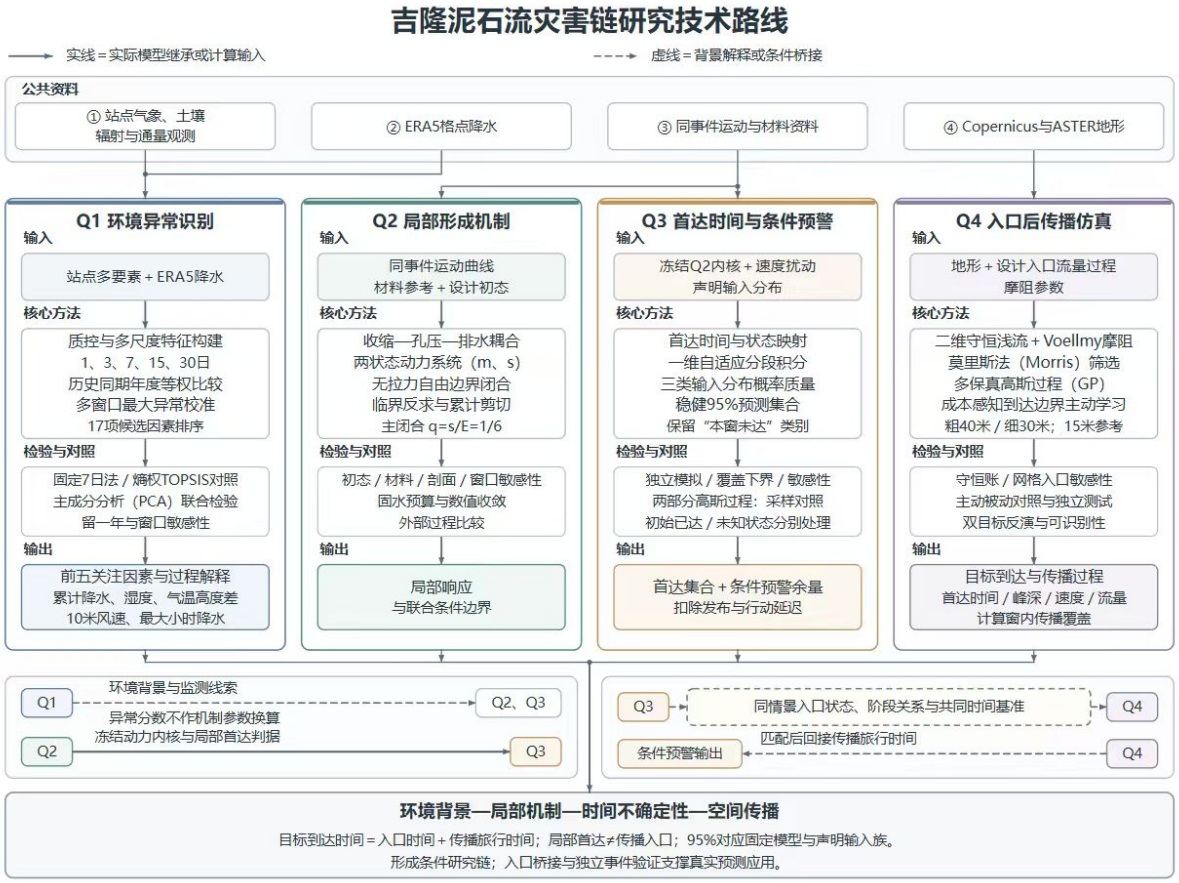


图 1-1　四问研究技术路线（含补充检验）

三、 模型假设

1. 问题一以北京时间 2026 年 8 月 26 日 10:00 为回溯截止时刻 t_0。站点主分析按标签 t 对应 $[t-1\text{h}, t)$ 的小时均值处理，并比较 $[t, t+1\text{h})$ 及当日 00:00 截止的情景。ERA5 总降水采用标签之前一小时的水当量。
2. 2021—2026 年同名观测变量采用相同对象、单位和统计方式。
3. 2021—2025 年同季、同钟点记录作为历史背景，各年赋予相同权重。
4. 问题二将局部细粒材料理想化为饱和柱，固定底面积和固体体积，沿坡面法线取厚度平均状态，顶面可交换液水。
5. 问题三沿用问题二材料柱与二次压力剖面，以选定速度窗起点设置初态；假设整条速度曲线受同一偏移，传播示例采用理想常坡和首达到入口的零阶段延时。
6. 问题四以规定的已形成流进入断面为时间零点，采用常密度固定床、初始干床和设计入流。

问题一以单次事件为对象，采用固定截止时刻的回溯比较。

四、 符号说明

表 1　主要符号及含义

符号	含义
j, w, t_0	因素、窗口长度及目标截止时刻
P_i, P_d, A_d	格点第 i 小时总降水量、截止对齐第 d 日块总降水量及第 d 块前期总降水指数，单位均为 mm
$X_{j,w}(t)$	端点 t 之前窗口 w 内的因素统计量
W_j	因素 j 固定适用的窗口集合
$B_{j,y}, Y_j$	年 y 的合格历史端点集、因素 j 的有效历史年集合
R, h	三参考年集合、留出的校准年
F^{mid}, q	经验中秩分布、年度等权历史分位位置
a, M, S	双侧异常、多窗最大异常、历史校准位置
z, P_k, Λ_k	标准化向量、保留主成分矩阵及特征值矩阵
T^2, Q	保留空间距离、重构残差平方和
m, H, s	问题二的固相体积分数（无量纲）、柱厚（m）及平均有效法向应力（Pa）；问题一的 H 另指感热通量
$q_0, \chi, \Gamma_{\mathrm{sh}}$	问题二初始应力比 s_{init}/E（无量纲）、剪切传递系数（m^{-1}）及累计剪切（无量纲）；E 为浮重法向应力尺度（Pa）
τ_L, d, a	问题三局部首达时间（s）、整条速度曲线的偏移及偏移支持半宽（m/s）
$d_{\mathrm{pub}}, d_{\mathrm{act}}$	窗起点至发布的延迟、发布后的行动历时（s）
$T_{\mathrm{in}}, T_A, L_L, L_A$	传播入口时刻、目标到达时刻、局部及目标行动余量（s）
$h, \mathbf{u}, z$	问题四竖直流深（m）、水平平均流速（m/s）和固定床高（m）

五、 模型建立与求解

5.1 问题一

第一问：结合站点逐小时记录与格点总降水，分析事件前区域环境过程，识别相对历史同期突出的五项特征。

1）模型的准备

采用吉隆综合观测站的气象、土壤、辐射和湍流逐小时记录 [2]。站点位于 28.86°N、85.29°E，海拔约 4140 m，距受影响口岸约 65 km；观测包括 1—20 m 气温与标量风速、2 m 相对湿度及气压，0.1、0.2、0.4、0.8 和 1.6 m 土壤温度（℃）与体积含水量（m³/m³），以及 0.1 m 和 0.2 m 板热流。Rsd 和 Rld 分别为短波与长波辐射原通道，H 和 LE 为感热与潜热通量原通道，单位均为 W/m²；USTAR 为摩擦速度类原通道（m/s）。2021—2026 年小时总降水取 ERA5 同一最近格点 [3]：28.75°N、85.25°E、地形高程 4756 m、约 0.25° 分辨率，关闭高程降尺度；总降水为雨雪水当量。

站点数据保留原时间位置与缺测，有质量标记的变量采用质量标记为 0 的记录，剔除不可解析值、非有限值及缺测标记 9999.9。物理范围为摄氏温度高于 –273.15 ℃、MET 气压为正、土壤体积含水量在 [0,1]；时间标签须可解析、唯一并符合所属年份及小时网格。辐射与通量采用原通道及符号 [4]。

长度为 w 日的回溯窗口覆盖截止前 $24w$ 小时。站点主情景将标签 t 记为 $[t-1\mathrm{h}, t)$ 的小时均值；ERA5 标签为此前一小时总降水，纳入 $t_0 - 24w\mathrm{h} < t \le t_0$ 的记录。末 7 日为 8 月 19 日 10:00 至 8 月 26 日 10:00，末 30 日为 7 月 27 日 10:00 至 8 月 26 日 10:00。

近地层结构与灾前过程

8 月 26 日 09:00—10:00 的小时均温在 1—20 m 由 12.05 ℃ 降至 11.24 ℃；标量风速随高度变化（图 1-2）。20 m 与 2 m 共同有效小时的 30 日平均温差为 –0.497 ℃，较历史同期偏高。

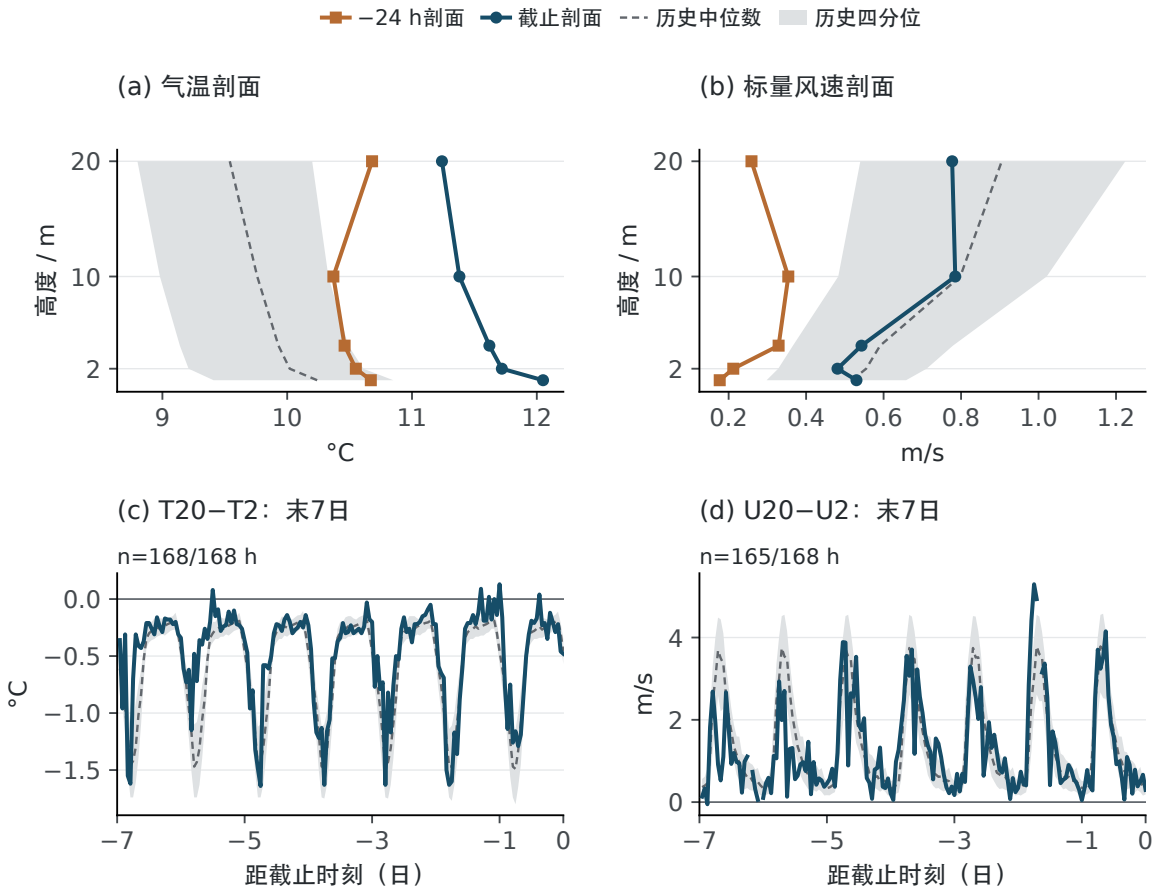


图 **1-2**　近地层剖面与配对层间差。上排为截止时刻和提前 24 h 时刻的两个气温、标量风速剖面，灰带为历史同期、同钟点记录的年度等权四分位范围。下排 T20–T2、U20–U2 分别为20 m 减 2 m的气温差与标量风速差；n 表示两高度同一小时共同有效的小时数 / 应有小时数，缺测保留断线。

长期背景与临近变化并不相同：30 日相对湿润的背景下，末 7 日出现升温和相对湿度下降（图 1-3）。2 m 气温末 7 日均值为 12.51 ℃，首、末 24 h 均值由 11.88 ℃ 升至 13.78 ℃；相对湿度 30 日均值为 71.25%，末 24 h 为 60.49%。末 7 日首末日块中，0.1 m 和 0.2 m 土壤体积含水量均值下降，0.1—0.4 m 土温均值上升，0.8 m 和 1.6 m 土温均值略降。

图 1-3以三角标记逐小时历史分位数：对每个过程小时，汇集 2021—2025 年相应历日前后 15 日的同钟点记录，逐年等权确定第 5 和第 95 百分位；气温分别用两阈值标记偏冷、偏热，风速用第 95 百分位标记相对强风。

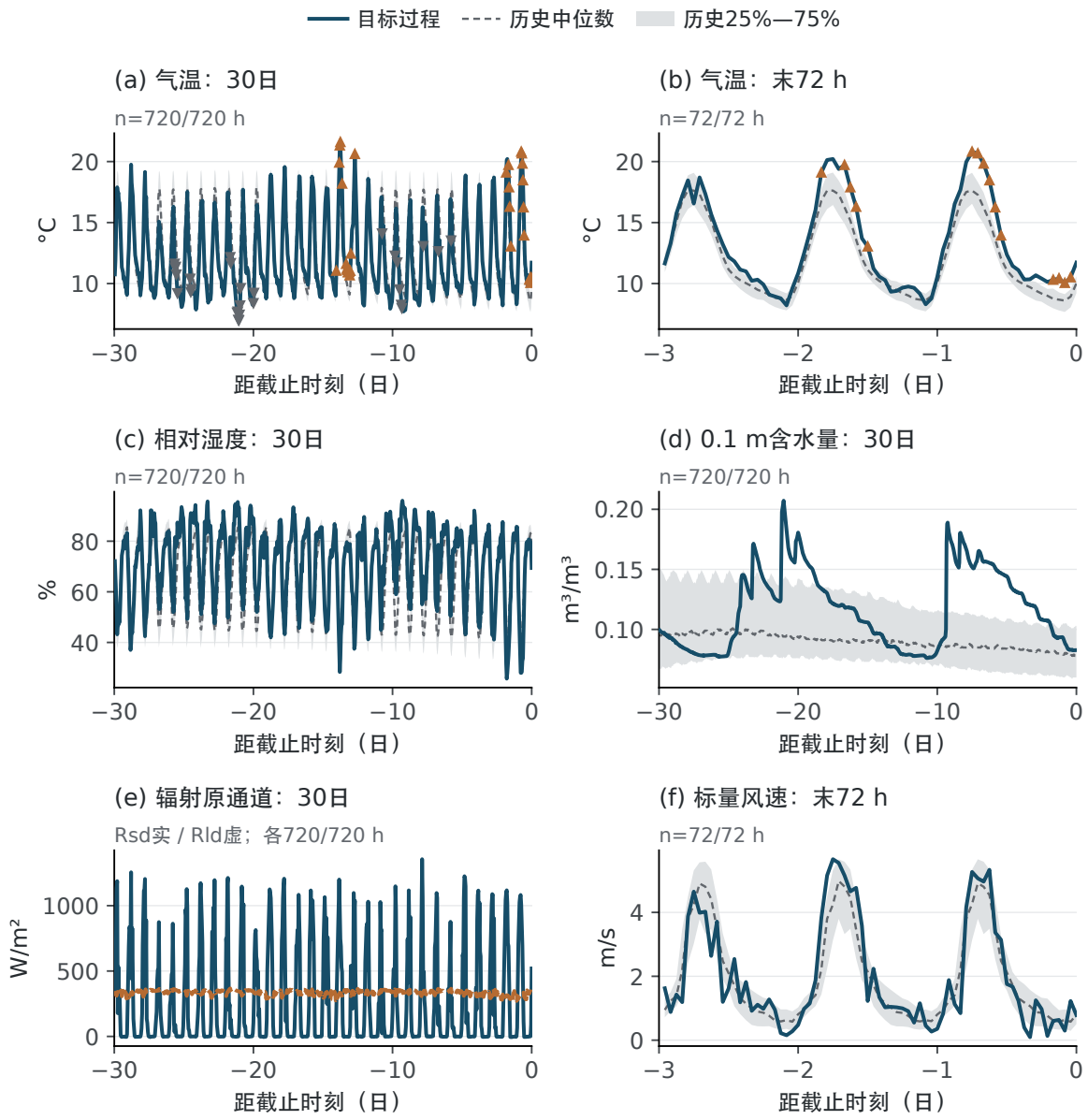


图 **1-3**　灾前天气与水热过程。展示 30 日温湿、水分及辐射过程，并与末 72 h 气温、风速局部对照；历史背景为同期、同钟点的年度等权分布。三角符号标出相对极端的气温或风速时刻；n 表示各通道有效小时数 / 应有小时数。辐射展示原观测通道。

10 m 风速 30 日均值为 1.70 m/s，末 72 h 均值为 2.01 m/s。末 7 日首末 24 h 的 Rsd、Rld、H 和 LE 均值分别变化约 +115.20、−27.66、+21.52 和 −23.25 W/m²，与同期升温及湿度下降共同呈现。

按逐时阈值，30 日内 2 m 气温有 30 小时高于第 95 百分位、28 小时低于第 5 百分位；末 7 日 15 个高温超阈小时均在末 72 h，10 m 风速末 72 h 无高于第 95 百分位的小时。

ERA5 格点 30 日、7 日及末 72 h 累计总降水分别为 195.7、26.9 和 5.1 mm（图 1-4）。逐小时第 95 百分位以 2021—2025 年相应历日前后 15 日同钟点记录按年度等权确定；30 日、7 日及 72 h 分别有 48/720、1/168 和 0/72 小时超过该阈值。

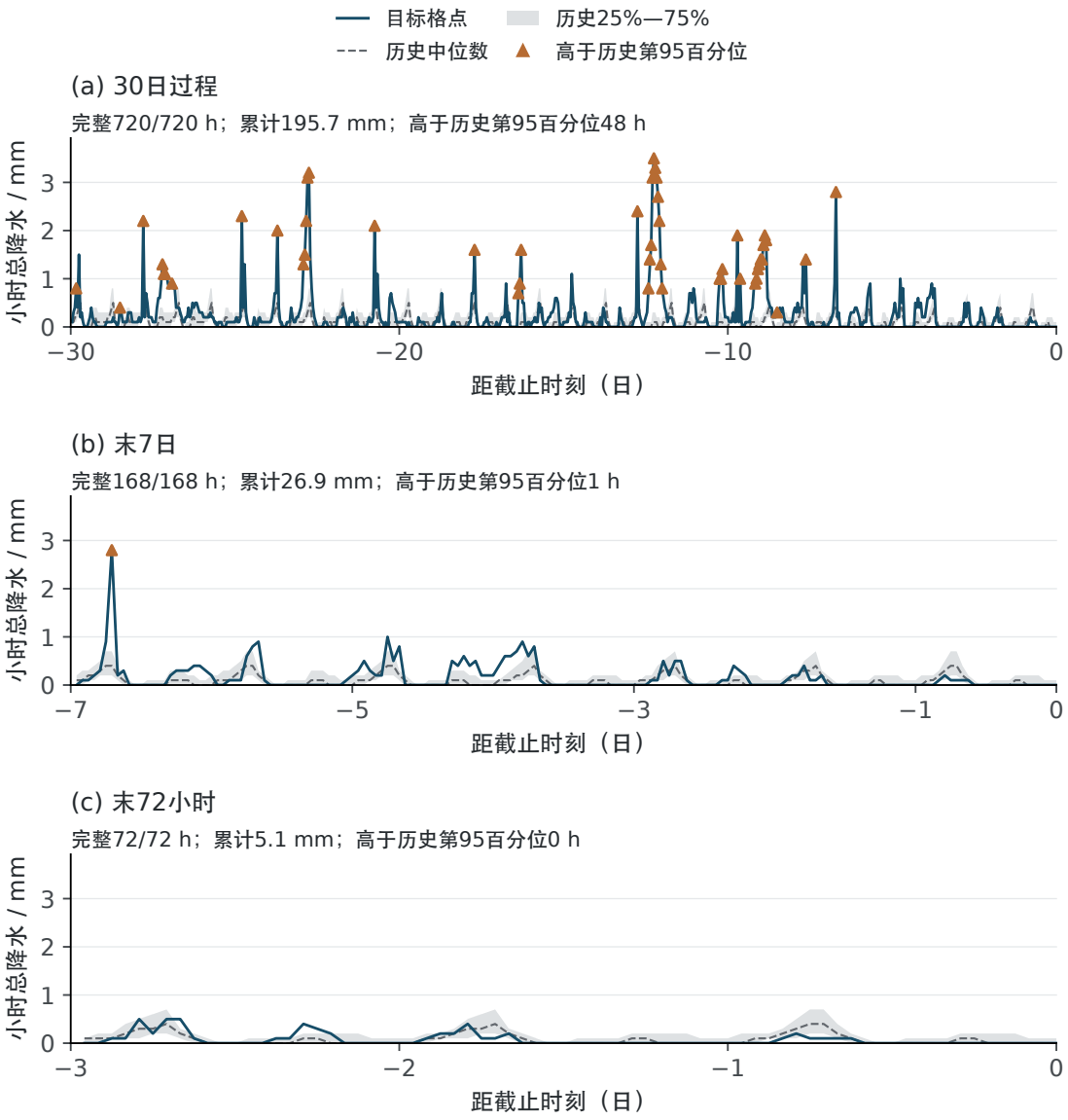


图 **1-4**　ERA5 格点小时总降水过程与历史第 95 百分位。分别展示 30 日、7 日和 72 h；降水为雨雪水当量，阈值由 2021—2025 年记录构造。

表 **1-1**　ERA5 格点总降水五窗口统计量。最大小时量除以 1 h 为小时平均率，有降水小时按单小时总量不小于 0.1 mm 计。

窗口	累计/mm	最大小时总量/mm	有降水小时/h	前期指数/mm
1 日	0.7	0.2	6	0.700
3 日	5.1	0.5	25	4.381
7 日	26.9	2.8	74	17.920
15 日	102.5	3.5	212	43.821
30 日	195.7	3.5	471	53.203

2）模型的建立

因素定义与历史样本

采用 17 项候选（表 1-2），包含格点降水、站点热力、辐射、浅土、层间差及常规天气量。窗口内最大小时均温为 $\max_i T_i$，T_i 为该窗第 i 小时的 2 m 均温。

表 1-2　17 项候选因素的物理分类

物理类别	具体变量、统计量及单位
格点降水（4 项）	累计总降水、最大小时总量、前期总降水指数（mm）；有降水小时数（h）
近地热力（3 项）	正温指数（℃ · h）；最大小时均温（℃）；24 h 块均温变化率（℃/d）
辐射（2 项）	Rsd、Rld 原通道均值（W/m²）
浅土水热（3 项）	0.1 m 体积含水量均值（m³/m³）、土温均值（℃）、板热流均值（W/m²）
近地层结构（2 项）	20 m–2 m 气温差均值（℃）及标量风速差均值（m/s）
常规天气（3 项）	MET 气压均值（hPa）、2 m 相对湿度均值（%）、10 m 标量风速均值（m/s）

均值、PDD 和最大小时均温采用 1、3、7、15、30 日窗口，均温变化率使用 3 日及以上窗口。站点均值要求有效小时达到全窗 90%，PDD、最大小时均温及四项格点降水要求全窗完整。

正温指数 PDD 按 $\sum_i \max(T_i, 0)\Delta t$ 计算，$\Delta t = 1\text{h}$，单位 ℃ · h。均温变化率以截止对齐的 24 h 合格块均温对实际时间线性拟合，斜率单位 ℃/d；主设置要求每块至少 22 小时有效、全窗有效小时和合格块均不少于 90%，至少 3 块且首末块合格。

设 P_i 为 ERA5 前一小时总降水量（mm），四项统计量为累计量 $\sum_i P_i$、最大小时量 $\max_i P_i$、$P_i \geq 0.1$ mm 的小时数及前期总降水指数。后者将窗口分成截止对齐的 24 h 块并按时间由早到晚编号，首块前 $A_0 = 0$，按 $A_d = P_d + 0.9A_{d-1}$ 递推；P_d 为第 d 块总降水，指数单位为 mm，日衰减系数 0.9 无量纲。每个窗口均从零开始；2.8 mm 的小时总量折合小时平均率 2.8 mm/h。

两项高度差先在同一小时的两层记录共同有效时作差，再求窗口均值：

$$\Delta T(t) = T_{20\text{m}}(t) - T_{2\text{m}}(t), \qquad \Delta U(t) = U_{20\text{m}}(t) - U_{2\text{m}}(t). \tag{1}$$

式（1）中 T 和 U 分别为所标高度的气温和标量风速；差值单位分别为 ℃ 和 m/s。

同期背景取 2021—2025 年各年 8 月 26 日前后 15 日、与截止同钟点的 31 个候选端点。对因素 j，端点的全部适用窗口 W_j 均须有效；每年至少保留 25 个端点，且至少 4 年合格，目标采用相同的窗口集合。

年度等权的多尺度历史校准

依次计算实际统计量、单窗历史位置 q、双侧择窗结果 M 与留出年校准位置 S：先将窗口统计值与三参考年同期记录比较，得到 q；再取该因素适用窗口中的最大双侧异常 M；最后用留出历史端点的 M 分布确定目标位置 S。每次留出一年 h 作为**校准年**，从其余有效年中选三年组成参考集合 R，记一组配置为 (h, R)。

记 $B_{j,y}$ 为因素 j 在年 y 的合格端点集，$n_{j,y}$ 为端点数，u 为历史端点，x 为待比较的窗口统计值。指示函数 $\mathbf{1}_{\{C\}}$ 在条件 C 成立时取 1，否则取 0；中秩将小于 x 的端点计 1、等于 x 的端点计 1/2。年内经验分布为

$$F^{\text{mid}}_{j,w,y}(x)=\frac{1}{n_{j,y}}\sum_{u\in B_{j,y}}\left[\mathbf{1}_{\{X_{j,w}(u)<x\}}+\tfrac{1}{2}\mathbf{1}_{\{X_{j,w}(u)=x\}}\right]. \tag{2}$$

对三参考年的年内位置取等权平均，得到端点 t 在窗口 w 上的历史分位位置

$$q_{j,w,R}(t)=\frac{1}{3}\sum_{y\in R}F^{\text{mid}}_{j,w,y}\big(X_{j,w}(t)\big). \tag{3}$$

q 接近 1 表示高于大多数同期记录，接近 0 表示偏低，接近 0.5 表示居中。将 q 距中间位置的程度记为双侧异常 a，在固定窗口集合中取最大值 M：

$$a_{j,w,R}(t)=\left|2q_{j,w,R}(t)-1\right|,\qquad M_{j,R}(t)=\max_{w\in W_j}a_{j,w,R}(t). \tag{4}$$

对目标与留出校准年的所有端点均执行式（4）；并列的最大异常窗口全部保留。

记 $F^{\text{mid}}_{M,j,h,R}$ 为留出年 h 各合格端点的多窗最大异常 M 构成的中秩分布，$\mathcal{R}_{j,h}$ 为该校准年对应的合法三参考年集合，则历史校准位置为

$$S_j=\frac{1}{|Y_j|}\sum_{h\in Y_j}\frac{1}{|\mathcal{R}_{j,h}|}\sum_{R\in\mathcal{R}_{j,h}}F^{\text{mid}}_{M,j,h,R}\big(M_{j,R}(t_0)\big). \tag{5}$$

式中 $|Y_j|$ 为有效校准年数，$|\mathcal{R}_{j,h}|$ 为对应的三参考年集合数。五个有效年形成 20 个参考—校准配置，四年形成 4 个。S 先在同一校准年内对参考集合平均，再对校准年等权平均；S 越大，目标最大异常在历史背景中越突出。

对物理信息重复的候选，采用 7 日窗、至少 4 个共同有效年及每年至少 25 个配对端点；历史 Spearman 秩相关绝对值达到 0.90 时合并。均温与 PDD 的相关为 1 且 S 相同，保留 PDD 代表。

3）模型的求解与分析

关注排序与过程解释

五项关注特征依次为 ERA5 格点累计总降水、2 m 相对湿度均值、20 m–2 m 气温差均值、10 m 标量风速均值及 ERA5 格点最大小时总降水量，S 分别为 0.902、0.869、0.854、0.852 和 0.845（表 1-3）。S 综合全部适用窗口；表中主要窗口为参考—校准配置中达到最大双侧异常频次最高的窗口，并列最大均计数。四项主要窗口为 30 日、最大小时量为 7 日。

表 **1-3**　前五关注特征及主要窗口

关注特征	S	主要窗口	目标值	该窗平均 q 及方向
ERA5 格点累计总降水	0.902	30 日	195.7 mm	0.994，偏高
2 m 相对湿度均值	0.869	30 日	71.25%	1.000，偏高
20 m–2 m 气温差均值	0.854	30 日	–0.497 ℃	1.000，偏高
10 m 标量风速均值	0.852	30 日	1.70 m/s	0.000，偏低
ERA5 格点最大小时总降水量	0.845	7 日	2.8 mm	1.000，偏高

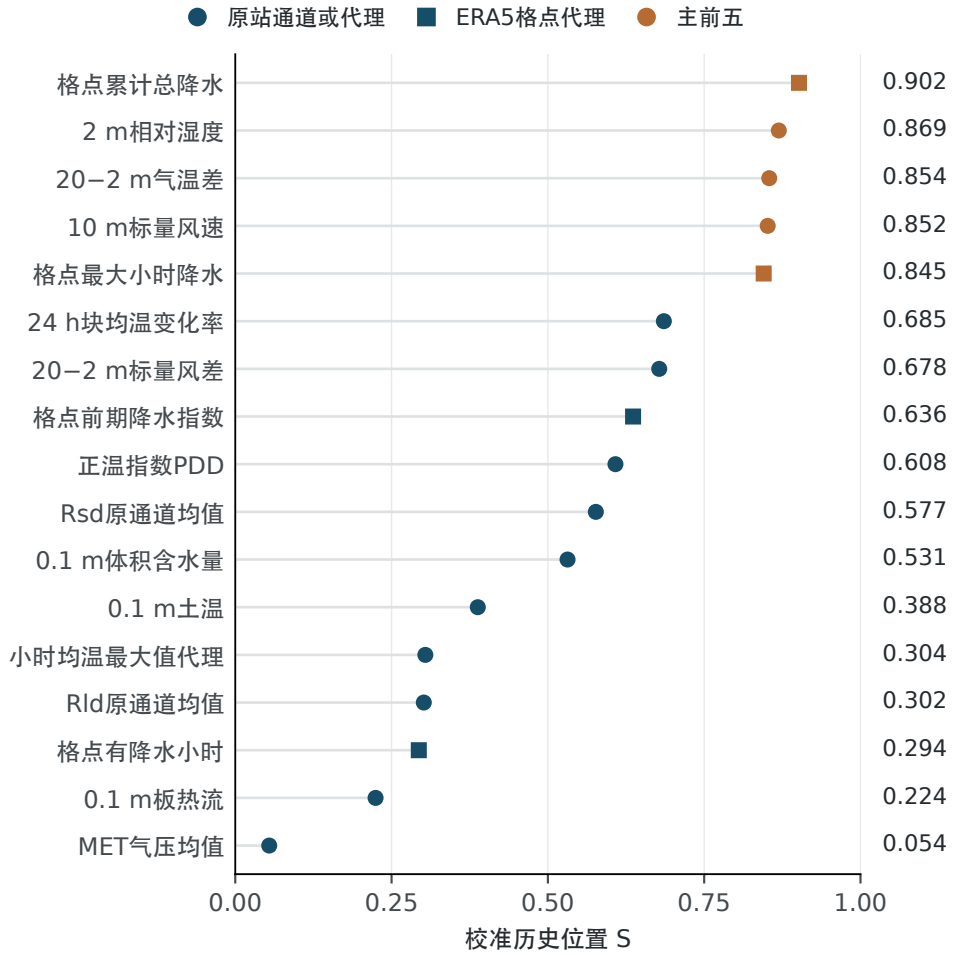


图 **1-5**　17 项候选的历史校准位置。颜色区分前五与其余候选，形状区分站点变量与格点表达；S 为历史校准位置。图中“小时均温最大值代理”对应窗口内最大小时均温。

降水的历史位置随尺度分化：30 日累计 195.7 mm、15 日 102.5 mm 的平均 q 分别为 0.994 和 0.981，均偏高；末 24 h 和 72 h 累计 0.7 mm、5.1 mm，平均 q 为 0.084 和 0.158，均偏低。7 日最大小时量为 2.8 mm，平均 $q = 1$；末 24 h 和 72 h 的最大值分别为 0.2 和 0.5 mm。站点 30 日相对湿度亦偏高，而末 24 h 与 72 h 平均 q 分别为 0.168 和 0.252。

气温高度差 30 日均值为 –0.497 ℃，较历史同期偏高；末 7 日首末 24 h 均值由 –0.429 ℃ 变为 –0.520 ℃。10 m 风速 30 日位置偏低，3 日平均 $q = 0.510$；3 日标量风速高度差的方向随参考配置变化。

均温变化率的 7 日斜率为 +0.349 ℃/d，末 72 h 为 +0.552 ℃/d；7 日单窗双侧异常为 0.897，多窗历史校准位置为 0.685。

敏感性分析

为考察前五特征对分析设置和历史年份的依赖，比较 11 种时间、完整性、窗口及历史支持设置，并逐次剔除一个历史年；表 1-4给出各设置的入选和可评分次数。

表 1-4　前五特征对分析设置和历史年份的敏感性

特征	11 种设置的 S 范围	设置入选/可算	逐年剔除入选/可算	逐年剔除名次范围
格点累计总降水	0.856—0.902	11/11	5/5	1—2
相对湿度	0.670—0.874	10/11	5/5	2—5
气温高度差	0.685—0.857	10/11	5/5	1—5
10 m 标量风速	0.315—0.853	10/11	5/5	3—5
格点最大小时量	0.844—0.918	11/11	5/5	1—5

11 种设置涵盖站点时标、截止时刻、覆盖率、窗口、历史端点及去重阈值，并比较共同历史支持；后者取合并重复温度信息前 18 项可评分因素的适用窗口均有效的历史端点。“入选 / 可算”为进入前五的次数与可评分次数之比。

五项在 5 次逐年剔除中均保留；删除 30 日窗后，前五变为格点最大小时量、格点累计量、PDD、均温变化率和 Rsd。

多变量联合结构核验

为判断各因素共同出现时是否偏离历史，将同一窗口的 17 个实际统计量组成向量，分别在 7 日和 30 日开展主成分分析（PCA）。每行对应一个历史端点，两个尺度共用 148 个完整端点，2021—2025 年依次为 31、31、31、26、29 行。每个参考—校准配置用三参考年按年等权拟合中心、标准差和协方差；年 y 有 n_y 个端点时，每行权重为 $1/(3n_y)$，目标与校准年应用同一组参数。

保留成分数 k 取累计解释方差首次达到 85% 的最小值：7 日模型 $k=5$ 或 6，30 日模型 $k=3$ 或 4；合法的 $k-1$ 和 $k+1$ 另作敏感性比较。

设 x_j 为该窗口的第 j 项统计量，μ_j 和 σ_j 为三参考年的加权中心和标准差，$z_j=(x_j-\mu_j)/\sigma_j$ 为标准化后的第 j 项，z 为所得 17 维列向量，P_k 的列为保留的 k 个单位正交主成分方向，Λ_k 为对应特征值组成的对角矩阵；上标 T 表示转置。

用保留空间中的距离 T^2 和重构残差平方和 Q 分别描述联合偏离：

$$T^2=z^{\mathsf{T}}P_k\Lambda_k^{-1}P_k^{\mathsf{T}}z,\qquad e=z-P_kP_k^{\mathsf{T}}z,\qquad Q=e^{\mathsf{T}}e. \tag{6}$$

T^2 衡量沿历史共同变化方向的标准化距离；e 是原标准化向量减去保留成分重构值后得到的残差，Q 为残差平方和，衡量未被保留成分解释的偏离 [5]。分别将目标的两量与同一模型的校准年比较，求其中秩位置，再依式（5）先平均同一校准年的参考组合、后跨年等权汇总。两量分别比较。

表 1-5　PCA 联合指标在留出背景中的历史位置

尺度	T^2 汇总位置	T^2 组合范围	Q 汇总位置	Q 组合范围
7 日	0.677	0.069—1.000	0.628	0.161—1.000
30 日	0.745	0.231—1.000	0.142	0—0.806

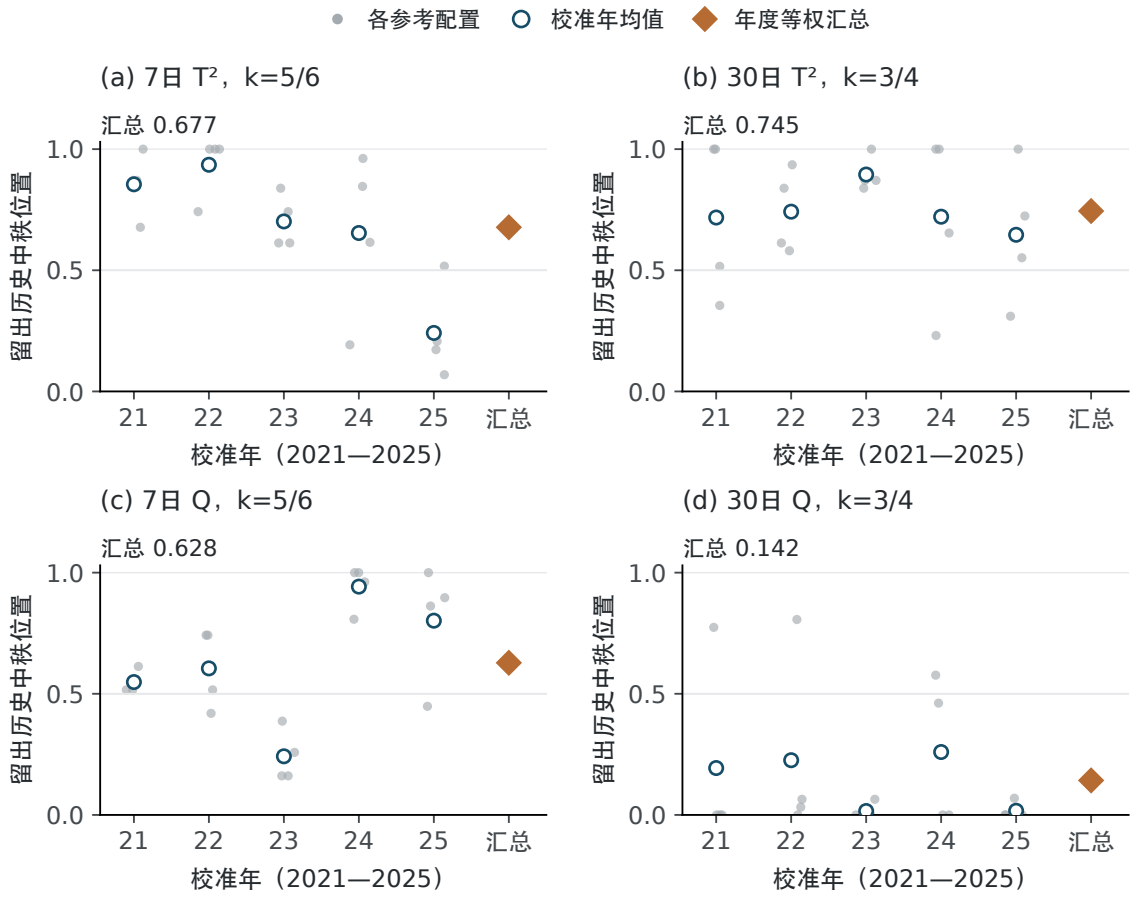


图 **1-6**　两个固定尺度的 17 项联合历史位置。展示各参考组合、校准年均值和年度等权汇总；散点横向错位用于区分不同配置。7 日 k 取 5 或 6，30 日取 3 或 4。

联合位置随参考年份组合变化（表 1-5、图 1-6）；改变成分数后，7 日和 30 日 T^2 汇总位置分别为 0.677—0.739 和 0.677—0.816。

为定位各变量的联合偏离，将 T^2 和 Q 分解为可相加的原变量项：

$$c_j^T = z_j\left[P_k\Lambda_k^{-1}P_k^{\mathsf{T}}z\right]_j, \qquad c_j^Q = e_j^2, \qquad \sum_j c_j^T = T^2, \quad \sum_j c_j^Q = Q. \tag{7}$$

式中方括号下标 j 表示向量的第 j 个分量，c_j^T 和 c_j^Q 为因素 j 对 T^2 和 Q 的可加项；对非零的对应统计量，各模型分别计算 c_j^T/T^2 和 c_j^Q/Q，先对同一校准年的参考集合平均，再对校准年等权平均。T^2 的有符号分解允许个别份额为负。7 日距离中均温变化率和浅层含水量份额较大，残差中最大小时总降水量份额较大；30 日距离中累计降水、湿度及高度差等份额较大。

5.2 问题二

第二问：建立饱和细粒基质的孔压反馈模型，分析初始状态与累计剪切共同决定的起始条件。

1）模型的准备

研究对象和输入资料

问题二研究吉隆冰崩—泥石流灾害链的触发与物质转化条件。此次“8 · 26”灾害链源于蓝塘里壤峰高位冰岩崩：冰岩体撞击错坚河谷底后，运动物质裹挟冰川冰和湿细粒冰碛物，沿错坚河下行至普热普藏布 [6]。本文据此研究湿细粒冰碛物受剪时的孔压变化，将其中细颗粒与孔隙水理想化为单位平面面积的饱和材料柱：柱厚沿坡面法线方向，固体体积固定，基底不透水，顶面可交换液水；以固相体积分数和厚度平均有效法向应力描述柱体状态。

颗粒骨架受剪收缩、孔隙水排出滞后时，孔隙水压力（孔压）上升，颗粒承担的接触应力降低；排水则缓解这一变化。柱上部接触应力降至零时，孔压承担法向荷载；零接触应力区域开始具有正厚度的边界称为**孔压支承层起始条件**。

由撞击谷底至错坚河沟口的 8 个地图顶点得到平面折线长度约 5.37 km；以两端高程 3900、3020 m 的差值除以该长度并取反正切，得平均坡度 $\theta = 9.31°$[6]，用于法向浮重计算。同文纵剖面的另一组距离和高差给出 8.51°，用于几何敏感性比较。

外部速度 U_{ext} 采用同事件地震信号反演的整体沿坡速度，按图中约 5 s 间隔读点线性插值，读取范围约为 ± 1.5 m/s[6]。取源释放后 27—100 s 的曲线驱动材料柱，模型时间 $t = 0$ 对应相对 27 s，窗长 73 s。速度积分表示外部累计运动尺度。

图 2-1展示运动输入及瑞士 Blatten 冰岩崩沉积物的细粒样本。采用文献报告的饱和导水率均值 $K_{fs} = 1.5 \times 10^{-5}$ m/s[7, 8]，以清水动力黏度 $\mu_w = 0.001$ Pa · s 和水密度 $\rho_w = 1000$ kg/m³ 换算，得到初始渗透率 $k_i = K_{fs}\mu_w/(\rho_w g) = 1.529 \times 10^{-12}$ m²；模型另取细粒悬液有效黏度 $\mu = 0.01$ Pa · s。

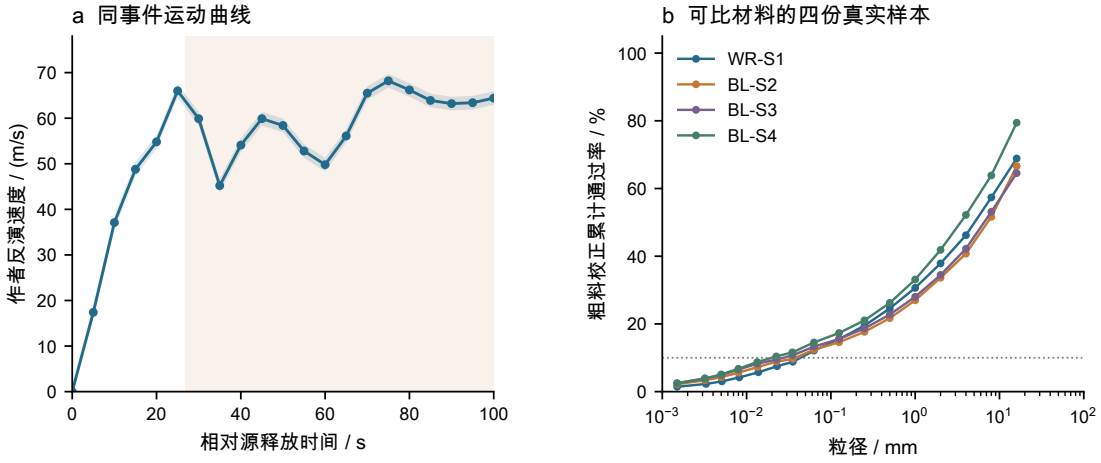


图 **2-1**　反演运动输入与 Blatten 沉积物粒度。速度包络表示图示读取范围；右图为四个细粒样本。

初态与工作假设

设计初态取固相体积分数 $m_0 = 0.54$、柱厚 $H_0 = 1$ m 和初始有效应力比 $q_0 = 0.17, 0.30, 0.50, 0.75$；强施剪情景取 $\chi = 0.02\,\mathrm{m}^{-1}$。材料参数取临界固相分数 $m_{\mathrm{crit}} = 0.60$、无量纲压缩性参数 $a_c = 0.03$、应力正则量 $\sigma_0 = 100$ Pa、颗粒碰撞长度尺度 $\delta = 0.001$ m 和固体密度 $\rho_s = 2700$ kg/m³[9]。

2）模型的建立

状态、本构与外部剪切

记 m 为固相体积分数（无量纲），H 为沿坡面法线的柱厚（m），s 为厚度平均有效法向应力（Pa），即扣除孔压后颗粒骨架承担的法向应力。单位面积固体体积 $H_s = m_0 H_0 = mH$，固体浮重的法向应力尺度为 $E = (\rho_s - \rho_w) g \cos\theta H_s$（Pa），其中 $g = 9.81$ m/s²。固定固体体积与坡度下 E 为常数，H 由 m 恢复；静水孔压时 $s = E/2$。初始值 $s_{\mathrm{init}} = q_0 E$，因此 $q_0 = 0.50$ 是初始超孔压为零的参考状态。

源模型的平均有效应力为 $\sigma_e = \mathrm{tr}(T_e)/3$，$T_e$ 为有效应力张量。以厚度平均法向应力 s 近似替代 σ_e，代入剪胀—压缩性关系 [9]：

$$\begin{aligned} \alpha &= \frac{a_c}{m(s+\sigma_0)}, \qquad N = \frac{\mu\dot{\gamma}}{\rho_s \dot{\gamma}^2 \delta^2 + s}, \\ \tan\psi &= m - \frac{m_{\mathrm{crit}}}{1+\sqrt{N}}, \quad k = k_i \exp[(m_0 - m)/0.04]. \end{aligned} \tag{8}$$

式中 α 为压缩性（Pa^{-1}），N 为无量纲黏性数，ψ 为剪胀角，k 为渗透率（m²），$\dot{\gamma}$ 为剪切率（s^{-1}）；$\dot{\gamma} = 0$ 时取 $N = 0$。m 与经 N 修正的临界分数比较决定 $\tan\psi$ 的符号：负值表示收缩，正值表示剪胀。a_c 控制压缩性，σ_0 避免低应力时压缩性发散，δ 决定颗粒碰撞应力的长度尺度。渗透率以初态 m_0 的初始值 k_i 为锚点；指数中的 0.04 为无量纲固相分数尺度，控制渗透率对固相分数变化的敏感程度，取自源经验关系 [9]。χ 综合局部滑移、受剪尺度与应变分配，将整体速度映射为局部剪切率，单位为 m^{-1}：

$$\dot{\gamma}(t) = \chi U_{\mathrm{ext}}(t + 27\,\mathrm{s}), \qquad 0 \le t \le 73\,\mathrm{s}. \tag{9}$$

无拉力分层与水量平衡

取沿坡面法线自基底向上的坐标 z（m），令 $x = z/H$；p_e 为相对静水分布的超孔压（Pa），$e = p_e(0)$ 为基底超孔压，J 为向上的等效 Darcy 通量（m/s）。基底不透水，顶面超孔压为零。采用二次超孔压剖面 [9]，由平均有效应力得到基底超孔压和顶面通量；未分层时有

$$e = \tfrac{3}{2}(E/2 - s), \qquad p_e(x) = e(1 - x^2), \qquad J = \frac{2ke}{\mu H}, \qquad s/E \ge \tfrac{1}{6}. \tag{10}$$

当二次剖面的上部接触应力降至零后，令 $0<s/E<1/6$，并设 $r=\sqrt{6s/E}$、$f=1-r$ 为孔压支承层厚度占比。本文采用无拉力分层及相应等效 Darcy 通量，得到

$$e=E(1-r/2),\quad J=\frac{kE}{\mu H},\quad p_e(x)=\begin{cases}e-Ex^2/(2r), & 0\le x\le r,\\ E(1-x), & r<x\le 1.\end{cases} \tag{11}$$

总孔压为 $\rho_w g\cos\theta H(1-x)+p_e$；式（11）中下部接触应力为 $E(r-x)^2/(2r)$，上部为零，其深度平均为 $s=Er^2/6$。两支在 $r=1$ 处压力、梯度和通量连续，因此 $q_c=s/E=1/6$ 是本剖面的支承层起始边界；等号处 $f=0$，继续越过才有正厚度支承层。

由固体守恒、饱和体积预算及式（8）的本构关系得到，其中上点表示对模型时间 t 求导：

$$\dot m=\frac{mJ}{H},\qquad \dot s=\frac{\dot\gamma\tan\psi+J/H}{\alpha},\qquad \dot H=-J. \tag{12}$$

$J<0$ 表示顶面吸水，依赖外部供水；$J>0$ 表示排水，使柱厚减小、固相分数增大。式（12）中剪切收缩项降低 s，排水项提高 s，两者共同决定应力变化。一般初态的初始通量 $J_0=2k_ie_0/(\mu H_0)$，$e_0=\frac{3}{2}(E/2-s_{\text{init}})$，只有 $q_0=0.50$ 的 J_0 为零。高于静水参考的初始有效应力对应负超孔压；饱和情景要求总孔压非负，达到这一边界时结束该饱和情景。初态工作范围为 $1/6<q_0<q_{\max}$，其中

$$q_{\max}=\frac{1}{2}+\frac{\rho_w}{3(\rho_s-\rho_w)m_0}\approx 0.863108. \tag{13}$$

四个设计初态均在此范围内。

累计剪切与临界反求

若排水及材料结构变化都较小，收缩导致的应力下降主要由累计剪切决定；以下用数值轨迹检验这一近似。令 $I_U(t)=\int_0^t U_{\text{ext}}(\tau+27\,\text{s})\,d\tau$（m），累计剪切 $\Gamma_{\text{sh}}=\chi I_U$ 为无量纲量。由式（8）、（12）可积分得到

$$\ln\frac{s(t)+\sigma_0}{s_{\text{init}}+\sigma_0}=\frac{1}{a_c}\int_0^t m\dot\gamma\tan\psi\,d\tau+\frac{m(t)-m_0}{a_c}. \tag{14}$$

右侧第一项为剪切贡献，第二项是与排水相关的体变贡献。若到达起始边界前排水弱、$m\simeq m_0$，且 N 对 $\Delta=m_{\text{crit}}-m_0>0$ 的修正较小，则

$$\begin{aligned} s+\sigma_0&\simeq(s_{\text{init}}+\sigma_0)\exp\!\left(-\frac{m_0\Delta}{a_c}\Gamma_{\text{sh}}\right),\\ \Gamma_{c,\text{asy}}&=\frac{a_c}{m_0\Delta}\ln\frac{s_{\text{init}}+\sigma_0}{q_cE+\sigma_0},\qquad \chi_{\text{asy}}=\frac{\Gamma_{c,\text{asy}}}{I_U(T)}. \end{aligned} \tag{15}$$

式中 T 为规定窗长，$\Gamma_{c,\mathrm{asy}}$ 与 χ_{asy} 分别为近似临界累计剪切和传递系数；要求 $s_{\mathrm{init}} > q_c E$ 且 $I_U(T) > 0$。为检验近似，定义对数目标间隔和排水绝对贡献比

$$L_{\mathrm{goal}} = \ln \frac{s_{\mathrm{init}} + \sigma_0}{q_c E + \sigma_0}, \qquad \epsilon_J = \frac{\int_0^{t_{\mathrm{cut}}} |mJ/(a_c H)|\, dt}{L_{\mathrm{goal}}}, \tag{16}$$

其中 t_{cut} 为首次到达边界、计算窗结束或越出饱和模型有效域的时刻。ϵ_J 衡量排水相对于所需应力下降的量级；在 $0 \le t \le t_{\mathrm{cut}}$ 上，同时定义无量纲诊断量 $\epsilon_m = \max |m - m_0|/\Delta$ 和 $\epsilon_N = \max[m_{\mathrm{crit}}\sqrt{N}/((1+\sqrt{N})\Delta)]$，分别衡量固相变化及 N 修正相对于 Δ 的大小。

固定初态、材料与外部速度，改变 χ，在 73 s 窗内求最小应力恰好触及 $q_c E$ 的条件根：令 $G(\chi) = \min_{0\le t\le T} s(t)/E - q_c$，采用 Radau 积分，并在有效异号区间用 Brent 法求 $G(\chi^*) = 0$。

3）模型的求解与分析

初态与累计剪切联合边界

主材料条件与 73 s 速度窗下，四个初态的临界根、累计剪切及共同施剪响应见表 2-1；完整窗速度积分 $I_U(73) = 4329.44$ m。

共同采用 $\chi_{\mathrm{cmp}} = 2.265378784906 \times 10^{-4}\ \mathrm{m}^{-1}$ 时，前三个初态分别在 1.21、41.7 和 72.9971 s 到达边界，$q_0 = 0.75$ 在 73 s 内未到。强施剪档 $\chi = 0.02\ \mathrm{m}^{-1}$ 下，$q_0 = 0.50$ 的到达时间为 0.792 s。

表 2-1　四个设计初态的临界根与共同比较档响应

q_0	χ^*（m^{-1}）	$\chi^* I_U(73)$	共同 χ_{cmp} 首次到边界（s）
0.17	4.35×10^{-6}	0.0188	1.21
0.30	1.20×10^{-4}	0.520	41.7
0.50	2.27×10^{-4}	0.981	72.9971
0.75	3.12×10^{-4}	1.35	73 s 内未到达边界

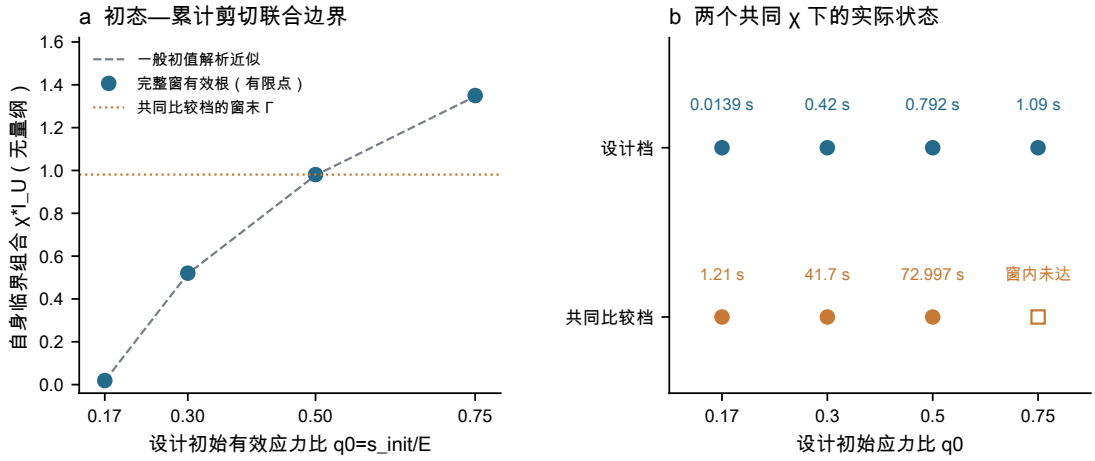


图 2-2　四个初态的临界根与共同施剪响应。左图为临界根及近似值，右图比较两档剪切；空方块表示完整规定窗内未到达支承层起始边界。事件时刻由折线输入的数值解给出。

排水对近似的影响

初态接近起始边界时，达到边界所需的应力下降较小，排水相对作用增强。四个初态的临界根相对近似值偏差 $(\chi^{*}-\chi_{\mathrm{asy}})/\chi_{\mathrm{asy}}$ 依次为 9.66%、0.482%、0.347% 和 0.289%。$q_0=0.17$ 时，略高于临界剪切系数的弱施剪轨迹，其排水绝对贡献占目标对数应力下降的 9.54%；同一初态在共同档 χ_{cmp} 下为 0.159%；前者固相变化和 N 修正相对 Δ 分别为 8.86×10^{-4} 和 4.47×10^{-4}（图 2-3）。

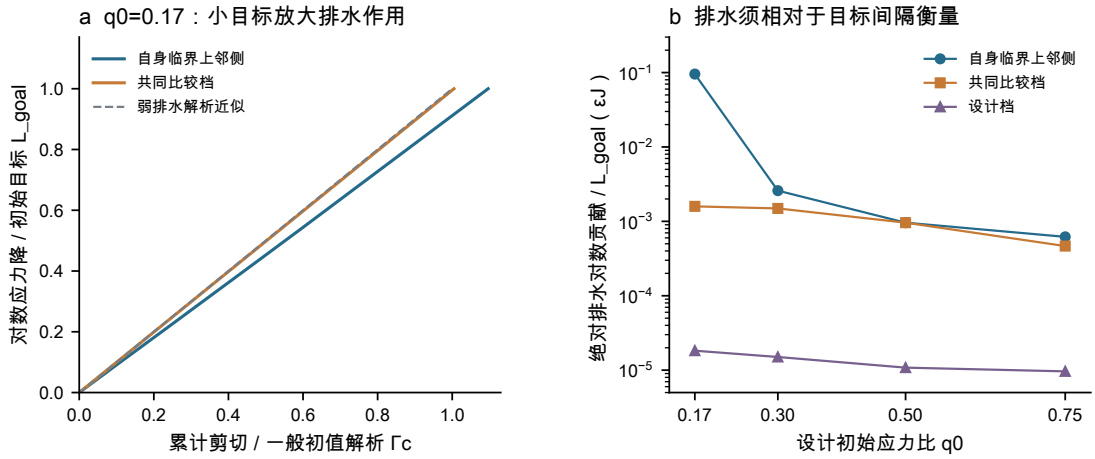


图 **2-3** 近边界初态的排水诊断。左图对比略高于临界剪切系数的弱施剪档与共同 χ_{cmp} 档的应力变化；右图为各诊断轨迹的排水绝对贡献与目标间隔之比。

材料、时间窗与压力形状的影响

固定 $H_0=1$ m、$q_0=0.50$，分别取 $m_0=0.54$ 和 0.57；与 $m_{\mathrm{crit}}=0.60$ 相比，改为 0.59 时临界根分别增加 20.1% 和 50.8%。$m_{\mathrm{crit}}=0.54$ 时初始有效应力上升。对 $m_0=0.54$、$H_0=1$ m、$q_0=0.50$，在相对 27 s 起始的 33、53、73 s 窗口分别求根，$\chi^{*}I_U$ 依次为 0.98124、0.98084 和 0.98074。

压力剖面的指数 n 控制超孔压随厚度方向衰减的形状。改变该形状时，平均应力、顶面排水与零接触应力层条件须相应改变。对未分层形状 $p_e=e(1-x^{n})$ $(n>1)$，可得

$$s=E/2-\frac{n}{n+1}e,\qquad J=\frac{nke}{\mu H},\qquad q_c(n)=\frac{n-1}{2(n+1)}. \tag{17}$$

对应分层支取 $r=\sqrt{(s/E)/q_c(n)}$、$e=E[1-(n-1)r/n]$，下部 $p_e=e-Ex^{n}/(nr^{n-1})$，上部 $p_e=E(1-x)$，通量为 $kE/(\mu H)$。在 $m_0=0.54$、$H_0=1$ m、$q_0=0.50$ 及 73 s 速度窗下，$n=2$ 和 $n=3$ 的起始应力比分别为 $1/6$ 和 $1/4$，$n=3$ 的条件根低 36.4%（图 2-4）。在 $n=2$ 和相同条件下，将速度曲线整体上移或下移 1.5 m/s，条件根为 2.21—2.32×10^{-4} m^{-1}。

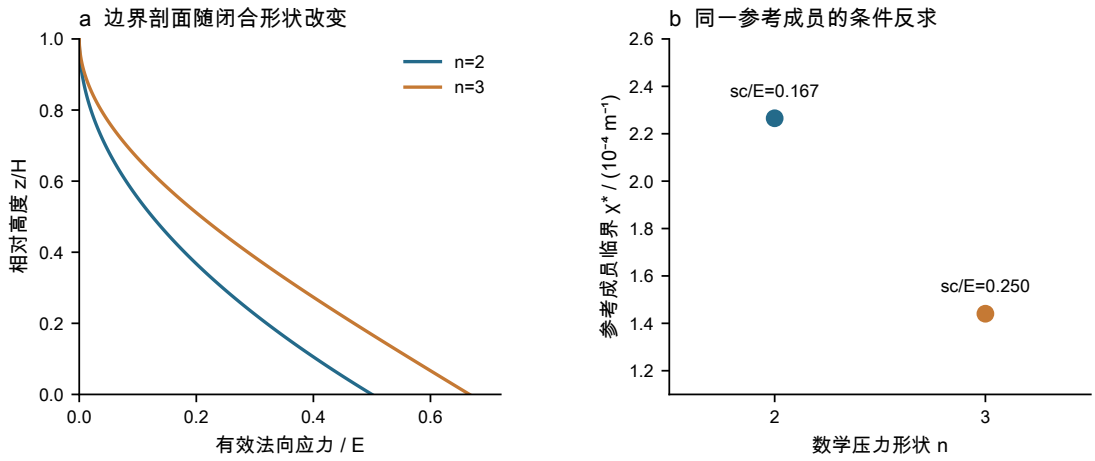


图 **2-4**　不同压力剖面假设下的起始边界比较。两条曲线对应各自的模型内起始边界。

5.3 问题三

第三问：预测局部临界条件的首达时间，分析覆盖率、预警余量与目标到达。

1）模型的准备

预测对象与输入

问题二给出了局部饱和材料柱的孔压支承层起始边界；问题三在该动力系统上进一步预测何时首次达到边界。记平均有效法向应力比 $q = s/E$，二次压力剖面的边界 $q_c = 1/6$。沿用事件反演速度曲线的选定区间 $t_{\mathrm{ref}} \in [27, 100]$ s，令 $t = t_{\mathrm{ref}} - 27$ s，预测窗为 $0 \le t \le 73$ s；$t = 0$ 为选定计算窗起点，初态在此设置。对初始有效且 $q_0 > q_c$ 的材料柱，定义局部首达时间

$$\tau_L = \inf\{t \in (0, 73\,\mathrm{s}] : q(t) \le q_c\}. \tag{18}$$

窗内首达输出有限 τ_L；完整有效计算窗内未达到边界时记为独立的“窗内未达”类别 $\perp$。

参考速度为该事件的事后反演读图曲线。对整条曲线的每个速度节点加入同一偏移 d，逐节点截零后再分段线性插值。记 $t_i = t_{\mathrm{ref,i}} - 27$ s，节点 t_i 处参考速度为 $U_{\mathrm{ref,i}}$、处理后为 U_i，则

$$\begin{aligned} U_i(d) &= \max\{0, U_{\mathrm{ref,i}} + d\}, \qquad U(t; d) = (1 - \lambda_i)U_i(d) + \lambda_i U_{i+1}(d), \\ \lambda_i &= (t - t_i)/(t_{i+1} - t_i), \qquad d \in [-a, a], \quad a = 1.5\ \mathrm{m/s}. \end{aligned} \tag{19}$$

剪切率沿用问题二的 $\dot{\gamma} = \chi U$，单位为 s^{-1}。主要概率分析固定初态 $q_0 = 0.50$，选用强施剪 $\chi = 0.02\ \mathrm{m}^{-1}$ 和近临界 $\chi = 2.265378784906 \times 10^{-4}\ \mathrm{m}^{-1}$ 两种设计条件；d 分别采用对称三角分布 $\mathrm{Tri}(-a, 0, a)$、均匀分布 $\mathrm{U}(-a, a)$ 和低速偏向三角分布 $\mathrm{Tri}(-a, -a, a)$，其中三角分布三个参数依次为下界、众数和上界。

2）模型的建立

分段映射与稳健预测集合

固定初态、材料和施剪条件后，材料柱方程将速度偏移 d 映射为有限首达时间或窗内状态。对第 r 种偏移分布 F_r，首达子累积分布 $H_r(t)$ 给出全体情景中至时刻 t 已首达的质量：

$$H_r(t) = \int \mathbf{1}\{\text{窗内首达},\ \tau_L(d) \le t\}\,\mathrm{d}F_r(d). \tag{20}$$

对输入域的类别分界和有限首达段进行自适应细分，有限段用形状保持分段三次插值（PCHIP）[10]，再在每个输入段反解 $\tau_L(d) \le t$ 得到偏移区间 $[\ell_j, u_j]$，按 $F_r(u_j) - F_r(\ell_j)$ 计算该段质量并对各段求和。窗末的 $H_r(73\,\mathrm{s})$ 为窗内首达质量，未达质量单独保留。

令 $I = [L, U]$ 为窗内有限时间区间，$\eta \in \{0, 1\}$ 表示预测集合是否包含未达类别；$p_{\perp,r}$ 为第 r 种分布的窗内未达质量。对声明的三种分布同时要求集合质量不低于 $1 - \alpha_{\mathrm{cov}} = 0.95$，在共同固定时间网格上选择有限宽度最短的集合：

$$\min_{I,\eta}(U - L), \quad \text{s.t.} \quad \min_{r}\{H_r(U) - H_r(L^-) + \eta p_{\perp,r}\} \ge 1 - \alpha_{\mathrm{cov}}. \tag{21}$$

其中 $H_r(L^-)$ 使闭区间左端点计入，$\alpha_{\mathrm{cov}} = 0.05$ 为目标漏覆盖率。三种输入分布共享同一确定性响应映射，改变的是概率权重。名义分布为对称三角分布：名义中央集合按 2.5% 与 97.5% 累计质量定位端点，高分位超出有限首达质量时附加窗内未达；名义最短集合在该分布下选择有限宽度最短的集合；稳健最短集合同时满足三种分布的质量要求。

3）模型的求解与分析

时间预测与模拟覆盖

强施剪的稳健 95% 有限预测区间为 **[0.775, 0.815] s**；近临界预测集合由有限区间 **[71.40, 72.95] s** 与 73 s 内未达类别组成（表 3-1，图 3-1）。时间均相对选定计算窗起点。连续积分得到的最差分布质量分别为 0.975799 和 0.964297。

表 3-1 两种施剪条件下的稳健 95% 首达预测集合

条件	窗内时间区间/s	窗内未达	最差族积分质量
强施剪	[0.775, 0.815]	不含	0.975799
近临界	[71.40, 72.95]	包含	0.964297

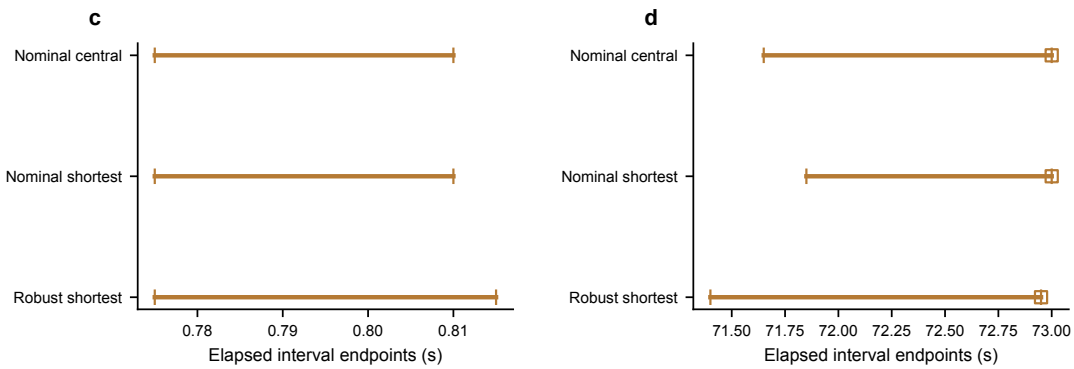


图 3-1 强施剪（c）与近临界（d）的首达预测集合。每栏从上至下为名义中央、名义最短和三分布稳健最短集合；空心方框表示附加窗内未达类别，横轴为自选定计算窗起点计的有限区间端点。

为检验上述两套集合在声明分布下的模型模拟包含率，分别从两种施剪条件和三种偏移分布生成随机成员，每项 4096 个，共 24576 个。目标漏覆盖率为 $\alpha_{\mathrm{cov}} = 0.05$；六项声明的总体模拟错误预算取 $\delta_{\mathrm{sim}} = 0.05$，每项分配 $\delta_{\mathrm{sim}}/6$。对包含数 k 和样本数 n，以单侧 Clopper–Pearson 精确二项下界 [11]$\ell = B^{-1}(\delta_{\mathrm{sim}}/6; k, n-k+1)$ 评价包含概率，其中 B^{-1} 为对应 Beta 分布分位数。六项模拟包含比例和同时保护下界见表 3-2；强施剪和近临界的最低下界分别为 0.970003 和 0.956011，均高于 0.95。

表 3-2 两种施剪条件、三种速度偏移分布下的模拟包含率

条件	分布	包含数/4096	包含比例	精确单侧下界
强施剪	对称三角	4090	0.998535	0.996376
强施剪	均匀	3999	0.976318	0.970003
强施剪	低速偏向三角	4094	0.999512	0.997893
近临界	对称三角	3979	0.971436	0.964584
近临界	均匀	3947	0.963623	0.956011
近临界	低速偏向三角	4047	0.988037	0.983319

初态与临界响应

19 个初始应力比与 17 个速度偏移组成 323 个设计点，两种施剪条件共 646 个情景：强施剪 323 个窗内首达，近临界 179 个首达、144 个窗内未达（图 3-2）。近临界且 $d = 0$ 时，$q_0 = 0.30$ 与 0.50 的首达时间分别为 41.682 s 和 72.997 s，$q_0 = 0.75$ 为窗内未达；在 $q_0 = 0.50$、$d = 0$ 下，将近临界 χ 降低 1% 得到窗内未达，提高 1% 得到 72.331 s 首达。

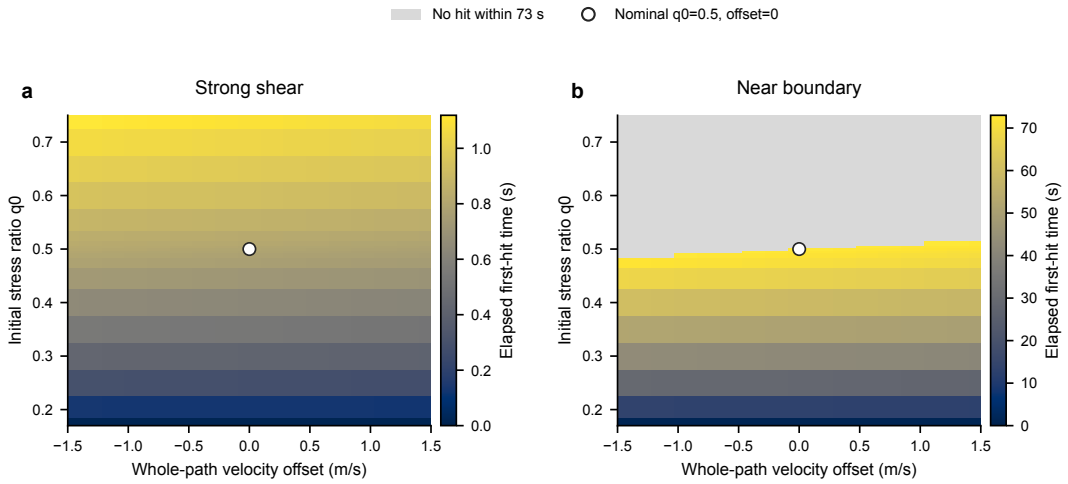


图 3-2 初始应力比与整条速度偏移对应的首达响应。左栏为强施剪、右栏为近临界；横轴 d 为整条速度偏移（m/s），纵轴 q_0 为初始应力比，两栏色标范围不同，均表示自选定窗起点计的有限首达时间（s）；灰色为 73 s 内未达，白圈为 $q_0 = 0.50$、$d = 0$。

在首达处横截且 $\dot{q}(\tau_L) \neq 0$ 时，相同局部状态增量 δq 对应的一阶时间变化为

$$\delta\tau_L \simeq -\frac{\delta q}{\dot{q}(\tau_L)}. \tag{22}$$

取 $\delta q = 0.001$，强施剪与近临界的时间响应分别为 0.00428 s 和 0.35857 s，后者约为前者的 83.8 倍。该比较说明慢横截对局部状态裕量的报时放大；近临界名义首达距窗末约 0.002905 s，扰动可使结果在首达与未达之间切换。三种偏移分布下，近临界窗内未达质量依次为 0.4983、0.4991 和 0.7491。

发布、行动与局部预警余量

设 d_{pub} 为自选定窗起点至发布的延迟，d_{act} 为发布后的行动历时，则局部首达相对行动就绪时刻的余量为

$$L_L = \tau_L - d_{\mathrm{pub}} - d_{\mathrm{act}}. \tag{23}$$

$L_L > 0$ 表示行动就绪后尚有时间，$L_L < 0$ 表示就绪晚于局部首达。强施剪下发布延迟 1 s、行动历时 0 s，对应有限余量 [–0.225, –0.185] s；近临界下发布延迟 1 s、行动历时 15 s，对应 [55.40, 56.95] s，另保留窗内未达类别。图 3-3给出不同延迟下的变化，强施剪的负余量提示预警启动需要前置。

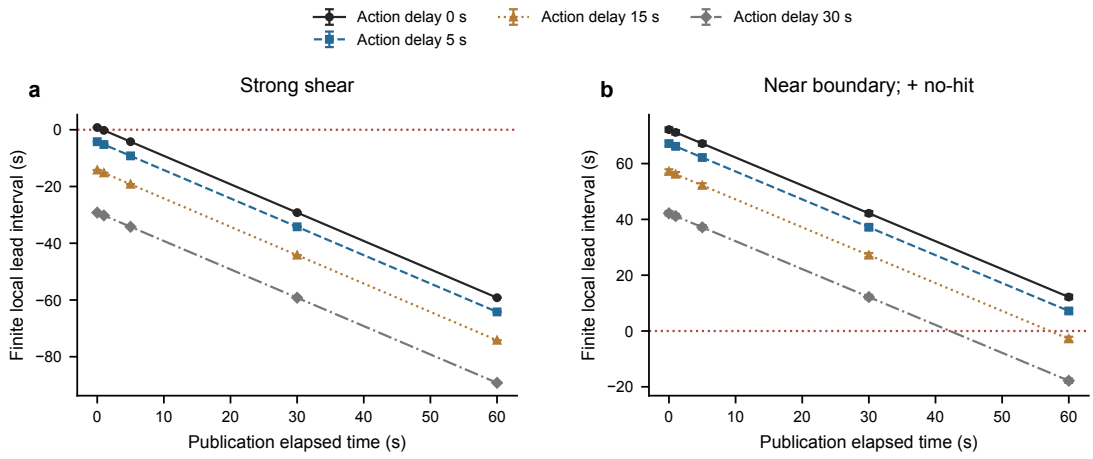


图 **3-3**　不同发布延迟和行动历时下的局部有限余量。左栏为强施剪、右栏为近临界；横轴为发布延迟（s），四条线对应行动历时 0、5、15、30 s，纵轴为有限余量（s）。零线区分行动就绪早于或晚于局部首达；近临界集合保留窗内未达类别。

理想常坡目标到达

为计算空间目标的条件余量，设传播入口相对时刻 T_{in} 与局部首达时刻 $T_L = \tau_L$ 相同；目标旅行时长为 ΔT_A，到达时刻与行动余量分别为

$$T_A = T_{\mathrm{in}} + \Delta T_A, \qquad L_A = T_A - d_{\mathrm{pub}} - d_{\mathrm{act}}. \tag{24}$$

在 8° 理想常坡、恒定法向层厚 2 m 的算例中，以沿坡距入口 1000 m 处为固定测试断面，自传播入口起算的传播时间窗取 180 s。采用 Voellmy 摩擦参数 μ_V（无量纲）和湍阻参数 ξ（m/s²）构造五个固定参数情景；μ_V 与问题二的悬液黏度 μ 分别定义。记 z 为自入口起算的传播时间、x 为沿坡位移、v 为速度，$\vartheta = 8°$、恒定法向层厚 $H_A = 2$ m、$g = 9.81$ m/s²，则单方向标量运动关系和初始条件为

$$\frac{\mathrm{d}x}{\mathrm{d}z} = v, \qquad \frac{\mathrm{d}v}{\mathrm{d}z} = g(\sin\vartheta - \mu_V \cos\vartheta) - \frac{g v^2}{\xi H_A}, \qquad x(0) = v(0) = 0. \tag{25}$$

由解析位移关系解 $x(z) = 1000$ m 得到旅行时长；五个情景中三例到达目标，两例在 180 s 内未到（表 3-3）。将各情景旅行时间与对应局部首达集合相加，即得目标到达时间与行动余量；图 3-4展示发布延迟 60 s、行动历时 30 s 时的有限余量。

表 **3-3**　8° 理想常坡至 1000 m 测试断面的五个传播情景

情景	μ_V	ξ/(m/s²)	旅行时长或传播窗状态
S1	0.03	200	155.395 s
S2	0.03	2000	61.260 s
S3	0.13	200	180 s 内未到
S4	0.13	2000	180 s 内未到
S5	0.09	399.052	167.147 s

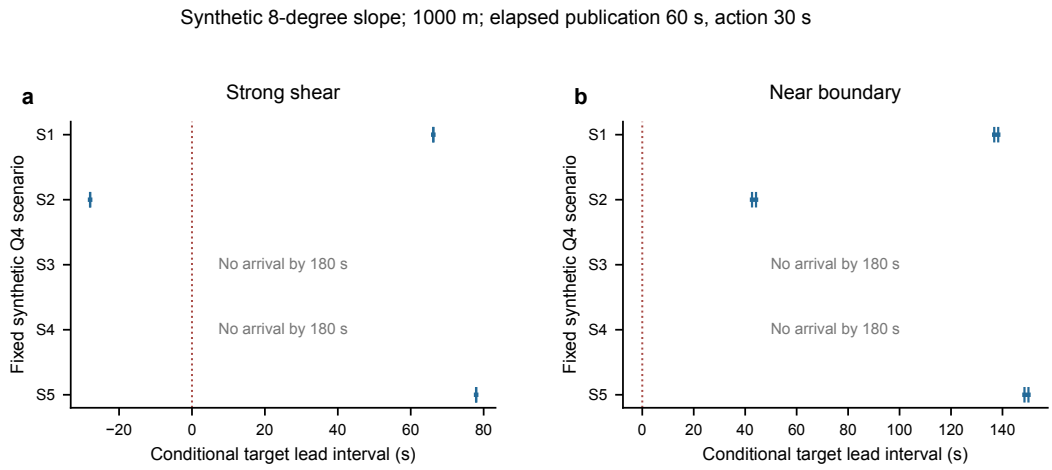


图 **3-4** 理想常坡固定 1000 m 目标的条件行动余量。左栏为强施剪、右栏为近临界；横轴为目标行动余量（s），纵轴 S1—S5 为表 3-3的参数情景。发布延迟 60 s、行动历时 30 s；S3、S4 在 180 s 传播窗内未到，虚线表示零余量。

5.4 问题四

第四问：建立规定入口后的二维传播仿真，分析到达、覆盖及关键参数。

1）模型的准备

计算区域与入口条件

本问以已形成的等效流体进入指定断面为传播起点，入口开始时刻记为相对 0 s。局部目标断面和口岸目标断面距入口的参考轴线长度分别约 2.119 km 和 14.188 km。问题三的材料柱首达与本问的规定入口分别定义，两者采用各自的时间原点。计算在水平坐标下进行，流深为竖直厚度；地形采用投影至 UTM 45N（EPSG:32645）的 Copernicus GLO-30 及 ASTER GDEM V003[12, 13]，各产品独立形成 15 m 插值支持，再采样到不同计算网格；15 m 支持为插值表达。

规定入口宽 30 m、向西速度 30 m/s，初始域为干床；入口脉冲结束后关闭，规定的下游区段允许出流。入口流量采用总体积 V、历时 T 确定的对称三角脉冲，峰值为 $2V/T$。基准设定 $V = 8 \times 10^5$ m³、$T = 960$ s；双峰情景由两个相邻的半体积、半历时对称三角脉冲组成。目标断面上的最大竖直流深首次达到 0.1 m 时记为到达，首达时刻保留计算步构成的括区。局部域分析采用 1800 s 计算窗，完整河段采用 3600 s 计算窗。

2）模型的建立

固定床二维守恒浅流

设 h 为竖直流深（m），$\mathbf{u}$ 为深度平均水平速度（m/s），z 为固定床高（m），$g = 9.81$ m/s²；μ_V 为 Voellmy 摩擦系数，ξ 为湍阻参数。固定床、常密度条件下，以质量和动量守恒描述流动 [14]：

$$\frac{\partial h}{\partial t} + \nabla\cdot(h\mathbf{u}) = 0, \tag{26}$$

$$\frac{\partial (h\mathbf{u})}{\partial t} + \nabla\cdot\left(h\mathbf{u}\otimes\mathbf{u} + \tfrac{1}{2}gh^2\mathbf{I}\right) = -gh\nabla z - \mu_V gh\frac{\mathbf{u}}{|\mathbf{u}|} - \frac{g}{\xi}|\mathbf{u}|\mathbf{u}. \tag{27}$$

其中 **I** 为单位张量，$\otimes$ 为张量积；μ_V 无量纲，ξ 的单位为 m/s²。零速度处摩擦按停止条件处理。空间上采用有限体积 HLL 通量、受限重构和静水重构 [15]，时间推进结合非反向隐式摩阻更新。入口流量以时间步上的解析积分分配；按边界通量核对体积收支，并记录有效到达和有效计算窗内未达。

参数筛选与多保真代理

以局部断面的最大深度余量 $h_{\max} - 0.1\,\mathrm{m}$ 为筛选响应，以归一化参数步长 0.6 计算基本效应，其八条轨迹的绝对值均值记为 μ^*（m），在 $V \in [2, 8] \times 10^5$ m³、$T \in [720, 1200]$ s、$\mu_V \in [0.03, 0.15]$、$\xi \in [250, 1000]$ m/s² 内进行 Morris 分析。Morris 计算采用 40 m 网格和 1800 s 窗；八条完整轨迹共有 40 个节点、38 个不同查询；V、T 及 ξ 在对数尺度构造轨迹。再固定 $T = 960$ s、$\xi = 500$ m/s²，在 $(\log V, \mu_V)$ 切片上比较代理的到达边界。

取同一连续地形支持上的 40 m 和 30 m 网格为低、高分辨率层。高斯过程（GP）代理采用 Matérn 5/2 核，以无量纲响应 y 表示最大断面流深：

$$y = \log\left(\frac{h_{\max} + 0.1\,\mathrm{m}}{0.2\,\mathrm{m}}\right), \qquad y = 0 \iff h_{\max} = 0.1\,\mathrm{m}. \tag{28}$$

多保真代理以高层响应 $f_H = \rho f_L + e$ 连接低层 f_L 与层间差异 e；ρ 为层间系数，f_L 和 e 的先验相互独立，ρ 和核参数由训练集最大似然拟合；主动策略按边界附近高层预测方差的预期下降及物理调用成本选择新点 [16]。比较四组：A 为 16 个高层点的单层被动代理；B 为 16 个高层和 24 个低层点的被动多保真代理；Bm 与 C 均为 16 个高层、32 个低层点，分别采用被动和主动选点；C 从共同 8 个高层与 24 个低层初始点出发，再主动增加 8 个高层及 8 个低层点。

3）模型的求解与分析

局部传播与关键因素

在 $\mu_V = 0.05$、$\xi = 500$ m/s² 的基准入口及 30 m 网格下，局部目标 0.1 m 首达括区为 **[731.535, 731.808] s**，断面最大流深为 10.020 m（图 4-1）。入口速度由 30 降至 24 m/s，或入口宽度由 30 增至 40 m 时，首达括区上界分别为 733.868 s 和 733.380 s，显示这两个指定扰动下局部结果的变化。

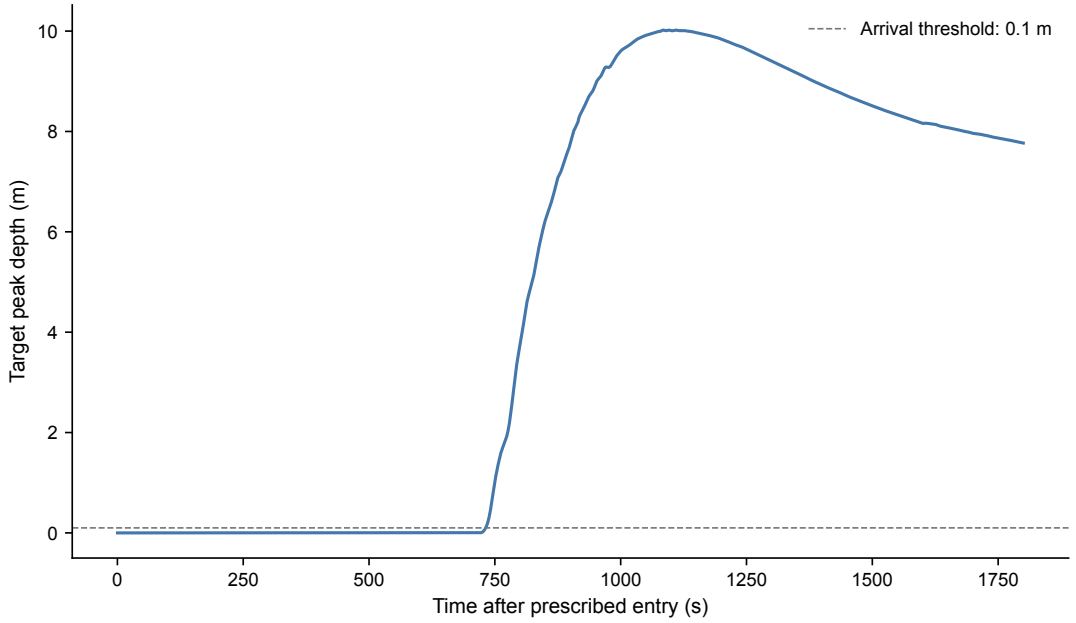


图 **4-1**　局部基准断面流深过程。横轴为自规定入口开始计的相对时间，纵轴为断面瞬时最大竖直流深，虚线为 0.1 m 到达阈值。

Morris 八条轨迹中有 14 个不同查询到达、24 个在计算窗内未达。μ^* 按 μ_V、V、T、ξ 依次为 7.176、2.489、0.819、0.602 m，表明在所设范围和断面深度余量响应下，摩擦系数与入口体积的影响较大。

完整河段覆盖

对 Copernicus 30 m、Copernicus 40 m、ASTER 40 m 及 Copernicus 40 m 双峰四个情景，口岸目标断面均在 3600 s 窗内未达到 0.1 m，且窗末保持流体运动。四组均采用上述基准总体积、历时、入口宽度、速度和摩阻参数；双峰情景采用双峰入流过程。表 4-1分别列出计算窗内曾达到 0.1 m 的覆盖面积与窗末瞬时覆盖面积；图 4-2给出 Copernicus 40 m 单峰情景的空间分布。

表 4-1　完整河段四个条件情景的 3600 s 窗内结果

地形与网格	域内最大流深/m	曾过阈面积/m²	窗末面积/m²
Copernicus 30 m	22.180	301500	201600
Copernicus 40 m	23.571	313600	212800
ASTER 40 m	29.089	145600	102400
Copernicus 40 m 双峰	23.686	313600	212800

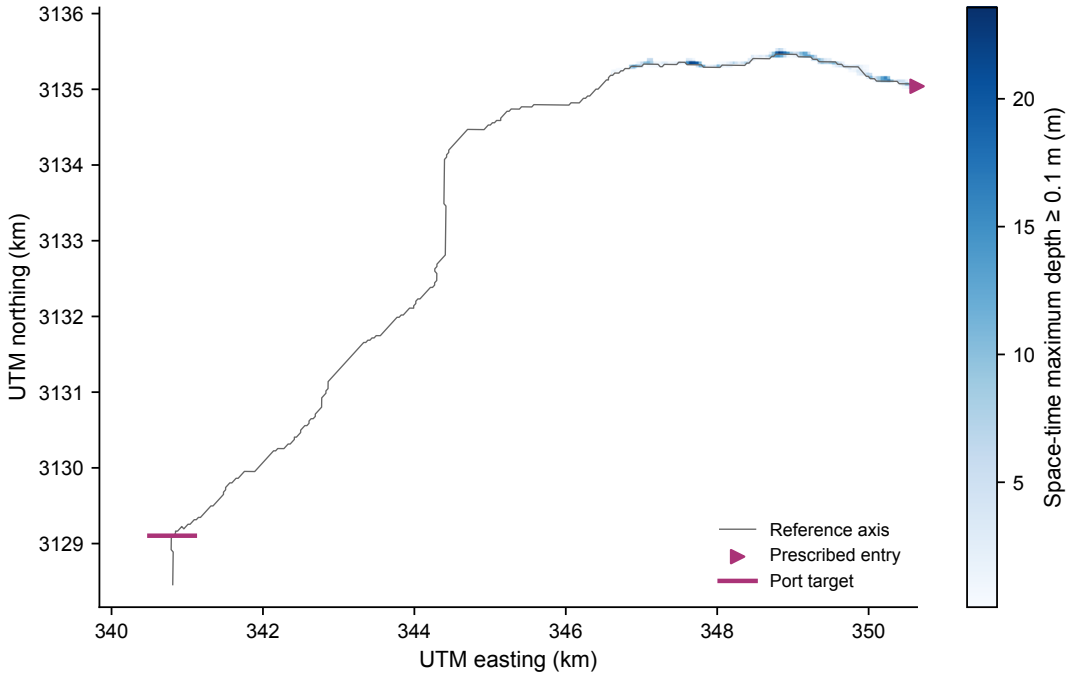


图 **4-2**　Copernicus 40 m 完整河段情景的窗内覆盖。色值表示 3600 s 内曾达到 0.1 m 单元的最大竖直流深；灰线为参考轴线，三角标记为规定入口，短线为口岸目标断面；横、纵轴为 UTM 东、北坐标（km）。

四策略到达边界判断

采用种子 137 的 64 个不同训练查询（40 个低层、24 个高层）；测试采用 16 个全域点和 8 个开发预测边界点，预测完成后再取得相同坐标的高、低层结果。全域 16 点中有 4 个高层模拟到达，四策略均识别这 4 点；边界带 8 点全部在窗内未达。各策略的误报数和全域均方根误差（RMSE）见表 4-2。

表 **4-2**　种子 137 四策略在原 24 个独立测试坐标上的到达判断与误差

策略	全域误报/16	边界误报/8	全域 y RMSE	深度余量 RMSE/m
A 单层被动	3	2	0.64657	1.55064
B 多保真被动	2	2	0.79706	1.73590
Bm 数量匹配被动	1	0	0.30945	0.53211
C 多保真主动	0	0	0.30415	1.64786

原 24 点对照中，C 的到达类别全部正确；预测边界带没有高层模拟到达成员，故该带漏达率为不适用。全域原始深度余量 RMSE 中，Bm 为 0.53211 m、C 为 1.64786 m，反映到达类别和深度幅值是两个不同评价对象。四组采用同一固定历时和湍阻参数切片。

临界摩擦区间与端点检验

为检验到达边界，在固定 $T = 960$ s、$\xi = 500$ m/s² 及局部 30 m 网格、1800 s 窗下，分别取 $V_1 = 336358.566$ m³ 和 $V_2 = 672717.132$ m³。两条体积切片共完成 14 次高层模拟查询，所得到达与未达之间的摩擦系数括区依次为 **[0.06000, 0.06375]** 和 **[0.07875, 0.08250]**。每条切片的两个端点分别为到达和窗内未达；对四端点评价种子 137 对应的四组代理，结果见表 4-3。这些端点由临界搜索选出，展示的是指定模型和时窗下的边界判断。

表 4-3　四策略在两条体积切片的四个高层模拟端点上的表现

策略	TP/FN/FP/TN	y RMSE	深度余量 RMSE/m	V_1 根距	V_2 根距
A	2/0/2/0	1.77867	0.95205	0.01286	0.01746
B	2/0/2/0	2.10412	1.79631	0.00937	0.03562
Bm	2/0/1/1	1.09126	0.45808	0.00861	0
C	2/0/2/0	0.89954	0.27562	0.00378	0.00182

表中 TP、FN、FP、TN 依次为正确到达、漏达、误报和正确未达的点数；根距为各代理预测摩擦临界根到对应高层模拟括区的距离，根在括区内时为零。主动组 C 在四端点误报 2 个，其 y 响应 RMSE 为 0.89954；Bm 误报 1 个，V_2 预测根落在括区内。分类、连续响应和根定位分别评价。种子 271 的四组模型对既有 24 点作描述性回放，A、B、Bm、C 的误报数依次为 2、2、2、0，均无漏达。

物理核验与状态发布

将 GP 结果作为预估，以同配置 30 m 传播计算核定到达状态；核对输入、网格与参数、时窗、目标阈值及质量和动量收支后，发布有效“到达”或“1800 s 窗内未达”结果。相同配置的有效物理结果可复用。对 $V = 475682.846$ m³ 的两个新增参数点，$\mu_V = 0.0675$ 时目标到达、峰深 1.743419 m，首达括区为 [1467.34219, 1467.62929] s；$\mu_V = 0.075$ 时 1800 s 内未达。两次新增传播计算的子进程墙时之和为 67.30125 s。四个历史端点与两个新增点的发布结果均与各自有效物理证据一致；原 GP 对这六条记录预估时产生 3 个误报，该比较反映核验接口的一致性。

六、 模型评价、改进与推广

6.1 模型的优点

问题一以年度等权多窗口比较描述环境特征，兼顾因素排序与联合结构。问题二以守恒与剪胀关系连接受剪、排水和孔压；问题三同时保留首达时间与窗内未达类别。问题四将地形、规定入流与守恒浅流结合，并以物理核验支撑到达状态发布。

6.2 模型的缺点

历史参考年份有限，因素位置随窗口和参考配置变化。材料柱采用局部平均状态及理想化压力剖面；局部首达与已形成流入口的实测对应尚待建立。传播结果依赖固定床、地形和设计入流；临界代理出现误报，物理核验增加计算费用。

6.3 模型的改进与推广

可补充多年份、多站点观测，扩展历史比较；通过材料试验与现场观测校核初态、渗透率和压力剖面。进一步建立局部触发至入口的状态和时钟对应，并结合实测河床、断面流量与多时空资料校核传播，在其他沟谷按地形与入口条件重新设置参数。

七、 参考文献

[1] 2026 年第七届 MathorCup 数学应用挑战赛——大数据竞赛组委会. 赛道 B：泥石流灾害预报及防灾策略 [Z]. 2026 年初赛题面，2026.

[2] 马耀明，马伟强，谢志鹏，王宾宾，陈学龙，韩存博，西藏吉隆高寒山区地气相互作用逐小时综合观测数据集（2021—2026），国家青藏高原科学数据中心，https://doi.org/10.11888/Atmos.tpdc.303600，访问日期 2026-10-24。

[3] Open-Meteo，Historical Weather API: models, hourly variables, time zones and grid-cell selection，https://open-meteo.com/en/docs/historical-weather-api，访问日期 2026-10-30。

[4] Wang B, Ma Y, Hu Z, et al. Quantifying the spatial-seasonal patterns of land–atmosphere water, heat and CO2 flux exchange over the Tibetan Plateau from an observational perspective，Earth System Science Data，18：1147–1164，2026. https://doi.org/10.5194/essd-18-1147-2026.

[5] Jackson J E, Mudholkar G S. Control Procedures for Residuals Associated With Principal Component Analysis，Technometrics，21(3)：341–349，1979. https://doi.org/10.1080/00401706.1979.10489779.

[6] 殷跃平，张仕林，高少华，等. 尼泊尔蓝塘里壤峰“8 · 26”冰岩崩致吉隆泥石流灾难研究 [Z]. 录用稿, 2026. https://doi.org/10.16031/j.cghc.202610008.

[7] Kang J, Lucas A, Mangeney A, et al. Frictional weakening in the highly mobile 2025 Blatten rock and ice avalanche in Switzerland，Communications Earth & Environment，7：722，2026. https://doi.org/10.1038/s43247-026-03983-1.

[8] Kang J, Lucas A, Mangeney A, et al.，Dataset for paper “Frictional weakening in the highly mobile 2025 Blatten rock and ice avalanche in Switzerland”，Zenodo，https://doi.org/10.5281/zenodo.21511788，访问日期 2026-10-30。

[9] Iverson R M, George D L. A depth-averaged debris-flow model that includes the effects of evolving dilatancy. I. Physical basis，Proceedings of the Royal Society A，470(2170)：20130819，2014. https://doi.org/10.1098/rspa.2013.0819.

[10] Fritsch F N, Butland J. A method for constructing local monotone piecewise cubic interpolants，SIAM Journal on Scientific and Statistical Computing，5(2)：300–304，1984. https://doi.org/10.1137/0905021.

[11] Clopper C J, Pearson E S. The use of confidence or fiducial limits illustrated in the case of the binomial，Biometrika，26(4)：404–413，1934. https://doi.org/10.1093/biomet/26.4.404.

[12] Copernicus，Copernicus WorldDEM-30 / GLO-30，https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM，访问日期 2026-10-30。

[13] NASA and METI，ASTER Global Digital Elevation Model V003，NOAA InPort，https://www.fisheries.noaa.gov/inport/item/59575/full-list，访问日期 2026-10-30。

[14] Langham J, Woodhouse M J, Hogg A J, et al.，Simulating shallow morphodynamic flows on evolving topographies，https://doi.org/10.48550/arXiv.2306.16185，访问日期 2026-10-30。

[15] Delestre O, Cordier S, Darboux F, James F. A limitation of the hydrostatic reconstruction technique for Shallow Water equations，Comptes Rendus Mathématique，350：677–681，2012. https://doi.org/10.1016/j.crma.2012.08.004.

[16] Renganathan S A, Rao V, Navon I M. CAMERA: A method for cost-aware, adaptive, multifidelity, efficient reliability analysis，Journal of Computational Physics，472：111698，2023. https://doi.org/10.1016/j.jcp.2022.111698.

[17] ZCode，GLM-5.3-Flash，智谱 Z.ai，2026-10-27 至 2026-10-30。

[18] DeepSeek，DeepSeek-V4.1-Flash-1017，深度求索，2026-10-25 至 2026-10-30。

AI 工具使用声明：使用的人工智能工具与日期见文献 [17, 18]。

八、 附录

附录 1：单侧假说与多尺度结果

为检查特定方向的历史位置，另列热力、浅土水分及 ERA5 格点降水的偏高单侧对照（表 A1）。单侧比较采用 $\max_w q$，目标与历史端点按相同择窗和式（5）校准。图 A1给出 17 项候选在各窗的双侧异常及方向。

表 A1　偏高方向对照的历史校准位置

因素	比较方向	单侧 S
2 m 均温 / 正温指数	偏高	0.788
2 m 窗口内最大小时均温	偏高	0.683
24 h 块均温变化率	偏高	0.824
0.1 m 含水量	偏高	0.776
0.1 m 土温	偏高	0.224
ERA5 累计总降水	偏高	0.948
ERA5 最大小时总降水量	偏高	0.948
ERA5 有降水小时数	偏高	0.613
ERA5 前期总降水指数	偏高	0.740

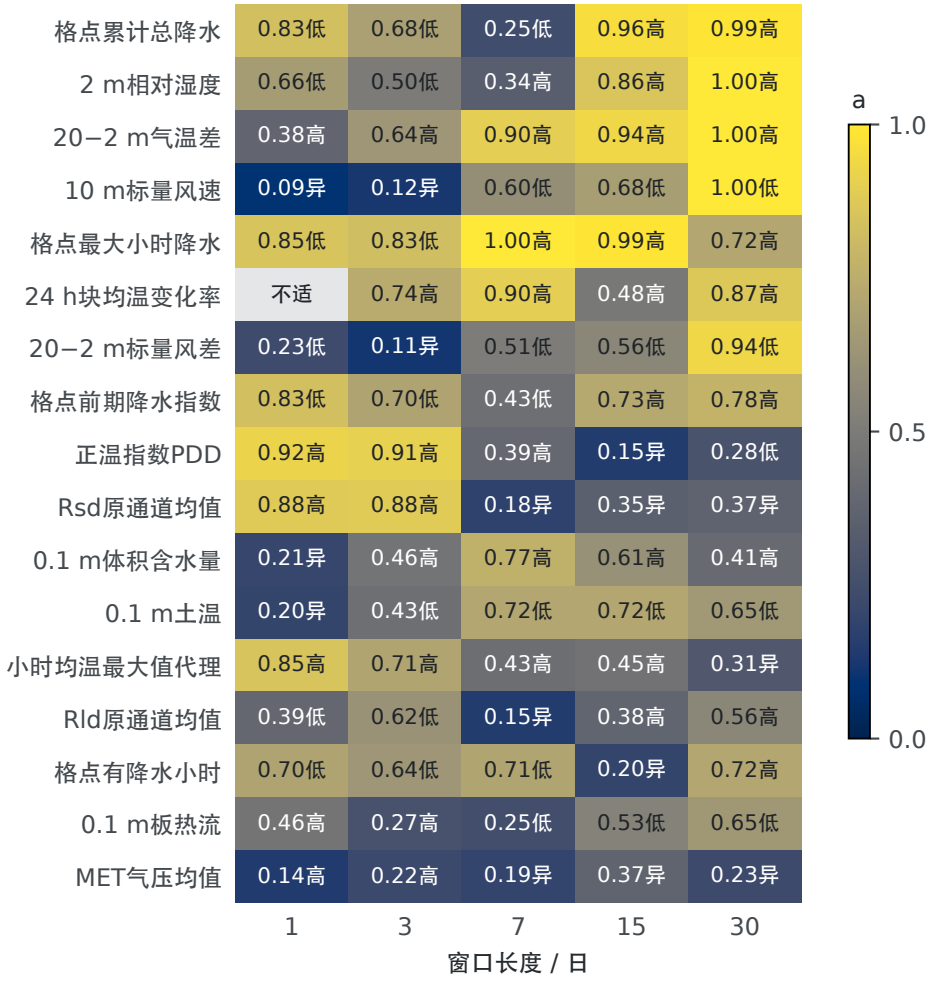


图 **A1**　17 项候选在各固定尺度的异常与方向。颜色和数字为各参考配置平均的单窗双侧异常 a；“高”“低”表示参考配置方向一致，“异”表示方向分歧，灰色为窗口不适用。

附录 2：软件与运行命令

以下列出运行环境及命令，输入路径和输出目录以附件实际内容为准。

软件与运行命令

Python 3.12。各问题依赖版本见对应requirements文件；数据准备另见prepare/requirements.txt。
↪ 第四问C加速需要C99编译器。中文绘图需要可用的中文字体。以下命令在对应问题目录运行，输出目录须为新目录。

问题一

```
python -B sol.py --mode enhance --d1-csv data/d1_era5/era5_precipitation_hourly_2021_2026.csv
↪ --d0-cache results/d0_window_cache.json.gz --baseline-results results/baseline
↪ --output-dir new_run
python -B draw.py --results-dir results --output-dir new_figures
```

问题二

```
python -B sol.py --output-dir new_results
python -B draw.py --results-dir results --output-dir new_figures
```

问题三

```
python -B sol.py --data data --results results_new --workers 2
python -B draw.py --results results --output figures_new
```

问题四（以下从附件根目录运行）

```
python -m pip install -r q4/requirements-lock.txt
python -B q4/code/sol.py decide --out q4_run/new2 --inputs q4/data/local --backend c --volume
↪ 475682.8460010884 --mu .075 --gp-state q4/data/GP_STATE.json --cache-entry
↪ q4/data/cache/new2/CACHE_ENTRY.json --cache-only
python -B q4/code/draw.py --root q4 --out q4_guard_figures
python -B q4/followup/code/draw.py --root q4/followup --out q4_followup_figures
python -B q4/base/code/draw.py --root q4/base --out q4_base_figures
```

原始附件清理（在 prepare 目录内）

```
python -B clean_all.py --input-root " 官方附件目录" --output-dir "new_cleaned_all"
python -B clean_data.py --input-root " 官方附件目录" --output-dir "new_station_daily"
python -B era5/download.py
python -B route.py --output route_new.csv
```

全清理需要使用者提供原始1—8附件，站点子入口只生成站点/日温结果；取数脚本缺原响应时会联网取得公开数据。
↪ 主计算已带必要输入，通常不必重新取数或清理。

附录 3：完整程序代码

以下依次列出数据准备及四问源程序。

数据准备

prepare/clean_all.py

```
import argparse
import csv
import datetime as dt
import hashlib
import importlib.metadata
import json
from pathlib import Path
import shutil
import sys

import clean_data
from netcdf_cleaning import clean_netcdf
from raster_cleaning import clean_rasters
from vector_cleaning import clean_vectors


BASE = Path(__file__).resolve().parent
CODE_FILES = ["clean_all.py", "clean_data.py", "quality_policy.py", "quality_diagnostics.py",
              "quality_rules.json", "netcdf_cleaning.py", "raster_cleaning.py",
              ↪  "vector_cleaning.py"]


def sha(path):
    h = hashlib.sha256()
    with path.open("rb") as source:
        for block in iter(lambda: source.read(1024 * 1024), b""):
            h.update(block)
    return h.hexdigest()


def dump(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
    ↪  encoding="utf-8")


def inventory(input_root):
    folders = {}
    for number in range(1, 9):
        matches = [p for p in input_root.iterdir() if p.is_dir() and p.name.startswith((f"
        ↪  ({number}) ", f"({number})"))]
        if len(matches) != 1:
            raise ValueError(f"Expected one folder for attachment {number}, found
            ↪  {len(matches)}")
        folders[number] = matches[0]
    records = []
    for number, folder in folders.items():
        for path in sorted(p for p in folder.rglob("*") if p.is_file()):
```

```
            suffix = path.suffix.lower()
            primary = {1: {".csv": "station_csv"}, 2: {".xlsx": "daily_xlsx"},
                       3: {".tif": "glacier_raster", ".shp": "glacier_vector"},
                       4: {".nc": "glacier_netcdf"}, 5: {".tif": "landcover_raster"},
                       6: {".shp": "historical_disaster_vector"}, 7: {".tif": "dem_raster"},
                       8: {".shp": "population_vector"}}
            if path.name.startswith(("._", ".DS_Store", "~$")) or "__MACOSX" in path.parts:
                role, handling = "operating_system_metadata", "listed_only_not_data"
            elif suffix == ".lock":
                role, handling = "software_lock", "listed_only_not_data"
            elif suffix in primary[number]:
                role, handling = primary[number][suffix], "cleaned"
            elif suffix in {".doc", ".docx", ".pdf", ".jpg", ".png"}:
                role, handling = "reference_document_or_image", "copied_unchanged_to_support"
            elif suffix in {".dbf", ".shx", ".prj", ".cpg", ".sbn", ".sbx", ".xml", ".tfw",
            ↪  ".ovr"}:
                role, handling = "source_companion", "copied_unchanged_to_support"
            else:
                raise ValueError(f"Unclassified attachment file (not silently skipped):
                ↪  {path}")
            archive = handling == "copied_unchanged_to_support" or suffix == ".shp"
            support = (Path("support") / f"attachment{number:02d}" /
            ↪  path.relative_to(folder)).as_posix() if archive else ""
            records.append(dict(attachment=number,
            ↪  input_path=path.relative_to(input_root).as_posix(),
                                bytes=path.stat().st_size, sha256=sha(path), role=role,
                                ↪  handling=handling,
                                support_copy=support))
    return records


def main(input_root, output_dir):
    input_root, out = input_root.resolve(), output_dir.resolve()
    if not input_root.is_dir():
        raise ValueError("--input-root must be the official 附件 directory")
    if out.exists():
        raise ValueError("Output exists; choose a new directory to preserve previous
        ↪  deliveries")
    if input_root == out or input_root in out.parents:
        raise ValueError("Output must be outside the original attachment directory")
    started = dt.datetime.now(dt.timezone.utc).isoformat()
    sources = inventory(input_root)
    code_before = {name: sha(BASE / name) for name in CODE_FILES}
    out.mkdir(parents=True)
    (out / "quality").mkdir()
    with (out / "input_inventory_all.csv").open("w", encoding="utf-8-sig", newline="") as
    ↪  text:
        writer = csv.DictWriter(text, fieldnames=list(sources[0]))
        writer.writeheader()
        writer.writerows(sources)

    clean_data.main(input_root, out / "attachment01_02")
    tabular = json.loads((out /
    ↪  "attachment01_02/run_summary.json").read_text(encoding="utf-8"))
```

```
rasters = clean_rasters(input_root, out)
vectors = clean_vectors(input_root, out)
netcdf = clean_netcdf(input_root, out)
for source in sources:
    if source["support_copy"]:
        target = out / source["support_copy"]
        target.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(input_root / source["input_path"], target)
        if sha(target) != source["sha256"]:
            raise RuntimeError(f"Support file changed: {target.name}")
if any(sha(input_root / source["input_path"]) != source["sha256"] for source in sources):
    raise RuntimeError("An original attachment changed during cleaning")
if any(sha(BASE / name) != digest for name, digest in code_before.items()):
    raise RuntimeError("Cleaner code changed during execution")
versions = {name: importlib.metadata.version(name) for name in [
    "numpy", "pandas", "openpyxl", "rasterio", "geopandas", "pyogrio", "pyproj",
    ↪  "shapely", "netCDF4"]}
summary = dict(scope="all eight official attachments; basic cleaning only",
↪  started_utc=started,
    finished_utc=dt.datetime.now(dt.timezone.utc).isoformat(),
    input_files=len(sources), primary_files=sum(s["handling"] == "cleaned" for s in
    ↪  sources),
    copied_support_files=sum(bool(s["support_copy"]) for s in sources),
    ignored_non_data_files=[s["input_path"] for s in sources if s["handling"] ==
    ↪  "listed_only_not_data"],
    counts_by_attachment={str(i): dict(
        files=sum(s["attachment"] == i for s in sources),
        primary_files=sum(s["attachment"] == i and s["handling"] == "cleaned" for s in
        ↪  sources)) for i in range(1, 9)},
    tabular=dict(source_files=tabular["input_files"],
    ↪  station_rows=tabular["station_rows"], daily_rows=tabular["daily_rows"],
                measurement_cells=tabular["cells"],
                ↪  retained_values=tabular["analysis_present"],
                ↪  statuses=tabular["statuses"]),
    rasters=rasters, vectors=vectors, netcdf=netcdf,
    source_inputs_unchanged=True, code_sha256=code_before,
    environment=dict(python=sys.version.split()[0], **versions),
    imputation=False, clipping=False, reprojection=False, resampling=False,
    model_training=False, new_observations=False,
    limitations=[
        "Station interval/aggregation and daily temperature units/day boundary remain
        ↪  unconfirmed.",
        "Raster native grids/CRS/NoData are preserved; different grids are not
        ↪  automatically aligned.",
        "Glacier dhdt units, land-cover class meanings and vertical datums remain
        ↪  partially unconfirmed.",
        "Original invalid vector geometries remain in all_features layers and are
        ↪  isolated in analysis_valid_geometry layers.",
        "Historical-disaster geometries are polygons; no centroids or artificial
        ↪  dates/labels are created.",
        "Attachment04 velocity fields are climatological, with no time dimension.",
        "Population attribute meanings/totals are preserved; no guessed reallocation or
        ↪  normalization."],
    output_sha256={p.relative_to(out).as_posix(): sha(p) for p in sorted(out.rglob("*"))
    ↪  if p.is_file()})
```

```
    dump(out / "run_summary_all.json", summary)
    print(json.dumps({k: summary[k] for k in ["scope", "input_files", "primary_files",
    ↪  "copied_support_files", "tabular"]},
                     ensure_ascii=False, indent=2), flush=True)


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input-root", type=Path, required=True, help="Official 附件 folder
    ↪  with （1）…（8）")
    parser.add_argument("--output-dir", type=Path, required=True, help="New output folder
    ↪  outside originals")
    args = parser.parse_args()
    main(args.input_root, args.output_dir)
```

prepare/clean_data.py

```
import argparse
from contextlib import ExitStack, contextmanager
from decimal import Decimal, InvalidOperation
import collections
import csv
import datetime as dt
import gzip
import hashlib
import io
import json
import math
import pathlib
import re
import sys
import xml.etree.ElementTree as ET
import zipfile
import pandas as pd
import quality_policy as API
from quality_diagnostics import diagnose

BASE = pathlib.Path(__file__).resolve().parent
RULES = json.loads((BASE / "quality_rules.json").read_text(encoding="utf-8"))
NS = {"m": "http://schemas.openxmlformats.org/spreadsheetml/2006/main"}


def physical(variable, station=True):
    if not station: return "none_daily_unit_pending", None
    rules = RULES["physical_rules"]
    if variable.startswith(("Ta_", "soilT_")) or variable == "T_SONIC(deg C)": key =
    ↪  "temperature_C"
    elif variable in rules["pressure"]["columns"]: key = "pressure"
    elif variable.startswith("VWC_") and variable.endswith("(m3/m3)"): key = "vwc"
    else: return "none_source_definition_pending", None
    r = rules[key]
    return key, API.PhysicalBounds(lower=r.get("lower", -math.inf), upper=r.get("upper",
    ↪  math.inf),
        lower_inclusive=r.get("lower_inclusive", True),
        ↪  upper_inclusive=r.get("upper_inclusive", True))
```

```
NUMBER_GRAMMAR = re.compile(r"[+-]?(?:[0-9]+(?:\.[0-9]*)?|\.[0-9]+)(?:[eE][+-]?[0-9]+)?\Z",
↪  re.ASCII)
ASCII_SPACE = " \t\r\n\v\f"


def parse_number(raw, station=True):

    if not isinstance(raw, str):
        return None, "nonnumeric"
    token = raw.strip(ASCII_SPACE)
    if not token:
        return None, "empty_token"
    if re.fullmatch(r"[+-]?(?:inf(?:inity)?|s?nan[0-9]*)", token, re.IGNORECASE | re.ASCII):
        return None, "nonfinite"
    if not NUMBER_GRAMMAR.fullmatch(token):
        return None, "nonnumeric"
    try:
        exact = Decimal(token)
    except InvalidOperation:
        return None, "decimal_conversion_invalid"
    if not exact.is_finite():
        return None, "nonfinite"
    if station and exact == Decimal("9999.9"):
        return None, "declared_sentinel"
    try:
        value = float(exact)
    except (ValueError, OverflowError):
        return None, "numeric_conversion_nonfinite"
    if not math.isfinite(value):
        return None, "numeric_conversion_nonfinite"
    if exact != 0 and value == 0:
        return None, "numeric_conversion_underflow"
    return value, None


def assess(raw, variable, presence, qc_raw, label_status="unique", station=True):
    qc = None
    if qc_raw.strip():
        try: qc = float(qc_raw)
        except ValueError: qc = qc_raw
    rule, bounds = physical(variable, station)
    val, parse_reason = parse_number(raw, station)

    out = API.assess_cell(val if parse_reason is None else math.nan,
        source_qc_presence=presence, source_qc=qc, physical_bounds=bounds,
        policy=API.QualityPolicy(no_source_qc=RULES["no_source_qc"],
            retain_flagged_codes=tuple(c for c in RULES["main_allowed_qc_codes"] if c != 0),
            sentinel_tokens=()))
    out["raw_value"] = raw
    out["structural_status"] = "pass" if parse_reason is None else "fail"
    reasons = ([parse_reason] if parse_reason else []) + [r for r in out["reasons"] if r not
    ↪  in {"nonfinite","nonnumeric","declared_sentinel"}]
```

```
    flags = []
    if station and variable == "PA(kPa)":
        flags.append("pa_definition_reference_pending")
        reasons.append("pa_definition_reference_pending")
    if val is not None and parse_reason is None:
        if val < 0 and variable == "Precip(mm)":
        ↪  flags.append("precip_nonnegative_definition_pending")
        if val < 0 and variable == "USTAR(m/s)":
        ↪  flags.append("ustar_nonnegative_definition_pending")
        if val < 0 and variable in {"Rsd(W/m2)","Rsu(W/m2)","Rld(W/m2)","Rlu(W/m2)"}:
            flags.append("radiation_direction_noise_pending")
    if label_status != "unique": reasons.append("time_label_"+label_status)
    structural = parse_reason is None
    source_candidate = structural and out["source_policy_eligible"]
    candidate = source_candidate and out["physical_status"] != "fail" and label_status ==
    ↪  "unique"
    screened = candidate and out["physical_status"] == "pass"
    if not structural: part = "quarantine_structure"
    elif not out["source_policy_eligible"]: part = "quarantine_source_qc"
    elif out["physical_status"] == "fail": part = "quarantine_physical"
    elif label_status != "unique": part = "hold_time_label_conflict"
    elif out["physical_status"] != "pass": part = "retain_physical_pending"
    else: part = "retain_screened_candidate"
    out.update(physical_rule=rule, conditional_flags=";".join(flags),
    ↪  primary_disposition=part, empirical_status="not_evaluated",
        reasons=";".join(dict.fromkeys(reasons)), time_label_status=label_status,
        source_candidate_value=val if source_candidate else None,
        candidate_value=val if candidate else None, screened_candidate_value=val if screened
        ↪  else None,
        research_candidate=bool(screened))
    return out


def parse_label(raw):
    try: return dt.datetime.strptime(raw.replace("/", "-"), "%Y-%m-%d %H:%M")
    except ValueError: return None


def label_statuses(labels, records, year, hourly):
    counts = collections.Counter(x for x in labels if x is not None)
    statuses = []
    prev = None
    for i, t in enumerate(labels):
        why = []
        if t is None: why.append("unparseable")
        else:
            if counts[t] > 1: why.append("duplicate")
            if t.year != year: why.append("year_mismatch")
            if t.minute or t.second or t.microsecond or (not hourly and t.hour):
            ↪  why.append("off_grid")
            if prev and t < prev: why.append("out_of_order")
            prev = t
        statuses.append("+".join(why) or "unique")
    conflicts = []
```

```
    for t, n in counts.items():
        if n > 1:
            ix = [i for i, x in enumerate(labels) if x == t]
            conflicts.append({"timestamp_label":t.isoformat(),"rows":[i+2 for i in
            ↪  ix],"count":n,
                              "identical_full_records":len({tuple(map(str,records[i])) for i
                              ↪  in ix}) == 1})
    return statuses, conflicts


def excel_rows(path):

    import openpyxl
    decoded = openpyxl.load_workbook(path, read_only=True, data_only=False)
    try:
        with zipfile.ZipFile(path) as z:
            strings = []
            if "xl/sharedStrings.xml" in z.namelist():
                strings = ["".join(n.itertext()) for n in
                ↪  ET.fromstring(z.read("xl/sharedStrings.xml"))]

            relns = "{http://schemas.openxmlformats.org/package/2006/relationships}"
            rels = {a.attrib["Id"]: a.attrib["Target"] for a in
            ↪  ET.fromstring(z.read("xl/_rels/workbook.xml.rels"))}
            names = ET.fromstring(z.read("xl/workbook.xml")).find("m:sheets", NS)
            for s in names:
                rid = s.attrib["{http://schemas.openxmlformats.org/officeDocument/2006/relati⌟
                ↪  onships}id"]
                target = rels[rid]
                target = target.lstrip("/") if target.startswith("/") else "xl/"+target
                sheet = s.attrib["name"]
                xml = ET.fromstring(z.read(target))
                raw_rows = []
                for row in xml.findall("m:sheetData/m:row", NS):
                    cells = {}
                    for c in row:
                        ref = c.attrib["r"]
                        typ = c.attrib.get("t", "n")
                        v = c.find("m:v", NS)
                        token = v.text if v is not None and v.text is not None else ""
                        if typ == "s": value = strings[int(token)]
                        elif typ == "inlineStr": value = "".join(c.find("m:is",
                        ↪  NS).itertext())
                        else: value = token
                        formula = c.find("m:f", NS)
                        cells[re.sub(r"[0-9]", "", ref)] = (value,typ,formula.text if formula
                        ↪  is not None else "",ref)
                    raw_rows.append((int(row.attrib["r"]), cells))
                yield sheet, raw_rows, list(decoded[sheet].values)
    finally: decoded.close()


LOCATION = ["source_file", "source_sheet", "source_row", "timestamp_raw", "timestamp_label"]
FLAG_FIELDS = ["dataset", "source_file", "source_sheet", "source_row", "source_column",
```

```
              "cell_reference", "timestamp_label", "variable", "raw_value", "source_qc",
              "source_quality_status", "status", "reasons"]


def sha(path):
    h = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            h.update(block)
    return h.hexdigest()


@contextmanager
def gz_writer(path, fields):

    with path.open("wb") as binary, gzip.GzipFile(
            filename="", fileobj=binary, mode="wb", mtime=0, compresslevel=6) as gz:
        with io.TextIOWrapper(gz, encoding="utf-8", newline="") as text:
            writer = csv.DictWriter(text, fieldnames=fields)
            writer.writeheader()
            yield writer


def write_csv(path, rows, fields=None):
    fields = fields or list(rows[0])
    with path.open("w", encoding="utf-8-sig", newline="") as text:
        writer = csv.DictWriter(text, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)


def discover(root):

    found = []
    for number, pattern, station in [(1, "*.csv", True), (2, "*.xlsx", False)]:
        folders = [p for p in root.iterdir() if p.is_dir() and
                   p.name.startswith((f"（{number}）", f"({number})"))]
        if len(folders) != 1:
            raise ValueError(f"Expected one attachment ({number}) folder directly inside
            ↪  {root}")
        paths = sorted(p for p in folders[0].rglob(pattern)
                       if not p.name.startswith(("._", "~$")) and "__MACOSX" not in p.parts)
        if not paths:
            raise ValueError(f"No {pattern} inputs inside {folders[0]}")
        for path in paths:
            if station and not re.fullmatch(r"(MET|RADM|SOIL|FLUX)_Gyirong_20[0-9]{2}\.csv",
            ↪  path.name):
                raise ValueError(f"Unrecognized station CSV (not silently skipped): {path}")
            year_match = re.search(r"(20[0-9]{2})(?:\.csv|\.xlsx)$", path.name)
            if not year_match:
                raise ValueError(f"No source year in filename: {path.name}")
            found.append(dict(path=path, station=station, year=int(year_match[1]),
                              dataset=path.name.split("_")[0] if station else
                              ↪  "daily_temperature"))
```

```
    return found


def read_sheets(source):

    path, station = source["path"], source["station"]
    if station:
        with path.open(encoding="utf-8-sig", newline="") as text:
            reader = csv.reader(text)
            header, data = next(reader), list(reader)
        if len(set(header)) != len(header) or any(len(row) != len(header) for row in data):
            raise ValueError(f"Duplicate header or inconsistent row width: {path.name}")
        columns = [j for j, name in enumerate(header) if j and not name.startswith("QC_")]
        qcmap = {}
        for j in columns:
            wanted = "qc_" + header[j].split("(")[0].lower()
            hits = [k for k, name in enumerate(header) if name.lower() == wanted]
            if len(hits) > 1:
                raise ValueError(f"Ambiguous QC column: {path.name}, {header[j]}")
            qcmap[j] = hits[0] if hits else None
        if {k for k, name in enumerate(header) if name.startswith("QC_")} != set(
                k for k in qcmap.values() if k is not None):
            raise ValueError(f"Unmapped QC column: {path.name}")
        yield "", header, data, columns, qcmap, [
            (i + 2, parse_label(row[0]), {}) for i, row in enumerate(data)]
    else:
        for sheet, raw, decoded in excel_rows(path):
            if not raw or raw[0][0] != 1:
                raise ValueError(f"Missing row-1 header: {path.name}/{sheet}")
            letters = list(raw[0][1])
            header = [raw[0][1][letter][0] for letter in letters]
            if header[0] != " 时间" or len(set(header)) != len(header):
                raise ValueError(f"Unexpected daily header: {path.name}/{sheet}")
            data, extra = [], []
            for rn, cells in raw[1:]:
                row = [cells.get(c, ("", "missing", "", f"{c}{rn}"))[0] for c in letters]
                data.append(row)
                value = decoded[rn - 1][0]
                label = value if isinstance(value, dt.datetime) else None
                extra.append((rn, label, {j: cells.get(c, ("", "missing", "", f"{c}{rn}"))
                                          for j, c in enumerate(letters)}))
            yield sheet, header, data, list(range(1, len(header))), {
                j: None for j in range(1, len(header))}, extra


def disposition(a):

    part = a["primary_disposition"]
    if part == "quarantine_structure":
        return "missing" if a["reasons"].split(";")[0] in {
            "empty_token", "declared_sentinel", "nonfinite"} else "excluded_parse"
    if part == "quarantine_source_qc":
        return {"source_flag_8_missing": "missing", "source_flag_2_wrong":
        ↪  "excluded_source_error"}.get(
```

```
            a["source_quality_status"], "excluded_qc_policy")
    if part == "quarantine_physical":
        return "excluded_physical"
    if part == "hold_time_label_conflict":
        return "excluded_time"
    if a["physical_status"] != "pass" or a["source_qc_presence"] == "not_provided" or
    ↪  a["conditional_flags"]:
        return "retained_flagged"
    return "retained"


def flag_reasons(a):
    reasons = [x for x in a["reasons"].split(";")
               if x and x != "unknown_or_invalid_support"]
    if a["source_qc_presence"] == "not_provided":
        reasons.append("source_qc_not_provided")
    reasons.extend(a["conditional_flags"].split(";") if a["conditional_flags"] else [])
    return ";".join(dict.fromkeys(reasons))


def main(input_root, output_dir):
    root, out = input_root.resolve(), output_dir.resolve()
    inputs = discover(root)
    if out.exists():
        raise ValueError(f"Output already exists; choose a new directory to preserve prior
        ↪  results: {out}")
    if out == root or root in out.parents:
        raise ValueError("Put outputs outside the original attachment directory")
    started = dt.datetime.now(dt.timezone.utc).isoformat()
    before = {s["path"]: sha(s["path"]) for s in inputs}

    label_counts = collections.defaultdict(collections.Counter)
    schemas = {}
    for source in inputs:
        for sheet, header, data, columns, qcmap, extra in read_sheets(source):
            dataset = source["dataset"]
            if dataset in schemas and schemas[dataset] != header:
                raise ValueError(f"Schema drift in {source['path'].name}; explicit mapping
                ↪  needed")
            schemas[dataset] = header
            label_counts[dataset].update(t for _, t, _ in extra if t is not None)
    out.mkdir(parents=True)
    (out / "station").mkdir()
    (out / "daily").mkdir()
    inventory, summaries, dictionary, diagnostics, time_issues, gaps = [], [], {}, [], [], []
    total_status = collections.Counter()
    totals = collections.Counter()
    with ExitStack() as stack:
        flags = stack.enter_context(gz_writer(out / "quality_flags.csv.gz", FLAG_FIELDS))
        writers = {}
        for dataset, header in schemas.items():
            target = (out / "daily/temperature.csv.gz" if dataset == "daily_temperature"
                      else out / f"station/{dataset}.csv.gz")
            writers[dataset] = stack.enter_context(gz_writer(target, LOCATION + header[1:]))
```

```
for source in inputs:
    path, station, year, dataset = (source[k] for k in ["path", "station", "year",
    ↪  "dataset"])
    for sheet, header, data, columns, qcmap, extra in read_sheets(source):
        labels = [e[1] for e in extra]
        statuses, _ = label_statuses(labels, data, year, station)
        for i, label in enumerate(labels):
            if label is not None and label_counts[dataset][label] > 1 and "duplicate"
            ↪  not in statuses[i]:
                statuses[i] = "cross_file_duplicate" if statuses[i] == "unique" else
                ↪  statuses[i] + "+cross_file_duplicate"
        stats = {j: dict(status=collections.Counter(), qc=collections.Counter(),
                         reasons=collections.Counter(), present=0, physical_pass=0,
                         first="", last="", minimum=None, maximum=None) for j in
                         ↪  columns}
        cleaned = []
        for i, row in enumerate(data):
            rn, label, cell_info = extra[i]
            label_text = (label.isoformat() + ("+08:00" if station else "")) if label
            ↪  else ""
            result = dict(source_file=path.name, source_sheet=sheet, source_row=rn,
                          timestamp_raw=row[0], timestamp_label=label_text)
            if statuses[i] != "unique":
                time_issues.append(dict(dataset=dataset, source_file=path.name,
                ↪  source_sheet=sheet,
                                        source_row=rn, timestamp_raw=row[0],
                                        ↪  status=statuses[i]))
            for j in columns:
                qi = qcmap[j]
                qraw = row[qi] if qi is not None else ""
                a = assess(row[j], header[j], "provided" if qi is not None else
                ↪  "not_provided",
                           qraw, statuses[i], station)
                ref = str(j + 1)
                if not station:
                    _, typ, formula, ref = cell_info[j]
                    if formula or typ in {"b", "e"}:
                        a.update(candidate_value=None,
                        ↪  primary_disposition="quarantine_structure",
                                 structural_status="fail",
                                 ↪  reasons="formula_not_evaluated" if formula else
                                 ↪  "nonnumeric_xml_type")
                value, status = a["candidate_value"], disposition(a)
                result[header[j]] = value
                if qi is not None:
                    result[header[qi]] = qraw
                s = stats[j]
                s["status"][status] += 1
                s["qc"][qraw if qi is not None else "not_provided"] += 1
                s["present"] += value is not None
                s["physical_pass"] += value is not None and a["physical_status"] ==
                ↪  "pass"
                if value is not None:
                    s["first"] = s["first"] or label_text
```

```
            s["last"] = label_text
            s["minimum"] = value if s["minimum"] is None else
            ↪  min(s["minimum"], value)
            s["maximum"] = value if s["maximum"] is None else
            ↪  max(s["maximum"], value)
        if status != "retained":
            reason = flag_reasons(a)
            s["reasons"].update(reason.split(";"))
            flags.writerow(dict(dataset=dataset, source_file=path.name,
            ↪  source_sheet=sheet,
                source_row=rn, source_column=j + 1, cell_reference=ref,
                timestamp_label=label_text, variable=header[j],
                ↪  raw_value=row[j], source_qc=qraw,
                source_quality_status=a["source_quality_status"],
                ↪  status=status, reasons=reason))
        key = dataset, header[j]
        unit_match = re.search(r"\((.*)\)$", header[j]) if station else None
        dictionary[key] = dict(dataset=dataset, variable=header[j],
            unit=unit_match[1] if unit_match else "unknown_in_local_workbook",
            unit_basis="source_header" if unit_match else "not_declared",
            qc_column=header[qi] if qi is not None else "not_provided",
            physical_rule=physical(header[j], station)[0],
            timezone="UTC+08:00" if station else "not_declared",
            time_support="label_only; interval/aggregation_unconfirmed" if
            ↪  station else "date_only; day_boundary_unconfirmed")
    writers[dataset].writerow(result)
    cleaned.append(result)
valid_labels = sorted(set(t for t in labels if t is not None))
step = dt.timedelta(hours=1) if station else dt.timedelta(days=1)
for left, right in zip(valid_labels, valid_labels[1:]):
    if right - left > step:
        gaps.append(dict(dataset=dataset, source_file=path.name,
        ↪  source_sheet=sheet,
                         start=(left + step).isoformat(), end=(right -
                         ↪  step).isoformat(),
                         nominal_missing_labels=int((right - left) / step) -
                         ↪  1,
                         basis="between_existing_source_labels; no rows
                         ↪  inserted"))
for j in columns:
    s = stats[j]
    if sum(s["status"].values()) != len(data):
        raise RuntimeError("Cell counts do not close")
    summaries.append(dict(dataset=dataset, source_file=path.name,
    ↪  source_sheet=sheet,
        variable=header[j], cells=len(data), analysis_present=s["present"],
        analysis_missing=len(data) - s["present"],
        necessary_physical_screen_present=s["physical_pass"],
        first_analysis_label=s["first"], last_analysis_label=s["last"],
        min_analysis_value=s["minimum"], max_analysis_value=s["maximum"],
        statuses=json.dumps(s["status"], ensure_ascii=False, sort_keys=True),
        source_qc_counts=json.dumps(s["qc"], ensure_ascii=False,
        ↪  sort_keys=True),
        reason_counts=json.dumps(s["reasons"], ensure_ascii=False,
        ↪  sort_keys=True)))
```

```
                total_status.update(s["status"])
                totals.update(cells=len(data), analysis_present=s["present"],
                ↪  physical_screen_present=s["physical_pass"])
            inventory.append(dict(dataset=dataset, source_file=path.name,
            ↪  source_sheet=sheet,
                input_path=path.relative_to(root).as_posix(), sha256=before[path],
                encoding="utf-8-sig strict" if station else "OOXML raw numeric tokens +
                ↪  decoded date labels",
                rows=len(data), measurement_columns=len(columns),
                first_label=valid_labels[0].isoformat() if valid_labels else "",
                last_label=valid_labels[-1].isoformat() if valid_labels else "",
                time_status_counts=json.dumps(collections.Counter(statuses),
                ↪  ensure_ascii=False),
                output_path=f"station/{dataset}.csv.gz" if station else
                ↪  "daily/temperature.csv.gz"))
            diagnostics.extend(diagnose(pd.DataFrame(cleaned), [header[j] for j in
            ↪  columns],
                                        dataset=dataset, step_hours=1 if station else 24))
            print(f"{path.name}: {len(data):,} rows, {len(columns)} measurement columns",
            ↪  flush=True)
if not all(sha(s["path"]) == before[s["path"]] for s in inputs):
    raise RuntimeError("A source input changed during cleaning")
write_csv(out / "input_inventory.csv", inventory)
write_csv(out / "variable_summary.csv", summaries)
write_csv(out / "variable_dictionary.csv", list(dictionary.values()))
write_csv(out / "internal_diagnostics.csv", diagnostics,
          ["dataset", "source_file", "check", "variable", "count", "start", "end",
          ↪  "value_before", "value_after", "detail"])
write_csv(out / "time_issues.csv", time_issues,
          ["dataset", "source_file", "source_sheet", "source_row", "timestamp_raw",
          ↪  "status"])
write_csv(out / "record_gaps.csv", gaps,
          ["dataset", "source_file", "source_sheet", "start", "end",
          ↪  "nominal_missing_labels", "basis"])
summary = dict(started_utc=started,
↪  finished_utc=dt.datetime.now(dt.timezone.utc).isoformat(),
    scope="official attachments 1 and 2 only", input_files=len(inputs),
    station_files=sum(s["station"] for s in inputs), daily_files=sum(not s["station"] for
    ↪  s in inputs),
    station_rows=sum(x["rows"] for x in inventory if x["dataset"] != "daily_temperature"),
    daily_rows=sum(x["rows"] for x in inventory if x["dataset"] == "daily_temperature"),
    **totals, statuses=dict(total_status), source_inputs_unchanged=True,
    imputed_here=0, clipped_here=0, unit_conversions=0, inserted_observation_rows=0,
    model_or_factor_computation=False, time_issue_rows=len(time_issues),
    ↪  record_gap_intervals=len(gaps),
    code_sha256={p.name: sha(p) for p in [BASE / "clean_data.py", BASE /
    ↪  "quality_policy.py",
                                        BASE / "quality_diagnostics.py", BASE /
                                        ↪  "quality_rules.json"]},
    environment=dict(python=sys.version.split()[0], pandas=pd.__version__,
                     openpyxl=__import__("openpyxl").__version__),
    output_sha256={p.relative_to(out).as_posix(): sha(p) for p in sorted(out.rglob("*"))
    ↪  if p.is_file()},
    limitations=["Retained numbers have passed the declared cleaning policy, not every
    ↪  possible quality check.",
```

```
                    "No-source-QC and physical-definition-pending values are retained and
                    ↪  flagged.",
                    "Station time labels are UTC+8; interval endpoints/aggregation remain
                    ↪  unconfirmed.",
                    "Daily units/day boundary/spatial extraction are not declared locally;
                    ↪  no conversion.",
                    "PA is not treated as MET pressure; precipitation is not accumulated
                    ↪  here."])
    (out / "run_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2) +
    ↪  "\n", encoding="utf-8")
    print(json.dumps({k: summary[k] for k in ["input_files", "station_rows", "daily_rows",
    ↪  "cells", "analysis_present", "statuses"]},
                     ensure_ascii=False, indent=2))


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input-root", type=pathlib.Path, required=True, help="Official 附件
    ↪  directory containing （1） and （2）")
    parser.add_argument("--output-dir", type=pathlib.Path, required=True, help="New output
    ↪  directory outside original attachments")
    args = parser.parse_args()
    main(args.input_root, args.output_dir)
```

prepare/era5/download.py

```
from __future__ import annotations

import argparse
import csv
import datetime as dt
import hashlib
import json
import math
from pathlib import Path
from urllib.parse import urlencode
from urllib.request import Request, urlopen

BASE = Path(__file__).resolve().parent
ENDPOINT = "https://archive-api.open-meteo.com/v1/archive"
DOC_URL = "https://open-meteo.com/en/docs/historical-weather-api"
ZONE = dt.timezone(dt.timedelta(hours=8))


def sha(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def fetch(url: str) -> tuple[bytes, dict]:
    request = Request(url, headers={"User-Agent": "MathorCup-Q1-research/1.0"})
    with urlopen(request, timeout=55) as response:
        body = response.read()
        metadata = {"http_status": response.status,
                    "retrieved_at_utc": dt.datetime.now(dt.timezone.utc).isoformat(),
                    "response_headers": dict(response.headers)}
```

```
    return body, metadata


def parse_year(year: int, obj: dict) -> tuple[list[dict], dict]:
    if obj.get("error"):
        raise ValueError(f"API error: {obj}")
    assert obj["utc_offset_seconds"] == 28800
    assert obj["timezone"] == "Asia/Shanghai"
    assert obj["hourly_units"]["precipitation"] == "mm"
    assert obj["hourly_units"]["rain"] == "mm"
    times = obj["hourly"]["time"]
    precip = obj["hourly"]["precipitation"]
    rain = obj["hourly"]["rain"]
    assert len(times) == len(precip) == len(rain)
    start = dt.datetime(year, 5, 1, tzinfo=ZONE)
    last = dt.datetime(year, 10, 16, 23, tzinfo=ZONE)
    expected = int((last - start).total_seconds() / 3600) + 1
    assert len(times) == expected, (year, len(times), expected)
    rows = []
    for i, (label, p, r) in enumerate(zip(times, precip, rain)):
        end = dt.datetime.fromisoformat(label).replace(tzinfo=ZONE)
        assert end == start + dt.timedelta(hours=i), (year, i, label)
        for value in (p, r):
            assert value is not None and math.isfinite(value) and value >= 0, (label, p, r)

        rows.append({"interval_start": (end-dt.timedelta(hours=1)).isoformat(),
                     "interval_end": end.isoformat(),
                     "precipitation_mm": p, "rain_mm": r,
                     "source_product": "Open-Meteo archive / ERA5",
                     "source_qc_status": "not_provided",
                     "grid_latitude": obj["latitude"],
                     "grid_longitude": obj["longitude"],
                     "grid_elevation_m": obj["elevation"]})
    checks = {"year": year, "status": "pass", "requested_first_label": start.isoformat(),
              "requested_last_label": last.isoformat(), "expected_count": expected,
              "actual_count": len(rows), "missing_precipitation": 0, "missing_rain": 0,
              "nonfinite_values": 0, "negative_values": 0, "duplicate_labels": 0,
              "hourly_continuity": True, "sum_precipitation_mm": round(sum(precip), 10),
              "sum_rain_mm": round(sum(rain), 10),
              "max_precipitation_mm": max(precip), "wet_hours_gt_zero": sum(p>0 for p in
              ↪  precip),
              "zero_hours": sum(p==0 for p in precip),
              "returned_grid": {k:obj[k] for k in ["latitude","longitude","elevation"]},
              "timezone": obj["timezone"], "utc_offset_seconds": obj["utc_offset_seconds"],
              "hourly_units": obj["hourly_units"]}
    return rows, checks


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--refresh", action="store_true", help="refetch existing raw years")
    args = parser.parse_args()
    BASE.mkdir(exist_ok=True, parents=True)
    rows, checks, requests = [], [], []
```

```
    for year in range(2021, 2027):
        params = dict(latitude=28.86, longitude=85.29, start_date=f"{year}-05-01",
                      end_date=f"{year}-10-16", hourly="precipitation,rain", models="era5",
                      cell_selection="nearest", elevation="nan", timezone="Asia/Shanghai",
                      precipitation_unit="mm", timeformat="iso8601")
        url = ENDPOINT + "?" + urlencode(params)
        raw = BASE / f"era5_hourly_{year}.json"
        receipt = BASE / f"request_{year}.json"
        if args.refresh or not raw.exists():
            body, metadata = fetch(url)
            raw.write_bytes(body)
            receipt.write_text(json.dumps({"url":url,"params":params,**metadata},
                                          ensure_ascii=False, indent=2), encoding="utf-8")
        request_record = json.loads(receipt.read_text(encoding="utf-8"))
        assert request_record["url"] == url
        year_rows, year_checks = parse_year(year, json.loads(raw.read_text(encoding="utf-8")))
        year_checks["raw_sha256"] = sha(raw)
        rows.extend(year_rows)
        checks.append(year_checks)
        requests.append(request_record)
        print(f"{year}: {len(year_rows)} hourly rows checked", flush=True)
    grids = {(c["returned_grid"]["latitude"], c["returned_grid"]["longitude"],
              c["returned_grid"]["elevation"]) for c in checks}
    assert len(grids) == 1, grids
    out = BASE / "era5_precipitation_hourly_2021_2026.csv"
    with out.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    report = {"status":"pass", "script_sha256":sha(Path(__file__)),
              "csv_sha256":sha(out), "rows":len(rows), "same_product_all_years":True,
              "same_returned_grid_all_years":True, "year_checks":checks,
              "source_qc_status":"not_provided", "requests":requests,
              "support":"Each API label t is preceding-hour sum, assigned [t-1h,t).",
              "role":"D1 independent grid proxy; never inserted into D0 station missing
              ↪  values."}
    (BASE / "DOWNLOAD_CHECKS.json").write_text(json.dumps(report,ensure_ascii=False,indent=2)」
    ↪  ,encoding="utf-8")
    doc = BASE / "open_meteo_historical_weather_api.html"
    if args.refresh or not doc.exists():
        body, metadata = fetch(DOC_URL)
        doc.write_bytes(body)
        (BASE / "documentation_request.json").write_text(
            json.dumps({"url":DOC_URL,
            ↪  **metadata},ensure_ascii=False,indent=2),encoding="utf-8")


if __name__ == "__main__":
    main()
```

prepare/netcdf_cleaning.py

```
from pathlib import Path
import csv
import gzip
```

```
import io
import json
import shutil

from netCDF4 import Dataset
import numpy as np
from pyproj import CRS


def _jsonable(value):
    if isinstance(value, dict):
        return {k: _jsonable(v) for k, v in value.items()}
    if isinstance(value, (tuple, list, np.ndarray)):
        return [_jsonable(v) for v in value]
    if isinstance(value, np.generic):
        return _jsonable(value.item())
    if isinstance(value, float) and not np.isfinite(value):
        return str(value)
    return value


def _decoded(variable):
    variable.set_auto_maskandscale(True)
    masked = np.ma.asarray(variable[:], dtype=np.float64)
    values = np.asarray(masked.filled(np.nan))
    source_missing = np.ma.getmaskarray(masked) | np.isnan(values)
    nonfinite = ~np.isfinite(values) & ~source_missing
    return values, source_missing, nonfinite


def _physical(name, units, values):

    failed = np.zeros(values.shape, dtype=bool)
    rule = "none; retain_without_inventing_bounds"
    if name == "slope" and units == "degree":
        failed = (values < 0) | (values > 90)
        rule = "terrain_slope_degrees_in_0_90"
    elif name == "thickness" and units == "m":
        failed = values < 0
        rule = "ice_thickness_metres_nonnegative"
    elif name in {"v", "v_amp", "vx_amp", "vy_amp"} and units == "meter/year":
        failed = values < 0
        rule = "speed_or_amplitude_nonnegative; signed_vx_vy_exempt"
    return failed & np.isfinite(values), rule


def clean_netcdf(input_root: Path, output_root: Path) -> dict:
    folders = [p for p in input_root.iterdir() if p.is_dir() and p.name.startswith((" (4) ",
    ↪  "(4)"))]
    if len(folders) != 1:
        raise ValueError("Expected one attachment 4 directory")
    files = sorted(p for p in folders[0].rglob("*.nc") if not p.name.startswith("._"))
    if not files:
        raise ValueError("No attachment 4 NetCDF files")
```

```
destination = output_root / "attachment04"
destination.mkdir(parents=True, exist_ok=False)
quality = output_root / "quality"
quality.mkdir(exist_ok=True)
reports = []
outputs = []
for source in files:
    target = destination / source.name
    table = destination / (source.stem + "_grid_cells.csv.gz")
    flags_file = destination / (source.stem + "_cell_quality.csv.gz")
    shutil.copy2(source, target)
    grids, statuses, variable_reports = {}, {}, []
    updates = {}
    with Dataset(source) as dataset:
        dimensions = {name: dict(size=len(dim), unlimited=dim.isunlimited())
                      for name, dim in dataset.dimensions.items()}
        global_attrs = {name: dataset.getncattr(name) for name in dataset.ncattrs()}
        if not {"x", "y", "spatial_ref"}.issubset(dataset.variables):
            raise ValueError(f"No explicit x/y/CRS in {source.name}; cannot create a
            ↪  spatial table")
        x, xm, xn = _decoded(dataset["x"])
        y, ym, yn = _decoded(dataset["y"])
        if xm.any() or ym.any() or xn.any() or yn.any():
            raise ValueError("Invalid coordinate labels; raw NetCDF is retained,
            ↪  coordinate alignment requires review")
        reference = dataset["spatial_ref"]
        wkt = getattr(reference, "crs_wkt", getattr(reference, "spatial_ref", None))
        crs = CRS.from_wkt(wkt) if wkt else None
        transform_text = getattr(reference, "GeoTransform", "")
        transform = [float(v) for v in transform_text.split()] if transform_text else None
        coordinate_check = "not_declared"
        if transform:
            if len(transform) != 6:
                raise ValueError("Invalid source GeoTransform")
            a, b, c, d, e, f = transform
            if c or e:
                raise ValueError("Rotated transform cannot be inferred from 1D
                ↪  coordinates")
            expected_x = a + (np.arange(len(x)) + 0.5) * b
            expected_y = d + (np.arange(len(y)) + 0.5) * f
            if not np.allclose(x, expected_x, rtol=0, atol=1e-7) or not np.allclose(y,
            ↪  expected_y, rtol=0, atol=1e-7):
                raise ValueError("Source coordinates conflict with source transform; no
                ↪  automatic shift")
            coordinate_check = "source_coordinates_equal_transform_pixel_centres"
        if len(np.unique(x)) != len(x) or len(np.unique(y)) != len(y):
            raise ValueError("Duplicate coordinates require an explicit reconciliation
            ↪  policy")
        for name, variable in dataset.variables.items():
            if variable.dtype.kind not in "iuf":
                raise ValueError(f"Unsupported variable type (not silently skipped):
                ↪  {name}")
            values, missing, nonfinite = _decoded(variable)
            units = getattr(variable, "units", "not_declared")
```

```
        failed, rule = _physical(name, units, values)
        status = np.ones(values.shape, dtype=np.uint8)
        status[missing] = 0
        status[nonfinite] = 2
        status[failed] = 3
        analysis = values.copy()
        analysis[status != 1] = np.nan
        if np.any(nonfinite | failed):
            updates[name] = nonfinite | failed
        if variable.dimensions == ("y", "x"):
            grids[name], statuses[name] = analysis, status
        valid = analysis[np.isfinite(analysis)]
        variable_reports.append(dict(name=name, dimensions=list(variable.dimensions),
        ↪  shape=list(values.shape),
            source_dtype=str(variable.dtype), units=units,
            attrs={k: variable.getncattr(k) for k in variable.ncattrs()},
            source_missing=int(missing.sum()),
            ↪  nonfinite_excluded=int(nonfinite.sum()),
            physical_excluded=int(failed.sum()),
            ↪  retained=int(np.isfinite(analysis).sum()),
            min=float(valid.min()) if valid.size else None, max=float(valid.max()) if
            ↪  valid.size else None,
            physical_rule=rule, source_qc="not_provided"))
    velocity_check = None
    if {"v", "vx", "vy"}.issubset(grids):
        common = np.isfinite(grids["v"]) & np.isfinite(grids["vx"]) &
        ↪  np.isfinite(grids["vy"])
        discrepancy = np.abs(grids["v"][common] - np.hypot(grids["vx"][common],
        ↪  grids["vy"][common]))
        velocity_check = dict(paired_cells=int(common.sum()),
            max_absolute_difference=float(discrepancy.max()) if discrepancy.size else
            ↪  None,
            action="observation_only; bilinear-source-processing-can-change-identity;
            ↪  no values altered")
if updates:
    with Dataset(target, "r+") as dataset:
        for name, exclusion in updates.items():
            variable = dataset[name]
            if "_FillValue" not in variable.ncattrs():
                raise ValueError(f"No declared fill value for excluding {name};
                ↪  cannot invent one")
            variable.set_auto_maskandscale(False)
            raw = np.asarray(variable[:]).copy()
            raw[exclusion] = variable.getncattr("_FillValue")
            variable[:] = raw

with Dataset(source) as original, Dataset(target) as cleaned:
    if original.ncattrs() != cleaned.ncattrs() or set(original.variables) !=
    ↪  set(cleaned.variables):
        raise RuntimeError("NetCDF metadata/schema changed unexpectedly")
    for key in original.ncattrs():
        if _jsonable(original.getncattr(key)) != _jsonable(cleaned.getncattr(key)):
            raise RuntimeError(f"Changed global attribute: {key}")
    for name, variable in original.variables.items():
```

```
        copy = cleaned[name]
        if variable.dimensions != copy.dimensions or variable.dtype != copy.dtype:
            raise RuntimeError(f"Changed variable schema: {name}")
        if _jsonable({k: variable.getncattr(k) for k in variable.ncattrs()}) !=
        ↪  _jsonable({k: copy.getncattr(k) for k in copy.ncattrs()}):
            raise RuntimeError(f"Changed variable attributes: {name}")
        variable.set_auto_maskandscale(False)
        copy.set_auto_maskandscale(False)
        raw, new = np.asarray(variable[:]), np.asarray(copy[:])
        keep = ~updates[name] if name in updates else np.ones(raw.shape, dtype=bool)
        if not np.array_equal(raw[keep], new[keep], equal_nan=True):
            raise RuntimeError(f"Changed retained raw values: {name}")
        if name in grids:
            decoded, missing, nonfinite = _decoded(copy)
            if not np.array_equal(np.where(np.isfinite(decoded), decoded, np.nan),
            ↪  grids[name], equal_nan=True):
                raise RuntimeError(f"NetCDF/table analysis value mismatch: {name}")
for filename, fields, flag_table in [(table, ["source_row", "source_column", "x",
↪  "y", *grids], False),
                                     (flags_file, ["source_row", "source_column",
                                     ↪  "x", "y", "variable", "status"], True)]:
    with filename.open("wb") as binary, gzip.GzipFile(filename="", fileobj=binary,
    ↪  mode="wb", mtime=0) as gz:
        with io.TextIOWrapper(gz, encoding="utf-8", newline="") as text:
            writer = csv.DictWriter(text, fieldnames=fields)
            writer.writeheader()
            for row in range(len(y)):
                for col in range(len(x)):
                    location = dict(source_row=row + 1, source_column=col + 1,
                    ↪  x=float(x[col]), y=float(y[row]))
                    if flag_table:
                        for name, status in statuses.items():
                            if status[row, col] != 1:
                                writer.writerow(dict(location, variable=name,
                                ↪  status={0: "source_missing", 2:
                                ↪  "excluded_nonfinite", 3:
                                ↪  "excluded_physical"}[int(status[row, col])]))
                    else:
                        writer.writerow(dict(location, **{name: float(values[row,
                        ↪  col]) if np.isfinite(values[row, col]) else None
                                                          for name, values in
                                                          ↪  grids.items()}))
report = dict(source_path=source.relative_to(input_root).as_posix(),
    output_path=target.relative_to(output_root).as_posix(),
    ↪  grid_table=table.relative_to(output_root).as_posix(),
    quality_table=flags_file.relative_to(output_root).as_posix(),
    ↪  dimensions=dimensions, global_attrs=global_attrs,
    crs=crs.to_string() if crs else None, crs_wkt=wkt, geotransform=transform,
    coordinate_check=coordinate_check, has_time_dimension="time" in dimensions,
    time_note="No time dimension: calendar/time decoding not applicable; velocities
    ↪  are source-described 2014–2024 climatology.",
    variables=variable_reports, grid_rows=len(x) * len(y),
    ↪  velocity_consistency_observation=velocity_check,
    vertical_datum="not_declared; do not assume compatibility with attachment07 DEM",
    ↪  decoded_once=True,
```

```
            nonfinite_or_physical_exclusions=sum(int(m.sum()) for m in updates.values()),
            no_resampling=True, no_imputation=True)
        reports.append(_jsonable(report))
        outputs.extend([p.relative_to(output_root).as_posix() for p in [target, table,
        ↪  flags_file]])
        print(f"NetCDF {source.name}: {len(x) * len(y):,} grid cells; coordinates/units/masks
        ↪  retained", flush=True)
    (quality / "netcdf_inventory.json").write_text(json.dumps(reports, ensure_ascii=False,
    ↪  indent=2, allow_nan=False) + "\n", encoding="utf-8")
    return dict(files=len(files), grid_rows=sum(r["grid_rows"] for r in reports),
    ↪  outputs=outputs,
                exclusions=sum(r["nonfinite_or_physical_exclusions"] for r in reports),
                inventory="quality/netcdf_inventory.json")
```

prepare/quality_diagnostics.py

```
import json
import math
from numbers import Integral

import pandas as pd


def _scalar(value):

    if value is None or pd.isna(value):
        return None
    if hasattr(value, "item"):
        value = value.item()
    return value


def _label(frame, position):
    value = _scalar(frame.at[position, "timestamp_label"])
    return str(value) if value is not None else None


def _row(frame, position):
    return _scalar(frame.at[position, "source_row"])


def _json(details):


    def safe(value):
        if isinstance(value, dict):
            return {str(key): safe(item) for key, item in value.items()}
        if isinstance(value, (tuple, list)):
            return [safe(item) for item in value]
        value = _scalar(value)
        if isinstance(value, float) and not math.isfinite(value):
            return str(value)
        return value
```

```
    return json.dumps(safe(details), ensure_ascii=False, separators=(",", ":"),
    ↪  allow_nan=False)


def diagnose(data: pd.DataFrame, measurement_columns: list[str], *,
             dataset: str, step_hours: int) -> list[dict]:


    if isinstance(step_hours, bool) or not isinstance(step_hours, Integral) or step_hours <=
    ↪  0:
        raise ValueError("step_hours must be a positive integer")
    if data.empty:
        return []
    columns = list(dict.fromkeys(measurement_columns))
    required = ["timestamp_label", "source_file", "source_sheet", "source_row", *columns]
    missing = [column for column in required if column not in data.columns]
    if missing:
        raise ValueError(f"Missing diagnostic input columns: {missing}")
    if not data.columns.is_unique:
        raise ValueError("Diagnostic input column names must be unique")

    output = []
    step = pd.Timedelta(hours=int(step_hours))
    kind = dataset.upper()
    for (source_file, source_sheet), source in data.groupby(
            ["source_file", "source_sheet"], sort=False, dropna=False):
        frame = source.reset_index(drop=True)


        times = pd.to_datetime(frame["timestamp_label"], format="ISO8601", errors="coerce",
        ↪  utc=True)
        numeric = frame[columns].apply(pd.to_numeric, errors="coerce")
        finite = numeric.notna() & numeric.abs().lt(float("inf"))
        sheet_detail = {"source_sheet": _scalar(source_sheet)}

        def emit(check, variable, count, start=None, end=None,
                 before=None, after=None, **details):
            output.append({
                "dataset": dataset,
                "source_file": _scalar(source_file),
                "check": check,
                "variable": variable,
                "count": int(count),
                "start": _label(frame, start) if start is not None else None,
                "end": _label(frame, end) if end is not None else None,
                "value_before": _scalar(before),
                "value_after": _scalar(after),
                "detail": _json({**sheet_detail, **details}),
            })

        for column in columns:
            usable = finite[column] & times.notna()
            observed = numeric.loc[usable, column]
            eligible = times.loc[usable].diff().eq(step)
```

```
    changes = observed.diff().loc[eligible]
    if changes.empty:
        emit("max_adjacent_change", column, 0,
             reason="no_adjacent_valid_pair_at_declared_step",
             ↪  step_hours=int(step_hours))
    else:
        end = int(changes.abs().idxmax())
        end_offset = observed.index.get_loc(end)
        start = int(observed.index[end_offset - 1])
        emit("max_adjacent_change", column, len(changes), start, end,
             float(observed.at[start]), float(observed.at[end]),
             source_rows=[_row(frame, start), _row(frame, end)],
             signed_change=float(changes.at[end]),
             ↪  absolute_change=float(abs(changes.at[end])),
             step_hours=int(step_hours))


    continuation = (usable & usable.shift(1, fill_value=False)
                    & times.diff().eq(step) &
                    ↪  numeric[column].eq(numeric[column].shift(1)))
    run_ids = (~continuation).cumsum()
    runs = numeric.loc[usable, column].groupby(run_ids.loc[usable], sort=False)
    sizes = runs.size()
    if sizes.empty:
        emit("longest_exact_constant_run", column, 0, reason="no_valid_timed_value")
    else:
        best_run = sizes.idxmax()
        positions = run_ids.index[usable & run_ids.eq(best_run)]
        start, end = int(positions[0]), int(positions[-1])
        value = float(numeric.at[start, column])
        emit("longest_exact_constant_run", column, int(sizes.at[best_run]),
             start, end, value, value,
             source_rows=[_row(frame, start), _row(frame, end)],
             equality="exact_float_equality", step_hours=int(step_hours))

def layer_spread(prefix, group_name):
    selected = [column for column in columns if column.lower().startswith(prefix)]
    if len(selected) < 2:
        return
    values = numeric[selected].where(finite[selected])
    simultaneous = values.notna().sum(axis=1).ge(2) & times.notna()
    spreads = (values.max(axis=1) - values.min(axis=1)).loc[simultaneous]
    if spreads.empty:
        emit("max_simultaneous_layer_spread", group_name, 0,
             reason="no_time_with_two_valid_layers", total_layers=len(selected))
        return
    position = int(spreads.idxmax())
    valid_layers = values.loc[position].dropna()
    low, high = valid_layers.idxmin(), valid_layers.idxmax()
    emit("max_simultaneous_layer_spread", group_name, len(spreads),
         position, position, float(valid_layers.at[low]),
         ↪  float(valid_layers.at[high]),
         source_row=_row(frame, position), spread=float(spreads.at[position]),
         low_layer=low, high_layer=high, valid_layers=len(valid_layers),
         ↪  total_layers=len(selected),
```

```
                layer_values={column: float(value) for column, value in
                ↪  valid_layers.items()},
                interpretation="observation_only_not_an_error_test")

        def range_observation(selected, name, predicate):
            if not selected:
                return
            mask = predicate(numeric[selected]) & finite[selected]
            cell_count = int(mask.sum().sum())
            details = {"count_unit": "observed_cells", "distinct_source_rows":
            ↪  int(mask.any(axis=1).sum()),
                       "counts_by_variable": {column: int(mask[column].sum()) for column in
                       ↪  selected},
                       "interpretation": "observation_only_no_value_removed"}
            if not cell_count:
                emit(name, ";".join(selected), 0, **details)
                return
            position = int(mask.any(axis=1).loc[lambda series: series].index[0])
            column = next(column for column in selected if mask.at[position, column])
            emit(name, ";".join(selected), cell_count, position, position,
                 float(numeric.at[position, column]), source_row=_row(frame, position),
                 representative_variable=column, representative="first_in_source_order",
                 ↪  **details)

        if kind == "MET":
            layer_spread("ta_", "Ta_layers")
            range_observation([c for c in columns if c.upper().startswith(("RH_", "RH("))],
                              "observed_RH_outside_0_100", lambda values: (values < 0) |
                              ↪  (values > 100))
            range_observation([c for c in columns if c.upper().startswith(("WS_", "WS("))],
                              "observed_negative_WS", lambda values: values < 0)
            range_observation([c for c in columns if c.upper().startswith(("WD_", "WD("))],
                              "observed_WD_outside_0_360", lambda values: (values < 0) |
                              ↪  (values > 360))
        elif kind == "SOIL":
            layer_spread("soilt_", "soilT_layers")
            layer_spread("vwc_", "VWC_layers")
        elif kind == "RADM":
            radiation = [c for c in columns if c.split("(", 1)[0].lower() in {"rsd", "rsu",
            ↪  "rld", "rlu"}]
            range_observation(radiation, "observed_negative_radiation", lambda values: values
            ↪  < 0)
    return output
```

prepare/quality_policy.py

```
from dataclasses import dataclass
from datetime import datetime, timezone
from math import isfinite, isnan
from numbers import Real


@dataclass(frozen=True)
class PhysicalBounds:
    lower: float = float("-inf")
```

```
    upper: float = float("inf")
    lower_inclusive: bool = True
    upper_inclusive: bool = True

    def __post_init__(self):
        if isnan(self.lower) or isnan(self.upper) or self.lower > self.upper:
            raise ValueError("Invalid physical bounds")
        if self.lower == self.upper and not (self.lower_inclusive and self.upper_inclusive):
            raise ValueError("Empty physical domain")

    def accepts(self, value):
        return ((value >= self.lower if self.lower_inclusive else value > self.lower)
                and (value <= self.upper if self.upper_inclusive else value < self.upper))


@dataclass(frozen=True)
class QualityPolicy:
    no_source_qc: str = "reject"
    retain_flagged_codes: tuple = ()
    sentinel_tokens: tuple = (9999.9,)

    def __post_init__(self):

        object.__setattr__(self, "retain_flagged_codes", tuple(self.retain_flagged_codes))
        object.__setattr__(self, "sentinel_tokens", tuple(self.sentinel_tokens))
        if self.no_source_qc not in {"reject", "retain_flagged"}:
            raise ValueError("Unknown no-source-QC policy")
        if any(type(c) is not int or c not in {1, 4, 9} for c in self.retain_flagged_codes):
            raise ValueError("Only QC1/4/9 may be explicitly retained")
        if any(isinstance(x, bool) or not isinstance(x, Real) or not isfinite(x)
               for x in self.sentinel_tokens):
            raise ValueError("Sentinel tokens must be explicit finite numbers")


def _aware(value):
    return isinstance(value, datetime) and value.tzinfo is not None and value.utcoffset() is
    ↪  not None


def assess_cell(raw_value, *, source_qc_presence, source_qc=None,
                physical_bounds=None, policy=QualityPolicy(),
                interval_start=None, interval_end=None, aggregation=None,
                available_at=None, mode="retrospective", as_of=None,
                empirical_flags=()):


    if source_qc_presence not in {"not_provided", "provided"}:
        raise ValueError("source_qc_presence must be explicit")
    if source_qc_presence == "not_provided" and source_qc is not None:
        raise ValueError("Do not fabricate a code for a column without source QC")
    if mode not in {"retrospective", "causal"}:
        raise ValueError("Unknown mode")
    if mode == "causal" and not _aware(as_of):
        raise ValueError("Causal mode requires timezone-aware as_of")
```

```
if mode == "retrospective" and as_of is not None:
    raise ValueError("as_of applies only in causal mode")
if isinstance(empirical_flags, str):
    raise ValueError("empirical_flags must be a collection of nonempty strings")
flags = tuple(empirical_flags)
if any(not isinstance(x, str) or not x for x in flags):
    raise ValueError("empirical_flags must be a collection of nonempty strings")
reasons = []
numeric = None
if isinstance(raw_value, bool):
    reasons.append("nonnumeric")
else:
    try:
        numeric = float(raw_value)
    except (TypeError, ValueError, OverflowError):
        reasons.append("nonnumeric")
structural = numeric is not None and isfinite(numeric)
if numeric is not None and not isfinite(numeric):
    reasons.append("nonfinite")
if structural and numeric in policy.sentinel_tokens:
    structural = False
    reasons.append("declared_sentinel")

qc_known = (isinstance(source_qc, Real) and not isinstance(source_qc, bool)
            and isfinite(source_qc) and source_qc in {0, 1, 2, 4, 8, 9})
if source_qc_presence == "not_provided":
    source_status = "source_quality_unverified"
    source_allowed = policy.no_source_qc == "retain_flagged"
elif source_qc is None:
    source_status, source_allowed = "provided_code_missing", False
elif not qc_known:
    source_status, source_allowed = "provided_code_unknown", False
else:
    source_status = {0: "source_flag_0_correct", 1: "source_flag_1_suspicious",
                     2: "source_flag_2_wrong", 4: "source_flag_4_imputed",
                     8: "source_flag_8_missing", 9:
                     ↪  "source_flag_9_uncontrolled"}[source_qc]
    source_allowed = (source_qc not in {2, 8}
                      and (source_qc == 0 or source_qc in policy.retain_flagged_codes))
if not source_allowed:
    reasons.append(source_status)

if physical_bounds is not None and not isinstance(physical_bounds, PhysicalBounds):
    raise TypeError("physical_bounds must be an explicit PhysicalBounds")
physical = ("not_evaluated" if not structural or physical_bounds is None else
            "pass" if physical_bounds.accepts(numeric) else "fail")
if physical != "pass":
    reasons.append("physical_" + physical)

aware_support = _aware(interval_start) and _aware(interval_end)

start_utc = interval_start.astimezone(timezone.utc) if aware_support else None
end_utc = interval_end.astimezone(timezone.utc) if aware_support else None
time_known = bool(aware_support and (
```

```
        (aggregation == "instantaneous" and start_utc == end_utc)
        or (aggregation in {"mean", "sum"} and start_utc < end_utc)))
    time_status = "declared_support_known" if time_known else "unknown_or_invalid_support"
    if not time_known:
        reasons.append(time_status)
    release_known = _aware(available_at)
    release_utc = available_at.astimezone(timezone.utc) if release_known else None
    cutoff_utc = as_of.astimezone(timezone.utc) if mode == "causal" else None
    release_consistent = release_known and (not time_known or release_utc >= end_utc)
    availability_status = ("unknown" if not release_known else
                           "invalid_before_support_end" if not release_consistent else
                           ↪  "known")
    causal_available = bool(time_known and release_consistent and mode == "causal"
                            and end_utc <= cutoff_utc and release_utc <= cutoff_utc)
    if mode == "causal" and not causal_available:
        reasons.append("causal_availability_not_established")
    if release_known and not release_consistent:
        reasons.append(availability_status)
    candidate = structural and physical == "pass" and source_allowed
    usable = bool(candidate and time_known and (not release_known or release_consistent)
                  and (mode == "retrospective" or causal_available))
    return dict(raw_value=raw_value, source_qc=source_qc,
                source_qc_presence=source_qc_presence, source_quality_status=source_status,
                source_policy_eligible=bool(source_allowed),
                structural_status="pass" if structural else "fail", physical_status=physical,
                time_support_status=time_status, interval_start=interval_start,
                ↪  interval_end=interval_end,
                aggregation=aggregation, availability_status=availability_status,
                available_at=available_at, causal_available=causal_available, mode=mode,
                research_candidate=bool(candidate), research_usable=usable,
                research_value=numeric if usable else None, empirical_flags=flags,
                empirical_status="suspect" if flags else "not_flagged",
                source_imputed=bool(qc_known and source_qc == 4), imputed_here=False,
                clipped_here=False, reasons=tuple(reasons))
```

prepare/raster_cleaning.py

```
from __future__ import annotations

import csv
import hashlib
import json
import math
import re
import xml.etree.ElementTree as ET
from collections import Counter
from pathlib import Path
from typing import Any, Iterator

import numpy as np
import rasterio
from rasterio.shutil import copy as copy_raster
from rasterio.windows import Window
```

```
ATTACHMENTS = (3, 5, 7)
WINDOW_SIZE = 512


LAYOUT_TAG_DOMAINS = {"IMAGE_STRUCTURE", "DERIVED_SUBDATASETS"}


def _json_value(value: Any) -> Any:
    if isinstance(value, (np.integer, np.floating)):
        value = value.item()
    if isinstance(value, float) and not math.isfinite(value):
        return "NaN" if math.isnan(value) else ("Infinity" if value > 0 else "-Infinity")
    if isinstance(value, dict):
        return {str(k): _json_value(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
        return [_json_value(v) for v in value]
    return value


def _dump_json(path: Path, data: Any) -> None:
    with path.open("x", encoding="utf-8") as handle:
        json.dump(_json_value(data), handle, ensure_ascii=False, indent=2, allow_nan=False)
        handle.write("\n")


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def _windows(width: int, height: int) -> Iterator[Window]:

    for row in range(0, height, WINDOW_SIZE):
        for col in range(0, width, WINDOW_SIZE):
            yield Window(col, row, min(WINDOW_SIZE, width - col), min(WINDOW_SIZE, height -
            ↪  row))


def _tags(src: Any, band: int = 0) -> dict[str, dict[str, str]]:
    tags = {"default": src.tags(band)}
    for namespace in src.tag_namespaces(band):
        if namespace not in LAYOUT_TAG_DOMAINS:
            tags[namespace] = src.tags(band, ns=namespace)
    return tags


def _metadata(src: Any) -> dict[str, Any]:
    crs_wkt = src.crs.to_wkt() if src.crs else None
    return {
        "driver": src.driver,
        "width": src.width,
```

```
        "height": src.height,
        "count": src.count,
        "dtypes": list(src.dtypes),
        "crs": str(src.crs) if src.crs else None,
        "crs_wkt": crs_wkt,
        "epsg": src.crs.to_epsg() if src.crs else None,
        "transform": list(src.transform),
        "resolution": list(src.res),
        "bounds": list(src.bounds),
        "nodata": _json_value(src.nodatavals),
        "scales": list(src.scales),
        "offsets": list(src.offsets),
        "units": list(src.units),
        "descriptions": list(src.descriptions),
        "color_interpretation": [item.name for item in src.colorinterp],
        "mask_flags": [[item.name for item in band] for band in src.mask_flag_enums],
        "tags": _tags(src),
        "band_tags": [_tags(src, band) for band in src.indexes],
    }


def _grid_signature(metadata: dict[str, Any]) -> str:
    grid = {key: metadata[key] for key in ("crs_wkt", "transform", "width", "height")}
    grid["pixel_registration"] = metadata["tags"]["default"].get("AREA_OR_POINT",
    ↪  "unspecified")
    return hashlib.sha256(json.dumps(grid, sort_keys=True,
    ↪  ensure_ascii=False).encode("utf-8")).hexdigest()


def _copy_metadata(src: Any, dst: Any) -> None:
    dst.scales = src.scales
    dst.offsets = src.offsets
    dst.units = src.units
    dst.descriptions = src.descriptions
    dst.colorinterp = src.colorinterp
    for band in (0, *src.indexes):
        for namespace, tags in _tags(src, band).items():
            if tags:
                dst.update_tags(band, ns=None if namespace == "default" else namespace,
                ↪  **tags)


def _flag(code: str, detail: str, severity: str = "pending") -> dict[str, str]:
    return {"code": code, "severity": severity, "detail": detail}


def _metadata_flags(metadata: dict[str, Any], number: int) -> list[dict[str, str]]:
    flags = []
    if not metadata["crs"]:
        flags.append(_flag("crs_unconfirmed", "Source has no declared CRS; coordinates were
        ↪  not relabelled."))
    if any(value is None for value in metadata["nodata"]):
        flags.append(_flag("nodata_not_declared", "No new sentinel was inferred; source masks
        ↪  and nonfinite values determine validity."))
```

```
    if number == 3 and any(not unit for unit in metadata["units"]):
        flags.append(_flag("dhdt_units_unconfirmed", "Provider document identifies
        ↪  elevation-change rate/error but does not state their numeric units; source scale
        ↪  and offset are retained."))
    if number == 5:
        flags.append(_flag("land_cover_class_dictionary_unconfirmed", "Observed integer codes
        ↪  are retained without assigning class names; no provider class dictionary was
        ↪  found in this attachment."))
    if number == 7:
        if any(not unit for unit in metadata["units"]):
            flags.append(_flag("elevation_units_unconfirmed", "Elevation band units are
            ↪  missing; no conversion was applied."))

        flags.append(_flag("vertical_datum_unconfirmed", "A confirmed vertical datum is not
        ↪  supplied by the raster CRS; consult provider documentation before using absolute
        ↪  elevations across sources."))
    transform = metadata["transform"]
    if transform[1] != 0 or transform[3] != 0:
        flags.append(_flag("rotated_grid_preserved", "Source affine rotation/shear is
        ↪  retained.", "info"))
    return flags


def _source_document_notes(folder: Path, number: int) -> list[dict[str, str]]:

    notes = []
    if number == 3:
        for path in sorted(folder.rglob("*.docx")):


            notes.append({"path": path.relative_to(folder).as_posix(), "sha256":
            ↪  _sha256(path), "role": "provider_document; rate/error numeric unit remains
            ↪  unconfirmed"})
    if number == 7:
        for path in sorted(folder.rglob("*.tif.xml")):
            note = {"path": path.relative_to(folder).as_posix(), "sha256": _sha256(path),
            ↪  "role": "provider_spatial_metadata"}
            try:
                text = " ".join(ET.parse(path).getroot().itertext())
                if re.search(r"VDATUM\s*\[\s*['\"]unknown['\"]", text, flags=re.I):
                    note["vertical_datum_evidence"] = "provider XML explicitly declares
                    ↪  VDATUM unknown"
            except (ET.ParseError, UnicodeError, OSError) as exc:
                note["read_status"] = f"pending: {type(exc).__name__}"
            notes.append(note)
    return notes


def _new_band_stats(band: int, src: Any) -> dict[str, Any]:
    return {
        "band": band, "dtype": src.dtypes[band - 1],
        "scale": src.scales[band - 1], "offset": src.offsets[band - 1],
        "unit": src.units[band - 1], "sample_count": 0,
        "source_masked_count": 0, "finite_source_masked_count": 0,
```

```
        "nonfinite_count": 0, "nonfinite_source_unmasked_count": 0,
        "analysis_valid_count": 0, "analysis_invalid_count": 0,
        "valid_raw_min": None, "valid_raw_max": None,
        "valid_scaled_min": None, "valid_scaled_max": None,
        "valid_negative_count": 0,
    }


def _update_stats(stats: dict[str, Any], values: np.ndarray, source_mask: np.ndarray,
                  analysis_valid: np.ndarray, class_counts: Counter | None) -> None:
    finite = np.isfinite(values)
    source_masked = source_mask == 0
    valid_values = values[analysis_valid]
    stats["sample_count"] += int(values.size)
    stats["source_masked_count"] += int(np.count_nonzero(source_masked))
    stats["finite_source_masked_count"] += int(np.count_nonzero(source_masked & finite))
    stats["nonfinite_count"] += int(np.count_nonzero(~finite))
    stats["nonfinite_source_unmasked_count"] += int(np.count_nonzero(~source_masked &
    ↪  ~finite))
    count = int(valid_values.size)
    stats["analysis_valid_count"] += count
    stats["analysis_invalid_count"] += int(values.size) - count
    if count:
        low, high = valid_values.min().item(), valid_values.max().item()
        stats["valid_raw_min"] = low if stats["valid_raw_min"] is None else min(low,
        ↪  stats["valid_raw_min"])
        stats["valid_raw_max"] = high if stats["valid_raw_max"] is None else max(high,
        ↪  stats["valid_raw_max"])
        stats["valid_negative_count"] += int(np.count_nonzero(valid_values < 0))
        if class_counts is not None:
            unique, counts = np.unique(valid_values, return_counts=True)
            class_counts.update({str(value.item()): int(n) for value, n in zip(unique,
            ↪  counts)})


def _finalize_stats(stats: dict[str, Any], class_counts: Counter | None) -> None:
    if stats["valid_raw_min"] is not None:
        limits = [float(stats[key]) * stats["scale"] + stats["offset"] for key in
        ↪  ("valid_raw_min", "valid_raw_max")]
        stats["valid_scaled_min"], stats["valid_scaled_max"] = min(limits), max(limits)
    if class_counts is not None:
        stats["observed_valid_code_counts"] = dict(sorted(class_counts.items(), key=lambda
        ↪  item: float(item[0])))


def _prepare_one(source: Path, folder: Path, number: int, input_root: Path,
                 output_root: Path) -> dict[str, Any]:
    relative = source.relative_to(folder)
    destination = output_root / f"attachment{number:02d}" / "rasters" / relative
    validity_path = destination.with_name(destination.stem + ".analysis_valid_mask.tif")
    destination.parent.mkdir(parents=True, exist_ok=True)
    source_hash = _sha256(source)
    with rasterio.open(source) as src:
        source_metadata = _metadata(src)
```

```
source_dependencies = []
for filename in src.files:
    dependency = Path(filename).resolve()
    if dependency != source and dependency.is_file():
        source_dependencies.append({
            "path": dependency.relative_to(input_root).as_posix(),
            "sha256": _sha256(dependency),
            "role": "source GDAL companion; archived separately by caller",
        })
flags = _metadata_flags(source_metadata, number)


with rasterio.Env(GDAL_TIFF_INTERNAL_MASK=True):
    copy_raster(src, destination, driver="GTiff", strict=True,
                compress="DEFLATE", tiled=True, blockxsize=256, blockysize=256)
with rasterio.open(destination, "r+") as dst:
    _copy_metadata(src, dst)
mask_profile = {
    "driver": "GTiff", "width": src.width, "height": src.height,
    "count": src.count, "dtype": "uint8", "crs": src.crs,
    "transform": src.transform, "nodata": 0,
    "compress": "DEFLATE", "tiled": True, "blockxsize": 256, "blockysize": 256,
}
band_stats = [_new_band_stats(band, src) for band in src.indexes]
class_counts = [Counter() if number == 5 and np.issubdtype(np.dtype(src.dtypes[band -
↪  1]), np.integer) else None for band in src.indexes]
checked_windows = 0
with rasterio.open(destination) as dst, rasterio.open(validity_path, "w",
↪  **mask_profile) as mask:
    copied_metadata = _metadata(dst)


    compare_keys = [key for key in source_metadata if key != "mask_flags"]
    mismatch = [key for key in compare_keys if source_metadata[key] !=
    ↪  copied_metadata[key]]
    if mismatch:
        raise ValueError(f"Raster metadata readback differs ({mismatch}): {source}")
    mask.update_tags(
        mask_meaning="0=invalid/missing; 1=analysis-valid",
        validity_rule="source_band_mask_nonzero AND isfinite(source_raw_value)",
        source_raster=destination.relative_to(output_root).as_posix(),
        source_sha256=source_hash,
        numeric_policy="Raw source tokens, nodata, scales and offsets are preserved
        ↪  in source_raster; no imputation or inferred sentinel.",
    )
    mask.descriptions = tuple(f"analysis_valid_band_{band}" for band in src.indexes)
    for band in src.indexes:
        for window in _windows(src.width, src.height):
            values = src.read(band, window=window)
            readback = dst.read(band, window=window)
            source_mask = src.read_masks(band, window=window)
            readback_mask = dst.read_masks(band, window=window)
            if not np.array_equal(values, readback, equal_nan=True):
                raise ValueError(f"Raster values differ on readback: {source}, band
                ↪  {band}, {window}")
```

```
                if not np.array_equal(source_mask, readback_mask):
                    raise ValueError(f"Source mask differs on readback: {source}, band
                    ↪  {band}, {window}")
                valid = (source_mask != 0) & np.isfinite(values)
                mask.write(valid.astype("uint8"), band, window=window)
                _update_stats(band_stats[band - 1], values, source_mask, valid,
                ↪  class_counts[band - 1])
                checked_windows += 1


    with rasterio.open(validity_path) as mask:
        if mask.crs != src.crs or mask.transform != src.transform or mask.shape !=
        ↪  src.shape or mask.count != src.count:
            raise ValueError(f"Analysis mask grid differs: {validity_path}")
        for band in src.indexes:
            for window in _windows(src.width, src.height):
                values = src.read(band, window=window)
                expected = ((src.read_masks(band, window=window) != 0) &
                ↪  np.isfinite(values)).astype("uint8")
                if not np.array_equal(mask.read(band, window=window), expected):
                    raise ValueError(f"Analysis mask readback differs: {validity_path},
                    ↪  band {band}, {window}")
    for stats, counts in zip(band_stats, class_counts):
        _finalize_stats(stats, counts)
        if stats["nonfinite_source_unmasked_count"]:
            flags.append(_flag("nonfinite_unmasked_samples", f"Band {stats['band']}:
            ↪  {stats['nonfinite_source_unmasked_count']} source-unmasked NaN/Inf
            ↪  samples are excluded by analysis mask only.", "warning"))
        if not stats["analysis_valid_count"]:
            flags.append(_flag("no_analysis_valid_samples", f"Band {stats['band']} has no
            ↪  finite unmasked samples.", "warning"))
        if "dhdt_err_UTM" in source.parts and stats["valid_negative_count"]:
            flags.append(_flag("negative_error_values_retained", f"Band {stats['band']}:
            ↪  {stats['valid_negative_count']} finite unmasked negative error values
            ↪  require provider review; no values were removed.", "warning"))
if _sha256(source) != source_hash:
    raise RuntimeError(f"Source changed during raster preparation: {source}")
return {
    "attachment": f"attachment{number:02d}",
    "source": source.relative_to(input_root).as_posix(),
    "source_sha256": source_hash,
    "source_companion_files": source_dependencies,
    "output": destination.relative_to(output_root).as_posix(),
    "output_sha256": _sha256(destination),
    "analysis_valid_mask": validity_path.relative_to(output_root).as_posix(),
    "analysis_valid_mask_sha256": _sha256(validity_path),
    "metadata": source_metadata,
    "output_mask_flags": copied_metadata["mask_flags"],
    "grid_signature": _grid_signature(source_metadata),
    "band_statistics": band_stats,
    "flags": flags,
    "verification": {
        "metadata": "pass", "all_values": "pass", "source_masks": "pass",
        "analysis_mask_values": "pass", "analysis_mask_grid": "pass",
```

```
            "band_windows_checked": checked_windows, "maximum_window_shape": [WINDOW_SIZE,
            ↪  WINDOW_SIZE],
            "scope": "Exact source/output equality and mask rule; this is not independent
            ↪  scientific validation.",
        },
    }


def _attachment_folder(input_root: Path, number: int) -> Path:
    pattern = re.compile(rf"^[ ((]0?{number}[) )]")
    found = [path for path in input_root.iterdir() if path.is_dir() and
    ↪  pattern.match(path.name)]
    if len(found) != 1:
        raise ValueError(f"Expected exactly one attachment {number} directory under
        ↪  {input_root}; found {len(found)}")
    return found[0]


def clean_rasters(input_root: Path, output_root: Path) -> dict[str, Any]:


    input_root, output_root = Path(input_root).resolve(), Path(output_root).resolve()
    if not input_root.is_dir() or not output_root.is_dir():
        raise ValueError("Input and output roots must already exist as directories")
    if input_root == output_root or input_root in output_root.parents:
        raise ValueError("Output root must be outside the source attachment tree")
    folders = {number: _attachment_folder(input_root, number) for number in ATTACHMENTS}
    sources = {number: sorted(path for path in folder.rglob("*") if path.is_file() and
    ↪  path.suffix.lower() in {".tif", ".tiff"}) for number, folder in folders.items()}
    inventory_json = output_root / "quality" / "raster_inventory.json"
    inventory_csv = output_root / "quality" / "raster_inventory.csv"
    targets = [inventory_json, inventory_csv]
    for number, paths in sources.items():
        if not paths:
            raise ValueError(f"No source TIFFs found in attachment {number}:
            ↪  {folders[number]}")
        for source in paths:
            if source.is_symlink():
                raise ValueError(f"Symlink source is not accepted: {source}")
            destination = output_root / f"attachment{number:02d}" / "rasters" /
            ↪  source.relative_to(folders[number])
            targets.extend([destination, destination.with_name(destination.stem +
            ↪  ".analysis_valid_mask.tif")])
    if len(set(targets)) != len(targets):
        raise ValueError("Raster output paths collide with one another")
    for target in targets:
        if target.exists():
            raise FileExistsError(f"Refusing to overwrite raster output: {target}")


        if not target.resolve().is_relative_to(output_root):
            raise ValueError(f"Raster output escapes output root: {target}")
    inventory_json.parent.mkdir(parents=True, exist_ok=True)
    records = []
```

```
document_notes = {}
for number in ATTACHMENTS:
    document_notes[f"attachment{number:02d}"] = _source_document_notes(folders[number],
    ↪  number)
    for source in sources[number]:
        records.append(_prepare_one(source, folders[number], number, input_root,
        ↪  output_root))
grid_groups: dict[str, list[str]] = {}
for record in records:
    grid_groups.setdefault(record["grid_signature"], []).append(record["output"])
inventory = {
    "schema_version": "1.0", "scope": "basic_lossless_raster_preparation",
    "policy": {
        "raw_values": "preserved per band, including finite source NoData tokens and
        ↪  nonfinite values",
        "source_masks": "preserved and fully compared after writing",
        "analysis_valid_mask": "uint8 per band: 1 iff source mask is nonzero and source
        ↪  raw numeric value is finite; 0 means missing/invalid",
        "spatial_operations": "none: no reprojection, resampling, mosaic, crop or forced
        ↪  alignment",
        "numeric_operations": "none: no imputation, clipping, inferred sentinel, unit
        ↪  conversion or class remapping",
        "statistics": "computed only on finite source-unmasked values; raw and
        ↪  scale/offset-derived ranges are separate",
        "tags": "source default/band and substantive namespace tags preserved; layout and
        ↪  generated DERIVED_SUBDATASETS domains describe new files and are not copied",
        "grid_signatures": "exact CRS WKT, affine, dimensions and pixel registration;
        ↪  different signatures must not be multiplied by array index without a separate
        ↪  alignment decision; tiny floating differences are retained",
        "metadata_pending": "missing units, class definitions and vertical datum remain
        ↪  pending locally; numerical preservation may pass independently",
    },
    "counts": {f"attachment{number:02d}": len(sources[number]) for number in ATTACHMENTS},
    "provider_document_notes": document_notes,
    "grid_groups": grid_groups,
    "records": records,
}
_dump_json(inventory_json, inventory)
with inventory_csv.open("x", encoding="utf-8-sig", newline="") as handle:
    columns = ["attachment", "source", "output", "analysis_valid_mask", "grid_signature",
    ↪  "crs", "width", "height", "count", "dtype", "resolution", "bounds", "affine",
    ↪  "nodata", "units", "analysis_valid_count", "source_masked_count",
    ↪  "nonfinite_count", "flags", "readback_verification"]
    writer = csv.DictWriter(handle, fieldnames=columns)
    writer.writeheader()
    for record in records:
        metadata = record["metadata"]
        row = {key: record[key] for key in ("attachment", "source", "output",
        ↪  "analysis_valid_mask", "grid_signature")}
        row.update({key: metadata[key] for key in ("crs", "width", "height", "count",
        ↪  "resolution", "bounds", "nodata", "units")})
        row.update(dtype=metadata["dtypes"], affine=metadata["transform"],
        ↪  flags=record["flags"], readback_verification="pass")
        for key in ("analysis_valid_count", "source_masked_count", "nonfinite_count"):
```

```
                row[key] = [band[key] for band in record["band_statistics"]]
            writer.writerow({key: json.dumps(_json_value(value), ensure_ascii=False,
            ↪  allow_nan=False) if isinstance(value, (dict, list)) else value for key, value
            ↪  in row.items()})
    outputs = [record[key] for record in records for key in ("output", "analysis_valid_mask")]
    outputs.extend([inventory_json.relative_to(output_root).as_posix(),
    ↪  inventory_csv.relative_to(output_root).as_posix()])
    summary = {
        "status": "completed_with_metadata_pending" if any(flag["severity"] == "pending" for
        ↪  record in records for flag in record["flags"]) else "completed",
        "raster_count": len(records), "analysis_mask_count": len(records),
        "counts_by_attachment": inventory["counts"], "grid_signature_count": len(grid_groups),
        "readback_verification": "pass", "independent_scientific_validation": "not_performed",
        "pending_metadata_flag_count": sum(flag["severity"] == "pending" for record in
        ↪  records for flag in record["flags"]),
        "warning_flag_count": sum(flag["severity"] == "warning" for record in records for
        ↪  flag in record["flags"]),
        "inventory_json": inventory_json.relative_to(output_root).as_posix(),
        "inventory_csv": inventory_csv.relative_to(output_root).as_posix(),
        "outputs": outputs,
    }

    summary.update(
        primary_files=len(records),
        band_cells=sum(band["sample_count"] for record in records for band in
        ↪  record["band_statistics"]),
        analysis_valid_cells=sum(band["analysis_valid_count"] for record in records for band
        ↪  in record["band_statistics"]),
        output_paths=outputs,
        inventory_path=inventory_json.relative_to(output_root).as_posix(),
    )
    return summary
```

prepare/route.py

```
import argparse
from pathlib import Path

import numpy as np
import pandas as pd


def convert(src, out):
    with np.load(src, allow_pickle=False) as d:
        path = pd.DataFrame({
            'latitude': d['lat'],
            'longitude': d['lon'],
            'elevation_m': d['elev'],
            'author_chainage_m': d['dist_m'],
            'segment_azimuth_deg': d['seg_azimuth_deg'],
        })
        index = int(d['i_confluence'])
        assert np.all(np.diff(path.author_chainage_m) >= 0)
        path['author_confluence_node'] = False
        path.loc[index, 'author_confluence_node'] = True
```

```
    path.to_csv(out, index=False)


def sol():
    p = argparse.ArgumentParser()
    p.add_argument('--input', type=Path, default=Path(__file__).resolve().parent /
    ↪  'route_data/path.npz')
    p.add_argument('--output', type=Path, default=Path('route_new.csv'))
    a = p.parse_args()
    if a.output.exists():
        raise FileExistsError(a.output)
    a.output.parent.mkdir(parents=True, exist_ok=True)
    convert(a.input, a.output)


if __name__ == '__main__':
    sol()
```

prepare/vector_cleaning.py

```
from __future__ import annotations

import codecs
import csv
import gzip
import hashlib
import json
import re
import warnings
import xml.etree.ElementTree as ET
from pathlib import Path
from typing import Any

import geopandas as gpd
import numpy as np
import pandas as pd
import pyogrio
import shapely


_ATTACHMENT_RE = re.compile(r"^[ ((]\s*([368])\s*[) )]")
_ATTACHMENTS = (3, 6, 8)
_OUTPUT_NAMES = {
    "JilongFlood_Buffer_200km": "flood_buffer_200km",
    "RGI_V6_JilongFlood_Buffer_200km": "rgi_v6_glaciers",
    "RGI_V7_JilongFlood_Buffer_200km": "rgi_v7_glaciers",
    " 西藏日喀则市吉隆县泥石流灾害区历史灾点数据": "historical_hazards",
    " 西藏日喀则市吉隆县第七次人口普查数据": "census_population",
}
_ID_COLUMNS = {
    "rgiid", "rgi_id", "glimsid", "glims_id", "code", "orig_fid",
    " 区划码", " 县级码",
}
_RESERVED = {
    "source_row", "source_fid", "source_file", "qc_source_presence",
```

```
    "qc_source_encoding", "qc_geometry_status",
}


def _sha(path: Path) -> str:
    result = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            result.update(chunk)
    return result.hexdigest()


def _relative(path: Path, root: Path) -> str:
    return path.relative_to(root).as_posix()


def _write_json(path: Path, value: Any) -> None:
    path.write_text(
        json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
        encoding="utf-8",
    )


def _write_csv(path: Path, frame: pd.DataFrame) -> None:


    frame.to_csv(
        path, index=False, encoding="utf-8", na_rep="", lineterminator="\n",
        compression={"method": "gzip", "mtime": 0},
        date_format="%Y-%m-%dT%H:%M:%S",
    )


def _dbf_layout(path: Path) -> dict[str, Any]:
    data = path.read_bytes()
    if len(data) < 33:
        raise ValueError("DBF header is truncated")
    count = int.from_bytes(data[4:8], "little")
    header_size = int.from_bytes(data[8:10], "little")
    record_size = int.from_bytes(data[10:12], "little")
    if header_size < 33 or record_size < 1:
        raise ValueError("DBF header/record length is invalid")
    if len(data) < header_size + count * record_size:
        raise ValueError("DBF declared records exceed available source bytes")
    fields: list[dict[str, Any]] = []
    offset = 1
    position = 32
    while position < header_size and data[position] != 0x0D:
        descriptor = data[position:position + 32]
        if len(descriptor) != 32:
            raise ValueError("DBF field descriptor is truncated")
        field = {
            "name_bytes": descriptor[:11].split(b"\0", 1)[0],
            "type": chr(descriptor[11]), "width": int(descriptor[16]),
```

```
            "decimals": int(descriptor[17]), "record_offset": offset,
        }
        fields.append(field)
        offset += field["width"]
        position += 32
    if position >= header_size or data[position] != 0x0D or offset != record_size:
        raise ValueError("DBF field layout does not match declared record length")
    flags = [data[header_size + row * record_size] for row in range(count)]
    if any(flag not in (0x20, 0x2A) for flag in flags):
        raise ValueError("Unrecognised DBF deletion flag; refusing silent row loss")
    return {
        "bytes": data, "version": data[0], "record_count": count,
        "header_size": header_size, "record_size": record_size,
        "language_driver_id": data[29], "fields": fields, "flags": flags,
    }


def _dbf_text_cells(dbf: dict[str, Any]):
    for field in dbf["fields"]:
        yield None, field["name_bytes"]
    for row in range(dbf["record_count"]):
        start = dbf["header_size"] + row * dbf["record_size"]
        for field in dbf["fields"]:
            if field["type"] == "C":
                offset = start + field["record_offset"]
                yield row, dbf["bytes"][offset:offset + field["width"]].rstrip(b" \0")


def _decode_probe(dbf: dict[str, Any], encoding: str) -> dict[str, Any]:
    failures = names = values = 0
    for row, raw in _dbf_text_cells(dbf):
        if row is None:
            names += 1
        else:
            values += 1
        try:
            raw.decode(encoding, errors="strict")
        except UnicodeDecodeError:
            failures += 1
    return {
        "encoding": encoding, "strict_decode_errors": failures,
        "field_names_checked": names, "character_values_checked": values,
        "all_text_bytes_decodable": failures == 0,
    }


def _normalise_encoding(label: str) -> str:
    aliases = {"65001": "utf-8", "936": "gbk", "cp936": "gbk"}
    return codecs.lookup(aliases.get(label.strip().lower(), label.strip())).name


def _encoding_decision(path: Path, dbf: dict[str, Any]) -> dict[str, Any]:
    cpg = path.with_suffix(".cpg")
    label = cpg.read_text(encoding="utf-8-sig").strip() if cpg.exists() else None
```

```
    probes = [_decode_probe(dbf, name) for name in ("utf-8", "gbk", "gb18030")]
    evidence = [
        f"DBF language driver byte: 0x{dbf['language_driver_id']:02X}",
        "Strict decoding checks include every DBF field name and character cell.",
        "GDAL reports its Unicode transfer encoding, which is not proof of source byte
        ↪  encoding.",
    ]
    selected = None
    reason = None
    if label:
        try:
            candidate = _normalise_encoding(label)
            probe = _decode_probe(dbf, candidate)
            if probe["all_text_bytes_decodable"]:
                selected, reason = candidate, "source_cpg_and_strict_byte_check"
            else:
                evidence.append("Source CPG conflicts with DBF text bytes; no automatic
                ↪  override.")
        except LookupError:
            evidence.append("Source CPG label is not recognised by this Python codec
            ↪  registry.")
    elif dbf["language_driver_id"] == 0x4D:

        if _decode_probe(dbf, "gbk")["all_text_bytes_decodable"]:
            selected, reason = "gbk", "dbf_ldid_0x4d_cp936_and_strict_byte_check"
            evidence.append("DBF LDID 0x4D identifies Chinese CP936/GBK.")
    elif all(all(byte < 128 for byte in raw) for _, raw in _dbf_text_cells(dbf)):
        selected, reason = "ascii", "all_field_names_and_character_bytes_are_ascii"
    unresolved = selected is None
    return {
        "source_cpg": label, "source_cpg_present": cpg.exists(),
        "dbf_language_driver_id_hex": f"0x{dbf['language_driver_id']:02X}",
        "selected_source_encoding": selected,
        "reader_encoding": selected if selected else "ISO-8859-1",
        "status": "unresolved_raw_bytes" if unresolved else "source_supported",
        "decision_basis": reason, "evidence": evidence, "candidate_probes": probes,
        "fallback_policy": (
            "Undecided bytes use reversible ISO-8859-1 transport only; they are not
            ↪  interpreted as human text. "
            "Raw DBF records and candidate decodings are retained. Confirm source encoding
            ↪  before text use."
            if unresolved else None
        ),
        "recommended_options_if_unresolved": [
            "Retain raw bytes and candidate decodings until the producer supplies encoding
            ↪  metadata.",
            "Confirm a candidate against independent source labels before selecting it in a
            ↪  later version.",
        ] if unresolved else [],
    }


def _xml_evidence(path: Path, field_names: list[str]) -> dict[str, Any]:
    xml_path = path.with_suffix(".shp.xml")
```

```
    if not xml_path.exists():
        return {"present": False}
    data = xml_path.read_bytes()
    try:
        root = ET.fromstring(data)
        all_text = "\n".join(text for text in root.itertext() if text.strip())
        matches = [name for name in field_names if any(ord(c) > 127 for c in name) and name
        ↪  in all_text]
        declared = re.search(br'<\?xml[^>]*encoding=[\'\"]([^\'\"]+)', data[:256])
        return {
            "present": True, "sha256": _sha(xml_path), "xml_parse": "pass",
            "xml_document_encoding": declared.group(1).decode("ascii") if declared else None,
            "matched_non_ascii_dbf_field_names": matches,
            "arcgis_process_entries": sum(element.tag.rsplit("}", 1)[-1] == "Process" for
            ↪  element in root.iter()),
            "interpretation": "XML document encoding is independent of DBF attribute
            ↪  encoding. Processing history corroborates field labels only; it is not a
            ↪  current attribute codebook.",
        }
    except (ET.ParseError, UnicodeError) as error:
        return {"present": True, "sha256": _sha(xml_path), "xml_parse": "pending", "error":
        ↪  str(error)}


def _write_raw_dbf(path: Path, dbf: dict[str, Any]) -> None:
    with path.open("wb") as binary:
        with gzip.GzipFile(filename="", mode="wb", fileobj=binary, mtime=0) as compressed:
            import io
            with io.TextIOWrapper(compressed, encoding="utf-8", newline="") as stream:
                writer = csv.writer(stream, lineterminator="\n")
                writer.writerow(["source_row", "source_fid", "dbf_deleted", "record_hex"])
                for row, flag in enumerate(dbf["flags"]):
                    start = dbf["header_size"] + row * dbf["record_size"]
                    writer.writerow([row + 1, row, flag == 0x2A, dbf["bytes"][start:start +
                    ↪  dbf["record_size"]].hex()])


def _write_candidate_decodings(path: Path, dbf: dict[str, Any]) -> None:
    rows = []
    for index, field in enumerate(dbf["fields"]):
        name_bytes = field["name_bytes"]
        for encoding in ("utf-8", "gbk", "gb18030"):
            try:
                text, state = name_bytes.decode(encoding, errors="strict"),
                ↪  "decoded_candidate_only"
            except UnicodeDecodeError:
                text, state = "", "strict_decode_failed"
            rows.append({"source_row": 0, "field_index": index, "encoding": encoding,
            ↪  "raw_hex": name_bytes.hex(), "decoded_candidate": text, "status": state})
        if field["type"] != "C":
            continue
        for row in range(dbf["record_count"]):
            start = dbf["header_size"] + row * dbf["record_size"] + field["record_offset"]
            raw = dbf["bytes"][start:start + field["width"]].rstrip(b" \0")
```

```
            for encoding in ("utf-8", "gbk", "gb18030"):
                try:
                    text, state = raw.decode(encoding, errors="strict"),
                    ↪  "decoded_candidate_only"
                except UnicodeDecodeError:
                    text, state = "", "strict_decode_failed"
                rows.append({"source_row": row + 1, "field_index": index, "encoding":
                ↪  encoding, "raw_hex": raw.hex(), "decoded_candidate": text, "status":
                ↪  state})
    _write_csv(path, pd.DataFrame(rows, columns=["source_row", "field_index", "encoding",
    ↪  "raw_hex", "decoded_candidate", "status"]))


def _geometry_quality(frame: gpd.GeoDataFrame) -> pd.DataFrame:
    geometries = frame.geometry.array
    missing = np.asarray(shapely.is_missing(geometries))
    empty = np.asarray(shapely.is_empty(geometries))
    valid = np.asarray(shapely.is_valid(geometries))
    status = np.full(len(frame), "valid", dtype=object)
    status[~valid] = "invalid"
    status[empty] = "empty"
    status[missing] = "missing"
    wkbs = shapely.to_wkb(geometries)
    return pd.DataFrame({
        "source_row": frame["source_row"].to_numpy(), "source_fid":
        ↪  frame["source_fid"].to_numpy(),
        "geometry_status": status, "geometry_type": frame.geometry.geom_type.to_numpy(),
        "validity_reason": shapely.is_valid_reason(geometries),
        "has_z": shapely.has_z(geometries), "has_m": shapely.has_m(geometries),
        "coordinate_dimension": shapely.get_coordinate_dimension(geometries),
        "geometry_wkb_sha256": [hashlib.sha256(wkb).hexdigest() if wkb is not None else None
        ↪  for wkb in wkbs],
        "source_qc_presence": "not_provided",
        "qc_interpretation": "structural diagnostics only; source scientific QC is not
        ↪  supplied",
    })


def _attribute_quality(frame: gpd.GeoDataFrame, columns: list[str], dbf: dict[str, Any],
↪  encoding: str):
    layout = {field["name_bytes"].decode(encoding): field for field in dbf["fields"]}
    profile = []
    duplicates = []
    issues = []
    for column in columns:
        series = frame[column]
        field = layout[column]
        blanks = series.map(lambda value: isinstance(value, str) and not value.strip())
        profile.append({
            "field": column, "decoded_dtype": str(series.dtype), "dbf_type": field["type"],
            "dbf_width": field["width"], "dbf_decimals": field["decimals"],
            "rows": len(frame), "null_count": int(series.isna().sum()),
            "blank_string_count": int(blanks.sum()), "non_null_distinct_count":
            ↪  int(series.nunique(dropna=True)),
```

```
            "source_qc_presence": "not_provided", "missing_value_policy": "preserved; no
            ↪  null-to-zero or undeclared sentinel substitution",
        })
        if column.casefold() in _ID_COLUMNS:
            duplicated = series.notna() & series.duplicated(keep=False)
            for position in np.flatnonzero(duplicated.to_numpy()):
                duplicates.append({
                    "source_row": int(frame.iloc[position]["source_row"]), "source_fid":
                    ↪  int(frame.iloc[position]["source_fid"]),
                    "field": column, "value": str(series.iloc[position]), "action":
                    ↪  "retained_without_deduplication",
                    "key_status": "identifier_candidate; uniqueness is not assumed",
                })

        sentinel_like = series.astype("string").isin(["-9999", "-9999.0", "-9999999",
        ↪  "9999.9"])
        for position in np.flatnonzero(sentinel_like.to_numpy(dtype=bool, na_value=False)):
            issues.append({
                "source_row": int(frame.iloc[position]["source_row"]), "source_fid":
                ↪  int(frame.iloc[position]["source_fid"]),
                "field": column, "observed_value": str(series.iloc[position]),
                "observation": "placeholder_like_token_uninterpreted", "action": "retained;
                ↪  field-specific source definition required before recoding",
            })
    return (
        pd.DataFrame(profile),
        pd.DataFrame(duplicates, columns=["source_row", "source_fid", "field", "value",
        ↪  "action", "key_status"]),
        pd.DataFrame(issues, columns=["source_row", "source_fid", "field", "observed_value",
        ↪  "observation", "action"]),
    )


def _verify_layer(expected: gpd.GeoDataFrame, path: Path, layer: str) -> dict[str, Any]:
    actual = pyogrio.read_dataframe(path, layer=layer)
    if len(expected) != len(actual):
        raise ValueError(f"Readback row-count mismatch for {layer}")
    if expected.crs != actual.crs:
        raise ValueError(f"Readback CRS mismatch for {layer}")
    if list(expected.columns) != list(actual.columns):
        raise ValueError(f"Readback attribute/geometry column mismatch for {layer}")
    pd.testing.assert_frame_equal(
        expected.drop(columns=expected.geometry.name).reset_index(drop=True),
        actual.drop(columns=actual.geometry.name).reset_index(drop=True),
        check_dtype=False, check_exact=True,
    )
    before, after = shapely.to_wkb(expected.geometry.array),
    ↪  shapely.to_wkb(actual.geometry.array)
    if not all(left == right for left, right in zip(before, after)):
        raise ValueError(f"Readback WKB mismatch (coordinates/order/dimension changed) for
        ↪  {layer}")
    return {
        "status": "pass", "rows": len(actual), "all_attribute_values_equal": True,
        "row_order_and_stable_ids_equal": True, "crs_equivalent": True,
```

```
        "geometry_wkb_exact": True, "coordinates_rings_parts_and_z_unchanged": True,
        "dtype_policy": "GeoPackage integer/date storage widths can differ; all decoded
        ↪  values and missingness compared exactly",
    }


def _clean_layer(path: Path, attachment: int, input_root: Path, output_root: Path) ->
↪  dict[str, Any]:
    source_file = _relative(path, input_root)
    slug = _OUTPUT_NAMES.get(path.stem, "layer_" +
    ↪  hashlib.sha256(source_file.encode()).hexdigest()[:12])
    target = output_root / f"attachment{attachment:02d}" / "vectors"
    target.mkdir(parents=True, exist_ok=True)
    required = [path, path.with_suffix(".shx"), path.with_suffix(".dbf"),
    ↪  path.with_suffix(".prj")]
    missing = [item.suffix for item in required if not item.is_file()]
    if missing:
        raise ValueError(f"Required shapefile components are missing: {missing}")
    dbf = _dbf_layout(path.with_suffix(".dbf"))
    encoding = _encoding_decision(path, dbf)
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        info = pyogrio.read_info(path, encoding=encoding["reader_encoding"])
        frame = pyogrio.read_dataframe(path, encoding=encoding["reader_encoding"],
        ↪  fid_as_index=True)
    source_columns = [column for column in frame.columns if column != frame.geometry.name]
    if any(column in _RESERVED for column in source_columns):
        raise ValueError("Source attributes collide with reserved provenance columns;
        ↪  explicit name mapping is required")
    if len(source_columns) != len(set(source_columns)):
        raise ValueError("Duplicate decoded field names are ambiguous; raw DBF bytes remain
        ↪  authoritative")
    fields_encoding = encoding["selected_source_encoding"] or "latin-1"
    if source_columns != [field["name_bytes"].decode(fields_encoding) for field in
    ↪  dbf["fields"]]:
        raise ValueError("Decoded DBF field order does not match the GIS reader")
    active_fids = [row for row, flag in enumerate(dbf["flags"]) if flag == 0x20]
    if list(frame.index) != active_fids:
        raise ValueError("GIS reader omitted/reordered source DBF records; refusing silent
        ↪  row loss")
    if len(active_fids) != dbf["record_count"]:
        raise ValueError("Deleted DBF records require an explicit retained-record import; no
        ↪  source rows are silently dropped")
    frame.insert(0, "source_fid", np.asarray(frame.index, dtype="int64"))
    frame.insert(0, "source_row", frame["source_fid"].to_numpy() + 1)
    frame.insert(2, "source_file", source_file)
    frame["qc_source_presence"] = "not_provided"
    frame["qc_source_encoding"] = encoding["status"]
    quality = _geometry_quality(frame)
    frame["qc_geometry_status"] = quality["geometry_status"].to_numpy()


    geometry_name = frame.geometry.name
    frame = frame[[column for column in frame.columns if column != geometry_name] +
    ↪  [geometry_name]]
```

```
frame = frame.reset_index(drop=True)
profile, duplicate_ids, issues = _attribute_quality(frame, source_columns, dbf,
↪  fields_encoding)
analysis = frame.loc[frame["qc_geometry_status"].eq("valid")].copy()
files = {
    "geopackage": target / f"{slug}.gpkg",
    "attributes": target / f"{slug}_attributes.csv.gz",
    "geometry_quality": target / f"{slug}_geometry_quality.csv.gz",
    "attribute_profile": target / f"{slug}_attribute_profile.csv.gz",
    "duplicate_ids": target / f"{slug}_duplicate_ids.csv.gz",
    "attribute_observations": target / f"{slug}_attribute_observations.csv.gz",
    "raw_dbf_records": target / f"{slug}_raw_dbf_records.csv.gz",
    "metadata": target / f"{slug}_metadata.json",
}
if any(item.exists() for item in files.values()):
    raise FileExistsError(f"Refusing to overwrite an existing vector layer output:
    ↪  {slug}")
pyogrio.write_dataframe(frame, files["geopackage"], layer="all_features", driver="GPKG",
↪  promote_to_multi=False)
pyogrio.write_dataframe(analysis, files["geopackage"], layer="analysis_valid_geometry",
↪  driver="GPKG", promote_to_multi=False, append=False)
checks = {
    "all_features": _verify_layer(frame, files["geopackage"], "all_features"),
    "analysis_valid_geometry": _verify_layer(analysis, files["geopackage"],
    ↪  "analysis_valid_geometry"),
}
_write_csv(files["attributes"], frame.drop(columns=frame.geometry.name))
_write_csv(files["geometry_quality"], quality)
_write_csv(files["attribute_profile"], profile)
_write_csv(files["duplicate_ids"], duplicate_ids)
_write_csv(files["attribute_observations"], issues)
_write_raw_dbf(files["raw_dbf_records"], dbf)
if encoding["status"] == "unresolved_raw_bytes":
    files["candidate_decodings"] = target / f"{slug}_candidate_decodings.csv.gz"
    _write_candidate_decodings(files["candidate_decodings"], dbf)
source_components = []
for component in sorted(path.parent.iterdir()):
    if component.is_file() and (component.stem == path.stem or
    ↪  component.name.startswith(path.name + ".")):
        source_components.append({"path": _relative(component, input_root), "bytes":
        ↪  component.stat().st_size, "sha256": _sha(component)})
geometry_counts = {str(key): int(value) for key, value in
↪  quality["geometry_status"].value_counts().items()}
metadata = {
    "attachment": attachment, "source_file": source_file, "output_name": slug,
    "source_components": source_components, "source_rows": dbf["record_count"],
    "retained_rows": len(frame), "analysis_geometry_rows": len(analysis),
    "source_fid_definition": "GDAL Shapefile FID, zero-based physical DBF/SHP record
    ↪  position",
    "source_row_definition": "one-based physical DBF/SHP record position = source_fid +
    ↪  1",
    "source_attribute_columns": source_columns, "source_qc_presence": "not_provided",
    "source_qc_note": "No dedicated scientific QC field is documented in the supplied
    ↪  vector attributes; inventory status/classification attributes are retained
    ↪  without reinterpretation.",
```

```
    "source_crs_wkt": frame.crs.to_wkt() if frame.crs else None,
    "source_crs_authority": list(frame.crs.to_authority()) if frame.crs and
    ↪  frame.crs.to_authority() else None,
    "source_prj_sha256": _sha(path.with_suffix(".prj")),
    "crs_status": "source_metadata_preserved" if frame.crs else "unknown_not_guessed",
    "source_geometry_type": str(info["geometry_type"]), "geometry_status_counts":
    ↪  geometry_counts,
    "geometry_types": {str(key): int(value) for key, value in
    ↪  frame.geometry.geom_type.value_counts(dropna=False).items()},
    "geometry_with_z": int(quality["has_z"].sum()), "geometry_with_m":
    ↪  int(quality["has_m"].sum()),
    "encoding": encoding, "xml_evidence": _xml_evidence(path, source_columns),
    "dbf_layout": {
        "version": dbf["version"], "header_size": dbf["header_size"], "record_size":
        ↪  dbf["record_size"],
        "deleted_record_count": dbf["flags"].count(0x2A),
        "fields": [{**{key: value for key, value in field.items() if key !=
        ↪  "name_bytes"}, "name": field["name_bytes"].decode(fields_encoding),
        ↪  "name_bytes_hex": field["name_bytes"].hex()} for field in dbf["fields"]],
    },
    "policies": {
        "all_features": "all source rows and decoded attributes; original geometry
        ↪  including invalid geometry retained",
        "analysis_valid_geometry": "only structural geometry screening; excludes
        ↪  missing/empty/invalid geometry without repairing or splitting",
        "geometry_repair": "none", "coordinate_transformation": "none", "deduplication":
        ↪  "none",
        "missing_values": "no fill, zero substitution, global sentinel recoding or
        ↪  undocumented missingness inference",
        "units": "source attributes retained; no field units are guessed",
        "csv": "UTF-8 compressed CSV, decoded nulls exported as empty fields; profiles
        ↪  and raw DBF bytes preserve interpretation evidence",
        "raw_dbf": "record_hex includes the full deletion marker and fixed-width record
        ↪  bytes; use dbf_layout to decode without losing whitespace",
    },
    "duplicate_identifier_rows": len(duplicate_ids), "attribute_observations":
    ↪  len(issues),
    "reader_warnings": [str(item.message).replace(str(path), source_file) for item in
    ↪  caught],
    "readback_verification": checks,
    "engine_versions": {"geopandas": gpd.__version__, "pyogrio": pyogrio.__version__,
    ↪  "gdal": pyogrio.__gdal_version_string__, "shapely": shapely.__version__, "geos":
    ↪  shapely.geos_version_string, "pandas": pd.__version__},
    "outputs": {key: _relative(value, output_root) for key, value in files.items()},
    "remaining_semantic_questions": [
        "Attribute units, categorical codes and observation/interpretation date meanings
        ↪  require the field-specific source documentation before analytical use."
    ],
}
if attachment == 6:
    metadata["remaining_semantic_questions"].append("The source is Polygon Z, despite
    ↪  being called historical disaster points. Resolution/imagery/river fields contain
    ↪  potentially inconsistent semantics; values are retained exactly.")
if attachment == 8:
```

```
        metadata["remaining_semantic_questions"].append("合计 and 人口数 are distinct source
        ↪  fields; population/household scope is not equated or overwritten.")
    _write_json(files["metadata"], metadata)
    return {
        "attachment": attachment, "source_file": source_file, "name": slug,
        "status": "complete" if encoding["status"] == "source_supported" else
        ↪  "pending_encoding",
        "rows": len(frame), "analysis_geometry_rows": len(analysis),
        "geometry_status_counts": geometry_counts, "source_encoding":
        ↪  encoding["selected_source_encoding"],
        "source_qc_presence": "not_provided", "duplicate_identifier_rows": len(duplicate_ids),
        "readback_verification": checks,
        "outputs": metadata["outputs"],
        "output_fingerprints": {key: {"sha256": _sha(value), "bytes": value.stat().st_size}
        ↪  for key, value in files.items()},
    }


def clean_vectors(input_root: Path, output_root: Path) -> dict[str, Any]:


    input_root, output_root = Path(input_root).resolve(), Path(output_root).resolve()
    if not input_root.is_dir():
        raise FileNotFoundError("input_root must be an existing local attachment directory")
    if output_root == input_root or input_root.is_relative_to(output_root) or
    ↪  output_root.is_relative_to(input_root):
        raise ValueError("Input and output roots must be separate directories, with neither
        ↪  containing the other")
    discovered = []
    for path in sorted(input_root.rglob("*.shp")):
        match = next((match for part in path.relative_to(input_root).parts[:-1] if (match :=
        ↪  _ATTACHMENT_RE.match(part))), None)
        if match:
            discovered.append((int(match.group(1)), path))
    layers, failures = [], []
    for attachment, path in discovered:
        try:
            layers.append(_clean_layer(path, attachment, input_root, output_root))
        except Exception as error:

            failures.append({"attachment": attachment, "source_file": _relative(path,
            ↪  input_root), "status": "pending", "error_type": type(error).__name__,
            ↪  "error": str(error).replace(str(input_root),
            ↪  "<input_root>").replace(str(output_root), "<output_root>")})
    counts = {number: sum(attachment == number for attachment, _ in discovered) for number in
    ↪  _ATTACHMENTS}
    inventory = {f"attachment{number:02d}": {"minimum_expected_layers": 1, "found_layers":
    ↪  counts[number], "source_layers": [_relative(path, input_root) for attachment, path in
    ↪  discovered if attachment == number], "status": "pass" if counts[number] >= 1 else
    ↪  "pending"} for number in _ATTACHMENTS}
    complete = not failures and all(record["status"] == "pass" for record in
    ↪  inventory.values()) and all(layer["status"] == "complete" for layer in layers)
    result = {
        "status": "complete" if complete else "pending", "scope": "attachments 03, 06, 08
        ↪  vector basic cleaning only",
```

```
        "source_qc_presence": "not_provided", "inventory": inventory,
        "layers": layers, "failures": failures,
        "total_retained_rows": sum(layer["rows"] for layer in layers),
        "no_geometry_repair_or_coordinate_transform": True,
        "independent_review": "not_performed_by_this_executor",
        "inventory_path": "quality/vector_inventory.json",
    }

    json.dumps(result, ensure_ascii=False, allow_nan=False)
    inventory_path = output_root / result["inventory_path"]
    inventory_path.parent.mkdir(parents=True, exist_ok=True)
    if inventory_path.exists():
        raise FileExistsError("Refusing to overwrite quality/vector_inventory.json")
    _write_json(inventory_path, result)
    return result
```

问题一

q1/draw.py

```
from __future__ import annotations
import argparse
import gzip
import json
import math
import os
from pathlib import Path
import tempfile
os.environ.setdefault('MPLCONFIGDIR', str(Path(tempfile.gettempdir())/'q1_final_mplcache'))
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
import numpy as np

STYLE={'width_mm':156.,'dpi':400,'font':9.,'small':8.,'panel':9.5,'line':1.5,
       'target':'#164D68','accent':'#B66B32','neutral':'#62676D','band':'#DEE1E3',
       'grid':'#E6E8EA','missing':'#E4E6E8','colors':['#164D68','#B66B32','#718B9A','#6E637C'」
       ↪  ,'#8C8C66'],
       'display_seed':20261026,
       'fonts':['Noto Sans CJK SC','Source Han Sans SC','Microsoft YaHei','PingFang
       ↪  SC','Heiti SC','SimHei','Arial Unicode MS']}
SHORT={'rh2_mean':'2 m 相对湿度','t20_t2_diff':'20–2 m 气温差','ws10_mean':'10 m 标量风速',
       'warm_rate':'24 h 块均温变化率','ws20_ws2_diff':'20–2 m 标量风差','pdd':'正温指数 PDD',
       'rsd_mean':'Rsd 原通道均值','vwc01':'0.1 m 体积含水量','soilt01':'0.1 m 土温',
       't2_max':'小时均温最大值代理','rld_mean':'Rld原通道均值','g01':'0.1
       ↪  m板热流','p_mean':'MET气压均值'}
WINDOWS=(24,72,168,360,720)


def configure_style():
    fonts={f.name for f in font_manager.fontManager.ttflist}
    chinese=next((s for s in STYLE['fonts'] if s in fonts),None)
```

```
    if chinese is None:raise RuntimeError('需要可用中文字体，如 Noto Sans CJK SC。')
    matplotlib.rcParams.update({'font.family':['DejaVu
    ↪  Sans',chinese],'font.sans-serif':[chinese,'DejaVu Sans'],
        'font.size':STYLE['font'],'axes.labelsize':STYLE['font'],'axes.titlesize':STYLE['pane」
        ↪  l'],
        'xtick.labelsize':STYLE['font'],'ytick.labelsize':STYLE['font'],'legend.fontsize':8.5,
        'axes.spines.right':False,'axes.spines.top':False,'axes.linewidth':.7,
        'axes.unicode_minus':True,'pdf.fonttype':42,'ps.fonttype':42,'svg.fonttype':'none',
        'figure.facecolor':'white','axes.facecolor':'white','savefig.facecolor':'white',
        'text.color':'#252A2E','axes.labelcolor':'#252A2E','xtick.color':'#44494D','ytick.col」
        ↪  or':'#44494D',
        'legend.frameon':False,'lines.linewidth':STYLE['line']})
    return chinese


def load_json(path):
    p=Path(path)
    with gzip.open(p,'rt',encoding='utf-8') if p.suffix=='.gz' else p.open(encoding='utf-8')
    ↪  as stream:
        return json.load(stream)


def array(values):
    return np.asarray([float(v) if v is not None and math.isfinite(float(v)) else np.nan for
    ↪  v in values],dtype=float)


def grid(rows,cols,height,left=16,right=4,bottom=16,top=14,hgap=18,vgap=21):

    w=STYLE['width_mm'];fig=plt.figure(figsize=(w/25.4,height/25.4))
    pw=(w-left-right-(cols-1)*hgap)/cols;ph=(height-top-bottom-(rows-1)*vgap)/rows
    axes=[]
    for row in range(rows):
        axes.append([fig.add_axes([(left+col*(pw+hgap))/w,(height-top-(row+1)*ph-row*vgap)/he」
        ↪  ight,pw/w,ph/height]) for col in
        ↪  range(cols)])
    return fig,np.asarray(axes)


def panel(ax,letter,title):
    ax.text(0,1.20,f'({letter})
    ↪  {title}',transform=ax.transAxes,ha='left',va='bottom',fontsize=STYLE['panel'])
    ax.tick_params(direction='out',length=3,width=.7,pad=3)
    ax.grid(axis='y',color=STYLE['grid'],linewidth=.5,zorder=0)
    ax.set_axisbelow(True)


def note(fig,text,y=.012):
    fig.text(.025,y,text,fontsize=STYLE['small'],color=STYLE['neutral'],va='bottom')


def time_axes(ax,start,label=True):
    ax.set_xlim(start,0)
    ax.set_xticks([-30,-20,-10,0] if start==-30 else [-7,-5,-3,-1,0] if start==-7 else
    ↪  [-3,-2,-1,0])
```

```
    if label:ax.set_xlabel('距截止时刻（日）',labelpad=3)


def support(ch,period):
    s=ch['summaries'][period]
    return f"{s['available_hours']}/{s['expected_hours']} h"


def background(ax,x,ch,start):
    mask=(x>start)&(x<=0);xx=x[mask]
    lo=array(ch['background_q25'])[mask];mid=array(ch['background_median'])[mask];hi=array(ch」
    ↪  ['background_q75'])[mask]
    finite=np.isfinite(lo)&np.isfinite(hi)
    if np.any(lo[finite]>hi[finite]):raise ValueError('背景分位次序错误。')
    ax.fill_between(xx,lo,hi,color=STYLE['band'],linewidth=0,zorder=1)
    ax.plot(xx,mid,color=STYLE['neutral'],lw=.9,ls=(0,(3,2)),zorder=2)


def target(ax,x,ch,start,color=None,label=None,linestyle='-',bg=True,lw=None):
    if bg:background(ax,x,ch,start)
    mask=(x>start)&(x<=0)
    ax.plot(x[mask],array(ch['values'])[mask],color=color or STYLE['target'],lw=lw or
    ↪  STYLE['line'],
            ls=linestyle,label=label,zorder=3)


def context_legend(fig,y=.99,ncol=3):
    handles=[Line2D([],[],color=STYLE['target'],lw=1.5,label='目标过程'),
             Line2D([],[],color=STYLE['neutral'],ls=(0,(3,2)),lw=.9,label='历史中位数'),
             Patch(facecolor=STYLE['band'],edgecolor='none',label='历史 25%—75%')]
    fig.legend(handles=handles,loc='upper center',bbox_to_anchor=(.52,y),ncol=ncol,
               handlelength=1.8,columnspacing=1.0,handletextpad=.4,fontsize=8.5)


def channel_map(process):
    return {c['id']:c for c in process['weather']['channels']}


def validate(data,process,pca):
    branch=data['primary_branch'];meta=process['metadata']
    if branch['endpoint']!='H-END' or branch['cutoff_hour']!=10 or
    ↪  meta['primary_cutoff']!='2026-08-26T10:00:00+08:00':
        raise ValueError('输入不是冻结的 UTC+8 主截止/端点分支。')
    if pca['metadata']['endpoint']!='H-END' or
    ↪  pca['metadata']['cutoff']!=meta['primary_cutoff']:
        raise ValueError('PCA 与过程时间条件不一致。')
    ids=[r['id'] for r in data['primary']['candidate_ranking']]
    if ids!=pca['feature_ids'] or len(ids)!=13:raise ValueError('候选池/PCA 输入不一致。')
    return ids


def figure_structure(data,process,pca):
    fig,axes=grid(2,2,130,top=22,bottom=20,vgap=24)
    near=process['near_surface'];heights=array(near['heights_m'])
```

```
    profiles={int(p['support_end_relative_hours']):p for p in near['profiles']}
    for col,(key,unit,title) in
    ↪  enumerate((('temperature','°C','气温剖面'),('wind','m/s','标量风速剖面'))):
        ax=axes[0,col]
        ref=profiles[0][key]
        ax.fill_betweenx(heights,array(ref['background_q25']),array(ref['background_q75']),co」
        ↪  lor=STYLE['band'],lw=0)
        ax.plot(array(ref['background_median']),heights,ls=(0,(3,2)),color=STYLE['neutral'],l」
        ↪  w=.9)
        for hour,color,marker in ((-24,STYLE['accent'],'s'),(0,STYLE['target'],'o')):
            ax.plot(array(profiles[hour][key]['values']),heights,color=color,marker=marker,lw」
            ↪  =1.5,ms=3.5)
        ax.set_ylim(0,21);ax.set_yticks([2,10,20]);ax.set_ylabel('高度 /
        ↪  m');ax.set_xlabel(unit)
        panel(ax,chr(97+col),title)
    th=near['time_height'];x=array(th['support_end_relative_hours'])/24
    for col,(value,prefix,skey,unit,title) in enumerate((
        ('temperature_difference_c','pair_temperature','temperature_difference_summaries','°C」
        ↪  ','T20−T2：
        ↪  末7日'),
        ('wind_difference_ms','pair_wind','wind_difference_summaries','m/s','U20−U2：末7日'))):
        ch={'values':th[value],**{f'background_{q}':th[f'{prefix}_background_{q}'] for q in
        ↪  ('q25','median','q75')}}
        ax=axes[1,col];target(ax,x,ch,-7);ax.axhline(0,color=STYLE['neutral'],lw=.65)
        ax.set_ylabel(unit);time_axes(ax,-7);panel(ax,chr(99+col),title)
        s=th[skey]['7d'];ax.text(0,1.015,f"n={s['available_hours']}/{s['expected_hours']}
        ↪  h",transform=ax.transAxes,va='bottom',fontsize=8)
    fig.legend(handles=[Line2D([],[],color=STYLE['accent'],marker='s',ms=3.5,label='−24
    ↪  h剖面'),
        Line2D([],[],color=STYLE['target'],marker='o',ms=3.5,label='截止剖面'),
        Line2D([],[],color=STYLE['neutral'],ls=(0,(3,2)),lw=.9,label='历史中位数'),
        Patch(facecolor=STYLE['band'],edgecolor='none',label='历史四分位')],loc='upper
        ↪  center',bbox_to_anchor=(.53,.995),ncol=4,
        fontsize=8,handlelength=1.3,columnspacing=.8,handletextpad=.35)
    note(fig,'北京时间 8 月 26 日 10:00 截止；剖面背景取截止同钟点，层差背景逐时匹配。')
    return fig


def add_extremes(ax,process,x,ch_id,start):
    for e in process['extremes']['series']:
        if e['channel_id']!=ch_id:continue
        hit=np.asarray([v is True for v in e['exceedance']])&(x>start)
        color=STYLE['accent'] if e['direction']=='high' else STYLE['neutral']
        ax.plot(x[hit],array(e['values'])[hit],ls='none',marker='^' if e['direction']=='high'
        ↪  else 'v',ms=3.0,color=color,zorder=5)


def figure_process(data,process,pca):
    fig,axes=grid(3,2,171,top=22,bottom=20,vgap=23)
    ch=channel_map(process);x=array(process['weather']['time_relative_hours'])/24
    plan=[('t2',-30,'气温：30 日','°C'),('t2',-3,'气温：末 72 h','°C'),
          ('rh2',-30,'相对湿度：30 日','%'),('vwc01',-30,'0.1 m 含水量：30 日','m³/m³'),
          ('radiation',-30,'辐射原通道：30 日','W/m²'),('ws10',-3,'标量风速：末 72 h','m/s')]
    for i,(key,start,title,unit) in enumerate(plan):
```

```
        ax=axes.flat[i]
        if key=='radiation':
            for g,color in (('rsd',STYLE['target']),('rld',STYLE['accent'])):
                target(ax,x,ch[g],start,color=color,label=g.upper(),linestyle='-' if g=='rsd'
                ↪  else '--',bg=False)
            ns=[support(ch[g],'30d') for g in ('rsd','rld')]
            ntext='Rsd实 / Rld虚；各'+ns[0] if ns[0]==ns[1] else 'Rsd实 '+ns[0]+'；Rld虚 '+ns[1]
        else:
            target(ax,x,ch[key],start)
            if key in ('t2','ws10'):add_extremes(ax,process,x,key,start)
            ntext='n='+support(ch[key],'30d' if start==-30 else '72h')
        ax.set_ylabel(unit);time_axes(ax,start);panel(ax,chr(97+i),title)
        ax.text(0,1.015,ntext,transform=ax.transAxes,va='bottom',fontsize=8,color=STYLE['neut」
        ↪  ral'])

    limits=[axes[0,j].get_ylim() for j in range(2)];lo=min(a for a,b in limits);hi=max(b for
    ↪  a,b in limits)
    for ax in axes[0]:ax.set_ylim(lo,hi)
    context_legend(fig)
    note(fig,'△ 高于历史第 95 百分位；▽ 低于历史第 5 百分位；辐射仅列原值。')
    return fig


def figure_multiscale(data,process,pca):
    ids=[r['id'] for r in data['primary']['candidate_ranking']];rows=[];texts=[]
    for g in ids:
        source=data['primary']['factors'][g]['windows'];vals=[];txt=[]
        for hours in WINDOWS:
            r=source.get(str(hours))
            if r is None or r.get('a_mean') is
            ↪  None:vals.append(np.nan);txt.append('不适');continue
            a=float(r['a_mean']);lo=float(r['q_min']);hi=float(r['q_max'])
            direction='高' if lo>.5 else '低' if hi<.5 else '中' if lo==hi==.5 else '异'
            vals.append(a);txt.append(f'{a:.2f}{direction}')
        rows.append(vals);texts.append(txt)
    fig=plt.figure(figsize=(156/25.4,127/25.4));ax=fig.add_axes([.285,.14,.55,.82]);cm=plt.ge」
    ↪  t_cmap('cividis').copy();cm.set_bad(STYLE['missing'])
    vals=np.asarray(rows);mesh=ax.pcolormesh(np.arange(6)-.5,np.arange(len(ids)+1)-.5,np.ma.m」
    ↪  asked_invalid(vals),cmap=cm,vmin=0,vmax=1,shading='flat',linewidth=0,rasterized=False)
    ax.set_ylim(len(ids)-.5,-.5);ax.set_xlim(-.5,4.5);ax.set_yticks(range(len(ids)),[SHORT[g]
    ↪  for g in ids]);ax.set_xticks(range(5),['1','3','7','15','30']);ax.set_xlabel('窗口长度
    ↪  / 日')
    ax.tick_params(length=0,pad=5);[s.set_visible(False) for s in ax.spines.values()]
    for i,row in enumerate(texts):
        for j,txt in enumerate(row):
            value=vals[i,j];ax.text(j,i,txt,ha='center',va='center',fontsize=8.5,color='white」
            ↪  ' if np.isfinite(value) and value<.49 else
            ↪  '#202428')
    cax=fig.add_axes([.855,.265,.016,.56]);cb=fig.colorbar(mesh,cax=cax,ticks=[0,.5,1]);cb.ax」
    ↪  .set_title('a',fontsize=9,pad=6)
    if cb.solids is not None:cb.solids.set_rasterized(False)
    note(fig,'高/低：全部配置 q 在 0.5 同侧；异：跨侧；灰格为不适用，真实 0 值仍写数字。')
    return fig
```

```
def figure_ranking(data,process,pca):
    rows=data['primary']['candidate_ranking'];ids=[r['id'] for r in
    ↪  rows];values=np.array([r['score_S'] for r in rows])
    fig=plt.figure(figsize=(156/25.4,108/25.4));ax=fig.add_axes([.285,.16,.60,.79]);y=np.aran」
    ↪  ge(len(rows));top=set(data['primary']['top_candidates'])
    for i,(g,s) in enumerate(zip(ids,values)):
        color=STYLE['accent'] if g in top else
        ↪  STYLE['target'];ax.hlines(i,0,s,color=STYLE['band'],lw=1.0)
        ax.plot(s,i,'o',color=color,ms=5.5 if g in top else
        ↪  4.4);ax.text(1.025,i,f'{s:.3f}',ha='left',va='center',fontsize=9,clip_on=False)
    cutoff=data['primary'].get('top_cutoff_score_S')
    if cutoff is not None:ax.axvline(cutoff,color=STYLE['accent'],ls=(0,(3,3)),lw=.8)
    ax.set_yticks(y,[SHORT[g] for g in ids]);ax.set_ylim(len(rows)-.45,-.55);ax.set_xlim(0,1)」
    ↪  ;ax.set_xticks([0,.25,.5,.75,1]);ax.set_xlabel('校准历史位置
    ↪  S')
    ax.grid(axis='x',color=STYLE['grid'],lw=.5);ax.tick_params(axis='y',length=0,pad=5)
    fig.legend(handles=[Line2D([],[],color=STYLE['accent'],marker='o',ls='none',label='主前五'」
    ↪  ),Line2D([],[],color=STYLE['target'],marker='o',ls='none',label='其余候选')],loc='uppe」
    ↪  r
    ↪  right',bbox_to_anchor=(.98,1),ncol=2,fontsize=8.5,handletextpad=.25,columnspacing=.8)
    note(fig,f" 去重后 {len(rows)} 项竞争；虚线为第五项分数，不是概率、显著性或灾害阈值。")
    return fig


def figure_pca(data,process,pca):
    fig,axes=grid(2,2,130,top=21,bottom=20,vgap=23)
    rng=np.random.default_rng(STYLE['display_seed']);years=list(pca['support']['valid_years'])
    for i,(hours,metric) in enumerate(((168,'T2'),(720,'T2'),(168,'Q'),(720,'Q'))):
        ax=axes.flat[i];scale=pca['scales'][str(hours)];summary=scale['summaries']['main'];ag」
        ↪  g=summary['metrics'][metric]
        for j,year in enumerate(years):
            conf=[c for c in scale['configurations'] if c['status']=='ok' and
            ↪  c['k_branch']=='main' and c['calibration_year']==year]
            vals=[c['positions'][metric] for c in conf];valid=[v for v in vals if v is not
            ↪  None]
            jitter=rng.uniform(-.15,.15,len(valid));ax.scatter(j+jitter,valid,s=12,color='#A4」
            ↪  ABB0',alpha=.65,zorder=2,linewidths=0)
        for row in agg['calibration_years']:
            j=years.index(row['year']);ax.plot(j,row['position_mean'],'o',mfc='white',mec=STY」
            ↪  LE['target'],mew=1,ms=5,zorder=4)
        if agg['position_mean'] is not None:ax.plot(len(years),agg['position_mean'],'D',ms=6,」
        ↪  color=STYLE['accent'],zorder=5)
        ax.axhline(.5,color=STYLE['grid'],lw=.7);ax.set_ylim(0,1.03);ax.set_yticks([0,.5,1]);」
        ↪  ax.set_xlim(-.45,len(years)+.5)
        ax.set_xticks(range(len(years)+1),[y[-2:] for y in
        ↪  years]+['汇总']);ax.set_xlabel('校准年（2021—2025）');ax.set_ylabel('留出历史中秩位置')
        kvals='/'.join(map(str,summary['k_values']));title=f"{hours//24}日 {'T²' if
        ↪  metric=='T2' else 'Q'}，k={kvals}"
        panel(ax,chr(97+i),title)
        text='NA' if agg['position_mean'] is None else f" 汇总 {agg['position_mean']:.3f}"
        ax.text(0,1.015,text,transform=ax.transAxes,va='bottom',fontsize=8.5)
```

```
    fig.legend(handles=[Line2D([],[],marker='o',ms=3,color='#A4ABB0',ls='none',label='各参考配
    ↪ 置'),Line2D([],[],marker='o',mfc='white',mec=STYLE['target'],ls='none',label='校准年均
    ↪ 值'),Line2D([],[],marker='D',color=STYLE['accent'],ls='none',label='年度等权汇总')],loc
    ↪ ='upper
    ↪ center',bbox_to_anchor=(.52,.995),ncol=3,fontsize=8.5,columnspacing=.8,handletextpad=
    ↪ .3)
    counts={(s['summaries']['main']['metrics'][m]['valid_configurations'],s['summaries']['mai
    ↪ n']['metrics'][m]['valid_calibration_years']) for s in pca['scales'].values() for m
    ↪ in ('T2','Q')}
    count_text=f'各项{next(iter(counts))[0]}配置/{next(iter(counts))[1]}校准年' if
    ↪ len(counts)==1 else '有效分母见结果表'
    note(fig,count_text+f"；横向抖动种子 {STYLE['display_seed']}，纵值不变；0.5 非显著性阈值。")
    return fig


def figure_soil(data,process,pca):
    fig,axes=grid(2,2,143,top=22,bottom=20,vgap=24);ch=channel_map(process);x=array(process['
    ↪ weather']['time_relative_hours'])/24
    depths=('01','02','04','08','16');depth_labels=('0.1 m','0.2 m','0.4 m','0.8 m','1.6
    ↪ m');styles=('-','--','-.',':',(0,(5,1,1,1)))
    for row,(prefix,unit,title) in
    ↪ enumerate((('soilt','°C','五深度土温'),('vwc','m³/m³','五深度含水量'))):
        for col,start in enumerate((-30,-3)):
            ax=axes[row,col];background(ax,x,ch[prefix+'01'],start)
            for j,d in enumerate(depths):target(ax,x,ch[prefix+d],start,color=STYLE['colors']
            ↪ [j],linestyle=styles[j],bg=False,lw=1.35)
            ax.set_ylabel(unit);time_axes(ax,start);panel(ax,chr(97+row*2+col),title+('：30日'
            ↪ if col==0 else '：末72 h'))
        limits=[axes[row,j].get_ylim() for j in range(2)];lo=min(a for a,b in
        ↪ limits);hi=max(b for a,b in limits)
        for ax in axes[row]:ax.set_ylim(lo,hi)
    fig.legend(handles=[Line2D([],[],color=STYLE['colors'][j],ls=styles[j],label=lab,lw=1.35)
    ↪ for j,lab in enumerate(depth_labels)],loc='upper center',bbox_to_anchor=(.52,.995),nc
    ↪ ol=5,fontsize=8.5,handlelength=1.3,columnspacing=.65,handletextpad=.25)
    counts30={support(ch[p+d],'30d') for p in ('soilt','vwc') for d in
    ↪ depths};counts72={support(ch[p+d],'72h') for p in ('soilt','vwc') for d in depths}
    text='各深度n30='+next(iter(counts30))+'，n72='+next(iter(counts72)) if
    ↪ len(counts30)==len(counts72)==1 else '逐深度支持见结果表'
    note(fig,'灰线/灰带仅为 0.1 m 历史背景；'+text+'。')
    return fig


def figure_channels(data,process,pca):
    fig,axes=grid(4,2,232,left=20,top=13,bottom=20,hgap=19,vgap=23);ch=channel_map(process);x
    ↪ =array(process['weather']['time_relative_hours'])/24
    plan=[(('pressure',),'气压原列','hPa'),(('rsd','rsu'),'短波原通道','W/m²'),
          (('rld','rlu'),'长波原通道','W/m²'),(('h',),'H 原通道','W/m²'),
          (('le',),'LE 原通道','W/m²'),(('ustar',),'USTAR 原通道','m/s'),
          (('g01','g02'),'板热流原通道','W/m²')]
    for i,(keys,title,unit) in enumerate(plan):
        ax=axes.flat[i]
        for j,key in enumerate(keys):target(ax,x,ch[key],-30,color=STYLE['colors'][j],label=k
        ↪ ey.upper() if key not in ('g01','g02') else '0.1 m' if key=='g01' else '0.2
        ↪ m',linestyle='-' if j==0 else '--',bg=len(keys)==1,lw=1.35)
```

```
        ntext='; '.join((key.upper()+('实 ' if j==0 else '虚 ') if len(keys)>1 else
        ↪  'n=')+support(ch[key],'30d') for j,key in enumerate(keys))
        ax.text(0,1.015,ntext,transform=ax.transAxes,va='bottom',fontsize=8,color=STYLE['neut
        ↪  ral'])
        ax.set_ylabel(unit);time_axes(ax,-30);panel(ax,chr(97+i),title)
    ax=axes.flat[-1];ax.set_axis_off()
    ax.text(0,1,'全部保持有符号原值\n缺测处断线，
    ↪  不补零\n多通道面板只列目标\n单通道灰带为历史四分位\nH/LE/USTAR无源QC列\n不作净收支或蒸散换算',tra
    ↪  nsform=ax.transAxes,va='top',fontsize=9,linespacing=1.7)
    note(fig,'各通道支持以有效/应有小时列示；UTC+8 截止与历史背景口径同正文。')
    return fig


def figure_contributions(data,process,pca):
    ids=pca['feature_ids'];columns=((168,'T2'),(168,'Q'),(720,'T2'),(720,'Q'));values=np.full
    ↪  ((len(ids),4),np.nan);labels=[]
    for j,(hours,metric) in enumerate(columns):
        rows={r['id']:r for r in
        ↪  pca['scales'][str(hours)]['summaries']['main']['contributions'][metric]}
        column=[]
        for i,g in enumerate(ids):
            r=rows[g];q=r['position_mean'];share=r['normalized_share_mean'];values[i,j]=q if
            ↪  q is not None else np.nan
            if q is None or share is None:column.append('NA');continue
            rank=str(r['rank_min']) if r['rank_min']==r['rank_max'] else f"{r['rank_min']}–
            ↪  {r['rank_max']}"
            column.append(f"{share*100:.1f}%/{q:.2f}\n 秩 {rank}")
        labels.append(column)
    fig=plt.figure(figsize=(156/25.4,160/25.4));ax=fig.add_axes([.285,.18,.55,.76]);cm=plt.ge
    ↪  t_cmap('cividis').copy();cm.set_bad(STYLE['missing'])
    mesh=ax.pcolormesh(np.arange(5)-.5,np.arange(len(ids)+1)-.5,np.ma.masked_invalid(values),
    ↪  cmap=cm,vmin=0,vmax=1,shading='flat',rasterized=False,linewidth=0)
    ax.set_ylim(len(ids)-.5,-.5);ax.set_xlim(-.5,3.5);ax.set_yticks(range(len(ids)),[SHORT[g]
    ↪  for g in ids]);ax.set_xticks(range(4),['7日 cT/T²','7日 cQ/Q','30日 cT/T²','30日
    ↪  cQ/Q']);ax.xaxis.tick_top();ax.tick_params(length=0,pad=5,labelsize=8.5);[s.set_visib
    ↪  le(False) for s in
    ↪  ax.spines.values()]
    for j in range(4):
        for i,txt in enumerate(labels[j]):ax.text(j,i,txt,ha='center',va='center',fontsize=8,
        ↪  color='white' if np.isfinite(values[i,j]) and values[i,j]<.49 else
        ↪  '#202428',linespacing=1.25)
    cax=fig.add_axes([.855,.30,.016,.50]);cb=fig.colorbar(mesh,cax=cax,ticks=[0,.5,1]);cb.ax.
    ↪  set_title('q',fontsize=9,pad=6)
    if cb.solids is not None:cb.solids.set_rasterized(False)
    note(fig,'颜色为贡献历史位置q；格内：份额均值/q，第二行为配置名次范围。\n负cT不截断，份额不等于因果占比；
    ↪  整体联合位置仍以正文图5为准。')
    return fig


FIGURES={'q1_fig01_structure':figure_structure,'q1_fig02_process':figure_process,
         'q1_fig03_multiscale':figure_multiscale,'q1_fig04_ranking':figure_ranking,'q1_fig05_
         ↪  pca':figure_pca,
         'q1_app01_soil':figure_soil,'q1_app02_channels':figure_channels,'q1_app03_contributi
         ↪  ons':figure_contributions}
```

```
def build_figures(data,process,pca,only=None):
    validate(data,process,pca);wanted=set(only or FIGURES)
    if wanted-set(FIGURES):raise ValueError('未知图名。')
    return {name:fn(data,process,pca) for name,fn in FIGURES.items() if name in wanted}


def require_matplotlib_panel_alignment(fig, tolerance_pt=1.5):


    fig.canvas.draw()
    width_pt,height_pt=fig.get_size_inches()*72
    boxes=[np.array([a.get_position().x0*width_pt,a.get_position().y0*height_pt,
                     a.get_position().x1*width_pt,a.get_position().y1*height_pt])
           for a in fig.axes if a.axison and a.get_position().width>.10]
    for i,a in enumerate(boxes):
        for b in boxes[i+1:]:
            if abs(a[1]-b[1])<tolerance_pt:
                if abs(a[3]-b[3])>tolerance_pt or abs((a[2]-a[0])-(b[2]-b[0]))>tolerance_pt:
                    raise ValueError('同排证据面板的高度或宽度不一致。')
            if abs(a[0]-b[0])<tolerance_pt and abs(a[2]-b[2])>tolerance_pt:
                raise ValueError('同列证据面板没有对齐。')
    for item in fig.findobj(matplotlib.text.Text):
        if item.get_visible() and item.get_text() and item.get_fontsize()<STYLE['small']:
            raise ValueError('存在低于约定 8 pt 的绘图文字。')


def save_figure(fig,path,dpi=400,alignment_audit=None):
    fig.canvas.draw()
    require_matplotlib_panel_alignment(fig)
    if alignment_audit is not None:alignment_audit(fig)
    path=Path(path)
    fig.savefig(path.with_suffix('.png'),dpi=dpi)
    fig.savefig(path.with_suffix('.pdf'),metadata={'Creator':'Matplotlib','Author':'','Creati」
    ↪  onDate':None,'ModDate':None})
    fig.savefig(path.with_suffix('.svg'),metadata={'Creator':'Matplotlib','Date':None})


def main(argv=None):
    here=Path(__file__).resolve().parent;parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--results-dir',type=Path,default=here/'results');parser.add_argument」
    ↪  ('--output-dir',type=Path,default=here/'figures')
    parser.add_argument('--only',nargs='+',choices=list(FIGURES));parser.add_argument('--dpi'」
    ↪  ,type=int,default=STYLE['dpi'])
    args=parser.parse_args(argv)
    if args.dpi<300:parser.error('PNG 分辨率至少 300 dpi。')
    data=load_json(args.results_dir/'analysis_results.json.gz');process=load_json(args.result」
    ↪  s_dir/'process_results.json.gz');pca=load_json(args.results_dir/'pca_results.json.gz')
    font=configure_style();args.output_dir.mkdir(parents=True,exist_ok=True)
    for name,fig in build_figures(data,process,pca,args.only).items():save_figure(fig,args.ou」
    ↪  tput_dir/name,args.dpi);plt.close(fig)
    print(f'已生成{len(args.only or FIGURES)}张图，宽156 mm；PNG {args.dpi} dpi，PDF/SVG；中文字体：
    ↪  {font}。')
```

```
    return 0


SHORT.update({'d1_era5_precip_sum':'格点累计总降水','d1_era5_precip_max':'格点最大小时降水',
              'd1_era5_rain_hours':'格点有降水小时','d1_era5_api':'格点前期降水指数'})


def figure_precipitation(record):

    fig,axes=grid(3,1,169,left=17,right=5,top=21,bottom=25,vgap=23)
    x=array(record['time_relative_hours'])/24
    values=array(record['values'])
    ymax=max(float(np.nanmax(values)),float(np.nanmax(array(record['background_q95']))))*1.12
    for i,(key,start,title) in
    ↪  enumerate((('30d',-30,'30日过程'),('7d',-7,'末7日'),('72h',-3,'末72小时'))):
        ax=axes[i,0];mask=(x>start)&(x<=0);xx=x[mask]
        lo=array(record['background_q25'])[mask];hi=array(record['background_q75'])[mask]
        ax.fill_between(xx,lo,hi,color=STYLE['band'],lw=0)
        ax.plot(xx,array(record['background_median'])[mask],color=STYLE['neutral'],ls='--',lw」
        ↪  =.8)
        ax.plot(xx,values[mask],color=STYLE['target'],lw=1.05)
        exceed=np.asarray(record['high_p95_exceedance'],dtype=bool)[mask]
        ax.scatter(xx[exceed],values[mask][exceed],marker='^',s=14,color=STYLE['accent'],zord」
        ↪  er=5)
        s=record['summaries'][key]
        panel(ax,chr(97+i),title)
        ax.text(0,1.015,f"完整{s['available_hours']}/{s['expected_hours']} h；
        ↪  累计{s['sum_precipitation_mm']:.1f} mm；高于历史第95百分位{s['high_p95_hours']}
        ↪  h",transform=ax.transAxes,fontsize=8,va='bottom')
        ax.set_ylim(0,ymax);ax.set_ylabel('小时总降水 / mm');time_axes(ax,start)
    handles=[Line2D([],[],color=STYLE['target'],lw=1.1,label='目标格点'),
             Line2D([],[],color=STYLE['neutral'],ls='--',lw=.8,label='历史中位数'),
             Patch(facecolor=STYLE['band'],label='历史 25%—75%'),
             Line2D([],[],color=STYLE['accent'],marker='^',ls='none',ms=4,label='高于历史第95百」
             ↪  分位')]
    fig.legend(handles=handles,loc='upper
    ↪  center',bbox_to_anchor=(.52,.995),ncol=2,fontsize=8,handlelength=1.5,columnspacing=.9)
    note(fig,'固定ERA5格点总降水（含雨雪），各图采用同一量程。\n历史按年等权；重叠轨迹不是独立事件。
    ↪  格点结果不代表源坡实测。',y=.008)
    return fig


def enhanced_label(feature_id,features):
    if feature_id in SHORT:return SHORT[feature_id]
    return features.get(feature_id,{}).get('label',feature_id).replace('ERA5','格点')


def figure_enhanced_ranking(enhanced):
    rows=enhanced['primary']['candidate_ranking'];features={f['id']:f for f in
    ↪  enhanced['features']}
    fig=plt.figure(figsize=(156/25.4,139/25.4));ax=fig.add_axes([.34,.15,.53,.78])
    top=set(enhanced['primary']['top_candidates'])
    for i,row in enumerate(rows):
        g=row['id'];s=row['score_S'];color=STYLE['accent'] if g in top else STYLE['target']
```

```
        ax.hlines(i,0,s,color=STYLE['band'],lw=1)
        ax.plot(s,i,'s' if g.startswith('d1_') else 'o',ms=5,color=color)
        ax.text(1.035,i,f'{s:.3f}',va='center',ha='left',fontsize=8.5,clip_on=False)
    ax.set_yticks(range(len(rows)),[enhanced_label(r['id'],features) for r in
    ↪  rows]);ax.tick_params(axis='y',length=0,pad=4)
    ax.set_xlim(0,1);ax.set_ylim(len(rows)-.4,-.6);ax.set_xticks([0,.25,.5,.75,1]);ax.set_xla」
    ↪  bel('校准历史位置
    ↪  S')
    ax.grid(axis='x',color=STYLE['grid'],lw=.5)
    fig.legend(handles=[Line2D([],[],color=STYLE['target'],marker='o',ls='none',label='原站通」
    ↪  道或代理'),
                        Line2D([],[],color=STYLE['target'],marker='s',ls='none',label='ERA5格」
                        ↪  点代理'),
                        Line2D([],[],color=STYLE['accent'],marker='o',ls='none',label='主前五'」
                        ↪  )],
               loc='upper center',bbox_to_anchor=(.53,.995),ncol=3,fontsize=8.5,handletextpad」
               ↪  =.3,columnspacing=.8)
    note(fig,f'加入格点降水后的去重池：{len(rows)}项；S为历史相对位置，不是灾害概率。
    ↪  \n站点与格点空间支持不同；方形标识不能解释成新增站点观测。',y=.012)
    return fig


def figure_enhanced_multiscale(enhanced):
    rows=enhanced['primary']['candidate_ranking'];features={f['id']:f for f in
    ↪  enhanced['features']}
    ids=[r['id'] for r in rows];vals=[];texts=[]
    for g in ids:
        v=[];t=[]
        for h in WINDOWS:
            r=enhanced['primary']['factors'][g]['windows'].get(str(h),{})
            a=r.get('a_mean')
            if a is None:v.append(np.nan);t.append('不适');continue
            lo,hi=r['q_min'],r['q_max'];direction='高' if lo>.5 else '低' if hi<.5 else '中'
            ↪  if lo==hi==.5 else '异'
            v.append(a);t.append(f'{a:.2f}{direction}')
        vals.append(v);texts.append(t)
    fig=plt.figure(figsize=(156/25.4,145/25.4));ax=fig.add_axes([.34,.13,.48,.82]);cm=plt.get」
    ↪  _cmap('cividis').copy();cm.set_bad(STYLE['missing'])
    z=np.asarray(vals);mesh=ax.pcolormesh(np.arange(6)-.5,np.arange(len(ids)+1)-.5,np.ma.mask」
    ↪  ed_invalid(z),cmap=cm,vmin=0,vmax=1,shading='flat')
    ax.set_xlim(-.5,4.5);ax.set_ylim(len(ids)-.5,-.5);ax.set_yticks(range(len(ids)),[enhanced」
    ↪  _label(g,features) for g in
    ↪  ids]);ax.set_xticks(range(5),['1','3','7','15','30']);ax.set_xlabel('窗口长度 /
    ↪  日');ax.tick_params(length=0,pad=5)
    for spine in ax.spines.values():spine.set_visible(False)
    for i,row in enumerate(texts):
        for j,t in
        ↪  enumerate(row):ax.text(j,i,t,ha='center',va='center',fontsize=8,color='white' if
        ↪  np.isfinite(z[i,j]) and z[i,j]<.49 else '#202428')
    cax=fig.add_axes([.86,.28,.018,.56]);cb=fig.colorbar(mesh,cax=cax,ticks=[0,.5,1]);cb.ax.s」
    ↪  et_title('a',fontsize=9,pad=6)
    if cb.solids is not None:cb.solids.set_rasterized(False)
    note(fig,'高/低：所有参考配置的q同侧；异：跨侧。\n逐窗双侧异常a与多尺度校准S分开，灰格为该尺度不适用。
    ↪  ',y=.01)
```

```
    return fig


def figure_method_comparison(enhanced):
    rows=enhanced['comparisons'];features={f['id']:f for f in enhanced['features']}
    fig=plt.figure(figsize=(156/25.4,133/25.4));ax=fig.add_axes([.34,.13,.50,.81])
    z=np.asarray([[r[k]['rank_min'] for k in ('E2_rank','B_rank','C_rank')] for r in
    ↪  rows],dtype=float)
    mesh=ax.pcolormesh(np.arange(4)-.5,np.arange(len(rows)+1)-.5,z,cmap='cividis_r',vmin=1,vm」
    ↪  ax=len(rows),shading='flat')
    ax.set_xlim(-.5,2.5);ax.set_ylim(len(rows)-.5,-.5);ax.set_yticks(range(len(rows)),[enhanc」
    ↪  ed_label(r['id'],features) for r in
    ↪  rows]);ax.set_xticks(range(3),['多尺度','熵权TOPSIS','固定7日']);ax.xaxis.tick_top();ax.」
    ↪  tick_params(length=0,pad=6)
    for s in ax.spines.values():s.set_visible(False)
    for i in range(len(rows)):
        for j in range(3):ax.text(j,i,str(int(z[i,j])),ha='center',va='center',fontsize=8.5,c」
        ↪  olor='white' if z[i,j]>=8 else
        ↪  '#202428')
    cax=fig.add_axes([.88,.28,.018,.56]);cb=fig.colorbar(mesh,cax=cax,ticks=[1,5,10,15]);cb.a」
    ↪  x.set_title('名次',fontsize=8,pad=6)
    if cb.solids is not None:cb.solids.set_rasterized(False)
    note(fig,'共同15项原因素及降水代理；两项高度差代理没有原机制等级，未列入。\nTOPSIS在共同集重新拟合；
    ↪  名次重合不表示预测准确率。',y=.01)
    return fig


def enhanced_main(argv=None):
    here=Path(__file__).resolve().parent;parser=argparse.ArgumentParser(description='Q1 D0 +
    ↪  fixed ERA5 result-only figures')
    parser.add_argument('--results-dir',type=Path,default=here/'results');parser.add_argument」
    ↪  ('--output-dir',type=Path,default=here/'figures')
    parser.add_argument('--only',nargs='+');parser.add_argument('--dpi',type=int,default=400)
    args=parser.parse_args(argv)
    if args.dpi<300:parser.error('PNG 分辨率至少 300 dpi。')
    baseline=args.results_dir/'baseline';data=load_json(baseline/'analysis_results.json.gz');」
    ↪  process=load_json(baseline/'process_results.json.gz');pca=load_json(baseline/'pca_res」
    ↪  ults.json.gz')
    validate(data,process,pca);e=load_json(args.results_dir/'enhanced_results.json.gz');rain=」
    ↪  load_json(args.results_dir/'d1_process.json')
    if rain['metadata']['primary_cutoff']!=process['metadata']['primary_cutoff']:raise
    ↪  ValueError('D0/D1截止不一致')
    epca=e.get('pca');functions={
        'q1_fig01_structure':lambda:figure_structure(data,process,pca),
        'q1_fig02_process':lambda:figure_process(data,process,pca),
        'q1_fig03_precipitation':lambda:figure_precipitation(rain),
        'q1_fig04_ranking':lambda:figure_enhanced_ranking(e),
        'q1_app01_multiscale':lambda:figure_enhanced_multiscale(e),
        'q1_app02_soil':lambda:figure_soil(data,process,pca),
        'q1_app03_channels':lambda:figure_channels(data,process,pca),
        'q1_app04_methods':lambda:figure_method_comparison(e)}
    if epca and epca.get('scales'):functions['q1_fig05_pca']=lambda:figure_pca(e,process,epca)
    chosen=args.only or list(functions)
    if set(chosen)-set(functions):parser.error('未知或当前不可计算图名:
    ↪  '+','.join(set(chosen)-set(functions)))
```

```
    font=configure_style();args.output_dir.mkdir(parents=True,exist_ok=True)
    audit_hook=globals().get('_external_alignment_hook')
    for name in chosen:
        fig=functions[name]()
        hook=(lambda fig,n=name:audit_hook(fig,n)) if audit_hook else None
        save_figure(fig,args.output_dir/name,args.dpi,alignment_audit=hook);plt.close(fig)
    print(f'实际结果绘图完成：{len(chosen)} 图；156 mm，PNG{args.dpi} dpi 与 PDF/SVG；字体 {font}')
    return 0


if __name__=='__main__':raise SystemExit(enhanced_main())
```

q1/sol.py

```
from __future__ import annotations
import json
from pathlib import Path


from bisect import bisect_left, bisect_right
from collections import defaultdict
from fractions import Fraction
from itertools import combinations
from math import isfinite, sqrt
from numbers import Number
from typing import Any, Iterable, Mapping, Sequence

A_DEFAULT_YEARS = (2021, 2022, 2023, 2024, 2025)
A_ALLOWED_OFFSETS = tuple(range(-15, 16))
A_DIRECTIONS = {"high", "low", "two_sided"}


def _a_get(mapping: Mapping, key: int, default=None):
    return mapping.get(key, mapping.get(str(key), default))


def _a_finite(value: Any) -> bool:

    if isinstance(value, (bool, complex)):return False
    if isinstance(value, (int, Fraction)):return True
    return isinstance(value, Number) and isfinite(value)


def _a_fraction_text(value: Fraction) -> str:
    return f"{value.numerator}/{value.denominator}"


def a_midrank_fraction(sorted_values: Sequence[Number], value: Number) -> Fraction:

    if not sorted_values:
        raise ValueError("midrank reference must be nonempty")
    left, right = bisect_left(sorted_values, value), bisect_right(sorted_values, value)
    return Fraction(left + right, 2 * len(sorted_values))
```

```
def _a_direction(q: Fraction, direction: str) -> Fraction:
    if direction == "high":
        return q
    if direction == "low":
        return 1 - q
    if direction == "two_sided":
        return abs(2 * q - 1)
    raise ValueError(f"unknown direction: {direction}")


def _a_mean(values: Sequence[Fraction]) -> Fraction:
    return sum(values, Fraction(0)) / len(values)


def _a_prepare(feature: Mapping[str, Any], data: Mapping[str, Any], years: tuple[int, ...],
             min_endpoints: int) -> dict[str, Any]:
    factor_id = str(feature["id"])
    direction = feature["direction"]
    if direction not in A_DIRECTIONS:
        raise ValueError(f"{factor_id}: direction must be one of {sorted(A_DIRECTIONS)}")
    windows = tuple(int(w) for w in feature["window_hours"])
    if not windows or len(set(windows)) != len(windows) or any(w <= 0 for w in windows):
        raise ValueError(f"{factor_id}: window_hours must be distinct positive integers")
    raw_target = data.get("target", {})
    target = {w: _a_get(raw_target, w) for w in windows}
    per_year = data.get("years", {})
    endpoints, n_valid = {}, {}
    for year in years:
        raw_endpoints = _a_get(per_year, year, {})
        accepted = {}
        for offset in A_ALLOWED_OFFSETS:
            row = _a_get(raw_endpoints, offset, {})
            vals = {w: _a_get(row, w) for w in windows}
            if all(_a_finite(v) for v in vals.values()):
                accepted[offset] = vals
        n_valid[year] = len(accepted)
        if len(accepted) >= min_endpoints:
            endpoints[year] = accepted
    return {"id": factor_id, "direction": direction, "windows": windows,
            "target": target, "target_valid": all(_a_finite(v) for v in target.values()),
            "endpoints": endpoints, "n_valid": n_valid}


def _a_score_event(values: Mapping[int, Number], windows: tuple[int, ...], direction: str,
                   sorted_by_year: Mapping[int, Mapping[int, list]],
                   reference_years: tuple[int, ...]) -> dict[str, Any]:
    q = {w: _a_mean([a_midrank_fraction(sorted_by_year[y][w], values[w])
                     for y in reference_years]) for w in windows}
    a = {w: _a_direction(q[w], direction) for w in windows}
    score_t = max(a.values())
    return {"q": q, "a": a, "T": score_t,
            "winning_windows": [w for w in windows if a[w] == score_t]}
```

```
def _a_event_json(event: Mapping[str, Any]) -> dict[str, Any]:
    winners = event["winning_windows"]
    return {"q_by_window": {str(w): float(q) for w, q in event["q"].items()},
            "a_by_window": {str(w): float(a) for w, a in event["a"].items()},
            "T": float(event["T"]), "T_exact": _a_fraction_text(event["T"]),
            "winning_windows": winners, "winning_window_tie": len(winners) > 1}


def _a_score_factor(feature: Mapping[str, Any], prepared: Mapping[str, Any],
                    years: tuple[int, ...], min_endpoints: int, min_years: int
                    ) -> tuple[dict[str, Any], Fraction | None]:
    windows, endpoints = prepared["windows"], prepared["endpoints"]
    valid_years = tuple(sorted(endpoints))
    factor = {"id": prepared["id"], "direction": prepared["direction"],
              "window_hours": list(windows), "eligible_for_top5":
              ↪  bool(feature.get("eligible_for_top5", False)),
              "score_S": None, "score_S_exact": None, "rank_eligible": False,
              "support": {"requested_years": list(years), "valid_years": list(valid_years),
                          "n_valid_endpoints_by_year": {str(y): prepared["n_valid"][y] for y
                          ↪  in years},
                          "min_endpoints_per_year": min_endpoints, "min_years": min_years,
                          "endpoint_rule": "all applicable windows jointly valid"},
              "windows": {str(w): {"target_value": prepared["target"][w]} for w in windows},
              "configurations": [], "calibration_year_scores": {}, "constant_flags": {}}
    if not prepared["target_valid"]:
        factor.update(status="not_computable", reason="target missing at least one predefined
        ↪  window")
        return factor, None
    if len(valid_years) < min_years:
        factor.update(status="not_computable", reason=f"only {len(valid_years)} valid
        ↪  historical years; require {min_years}")
        return factor, None

    sorted_by_year = {y: {w: sorted(row[w] for row in endpoints[y].values())
                          for w in windows} for y in valid_years}
    background_constant = {
        w: len({row[w] for y in valid_years for row in endpoints[y].values()}) == 1
        for w in windows}
    all_values_constant = {
        w: background_constant[w] and prepared["target"][w] ==
        ↪  sorted_by_year[valid_years[0]][w][0]
        for w in windows}
    window_q, window_a, winning_counts = defaultdict(list), defaultdict(list),
    ↪  defaultdict(int)
    scores_by_h = defaultdict(list)
    all_configs_calibration_T_constant, all_configs_target_tied = True, True
    for h in valid_years:
        for reference in combinations([y for y in valid_years if y != h], 3):
            target_event = _a_score_event(prepared["target"], windows, prepared["direction"],
                                          sorted_by_year, reference)
            pseudo_events = [(offset, _a_score_event(vals, windows, prepared["direction"],
                                                     sorted_by_year, reference))
                             for offset, vals in sorted(endpoints[h].items())]
            pseudo_t = sorted(event["T"] for _, event in pseudo_events)
```

```
            config_s = a_midrank_fraction(pseudo_t, target_event["T"])
            scores_by_h[h].append(config_s)
            constant_t = len(set(pseudo_t)) == 1
            target_tied = constant_t and pseudo_t[0] == target_event["T"]
            all_configs_calibration_T_constant &= constant_t
            all_configs_target_tied &= target_tied
            for w in windows:
                window_q[w].append(target_event["q"][w])
                window_a[w].append(target_event["a"][w])
                winning_counts[w] += int(w in target_event["winning_windows"])
            factor["configurations"].append({
                "calibration_year": h, "reference_years": list(reference),
                "target": _a_event_json(target_event),
                "pseudo_events": [{"offset": offset, **_a_event_json(event)}
                                  for offset, event in pseudo_events],
                "n_calibration_endpoints": len(pseudo_events),
                "score_S": float(config_s), "score_S_exact": _a_fraction_text(config_s),
                "calibration_T_all_tied": constant_t,
                "target_tied_with_every_calibration_T": target_tied})
    by_year = {h: _a_mean(scores_by_h[h]) for h in valid_years}
    score_s = _a_mean(list(by_year.values()))
    factor["calibration_year_scores"] = {
        str(h): {"score_S": float(by_year[h]), "score_S_exact": _a_fraction_text(by_year[h]),
                 "configuration_count": len(scores_by_h[h])} for h in valid_years}
    n_config = len(factor["configurations"])
    for w in windows:
        factor["windows"][str(w)].update({
            "q_mean": float(_a_mean(window_q[w])), "q_min": float(min(window_q[w])),
            "q_max": float(max(window_q[w])), "a_mean": float(_a_mean(window_a[w])),
            "a_min": float(min(window_a[w])), "a_max": float(max(window_a[w])),
            "winning_configuration_count": winning_counts[w],
            "winning_configuration_fraction": winning_counts[w] / n_config,
            "background_constant": background_constant[w],
            "background_and_target_constant": all_values_constant[w]})
    factor.update(score_S=float(score_s), score_S_exact=_a_fraction_text(score_s),
                  configuration_count=n_config,
                  constant_flags={"all_windows_background_constant":
                  ↪  all(background_constant.values()),
                                  "all_windows_background_and_target_constant":
                                  ↪  all(all_values_constant.values()),
                                  "all_configurations_calibration_T_all_tied":
                                  ↪  all_configs_calibration_T_constant,
                                  "all_configurations_target_and_calibration_T_all_tied":
                                  ↪  all_configs_target_tied})
    if all_configs_target_tied:
        factor.update(status="computable_uninformative", reason="target and all calibration T
        ↪  values tie in every configuration")
    else:
        factor.update(status="computable", reason=None, rank_eligible=True)
    return factor, score_s


def _a_average_ranks(values: Sequence[Number]) -> list[float]:
    ordered = sorted(range(len(values)), key=values.__getitem__)
```

```
    ranks = [0.0] * len(values)
    start = 0
    while start < len(values):
        stop = start + 1
        while stop < len(values) and values[ordered[stop]] == values[ordered[start]]:
            stop += 1
        rank = (start + 1 + stop) / 2
        for i in ordered[start:stop]:
            ranks[i] = rank
        start = stop
    return ranks


def a_spearman_midrank(x: Sequence[Number], y: Sequence[Number]) -> float | None:

    if len(x) != len(y) or not x:
        raise ValueError("Spearman arrays must be nonempty and equal length")
    rx, ry = _a_average_ranks(x), _a_average_ranks(y)
    mean_x, mean_y = sum(rx) / len(rx), sum(ry) / len(ry)
    dx, dy = [r - mean_x for r in rx], [r - mean_y for r in ry]
    ssx, ssy = sum(v * v for v in dx), sum(v * v for v in dy)
    if not ssx or not ssy:
        return None
    return max(-1.0, min(1.0, sum(a * b for a, b in zip(dx, dy)) / sqrt(ssx * ssy)))


def _a_explicitly_same_information(a: Mapping[str, Any], b: Mapping[str, Any]) -> bool:
    family = a.get("physical_family")
    if not family or family != b.get("physical_family"):
        return False
    explicit_pair = (b["id"] in a.get("duplicate_with", []) or
                     a["id"] in b.get("duplicate_with", []))
    return explicit_pair or (a.get("duplicate_information") is True and
                             b.get("duplicate_information") is True)


def _a_redundancy_pairs(features: Mapping[str, Mapping[str, Any]], prepared: Mapping[str,
↪  Any],
                        factor_results: Mapping[str, Any], min_endpoints: int,
                        min_years: int, threshold: float) -> list[dict[str, Any]]:
    pairs = []
    for a_id, b_id in combinations(sorted(features), 2):
        a, b = features[a_id], features[b_id]
        if not _a_explicitly_same_information(a, b):
            continue
        wa, wb = int(a.get("representative_hours", 168)), int(b.get("representative_hours",
        ↪  168))
        result = {"factor_a": a_id, "factor_b": b_id, "physical_family":
        ↪  a.get("physical_family"),
                  "representative_hours_a": wa, "representative_hours_b": wb,
                  "threshold": threshold, "rho": None, "abs_rho": None, "merge_eligible":
                  ↪  False,
                  "n_paired_endpoints_by_year": {}, "common_valid_years": []}
        if wa not in prepared[a_id]["windows"] or wb not in prepared[b_id]["windows"]:
```

```
            result.update(status="uncertain", reason="predeclared representative window
            ↪  unavailable")
            pairs.append(result)
            continue
        if factor_results[a_id]["score_S"] is None or factor_results[b_id]["score_S"] is None:
            result.update(status="uncertain", reason="one or both factors not computable")
            pairs.append(result)
            continue
        x, y = [], []
        a_years, b_years = prepared[a_id]["endpoints"], prepared[b_id]["endpoints"]
        for year in sorted(set(a_years) & set(b_years)):
            offsets = sorted(set(a_years[year]) & set(b_years[year]))
            result["n_paired_endpoints_by_year"][str(year)] = len(offsets)
            if len(offsets) >= min_endpoints:
                result["common_valid_years"].append(year)
                x.extend(a_years[year][o][wa] for o in offsets)
                y.extend(b_years[year][o][wb] for o in offsets)
        result["paired_endpoints_used"] = len(x)
        if len(result["common_valid_years"]) < min_years:
            result.update(status="uncertain", reason="insufficient jointly valid historical
            ↪  years/endpoints")
        else:
            rho = a_spearman_midrank(x, y)
            result.update(rho=rho, abs_rho=None if rho is None else abs(rho))
            if rho is None:
                result.update(status="uncertain", reason="constant representative-window
                ↪  margin")
            else:
                result.update(status="evaluated", reason=None, merge_eligible=abs(rho) >=
                ↪  threshold)
        pairs.append(result)
    return pairs


def _a_rank_rows(ids: Iterable[str], scores: Mapping[str, Fraction | None],
                 factor_results: Mapping[str, Any]) -> list[dict[str, Any]]:
    ordered = sorted(ids, key=lambda factor_id: (-scores[factor_id], factor_id))
    rows, start = [], 0
    while start < len(ordered):
        stop = start + 1
        while stop < len(ordered) and scores[ordered[stop]] == scores[ordered[start]]:
            stop += 1
        for factor_id in ordered[start:stop]:
            rows.append({"id": factor_id, "score_S": float(scores[factor_id]),
                         "score_S_exact": _a_fraction_text(scores[factor_id]),
                         "rank_min": start + 1, "rank_max": stop,
                         "score_tie_size": stop - start,
                         "eligible_for_top5": factor_results[factor_id]["eligible_for_top5"]})
        start = stop
    return rows


def summarize_one(features: Sequence[Mapping[str, Any]], values_by_factor: Mapping[str, Any],
↪  *,
```

```
              years: Sequence[int] = A_DEFAULT_YEARS, min_endpoints: int = 25,
              min_years: int = 4, redundancy_threshold: float = .90,
              top_n: int = 5) -> dict[str, Any]:


years = tuple(sorted(int(y) for y in years))
if len(set(years)) != len(years):
    raise ValueError("historical years must be distinct")
if min_years < 4 or not 1 <= min_endpoints <= 31 or top_n < 1:
    raise ValueError("require min_years >= 4, 1 <= min_endpoints <= 31 and top_n >= 1")
if not 0 <= redundancy_threshold <= 1:
    raise ValueError("redundancy_threshold must be in [0,1]")
specs = {str(feature["id"]): feature for feature in features}
if len(specs) != len(features):
    raise ValueError("feature ids must be unique")
prepared = {factor_id: _a_prepare(spec, values_by_factor.get(factor_id, {}), years,
↪  min_endpoints)
            for factor_id, spec in specs.items()}
factors, scores = {}, {}
for factor_id, spec in specs.items():
    factors[factor_id], scores[factor_id] = _a_score_factor(
        spec, prepared[factor_id], years, min_endpoints, min_years)
ranking_all = _a_rank_rows([factor_id for factor_id in specs if
↪  factors[factor_id]["rank_eligible"]],
                           scores, factors)
pairs = _a_redundancy_pairs(specs, prepared, factors, min_endpoints, min_years,
↪  redundancy_threshold)
pair_lookup = {frozenset((pair["factor_a"], pair["factor_b"])): pair
               for pair in pairs if pair["merge_eligible"]}
retained, suppressed = [], []
for row in ranking_all:
    factor_id = row["id"]
    if not row["eligible_for_top5"]:
        continue
    representatives = [r for r in retained if frozenset((factor_id, r)) in pair_lookup]
    if representatives:
        rep = representatives[0]
        pair = pair_lookup[frozenset((factor_id, rep))]
        suppressed.append({"id": factor_id, "representative_id": rep,
                           "reason": "predeclared duplicate information and supported
                           ↪  representative-window Spearman threshold",
                           "score_tied_with_representative": scores[factor_id] ==
                           ↪  scores[rep],
                           "rho": pair["rho"], "common_valid_years":
                           ↪  pair["common_valid_years"]})
    else:
        retained.append(factor_id)
candidate_ranking = _a_rank_rows(retained, scores, factors)
cutoff = scores[retained[top_n - 1]] if len(retained) >= top_n else None
top_candidates = [row["id"] for row in candidate_ranking
                  if cutoff is None or scores[row["id"]] >= cutoff]
boundary = [row["id"] for row in candidate_ranking
            if row["rank_min"] <= top_n < row["rank_max"]]
```

```
    return {"method": "A_equal_year_cross_calibrated_window_max", "score_interpretation":
    ↪  "descriptive cross-year calibrated historical anomaly position; not a p-value or
    ↪  disaster probability",
            "settings": {"years": list(years), "min_endpoints": min_endpoints,
                         "min_years": min_years, "redundancy_threshold": redundancy_threshold,
                         "top_n": top_n, "tie_rule": "exact midrank on caller-supplied
                         ↪  source-precision values; exact rational ranks",
                         "redundancy_rule": "same explicitly duplicated physical information;
                         ↪  pooled endpoint Spearman on predeclared representative windows;
                         ↪  sufficient paired support in each of >=4 years"},
            "factors": factors, "ranking_all": ranking_all,
            "ranking_exploratory": [r for r in ranking_all if not r["eligible_for_top5"]],
            "redundancy": {"pairs": pairs, "suppressed": suppressed,
                           "representative_tie_rule": "equal score: lexicographic factor id;
                           ↪  score tie remains explicitly recorded"},
            "candidate_ranking": candidate_ranking, "top_candidates": top_candidates,
            "top_core_candidates": [r["id"] for r in candidate_ranking if r["rank_max"] <=
            ↪  top_n],
            "boundary_candidates": boundary,
            "top_cutoff_score_S": None if cutoff is None else float(cutoff),
            "top_candidate_count": len(top_candidates), "available_candidate_count":
            ↪  len(retained),
            "fewer_than_requested_candidates": len(retained) < top_n}


def summarize_selection_frequency(named_runs: Mapping[str, Mapping[str, Any]],
                                  features: Sequence[Mapping[str, Any]] | None = None
                                  ) -> dict[str, Any]:


    ids = sorted({str(f["id"]) for f in features} if features is not None else
                 {factor_id for run in named_runs.values() for factor_id in
                 ↪  run.get("factors", {})})
    summaries = {}
    for factor_id in ids:
        records, eligible = [], False
        for name, run in named_runs.items():
            factor = run.get("factors", {}).get(factor_id, {})
            eligible |= bool(factor.get("eligible_for_top5", False))
            computable = factor.get("score_S") is not None
            all_rank = next((r for r in run.get("ranking_all", []) if r["id"] == factor_id),
            ↪  None)
            candidate_rank = next((r for r in run.get("candidate_ranking", []) if r["id"] ==
            ↪  factor_id), None)
            records.append({"run": str(name), "computable": computable,
                            "omitted_year_was_valid": factor.get("omitted_year_was_valid"),
                            "no_op": bool(factor.get("no_op", False)),
                            "rankable": bool(factor.get("rank_eligible", False)),
                            "selected": factor_id in run.get("top_candidates", []),
                            "score_S": factor.get("score_S"),
                            "rank_min": None if candidate_rank is None else
                            ↪  candidate_rank["rank_min"],
                            "rank_max": None if candidate_rank is None else
                            ↪  candidate_rank["rank_max"],
```

```
                            "pre_redundancy_rank_min": None if all_rank is None else
                            ↪  all_rank["rank_min"],
                            "pre_redundancy_rank_max": None if all_rank is None else
                            ↪  all_rank["rank_max"],
                            "status": factor.get("status", "not_computable"),
                            "reason": factor.get("reason", "factor absent from this run")})
        n_computable = sum(r["computable"] for r in records)
        n_selected = sum(r["selected"] for r in records)
        perturbed = [r for r in records if not r["no_op"]]
        n_perturbed_computable = sum(r["computable"] for r in perturbed)
        n_perturbed_selected = sum(r["selected"] for r in perturbed)
        available_ranks = [r for r in records if r["rank_min"] is not None]
        all_ranks = [r for r in records if r["pre_redundancy_rank_min"] is not None]
        summaries[factor_id] = {
            "eligible_for_top5": eligible, "selected_count": n_selected,
            "computable_count": n_computable, "not_computable_count": len(records) -
            ↪  n_computable,
            "rankable_count": sum(r["rankable"] for r in records),
            "selection_fraction": n_selected / n_computable if eligible and n_computable else
            ↪  None,
            "no_op_count": len(records) - len(perturbed),
            "perturbed_selected_count": n_perturbed_selected,
            "perturbed_computable_count": n_perturbed_computable,
            "perturbed_not_computable_count": len(perturbed) - n_perturbed_computable,
            "perturbed_fraction": (n_perturbed_selected / n_perturbed_computable
                                   if eligible and n_perturbed_computable else None),
            "rank_min": min((r["rank_min"] for r in available_ranks), default=None),
            "rank_max": max((r["rank_max"] for r in available_ranks), default=None),
            "pre_redundancy_rank_min": min((r["pre_redundancy_rank_min"] for r in all_ranks),
            ↪  default=None),
            "pre_redundancy_rank_max": max((r["pre_redundancy_rank_max"] for r in all_ranks),
            ↪  default=None),
            "run_records": records}
    return summaries


def leave_one_year_out(features: Sequence[Mapping[str, Any]], values_by_factor: Mapping[str,
↪  Any], *,
                       years: Sequence[int] = A_DEFAULT_YEARS, min_endpoints: int = 25,
                       min_years: int = 4, redundancy_threshold: float = .90,
                       top_n: int = 5) -> dict[str, Any]:

    years = tuple(int(y) for y in years)
    runs = {str(omitted): summarize_one(
        features, values_by_factor, years=[y for y in years if y != omitted],
        min_endpoints=min_endpoints, min_years=min_years,
        redundancy_threshold=redundancy_threshold, top_n=top_n) for omitted in years}
    baseline_valid_years = {
        str(spec["id"]): set(_a_prepare(spec, values_by_factor.get(str(spec["id"]), {}),
                                        years, min_endpoints)["endpoints"])
        for spec in features}
    for omitted in years:
        run = runs[str(omitted)]
        run["omitted_year"] = omitted
```

```
        for factor_id, factor in run["factors"].items():
            was_valid = omitted in baseline_valid_years[factor_id]
            factor["omitted_year_was_valid"] = was_valid
            factor["no_op"] = not was_valid
    return {"runs": runs, "selection_frequency": summarize_selection_frequency(runs,
    ↪  features),
            "interpretation": "sensitivity to omitted historical years; public-fold and
            ↪  actually-perturbed denominators are distinct; deleting a previously invalid
            ↪  year is a factor-specific no-op; unavailable perturbed folds stay NA; not
            ↪  independent-event sampling or disaster prediction coverage"}


from collections import Counter
from datetime import date, timedelta
from typing import Any

import numpy as np
from scipy.stats import kendalltau, rankdata


SEED = 20261024
WINDOWS_H = (24, 72, 168, 360, 720)
CLIM_YEARS = (2021, 2022, 2023, 2024, 2025)
N_PERM = 2000
N_BOOT = 1000
N_PERM_BOOT = 500
MIN_REFERENCE = 100


ORIGINAL_FACTORS = (
    ("precip_sum", " 累计降水", 3),
    ("precip_max", " 最大小时雨强", 3),
    ("rain_hours", " 有雨小时数", 2),
    ("api", " 前期降水指数 API", 3),
    ("t2_mean", "2 m 气温", 3),
    ("t2_max", "2 m 日最高气温", 2),
    ("pdd", " 站点正积温 PDD", 3),
    ("warm_rate", " 升温速率", 3),
    ("rn_mean", " 净辐射 Rn", 3),
    ("rsd_mean", " 入射短波 Rsd", 2),
    ("rld_mean", " 入射长波 Rld", 2),
    ("vwc01", "0.1 m 体积含水量", 3),
    ("vwc02", "0.2 m 体积含水量", 2),
    ("soilt01", "0.1 m 土温", 2),
    ("g01", "0.1 m 土壤热通量 G", 1),
    ("dthdz_night", " 夜间位温梯度", 1),
    ("rig_day", " 日间梯度理查森数 Ri_g 中位数", 1),
    ("zeta_med", " 稳定度参数 ζ 中位数", 1),
    ("h_mean", " 感热 H", 2),
    ("le_mean", " 潜热 LE", 2),
    ("p_mean", " 气压", 1),
    ("rh2_mean", "2 m 相对湿度", 2),
    ("ws10_mean", "10 m 风速", 1),
    ("ustar_mean", " 摩擦速度 u*", 1),
```

```
)
MECHANISM = {key: mechanism for key, _, mechanism in ORIGINAL_FACTORS}
ORIGINAL_NAMES = {key: name for key, name, _ in ORIGINAL_FACTORS}


def _number(x: Any) -> float | int:
    if isinstance(x, (int, np.integer)):
        return int(x)
    try:
        value = float(x)
        return value if np.isfinite(value) else np.nan
    except (TypeError, ValueError):
        return np.nan


def _finite_values(values: np.ndarray) -> np.ndarray:

    values = np.asarray(values)
    if values.dtype != object:
        return np.isfinite(values)
    return np.fromiter((isinstance(v, (int, np.integer)) or
                        (isinstance(v, (float, np.floating)) and np.isfinite(v))
                       for v in values.flat), dtype=bool,
                       ↪  count=values.size).reshape(values.shape)


def _window_value(mapping: dict, hours: int) -> float | int:
    labels = (hours, str(hours), f"{hours}h", f"{hours // 24}d")
    return next((_number(mapping[k]) for k in labels if k in mapping), np.nan)


def _features(features: list[dict] | dict) -> list[dict]:
    if isinstance(features, dict):
        if isinstance(features.get("features"), list):
            source = features["features"]
        elif isinstance(features.get("factors"), list):
            source = features["factors"]
        else:
            source = [dict(v, factor_id=k) for k, v in features.items()]
    else:
        source = features
    out = []
    for feature in source:
        key = feature.get("factor_id", feature.get("id", feature.get("key")))
        if key not in MECHANISM:

            continue
        raw_windows = feature.get("windows_h", feature.get("window_hours",
        ↪  feature.get("windows_hours", feature.get("windows", WINDOWS_H))))
        raw_windows = [] if raw_windows is None else raw_windows
        if isinstance(raw_windows, dict):
            raw_windows = list(raw_windows.values())
        windows = []
        for value in raw_windows:
```

```
            if isinstance(value, str) and value.endswith("d"):
                value = float(value[:-1]) * 24
            elif isinstance(value, str) and value.endswith("h"):
                value = value[:-1]
            hours = int(value)
            if hours not in WINDOWS_H:
                raise ValueError(f"Unknown original 参考方法 window: {key}/{hours}")
            if hours not in windows:
                windows.append(hours)
        out.append({**feature, "factor_id": key, "windows_h": windows})
    if len({f["factor_id"] for f in out}) != len(out):
        raise ValueError('Repeated 参考方法 factor IDs')
    return out


def bh_qvalues(p: np.ndarray) -> np.ndarray:

    p = np.asarray(p, float)
    out = np.full(p.shape, np.nan)
    finite = np.isfinite(p)
    pv = p[finite]
    if not len(pv):
        return out
    order = np.argsort(pv, kind="stable")
    q = np.minimum.accumulate((pv[order] * len(pv) / np.arange(1, len(pv) + 1))[::-1])[::-1]
    values = np.empty(len(pv))
    values[order] = np.minimum(1.0, q)
    out[finite] = values
    return out


def entropy_topsis(matrix: np.ndarray) -> tuple[np.ndarray, np.ndarray, str]:

    matrix = np.asarray(matrix, float)
    n, columns = matrix.shape
    if n < 2:
        return np.full(n, np.nan), np.full(columns, np.nan), "fewer_than_two_rows"
    ranges = np.ptp(matrix, axis=0)
    scaled = np.divide(matrix - matrix.min(axis=0), ranges,
                       out=np.zeros_like(matrix), where=ranges > 0)
    probability = (scaled + 1e-12) / (scaled + 1e-12).sum(axis=0)
    entropy = -(probability * np.log(probability)).sum(axis=0) / np.log(n)
    diversity = np.maximum(0.0, 1.0 - entropy)
    diversity[ranges == 0] = 0.0
    if not diversity.sum() > 1e-14:
        return np.full(n, np.nan), np.full(columns, np.nan), "all_columns_constant"
    weights = diversity / diversity.sum()
    weighted = scaled * weights
    positive = np.linalg.norm(weighted - weighted.max(axis=0), axis=1)
    negative = np.linalg.norm(weighted - weighted.min(axis=0), axis=1)
    denominator = positive + negative
    scores = np.divide(negative, denominator, out=np.full(n, np.nan), where=denominator > 0)
    return scores, weights, "ok"
```

```
def tied_ranks(scores: np.ndarray, k: int = 5) -> tuple[np.ndarray, np.ndarray, np.ndarray,
↪  np.ndarray]:

    scores = np.asarray(scores, float)
    rank = np.full(scores.shape, np.nan)
    low = rank.copy()
    high = rank.copy()
    inclusion = rank.copy()
    good = np.isfinite(scores)
    if good.any():
        vals = -scores[good]
        rank[good] = rankdata(vals, method="average")
        low[good] = rankdata(vals, method="min")
        high[good] = rankdata(vals, method="max")
        size = high[good] - low[good] + 1
        inclusion[good] = np.clip((k - low[good] + 1) / size, 0, 1)
    return rank, low, high, inclusion


def _rank_all(anomaly: np.ndarray, p: np.ndarray, mechanism: np.ndarray):
    good = np.isfinite(anomaly) & np.isfinite(p) & (p > 0) & np.isfinite(mechanism)
    score = np.full(len(anomaly), np.nan)
    weights = np.full(3, np.nan)
    if good.any():
        score[good], weights, status = entropy_topsis(np.column_stack((anomaly[good],
        ↪  -np.log10(p[good]), mechanism[good])))
    else:
        status = "no_computable_rows"
    return score, weights, status


def _max_integer_ratio(numerator: np.ndarray, denominator: np.ndarray) -> tuple[np.ndarray,
↪  np.ndarray]:


    numerator, denominator = np.asarray(numerator, np.int64), np.asarray(denominator,
    ↪  np.int64)
    if numerator.shape != denominator.shape or not numerator.shape[-1]:
        raise ValueError("Nonempty equal-shaped ratio arrays are required")
    best_numerator, best_denominator = numerator[..., 0].copy(), denominator[..., 0].copy()
    for wi in range(1, numerator.shape[-1]):
        better = numerator[..., wi] * best_denominator > best_numerator * denominator[..., wi]
        best_numerator = np.where(better, numerator[..., wi], best_numerator)
        best_denominator = np.where(better, denominator[..., wi], best_denominator)
    return best_numerator, best_denominator


class _FactorCache:


    def __init__(self, key: str, windows: list[int], values: dict):
        self.key, self.windows = key, windows
        self.years = CLIM_YEARS
```

```
    seasonal = values.get("seasonal_years", {})
    if not seasonal:
        raise ValueError('seasonal_years cache is required for original 参考方法
        ↪  pseudo-events')
    anchor = date(2025, 5, 31)
    self.days = [(anchor + timedelta(days=i)).strftime("%m-%d") for i in range(138)]
    self.day_index = {day: i for i, day in enumerate(self.days)}
    season_start = self.day_index["06-15"]
    season_end = self.day_index["09-30"] + 1
    self.candidates = [(yi, di) for yi in range(len(self.years)) for di in
    ↪  range(season_start, season_end)]
    self.candidate_year = np.array([yi for yi, _ in self.candidates], int)
    self.candidate_day = np.array([di for _, di in self.candidates], int)


    self.values = np.full((len(self.years), len(self.days), len(windows)), np.nan,
    ↪  dtype=object)
    for yi, year in enumerate(self.years):
        year_values = seasonal.get(year, seasonal.get(str(year), {}))
        for di, day in enumerate(self.days):
            row = year_values.get(day, {})
            for wi, hours in enumerate(windows):
                self.values[yi, di, wi] = _window_value(row, hours)
    self.target = np.array([_window_value(values.get("target", {}), h) for h in windows],
    ↪  dtype=object)
    center = self.day_index["08-26"]
    self.target_ref = self.values[:, center - 15:center + 16, :]
    valid = _finite_values(self.target_ref)
    self.target_n = valid.sum(axis=1)
    self.target_less = ((self.target_ref < self.target[None, None, :]) &
    ↪  valid).sum(axis=1)
    self.target_equal = ((self.target_ref == self.target[None, None, :]) &
    ↪  valid).sum(axis=1)
    shape = (len(self.candidates), len(self.years), len(windows))
    self.pseudo_n = np.zeros(shape, np.int32)
    self.pseudo_less = np.zeros(shape, np.int32)
    self.pseudo_equal = np.zeros(shape, np.int32)
    self.pseudo_values = self.values[self.candidate_year, self.candidate_day, :]
    for ci, (yi, di) in enumerate(self.candidates):
        reference = self.values[:, di - 15:di + 16, :]
        finite = _finite_values(reference)
        target = self.pseudo_values[ci]
        self.pseudo_n[ci] = finite.sum(axis=1)
        self.pseudo_less[ci] = ((reference < target[None, None, :]) & finite).sum(axis=1)
        self.pseudo_equal[ci] = ((reference == target[None, None, :]) &
        ↪  finite).sum(axis=1)
    self.memo: dict[tuple[int, ...], dict] = {}
    finite_refs = _finite_values(self.target_ref)
    self.constant = bool(len(windows) and finite_refs.any() and
    ↪  _finite_values(self.target).all() and
                         np.all((self.target_ref == self.target[None, None, :]) |
                         ↪  ~finite_refs))

def distribution(self, weights: tuple[int, ...]) -> dict:
```

```
if weights in self.memo:
    return self.memo[weights]
weight = np.asarray(weights, int)
target_n = (self.target_n * weight[:, None]).sum(axis=0)
target_twice_counts = ((2 * self.target_less + self.target_equal) * weight[:,
↪  None]).sum(axis=0)
target_q = np.divide(target_twice_counts, 2 * target_n,
↪  out=np.full(len(self.windows), np.nan), where=target_n >= MIN_REFERENCE)
target_q[~_finite_values(self.target)] = np.nan


target_anomaly_numerator = np.abs(target_twice_counts - target_n)
window_a = np.divide(target_anomaly_numerator, target_n,
                     out=np.full(len(self.windows), np.nan), where=target_n >=
                     ↪  MIN_REFERENCE)
window_a[~_finite_values(self.target)] = np.nan

target_complete = len(self.windows) and np.isfinite(window_a).all() and not
↪  self.constant
if target_complete:
    target_max_num, target_max_den = _max_integer_ratio(target_anomaly_numerator,
    ↪  target_n)
    target_max_num, target_max_den = int(target_max_num), int(target_max_den)
    anomaly = target_max_num / target_max_den
else:
    target_max_num = target_max_den = None
    anomaly = np.nan
selected_weights = np.broadcast_to(weight, (len(self.candidates), len(weight))).copy()
selected_weights[np.arange(len(self.candidates)), self.candidate_year] = 0
pseudo_n = np.einsum("cyw,cy->cw", self.pseudo_n, selected_weights, optimize=False)
pseudo_twice_counts = np.einsum("cyw,cy->cw", 2 * self.pseudo_less +
↪  self.pseudo_equal,
                                selected_weights, optimize=False)
pseudo_anomaly_numerator = np.abs(pseudo_twice_counts - pseudo_n)
pseudo_a = np.divide(pseudo_anomaly_numerator, pseudo_n,
                     out=np.full(pseudo_n.shape, np.nan), where=pseudo_n >=
                     ↪  MIN_REFERENCE)
full_support = _finite_values(self.pseudo_values).all(axis=1) &
↪  np.isfinite(pseudo_a).all(axis=1)
present_year = weight[self.candidate_year] > 0
valid = full_support & present_year
pseudo_max_num, pseudo_max_den = _max_integer_ratio(pseudo_anomaly_numerator[valid],
↪  pseudo_n[valid])
pseudo_stat = pseudo_max_num / pseudo_max_den
result = {"A": anomaly, "A_num": target_max_num, "A_den": target_max_den,
          "q": target_q, "q_twice_count": target_twice_counts,
          "a": window_a, "a_num": target_anomaly_numerator, "nref": target_n,
          "pseudo": pseudo_stat, "pseudo_num": pseudo_max_num, "pseudo_den":
          ↪  pseudo_max_den,
          "candidate_indices": np.flatnonzero(valid),
          "n_pool": int(valid.sum()), "n_pool_total": int(present_year.sum())}
self.memo[weights] = result
return result
```

```
def _draw_p(distribution: dict, draws: int, rng: np.random.Generator) -> tuple[float, int,
↪  int]:
    if distribution["A_num"] is None or not distribution["n_pool"]:
        return np.nan, 0, 0
    picks = rng.integers(0, distribution["n_pool"], size=draws)

    greater_equal = int(np.count_nonzero(distribution["pseudo_num"][picks] *
    ↪  distribution["A_den"] >=
                                         distribution["A_num"] *
                                         ↪  distribution["pseudo_den"][picks]))
    return (1 + greater_equal) / (1 + draws), draws, greater_equal


def _comparison(main: np.ndarray, other: np.ndarray, ids: list[str]) -> dict:
    common = np.isfinite(main) & np.isfinite(other)
    main_r, main_lo, _, main_top = tied_ranks(np.where(common, main, np.nan))
    other_r, other_lo, _, other_top = tied_ranks(np.where(common, other, np.nan))
    tau = kendalltau(main_r[common], other_r[common]).statistic if common.sum() >= 2 else
    ↪  np.nan
    return {"n_common_factors": int(common.sum()), "common_factor_ids": [ids[i] for i in
    ↪  np.flatnonzero(common)],
            "kendall_tau_b": tau,
            "top5_fractional_overlap": float(np.nansum(np.minimum(main_top, other_top))) if
            ↪  common.any() else None,
            "top5_possible_overlap": int(np.sum((main_lo <= 5) & (other_lo <= 5))) if
            ↪  common.any() else None,
            "other_top5_possible": [ids[i] for i in np.flatnonzero(other_lo <= 5)],
            "tie_rule": "average ranks; fractional allocation at Top5 boundary; tau_b"}


def _json_safe(value: Any) -> Any:
    if isinstance(value, dict):
        return {str(k): _json_safe(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
        return [_json_safe(v) for v in value]
    if isinstance(value, np.ndarray):
        return _json_safe(value.tolist())
    if isinstance(value, (np.integer,)):
        return int(value)
    if isinstance(value, (float, np.floating)):
        return float(value) if np.isfinite(value) else None
    if isinstance(value, np.bool_):
        return bool(value)
    return value


def run_ref(features: list[dict] | dict, values_by_factor: dict, seed: int = SEED, *,
            label: str = "B-aligned-corrected", n_perm: int = N_PERM,
            n_boot: int = N_BOOT, n_perm_boot: int = N_PERM_BOOT) -> dict:


    if min(n_perm, n_boot, n_perm_boot) < 1:
        raise ValueError("Draw and bootstrap counts must be positive")
```

```
selected = _features(features)
ids = [f["factor_id"] for f in selected]
mechanism = np.array([MECHANISM[key] for key in ids], float)
size = len(ids)
rng = np.random.default_rng(seed)
caches: list[_FactorCache | None] = []
unavailable = {}
for feature in selected:
    key = feature["factor_id"]
    if key not in values_by_factor or not feature["windows_h"]:
        caches.append(None)
        unavailable[key] = "no_window_cache_or_no_applicable_windows"
        continue
    try:
        caches.append(_FactorCache(key, feature["windows_h"], values_by_factor[key]))
    except ValueError as error:
        caches.append(None)
        unavailable[key] = str(error)
full_weights = (1,) * len(CLIM_YEARS)
main = [cache.distribution(full_weights) if cache else None for cache in caches]
anomaly = np.array([d["A"] if d else np.nan for d in main])
p = np.full(size, np.nan)
draws_run = np.zeros(size, int)
exceed = np.zeros(size, int)
for j, distribution in enumerate(main):
    if distribution:
        p[j], draws_run[j], exceed[j] = _draw_p(distribution, n_perm, rng)
score, entropy_weight, topsis_status = _rank_all(anomaly, p, mechanism)
ranks, rank_min, rank_max, top_weight = tied_ranks(score)
bh = bh_qvalues(p)
borda_score = np.full(size, np.nan)
good = np.isfinite(score)
if good.any():
    borda_score[good] = -(rankdata(-anomaly[good], method="average") +
                          rankdata(p[good], method="average") +
                          rankdata(-mechanism[good], method="average"))
borda_rank, _, _, _ = tied_ranks(borda_score)
zscore = np.full(size, np.nan)
for j, cache in enumerate(caches):
    if cache and 168 in cache.windows:
        wi = cache.windows.index(168)
        reference = cache.target_ref[:, :, wi].ravel()
        reference = reference[_finite_values(reference)]
        if len(reference) >= MIN_REFERENCE and _finite_values(cache.target)[wi]:
            center = reference[0]
            differences = [v - center for v in reference]
            target_difference = cache.target[wi] - center
            common_scale = max([abs(v) for v in differences] + [abs(target_difference)])
            if not common_scale > 0:
                continue


            centered = np.array([v / common_scale for v in differences], float)
            target_centered = target_difference / common_scale
```

```
            deviation = centered.std(ddof=1)
            if deviation > 0:
                zscore[j] = abs(target_centered - centered.mean()) / deviation
zrank, _, _, _ = tied_ranks(zscore)

boot_rng = np.random.default_rng(seed + 1)
year_draws = boot_rng.choice(len(CLIM_YEARS), size=(n_boot, len(CLIM_YEARS)),
↪  replace=True)
boot_weights = np.array([np.bincount(draw, minlength=len(CLIM_YEARS)) for draw in
↪  year_draws])
boot_ranks = np.full((n_boot, size), np.nan)
boot_scores = boot_ranks.copy()
boot_p = boot_ranks.copy()
boot_top = boot_ranks.copy()
boot_draws = np.zeros((n_boot, size), int)
boot_pools = np.zeros((n_boot, size), int)
boot_status = []
for b, weights in enumerate(boot_weights):
    weights_tuple = tuple(int(w) for w in weights)
    ab = np.full(size, np.nan)
    pb = np.full(size, np.nan)
    for j, cache in enumerate(caches):
        if cache:
            distribution = cache.distribution(weights_tuple)
            ab[j] = distribution["A"]
            boot_pools[b, j] = distribution["n_pool"]
            pb[j], boot_draws[b, j], _ = _draw_p(distribution, n_perm_boot, boot_rng)
    sb, _, status = _rank_all(ab, pb, mechanism)
    boot_scores[b], boot_p[b] = sb, pb
    boot_ranks[b], _, _, boot_top[b] = tied_ranks(sb)
    boot_status.append(status)
rows, window_rows, bootstrap_rows = [], [], []
for j, feature in enumerate(selected):
    key = ids[j]
    cache, distribution = caches[j], main[j]
    if cache is None:
        status = unavailable[key]
    elif cache.constant:
        status = "constant_target_and_reference_windows"
    elif not np.isfinite(anomaly[j]):
        status = "target_missing_or_some_window_reference_n_below_100"
    elif not np.isfinite(p[j]):
        status = "no_complete_pseudo_event_reference_pool"
    elif not np.isfinite(score[j]):
        status = f"topsis_{topsis_status}"
    else:
        status = "computed"
    winning = []
    if distribution and np.isfinite(anomaly[j]):
        winning = [h for wi, h in enumerate(cache.windows)
                   if int(distribution["a_num"][wi]) * distribution["A_den"] ==
                   distribution["A_num"] * int(distribution["nref"][wi])]
    main_row = {"factor_id": key, "factor": feature.get("name", ORIGINAL_NAMES[key]),
    ↪  "method": label,
```

```
            "status": status, "windows_h": feature["windows_h"], "A": anomaly[j],
            "A_exact_numerator": distribution["A_num"] if distribution else None,
            "A_exact_denominator": distribution["A_den"] if distribution else None,
            "value_encoding": "rank_ticks" if "rank_scale" in feature else
            ↪  "physical_value",
            "rank_scale": feature.get("rank_scale"),
            "best_windows_h": winning, "perm_p": p[j], "bh_q": bh[j],
            "bh_family_n": int(np.isfinite(p).sum()), "mech_score": int(mechanism[j]),
            "topsis_score": score[j], "rank": ranks[j], "rank_min": rank_min[j],
            ↪  "rank_max": rank_max[j],
            "top5_fractional_allocation": top_weight[j], "borda_rank": borda_rank[j],
            "baseline_7d_absolute_z": zscore[j], "baseline_7d_rank": zrank[j],
            "pseudo_pool_valid": distribution["n_pool"] if distribution else 0,
            "pseudo_pool_total": distribution["n_pool_total"] if distribution else 0,
            "perm_draws_requested": n_perm, "perm_draws_executed": draws_run[j],
            "perm_greater_equal": exceed[j] if draws_run[j] else None}
finite = np.isfinite(boot_ranks[:, j])
count = int(finite.sum())
bounds = np.percentile(boot_ranks[finite, j], [2.5, 97.5]) if count else [np.nan,
↪  np.nan]
bootstrap_row = {"factor_id": key, "bootstrap_requested": n_boot,
                 "bootstrap_rank_computable": count, "bootstrap_rank_not_computable":
                 ↪  n_boot - count,
                 "rank_ci_low": bounds[0], "rank_ci_high": bounds[1],
                 "top5_fractional_count": float(np.nansum(boot_top[:, j])) if count
                 ↪  else None,
                 "top5_frequency_computable": float(np.mean(boot_top[finite, j])) if
                 ↪  count else None,
                 "top5_definite_count": int(np.sum(boot_top[finite, j] == 1)),
                 "top5_possible_count": int(np.sum(boot_top[finite, j] > 0)),
                 "pseudo_draws_executed": int(boot_draws[:, j].sum()),
                 "pseudo_draws_per_computable_distribution": n_perm_boot,
                 "pseudo_distributions_computable": int(np.sum(boot_draws[:, j] ==
                 ↪  n_perm_boot))}
main_row.update(bootstrap_row)
rows.append(main_row)
bootstrap_rows.append(bootstrap_row)
if cache and distribution:
    for wi, hours in enumerate(cache.windows):
        q = distribution["q"][wi]
        window_rows.append({"factor_id": key, "hours": hours, "event_value":
        ↪  cache.target[wi],
                            "value_encoding": "rank_ticks" if "rank_scale" in feature
                            ↪  else "physical_value",
                            "rank_scale": feature.get("rank_scale"),
                            "pooled_midrank_quantile": q, "two_sided_anomaly":
                            ↪  distribution["a"][wi],
                            "quantile_twice_midrank_count":
                            ↪  distribution["q_twice_count"][wi],
                            "anomaly_exact_numerator": distribution["a_num"][wi],
                            "anomaly_exact_denominator": distribution["nref"][wi],
                            "direction": "high" if q > .5 else "low" if q < .5 else
                            ↪  "middle" if q == .5 else None,
                            "pooled_reference_n": distribution["nref"][wi],
```

```
                                "reference_n_by_year": dict(zip(CLIM_YEARS,
                                ↪  cache.target_n[:, wi].tolist())),
                                "is_best_window": hours in winning})
comparisons = {"borda": _comparison(score, borda_score, ids),
               "baseline_7d_absolute_z": _comparison(score, zscore, ids)}
rows.sort(key=lambda row: (row["rank"] if np.isfinite(row["rank"]) else np.inf,
↪  ids.index(row["factor_id"])))
result = {"factor_rows": rows, "window_rows": window_rows, "bootstrap_rows":
↪  bootstrap_rows,
          "comparisons": comparisons,
          "bootstrap_detail": {"factor_ids": ids, "year_draws":
          ↪  np.array(CLIM_YEARS)[year_draws],
                               "year_multiplicity": boot_weights, "ranks": boot_ranks,
                               ↪  "scores": boot_scores,
                               "p_values": boot_p, "top5_fractional_allocation": boot_top,
                               "pseudo_draws_executed": boot_draws, "pseudo_pool_sizes":
                               ↪  boot_pools},
          "metadata": {"label": label, "source_version": "MODELING_REPORT §4.4–4.5 /
          ↪  final problem1.py",
                       "seed": seed, "climate_years": CLIM_YEARS, "n_original_factors":
                       ↪  24,
                       "n_input_original_factors": size, "n_computable_topsis_factors":
                       ↪  int(np.isfinite(score).sum()),
                       "min_pooled_reference": MIN_REFERENCE, "n_perm_requested": n_perm,
                       "n_boot_year_resamples_executed": n_boot, "n_perm_boot_requested":
                       ↪  n_perm_boot,
                       "formal_counts": (n_perm, n_boot, n_perm_boot) == (N_PERM, N_BOOT,
                       ↪  N_PERM_BOOT),
                       "execution_class": "formal_counts" if (n_perm, n_boot,
                       ↪  n_perm_boot) == (N_PERM, N_BOOT, N_PERM_BOOT) else
                       ↪  "reduced_count_selfcheck",
                       "entropy_weights_A_neglogp_mechanism": entropy_weight,
                       ↪  "topsis_status": topsis_status,
                       "bootstrap_topsis_status_counts": dict(Counter(boot_status)),
                       "n_boot_year_multiplicity_patterns": len({tuple(w) for w in
                       ↪  boot_weights}),
                       "pseudo_endpoint_season": "06-15..09-30, inclusive; source year
                       ↪  excluded",
                       "window_reference_rule": "same month-day ±15 days; same full
                       ↪  applicable W for target and pseudo",
                       "rank_value_comparison": "exact object-integer comparisons when
                       ↪  rank ticks are supplied; baseline centers integers before
                       ↪  float standardization",
                       "T_and_p_comparison": "exact integer anomaly numerator/reference
                       ↪  denominator; multiwindow max and null>=target use integer
                       ↪  cross-products; floats only enter display and TOPSIS",
                       "weight_conflict": "Source §4.5 fits entropy weights to the event
                       ↪  A/p matrix, contradicting assumption 3. Computational §4.5 is
                       ↪  implemented and this is not labeled climate-only weighting.",
                       "p_interpretation": "Monte Carlo pseudo-event comparison under a
                       ↪  seasonal sampling scheme; exchangeability is not established;
                       ↪  not disaster probability or predictive accuracy",
                       "corrections": ["D0 cleaning and caller-defined time/support
                       ↪  instead of original QC/interpolation",
```

```
                                        "calendar month-day matching instead of leap-year
                                        ↪  dayofyear distance",
                                        "full fixed W required; no subset-window nanmax",
                                        "eligible complete pseudo pools, exactly requested
                                        ↪  draws, unavailable p stays NA",
                                        "constant factors/degenerate TOPSIS stay NA",
                                        "average ranks and explicit Top5 tie fractions
                                        ↪  instead of ordinal tie breaks",
                                        "BH family is the reported computable factor
                                        ↪  family, not fictitious 24 valid tests"],
                            "independent_review_status": "pending; execution is not scientific
                            ↪  approval"}}
    return _json_safe(result)


Q1_FROZEN_DICTIONARY = json.loads((Path(__file__).resolve().parent / 'data' /
↪  'features.json').read_text(encoding='utf-8'))

def q1_sha(path):
    h = hashlib.sha256()
    with Path(path).open('rb') as f:
        for block in iter(lambda: f.read(1024 * 1024), b''):
            h.update(block)
    return h.hexdigest()

def q1_fp(path):
    p = Path(path).resolve()
    try:
        identity=str(p.relative_to(Path.cwd().resolve()))
    except ValueError:
        identity='external_input/' + p.name
    return {'path': identity, 'bytes': p.stat().st_size, 'sha256': q1_sha(p)}

def q1_dump(path, value):
    Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) +
    ↪  '\n', encoding='utf-8')

def q1_csv(path, rows):
    if not rows:
        Path(path).write_text('status\nno_rows\n', encoding='utf-8')
        return
    with Path(path).open('w', encoding='utf-8-sig', newline='') as f:
        fields=list(dict.fromkeys(k for row in rows for k in row))
        w = csv.DictWriter(f, fieldnames=fields); w.writeheader()
        for row in rows:
            w.writerow({k: json.dumps(v, ensure_ascii=False) if isinstance(v, (dict, list))
            ↪  else v for k, v in row.items()})

def q1_hour(value):
    t = dt.datetime.fromisoformat(value)
    if t.tzinfo is None or t.utcoffset() != dt.timedelta(hours=8) or t.minute or t.second or
    ↪  t.microsecond:
        raise ValueError('D0 标签须为 UTC+8 整点且唯一')
    return int((t - Q1_ORIGIN).total_seconds() // 3600)
```

```
def q1_cache(arrays, features, endpoint, cutoff_hour, coverage, ledger):
    values = {}; source_features = [f for f in features if f.get('computable')]
    first = dt.date(2021, 5, 31); last = dt.date(2021, 10, 15)
    monthdays = [(first + dt.timedelta(days=d)).strftime('%m-%d') for d in
    ↪  range((last-first).days+1)]
    for f in source_features:
        g = f['id']; data = {'target': {}, 'years': {}, 'seasonal_years': {}}
        end = q1_hour(f'2026-08-26T{cutoff_hour:02d}:00:00+08:00')
        for h in f['window_hours']:
            value, meta = q1_window(arrays, f, end, h, endpoint, coverage)
            data['target'][str(h)] = value
            ledger.append(dict(factor_id=g,endpoint=endpoint,cutoff_hour=cutoff_hour,coverage
            ↪  _threshold=coverage,year=2026,monthday='08-26',window_hours=h,value_exact_raw
            ↪  _grid_fraction=str(value) if value is not None
            ↪  None,physical_value=float(value/f['source_grid_scale']) if value is not None
            ↪  else None,**meta))
        for year in A_DEFAULT_YEARS:
            seasonal = {}
            for md in monthdays:
                end = q1_hour(f'{year}-{md}T{cutoff_hour:02d}:00:00+08:00'); row = {}
                for h in f['window_hours']:
                    value, meta = q1_window(arrays, f, end, h, endpoint, coverage);
                    ↪  row[str(h)] = value
                    ledger.append(dict(factor_id=g,endpoint=endpoint,cutoff_hour=cutoff_hour,
                    ↪  coverage_threshold=coverage,year=year,monthday=md,window_hours=h,valu
                    ↪  e_exact_raw_grid_fraction=str(value) if value is not None else
                    ↪  None,physical_value=float(value/f['source_grid_scale']) if value is
                    ↪  not None else None,**meta))
                seasonal[md] = row
            data['seasonal_years'][str(year)] = seasonal
            data['years'][str(year)] = {str(offset):
            ↪  seasonal[(dt.date(year,8,26)+dt.timedelta(days=offset)).strftime('%m-%d')]
            ↪  for offset in range(-15,16)}
        values[g] = data
    return values

def q1_encode_exact_cache(cache, features):

    for feature in features:
        g=feature['id']
        if g not in Q1_COLUMNS:continue
        rows=[]
        for branch in cache.values():
            data=branch[g];rows.append(data['target'])
            rows.extend(r for year in data['seasonal_years'].values() for r in year.values())
        denominator=1
        for row in rows:
            for value in row.values():
                if value is not None:denominator=math.lcm(denominator,value.denominator)
        for row in rows:
            for h,value in row.items():
                if value is not None:
                    encoded=value*denominator
```

```
                    assert encoded.denominator==1
                    row[h]=encoded.numerator
        feature['rank_scale']=feature['source_grid_scale']*denominator
        feature['exact_common_denominator']=denominator

def q1_trim_a(result):
    r = copy.deepcopy(result)
    for f in r.get('factors', {}).values():
        f.pop('configurations', None)
    return r

def q1_common_support(features, values, primary):
    ids = [g for g,f in primary['factors'].items() if f.get('score_S') is not None]
    out = copy.deepcopy(values)
    counts = {}
    for year in A_DEFAULT_YEARS:
        ys = str(year); common = []
        for offset in range(-15,16):
            os_ = str(offset)
            if all(all(v is not None for v in out[g]['years'][ys][os_].values()) for g in
            ↪  ids): common.append(os_)
        counts[ys] = len(common)
        for g in out:
            out[g]['years'][ys] = {o:v for o,v in out[g]['years'][ys].items() if o in common}
    return out, {'factor_ids': ids, 'paired_endpoints_by_year': counts, 'note':
    ↪  '共同全部窗口/因素支持；不满足25/4仍NA'}

def q1_rank_rows(scores):
    valid = sorted([(k,v) for k,v in scores.items() if v is not None], key=lambda
    ↪  z:(-z[1],z[0]))
    result = []
    for k,v in valid:
        result.append({'id':k,'rank_min':1+sum(x>v for _,x in valid),'rank_max':sum(x>=v for
        ↪  _,x in valid),'score':v})
    return result


import datetime as dt
from fractions import Fraction
import math

import numpy as np


P2_YEARS = (2021, 2022, 2023, 2024, 2025)
P2_HEIGHTS = (1, 2, 4, 10, 20)
P2_QUANTILES = ((1, 20), (1, 4), (1, 2), (3, 4), (19, 20))
P2_QKEYS = ('q05', 'q25', 'median', 'q75', 'q95')
P2_CHANNELS = (
    ('t2', 'Ta_2m(degree C)', '2 m 气温', '°C', 'temperature'),
    ('rh2', 'RH_2m(%)', '2 m 相对湿度', '%', 'humidity'),
    ('ws10', 'WS_10m(m/s)', '10 m 标量风速', 'm s⁻¹', 'wind'),
    ('pressure', 'Pressure(hPa)', '气压原列', 'hPa', 'pressure'),
    *((f'soilt{str(z).replace(".", "")}', f'soilT_{z}m(degree C)',
```

```
       f'{z} m 土温', '°C', 'soil_temperature') for z in (0.1, 0.2, 0.4, 0.8, 1.6)),
    *((f'vwc{str(z).replace(".", "")}', f'VWC_{z}m(m3/m3)',
       f'{z} m 体积含水量', 'm³ m⁻³', 'soil_water') for z in (0.1, 0.2, 0.4, 0.8, 1.6)),
    *((x.lower(), f'{x}(W/m2)', f'{x} 辐射原通道', 'W m⁻²', 'radiation')
      for x in ('Rsd', 'Rsu', 'Rld', 'Rlu')),
    ('h', 'H(W/m2)', 'H 有符号原通道', 'W m⁻²', 'flux'),
    ('le', 'LE(W/m2)', 'LE 有符号原通道', 'W m⁻²', 'flux'),
    ('ustar', 'USTAR(m/s)', 'USTAR 原通道', 'm s⁻¹', 'friction_velocity'),
    ('g01', 'G_0.1m(W/m2)', '0.1 m 板热流原通道', 'W m⁻²', 'ground_heat'),
    ('g02', 'G_0.2m(W/m2)', '0.2 m 板热流原通道', 'W m⁻²', 'ground_heat'),
)


def _p2_plain(value):

    if isinstance(value, dict):
        return {str(k): _p2_plain(v) for k, v in value.items()}
    if isinstance(value, (list, tuple, np.ndarray)):
        return [_p2_plain(v) for v in value]
    if isinstance(value, (bool, np.bool_)):
        return bool(value)
    if isinstance(value, (int, np.integer)):
        return int(value)
    if isinstance(value, (float, np.floating, Fraction)):
        number = float(value)
        return number if math.isfinite(number) else None
    return value


def _p2_equal_year_quantile(groups, probabilities=P2_QUANTILES):


    if len(groups) != len(P2_YEARS) or any(not len(g) for g in groups):
        return [None] * len(probabilities)
    denominator = math.lcm(*(len(g) for g in groups))
    rows = sorted((v, denominator // len(g)) for g in groups for v in g)
    total = len(groups) * denominator
    result = []
    for numerator, divisor in probabilities:
        cumulative = 0
        answer = None
        for value, weight in rows:
            cumulative += weight
            if cumulative * divisor >= numerator * total:
                answer = value
                break
        result.append(answer)
    return result


def _p2_background(values, valid, indices, scale):

    quantiles = {key: [] for key in P2_QKEYS}
    exact = {key: [] for key in P2_QKEYS}
```

```
    counts = np.zeros((5, indices.shape[-1]), dtype=int)
    for k in range(indices.shape[-1]):
        groups = []
        for yi in range(5):
            ix = indices[yi, :, k]
            inside = (ix >= 0) & (ix < len(values))
            ix = ix[inside]
            ix = ix[valid[ix]]
            group = [int(v) for v in values[ix]]
            groups.append(group)
            counts[yi, k] = len(group)
        qs = _p2_equal_year_quantile(groups)
        for key, q in zip(P2_QKEYS, qs):
            exact[key].append(q)
            quantiles[key].append(float(q / scale) if q is not None else None)
    return {'quantiles': quantiles, 'exact_scaled_quantiles': exact,
            'history_n_by_year': counts.tolist(),
            'history_years_count': (counts > 0).sum(axis=0).tolist()}


def _p2_values(values, valid, indices, scale):
    return [float(values[i] / scale) if 0 <= i < len(values) and valid[i] else None
            for i in indices]


def _p2_summary(values):
    n = len(values)
    pairs = [(k, float(v)) for k, v in enumerate(values) if v is not None]
    result = {'expected_hours': n, 'available_hours': len(pairs),
              'complete_support': len(pairs) == n,
              'available_fraction': len(pairs) / n if n else None,
              'mean': None, 'min': None, 'max': None, 'slope_per_day': None,
              'first_24h_mean': None, 'last_24h_mean': None,
              'first_24h_available_hours': 0, 'last_24h_available_hours': 0,
              'last_minus_first_24h_mean': None}
    if pairs:
        x = np.array([p[0] for p in pairs], dtype=float)
        y = np.array([p[1] for p in pairs], dtype=float)
        result.update(mean=float(y.mean()), min=float(y.min()), max=float(y.max()))
        if len(x) > 1 and np.ptp(x) > 0:
            result['slope_per_day'] = float(np.dot(x-x.mean(), y-y.mean()) /
                                            np.dot(x-x.mean(), x-x.mean()) * 24)
    for part, seq in (('first', values[:24]), ('last', values[-24:])):
        good = [v for v in seq if v is not None]
        result[f'{part}_24h_available_hours'] = len(good)
        result[f'{part}_24h_mean'] = float(np.mean(good)) if good else None
    if result['first_24h_mean'] is not None and result['last_24h_mean'] is not None:
        result['last_minus_first_24h_mean'] = result['last_24h_mean'] -
        ↪  result['first_24h_mean']
    return result


def _p2_qc_note(identity):
    if identity == 'not_provided':
```

```
        return '源文件未提供 QC 列；D0 有效原值；未伪造 QC0，也不等于已通过通量专项质控。'
    if identity == 'source_not_read':
        return '所需源列未传入；本列为 NA、未构造有效值或 QC。'
    return '保留调用方 D0 有效掩码及真实 QC 身份；未新增插补、截尾、翻号或异常值筛除。'


def _p2_episode_records(values, valid, thresholds, target_indices, relative,
                        support_ends, direction, scale):
    assessed, excess, exceedance = [], [], []
    for k, i in enumerate(target_indices):
        available = 0 <= i < len(values) and bool(valid[i]) and thresholds[k] is not None
        assessed.append(available)
        if not available:
            exceedance.append(None)
            excess.append(None)
        else:
            delta = (int(values[i]) - thresholds[k]) * (1 if direction == 'high' else -1)
            exceedance.append(bool(delta > 0))
            excess.append(float(delta / scale))
    episodes = []
    start = None
    for k in range(len(target_indices) + 1):
        hit = k < len(target_indices) and exceedance[k] is True
        if hit and start is None:
            start = k
        if not hit and start is not None:
            stop = k
            peak = max(range(start, stop), key=lambda j: excess[j])
            episodes.append({
                'start_support_iso': (support_ends[start] -
                ↪  dt.timedelta(hours=1)).isoformat(),
                'end_support_iso': support_ends[stop-1].isoformat(),
                'start_support_end_relative_hours': relative[start],
                'end_support_end_relative_hours': relative[stop-1],
                'duration_hours': stop-start,
                'peak_support_end_iso': support_ends[peak].isoformat(),
                'peak_value': float(values[target_indices[peak]] / scale),
                'peak_threshold': float(thresholds[peak] / scale),
                'peak_excess': excess[peak],
                'support_complete_within_episode': True,
            })
            start = None
    summaries = {}
    for label, hours in (('30d', 720), ('7d', 168), ('72h', 72)):
        hours = min(hours, len(target_indices))
        offset = len(target_indices) - hours
        sub = exceedance[offset:]
        available = sum(0 <= i < len(values) and bool(valid[i]) for i in
        ↪  target_indices[offset:])
        known = sum(v is not None for v in sub)
        hits = sum(v is True for v in sub)
        lengths = []
        window_episodes = []
        run = 0
```

```
        for local_k, hit in enumerate(sub + [None]):
            if hit is True:
                run += 1
            elif run:
                lengths.append(run)
                first_k, last_k = offset+local_k-run, offset+local_k-1
                window_episodes.append({
                    'start_support_iso':
                    ↪  (support_ends[first_k]-dt.timedelta(hours=1)).isoformat(),
                    'end_support_iso': support_ends[last_k].isoformat(),
                    'duration_hours': run,
                    'clipped_to_display_window': first_k == offset and offset > 0 and
                    ↪  exceedance[offset-1] is True,
                })
                run = 0
        good_k = [k for k in range(offset, len(target_indices)) if assessed[k]]
        peak_k = max(good_k, key=lambda k: excess[k]) if good_k else None
        observed_k = [k for k in range(offset, len(target_indices))
                      if 0 <= target_indices[k] < len(values) and valid[target_indices[k]]]
        physical_k = ((max if direction == 'high' else min)(observed_k,
                       key=lambda k: values[target_indices[k]]) if observed_k else None)
        summaries[label] = {
            'expected_hours': hours, 'observed_available_hours': int(available),
            'threshold_available_hours': sum(t is not None for t in thresholds[offset:]),
            'assessed_hours': known, 'unassessed_hours': hours-known,
            'complete_support': known == hours,
            'known_exceedance_hours': hits if known else None,
            'fraction_among_assessed_hours': hits/known if known else None,
            'longest_known_episode_hours': max(lengths, default=0) if known else None,
            'longest_known_episodes': [r for r in window_episodes
                                       if r['duration_hours'] == max(lengths, default=0)],
            'observed_min': float(min(values[target_indices[k]] for k in observed_k)/scale)
            ↪  if observed_k else None,
            'observed_max': float(max(values[target_indices[k]] for k in observed_k)/scale)
            ↪  if observed_k else None,
            'observed_extreme_value': float(values[target_indices[physical_k]]/scale) if
            ↪  physical_k is not None else None,
            'observed_extreme_support_end': support_ends[physical_k].isoformat() if
            ↪  physical_k is not None else None,
            'max_signed_excess': excess[peak_k] if peak_k is not None else None,
            'value_at_max_signed_excess': float(values[target_indices[peak_k]] / scale) if
            ↪  peak_k is not None else None,
            'threshold_at_max_signed_excess': float(thresholds[peak_k] / scale) if peak_k is
            ↪  not None else None,
            'support_end_at_max_signed_excess': support_ends[peak_k].isoformat() if peak_k is
            ↪  not None else None,
        }
    return {'exceedance': exceedance, 'signed_excess': excess, 'episodes': episodes,
            'summaries': summaries,
            'episode_rule': '严格>高阈值或严格<低阈值；每条为假设下完整1h支持；缺观测或缺历史阈值均断段，
            ↪  绝不视为0/非超阈。'}


def build_processes(arrays, endpoint='H-END', cutoff_hour=10):
```

```
if endpoint not in ('H-END', 'H-START'):
    raise ValueError('endpoint must be H-END or H-START')
if not isinstance(cutoff_hour, int) or not 0 <= cutoff_hour <= 23:
    raise ValueError('cutoff_hour must be an integer between 0 and 23')
tz = dt.timezone(dt.timedelta(hours=8))
origin = dt.datetime(2021, 1, 1, tzinfo=tz)
cutoff = dt.datetime(2026, 8, 26, cutoff_hour, tzinfo=tz)
end = int((cutoff-origin).total_seconds() // 3600)
relative = list(range(-719, 1))
support_ends = [cutoff + dt.timedelta(hours=h) for h in relative]
shift = 0 if endpoint == 'H-END' else -1
target_indices = np.array([end+h+shift for h in relative], dtype=int)
hist_indices = np.array([[[int((cutoff.replace(year=year)-origin).total_seconds() //
↪  3600) +
                           day*24+h+shift for h in relative]
                          for day in range(-15, 16)] for year in P2_YEARS], dtype=int)
grid_length = max((len(v['values']) for v in arrays.values()), default=end+1)
cache = {}

def source(column):
    if column in arrays:
        a = arrays[column]
        if len(a['values']) != len(a['valid']):
            raise ValueError(f'values/valid length mismatch: {column}')
        return a['values'], np.asarray(a['valid'], dtype=bool), 10**int(a['p']),
        ↪  a.get('qc_identity', 'not_supplied_to_process_builder')
    return np.zeros(grid_length, dtype=object), np.zeros(grid_length, dtype=bool), 1,
    ↪  'source_not_read'

def series(column):
    if column not in cache:
        values, valid, scale, qc = source(column)
        background = _p2_background(values, valid, hist_indices, scale)
        cache[column] = {'values': _p2_values(values, valid, target_indices, scale),
                         'background': background, 'qc_identity': qc}
    return cache[column]

channels = []
for key, column, label, unit, group in P2_CHANNELS:
    record = series(column)
    channel = {'id': key, 'column': column, 'label': label, 'unit': unit,
               'group': group, 'values': record['values'],
               'qc_identity': record['qc_identity'], 'qc_note':
               ↪  _p2_qc_note(record['qc_identity']),
               'history_n_by_year': record['background']['history_n_by_year'],
               'history_years_count': record['background']['history_years_count'],
               'summaries': {label: _p2_summary(record['values'][-hours:])
                             for label, hours in (('30d', 720), ('7d', 168), ('72h',
                             ↪  72))}}
    channel.update({f'background_{name}': values for name, values in
    ↪  record['background']['quantiles'].items()})
    if group in ('radiation', 'flux', 'ground_heat'):
```

```
        channel['interpretation'] = ('同名通道跨年可比的条件描述；保留有符号原值，不合成净能量、
        ↪  不取绝对值或翻号。'
                                     'G为给定深度板热流，不是订正地表热流；
                                     ↪  H/LE正向及处理链未由本统计新核实。')
    elif group == 'friction_velocity':
        channel['interpretation'] = 'USTAR同名原通道条件描述；未补低u*筛选，
        ↪  未推Obukhov长度或完整湍流机制。'
    channels.append(channel)

tcols = [f'Ta_{z}m(degree C)' for z in P2_HEIGHTS]
ucols = [f'WS_{z}m(m/s)' for z in P2_HEIGHTS]

def paired(upper, lower):
    high, vh, sh, qh = source(upper)
    low, vl, sl, ql = source(lower)
    scale = max(sh, sl)
    n = min(len(high), len(low))
    values = np.zeros(n, dtype=object)
    valid = vh[:n] & vl[:n]


    valid[min(len(valid), end+shift+1):] = False
    ix = np.flatnonzero(valid)
    values[ix] = (np.asarray(high[ix], dtype=object) * (scale // sh) -
                  np.asarray(low[ix], dtype=object) * (scale // sl))
    return {'values': values, 'valid': valid, 'scale': scale,
            'qc_identity': {'upper': qh, 'lower': ql},
            'target': _p2_values(values, valid, target_indices, scale),
            'background': _p2_background(values, valid, hist_indices, scale)}

td = paired(tcols[-1], tcols[1])
ud = paired(ucols[-1], ucols[1])

def joint_mask(columns):
    records = [source(c) for c in columns]
    n = min(len(r[1]) for r in records)
    return records, np.logical_and.reduce([r[1][:n] for r in records])

def snapshot(columns, relative_hour):
    records, joint = joint_mask(columns)
    k = relative.index(relative_hour)
    target = [series(c)['values'][k] for c in columns]
    groups_by_height = [[] for _ in columns]
    counts = []
    for yi in range(5):
        ix = hist_indices[yi, :, k]
        ix = ix[(ix >= 0) & (ix < len(joint))]
        ix = ix[joint[ix]]
        counts.append(len(ix))
        for z, (values, _, scale, _) in enumerate(records):
            groups_by_height[z].append([Fraction(int(values[i]), scale) for i in ix])
    q = [_p2_equal_year_quantile(groups) for groups in groups_by_height]
    return {'values': target, 'valid_count': sum(v is not None for v in target),
            'target_complete_profile': all(v is not None for v in target),
```

```
            'background_q25': [v[1] for v in q],
            'background_median': [v[2] for v in q],
            'background_q75': [v[3] for v in q],
            'history_n_by_year': counts,
            'historical_joint_mask': 'all five heights valid at the same hour before
            ↪  aggregation',
            'qc_identity': [r[3] for r in records]}

profiles = [{'support_end_relative_hours': h,
             'support_end_iso': (cutoff+dt.timedelta(hours=h)).isoformat(),
             'temperature': snapshot(tcols, h), 'wind': snapshot(ucols, h),
             'statistic': 'one nominal hourly record per height; not a 24h mean'}
            for h in (-72, -24, 0)]
time_height = {'support_end_relative_hours': relative,
               'heights_m': list(P2_HEIGHTS),
               'temperature_c': [series(c)['values'] for c in tcols],
               'wind_ms': [series(c)['values'] for c in ucols],
               'temperature_difference_c': td['target'],
               'wind_difference_ms': ud['target'],
               'temperature_difference_summaries': {label:
               ↪  _p2_summary(td['target'][-hours:])
                    for label, hours in (('30d', 720), ('7d', 168), ('72h', 72))},
               'wind_difference_summaries': {label: _p2_summary(ud['target'][-hours:])
                    for label, hours in (('30d', 720), ('7d', 168), ('72h', 72))},
               'temperature_pair_qc_identity': td['qc_identity'],
               'wind_pair_qc_identity': ud['qc_identity'],
               'temperature_pair_history_n_by_year':
               ↪  td['background']['history_n_by_year'],
               'wind_pair_history_n_by_year': ud['background']['history_n_by_year']}
for prefix, pair in (('temperature', td), ('wind', ud)):
    time_height.update({f'pair_{prefix}_background_{name}': value
                        for name, value in pair['background']['quantiles'].items()})


def period_profile(columns, period_mask):
    records, joint = joint_mask(columns)
    ix = target_indices[period_mask]
    ix = ix[(ix >= 0) & (ix < len(joint))]
    ix = ix[joint[ix]]
    values = [Fraction(sum(int(v[i]) for i in ix), len(ix)*scale) if len(ix) else None
              for v, _, scale, _ in records]
    groups_by_height = [[] for _ in columns]
    counts = []
    support_counts = []
    for yi in range(5):
        yearly = [[] for _ in columns]
        yearly_support = []
        for oi in range(31):
            hi = hist_indices[yi, oi, period_mask]
            hi = hi[(hi >= 0) & (hi < len(joint))]
            hi = hi[joint[hi]]
            yearly_support.append(len(hi))
            if len(hi):
                for zi, (v, _, scale, _) in enumerate(records):
```

```
                    yearly[zi].append(Fraction(sum(int(v[i]) for i in hi), len(hi)*scale))
        counts.append(len(yearly[0]))
        support_counts.append(yearly_support)
        for zi in range(len(columns)):
            groups_by_height[zi].append(yearly[zi])
    qs = [_p2_equal_year_quantile(g) for g in groups_by_height]
    return {'values': values, 'background_q25': [q[1] for q in qs],
            'background_median': [q[2] for q in qs], 'background_q75': [q[3] for q in qs],
            'valid_counts': [len(ix)]*len(columns),
            'expected_hours': int(period_mask.sum()), 'history_n_by_year': counts,
            'history_available_hours_by_year_trajectory': support_counts,
            'joint_all_heights_before_mean': True}

def period_pair(pair, period_mask):
    ix = target_indices[period_mask]
    ix = ix[(ix >= 0) & (ix < len(pair['valid']))]
    ix = ix[pair['valid'][ix]]
    groups, support_counts = [], []
    for yi in range(5):
        group, yearly_support = [], []
        for oi in range(31):
            hi = hist_indices[yi, oi, period_mask]
            hi = hi[(hi >= 0) & (hi < len(pair['valid']))]
            hi = hi[pair['valid'][hi]]
            yearly_support.append(len(hi))
            if len(hi):
                group.append(Fraction(sum(int(pair['values'][i]) for i in hi),
                ↪  len(hi)*pair['scale']))
        groups.append(group)
        support_counts.append(yearly_support)
    qs = _p2_equal_year_quantile(groups)
    return {'value': Fraction(sum(int(pair['values'][i]) for i in ix),
    ↪  len(ix)*pair['scale']) if len(ix) else None,
            'valid_count': len(ix), 'expected_hours': int(period_mask.sum()),
            'background_q25': qs[1], 'background_median': qs[2], 'background_q75': qs[3],
            'history_n_by_year': [len(g) for g in groups],
            'history_available_hours_by_year_trajectory': support_counts,
            'joint_pair_before_mean': True}

periods = []
for first in (0, 6, 12, 18):

    mask = np.array([first <= (t-dt.timedelta(hours=1)).hour < first+6 for t in
    ↪  support_ends])
    periods.append({'label': f'{first:02d}–{first+6:02d}', 'hours': list(range(first,
    ↪  first+6)),
                    'temperature': period_profile(tcols, mask),
                    'wind': period_profile(ucols, mask),
                    'temperature_difference': period_pair(td, mask),
                    'wind_difference': period_pair(ud, mask)})

extreme_series = []
for key, column, label, direction, quantile in (
    ('t2_low_p05', tcols[1], '2 m 气温低于历史 P05', 'low', 'q05'),
```

```
    ('t2_high_p95', tcols[1], '2 m 气温高于历史 P95', 'high', 'q95'),
    ('ws10_high_p95', ucols[3], '10 m 风速高于历史 P95', 'high', 'q95'),
):
    values, valid, scale, qc = source(column)
    background = series(column)['background']
    thresholds = background['exact_scaled_quantiles'][quantile]
    record = {'id': key, 'label': label, 'column': column, 'direction': direction,
              'channel_id': 't2' if column == tcols[1] else 'ws10',
              'unit': '°C' if column == tcols[1] else 'm s⁻¹',
              'quantile': quantile, 'values': series(column)['values'],
              'threshold': background['quantiles'][quantile],
              'threshold_exact_scaled': [str(t) if t is not None else None for t in
              ↪  thresholds],
              'scale': str(scale), 'history_n_by_year': background['history_n_by_year'],
              'history_years_count': background['history_years_count'], 'qc_identity': qc}
    record.update(_p2_episode_records(values, valid, thresholds, target_indices,
                                      relative, support_ends, direction, scale))
    extreme_series.append(record)

note = ('逐目标日期/钟点，2021–2025 每年同历日 ±15 日（31 个候选小时），'
        '按每年有效 n_y 给每条 1/(5 n_y) 权重；五年任一年 n_y=0 则分位 NA；'
        '逆经验 CDF inf{x:F(x)≥p}，不插值；分位带是历史背景，非置信区间。')
result = {
    'metadata': {'schema_version': 'q1-process-v2.1', 'primary_cutoff':
    ↪  cutoff.isoformat(),
                 'endpoint': endpoint, 'timezone': 'UTC+8', 'origin': origin.isoformat(),
                 'reference_years': list(P2_YEARS), 'seasonal_days_each_side': 15,
                 'hour_label_assumption': 'H-END: label h supports [h-1h,h); H-START:
                 ↪  label h supports [h,h+1h)',
                 'time_coordinate': 'support interval end relative to cutoff in hours',
                 'nominal_support_hours': 1, 'actual_processing_support_confirmed': False,
                 'channel_selection': 'fixed scientific process groups; independent of
                 ↪  Top5/rank',
                 'future_support_used': False,
                 'background_quantile_note': note,
                 'no_inference': '不据这些过程认定事件触发、致因、真实日照或完整 PBL 高度。'},
    'near_surface': {'heights_m': list(P2_HEIGHTS), 'profiles': profiles,
                     'time_height': time_height,
                     'diurnal': {'periods': periods,
                                 'scope_note': ('固定UTC+8 00–06/06–12/12–18/18–24，
                                 ↪  按支持区间起点小时分组，'
                                               '不等同真实昼夜。目标为 30d 内共同有效小时均值；'
                                               '历史为每年31条同季30d轨迹各自的同固定时段共同有」
                                               ↪  效均值，
                                               ↪  '
                                               '再按年等权经验分位；轨迹重叠，
                                               ↪  不是独立样本/置信区间。')},
                     'scope_note': '1–20 m近地层记录；T20–T2为温差而非位温梯度；
                     ↪  WS20–WS2为标量风速差而非矢量切变；跨列先共同有效配对。'},
    'weather': {'time_relative_hours': relative, 'support_end_iso': [t.isoformat() for t
    ↪  in support_ends],
                'support_start_iso': [(t-dt.timedelta(hours=1)).isoformat() for t in
                ↪  support_ends],
                'channels': channels, 'seasonal_background_note': note,
```

```
                    'display_windows_hours': [720, 168, 72],
                    'summary_note': ('均值/极值/OLS 仅描述已有效支持，附可用时长；不补齐缺格。'
                                     '末24h与首24h差/斜率不等于末段单调变化。窗口用于固定过程展示，
                                     ↪  不新增排名择窗。'),
                    'precipitation': {'status':
                    ↪  'not_computed_unknown_increment_or_accumulation',
                                      'note': '当前Precip增量/累计/复位语义未闭合；未求和/差分/雨强，
                                      ↪  不能据此声称无雨或无强雨。'}},
        'extremes': {'threshold_rule': note,
                     'predefined_tests': ['T2<P05', 'T2>P95', 'WS10>P95'],
                     'revision_context': 'V2为已看V1结果后的设计修订；
                     ↪  本组阈值固定后不按2026命中情况挑阈值，不称事前盲定。',
                     'time_relative_hours': relative, 'series': extreme_series,
                     'support_note': '完整支持/可用支持分别报告；已知超阈小时数仅为可评估小时内计数；
                     ↪  缺失/缺阈值断段，不推断缺口内事件。'},
    }
    return _p2_plain(result)


def process_synthetic_selftest():


    groups = [[0], [0], [0], [100], [100]*31]
    assert _p2_equal_year_quantile(groups, ((1, 2),))[0] == 0
    assert _p2_equal_year_quantile([[], [0], [0], [0], [0]]) == [None]*5

    big = 2**60
    assert _p2_equal_year_quantile([[big+i] for i in range(5)], ((1, 2),))[0] == big+2
    tz = dt.timezone(dt.timedelta(hours=8))
    ends = [dt.datetime(2026, 8, 26, h, tzinfo=tz) for h in range(1, 6)]
    vals = np.array([11, 11, 11, 11, 11], dtype=object)
    mask = np.array([True, True, False, True, True])
    rec = _p2_episode_records(vals, mask, [10]*5, np.arange(5), list(range(-4, 1)), ends,
    ↪  'high', 1)
    assert [e['duration_hours'] for e in rec['episodes']] == [2, 2]
    assert rec['exceedance'][2] is None

    origin = dt.datetime(2021, 1, 1, tzinfo=tz)
    cutoff = dt.datetime(2026, 8, 26, 10, tzinfo=tz)
    end = int((cutoff-origin).total_seconds()//3600)
    n = end+3
    data = {}
    for h in P2_HEIGHTS:
        for prefix, suffix, base in (('Ta', '(degree C)', h), ('WS', '(m/s)', h*2)):
            data[f'{prefix}_{h}m{suffix}'] = {'values': np.full(n, base, dtype=object),
                                              'valid': np.ones(n, dtype=bool), 'p': 0,
                                              'qc_identity': 'synthetic'}

    data['Ta_20m(degree C)']['values'][end] = 200
    data['Ta_2m(degree C)']['valid'][end] = False
    data['Ta_20m(degree C)']['values'][end+1:] = 99999
    result = build_processes(data)
    th = result['near_surface']['time_height']
    assert th['temperature_c'][-1][-1] == 200
```

```
    assert th['temperature_difference_c'][-1] is None
    assert th['temperature_difference_c'][-2] == 18
    assert result['near_surface']['diurnal']['periods'][1]['temperature']['values'][-1] == 20
    assert max(v for row in th['temperature_c'] for v in row if v is not None) == 200
    assert result['weather']['support_end_iso'][-1] == cutoff.isoformat()

    reference_before = result['weather']['channels'][0]['background_q95']
    data['Ta_2m(degree C)']['values'][end-719:end+1] = big
    changed = build_processes(data, endpoint='H-START')
    assert changed['weather']['channels'][0]['background_q95'] == reference_before
    assert changed['near_surface']['time_height']['temperature_c'][-1][-1] == 20
    import json
    json.dumps(result, allow_nan=False)
    return {'status': 'pass', 'checks': ['exact_equal_year_inverse_cdf',
    ↪  'all_five_years_required',
            'integer_threshold_precision', 'missing_hour_breaks_episode',
            ↪  'pairwise_common_mask',
            'complete_support_before_cutoff', 'H_END_vs_H_START',
            ↪  'historical_threshold_excludes_2026',
            'strict_json_no_nan'], 'scope': 'synthetic author self-check; not independent
            ↪  scientific acceptance'}


import copy as v2b_copy
import math as v2b_math
import numbers as v2b_numbers
from fractions import Fraction as v2b_Fraction


V2B_NATIVE_V1_IDS = (
    't2_mean', 't2_max', 'pdd', 'warm_rate', 'vwc01', 'vwc02',
    'soilt01', 'p_mean', 'rh2_mean', 'ws10_mean',
)


def v2b_finite(value):
    return (isinstance(value, v2b_numbers.Real) and not isinstance(value, bool)
            and v2b_math.isfinite(value))


def v2b_plain(value):

    if value is None or isinstance(value, (str, bool)):
        return value
    if isinstance(value, dict):
        return {str(key): v2b_plain(item) for key, item in value.items()}
    if isinstance(value, (tuple, list)):
        return [v2b_plain(item) for item in value]
    if isinstance(value, v2b_numbers.Integral):
        return int(value)
    if isinstance(value, v2b_numbers.Real):
        return float(value) if v2b_math.isfinite(value) else None
    raise TypeError('comparison output is not strict JSON: ' + type(value).__name__)
```

```
def v2b_specs(features):
    if isinstance(features, dict):
        features = features.get('factors', features.get('features', []))
    out = {str(feature['id']): feature for feature in features}
    if len(out) != len(features):
        raise ValueError('duplicate V2 comparison feature id')
    return out


def v2b_rows_by_id(method):
    return {str(row.get('factor_id', row.get('id'))): row
            for row in method.get('factor_rows', [])}


def v2b_a_scores(result, ids):
    scores = {}
    for factor_id in ids:
        factor = result.get('factors', {}).get(factor_id, {})
        if factor.get('rank_eligible') and v2b_finite(factor.get('score_S')):
            exact = factor.get('score_S_exact')
            scores[factor_id] = (v2b_Fraction(exact) if exact is not None
                                 else factor['score_S'])
    return scores


def v2b_ranks(scores):
    return {row['id']: row for row in q1_rank_rows(scores)}


def v2b_interval(rows, factor_id):
    row = rows.get(factor_id)
    if row is None or not v2b_finite(row.get('rank_min')):
        return None
    return {'rank_min': int(row['rank_min']), 'rank_max': int(row['rank_max'])}


def v2b_reuse_check(baseline, validation, seed):

    if not isinstance(validation, dict) or validation.get('status') != 'pass':
        return False, 'no passed caller-side input/code/configuration validation'
    digest = validation.get('file_sha')
    if (not isinstance(digest, str) or len(digest) != 64
            or any(char not in '0123456789abcdefABCDEF' for char in digest)):
        return False, 'validated baseline file SHA256 is absent or malformed'
    controls = validation.get('verified_controls')
    if not isinstance(controls, (dict, list, tuple)) or not controls:
        return False, 'verified_controls must contain caller-side evidence, not a boolean'
    if not isinstance(baseline, dict):
        return False, 'baseline analysis result is absent'
    method = baseline.get('methods', {}).get('B_aligned')
    if not isinstance(method, dict):
        return False, 'baseline methods.B_aligned is absent'
    rows = method.get('factor_rows', [])
```

```
    ids = [row.get('factor_id') for row in rows]
    if len(ids) != len(V2B_NATIVE_V1_IDS) or set(ids) != set(V2B_NATIVE_V1_IDS):
        return False, 'baseline B is not the fixed native V1 ten-factor family'
    metadata = method.get('metadata', {})
    expected = {'seed': seed, 'n_perm_requested': 2000,
                'n_boot_year_resamples_executed': 1000, 'n_perm_boot_requested': 500}
    if any(metadata.get(key) != value for key, value in expected.items()):
        return False, 'baseline seed or formal draw configuration differs'
    if metadata.get('formal_counts') is not True:
        return False, 'baseline does not record the required formal counts'
    return True, 'reuse authorized by caller-side evidence bound to baseline SHA256'


def v2b_fit(specs, values, ids, seed, label, role):

    if len(ids) < 2:
        return {'factor_rows': [
                    {'factor_id': factor_id, 'status': 'insufficient_comparison_family',
                     'topsis_score': None, 'rank': None, 'rank_min': None,
                     'rank_max': None, 'perm_draws_executed': 0}
                    for factor_id in ids],
                'window_rows': [], 'bootstrap_rows': [], 'comparisons': {},
                'metadata': {
                    'label': label, 'family_role': role, 'input_factor_ids': list(ids),
                    'n_input_original_factors': len(ids),
                    'n_computable_topsis_factors': 0, 'seed': seed,
                    'execution_in_v2': False, 'execution_class':
                    ↪  'not_executed_insufficient_family',
                    'reason': 'TOPSIS requires at least two rows; no support threshold
                    ↪  relaxed',
                    'n_perm_requested': 2000, 'n_boot_year_resamples_requested': 1000,
                    'n_perm_boot_requested': 500, 'n_boot_year_resamples_executed': 0,
                    'new_main_pseudo_draws_executed': 0,
                    'new_bootstrap_pseudo_draws_executed': 0,
                    'entropy_weights_A_neglogp_mechanism': [None, None, None],
                    'topsis_status': 'fewer_than_two_input_rows'}}
    result = qf_run_ref([specs[factor_id] for factor_id in ids], values, seed=seed,
                        label=label, n_perm=2000, n_boot=1000, n_perm_boot=500)
    metadata = result.setdefault('metadata', {})
    metadata.update({
        'family_role': role, 'input_factor_ids': list(ids), 'execution_in_v2': True,
        'reuse_status': 'new_actual_fit',
        'new_main_pseudo_draws_executed': sum(
            row.get('perm_draws_executed', 0) for row in result.get('factor_rows', [])),
        'new_bootstrap_pseudo_draws_executed': sum(
            row.get('pseudo_draws_executed', 0) for row in result.get('bootstrap_rows', [])),
        'fitting_scope': 'entropy weights, TOPSIS, BH and each bootstrap fit use exactly this
        ↪  input family; unavailable rows remain NA',
        'proxy_policy': 'only original24 factors with existing original mechanism scores; no
        ↪  proxy mechanism score invented',
        'D0_alignment': 'caller-supplied D0 cache and applicable W; no filling, clipping or
        ↪  recleaning',
        'original_pdd_unit_alignment': 'dictionary degree-C h; constant /24 to degree-C d
        ↪  leaves ranks and B unchanged',
```

```
    })
    if result.pop('_final_reused_modex',False):
        metadata.update(execution_in_final=False,reuse_status='frozen_参考方法_reused_after_ex」
        ↪  act_input_check',
                        new_main_pseudo_draws_executed=0,new_bootstrap_pseudo_draws_executed=」
                        ↪  0,
                        n_boot_year_resamples_executed_in_final=0,
                        count_fields_scope='source-run counts retained; no new 参考方法 draws
                        ↪  in this run')
    else:
        metadata['execution_in_final']=True
    for row in result.get('window_rows', []):
        factor_id = row['factor_id']
        scale = specs[factor_id].get('rank_scale')
        if row.get('event_value') is not None and isinstance(scale, int) and scale > 0:
            row['event_value_physical'] = row['event_value'] / scale
    return result


def v2b_common_table(ids, specs, e1, e2, b_result, c_index, *, scope):
    e1_ranks = v2b_ranks(v2b_a_scores(e1, ids))
    e2_ranks = v2b_ranks(v2b_a_scores(e2, ids))
    b_rows = v2b_rows_by_id(b_result)
    b_ranks = v2b_ranks({factor_id: b_rows[factor_id]['topsis_score']
                        for factor_id in ids if factor_id in b_rows
                        and v2b_finite(b_rows[factor_id].get('topsis_score'))})
    c_ranks = v2b_ranks({factor_id: c_index[factor_id]['A'] for factor_id in ids
                        if factor_id in c_index and v2b_finite(c_index[factor_id].get('A'))})
    e1_native = {row['id']: row for row in e1.get('ranking_all', [])}
    e2_native = {row['id']: row for row in e2.get('ranking_all', [])}
    e1_dedup = {row['id']: row for row in e1.get('candidate_ranking', [])}
    e2_dedup = {row['id']: row for row in e2.get('candidate_ranking', [])}
    return [{
        'id': factor_id, 'label': specs[factor_id].get('label', factor_id),
        'E1_rank': v2b_interval(e1_ranks, factor_id),
        'E2_rank': v2b_interval(e2_ranks, factor_id),
        'B_rank': v2b_interval(b_ranks, factor_id),
        'C_rank': v2b_interval(c_ranks, factor_id),
        'E1_score_S': e1.get('factors', {}).get(factor_id, {}).get('score_S'),
        'E2_score_S': e2['factors'][factor_id].get('score_S'),
        'B_topsis_score': b_rows.get(factor_id, {}).get('topsis_score'),
        'C_fixed7d_A': c_index[factor_id].get('A'),
        'E1_native_rank': v2b_interval(e1_native, factor_id),
        'E2_native_rank': v2b_interval(e2_native, factor_id),
        'E1_native_deduplicated_rank': v2b_interval(e1_dedup, factor_id),
        'E2_native_deduplicated_rank': v2b_interval(e2_dedup, factor_id),
        'C_native_rank': v2b_interval(c_index, factor_id),
        'scope': scope,
        'E1_scope': 'E1 original10 intersect displayed IDs only; newly restored factors are
        ↪  NA',
    } for factor_id in ids]


def build_v2_comparisons(features, values, e1, e2, c_rows, baseline=None,
```

```
                    baseline_validation=None, seed=20261024):


specs = v2b_specs(features)
absent = set(V2B_NATIVE_V1_IDS) - set(specs)
if absent:
    raise ValueError('V2 dictionary lacks native V1 factors: ' + ',
    ↪  '.join(sorted(absent)))
original_ids = [factor[0] for factor in ORIGINAL_FACTORS]
expanded_ids = [factor_id for factor_id in original_ids if factor_id in specs
                and str(specs[factor_id].get('status', '')).upper() != 'NA'
                and specs[factor_id].get('window_hours')]
reuse, reuse_reason = v2b_reuse_check(baseline, baseline_validation, seed)
if reuse:
    native = v2b_copy.deepcopy(baseline['methods']['B_aligned'])
    native.setdefault('metadata', {}).update({
        'family_role': 'native_V1_original10_preserved',
        'input_factor_ids': list(V2B_NATIVE_V1_IDS),
        'execution_in_v2': False, 'reuse_status': 'reused_verified_prior_run',
        'reuse_source': {'field': 'methods.B_aligned',
                         'baseline_file_sha256': baseline_validation['file_sha'],
                         'caller_validation': v2b_copy.deepcopy(baseline_validation)},
        'new_main_pseudo_draws_executed': 0,
        'new_bootstrap_pseudo_draws_executed': 0,
        'count_interpretation': 'original executed counts are preserved as source-run
        ↪  facts; no new V2 draws for this native result',
    })
else:
    native = v2b_fit(specs, values, list(V2B_NATIVE_V1_IDS), seed,
                     'B-native-original10-V2-actual-recalculation',
                     'native_V1_original10_recalculated')
    native['metadata']['reuse_not_taken_reason'] = reuse_reason
expanded = v2b_fit(specs, values, expanded_ids, seed,
                   'B-expanded-original24-V2-actual-fit',
                   'expanded_semantically_available_original24')
b_expanded = v2b_rows_by_id(expanded)
a_ids = set(v2b_a_scores(e2, original_ids))
b_ids = {factor_id for factor_id, row in b_expanded.items()
         if v2b_finite(row.get('topsis_score'))}
c_index = {str(row['id']): row for row in c_rows}
c_ids = {factor_id for factor_id, row in c_index.items()
         if factor_id in original_ids and row.get('rank_eligible',True) and not
         ↪  row.get('uninformative',False) and not row.get('reason')
         and v2b_finite(row.get('q')) and v2b_finite(row.get('A'))}
common_ids = [factor_id for factor_id in original_ids
              if factor_id in a_ids & b_ids & c_ids]
common = v2b_fit(specs, values, common_ids, seed,
                 'B-common-original24-V2-actual-refit',
                 'common_original24_before_uniform_redundancy')
common_set = set(common_ids)
applied, not_applied = [], []
for row in e2.get('redundancy', {}).get('suppressed', []):
    factor_id, representative = row['id'], row['representative_id']
    if factor_id not in common_set:
```

```
        continue
    if representative in common_set:
        applied.append(v2b_copy.deepcopy(row))
    else:
        not_applied.append({**v2b_copy.deepcopy(row),
                            'comparison_action': 'retained: E2 representative absent from
                            ↪  this common pool'})
removed = {row['id'] for row in applied}
dedup_ids = [factor_id for factor_id in common_ids if factor_id not in removed]
dedup = v2b_fit(specs, values, dedup_ids, seed,
                'B-common-original24-uniform-deduplicated-V2-actual-refit',
                'common_original24_after_uniform_E2_redundancy')
methods = {'B_native_v1': native, 'B_expanded_original24': expanded,
           'B_common_refit': common, 'B_common_deduplicated_refit': dedup,
           'C_factors': v2b_copy.deepcopy(c_rows)}
if reuse and isinstance(baseline.get('methods', {}).get('B_original_cutoff'), dict):
    original_cutoff = v2b_copy.deepcopy(baseline['methods']['B_original_cutoff'])
    original_cutoff.setdefault('metadata', {}).update({
        'execution_in_v2': False, 'reuse_status': 'prior_V1_C09_reference_only',
        'baseline_file_sha256': baseline_validation['file_sha'],
        'new_main_pseudo_draws_executed': 0,
        'new_bootstrap_pseudo_draws_executed': 0,
        'V2_scope': 'preserved source reference; excluded from every V2 common fit and
        ↪  comparison',
    })
    methods['B_native_v1_original_cutoff_reference'] = original_cutoff
original10_scores1 = v2b_a_scores(e1, V2B_NATIVE_V1_IDS)
original10_scores2 = v2b_a_scores(e2, V2B_NATIVE_V1_IDS)
paired10 = [factor_id for factor_id in V2B_NATIVE_V1_IDS
            if factor_id in original10_scores1 and factor_id in original10_scores2]
ranks1 = v2b_ranks({factor_id: original10_scores1[factor_id] for factor_id in paired10})
ranks2 = v2b_ranks({factor_id: original10_scores2[factor_id] for factor_id in paired10})
result = {
    'methods': methods,
    'comparisons_v2': v2b_common_table(
        dedup_ids, specs, e1, e2, dedup, c_index,
        scope='uniform E2 redundancy on common original24 IDs; B actually refitted; A/C
        ↪  scores retained'),
    'comparisons_common_before_redundancy': v2b_common_table(
        common_ids, specs, e1, e2, common, c_index,
        scope='common original24 IDs before redundancy; B actually refitted; A/C scores
        ↪  retained'),
    'comparison_factor_ids': dedup_ids,
    'common_before_redundancy_ids': common_ids,
    'E1_E2_original10_common': [
        {'id': factor_id, 'label': specs[factor_id].get('label', factor_id),
         'E1_rank': v2b_interval(ranks1, factor_id),
         'E2_rank': v2b_interval(ranks2, factor_id),
         'E1_score_S': float(original10_scores1[factor_id]),
         'E2_score_S': float(original10_scores2[factor_id]),
         'scope': 'same rankable original10 IDs; unchanged exact A scores, ranks
         ↪  reindexed without redundancy'}
        for factor_id in paired10],
    'native_scope': {
```

```
        'E1': {'factor_ids': list(e1.get('factors', {})), 'settings': e1.get('settings'),
               'rankable_ids': list(v2b_a_scores(e1, specs)),
               'deduplicated_ids': [row['id'] for row in e1.get('candidate_ranking',
               ↪  [])]},
        'E2': {'factor_ids': list(e2.get('factors', {})), 'settings': e2.get('settings'),
               'rankable_ids': list(v2b_a_scores(e2, specs)),
               'deduplicated_ids': [row['id'] for row in e2.get('candidate_ranking',
               ↪  [])]},
        'B_native': {'factor_ids': list(V2B_NATIVE_V1_IDS), 'reused': reuse,
                     'reuse_decision_reason': reuse_reason},
        'B_expanded': {'factor_ids': expanded_ids,
                       'original24_semantic_NA_ids': [factor_id for factor_id in
                       ↪  original_ids if factor_id not in expanded_ids],
                       'excluded_new_proxy_ids': [factor_id for factor_id in specs if
                       ↪  factor_id not in original_ids]},
        'C': {'factor_ids': list(c_index), 'common_legal_original_ids': [factor_id for
        ↪  factor_id in original_ids if factor_id in c_ids],
              'rule': 'caller-computed fixed 168h D0 C10; annual>=15 endpoints and >=4
              ↪  years; source gap rule retained'},
        'pool_differences': 'A: factor-specific >=25 joint full-W endpoints/year, >=4
        ↪  years, annual equal-weight 3+1 calibration; B: pooled >=100 same-date
        ↪  references, Jun15-Sep30 pseudo pool, own full-W support, event-fitted
        ↪  entropy(A,-log10p,original mechanism); C: fixed7d with own valid support.
        ↪  Common IDs do not equal identical annual/pseudo observations or algorithms.',
    },
    'refit_scope': {
        'common_rule': 'rankable non-all-tied E2 intersect computable expanded-B TOPSIS
        ↪  intersect legally computable fixed7d-C, restricted to original24',
        'common_before_redundancy_ids': common_ids,
        'uniform_deduplicated_ids': dedup_ids,
        'redundancy_applied': applied, 'redundancy_not_applied_missing_representative':
        ↪  not_applied,
        'redundancy_source': 'E2 existing suppression: predeclared duplicated physical
        ↪  information AND sufficiently supported historical representative-window
        ↪  abs(Spearman rho)>=0.90; apply only when representative belongs to the same
        ↪  common pool',
        'B_fit_roles': ['B_expanded_original24', 'B_common_refit',
        ↪  'B_common_deduplicated_refit'],
        'counts_rule': 'each executed new fit requests 2000 main pseudo, 1000 year-block
        ↪  resamples and 500 pseudo per computable bootstrap distribution; actual draws
        ↪  and NA remain in its core rows; <2 input factors is not executed',
        'shared_input_scope': 'same per-factor definitions, full applicable W, C10/H-END
        ↪  D0 cache; observation/support differences remain explicit',
        'A_C_scope': 'A and C own scores retained; exact A fractions used to reindex
        ↪  ranks on listed IDs; no combined score',
        'E1_scope': 'original10 only; E1 newly restored-factor ranks remain NA and E1
        ↪  available-subset ranks are not a full common-family comparison',
    },
    'comparison_note': 'Native methods preserve their own pools/weights/support. B
    ↪  common-family entropy, TOPSIS and BH are real new fits; A/C rankings restrict
    ↪  unchanged own scores. New proxies remain in A/C native outputs, outside B and
    ↪  common-original24 tables. Rank agreement/overlap is not accuracy or proof of V2
    ↪  superiority.',
}
```

```
    return v2b_plain(result)


import argparse
import copy
import csv
import datetime as dt
import decimal
import gzip
import hashlib
import json
import math
import os
from pathlib import Path
import platform
import sys
import time

Q1_TZ=dt.timezone(dt.timedelta(hours=8))
Q1_ORIGIN=dt.datetime(2021,1,1,tzinfo=Q1_TZ)
Q1_GRID_LENGTH=52584
Q2_E1_IDS=('t2_mean','t2_max','pdd','warm_rate','vwc01','vwc02','soilt01','p_mean','rh2_mean'
↪  ,'ws10_mean')
Q2_PROXY_IDS=('t20_t2_diff','ws20_ws2_diff')
Q1_COLUMNS={f['id']:(f['raw_columns'][0]['table'],f['raw_columns'][0]['column'],f['raw_column
↪  s'][0].get('qc_column'),0)
            for f in Q1_FROZEN_DICTIONARY['factors'] if f.get('computable')}
Q2_OUTPUTS=('configuration.json','feature_dictionary.json','analysis_results.json','process_v
↪  2.json',
            'window_cache.json.gz','window_quality.csv.gz','all_factors.csv','primary_windows
            ↪  .csv',
            'single_sided_hypotheses.csv','experiments.csv','sensitivity.csv','loo_frequency.
            ↪  csv',
            'method_comparison.csv','c_fixed7d.csv','extremes.csv','process_support.csv')


def q1_features():
    d=copy.deepcopy(Q1_FROZEN_DICTIONARY)
    for f in d['factors']:
        f['windows_h']=f['window_hours']
        if f.get('computable'):
            assert f['direction']=='two_sided' and f['eligible_for_top5']
            f['source_grid_scale']=1;f['rank_scale']=1
            f['rank_encoding']='exact_source_grid_Fraction_then_lossless_integer_LCM'
    d['design_status']='看过 V1 目标结果后的研究修订；不称盲定或独立终评'
    return d


def q2_requested_columns(features):
    wanted={c['column'] for f in features if f.get('computable') for c in f['raw_columns']}
    wanted.update(p[1] for p in P2_CHANNELS)
    wanted.update(f'Ta_{z}m(degree C)' for z in P2_HEIGHTS)
    wanted.update(f'WS_{z}m(m/s)' for z in P2_HEIGHTS)
    return wanted
```

```
def q1_read_d0(data_dir,features):
    base=Path(data_dir);summary=json.loads((base/'run_summary_all.json').read_text())
    with (base/'attachment01_02/variable_dictionary.csv').open(encoding='utf-8-sig') as
    ↪  stream:
        metadata=list(csv.DictReader(stream))
    wanted=q2_requested_columns(features);arrays={};counts={}
    lookup={r['variable']:r for r in metadata if r['dataset'] in ('MET','SOIL','RADM','FLUX')
    ↪  and r['variable'] in wanted}
    assert set(lookup)==wanted,'所需 D0 变量/元数据不完整'
    for table in ('MET','SOIL','RADM','FLUX'):
        file=base/'attachment01_02/station'/(table+'.csv.gz')
        expected=summary['output_sha256']['attachment01_02/station/'+table+'.csv.gz']
        if isinstance(expected,dict):expected=expected['sha256']
        assert q1_sha(file)==expected,'D0 已有清理版本不匹配：'+table
        specs={col:r for col,r in lookup.items() if r['dataset']==table}
        for col,r in specs.items():
            no_qc=r['qc_column'] in ('not_provided','',None)
            arrays[col]={'values':np.zeros(Q1_GRID_LENGTH,dtype=object),'valid':np.zeros(Q1_G」
            ↪  RID_LENGTH,dtype=bool),'p':0,
                        'unit':r['unit'],'table':table,'qc_column':None if no_qc else
                        ↪  r['qc_column'],
                        'qc_identity':'not_provided' if no_qc else 'provided',
                        'qc_note':'源QC列未提供，不能推定QC9或QC0；沿用D0保留值' if no_qc else
                        ↪  'D0原QC0有效掩码，不恢复排除值'}
        seen=set()
        with gzip.open(file,'rt',encoding='utf-8-sig',newline='') as stream:
            for row in csv.DictReader(stream):
                i=q1_hour(row['timestamp_label']);assert i not in seen and 0<=i<Q1_GRID_LENGTH
                seen.add(i)
                for col,r in specs.items():
                    token=row[col]
                    if token=='':continue
                    source=arrays[col];qc=source['qc_column']
                    if qc is not None:assert decimal.Decimal(row[qc])==0,'D0 源 QC 政策不匹配'
                    number=decimal.Decimal(token);assert number.is_finite()
                    places=max(source['p'],-number.as_tuple().exponent)
                    if places>source['p']:
                        source['values']*=10**(places-source['p']);source['p']=places
                    exact=Fraction(number)*10**places;assert exact.denominator==1
                    source['values'][i]=exact.numerator;source['valid'][i]=True
        counts[table]={'labels':len(seen),'channels':{col:{'valid':int(arrays[col]['valid'].s」
        ↪  um()),'qc_identity':arrays[col]['qc_identity'],'qc_column':arrays[col]['qc_column」
        ↪  '],'decimal_places':arrays[col]['p']} for col in
        ↪  specs}}
    return arrays,counts


def q2_inputs(arrays,feature):
    g=feature['id']
    cached=arrays.setdefault('_q2_feature_cache',{})
    if g in cached:return cached[g]
```

```
    names=['Ta_20m(degree C)','Ta_2m(degree C)'] if g=='t20_t2_diff' else
    ↪  ['WS_20m(m/s)','WS_2m(m/s)'] if g=='ws20_ws2_diff' else
    ↪  [feature['raw_columns'][0]['column']]
    p=max(arrays[n]['p'] for n in names)
    raw=arrays[names[0]]['values']*10**(p-arrays[names[0]]['p'])
    mask=arrays[names[0]]['valid'].copy()
    if len(names)==2:
        raw=raw-arrays[names[1]]['values']*10**(p-arrays[names[1]]['p'])
        mask &= arrays[names[1]]['valid']
    cached[g]=(raw,mask,p,names)
    return cached[g]


def q1_window(arrays,feature,end,hours,endpoint,coverage):
    raw_all,valid_all,p,names=q2_inputs(arrays,feature)
    shift=int(endpoint=='H-END');lo,hi=end-hours+shift,end+shift
    meta={'expected':hours,'valid':0,'coverage':0.,'reason':None,'joint_columns':names}
    if lo<0 or hi>Q1_GRID_LENGTH:meta['reason']='outside_D0';return None,meta
    raw,mask=raw_all[lo:hi],valid_all[lo:hi];n=int(mask.sum());meta.update(valid=n,coverage=n
    ↪  /hours)
    if n<math.ceil(coverage*hours):meta['reason']='hour_coverage_below_threshold';return
    ↪  None,meta
    g=feature['id']
    if g in ('pdd','t2_max'):
        if n!=hours:meta['reason']='complete_interval_required';return None,meta
        value=Fraction(int(np.maximum(raw,0).sum())) if g=='pdd' else Fraction(int(raw.max()))
    elif g=='warm_rate':
        days=hours//24;xs=[];ys=[]
        for d in range(days):
            m=mask[d*24:(d+1)*24];r=raw[d*24:(d+1)*24]
            if int(m.sum())>=math.ceil(coverage*24):xs.append(Fraction(2*d+1,2));ys.append(Fr
            ↪  action(int(r[m].sum()),int(m.sum())))
        meta['daily_blocks_valid']=len(xs)
        if len(xs)<max(3,math.ceil(coverage*days)) or not xs or xs[0]!=Fraction(1,2) or
        ↪  xs[-1]!=Fraction(2*days-1,2):
            meta['reason']='slope_block_support_or_endpoints';return None,meta
        xm,ym=sum(xs)/len(xs),sum(ys)/len(ys)
        value=sum((x-xm)*(y-ym) for x,y in zip(xs,ys))/sum((x-xm)**2 for x in xs)
    else:value=Fraction(int(raw[mask].sum()),n)
    assert hi-shift==end and lo-shift==end-hours
    return value,meta


def q2_c_route(arrays,features,values):
    rows=[];gap_exclusions=[]
    for f in features:
        g=f['id'];data=values.get(g,{})
        def value(year,offset,encoded):
            if encoded is None:return None
            if g in ('pdd','t2_max'):return encoded
            _,mask,_,_=q2_inputs(arrays,f);end=q1_hour(f'{year}-08-26T10:00:00+08:00')+24*off
            ↪  set
            run=longest=0
            for valid in mask[end-167:end+1]:run=0 if valid else
            ↪  run+1;longest=max(longest,run)
```

```
            if longest>3:gap_exclusions.append({'id':g,'year':year,'offset':offset,'gap':long
            ↪  est});return
            ↪  None
            return encoded
        event=value(2026,0,data.get('target',{}).get('168'));annual={};all_values=[]
        for year,records in data.get('years',{}).items():
            xs=[value(int(year),int(o),r.get('168')) for o,r in records.items()];xs=[x for x
            ↪  in xs if x is not None]
            if len(xs)>=15 and event is not None:annual[year]={'n':len(xs),'cdf':float(a_midr
            ↪  ank_fraction(sorted(xs),event))};all_values.extend(xs)
        q=sum(x['cdf'] for x in annual.values())/len(annual) if len(annual)>=4 else None
        uninformative=bool(all_values) and len(set(all_values+[event]))==1
        rows.append({'id':g,'label':f['label'],'unit':f['unit'],'value':event/f['rank_scale']
        ↪  if event is not None else None,'q':q,'A':abs(2*q-1) if q is not None else
        ↪  None,'annual':annual,'rank_eligible':not
        ↪  uninformative,'uninformative':uninformative,'reason':None if q is not None else
        ↪  'definition_target_or_15window_4year_support_NA'})
    ranks={r['id']:r for r in q1_rank_rows({r['id']:r['A'] for r in rows if
    ↪  r['rank_eligible']})}
    for row in rows:row.update({k:v for k,v in ranks.get(row['id'],{}).items() if k in
    ↪  ('rank_min','rank_max')})
    return rows,{'mean_gap_rule_hours':3,'gap_exclusions':gap_exclusions,'cutoff_hour':10,'en
    ↪  dpoint':'H-END'}


def q2_evidence(features,result,arrays):
    rows=[];index={f['id']:f for f in features};end=q1_hour('2026-08-26T10:00:00+08:00')
    for g,factor in result['factors'].items():
        f=index[g];windows=[];directions=set();winning=[];all_counts={'high':0,'low':0,'cente
        ↪  r':0};mean_directions=set()
        for h,row in factor['windows'].items():
            q=row.get('q_mean');v=row['target_value']
            counts={'high':0,'low':0,'center':0}
            for configuration in factor['configurations']:
                cq=configuration['target']['q_by_window'][str(h)]
                cd='high' if cq>.5 else 'low' if cq<.5 else 'center'
                counts[cd]+=1;all_counts[cd]+=1
            if q is not None:mean_directions.add('high' if q>.5 else 'low' if q<.5 else
            ↪  'center')
            windows.append({'hours':int(h),'config_direction_counts':counts,'configuration_di
            ↪  rection_inconsistent':sum(n>0 for n in
            ↪  counts.values())>1,'physical_value':v/f['rank_scale'] if v is not None else
            ↪  None,'unit':f['unit'],'q_mean':q,'q_min':row.get('q_min'),'q_max':row.get('q_
            ↪  max'),'a_mean':row.get('a_mean'),'position_direction':'high' if q is not None
            ↪  and q>.5 else 'low' if q is not None and q<.5 else 'center' if q is not None
            ↪  else None,'raw_sign':'positive' if v is not None and v>0 else 'negative' if v
            ↪  is not None and v<0 else 'zero' if v==0 else None})
        for config in factor['configurations']:
            for h in config['target']['winning_windows']:
                q=config['target']['q_by_window'][str(h)];direction='high' if q>.5 else 'low'
                ↪  if q<.5 else 'center';directions.add(direction)
                v=factor['windows'][str(h)]['target_value']
                winning.append({'calibration_year':config['calibration_year'],'reference_year
                ↪  s':config['reference_years'],'hours':h,'q':q,'direction':direction,'physi
                ↪  cal_value':v/f['rank_scale']})
```

```
        first,m1=q1_window(arrays,f,end-696,24,'H-END',.9) if g!='warm_rate' else (None,{})
        last,m2=q1_window(arrays,f,end,24,'H-END',.9) if g!='warm_rate' else (None,{})
        rows.append({'id':g,'label':f['label'],'unit':f['unit'],'S':factor['score_S'],'window
        ↪  s':windows,'winning_configurations':winning,'winning_directions':sorted(direction
        ↪  s),'direction_inconsistent_across_winning_windows_or_configs':len(directions)>1,'
        ↪  all_window_mean_directions':sorted(mean_directions),'all_config_window_direction_
        ↪  counts':all_counts,'direction_inconsistent_across_all_windows_or_configs':sum(n>0
        ↪  for n in
        ↪  all_counts.values())>1,'first24_of30d':float(first/f['source_grid_scale']) if
        ↪  first is not None else None,'last24':float(last/f['source_grid_scale']) if last
        ↪  is not None else
        ↪  None,'last_minus_first24':float((last-first)/f['source_grid_scale']) if first is
        ↪  not None and last is not None else None,'first_last_support':[m1,m2]})
    return rows


def q2_original_specs(features):
    specs=[]
    for f in features:
        if f['id'] not in Q2_E1_IDS:continue
        s=copy.deepcopy(f);s['direction']=f['hypothesis_direction'];s['eligible_for_top5']=f[
        ↪  'id'] not in
        ↪  ('p_mean','rh2_mean','ws10_mean')
        specs.append(s)
    return specs


def q2_hypotheses(features,old_policy):
    rows=[]
    for f in features:
        if f['id'] not in Q2_E1_IDS or f.get('hypothesis_direction')!='high':continue
        r=old_policy['factors'][f['id']];s=r['score_S']
        rows.append({'id':f['id'],'hypothesis':'原V1偏高假说，
        ↪  不反转','direction':'high','S_one_sided':s,'description':'相对突出线索' if s is not
        ↪  None and s>=.8 else '较弱线索' if s is not None and s>=.5 else '未见偏高支持' if s
        ↪  is not None else '支持不足NA','rule':'描述性分级：S>=.8/.5；不是p检验，
        ↪  不参与V2主榜','windows':r['windows']})
    return rows


def q2_validate_v1(args,cache,features,data,before):
    if args.v1_results is None:return None,None,[]
    directory=Path(args.v1_results);files=[directory/'analysis_results.json',directory/'windo
    ↪  w_cache.json.gz',directory/'RUN_RECEIPT.json']
    if not all(p.is_file() for p in files):raise ValueError('指定 V1 结果目录缺少已声明复用文件')
    baseline=json.loads(files[0].read_text());receipt=json.loads(files[2].read_text())
    assert receipt['inputs_unchanged'] and baseline['schema']=='Q1_FORMAL_V1'
    assert json.loads((directory/'configuration.json').read_text())['seed']==args.seed

    required=['run_summary_all.json','attachment01_02/variable_dictionary.csv',
              'attachment01_02/station/MET.csv.gz','attachment01_02/station/SOIL.csv.gz']
    verified_inputs=[]
    for relative in required:
```

```
        matches=[x for x in receipt['inputs_before'] if x['path'].endswith('/'+relative)]
        assert len(matches)==1 and matches[0]['sha256']==q1_sha(data/relative), 'V1 D0 source
        ↪  fingerprint differs'
        verified_inputs.append({'input':relative,'sha256':matches[0]['sha256']})
    for name in ('analysis_results.json','window_cache.json.gz','configuration.json'):
        matches=[x for x in receipt['outputs'] if x['path'].endswith('/'+name)]
        assert len(matches)==1 and matches[0]['sha256']==q1_sha(directory/name), 'V1 reusable
        ↪  result fingerprint differs'
    with gzip.open(files[1],'rt',encoding='utf-8') as stream:old=json.load(stream)
    old_scale={f['id']:f['rank_scale'] for f in baseline['features'] if f['id'] in
    ↪  Q2_E1_IDS};new_scale={f['id']:f['rank_scale'] for f in features if f['id'] in
    ↪  Q2_E1_IDS}
    n=0
    for case,records in cache.items():
        ep,ch,cov=case;old_case=old[f'{ep}_C{ch}_{cov}']
        for g in Q2_E1_IDS:
            current=records[g];previous=old_case[g]
            rows=[(current['target'],previous['target'])]
            rows.extend((current['seasonal_years'][y][md],previous['seasonal_years'][y][md])
            ↪  for y in current['seasonal_years'] for md in current['seasonal_years'][y])
            for a,b in rows:
                assert set(a)==set(b)
                for h,value in a.items():
                    other=b[h];assert (value is None)==(other is None)
                    if value is not None:assert value*old_scale[g]==other*new_scale[g]
                    n+=1
    bindings=[q1_fp(p) for p in files+[directory/'configuration.json']]
    return baseline,{'status':'pass','file_sha':q1_sha(files[0]),'verified_controls':['D0
    ↪  exact input hashes below','same seed, variables, formulas,
    ↪  endpoints/cutoffs/coverage','all original10 target and seasonal physical Fraction
    ↪  values identical'],'physical_window_comparisons':n,'verified_D0_inputs':verified_inpu
    ↪  ts,'source_bindings':bindings},files+[directory/'configuration.json']


def q2_write_tables(res,result,process):
    specs=result['features'];primary=result['primary'];rank={r['id']:r for r in
    ↪  primary['ranking_all']};merged={x['id']:x['representative_id'] for x in
    ↪  primary['redundancy']['suppressed']};rows=[];win=[]
    for f in specs:
        g=f['id'];r=primary['factors'].get(g,{});rr=rank.get(g,{})
        rows.append({'id':g,'label':f['label'],'unit':f['unit'],'definition_status':f['status
        ↪  '],'statistic_status':r.get('status','definition_NA'),'rank_eligible':r.get('rank
        ↪  _eligible',False),'merged_into':merged.get(g),'direction':f['direction'],'S':r.ge
        ↪  t('score_S'),'rank_min':rr.get('rank_min'),'rank_max':rr.get('rank_max'),'top_can
        ↪  didate':g in primary['top_candidates'],'boundary':g in
        ↪  primary['boundary_candidates'],'years':r.get('support',{}).get('valid_years'),'an
        ↪  nual_counts':r.get('support',{}).get('n_valid_endpoints_by_year'),'reason':r.get(
        ↪  'reason') or (f.get('status_reason') if not f.get('computable') else
        ↪  None)})
        for h,w in r.get('windows',{}).items():win.append({'id':g,'hours':h,'value':w['target
        ↪  _value']/f['rank_scale'] if w['target_value'] is not None else
        ↪  None,'unit':f['unit'],**{k:v for k,v in w.items() if k!='target_value'}})
    q1_csv(res/'all_factors.csv',rows);q1_csv(res/'primary_windows.csv',win)
    q1_csv(res/'single_sided_hypotheses.csv',result['single_sided_hypotheses'])
```

```
    experiments=[{'experiment':k,'support_min':r.get('min_endpoints',25),'computable':sum(f['
    ↪  score_S'] is not None for f in
    ↪  r['result']['factors'].values()),'pool_after_redundancy':len(r['result']['candidate_r
    ↪  anking']),'top':r['result']['top_candidates'],'boundary':r['result']['boundary_candid
    ↪  ates']} for k,r in
    ↪  result['experiments'].items()]
    q1_csv(res/'experiments.csv',experiments)
    q1_csv(res/'sensitivity.csv',[{'branch':b['name'],'min_endpoints':b.get('min_endpoints',2
    ↪  5),'computable':sum(f['score_S'] is not None for f in
    ↪  b['result']['factors'].values()),'pool_after_redundancy':len(b['result']['candidate_r
    ↪  anking']),'top':b['result']['top_candidates'],'boundary':b['result']['boundary_candid
    ↪  ates']} for b in
    ↪  result['branches']])
    q1_csv(res/'loo_frequency.csv',[{'id':g,**r} for g,r in
    ↪  result['loo']['selection_frequency'].items()])
    q1_csv(res/'method_comparison.csv',result['comparisons_v2']);q1_csv(res/'c_fixed7d.csv',r
    ↪  esult['methods']['C_factors'])
    q1_csv(res/'extremes.csv',[{'id':e['id'],'label':e['label'],'unit':e['unit'],'period':per
    ↪  iod,**summary} for e in process['extremes']['series'] for period,summary in
    ↪  e['summaries'].items()])
    q1_csv(res/'process_support.csv',[{'channel':c['column'] if 'column' in c else
    ↪  c.get('id'),'details':c.get('support',c.get('summaries',{}))} for c in
    ↪  process.get('weather',{}).get('channels',[])])


def q1_execute(args):
    started=dt.datetime.now(dt.timezone.utc).isoformat();t0=time.monotonic();out=Path(args.ou
    ↪  tput_dir);res=out/'results';res.mkdir(parents=True,exist_ok=True)
    data=Path(args.data_dir);dictionary=q1_features();features=dictionary['factors'];active=[
    ↪  f for f in features if
    ↪  f.get('computable')]
    inputs=[Path(__file__), Path(__file__).resolve().parent / 'data/features.json',
    ↪  Path(__file__).resolve().parent /
    ↪  'data/reference.json',data/'run_summary_all.json',data/'attachment01_02/variable_dict
    ↪  ionary.csv']+[data/'attachment01_02/station'/(t+'.csv.gz') for t in
    ↪  ('MET','SOIL','RADM','FLUX')]
    if args.v1_results:inputs += [Path(args.v1_results)/n for n in ('analysis_results.json','
    ↪  window_cache.json.gz','RUN_RECEIPT.json','configuration.json')]
    before=[q1_fp(p) for p in inputs];arrays,counts=q1_read_d0(data,features)
    for f in active:
        _,_,places,_=q2_inputs(arrays,f);f['source_grid_scale']=10**places;f['rank_scale']=10
        ↪  **places
    ledger=[];cache={}
    for ep,ch,cov in [('H-END',10,.9),('H-START',10,.9),('H-END',0,.9),('H-START',0,.9),('H-E
    ↪  ND',10,.95),('H-END',9,.9)]:
        print('V2_WINDOW_CACHE',ep,ch,cov,flush=True);cache[(ep,ch,cov)]=q1_cache(arrays,acti
        ↪  ve,ep,ch,cov,ledger)
    q1_encode_exact_cache(cache,features);q1_dump(res/'feature_dictionary.json',dictionary)
    with gzip.open(res/'window_quality.csv.gz','wt',encoding='utf-8',newline='') as stream:
        wr=csv.DictWriter(stream,fieldnames=sorted({k for r in ledger for k in
        ↪  r}));wr.writeheader();wr.writerows(ledger)
    del ledger
    print('FIXED_MULTILEVEL_WEATHER_EXTREMES',flush=True)
    process=build_processes({k:v for k,v in arrays.items() if k!='_q2_feature_cache'},endpoin
    ↪  t='H-END',cutoff_hour=10);q1_dump(res/'process_v2.json',process)
```

```
primary_values=cache[('H-END',10,.9)];e1spec=[f for f in active if f['id'] in Q2_E1_IDS]
print('E1_ALL_TWO_SIDED',flush=True);e1=summarize_one(e1spec,primary_values)
print('E2_EXPANDED_ALL_TWO_SIDED',flush=True);primary=summarize_one(active,primary_values)
assert all(e1['factors'][g]['score_S_exact']==primary['factors'][g]['score_S_exact'] for
↪  g in e1['factors'])
old_policy=summarize_one(q2_original_specs(features),primary_values)
experiments={'E0_V1_policy_reproduced':{'min_endpoints':25,'result':q1_trim_a(old_policy)
↪  },'E1_original10_two_sided':{'min_endpoints':25,'result':q1_trim_a(e1)},'E2_expanded_
↪  main':{'min_endpoints':25,'result':q1_trim_a(primary)}}
branches=[]
for ep,ch,cov in
↪  [('H-END',10,.9),('H-START',10,.9),('H-END',0,.9),('H-START',0,.9),('H-END',10,.95)]:
    r=primary if (ep,ch,cov)==('H-END',10,.9) else
    ↪  summarize_one(active,cache[(ep,ch,cov)])
    branches.append({'name':f'{ep}_C{ch:02d}_coverage{int(cov*100)}','endpoint':ep,'cutof
    ↪  f_hour':ch,'coverage':cov,'window_mode':'full','min_endpoints':25,'result':q1_tri
    ↪  m_a(r)})
reduced=copy.deepcopy(active)
for f in reduced:f['window_hours']=[w for w in f['window_hours'] if w!=720]
branches.append({'name':'drop30d_all_factors','window_mode':'drop30d','min_endpoints':25,
↪  'result':q1_trim_a(summarize_one(reduced,primary_values))})
for cutoff in (20,15):
    for name,spec in [('E1',e1spec),('E2',active)]:
        r=summarize_one(spec,primary_values,min_endpoints=cutoff)
        experiments[f'{name}_uniform_min{cutoff}']={'min_endpoints':cutoff,'result':q1_tr
        ↪  im_a(r)}
        if name=='E2':branches.append({'name':f'all_factors_min{cutoff}','min_endpoints':
        ↪  cutoff,'window_mode':'full','result':q1_trim_a(r)})
for rho in (.85,.95):branches.append({'name':f'rho{rho}','min_endpoints':25,'result':q1_t
↪  rim_a(summarize_one(active,primary_values,redundancy_threshold=rho))})
common,common_info=q1_common_support(active,primary_values,primary)
branches.append({'name':'common_history_support','min_endpoints':25,'common_support':comm
↪  on_info,'result':q1_trim_a(summarize_one(active,common))})
loo=leave_one_year_out(active,primary_values)
for k in loo['runs']:loo['runs'][k]=q1_trim_a(loo['runs'][k])
c_rows,c_quality=q2_c_route(arrays,features,primary_values)
baseline,validation,_=q2_validate_v1(args,cache,features,data,before)
print('B_NATIVE_AND_ACTUAL_COMMON_REFITS',flush=True)
methods=build_v2_comparisons(features,primary_values,e1,primary,c_rows,baseline=baseline,
↪  baseline_validation=validation,seed=args.seed)
methods['methods']['C_factors']=c_rows;methods['methods']['C_quality']=c_quality
result={'schema':'Q1_FORMAL_V2','research_revision_after_V1':True,'features':features,'pr
↪  imary_branch':{'name':'E2_H-END_C10_min25','endpoint':'H-END','cutoff_hour':10,'cover
↪  age':.9,'min_endpoints':25},'primary':primary,'experiments':experiments,'branches':br
↪  anches,'loo':loo,'single_sided_hypotheses':q2_hypotheses(features,old_policy),'factor
↪  _evidence':q2_evidence(features,primary,arrays),'data_counts':counts,'common_support'
↪  :common_info,'baseline_validation':validation,**methods,'scientific_acceptance':'pend
↪  ing V2 independent total
↪  review'}
```

```
    q1_dump(res/'configuration.json',{'seed':args.seed,'main_support_min':25,'support_sensiti」
    ↪  vities_all_factors':[20,15],'windows_hours':[24,72,168,360,720],'primary':'E2 C10
    ↪  H-END cov90 min25','revision_after_V1':True,'not_independent_2026_evaluation':True,'f」
    ↪  ixed_process_variables':'23 channels plus five-level T/U, not selected by
    ↪  ranking','fixed_thresholds':'T2 lowP05/highP95 and WS10 highP95; five-year
    ↪  equal-weight inverse empirical
    ↪  CDF','B_counts':[2000,1000,500],'baseline_validation':validation})
    q1_dump(res/'analysis_results.json',result);q1_dump(res/'process_v2.json',process)
    with gzip.open(res/'window_cache.json.gz','wt',encoding='utf-8') as
    ↪  stream:json.dump({f'{ep}_C{ch}_{cov}':v for (ep,ch,cov),v in
    ↪  cache.items()},stream,ensure_ascii=False,allow_nan=False)
    q2_write_tables(res,result,process)
    after=[q1_fp(p) for p in inputs];assert before==after,'实际输入/代码漂移'
    receipt={'record_type':'actual_V2_analysis_process','started_utc':started,'finished_utc':」
    ↪  dt.datetime.now(dt.timezone.utc).isoformat(),'elapsed_seconds':time.monotonic()-t0,'p」
    ↪  id':os.getpid(),'argv':sys.argv,'environment':{'python':sys.version,'executable':'pro」
    ↪  ject .venv
    ↪  Python','executable_sha256':q1_sha(Path(sys.executable).resolve()),'numpy':np.__versi」
    ↪  on__,'scipy':__import__('scipy').__version__,'platform':platform.platform(),'PYTHONDO」
    ↪  NTWRITEBYTECODE':os.environ.get('PYTHONDONTWRITEBYTECODE')},'inputs_before':before,'i」
    ↪  nputs_after':after,'inputs_unchanged':True,'outputs':[q1_fp(res/name) for name in
    ↪  Q2_OUTPUTS],'output_scope':'explicit analysis outputs; excludes all later receipts
    ↪  and rendering/writing files','status':'executed_pending_V2_independent_review'}
    q1_dump(res/'RUN_RECEIPT.json',receipt)
    print('V2_COMPLETE',json.dumps({'top':primary['top_candidates'],'boundary':primary['bound」
    ↪  ary_candidates'],'S':{g:r['score_S'] for g,r in
    ↪  primary['factors'].items()}},ensure_ascii=False),flush=True)


P3_FROZEN_V2 = json.loads((Path(__file__).resolve().parent / 'data' /
↪  'reference.json').read_text(encoding='utf-8'))


from itertools import combinations as p3_combinations
import math as p3_math
import numpy as np

P3_MIN_ENDPOINTS = 25
P3_MIN_YEARS = 4
P3_RETAINED_VARIANCE = 0.85
P3_RANK_FACTOR = 1.0
P3_NEAR_ZERO_FACTOR = 64.0
P3_ADD_RTOL = 1e-10
P3_ADD_ATOL = 1e-12
P3_BRANCHES = (('main', 0), ('k_minus1', -1), ('k_plus1', 1))


def _p3_plain(value):
    if isinstance(value, dict):
        return {str(k): _p3_plain(v) for k, v in value.items()}
    if isinstance(value, (tuple, list, np.ndarray)):
        return [_p3_plain(v) for v in value]
    if isinstance(value, (bool, np.bool_)):
        return bool(value)
```

```
    if isinstance(value, (int, np.integer)):
        return int(value)
    if isinstance(value, (float, np.floating)):
        number = float(value)
        return number if p3_math.isfinite(number) else None
    return value


def _p3_midrank(reference, value):

    x = np.asarray(reference, dtype=np.float64)
    if not p3_math.isfinite(float(value)) or not len(x) or not np.isfinite(x).all():
        return None
    return float((np.count_nonzero(x < value) + .5 * np.count_nonzero(x == value)) / len(x))


def _p3_ranks(values):

    x = np.asarray(values, dtype=np.float64)
    return [{'rank_min': int(1 + np.count_nonzero(x > v)),
             'rank_max': int(np.count_nonzero(x >= v))} for v in x]


def _p3_fit(reference_years, matrices):
    p = next(iter(matrices.values())).shape[1]
    blocks = [matrices[y] for y in reference_years]
    n = sum(len(x) for x in blocks)
    x = np.vstack(blocks)
    w = np.concatenate([np.full(len(block), 1. / (3 * len(block))) for block in blocks])
    assert len(reference_years) == 3 and np.isclose(w.sum(), 1., rtol=0., atol=1e-14)

    anchor = x[0].copy()
    mu = anchor + np.sum((x - anchor) * w[:, None], axis=0)
    d = x - mu
    variance = np.sum(w[:, None] * d * d, axis=0)
    base = {'reference_years': list(reference_years), 'training_rows': n,
            'training_rows_by_year': {y: len(matrices[y]) for y in reference_years},
            'row_weight_by_year': {y: 1. / (3 * len(matrices[y])) for y in reference_years},
            'weight_sum': float(w.sum()), 'mu': mu, 'variance': variance,
            'moment_convention': 'population weighted moment ddof=0; each reference year
            ↪  total weight 1/3'}
    if not np.isfinite(mu).all() or not np.isfinite(variance).all():
        return {**base, 'status': 'NA', 'reason': 'nonfinite_training_moments'}
    zero = np.flatnonzero(variance <= 0).tolist()
    if zero:
        return {**base, 'status': 'NA', 'reason':
        ↪  'training_zero_variance_no_variables_deleted',
                'zero_variance_indices': zero}
    sigma = np.sqrt(variance)
    z = d / sigma
    covariance = (z * w[:, None]).T @ z
    covariance = (covariance + covariance.T) * .5
    raw_values, vectors = np.linalg.eigh(covariance)
    order = np.argsort(raw_values)[::-1]
```

```
    raw_values, vectors = raw_values[order], vectors[:, order]

    for j in range(p):
        pivot = int(np.argmax(np.abs(vectors[:, j])))
        if vectors[pivot, j] < 0:
            vectors[:, j] *= -1
    largest = float(max(0., raw_values[0]))
    tolerance = P3_RANK_FACTOR * max(n, p) * np.finfo(np.float64).eps * largest
    negative = np.flatnonzero(raw_values < 0).tolist()
    substantial_negative = np.flatnonzero(raw_values < -tolerance).tolist()
    base.update(sigma=sigma, standardized_covariance=covariance,
                raw_eigenvalues=raw_values, eigenvectors=vectors,
                eigenvalue_rank_tolerance=tolerance,
                negative_eigenvalue_indices=negative,
                substantial_negative_eigenvalue_indices=substantial_negative,
                weighted_standardized_center=np.sum(w[:, None] * z, axis=0),
                eigendecomposition_max_abs_error=float(np.max(np.abs(covariance @ vectors -
                ↪  vectors * raw_values))),
                orthogonality_max_abs_error=float(np.max(np.abs(vectors.T @ vectors -
                ↪  np.eye(p)))))
    if substantial_negative or largest <= 0:
        return {**base, 'status': 'NA', 'reason': 'invalid_nonpositive_covariance_spectrum'}
    eigenvalues = np.maximum(raw_values, 0.)
    effective_rank = int(np.count_nonzero(eigenvalues > tolerance))
    if effective_rank < 1:
        return {**base, 'status': 'NA', 'reason': 'effective_rank_zero', 'effective_rank': 0}

    total = float(np.sum(eigenvalues[:effective_rank]))
    explained = np.zeros_like(eigenvalues)
    explained[:effective_rank] = eigenvalues[:effective_rank] / total
    cumulative = np.cumsum(explained)
    main_k = min(effective_rank, int(np.searchsorted(cumulative[:effective_rank],
    ↪  P3_RETAINED_VARIANCE, side='left') + 1))
    base.update(status='ok', reason=None, eigenvalues=eigenvalues,
                effective_rank=effective_rank, main_k=main_k,
                explained_variance_ratio=explained,
                cumulative_explained_variance=cumulative,
                discarded_numerically_near_zero_eigenvalue_indices=list(range(effective_rank,
                ↪  p)),
                retained_variance_at_main_k=float(cumulative[main_k - 1]),
                loading_sign_rule='largest absolute component in each eigenvector is
                ↪  positive; no cross-model loading averaging')
    return base


def _p3_score(model, physical_values, k):
    x = np.asarray(physical_values, dtype=np.float64)
    x = np.atleast_2d(x)
    z = (x - model['mu']) / model['sigma']
    p = z.shape[1]
    pk = model['eigenvectors'][:, :k]
    lam = model['eigenvalues'][:k]
    t = z @ pk
    t2 = np.sum(t * t / lam, axis=1)
```

```
    residual = z - t @ pk.T
    q = np.sum(residual * residual, axis=1)
    ct = z * ((t / lam) @ pk.T)
    cq = residual * residual
    ct_error = np.abs(ct.sum(axis=1) - t2)
    cq_error = np.abs(cq.sum(axis=1) - q)
    if not (np.isfinite(z).all() and np.isfinite(t2).all() and np.isfinite(q).all()
            and np.isfinite(ct).all() and np.isfinite(cq).all()):
        raise FloatingPointError('nonfinite_PCA_score_or_contribution')
    if not np.allclose(ct.sum(axis=1), t2, rtol=P3_ADD_RTOL, atol=P3_ADD_ATOL):
        raise ArithmeticError('T2_contributions_do_not_add')
    if not np.allclose(cq.sum(axis=1), q, rtol=P3_ADD_RTOL, atol=P3_ADD_ATOL):
        raise ArithmeticError('Q_contributions_do_not_add')
    near_zero = P3_NEAR_ZERO_FACTOR * np.finfo(np.float64).eps * np.maximum(1., np.sum(z * z,
    ↪  axis=1))
    shares_t, shares_q = [], []
    for i in range(len(x)):
        shares_t.append((ct[i] / t2[i]).tolist() if t2[i] > near_zero[i] else None)
        shares_q.append((cq[i] / q[i]).tolist() if k < p and q[i] > near_zero[i] else None)
    return {'z': z, 'scores': t, 'residual': residual, 'T2': t2, 'Q': q, 'cT': ct, 'cQ': cq,
            'share_T2': shares_t, 'share_Q': shares_q,
            'rank_T2': [_p3_ranks(v) for v in ct], 'rank_Q': [_p3_ranks(v) for v in cq],
            'Q_informative': k < p,
            'Q_reason': None if k < p else 'full_dimension_residual_degenerate',
            'near_zero_share_threshold': near_zero,
            'max_contribution_sum_error_T2': float(ct_error.max(initial=0.)),
            'max_contribution_sum_error_Q': float(cq_error.max(initial=0.))}


def _p3_target_record(score):
    fields = ('z', 'scores', 'residual', 'T2', 'Q', 'cT', 'cQ', 'share_T2', 'share_Q',
    ↪  'rank_T2', 'rank_Q',
              'near_zero_share_threshold')
    result = {key: score[key][0] for key in fields}
    result.update(Q_informative=score['Q_informative'], Q_reason=score['Q_reason'],
                  max_contribution_sum_error_T2=score['max_contribution_sum_error_T2'],
                  max_contribution_sum_error_Q=score['max_contribution_sum_error_Q'])
    return result


def _p3_equal_h(records, get_value):

    groups = {}
    for rec in records:
        value = get_value(rec)
        if value is not None and p3_math.isfinite(float(value)):
            groups.setdefault(rec['calibration_year'], []).append(float(value))
    by_year = [{'year': y, 'mean': float(np.mean(v)), 'min': min(v), 'max': max(v),
                'valid_configurations': len(v)} for y, v in sorted(groups.items())]
    values = [v for group in groups.values() for v in group]
    return {'mean': float(np.mean([r['mean'] for r in by_year])) if by_year else None,
            'min': min(values) if values else None, 'max': max(values) if values else None,
            'calibration_years': by_year, 'valid_configurations': len(values),
            'valid_calibration_years': len(by_year)}
```

```
def _p3_summary(configurations, ids, branch):
    records = [c for c in configurations if c['k_branch'] == branch]
    valid = [c for c in records if c['status'] == 'ok']
    result = {'k_branch': branch, 'expected_configurations': len(records),
              'valid_configurations': len(valid), 'NA_configurations': len(records) -
              ↪  len(valid),
              'metrics': {}, 'contributions': {}}
    for metric, ckey, skey in (('T2', 'cT', 'share_T2'), ('Q', 'cQ', 'share_Q')):
        agg = _p3_equal_h(valid, lambda c: c['positions'][metric])
        by_h = [{**v, 'position_mean': v['mean'], 'position_min': v['min'], 'position_max':
        ↪  v['max']}
                for v in agg['calibration_years']]
        result['metrics'][metric] = {'position_mean': agg['mean'], 'position_min': agg['min'],
                                     'position_max': agg['max'], 'calibration_years': by_h,
                                     'valid_configurations': agg['valid_configurations'],
                                     'valid_calibration_years':
                                     ↪  agg['valid_calibration_years'],
                                     'NA_configurations': len(records) -
                                     ↪  agg['valid_configurations'],
                                     'interpretation': 'held-out empirical middle-rank
                                     ↪  position; not p/probability/accuracy'}
        rows = []
        for j, factor_id in enumerate(ids):
            pos = _p3_equal_h(valid, lambda c: c['positions']['contribution_' + metric][j])
            share = _p3_equal_h(valid, lambda c: c['target'][skey][j] if c['target'][skey] is
            ↪  not None else None)
            zstat = _p3_equal_h(valid, lambda c: c['target']['z'][j])

            rank_records = [c for c in valid if metric != 'Q' or c['target']['Q_informative']]
            ranks = [c['target']['rank_' + metric][j] for c in rank_records]
            negative = sum(c['target'][ckey][j] < 0 for c in rank_records)
            rows.append({'id': factor_id,
                         'normalized_share_mean': share['mean'], 'normalized_share_min':
                         ↪  share['min'],
                         'normalized_share_max': share['max'], 'share_valid_configurations':
                         ↪  share['valid_configurations'],
                         'share_valid_calibration_years': share['valid_calibration_years'],
                         'position_mean': pos['mean'], 'position_min': pos['min'],
                         ↪  'position_max': pos['max'],
                         'position_calibration_years': pos['calibration_years'],
                         'rank_min': min((v['rank_min'] for v in ranks), default=None),
                         'rank_max': max((v['rank_max'] for v in ranks), default=None),
                         'negative_configurations': negative, 'valid_configurations':
                         ↪  len(rank_records),
                         'z_mean': zstat['mean'], 'z_min': zstat['min'], 'z_max':
                         ↪  zstat['max'],
                         'z_positive_configurations': sum(c['target']['z'][j] > 0 for c in
                         ↪  valid),
                         'z_negative_configurations': sum(c['target']['z'][j] < 0 for c in
                         ↪  valid),
                         'share_interpretation': 'signed cT/T2' if metric == 'T2' else 'cQ/Q',
                         'raw_contributions_not_averaged': True})
```

```
        result['contributions'][metric] = rows
    result['k_values'] = sorted(set(c['k'] for c in valid))
    return result


def _p3_structure_summary(models, ids):
    valid = [m for m in models if m['status'] == 'ok']
    rows = []
    for i, j in p3_combinations(range(len(ids)), 2):
        xs = [float(m['standardized_covariance'][i, j]) for m in valid]
        rows.append({'id_a': ids[i], 'id_b': ids[j], 'training_fits': len(xs),
                     'correlation_mean': float(np.mean(xs)) if xs else None,
                     'correlation_min': min(xs) if xs else None, 'correlation_max': max(xs)
                     ↪  if xs else None,
                     'positive_fits': sum(x > 0 for x in xs), 'negative_fits': sum(x < 0 for
                     ↪  x in xs),
                     'interpretation': 'equal-year training standardized covariance; each
                     ↪  distinct R counted once; no PC loading average'})
    return rows


def build_pca_v3(feature_ids, matrices_by_year, targets_by_scale, feature_meta=None):


    ids = [str(v) for v in feature_ids]
    if len(ids) < 2 or len(set(ids)) != len(ids):
        raise ValueError('PCA feature IDs must be unique and contain at least two factors')
    p = len(ids)
    result = {'schema': 'Q1_PCA_V3.1', 'feature_ids': ids, 'feature_meta': feature_meta,
              'settings': {'scales_hours': [168, 720], 'min_endpoints_per_year':
              ↪  P3_MIN_ENDPOINTS,
                           'minimum_years': P3_MIN_YEARS, 'reference_years_per_fit': 3,
                           'retained_variance_main': P3_RETAINED_VARIANCE,
                           'retained_variance_role': 'operational convention fixed for this
                           ↪  post-V2 exploration, not accuracy or official standard',
                           'moment_convention': 'population weighted variance/covariance
                           ↪  ddof=0; row weight 1/(3*n_y)',
                           'rank_rule': 'eigenvalue >
                           ↪  max(n_training_rows,p)*float64_eps*lambda_max',
                           'rank_factor': P3_RANK_FACTOR, 'small_negative_rule': 'record and
                           ↪  clip [-rank_tol,0) to zero; below -rank_tol is NA',
                           'zero_variance_rule': 'whole reference fit NA; no variable
                           ↪  deleted',
                           'Q_degenerate_rule': 'k==p: raw roundoff residual recorded, Q
                           ↪  position/contribution position/share/rank are NA',
                           'share_near_zero_rule': 'total <=64*eps*max(1,sum(z^2)): share NA,
                           ↪  raw total/contribution retained',
                           'contribution_sum_tolerance': {'rtol': P3_ADD_RTOL, 'atol':
                           ↪  P3_ADD_ATOL},
                           'aggregation': 'equal R combinations within calibration year, then
                           ↪  equal calibration years; raw totals and contributions never
                           ↪  averaged across models',
                           'sensitivity': 'only k-1/k+1 inside [1,effective_rank]; reuse same
                           ↪  decomposition',
```

```
                    'scope': 'auxiliary linear PCA after seeing V2; no independent
                    ↪  disaster evaluation'},
          'scales': {}}
for hours in (168, 720):
    source = matrices_by_year.get(hours, matrices_by_year.get(str(hours), {}))
    target = targets_by_scale.get(hours, targets_by_scale.get(str(hours)))
    matrices, excluded = {}, []
    for year, rows in sorted(source.items(), key=lambda q: int(q[0])):
        y = str(year)
        if int(y) not in range(2021, 2026):
            raise ValueError('Only 2021--2025 historical years may enter PCA
            ↪  fitting/calibration')
        x = np.asarray(rows, dtype=np.float64)
        if x.ndim != 2 or x.shape[1] != p:
            raise ValueError('Each historical PCA matrix must preserve the complete
            ↪  feature vector/order')
        if not np.isfinite(x).all():
            raise ValueError('PCA common-support matrices must contain no
            ↪  missing/nonfinite values')
        if len(x) < P3_MIN_ENDPOINTS:
            excluded.append({'year': y, 'rows': len(x), 'reason':
            ↪  'common_endpoints_below_25'})
        else:
            matrices[y] = x
    years = tuple(sorted(matrices, key=int))
    scale = {'hours': hours, 'feature_ids': ids, 'years': list(years),
             'rows_by_year': {y: len(matrices[y]) for y in years}, 'excluded_years':
             ↪  excluded,
             'status': 'pending', 'models': [], 'configurations': [], 'summaries': {},
             ↪  'structure_summary': []}
    result['scales'][str(hours)] = scale
    if len(years) < P3_MIN_YEARS:
        scale.update(status='NA', reason='fewer_than_four_common_support_years')
        continue
    tx = np.asarray(target, dtype=np.float64) if target is not None else np.array([])
    if tx.shape != (p,) or not np.isfinite(tx).all():
        scale.update(status='NA', reason='target_complete_physical_vector_unavailable')
        continue
    scale.update(status='ok', reason=None, target_physical=tx)
    model_cache = {}
    for reference_years in p3_combinations(years, 3):
        model_id = f'h{hours}_R' + '-'.join(reference_years)
        try:
            with np.errstate(over='raise', invalid='raise', divide='raise'):
                model = _p3_fit(reference_years, matrices)
        except (FloatingPointError, np.linalg.LinAlgError) as error:
            model = {'status': 'NA', 'reason': type(error).__name__ + ':' + str(error),
                     'reference_years': list(reference_years)}
        model['model_id'] = model_id
        model_cache[reference_years] = model
        scale['models'].append(model)
    for h in years:
        for reference_years in p3_combinations(tuple(y for y in years if y != h), 3):
            assert h not in reference_years
```

```
        model = model_cache[reference_years]
        for branch, delta in P3_BRANCHES:
            rec = {'id': f'h{hours}_h{h}_R' + '-'.join(reference_years) + '_' +
            ↪  branch,
                   'hours': hours, 'reference_years': list(reference_years),
                   'calibration_year': h, 'model_id': model['model_id'], 'k_branch':
                   ↪  branch,
                   'calibration_rows': len(matrices[h]), 'status': 'pending'}
            scale['configurations'].append(rec)
            if model['status'] != 'ok':
                rec.update(status='NA', reason='reference_fit_NA:' + model['reason'],
                ↪  k=None)
                continue
            k = model['main_k'] + delta
            rec['k'] = k
            if not 1 <= k <= model['effective_rank']:
                rec.update(status='NA', reason='adjacent_k_outside_valid_rank_range')
                continue
            try:
                with np.errstate(over='raise', invalid='raise', divide='raise'):
                    ts = _p3_score(model, tx, k)
                    hs = _p3_score(model, matrices[h], k)
                q_valid = bool(ts['Q_informative'])
                rec.update(status='ok', reason=None, target=_p3_target_record(ts),
                           calibration={key: hs[key] for key in ('z', 'scores',
                           ↪  'residual', 'T2', 'Q', 'cT', 'cQ', 'share_T2',
                           ↪  'share_Q')},
                           positions={'T2': _p3_midrank(hs['T2'], ts['T2'][0]),
                                      'Q': _p3_midrank(hs['Q'], ts['Q'][0]) if
                                      ↪  q_valid else None,
                                      'contribution_T2': [_p3_midrank(hs['cT'][:, j],
                                      ↪  ts['cT'][0, j]) for j in range(p)],
                                      'contribution_Q': [_p3_midrank(hs['cQ'][:, j],
                                      ↪  ts['cQ'][0, j]) if q_valid else None for j
                                      ↪  in range(p)]},
                           checks={'reference_and_calibration_disjoint': h not in
                           ↪  reference_years,
                                   'target_not_in_fit': True,
                                   'T2_additivity_target_max_error':
                                   ↪  ts['max_contribution_sum_error_T2'],
                                   'Q_additivity_target_max_error':
                                   ↪  ts['max_contribution_sum_error_Q'],
                                   'T2_additivity_calibration_max_error':
                                   ↪  hs['max_contribution_sum_error_T2'],
                                   'Q_additivity_calibration_max_error':
                                   ↪  hs['max_contribution_sum_error_Q']})
            except (FloatingPointError, ArithmeticError) as error:
                rec.update(status='NA', reason=type(error).__name__ + ':' +
                ↪  str(error))
    scale['summaries'] = {name: _p3_summary(scale['configurations'], ids, name) for name,
    ↪  _ in P3_BRANCHES}
    scale['structure_summary'] = _p3_structure_summary(scale['models'], ids)
    scale['fit_cache_count'] = len(model_cache)
    scale['decomposition_reused_for_calibration_years_and_adjacent_k'] = True
```

```
        scale['status'] = 'ok' if scale['summaries']['main']['valid_configurations'] else 'NA'
        if scale['status'] == 'NA':
            scale['reason'] = 'all_reference_fits_or_main_configurations_NA'
    return _p3_plain(result)


def pca_v3_selftest():

    rng = np.random.default_rng(20261026)
    mats = {str(y): rng.normal(size=(25 + (y % 3), 3)) * np.array([2., .5, 4.]) +
    ↪  np.array([10., -2., 20.]) for y in range(2021, 2026)}
    model = _p3_fit(('2021', '2022', '2023'), mats)
    assert model['status'] == 'ok'
    assert all(abs(model['row_weight_by_year'][y] * len(mats[y]) - 1/3) < 1e-14 for y in
    ↪  model['reference_years'])
    assert np.allclose(np.diag(model['standardized_covariance']), 1., atol=1e-13)
    assert model['orthogonality_max_abs_error'] < 1e-12
    assert model['eigendecomposition_max_abs_error'] < 1e-12
    target = np.array([13., -1., 23.])
    for k in range(1, model['effective_rank'] + 1):
        sc = _p3_score(model, target, k)
        assert np.allclose(sc['cT'].sum(axis=1), sc['T2'])
        assert np.allclose(sc['cQ'].sum(axis=1), sc['Q'])
        if k == 3:
            assert sc['share_Q'] == [None] and not sc['Q_informative']
    with_target_changed = build_pca_v3(['a', 'b', 'c'], {168: mats, 720: mats}, {168: target
    ↪  + 100, 720: target + 100})
    original = build_pca_v3(['a', 'b', 'c'], {168: mats, 720: mats}, {168: target, 720:
    ↪  target})
    assert original['scales']['168']['models'] ==
    ↪  with_target_changed['scales']['168']['models']

    altered = {y: v.copy() for y, v in mats.items()}
    altered['2025'] += 1000
    assert np.array_equal(_p3_fit(('2021', '2022', '2023'), mats)['mu'], _p3_fit(('2021',
    ↪  '2022', '2023'), altered)['mu'])
    constant = {str(y): np.column_stack([np.arange(25), np.ones(25), np.arange(25)**2]) for y
    ↪  in range(2021, 2026)}
    assert _p3_fit(('2021', '2022', '2023'), constant)['status'] == 'NA'

    manual = {'mu': np.zeros(2), 'sigma': np.ones(2), 'eigenvectors': np.array([[1., 1.],
    ↪  [1., -1.]]) / np.sqrt(2), 'eigenvalues': np.array([1.5, .5])}
    neg = _p3_score(manual, np.array([2., -1.]), 1)
    assert neg['cT'][0, 1] < 0 and np.isclose(neg['cT'].sum(), neg['T2'][0])
    import json
    json.dumps(original, allow_nan=False)
    return {'status': 'pass', 'checks': ['year_equal_population_moments',
    ↪  'reference_only_standardization',
                                         'eigen_orthogonality',
                                         ↪  'signed_contribution_additivity',
                                         'target_does_not_change_fit',
                                         ↪  'held_out_year_does_not_change_excluding_R_fit',
                                         'zero_training_variance_is_NA',
                                         ↪  'full_dimension_Q_is_uninformative',
```

```
                                    'strict_JSON'],
            'scope': 'bounded author self-check; not independent statistical or scientific
            ↪  acceptance'}


def p3_inputs(windows):
    ids=P3_FROZEN_V2['feature_ids'];specs={f['id']:f for f in P3_FROZEN_V2['features']}
    support_rows=[];n_by_year={};n_two_scale={};offsets={};matrices={str(h):{} for h in
    ↪  (168,720)};exact={str(h):{} for h in (168,720)}
    target_valid=all(windows[g]['target'].get(str(h)) is not None for g in ids for h in
    ↪  (168,720))
    targets={str(h):[float(windows[g]['target'][str(h)]) if windows[g]['target'].get(str(h))
    ↪  is not None else None for g in ids] for h in (168,720)}
    for year in map(str,range(2021,2026)):
        common=[];dual=0
        for offset in range(-15,16):
            o=str(offset);two_missing=[f'{g}/{h}' for g in ids for h in (168,720) if
            ↪  windows[g]['years'][year][o].get(str(h)) is None]
            wide_missing=[f'{g}/{h}' for g in ids for h in specs[g]['window_hours'] if
            ↪  windows[g]['years'][year][o].get(str(h)) is None]
            if not two_missing:dual+=1
            eligible=not wide_missing
            if eligible:common.append(o)
            endpoint=dt.datetime(int(year),8,26,10,tzinfo=Q1_TZ)+dt.timedelta(days=offset)
            support_rows.append({'year':year,'offset_days':offset,'endpoint_id':f'{year}:{off
            ↪  set:+d}','endpoint_UTC8':endpoint.isoformat(),'two_scale_complete':not
            ↪  two_missing,'V2_all_applicable_windows_complete':eligible,'included_in_PCA':e
            ↪  ligible,'two_scale_missing_fields':two_missing,'V2_all_window_missing_fields'
            ↪  :wide_missing,'exclusion_reason':None if eligible else
            ↪  'V2_all_window_common_support_conservatively_reused'})
        n_by_year[year]=len(common);n_two_scale[year]=dual;offsets[year]=common
        for h in (168,720):
            exact[str(h)][year]=[[str(windows[g]['years'][year][o][str(h)]) for g in ids] for
            ↪  o in common]
            matrices[str(h)][year]=[[float(windows[g]['years'][year][o][str(h)]) for g in
            ↪  ids] for o in common]
    supported=[y for y,n in n_by_year.items() if n>=25]
    for row in support_rows:
        if row['year'] not in supported:
            row['included_in_PCA']=False
            if row['V2_all_applicable_windows_complete']:row['exclusion_reason']='year_common
            ↪  _endpoints_below25'
    matrices={h:{y:v for y,v in data.items() if y in supported} for h,data in
    ↪  matrices.items()}
    support={'rule':'V2 all applicable windows common support, conservatively reused without
    ↪  adding newly eligible two-scale rows','n_by_year':n_by_year,'two_scale_available_n_by
    ↪  _year':n_two_scale,'included_offsets_by_year':{y:offsets[y] for y in
    ↪  supported},'included_endpoint_ids_by_year':{y:[f'{y}:{int(o):+d}' for o in
    ↪  offsets[y]] for y in supported},'valid_years':supported,'minimum_endpoints_per_year':
    ↪  25,'minimum_years':4,'included_rows_total':sum(n_by_year[y] for y in
    ↪  supported),'same_endpoints_for_both_scales':True,'extra_two_scale_available_not_used'
    ↪  :[r for r in support_rows if r['two_scale_complete'] and not
    ↪  r['included_in_PCA']],'target_complete':target_valid,'note':'Main PCA freezes
    ↪  approved V2 all-window shared support. 7/30-day-only availability is disclosed, not a
    ↪  sensitivity fit; no target-dependent selection.'}
```

```
    inputs={'feature_ids':ids,'matrices_by_year':matrices,'targets_by_scale':targets,'physica
    ↪  l_values_exact':exact,'target_physical_values_exact':{str(h):[str(windows[g]['target'
    ↪  ][str(h)]) if windows[g]['target'].get(str(h)) is not None else None for g in ids]
    ↪  for h in (168,720)},'support':support}
    return inputs,support_rows

def p3_tables(res,result):
    summary=[];contrib=[];structure=[];models=[]
    for h,scale in result['scales'].items():
        for branch,s in scale['summaries'].items():
            for metric,row in s['metrics'].items():summary.append({'hours':h,'k_branch':branc
            ↪  h,'metric':metric,'k_values':s['k_values'],**row})
            for metric,rows in s['contributions'].items():
                for row in
                ↪  rows:contrib.append({'hours':h,'k_branch':branch,'metric':metric,**row})
        structure.extend({'hours':h,**row} for row in scale['structure_summary'])
        models.extend({'hours':h,**{k:v for k,v in m.items() if k not in
        ↪  ('eigenvectors','standardized_covariance')}} for m in scale['models'])
    q1_csv(res/'pca_summary.csv',summary);q1_csv(res/'pca_contributions.csv',contrib);q1_csv(
    ↪  res/'pca_structure.csv',structure);q1_csv(res/'pca_models.csv',models)


import tempfile
import shutil

_QF_REF_CACHE=None
_QF_MANIFEST=None
_QF_REUSED_FAMILIES=[]


def qf_read_json(path):
    p=Path(path)
    text=gzip.decompress(p.read_bytes()).decode('utf-8') if p.suffix=='.gz' else
    ↪  p.read_text(encoding='utf-8')
    return json.loads(text)


def qf_write_gzip(path,value):
    data=(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+'\n').encode()
    Path(path).write_bytes(gzip.compress(data,compresslevel=9,mtime=0))


def qf_physical_fingerprint(features,values,seed,n_perm,n_boot,n_perm_boot):
    payload={'seed':seed,'counts':[n_perm,n_boot,n_perm_boot],'features':[]}
    for f in features:
        g=f['id'];scale=f['rank_scale'];record=values[g]
        convert=lambda row:{str(h):str(Fraction(row[str(h)],scale)) if row.get(str(h)) is not
        ↪  None else None for h in f['window_hours']}
        payload['features'].append({'id':g,'windows':f['window_hours'],'unit':f['unit'],'form
        ↪  ula':f['formula'],'target':convert(record['target']),'seasonal_years':{y:{md:conv
        ↪  ert(row) for md,row in rows.items()} for y,rows in
        ↪  record['seasonal_years'].items()}})
    return hashlib.sha256(json.dumps(payload,sort_keys=True,ensure_ascii=False,separators=(',
    ↪  ',':')).encode()).hexdigest()
```

```
def qf_run_ref(features,values,seed=20261024,*,label='B-aligned-corrected',n_perm=2000,n_boot
↪  =1000,n_perm_boot=500):
    if _QF_REF_CACHE is None:
        return run_ref(features,values,seed=seed,label=label,n_perm=n_perm,n_boot=n_boot,n_pe
        ↪  rm_boot=n_perm_boot)
    family='|'.join(f['id'] for f in features)
    expected=_QF_MANIFEST['modex_families'].get(family)
    if expected is None:raise ValueError('Frozen 参考方法 cache does not contain the exact
    ↪  ordered family')
    actual=qf_physical_fingerprint(features,values,seed,n_perm,n_boot,n_perm_boot)
    if actual!=expected['physical_window_sha256']:raise ValueError('参考方法 physical
    ↪  inputs/definitions/counts differ: full recomputation required')
    answer=copy.deepcopy(_QF_REF_CACHE['methods'][expected['method_key']])
    answer['_final_reused_modex']=True
    _QF_REUSED_FAMILIES.append({'method':expected['method_key'],'physical_window_sha256':actu
    ↪  al,'new_draws':0})
    return answer


def qf_pca_inputs_from_analysis(analysis,cache):
    ids=[r['id'] for r in analysis['primary']['candidate_ranking']]
    if ids!=P3_FROZEN_V2['feature_ids']:
        raise ValueError('Reconstructed candidate pool differs from frozen Q1; do not
        ↪  silently change PCA variables')
    specs={f['id']:f for f in analysis['features']};windows={}
    for g in ids:
        scale=specs[g]['rank_scale'];rec=cache[g]
        convert=lambda row:{str(h):Fraction(row[str(h)],scale) if row.get(str(h)) is not None
        ↪  else None for h in specs[g]['window_hours']}
        windows[g]={'target':convert(rec['target']),'years':{str(y):{str(o):convert(rec['year
        ↪  s'][str(y)][str(o)]) for o in range(-15,16)} for y in
        ↪  range(2021,2026)}}
    return p3_inputs(windows)


def qf_verify(results_dir):
    root=Path(results_dir);manifest=qf_read_json(root/'frozen_manifest.json')
    for name,expected in manifest['data_files'].items():
        p=root/name
        if not p.is_file() or q1_sha(p)!=expected['sha256']:raise ValueError('Packed frozen
        ↪  result missing or changed: '+name)
    a=qf_read_json(root/'analysis_results.json.gz');process=qf_read_json(root/'process_result
    ↪  s.json.gz');pca=qf_read_json(root/'pca_results.json.gz')
    assert len(a['primary']['candidate_ranking'])==len(pca['feature_ids'])==13
    assert [r['id'] for r in a['primary']['candidate_ranking']]==pca['feature_ids']
    assert process['metadata']['primary_cutoff']=='2026-08-26T10:00:00+08:00'
    assert pca['support']['included_rows_total']==148 and
    ↪  pca['support']['same_endpoints_for_both_scales']
    print('FROZEN_RESULTS_VERIFIED: complete single-factor/process/comparison/sensitivity/PCA
    ↪  inputs; no analysis or 参考方法 executed')
    return a,process,pca
```

```
def qf_recompute(args):
    global _QF_REF_CACHE,_QF_MANIFEST
    started=dt.datetime.now(dt.timezone.utc).isoformat();out=Path(args.output_dir);res=out/'r
    ↪  esults';res.mkdir(parents=True,exist_ok=True)
    files=[Path(__file__), Path(__file__).resolve().parent / 'data/features.json',
    ↪  Path(__file__).resolve().parent / 'data/reference.json',Path(args.data_dir)/'run_summ
    ↪  ary_all.json',Path(args.data_dir)/'attachment01_02/variable_dictionary.csv']+[Path(ar
    ↪  gs.data_dir)/'attachment01_02/station'/(t+'.csv.gz') for t in
    ↪  ('MET','SOIL','RADM','FLUX')]
    if args.reuse_ref:
        root=Path(args.reuse_ref);_QF_REF_CACHE,_,_=qf_verify(root);_QF_MANIFEST=qf_read_json
        ↪  (root/'frozen_manifest.json')
        for relative,expected in _QF_MANIFEST['frozen_inputs'].items():
            if q1_sha(Path(args.data_dir)/relative)!=expected['sha256']:raise ValueError('D0
            ↪  version differs; frozen 参考方法 must not be reused: '+relative)
        files += [root/'frozen_manifest.json',root/'analysis_results.json.gz']
    before=[q1_fp(p) for p in files]

    with tempfile.TemporaryDirectory(prefix='q1-compute-',dir=out) as temp:
        stage=Path(temp);q1_execute(argparse.Namespace(data_dir=args.data_dir,output_dir=str(
        ↪  stage),v1_results=None,seed=20261024))
        oldres=stage/'results';analysis=qf_read_json(oldres/'analysis_results.json');process=
        ↪  qf_read_json(oldres/'process_v2.json')
        with gzip.open(oldres/'window_cache.json.gz','rt',encoding='utf-8') as
        ↪  stream:cache=json.load(stream)['H-END_C10_0.9']
        inputs,support_rows=qf_pca_inputs_from_analysis(analysis,cache)
        feature_meta=[{'id':f['id'],'label':f['label'],'unit':f['unit']} for f in
        ↪  P3_FROZEN_V2['features']]
        pca=build_pca_v3(inputs['feature_ids'],inputs['matrices_by_year'],inputs['targets_by_
        ↪  scale'],feature_meta=feature_meta)
        pca.update(support=inputs['support'],metadata={'version':'final complete Q1','cutoff'
        ↪  :'2026-08-26T10:00:00+08:00','endpoint':'H-END','time_assumption':'hourly means
        ↪  and same channel cross-year; unconfirmed source facts remain
        ↪  conditional','V2_main_ranking_unchanged':True,'independent_2026_evaluation':False
        ↪  ,'scientific_acceptance':'new execution pending independent
        ↪  review','input_provenance':{'kind':'complete_D0_Q1_path','Modex_reused':bool(args
        ↪  .reuse_ref)}},v2_baseline={k:analysis['primary'][k] for k in
        ↪  ('candidate_ranking','top_candidates','boundary_candidates','available_candidate_
        ↪  count')},v2_factor_evidence=[e for e in analysis['factor_evidence'] if e['id'] in
        ↪  inputs['feature_ids']])
        for scale in pca['scales'].values():
            for model in scale['models']:model['training_endpoint_ids_by_year']={y:inputs['su
            ↪  pport']['included_endpoint_ids_by_year'][y] for y in
            ↪  model['reference_years']}
            for conf in scale['configurations']:conf['calibration_endpoint_ids']=inputs['supp
            ↪  ort']['included_endpoint_ids_by_year'][conf['calibration_year']]
        qf_write_gzip(res/'analysis_results.json.gz',analysis);qf_write_gzip(res/'process_res
        ↪  ults.json.gz',process);qf_write_gzip(res/'pca_results.json.gz',pca)
        q1_dump(res/'pca_inputs.json',inputs);q1_csv(res/'common_support.csv',support_rows);p
        ↪  3_tables(res,pca)
        q1_dump(res/'analysis_configuration.json',qf_read_json(oldres/'configuration.json'))
        shutil.copyfile(oldres/'window_cache.json.gz',res/'window_cache.json.gz')
        q1_dump(res/'pca_configuration.json',{'support':inputs['support'],'settings':pca['set
        ↪  tings'],'feature_ids':pca['feature_ids']})
```

```
        for name in
        ↪  ('all_factors.csv','primary_windows.csv','single_sided_hypotheses.csv','experimen」
        ↪  ts.csv','sensitivity.csv','loo_frequency.csv','method_comparison.csv','c_fixed7d.」
        ↪  csv','extremes.csv','process_support.csv'):shutil.copyfile(oldres/name,res/name)
    after=[q1_fp(p) for p in files];assert before==after,'Input/source changed during
    ↪  execution'
    outputs=[p for p in sorted(res.iterdir()) if p.is_file() and p.name not in
    ↪  ('RUN_RECEIPT.json','FIGURE_RUN_RECEIPT.json')]
    q1_dump(res/'RUN_RECEIPT.json',{'record_type':'actual_final_integrated_Q1_process','start」
    ↪  ed_utc':started,'finished_utc':dt.datetime.now(dt.timezone.utc).isoformat(),'argv':sy」
    ↪  s.argv,'inputs_before':before,'inputs_after':after,'inputs_unchanged':True,'outputs':」
    ↪  [q1_fp(p) for p in outputs],'scope':'D0 feature/A/process/sensitivity/C/PCA
    ↪  recomputed; 参考方法 reused after exact matching' if args.reuse_ref else 'complete D0
    ↪  Q1 including all 参考方法 computations','reused_Modex_families':_QF_REUSED_FAMILIES,'n」
    ↪  ew_Modex_computation_executed':not
    ↪  bool(args.reuse_ref),'scientific_acceptance':'pending independent review of this
    ↪  execution'})
    print('FINAL_Q1_COMPLETE; 参考方法 '+('REUSED, no new draws' if args.reuse_ref else
    ↪  'actually recomputed'))


def qf_main():
    here=Path(__file__).resolve().parent
    parser=argparse.ArgumentParser(description='Complete Q1: approved
    ↪  analysis/process/A-B-C/sensitivity and PCA, plus explicit frozen-result verification')
    parser.add_argument('--mode',choices=('verify','recompute'),default='verify')
    parser.add_argument('--results-dir',default=str(here/'results'))
    parser.add_argument('--data-dir',help='D0 cleaned_all directory, required only for
    ↪  recompute')
    parser.add_argument('--output-dir',default=str(here/'recomputed'))
    parser.add_argument('--reuse-ref',metavar='RESULTS_DIR',help='Reuse exact frozen 参考方法
    ↪  families only after D0 and physical window checks; no new draws')
    args=parser.parse_args()
    if args.mode=='verify':qf_verify(args.results_dir)
    else:
        if not args.data_dir:parser.error('--data-dir is required for recompute')
        qf_recompute(args)


QX_PRECIP_IDS=('precip_sum','precip_max','rain_hours','api')
QX_NEW_IDS=tuple('d1_era5_'+g for g in QX_PRECIP_IDS)


def qx_precip_specs(baseline_features):
    index={f['id']:f for f in baseline_features};out=[]
    labels={'precip_sum':'ERA5格点累计总降水','precip_max':'ERA5格点最大小时总降水量','rain_hours':」
    ↪  'ERA5格点有降水小时数（≥0.1 mm）','api':'ERA5格点前期总降水指数（K=0.9）
    ↪  '}
    formulas={'precip_sum':'sum(P_i)','precip_max':'max(P_i)/1
    ↪  h','rain_hours':'sum(1{P_i>=0.1 mm}) * 1 h','api':'Cutoff-aligned 24h blocks; A_0=0
    ↪  at each window start; A_d=P_d+0.9*A_(d-1); finite-window endpoint value'}
    for original in QX_PRECIP_IDS:
```

```
        f=copy.deepcopy(index[original]);f.update(id='d1_era5_'+original,name=labels[original]
        ↪  ],label=labels[original],original_factor_id=original,formula=formulas[original],w
        ↪  indow_hours=[24,72,168,360,720],windows_h=[24,72,168,360,720],representative_hour
        ↪  s=168,computable=True,eligible_for_top5=True,status='product_defined_proxy',statu
        ↪  s_label='来源定义明确的格点代理',status_reason='Open-Meteo fixed ERA5 previous-hour
        ↪  total precipitation; independent D1, not restoration of station
        ↪  Precip',source_grid_scale=1,rank_scale=1,raw_columns=[{'table':'D1_ERA5','column'
        ↪  :'precipitation_mm','qc_column':None,'source_qc_identity':'not_provided','support
        ↪  ':'[interval_end-1h,interval_end)'}],conditional_assumptions=['ERA5 gridded total
        ↪  precipitation represents an environmental proxy at the returned grid point; no
        ↪  station/source-slope identity claimed','API finite-window initial state zero and
        ↪  cutoff-aligned 24h blocks, fixed before scoring'],limitations=['ERA5 total
        ↪  precipitation includes liquid-equivalent rain/snow; not a station rain gauge','No
        ↪  source QC flags supplied; completeness does not imply no error','No
        ↪  interpolation, zero-filling, elevation correction or blending into D0','0.1 mm is
        ↪  an analysis threshold, not instrument
        ↪  accuracy'],physical_family='era5_total_precipitation_memory' if original=='api'
        ↪  else 'era5_total_precipitation_amount_or_intensity',historical_precision={'encodi
        ↪  ng':'exact Decimal/Fraction; lossless integer common
        ↪  denominator','measurement_accuracy_claim':False})
        f['window_applicability']={str(h):'fixed and identically used for target/pseudo' for
        ↪  h in f['window_hours']}
        out.append(f)
        MECHANISM[f['id']]=MECHANISM[original];ORIGINAL_NAMES[f['id']]=labels[original]
    return out


def qx_read_precip(path):
    values={};products=set();grids=set();qc=set()
    with Path(path).open(encoding='utf-8-sig',newline='') as stream:
        for row in csv.DictReader(stream):
            t=dt.datetime.fromisoformat(row['interval_end']);start=dt.datetime.fromisoformat(
            ↪  row['interval_start'])
            if t.utcoffset()!=dt.timedelta(hours=8) or t.minute or t.second or t.microsecond
            ↪  or t-start!=dt.timedelta(hours=1):raise ValueError('D1 support must be exact
            ↪  aware UTC+8 preceding hour')
            if t in values:raise ValueError('Duplicate D1 interval')
            token=row['precipitation_mm'];p=None if token=='' else
            ↪  Fraction(decimal.Decimal(token))
            if p is not None and p<0:raise ValueError('Negative D1 precipitation')
            values[t]=p;products.add(row['source_product']);grids.add((row['grid_latitude'],r
            ↪  ow['grid_longitude'],row['grid_elevation_m']));qc.add(row['source_qc_status'])
    if len(products)!=1 or len(grids)!=1 or qc!={'not_provided'}:raise ValueError('D1
    ↪  product/grid/QC identity inconsistent across target/history')
    return values,{'product':next(iter(products)),'returned_grid':list(next(iter(grids))),'so
    ↪  urce_qc_status':'not_provided','rows':len(values),'valid_rows':sum(v is not None for
    ↪  v in values.values()),'years':sorted({t.year for t in
    ↪  values}),'support':'[interval_end-1h,interval_end)','aggregation':'hourly total
    ↪  precipitation, rain/snow liquid equivalent','is_station_observation':False}


def qx_precip_window(series,end,hours,original):
    vals=[series.get(end-dt.timedelta(hours=i)) for i in range(hours-1,-1,-1)]
    if any(v is None for v in vals):return None
```

```
    if original=='precip_sum':return sum(vals,Fraction(0))
    if original=='precip_max':return max(vals)
    if original=='rain_hours':return Fraction(sum(v>=Fraction(1,10) for v in vals))
    if original=='api':
        state=Fraction(0)
        for i in range(0,hours,24):state=Fraction(9,10)*state+sum(vals[i:i+24],Fraction(0))
        return state
    raise ValueError(original)


def qx_precip_cache(series,specs,cutoff_hour):

    scale=1
    for value in series.values():
        if value is not None:scale=math.lcm(scale,value.denominator)
    raw=np.zeros(Q1_GRID_LENGTH,dtype=np.int64);valid=np.zeros(Q1_GRID_LENGTH,dtype=np.int64)
    for stamp,value in series.items():
        i=q1_hour(stamp.isoformat())
        if 0<=i<Q1_GRID_LENGTH and value is not None:
            n=value*scale;assert n.denominator==1
            raw[i]=n.numerator;valid[i]=1
    total=np.concatenate(([0],np.cumsum(raw)));count=np.concatenate(([0],np.cumsum(valid)))
    wet=np.concatenate(([0],np.cumsum((raw*10>=scale)&valid.astype(bool))))
    def all_windows(end):
        out={g:{} for g in QX_PRECIP_IDS}
        for hours in (24,72,168,360,720):
            lo,hi=end-hours+1,end+1;h=str(hours)
            if lo<0 or hi>Q1_GRID_LENGTH or count[hi]-count[lo]!=hours:
                for g in out:out[g][h]=None
                continue
            out['precip_sum'][h]=Fraction(int(total[hi]-total[lo]),scale)
            out['precip_max'][h]=Fraction(int(raw[lo:hi].max()),scale)
            out['rain_hours'][h]=Fraction(int(wet[hi]-wet[lo]))
            state=Fraction(0)
            for j in range(lo,hi,24):state=Fraction(9,10)*state+Fraction(int(total[j+24]-tota
            ↪  l[j]),scale)
            out['api'][h]=state
        return out
    first=dt.date(2021,5,31);last=dt.date(2021,10,15)
    monthdays=[(first+dt.timedelta(days=d)).strftime('%m-%d') for d in
    ↪  range((last-first).days+1)]
    target=all_windows(q1_hour(f'2026-08-26T{cutoff_hour:02d}:00:00+08:00'))
    annual={str(y):{md:all_windows(q1_hour(f'{y}-{md}T{cutoff_hour:02d}:00:00+08:00')) for md
    ↪  in monthdays} for y in A_DEFAULT_YEARS}
    values={}
    for f in specs:
        g=f['original_factor_id'];data={'target':target[g],'years':{},'seasonal_years':{}}
        for y in A_DEFAULT_YEARS:
            seasonal={md:annual[str(y)][md][g] for md in
            ↪  monthdays};data['seasonal_years'][str(y)]=seasonal
            data['years'][str(y)]={str(o):seasonal[(dt.date(y,8,26)+dt.timedelta(days=o)).str
            ↪  ftime('%m-%d')] for o in
            ↪  range(-15,16)}
        values[f['id']]=data
```

```
    return values


def qx_encode_precip(caches,specs):
    for f in specs:
        rows=[]
        for branch in caches.values():
            data=branch[f['id']];rows.append(data['target']);rows.extend(r for yr in
            ↪  data['seasonal_years'].values() for r in yr.values())
        denominator=1
        for row in rows:
            for v in row.values():
                if v is not None:denominator=math.lcm(denominator,v.denominator)
        for row in rows:
            for h,v in row.items():
                if v is not None:row[h]=(v*denominator).numerator
        f['rank_scale']=denominator;f['exact_common_denominator']=denominator


def qx_c_precip(specs,values):
    rows=[]
    for f in specs:
        g=f['id'];data=values[g];target=data['target']['168'];annual={};pool=[]
        for year,records in data['years'].items():
            valid=[r['168'] for r in records.values() if r['168'] is not None]
            if len(valid)>=15 and target is not None:annual[year]={'n':len(valid),'cdf':float
            ↪  (a_midrank_fraction(sorted(valid),target))};pool.extend(valid)
        q=sum(r['cdf'] for r in annual.values())/len(annual) if len(annual)>=4 else None
        uninformative=bool(pool) and len(set(pool+[target]))==1
        rows.append({'id':g,'label':f['label'],'unit':f['unit'],'value':target/f['rank_scale'
        ↪  ] if target is not None else None,'q':q,'A':abs(2*q-1) if q is not None else
        ↪  None,'annual':annual,'rank_eligible':not
        ↪  uninformative,'uninformative':uninformative,'reason':None if q is not None else
        ↪  'target_or_15window_4year_support_NA','D1_support_rule':'complete precipitation
        ↪  window; no mean-gap relaxation'})
    return rows


def qx_pca_inputs(specs,values,primary):
    ids=[r['id'] for r in primary['candidate_ranking']];index={f['id']:f for f in
    ↪  specs};windows={}
    for g in ids:
        s=index[g]['rank_scale'];rec=values[g]
        convert=lambda row:{str(h):Fraction(row[str(h)],s) if row.get(str(h)) is not None
        ↪  else None for h in index[g]['window_hours']}
        windows[g]={'target':convert(rec['target']),'years':{str(y):{str(o):convert(rec['year
        ↪  s'][str(y)][str(o)]) for o in range(-15,16)} for y in
        ↪  A_DEFAULT_YEARS}}
    global P3_FROZEN_V2
    old=P3_FROZEN_V2
    try:
        P3_FROZEN_V2={'feature_ids':ids,'features':[index[g] for g in ids]}
        inputs,rows=p3_inputs(windows)
    finally:P3_FROZEN_V2=old
```

```
    inputs['support']['rule']='All applicable windows of the full enhanced deduplicated
    ↪  candidate pool; same common endpoints at 7d/30d; no variable removed to salvage PCA'
    inputs['support']['note']='Support extension of approved V2 rule to D0+D1; no
    ↪  target-dependent endpoint selection'
    return inputs,rows


def qx_selftest():
    end=dt.datetime(2026,8,26,10,tzinfo=Q1_TZ)
    series={end-dt.timedelta(hours=i):Fraction(1) for i in range(72)}
    assert qx_precip_window(series,end,24,'precip_sum')==24
    assert qx_precip_window(series,end,24,'precip_max')==1
    assert qx_precip_window(series,end,24,'rain_hours')==24
    assert qx_precip_window(series,end,72,'api')==Fraction(24)*sum(Fraction(9,10)**i for i in
    ↪  range(3))

    series[end+dt.timedelta(hours=1)]=100000
    assert qx_precip_window(series,end,24,'precip_sum')==24
    missing=dict(series);del missing[end-dt.timedelta(hours=2)]
    assert qx_precip_window(missing,end,24,'precip_sum') is None
    zero={end-dt.timedelta(hours=i):Fraction(0) for i in range(24)}
    assert qx_precip_window(zero,end,24,'rain_hours')==0

    counter=[Fraction(0),Fraction(1),Fraction(3)];increments=[counter[i]-counter[i-1] for i
    ↪  in range(1,len(counter))]
    assert sum(increments)==3 and sum(counter)==4
    exact={'x':{g:{'target':{'24':Fraction(1,10)},'seasonal_years':{'2021':{'08-26':{'24':Fra
    ↪  ction(3,10)}}}} for g in
    ↪  ['g']}}
    spec=[{'id':'g','rank_scale':1}];qx_encode_precip(exact,spec)
    assert Fraction(exact['x']['g']['target']['24'],spec[0]['rank_scale'])==Fraction(1,10)
    return {'status':'pass','checks':['hour_end_inclusion_and_next_hour_exclusion','complete_
    ↪  hour_support','sum_max_threshold_count','zero_is_valid_not_missing','counter_vs_incre
    ↪  ment_distinction','fixed_API_recursion_initial_state_and_decay','lossless_exact_encod
    ↪  ing'],'scope':'author bounded checks, not independent
    ↪  approval'}


def qx_execute(args):
    started=dt.datetime.now(dt.timezone.utc).isoformat();clock0=time.monotonic();out=Path(arg
    ↪  s.output_dir);res=out/'results';res.mkdir(parents=True,exist_ok=True)
    if not args.d0_cache:
        if not args.data_dir:raise ValueError('enhance requires --data-dir to reconstruct D0
        ↪  windows, or explicit --d0-cache')
        rebuilt=out/'reconstructed_d0'
        qx_rebuild_windows(argparse.Namespace(data_dir=args.data_dir,output_dir=str(rebuilt)))
        args.d0_cache=str(rebuilt/'results/window_cache.json.gz')
    source_files=[Path(__file__), Path(__file__).resolve().parent / 'data/features.json',
    ↪  Path(__file__).resolve().parent /
    ↪  'data/reference.json',Path(args.d1_csv),Path(args.d0_cache),Path(args.baseline_result
    ↪  s)/'analysis_results.json.gz',Path(args.baseline_results)/'c_fixed7d.csv']
    before=[q1_fp(p) for p in source_files];baseline=qf_read_json(Path(args.baseline_results)
    ↪  /'analysis_results.json.gz')
```

```
features=copy.deepcopy(baseline['features']);new=qx_precip_specs(features);features+=new;
↪  active=[f for f in features if
↪  f.get('computable')]
print('D1_READ_AND_EXACT_WINDOWS',flush=True)
series,d1meta=qx_read_precip(args.d1_csv)
write_d1_process(args.d1_csv,res)
d1caches={hour:qx_precip_cache(series,new,hour) for hour in
↪  (10,0,9)};qx_encode_precip(d1caches,new)
d0caches=qf_read_json(args.d0_cache)
caches={key:{**data,**d1caches[int(key.split('_C')[1].split('_')[0])]} for key,data in
↪  d0caches.items()}
values=caches['H-END_C10_0.9']
print('ENHANCED_A_PRIMARY_AND_ALL_SENSITIVITIES',flush=True)
primary=summarize_one(active,values)
branches=[]
for ep,ch,cov in
↪  [('H-END',10,.9),('H-START',10,.9),('H-END',0,.9),('H-START',0,.9),('H-END',10,.95)]:
    key=f'{ep}_C{ch}_{cov}';answer=primary if key=='H-END_C10_0.9' else
    ↪  summarize_one(active,caches[key])
    branches.append({'name':f'{ep}_C{ch:02d}_coverage{int(cov*100)}','endpoint':ep,'cutof
    ↪  f_hour':ch,'coverage':cov,'min_endpoints':25,'result':q1_trim_a(answer),'D1_time_
    ↪  policy':'actual product support; D0 endpoint assumption does not shift
    ↪  D1'})
reduced=copy.deepcopy(active)
for f in reduced:f['window_hours']=[h for h in f['window_hours'] if h!=720]
branches.append({'name':'drop30d_all_factors','min_endpoints':25,'result':q1_trim_a(summa
↪  rize_one(reduced,values))})
for n in (20,15):branches.append({'name':f'all_factors_min{n}','min_endpoints':n,'result'
↪  :q1_trim_a(summarize_one(active,values,min_endpoints=n))})
for rho in (.85,.95):branches.append({'name':f'rho{rho}','min_endpoints':25,'result':q1_t
↪  rim_a(summarize_one(active,values,redundancy_threshold=rho))})
common,common_info=q1_common_support(active,values,primary)
branches.append({'name':'common_history_support','min_endpoints':25,'common_support':comm
↪  on_info,'result':q1_trim_a(summarize_one(active,common))})
print('ENHANCED_LEAVE_ONE_YEAR_OUT',flush=True);loo=leave_one_year_out(active,values)
for k in loo['runs']:loo['runs'][k]=q1_trim_a(loo['runs'][k])
singles=copy.deepcopy(new)
for f in singles:f['direction']='high'
single=summarize_one(singles,values)
c_rows=copy.deepcopy(baseline['methods']['C_factors'])+qx_c_precip(new,values);c_index={r
↪  ['id']:r for r in
↪  c_rows}
specs={f['id']:f for f in features}
common_ids=[r['id'] for r in primary['candidate_ranking'] if r['id'] in MECHANISM and
↪  c_index.get(r['id'],{}).get('A') is not None and
↪  c_index[r['id']].get('rank_eligible')]
print('ENHANCED_ACTUAL_B_REFIT',len(common_ids),flush=True)
b=v2b_fit(specs,values,common_ids,20261024,'B_D0_D1_common_actual_refit','enhanced_common
↪  _deduplicated_original_factor_analogues')
```

```
b['metadata'].update(proxy_policy='Original mechanism grades inherited for separately named four ERA5 precipitation analogues; two height-difference proxies excluded because no original grades',D0_alignment='Unchanged exact D0 window cache plus separate product-defined D1 total-precipitation windows; no filling, clipping, recleaning or cross-source substitution',execution_in_enhancement=True,execution_in_v2=False,D1_role='Four original precipitation factors represented by a separately named fixed ERA5 gridded proxy; original grades transparently inherited',D0_reused_without_change=True)
comparison=v2b_common_table(common_ids,specs,baseline['primary'],primary,b,c_index,scope='D0+D1 enhanced deduplicated shared original-factor analogues; no invented grades for height-difference proxies')
for row in comparison:
    row.update(E1_scope='Unchanged D0 baseline ranking displayed only for old IDs; added D1 analogues have no baseline score',E2_scope='Enhanced D0+D1 A scores reranked in this actual common family',B_scope='Actual common-family entropy/TOPSIS refit with original prior mechanism grades inherited for D1 analogues',C_scope='Fixed7d baseline D0 plus complete-window D1 analogues, common-family reranking')
print('ENHANCED_PCA_FIXED_7D_30D',flush=True)
inputs,support_rows=qx_pca_inputs(active,values,primary)
pca=build_pca_v3(inputs['feature_ids'],inputs['matrices_by_year'],inputs['targets_by_scale'],feature_meta=[{'id':g,'label':specs[g]['label'],'unit':specs[g]['unit']} for g in inputs['feature_ids']])
pca.update(support=inputs['support'],metadata={'version':'D0+D1 complete enhanced pool','cutoff':'2026-08-26T10:00:00+08:00','D0_time_assumption':'H-END hourly mean working assumption','D1_time_definition':'documented previous hour total precipitation','independent_2026_evaluation':False,'scientific_acceptance':'pending independent review'})
for scale in pca['scales'].values():
    for model in scale['models']:model['training_endpoint_ids_by_year']={y:inputs['support']['included_endpoint_ids_by_year'][y] for y in model['reference_years']}
    for conf in scale['configurations']:conf['calibration_endpoint_ids']=inputs['support']['included_endpoint_ids_by_year'][conf['calibration_year']]
old_ids=[r['id'] for r in baseline['primary']['candidate_ranking']];old_ranks={r['id']:r for r in baseline['primary']['candidate_ranking']};new_ranks={r['id']:r for r in primary['candidate_ranking']}
score_checks={g:baseline['primary']['factors'][g]['score_S_exact']==primary['factors'][g]['score_S_exact'] for g in old_ids}
if not all(score_checks.values()):raise ValueError('Unchanged D0 marginal A scores changed')
old_only_ranks=q1_rank_rows({g:primary['factors'][g]['score_S'] for g in old_ids});old_only={r['id']:r for r in old_only_ranks}
changes=[{'id':g,'label':specs[g]['label'],'old_rank_min':old_ranks[g]['rank_min'],'enhanced_rank_min':new_ranks[g]['rank_min'],'common_old_only_rank_min':old_only[g]['rank_min'],'S_old':baseline['primary']['factors'][g]['score_S'],'S_enhanced':primary['factors'][g]['score_S'],'score_exact_unchanged':score_checks[g],'change_reason':'candidate addition only; marginal definition and own historical support unchanged'} for g in old_ids]
```

```
result={'schema':'Q1_D0_D1_RECOMPUTED_20261030','features':features,'primary':primary,'br
↪  anches':branches,'loo':loo,'single_sided_precipitation_hypotheses':single,'methods':{
↪  'B_common_actual_refit':b,'C_factors':c_rows,'B_D0_unchanged_reference':baseline['met
↪  hods']['B_common_deduplicated_refit']},'comparisons':comparison,'pca':pca,'old_candid
↪  ate_changes':changes,'D1_metadata':d1meta,'common_support':common_info,'D0_marginal_s
↪  core_regression':score_checks,'scientific_acceptance':'pending independent Q1 total
↪  review','pre_score_operation_contract':'D1_OPERATION_CONTRACT.md'}
qf_write_gzip(res/'enhanced_results.json.gz',result);qf_write_gzip(res/'enhanced_pca_resu
↪  lts.json.gz',pca)
qf_write_gzip(res/'enhanced_window_cache.json.gz',caches)
q1_dump(res/'enhanced_pca_inputs.json',inputs);q1_csv(res/'enhanced_pca_common_support.cs
↪  v',support_rows)
pcadir=res/'enhanced_pca';pcadir.mkdir(exist_ok=True);p3_tables(pcadir,pca)
q1_csv(res/'enhanced_method_comparison.csv',comparison);q1_csv(res/'enhanced_old_candidat
↪  e_changes.csv',changes)
q1_csv(res/'enhanced_sensitivity.csv',[{'branch':r['name'],'pool':r['result']['available_
↪  candidate_count'],'top':r['result']['top_candidates'],'boundary':r['result']['boundar
↪  y_candidates']} for r in
↪  branches])
q1_csv(res/'enhanced_loo_frequency.csv',[{'id':g,**r} for g,r in
↪  loo['selection_frequency'].items()])
rank={r['id']:r for r in primary['candidate_ranking']};factor_rows=[];window_rows=[]
for f in features:
    g=f['id'];r=primary['factors'].get(g,{});factor_rows.append({'id':g,'label':f['label'
    ↪  ],'unit':f['unit'],'definition_status':f['status'],'score_status':r.get('status',
    ↪  'definition_NA'),'S':r.get('score_S'),'rank_min':rank.get(g,{}).get('rank_min'),'
    ↪  rank_max':rank.get(g,{}).get('rank_max'),'top_candidate':g in
    ↪  primary['top_candidates'],'reason':r.get('reason') or
    ↪  f.get('status_reason'),'support':r.get('support')})
    for h,w in r.get('windows',{}).items():window_rows.append({'id':g,'label':f['label'],
    ↪  'hours':int(h),'value':w['target_value']/f['rank_scale'] if w['target_value'] is
    ↪  not None else None,'unit':f['unit'],**{k:v for k,v in w.items() if
    ↪  k!='target_value'}})
q1_csv(res/'enhanced_all_factors.csv',factor_rows);q1_csv(res/'enhanced_primary_windows.c
↪  sv',window_rows)
checks={'bounded_precipitation_checks':qx_selftest(),'pca_reference_separation_and_contri
↪  bution_sum':pca_v3_selftest(),'D0_marginal_scores_unchanged':score_checks,'target_his
↪  tory_same_product_and_grid':d1meta,'all_new_factor_year_endpoint_counts':{g:primary['
↪  factors'][g]['support'] for g in
↪  QX_NEW_IDS},'formal_new_B_counts':b['metadata'],'D1_no_QC_flags':'not_provided; never
↪  represented as QC0','API_rule_fixed_before_scoring':True,'direction':'main two-sided;
↪  inherited high-only hypothesis separate'}
q1_dump(res/'ENHANCED_CHECKS.json',checks)
after=[q1_fp(p) for p in source_files]
if before!=after:raise ValueError('Input/source changed during enhanced execution')
outputs=[p for p in sorted(res.rglob('*')) if p.is_file() and p.name not in
↪  ('RUN_RECEIPT.json','ENHANCED_RUN_RECEIPT.json') and 'baseline' not in p.parts]
```

```
    q1_dump(res/'ENHANCED_RUN_RECEIPT.json',{'record_type':'actual_D0_D1_enhancement','starte
    ↪  d_utc':started,'finished_utc':dt.datetime.now(dt.timezone.utc).isoformat(),'elapsed_s
    ↪  econds':time.monotonic()-clock0,'argv':sys.argv,'environment':{'python':sys.version,'
    ↪  numpy':np.__version__,'scipy':__import__('scipy').__version__},'inputs_before':before
    ↪  ,'inputs_after':after,'inputs_unchanged':True,'outputs':[q1_fp(p) for p in
    ↪  outputs],'reused':'unchanged D0 windows and old separate B reference; baseline
    ↪  independently rerun by root with exact frozen cache checks','newly_computed':'D1
    ↪  exact windows; full enhanced A/sensitivity/LOO; common B formal draws and bootstrap;
    ↪  C precipitation; full enhanced PCA','scientific_acceptance':'pending Q1 total
    ↪  review'})
    print('ENHANCED_COMPLETE',json.dumps({'top':primary['top_candidates'],'pool':len(primary[
    ↪  'candidate_ranking']),'B_common':common_ids,'PCA_support':inputs['support']['n_by_yea
    ↪  r']},ensure_ascii=False),flush=True)


import argparse
import csv
import datetime as dt
from decimal import Decimal
from fractions import Fraction
import hashlib
import json
from pathlib import Path

D1_YEARS = (2021, 2022, 2023, 2024, 2025)
D1_PROBS = (("q05", Fraction(1,20)), ("q25", Fraction(1,4)),
            ("median", Fraction(1,2)), ("q75", Fraction(3,4)), ("q95", Fraction(19,20)))
D1_ZONE = dt.timezone(dt.timedelta(hours=8))


def d1_plain(obj):
    if isinstance(obj, dict):
        return {str(k):d1_plain(v) for k,v in obj.items()}
    if isinstance(obj, (tuple,list)):
        return [d1_plain(v) for v in obj]
    if isinstance(obj, (Fraction,Decimal)):
        return float(obj)
    return obj


def d1_equal_year_quantiles(groups):

    if len(groups) != 5 or any(not g for g in groups):
        return {name:None for name,_ in D1_PROBS}
    weighted = sorted((v,Fraction(1,5*len(g))) for g in groups for v in g)
    result = {}
    for name, p in D1_PROBS:
        weight = Fraction(0)
        for value, w in weighted:
            weight += w
            if weight >= p:
                result[name] = value
                break
    return result
```

```
def d1_load_precipitation(csv_path):
    records = {}
    grid = set()
    with Path(csv_path).open(encoding="utf-8", newline="") as stream:
        for row in csv.DictReader(stream):
            end = dt.datetime.fromisoformat(row["interval_end"])
            start = dt.datetime.fromisoformat(row["interval_start"])
            assert end.utcoffset() == dt.timedelta(hours=8)
            assert end.minute == end.second == end.microsecond == 0
            assert start == end-dt.timedelta(hours=1)
            assert end not in records, f"duplicate support end: {end}"
            text = row["precipitation_mm"].strip()
            value = Fraction(Decimal(text)) if text else None
            assert value is None or value >= 0
            assert row["source_product"] == "Open-Meteo archive / ERA5"
            assert row["source_qc_status"] == "not_provided"
            records[end] = value
            grid.add((float(row["grid_latitude"]),float(row["grid_longitude"]),float(row["gri⌟
            ↪  d_elevation_m"])))
    assert len(grid) == 1
    return records, next(iter(grid))


def d1_episodes(values, thresholds, ends, relative):
    flags = [None if v is None or q is None else v>q for v,q in zip(values,thresholds)]
    episodes, active = [], []
    def finish():
        if not active:
            return
        first,last = active[0],active[-1]
        episodes.append({"support_start":(ends[first]-dt.timedelta(hours=1)).isoformat(),
                         "support_end":ends[last].isoformat(),
                         "first_relative_hour":relative[first],"last_relative_hour":relative[⌟
                         ↪  last],
                         "duration_hours":len(active),
                         "sum_precipitation_mm":sum(values[k] for k in active),
                         "sum_excess_mm":sum(values[k]-thresholds[k] for k in active),
                         "max_hourly_precipitation_mm":max(values[k] for k in active),
                         "max_excess_mm":max(values[k]-thresholds[k] for k in active)})
        active.clear()
    for k,flag in enumerate(flags):
        if flag is True:
            active.append(k)
        else:
            finish()
    finish()
    return flags, episodes


def d1_period_summary(values, thresholds, ends, relative):
    available = [v for v in values if v is not None]
    flags,episodes = d1_episodes(values,thresholds,ends,relative)
```

```
    complete = len(available)==len(values)
    summary = {"expected_hours":len(values),"available_hours":len(available),
               "missing_hours":len(values)-len(available),"complete_support":complete,
               "support_start":(ends[0]-dt.timedelta(hours=1)).isoformat(),
               "support_end":ends[-1].isoformat(),
               "sum_precipitation_mm":sum(available) if complete else None,
               "available_sum_precipitation_mm":sum(available) if available else None,
               "max_hourly_precipitation_mm":max(available) if available else None,
               "wet_hours_ge_0_1mm":sum(v>=Fraction(1,10) for v in available),
               "zero_hours":sum(v==0 for v in available),
               "threshold_available_hours":sum(q is not None for q in thresholds),
               "assessed_hours":sum(f is not None for f in flags),
               "unassessed_hours":sum(f is None for f in flags),
               "high_p95_hours":sum(f is True for f in flags),
               "high_p95_episode_count":len(episodes),
               "longest_high_p95_episode_hours":max((e["duration_hours"] for e in
               ↪  episodes),default=0),
               "episodes":episodes,
               "count_note":"wet/zero/exceedance counts refer only to available/assessed
               ↪  hours; missing is never zero"}
    return summary


def build_d1_process(csv_path, cutoff=None):

    cutoff = cutoff or dt.datetime(2026,8,26,10,tzinfo=D1_ZONE)
    assert cutoff.utcoffset()==dt.timedelta(hours=8)
    assert cutoff.minute==cutoff.second==cutoff.microsecond==0
    records,grid = d1_load_precipitation(csv_path)
    relative = list(range(-719,1))
    ends = [cutoff+dt.timedelta(hours=h) for h in relative]
    values = [records.get(t) for t in ends]
    quantiles = {key:[] for key,_ in D1_PROBS}
    counts = [[] for _ in D1_YEARS]
    for end in ends:
        groups = []
        for yi,year in enumerate(D1_YEARS):
            group = []
            for day in range(-15,16):
                reference = end.replace(year=year)+dt.timedelta(days=day)
                assert reference.year in D1_YEARS
                v = records.get(reference)
                if v is not None:
                    group.append(v)
            counts[yi].append(len(group))
            groups.append(group)
        q = d1_equal_year_quantiles(groups)
        for key in quantiles:
            quantiles[key].append(q[key])
    thresholds = quantiles["q95"]
    flags,episodes = d1_episodes(values,thresholds,ends,relative)
    summaries = {name:d1_period_summary(values[-hours:],thresholds[-hours:],ends[-hours:],rel
    ↪  ative[-hours:])
                 for name,hours in (("30d",720),("7d",168),("72h",72))}
```

```
    assert len(ends)==720 and ends[0]-dt.timedelta(hours=1)==cutoff-dt.timedelta(days=30)
    assert ends[-1]==cutoff and all(t<=cutoff for t in ends)
    assert len(set(ends))==720
    assert all(all(n==31 for n in year) for year in counts), "incomplete D1 historical
    ↪  support"
    assert all(v is not None for v in values), "incomplete D1 target support"
    result = {"metadata":{"schema_version":"q1-d1-process-v1.0",
    ↪  "primary_cutoff":cutoff.isoformat(),
                          "timezone":"UTC+8", "source":"Open-Meteo archive API / fixed ERA5",
                          "requested_point":{"latitude":28.86,"longitude":85.29},
                          "returned_grid":{"latitude":grid[0],"longitude":grid[1],"elevation_
                          ↪  m":grid[2]},
                          "cell_selection":"nearest","elevation_request":"nan; downscaling
                          ↪  disabled",
                          "support":"API precipitation at t is total over preceding hour
                          ↪  [t-1h,t)",
                          "unit":"mm","source_qc_status":"not_provided",
                          "role":"D1 independent grid proxy, not D0 station or source-slope
                          ↪  observation",
                          "source_csv":str(csv_path),
                          "source_sha256":hashlib.sha256(Path(csv_path).read_bytes()).hexdige
                          ↪  st(),
                          "reference_years":list(D1_YEARS),"seasonal_days_each_side":15,
                          "quantile_rule":"Per relative hour: each of five years has weight
                          ↪  1/5; each eligible sample in year y has weight 1/(5*n_y);
                          ↪  inverse ECDF inf{x:F(x)>=p}, no interpolation; all five years
                          ↪  required.",
                          "threshold_rule":"strictly greater than historical q95; historical
                          ↪  years only, never target values",
                          "trajectory_note":"Historical same-calendar ±15d trajectories
                          ↪  overlap; not independent events or a confidence interval.",
                          "future_target_support_used":False,
                          "limitations":"Reanalysis grid precipitation, terrain-smoothed
                          ↪  spatial support; no causal attribution or no-rain-trigger
                          ↪  inference."},
              "time_relative_hours":relative,
              "support_start_iso":[(t-dt.timedelta(hours=1)).isoformat() for t in ends],
              "support_end_iso":[t.isoformat() for t in ends],
              "values":values,
              **{f"background_{key}":q for key,q in quantiles.items()},
              "history_n_by_year":counts,
              "history_years_count":[sum(c[k]>0 for c in counts) for k in range(720)],
              "high_p95_exceedance":flags,
              "high_p95_signed_excess_mm":[v-q if v is not None and q is not None else None
              ↪  for v,q in zip(values,thresholds)],
              "high_p95_episodes":episodes,"summaries":summaries}
    return d1_plain(result)


def d1_process_selftest():
    q=d1_equal_year_quantiles([[Fraction(0)],[Fraction(10)]*100,[Fraction(20)],[Fraction(30)]
    ↪  ,[Fraction(40)]])
    assert q["median"]==20, "annual weighting accidentally pooled"
    assert d1_equal_year_quantiles([[],[1],[1],[1],[1]])["q95"] is None
```

```
    ends=[dt.datetime(2026,8,26,7+i,tzinfo=D1_ZONE) for i in range(4)]
    flags,episodes=d1_episodes([Fraction(1),Fraction(2),None,Fraction(3)],[Fraction(1)]*4,end
    ↪  s,[-3,-2,-1,0])
    assert flags==[False,True,None,True] and len(episodes)==2
    s=d1_period_summary([Fraction(0),None],[Fraction(0),Fraction(0)],ends[:2],[-1,0])
    assert s["sum_precipitation_mm"] is None and s["available_sum_precipitation_mm"]==0
    assert s["zero_hours"]==1 and s["missing_hours"]==1 and s["high_p95_hours"]==0
    return {"unequal_annual_n_equal_weights":"pass","one_empty_year_quantiles_na":"pass",
            "strict_threshold_and_missing_breaks_episodes":"pass","zero_distinct_from_missing
            ↪  ":"pass"}


def write_d1_process(csv_path, output_dir):
    output_dir=Path(output_dir); output_dir.mkdir(parents=True,exist_ok=True)
    result=build_d1_process(csv_path)
    (output_dir/"d1_process.json").write_text(json.dumps(result,ensure_ascii=False,indent=2,a
    ↪  llow_nan=False),encoding="utf-8")
    fields=["relative_hour","support_start","support_end","precipitation_mm","background_q05",
            "background_q25","background_median","background_q75","background_q95",
            *[f"history_n_{year}" for year in
            ↪  D1_YEARS],"history_years_count","high_p95_exceedance",
            "high_p95_signed_excess_mm","source_product","source_qc_status"]
    with (output_dir/"process_source_data.csv").open("w",encoding="utf-8",newline="") as
    ↪  stream:
        writer=csv.DictWriter(stream,fieldnames=fields);writer.writeheader()
        for k,rel in enumerate(result["time_relative_hours"]):
            writer.writerow({"relative_hour":rel,"support_start":result["support_start_iso"][
            ↪  k],
                             "support_end":result["support_end_iso"][k],"precipitation_mm":re
                             ↪  sult["values"][k],
                             **{f"background_{key}":result[f"background_{key}"][k] for key,_
                             ↪  in D1_PROBS},
                             **{f"history_n_{year}":result["history_n_by_year"][yi][k] for
                             ↪  yi,year in enumerate(D1_YEARS)},
                             "history_years_count":result["history_years_count"][k],
                             "high_p95_exceedance":result["high_p95_exceedance"][k],
                             "high_p95_signed_excess_mm":result["high_p95_signed_excess_mm"][
                             ↪  k],
                             "source_product":"Open-Meteo archive /
                             ↪  ERA5","source_qc_status":"not_provided"})
    (output_dir/"d1_process_checks.json").write_text(json.dumps({"selftest":d1_process_selfte
    ↪  st(),
                    "actual_target_hours":len(result["values"]),"actual_target_missing_hours"
                    ↪  :sum(v is None for v in
                    ↪  result["values"]),
                    "historical_count_per_year_per_relative_hour":31,"historical_years":5,
                    "actual_cutoff_and_source_intervals":"pass","not_independent_review":True
                    ↪  },ensure_ascii=False,indent=2),encoding="utf-8")
    return result


def qx_rebuild_windows(args):

    started=dt.datetime.now(dt.timezone.utc).isoformat();out=Path(args.output_dir)/'results';
    ↪  out.mkdir(parents=True,exist_ok=True)
```

```
    base=Path(args.data_dir);files=[Path(__file__), Path(__file__).resolve().parent /
    ↪  'data/features.json', Path(__file__).resolve().parent /
    ↪  'data/reference.json',base/'run_summary_all.json',base/'attachment01_02/variable_dict
    ↪  ionary.csv']+[base/'attachment01_02/station'/(t+'.csv.gz') for t in
    ↪  ('MET','SOIL','RADM','FLUX')]
    before=[q1_fp(p) for p in
    ↪  files];dictionary=q1_features();features=dictionary['factors'];active=[f for f in
    ↪  features if f.get('computable')]
    arrays,counts=q1_read_d0(base,features)
    for f in active:
        _,_,places,_=q2_inputs(arrays,f);f['source_grid_scale']=10**places;f['rank_scale']=10
        ↪  **places
    caches={};ledger=[]
    for ep,ch,cov in [('H-END',10,.9),('H-START',10,.9),('H-END',0,.9),('H-START',0,.9),('H-E
    ↪  ND',10,.95),('H-END',9,.9)]:
        print('REBUILD_D0_EXACT_WINDOWS',ep,ch,cov,flush=True);caches[(ep,ch,cov)]=q1_cache(a
        ↪  rrays,active,ep,ch,cov,ledger)
        ledger.clear()
    q1_encode_exact_cache(caches,features)
    qf_write_gzip(out/'window_cache.json.gz',{f'{ep}_C{ch}_{cov}':data for (ep,ch,cov),data
    ↪  in caches.items()})
    q1_dump(out/'feature_dictionary.json',dictionary)
    after=[q1_fp(p) for p in files];assert before==after
    q1_dump(out/'WINDOW_REBUILD_RECEIPT.json',{'record_type':'actual_D0_exact_windows_reconst
    ↪  ruction','started_utc':started,'finished_utc':dt.datetime.now(dt.timezone.utc).isofor
    ↪  mat(),'inputs_before':before,'inputs_after':after,'inputs_unchanged':True,'outputs':[
    ↪  q1_fp(out/'window_cache.json.gz'),q1_fp(out/'feature_dictionary.json')],'data_counts'
    ↪  :counts,'analysis_or_B_sampling_executed':False,'scope':'D0 read/exact aggregation
    ↪  only; does not approve scientific results'})
    print('D0_WINDOWS_REBUILT',str(out/'window_cache.json.gz'),flush=True)


def sol():
    mode=sys.argv[sys.argv.index('--mode')+1] if '--mode' in sys.argv else 'verify'
    if mode not in ('enhance','rebuild-windows'):return qf_main()
    parser=argparse.ArgumentParser(description='Explicit independent ERA5 D1 precipitation
    ↪  enhancement; approved Q1 mathematics retained')
    parser.add_argument('--mode',choices=['enhance','rebuild-windows'],required=True);parser.
    ↪  add_argument('--d1-csv');parser.add_argument('--d0-cache');parser.add_argument('--dat
    ↪  a-dir');parser.add_argument('--baseline-results');parser.add_argument('--output-dir',
    ↪  required=True)
    args=parser.parse_args()
    if mode=='rebuild-windows':
        if not args.data_dir:parser.error('rebuild-windows requires --data-dir')
        qx_rebuild_windows(args)
    else:
        if not args.d1_csv or not args.baseline_results:parser.error('enhance requires
        ↪  --d1-csv and --baseline-results')
        if not args.d0_cache and not args.data_dir:parser.error('enhance requires --data-dir
        ↪  or --d0-cache')
        qx_execute(args)


if __name__=='__main__':sol()
```

问题二

q2/draw.py

```
import argparse
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path
import sys

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


STYLE={'font':'Arial Unicode MS','blue':'#246A8A','orange':'#C57934','purple':'#77608D',
       'grey':'#777E86','green':'#467F69','background':'#FFFFFF','dpi':300,
       'width_in':7.2,'height_in':3.65,'font_size':9}


def sha(p): return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def pair():
    fig,axes=plt.subplots(1,2,figsize=(STYLE['width_in'],STYLE['height_in']))
    fig.subplots_adjust(left=.10,right=.98,bottom=.23,top=.84,wspace=.40)
    for i,ax in enumerate(axes):
        ax.spines[['top','right']].set_visible(False)
        ax.tick_params(labelsize=8)
        ax.set_gid(chr(97+i))
    return fig,axes


def export(fig,axes,out,name,title,note):
    fig.suptitle(title,y=.975,fontsize=11)
    fig.text(.10,.055,note,ha='left',va='bottom',fontsize=8,color='#47505A')
    fig.canvas.draw()


    fig.savefig(out/(name+'.png'),facecolor='white',dpi=300)
    fig.savefig(out/(name+'.svg'),facecolor='white')
    fig.savefig(out/(name+'.pdf'),facecolor='white')
    plt.close(fig)


def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--delivery-dir',type=Path,default=Path(__file__).resolve().parent)
    parser.add_argument('--results-dir',type=Path)
    parser.add_argument('--output-dir',type=Path)
    a=parser.parse_args();d=a.delivery_dir.resolve();r=(a.results_dir or
    ↪  d/'results').resolve();out=(a.output_dir or d/'figures').resolve()
```

```
out.mkdir(parents=True,exist_ok=True)
started=datetime.now(timezone.utc).isoformat()
plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':[STYLE['font'],'DejaVu
↪ Sans'],
                     'font.size':STYLE['font_size'],'axes.labelsize':9,'axes.titlesize':1」
                     ↪ 0,
                     'legend.fontsize':8,'axes.unicode_minus':False,'svg.fonttype':'none',
                     'pdf.fonttype':42,'ps.fonttype':42,'savefig.dpi':STYLE['dpi']})
b=STYLE['blue'];o=STYLE['orange'];g=STYLE['grey'];p=STYLE['purple']
curve=pd.read_csv(d/'data/cghc_f7b_digitized.csv');gsd=pd.read_csv(d/'data/blatten_gsd_lo」
↪ ng.csv')
assert np.all(gsd.diameter_mm>0), 'Log grain-size axis requires strictly positive
↪ diameters'
f,ax=pair()
t=curve.time_since_refined_onset_s;u=curve.velocity_m_s
ax[0].fill_between(t,np.maximum(u-1.5,0),u+1.5,color=b,alpha=.16,lw=0)
ax[0].plot(t,u,'o-',color=b,ms=3,lw=1.3)
ax[0].axvspan(27,100,color=o,alpha=.10,zorder=0)
ax[0].set(xlabel='相对源释放时间 / s',ylabel='作者反演速度 / (m/s)',xlim=(0,100),ylim=(0,78))
ax[0].set_title('a  同事件运动曲线',loc='left')
for color,(name,group) in zip((b,o,p,STYLE['green']),gsd.groupby('sample',sort=False)):
    group=group.sort_values('diameter_mm')
    ax[1].semilogx(group.diameter_mm,group.coarse_adjusted_passing_percent,'o-',color=col」
    ↪ or,ms=2.5,lw=1.1,label=name.replace('V-',''))
ax[1].axhline(10,color=g,ls=':',lw=.8)
ax[1].set(xlabel='粒径 / mm',ylabel='粗料校正累计通过率 / %',ylim=(0,105),xlim=(.001,100))
ax[1].set_title('b  可比材料的四份真实样本',loc='left');ax[1].legend(loc='upper
↪ left',frameon=False)
export(f,ax,out,'fig01_new_evidence','新增证据：外部驱动与具名材料类比',
       '施剪窗 27–100 s；速度带仅为读图 ±1.5 m/s；粒度为 2025 Blatten 沉积样本。')

tr=pd.read_csv(r/'conditional_trajectories.csv')
f,ax=pair()
for name,label,color,ls in
↪ [('reference','有限排水参考',b,'-'),('ideal_drained','理想自由排水',g,'--')]:
    z=tr[(tr.case_id==name)&(tr.elapsed_s<=10)]
    ax[0].plot(z.elapsed_s,z.mean_effective_ratio,color=color,ls=ls,label=label)
ax[0].axhline(1/6,color=o,ls=':',lw=1.2,label='首层应力边界 1/6')
ax[0].set(xlabel='外部施剪开始后 / s',ylabel='平均有效应力比 s/E',xlim=(0,10),ylim=(-.01,.55))
ax[0].set_title('a  排水对照分离孔压反馈',loc='left');ax[0].legend(frameon=False,loc='center
↪ right')
for name,label,color,ls in [('reference','m0=0.54',b,'-'),('finite_H1_m0.57','m0=0.57',o,」
↪ '-'),('finite_H1_m0.63','m0=0.63（出域前）
↪ ',g,'--')]:
    z=tr[(tr.case_id==name)&(tr.elapsed_s<=10)]
    ax[1].plot(z.elapsed_s,z.fluid_supported_fraction,color=color,ls=ls,label=label)
    if name.endswith('0.63'):
        ax[1].plot(z.elapsed_s.iloc[-1],z.fluid_supported_fraction.iloc[-1],'x',color=g,m」
        ↪ s=6)
ax[1].set(xlabel='外部施剪开始后 / s',ylabel='条件孔压支承厚度比例
↪ f',xlim=(0,10),ylim=(-.03,1.03))
ax[1].set_title('b  初始结构改变条件响应',loc='left');ax[1].legend(frameon=False,loc='center
↪ right')
export(f,ax,out,'fig02_response_control','条件状态响应：相同驱动下的排水与结构差异',
```

```
       'H0=1 m、χ=0.02 /m 为设计参考;“出域”是饱和假说退出，不能判为安全。')

crit=pd.read_csv(r/'conditional_critical_chi.csv');grid=pd.read_csv(r/'critical_search_br
↪  ackets.csv')
assert np.all(crit.H0_m>0) and np.all(grid.chi_m_inv>0), 'Log axes require strictly
↪  positive input values'
f,ax=pair()
for m,color in ((.54,b),(.57,o)):
    z=crit[(crit.case_id.str.startswith('finite_'))&np.isclose(crit.m0,m)].sort_values('H
    ↪  0_m')
    ax[0].plot(z.H0_m,z.chi_critical_m_inv*1e4,'o-',color=color,label=f'm0={m:.2f}')
ax[0].set_xscale('log');ax[0].set(xlabel='设计初始厚度 H0 / m',ylabel='有效括区临界 χ / (10⁻⁴
↪  m⁻¹)',xlim=(.07,14))
ax[0].set_title('a
↪  12个有限成员中的有效反求',loc='left');ax[0].legend(frameon=False,loc='upper left')
z=grid[(grid.case_id=='finite_H1_m0.54')&(grid.record_type=='search_grid')].sort_values('
↪  chi_m_inv')
ax[1].semilogx(z.chi_m_inv,z.margin_mean_stress_ratio,'o-',color=b,ms=3,lw=1)
ax[1].axhline(0,color=o,lw=1,ls=':')
root=float(crit.loc[crit.case_id=='finite_H1_m0.54','chi_critical_m_inv'].iloc[0])
ax[1].plot(root,0,'D',color=o,ms=5,label='已核有效括区的根')
ax[1].set(xlabel='施剪转移组合 χ / m⁻¹',ylabel='窗内最小 s/E − 1/6')
ax[1].set_title('b  有限搜索保留非单调响应',loc='left');ax[1].legend(frameon=False,loc='upper
↪  right')
export(f,ax,out,'fig03_critical_conditions','反求临界组合：有限括区成立，全域单调性未证',
       '较密成员含提前出域；χ 及 1/χ 为组合条件，均非实测水深或剪切厚度。')

raw=pd.read_csv(d/'data/iverson2000_sparse_process_keypoints.csv')
obs=pd.concat([pd.read_csv(d/'data/iverson2000_matched_depth67_extra_keypoints.csv'),
               raw[raw.experiment_id.str.contains('dense')]],ignore_index=True)
f,ax=pair()
for axis,(exp,z),color,label in zip(ax,obs.groupby('experiment_id',sort=False),(b,o),('a
↪  松土：67 cm深传感器','b  密土：67 cm深传感器')):
    z=z.sort_values('time_since_water_application_s')
    time=z.time_since_water_application_s-z.time_since_water_application_s.iloc[0]
    axis.errorbar(time,z.pore_water_pressure_head_cm,xerr=z.time_reading_error_s,yerr=z.h
    ↪  ead_reading_error_cm,
                  fmt='o-',color=color,ms=4,capsize=2,lw=1.2)
    axis.set(xlabel='所读过程首点后 / s',ylabel='实测孔压水头 / cm',ylim=(0,72))
    axis.set_title(label,loc='left')
export(f,ax,out,'fig04_external_process','外部过程约束：松土增压与密土降压',
       '同一研究的两次代表过程、共 8 个稀疏读点；误差为读图支持，非独立样本置信区间。')
closure=pd.read_csv(r/'closure_sensitivity.csv');profiles=pd.read_csv(r/'closure_profile_
↪  samples.csv')
f,ax=pair()
for n,color in ((2,b),(3,o)):
    z=profiles[np.isclose(profiles.shape_n,n)]
    ax[0].plot(z.effective_stress_over_E,z.height_fraction,color=color,label=f'n={n}')
ax[0].set(xlabel='有效法向应力 / E',ylabel='相对高度 z/H',xlim=(0,.72),ylim=(0,1))
ax[0].set_title('a  边界剖面随闭合形状改变',loc='left');ax[0].legend(frameon=False,loc='upper
↪  right')
for n,color in ((2,b),(3,o)):
    z=closure[np.isclose(closure.shape_n,n)].iloc[0]
    ax[1].plot(n,z.chi_star_m_inv*1e4,'o',color=color,ms=7)
```

```
    ax[1].text(n,z.chi_star_m_inv*1e4+.09,f'sc/E={z.qcrit:.3f}',ha='center',fontsize=8)
ax[1].set(xlabel='数学压力形状 n',ylabel='参考成员临界 χ* / (10⁻⁴
↪  m⁻¹)',xlim=(1.5,3.5),ylim=(1.1,2.6),xticks=[2,3])
ax[1].set_title('b  同一参考成员的条件反求',loc='left')
export(f,ax,out,'fig05_closure_sensitivity','压力形状改变模型内边界，n=2 仍为主模型',
       'n=3 仅作数学压力测试；阈值比、应力映射、排水及停止边界成对改变。')

cum=pd.read_csv(r/'cumulative_shear_series.csv');mw=pd.read_csv(r/'material_window_sensit」
↪  ivity.csv')
f,ax=pair()
reference_id=str(closure.loc[np.isclose(closure.shape_n,2),'case_id'].iloc[0])
for regime,label,color,ls in [('design','设计施剪
↪  χ=0.02',b,'-'),('critical_upper_neighbor','临界根上邻侧',o,'-')]:
    z=cum[(cum.case_id==reference_id)&(cum.regime==regime)]
    ax[0].plot(z.Gamma_sh,z.s_ratio,color=color,ls=ls,lw=1.4,label=label)
z=cum[(cum.case_id==reference_id)&(cum.regime=='design')]
ax[0].plot(z.Gamma_sh,z.s_asy_ratio,color=g,ls='--',lw=1.2,label='弱排水、小 N 近似')
ax[0].axhline(1/6,color=p,ls=':',lw=.8)
ax[0].set(xlabel='模型输入累计剪切 Γsh',ylabel='首层前平均应力比
↪  s/E',xlim=(0,1.04),ylim=(.145,.52))
ax[0].set_title('a  相同累计剪切下的差别',loc='left');ax[0].legend(frameon=False,loc='upper
↪  right')
z=mw[mw.block=='window'].sort_values('T_s')
ax[1].plot(z.T_s,z.chi_star_m_inv*1e4,'o-',color=b,label='完整模型有效括区根')
ax[1].plot(z.T_s,z.chi_asy_m_inv*1e4,'s--',color=o,ms=4,label='累计剪切解析近似')
ax[1].set(xlabel='同起点前缀窗长 T / s',ylabel='临界 χ* / (10⁻⁴ m⁻¹)',xticks=[33,53,73])
ax[1].set_title('b  前缀窗口的临界组合',loc='left');ax[1].legend(frameon=False,loc='upper
↪  right')
export(f,ax,out,'fig06_cumulative_shear','累计剪切解释条件门槛，同时保留近似误差',
       'Γsh 无量纲；设计响应与临界根分开。窗口同时改变累计驱动和排水时间。')

boundary=pd.read_csv(r/'initial_state_boundary.csv').sort_values('q0')
response=pd.read_csv(r/'initial_state_response.csv')
diagnostics=pd.read_csv(r/'initial_state_cumulative_check.csv')
initial_series=pd.read_csv(r/'initial_state_cumulative_series.csv')
assert len(boundary)==4 and len(response)==8, 'Frozen four-state / two-rate matrix
↪  required'
f,ax=pair()
ax[0].plot(boundary.q0,boundary.Gamma_c_asy,'--',color=g,lw=1,label='一般初值解析近似')
ax[0].plot(boundary.q0,boundary.Gamma_star_window,'o',color=b,ms=6,label='完整窗有效根
↪  （有限点）')
comparison=float(response.loc[response.regime=='common_comparison','Gamma_window_prescrib」
↪  ed'].iloc[0])
ax[0].axhline(comparison,color=o,ls=':',lw=1,label='共同比较档的窗末 Γ')
ax[0].set(xlabel='设计初始有效应力比 q0=s_init/E',ylabel='自身临界组合 χ*I_U（无量纲）',
          xlim=(.13,.79),ylim=(-.04,1.62),xticks=boundary.q0)
ax[0].set_title('a  初态—累计剪切联合边界',loc='left')
ax[0].legend(frameon=False,loc='upper left',fontsize=7)
for rowindex,(regime,color) in enumerate([('common_comparison',o),('common_design',b)]):
    z=response[response.regime==regime].sort_values('q0')
    for j,row in enumerate(z.itertuples()):
        hit=pd.notna(row.onset_elapsed_s)
        ax[1].plot(j,rowindex,marker='o' if hit else 's',ms=6,
                   markerfacecolor=color if hit else 'white',markeredgecolor=color,lw=0)
```

```
        label=f'{row.onset_elapsed_s:.3g} s' if hit else '窗内未达'
        if hit and regime=='common_comparison' and row.q0==.5:
            label=f'{row.onset_elapsed_s:.3f} s'
        ax[1].text(j,rowindex+.20,label,ha='center',va='bottom',fontsize=7.5,color=color)
ax[1].set(xlabel='设计初始应力比 q0',xlim=(-.5,3.5),ylim=(-.45,1.65),
          xticks=range(4),xticklabels=[f'{x:g}' for x in boundary.q0],
          yticks=[0,1],yticklabels=['共同比较档','设计档'])
ax[1].set_title('b  两个共同 χ 下的实际状态',loc='left')
export(f,ax,out,'fig07_initial_state_boundary','约 0.98 的参考累计剪切条件依赖初始状态',
       '四点均为设计初态；数值根只显示实际有限点。空方块为 73 s 内未达，不赋到达时间。')

f,ax=pair()
near=str(boundary.iloc[0].case_id)
for regime,label,color in [('own_critical_upper_neighbor','自身临界上邻侧',b),
                           ('common_comparison','共同比较档',o)]:
    z=initial_series[(initial_series.case_id==near)&(initial_series.regime==regime)]
    rec=diagnostics[(diagnostics.case_id==near)&(diagnostics.regime==regime)].iloc[0]
    assert rec.Gamma_c_asy>0 and rec.L_goal>0, 'Positive pre-layer targets required'
    progress=-z.log_stress_ratio/rec.L_goal
    ax[0].plot(z.Gamma_sh/rec.Gamma_c_asy,progress,color=color,lw=1.4,label=label)
ax[0].plot([0,1],[0,1],'--',color=g,lw=1,label='弱排水解析近似')
ax[0].set(xlabel='累计剪切 / 一般初值解析 Γc',ylabel='对数应力降 / 初始目标 L_goal',
          xlim=(0,1.13),ylim=(0,1.17))
ax[0].set_title('a  q0=0.17：小目标放大排水作用',loc='left')
ax[0].legend(frameon=False,loc='upper left',fontsize=7)
for regime,label,color,marker in [('own_critical_upper_neighbor','自身临界上邻侧',b,'o'),
                                  ('common_comparison','共同比较档',o,'s'),
                                  ('common_design','设计档',p,'^')]:
    z=diagnostics[diagnostics.regime==regime].sort_values('q0')
    assert np.all(z.eps_J>0), 'Log drainage diagnostic requires positive values'
    ax[1].semilogy(z.q0,z.eps_J,marker+'-',color=color,lw=1,ms=4,label=label)
ax[1].set(xlabel='设计初始应力比 q0',ylabel='绝对排水对数贡献 / L_goal（εJ）',
          xlim=(.13,.79),xticks=boundary.q0,ylim=(5e-6,.3))
ax[1].set_title('b  排水须相对于目标间隔衡量',loc='left')
ax[1].legend(frameon=False,loc='upper right',fontsize=7)
export(f,ax,out,'fig08_initial_state_diagnostics','近边界的弱施剪可使排水改变累计剪切近似',
       '诊断由保存轨迹直接积分；不设置通过百分比。数值残差检验约化关系，不证明自然精度。')

sources=[d/'data/cghc_f7b_digitized.csv',d/'data/blatten_gsd_long.csv',d/'data/iverson200
↪  0_sparse_process_keypoints.csv',
         d/'data/iverson2000_matched_depth67_extra_keypoints.csv',
         r/'conditional_trajectories.csv',r/'conditional_critical_chi.csv',r/'critical_se
         ↪  arch_brackets.csv',
         r/'closure_sensitivity.csv',r/'closure_profile_samples.csv',r/'cumulative_shear_
         ↪  series.csv',r/'material_window_sensitivity.csv',
         r/'initial_state_boundary.csv',r/'initial_state_response.csv',r/'initial_state_c
         ↪  umulative_check.csv',r/'initial_state_cumulative_series.csv']
receipt={'started_utc':started,'finished_utc':datetime.now(timezone.utc).isoformat(),'cod
↪  e_sha256':sha(__file__),
         'input_sha256':{str(x):sha(x) for x in
         ↪  sources},'style':STYLE,'solver_started':False,
         'output_sha256':{x.name:sha(x) for x in sorted(out.iterdir()) if x.is_file() and
         ↪  x.name!='FIGURES_RUN_RECEIPT.json'},
         'visual_inspection':'pending'}
```

```
    (out/'FIGURES_RUN_RECEIPT.json').write_text(json.dumps(receipt,ensure_ascii=False,indent=
    ↪  2)+'\n')
    print('Rendered 8 figures as PNG/SVG/PDF')


if __name__=='__main__': main()
```

q2/sol.py

```
from __future__ import annotations
import argparse
import hashlib
import json
import math
import platform
import sys
from dataclasses import dataclass, replace, asdict
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd
import scipy
from scipy.integrate import solve_ivp, cumulative_simpson, simpson
from scipy.optimize import brentq

G = 9.81


def fingerprint(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def save_json(path, obj):
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False)+'\n')


@dataclass(frozen=True)
class Case:
    m0: float
    H0: float
    mcrit: float
    rho_s: float
    rho_f: float
    mu: float
    a: float
    sigma0: float
    delta: float
    Kfs_transfer_m_s: float
    water_conversion_mu_Pa_s: float
    theta_deg: float
    chi_m_inv: float
    curve_shift: float = 0.0
    start: float = 27.0
    end: float = 100.0
```

```
    mode: str = 'coupled'
    shape_n: float = 2.0
    initial_stress_ratio: float = 0.5

    @property
    def s_init(self):
        return self.initial_stress_ratio*self.E

    @property
    def qcrit(self):
        return (self.shape_n-1)/(2*(self.shape_n+1))

    @property
    def ki(self):
        return self.Kfs_transfer_m_s*self.water_conversion_mu_Pa_s/(self.rho_f*G)

    @property
    def Hs(self):
        return self.m0*self.H0

    @property
    def E(self):
        return (self.rho_s-self.rho_f)*G*math.cos(math.radians(self.theta_deg))*self.Hs


class Forcing:
    def __init__(self, path):
        d = pd.read_csv(path)
        self.t = d.time_since_refined_onset_s.to_numpy(float)
        self.u = d.velocity_m_s.to_numpy(float)
        if not (np.all(np.diff(self.t)>0) and np.all(self.u>=0)):
            raise ValueError('Forcing requires ordered times and nonnegative velocities')

    def speed(self, case, t):
        if case.start < self.t[0] or case.end > self.t[-1]:
            raise ValueError('Requested forcing window exceeds digitized data; no
            ↪  extrapolation')
        return max(0.0, float(np.interp(t+case.start, self.t, self.u))+case.curve_shift)


def constitutive(case, m, s, gamma):
    if not (0<m<1) or s <= -case.sigma0:
        raise ValueError('Constitutive state outside domain')
    alpha = case.a/(m*(s+case.sigma0))
    N = 0.0 if gamma == 0 else case.mu*gamma/(case.rho_s*gamma**2*case.delta**2+max(s,0.0))
    meq = case.mcrit/(1+math.sqrt(N))
    k = case.ki*math.exp((case.m0-m)/0.04)
    return alpha, N, meq, k


def closure(case, m, s):
    H = case.Hs/m
    zeta = s/case.E
    if zeta >= case.qcrit:
```

```
        e = (case.shape_n+1)/case.shape_n*(case.E/2-s)
        r = 1.0
        Jfactor = case.shape_n*e/H
    else:


        r = math.sqrt(max(0.0,zeta/case.qcrit))
        e = case.E*(1-(case.shape_n-1)*r/case.shape_n)
        Jfactor = case.E/H
    return H, e, r, Jfactor


def rhs(case, forcing, t, y):
    m,s,_=y
    U=forcing.speed(case,t)
    gamma=0.0 if case.mode=='zero_shear' else case.chi_m_inv*U
    alpha,N,meq,k=constitutive(case,m,s,gamma)
    H,e,r,Jfactor=closure(case,m,s)
    if case.mode=='drained':
        dm=-m*gamma*(m-meq)
        J=H*dm/m
        ds=0.0
    else:
        J=k*Jfactor/case.mu
        dm=m*J/H
        ds=(gamma*(m-meq)+J/H)/alpha
    return (dm,ds,J)


def profile(case,m,s,x):
    H,e,r,_=closure(case,m,s)
    if s/case.E >= case.qcrit:
        pe=e*(1-x**case.shape_n)
        effective=case.E*(1-x)-pe
    else:
        rn=max(r,1e-100)
        pe=np.where(x<=r,e-case.E*x**case.shape_n/(case.shape_n*rn**(case.shape_n-1)),case.E*
        ↪  (1-x))
        effective=np.where(x<=r,case.E*((case.shape_n-1)*r/case.shape_n-x+x**case.shape_n/(ca
        ↪  se.shape_n*rn**(case.shape_n-1))),0.0)
    hydro=case.rho_f*G*math.cos(math.radians(case.theta_deg))*H*(1-x)
    return pe, effective, hydro+pe


def initial_classification(case):
    q0=case.initial_stress_ratio
    B=case.rho_f*G*math.cos(math.radians(case.theta_deg))*case.H0
    qmax=.5+B/((case.shape_n+1)*case.E)
    if not math.isfinite(q0) or q0<0 or q0>qmax+1e-12:
        status='invalid_initial_state'
    elif math.isclose(q0,0,rel_tol=0,abs_tol=1e-12):
        status='initial_closure_limit'
    elif math.isclose(q0,qmax,rel_tol=0,abs_tol=1e-12):
        status='initial_saturation_boundary'
```

```
    elif q0<case.qcrit-1e-12:
        status='initial_positive_thickness_layer'
    elif math.isclose(q0,case.qcrit,rel_tol=0,abs_tol=1e-12):
        status='initial_first_layer_boundary'
    else:
        status='admissible_unreached_initial_state'
    f0=1-math.sqrt(q0/case.qcrit) if 0<=q0<case.qcrit else (0.0 if q0>=case.qcrit else None)
    if status=='invalid_initial_state':f0=None
    return {'q0':q0,'s_init_Pa':case.s_init,'qcrit':case.qcrit,'qmax':qmax,
            'initial_status':status,'initial_fluid_supported_fraction':f0,
            'initial_condition_satisfied':None if status in
            ↪  ('initial_closure_limit','invalid_initial_state') else status in
            ↪  ('initial_positive_thickness_layer','initial_first_layer_boundary'),
            'dynamic_first_hit_elapsed_s':None,
            'identity':'mathematical closure admission only; not local observed or credible
            ↪  range'}


def run_case(case, forcing, numerics, dense=True, keep_solution=False):
    if not (0<case.m0<1 and 0<case.mcrit<1 and case.H0>0 and case.ki>0 and case.mu>0 and
    ↪  case.shape_n>1):
        raise ValueError('Invalid physical input')
    admission=initial_classification(case)
    if admission['initial_status']!='admissible_unreached_initial_state':
        raise ValueError('Initial state is classified separately:
        ↪  '+admission['initial_status'])
    if case.mode=='drained' and case.initial_stress_ratio!=.5:
        raise ValueError('Historical ideal-drained control is restricted to hydrostatic
        ↪  initial state')
    def zero_stress(t,y): return y[1]
    zero_stress.terminal=True
    zero_stress.direction=-1
    def desaturation(t,y):
        H,e,r,_=closure(case,y[0],y[1])
        return e+case.rho_f*G*math.cos(math.radians(case.theta_deg))*H/case.shape_n
    desaturation.terminal=True
    desaturation.direction=-1
    def fluid_layer(t,y): return y[1]/case.E-case.qcrit
    fluid_layer.terminal=False
    fluid_layer.direction=-1
    duration=case.end-case.start
    sol=solve_ivp(lambda t,y:rhs(case,forcing,t,y),(0,duration),(case.m0,case.s_init,0.0),
                  method='Radau',rtol=numerics['rtol'],atol=(1e-11,1e-6,1e-10),
                  max_step=numerics['max_step_s'],dense_output=True,
                  events=(zero_stress,desaturation,fluid_layer))
    if not sol.success: raise RuntimeError(sol.message)
    state='completed_conditional_window'
    if len(sol.t_events[0]): state='mean_stress_zero_closure_limit'
    if len(sol.t_events[1]): state='saturation_hypothesis_boundary'
    if dense:
        ts=np.unique(np.r_[np.arange(0,sol.t[-1],numerics['output_step_s']),sol.t[-1],sol.t_e
        ↪  vents[2]])
    else:
        ts=np.unique(np.r_[sol.t,sol.t_events[2]])
```

```
ys=sol.sol(ts)
rows=[]
residual=[]
for t,(m,s,budget) in zip(ts,ys.T):
    H,e,r,_=closure(case,m,s)
    U=forcing.speed(case,t)
    gamma=0.0 if case.mode=='zero_shear' else case.chi_m_inv*U
    alpha,N,meq,k=constitutive(case,m,s,gamma)
    dm,ds,J=rhs(case,forcing,t,(m,s,budget))
    if case.mode=='drained': e=0.0; r=1.0
    resid=gamma*(m-meq)-alpha*ds+dm/m
    residual.append(resid)
    rows.append({'elapsed_s':t,'source_relative_s':t+case.start,'U_input_m_s':U,
                 'gamma_input_s_inv':gamma,'m':m,'H_m':H,'mean_effective_stress_Pa':s,
                 'mean_effective_ratio':s/case.E,'basal_excess_Pa':e,'basal_excess_ratio'
                 ↪  :e/case.E,
                 'fluid_supported_fraction':1-r,'drainage_flux_m_s':J,'cumulative_outflow
                 ↪  _m':budget,
                 'skeleton_thickness_m':H*r,'skeleton_to_delta_ratio':H*r/case.delta,
                 'alpha_Pa_inv':alpha,'k_m2':k,'N':N,'meq':meq,
                 'solid_volume_residual_m':m*H-case.Hs,
                 'water_budget_residual_m':H-case.H0+budget,'constitutive_residual_s_inv'
                 ↪  :resid})
frame=pd.DataFrame(rows)
x=np.linspace(0,1,1001)
profile_checks=[]
for j in np.unique(np.linspace(0,len(frame)-1,min(21,len(frame))).astype(int)):
    row=frame.iloc[j]
    pe,eff,total=profile(case,row.m,row.mean_effective_stress_Pa,x)
    if case.mode=='drained':
        eff=case.E*(1-x);total=case.rho_f*G*math.cos(math.radians(case.theta_deg))*row.H_
        ↪  m*(1-x)
    profile_checks.append((np.min(eff),np.min(total),abs(np.trapezoid(eff,x)-row.mean_eff
    ↪  ective_stress_Pa)/case.E))
onset=float(sol.t_events[2][0]) if len(sol.t_events[2]) else None
summary={'q0':case.initial_stress_ratio,'s_init_Pa':case.s_init,'initial_status':admissio
↪  n['initial_status'],'status':state,'onset_elapsed_s':onset,'onset_source_relative_s':
↪  None if onset is None else
↪  onset+case.start,
         'max_fluid_fraction':float(frame.fluid_supported_fraction.max()),
         'min_mean_effective_ratio':float(frame.mean_effective_ratio.min()),
         'final_m':float(frame.m.iloc[-1]),'final_H_m':float(frame.H_m.iloc[-1]),
         'end_elapsed_s':float(sol.t[-1]),'ki_m2':case.ki,'E_Pa':case.E,
         'max_solid_residual_m':float(frame.solid_volume_residual_m.abs().max()),
         'max_water_residual_m':float(frame.water_budget_residual_m.abs().max()),
         'max_constitutive_residual_s_inv':float(max(abs(x) for x in residual)),
         'min_profile_effective_stress_Pa':float(min(x[0] for x in profile_checks)),
         'min_profile_total_pore_Pa':float(min(x[1] for x in profile_checks)),
         'max_profile_mean_relative_error':float(max(x[2] for x in profile_checks)),
         'min_skeleton_to_delta_ratio':float(frame.skeleton_to_delta_ratio.min()),
         'continuum_scale_warning':bool(frame.skeleton_to_delta_ratio.min()<10),
         'continuum_warning_identity':'10 delta is a disclosed resolution diagnostic, not
         ↪  a calibrated formation boundary'}
if keep_solution: return frame,summary,sol
```

```
    return frame,summary


def threshold(case,forcing,cfg,numerics):
    admission=initial_classification(case)
    if admission['initial_status']!='admissible_unreached_initial_state':
        return {'threshold_status':admission['initial_status'],'chi_critical_m_inv':None,'cri
        ↪  tical_identity':'no new dynamic threshold for classified initial
        ↪  states'},[]
    search=cfg['threshold_search']

    maxu=float(max(forcing.u))+max(case.curve_shift,0)
    lo=search['chi_min_m_inv']; hi=search['max_gamma_s_inv']/maxu
    grid=np.geomspace(lo,hi,search['bracket_points'])
    values=[];records=[]
    def objective(chi):
        _,s=run_case(replace(case,chi_m_inv=float(chi)),forcing,numerics,False)
        if s['status']!='completed_conditional_window':
            return None,s
        return s['min_mean_effective_ratio']-case.qcrit,s
    for chi in grid:
        try:
            val,s=objective(chi)
        except (ValueError,RuntimeError) as error:
            val=None;s={'status':'numerical_or_physical_domain_failure','end_elapsed_s':None,
            ↪  'onset_elapsed_s':None,'failure':str(error)}
        values.append(val)
        records.append({'record_type':'search_grid','chi_m_inv':float(chi),'margin_mean_stres
        ↪  s_ratio':val,
                        'status':s['status'],'end_elapsed_s':s['end_elapsed_s'],'onset_elapse
                        ↪  d_s':s['onset_elapsed_s']})
    roots=[];root_records=[]
    for i in range(len(grid)-1):
        if values[i] is not None and values[i+1] is not None and values[i]*values[i+1]<0:
            def valid_objective(chi):
                val,s=objective(chi)
                if val is None: raise ValueError('Threshold bracket entered an
                ↪  incomplete/invalid forcing window')
                return val
            root=brentq(valid_objective,grid[i],grid[i+1],xtol=search['xtol_m_inv'])
            roots.append(float(root))
            eps=max(search['xtol_m_inv']*10,root*1e-5)
            left,ls=objective(max(grid[i],root-eps));right,rs=objective(min(grid[i+1],root+ep
            ↪  s))
            root_records.append({'record_type':'valid_root_bracket','chi_m_inv':float(root),
                                 'bracket_lo_m_inv':float(grid[i]),'bracket_hi_m_inv':float(g
                                 ↪  rid[i+1]),
                                 'bracket_margin_lo':values[i],'bracket_margin_hi':values[i+1
                                 ↪  ],
                                 'neighbor_left_margin':left,'neighbor_right_margin':right,
                                 'neighbor_left_chi_m_inv':float(max(grid[i],root-eps)),
                                 'neighbor_right_chi_m_inv':float(min(grid[i+1],root+eps)),
                                 'neighbor_left_status':ls['status'],'neighbor_right_status':
                                 ↪  rs['status'],
```

```
                                'root_event':'first-layer stress boundary within complete
                                ↪  prescribed window'})
    valid_values=[v for v in values if v is not None]
    invalid_count=len(values)-len(valid_values)
    monotone=bool(np.all(np.diff(valid_values)<=1e-5)) if len(valid_values)>1 else None
    if roots: threshold_state='root_found_in_valid_bracket'
    elif invalid_count:
    ↪  threshold_state='domain_exit_or_failure_prevents_complete_window_search'
    elif valid_values and min(valid_values)>0:
    ↪  threshold_state='no_onset_at_valid_sampled_chi_values'
    else: threshold_state='onset_at_search_lower_bound_or_no_crossing_found'
    out={'search_chi_min_m_inv':float(lo),'search_chi_max_m_inv':float(hi),
         'sampled_margin_nonincreasing':monotone,'crossings_found':len(roots),
         'chi_critical_m_inv':roots[0] if roots else None,
         'equivalent_length_at_critical_m':1/roots[0] if roots else None,
         'valid_complete_window_grid_points':len(valid_values),'incomplete_or_failed_grid_poi
         ↪  nts':invalid_count,
         'critical_identity':'first valid sampled bracket crossing; finite grid does not
         ↪  exclude missed tangencies or all other nonmonotone intervals',
         'threshold_status':threshold_state}
    return out,records+root_records


def speed_integral(forcing, start, end):
    if start<forcing.t[0] or end>forcing.t[-1] or end<start:
        raise ValueError('Integral outside actual curve support')
    t=np.unique(np.r_[start,forcing.t[(forcing.t>start)&(forcing.t<end)],end])
    return float(np.trapezoid(np.interp(t,forcing.t,forcing.u),t))


def diagnostic_grid(case,forcing,cut,max_step):
    nodes=np.unique(np.r_[0,forcing.t[(forcing.t>case.start)&(forcing.t<case.start+cut)]-case
    ↪  .start,cut])
    return np.unique(np.concatenate([np.linspace(a,b,max(2,int(math.ceil((b-a)/max_step))+1))
    ↪  for a,b in zip(nodes[:-1],nodes[1:])]))


def shear_diagnostics(case,forcing,sol,state,case_id,regime,max_step):
    hit=state['onset_elapsed_s'];cut=state['end_elapsed_s'] if hit is None else hit
    ts=diagnostic_grid(case,forcing,cut,max_step);ys=sol.sol(ts)
    m=ys[0];s=ys[1];gamma=np.asarray([(0.0 if case.mode=='zero_shear' else
    ↪  case.chi_m_inv*forcing.speed(case,t)) for t in ts])
    N=[];psi=[];jh=[]
    for t,mi,si,ga in zip(ts,m,s,gamma):
        alpha,ni,meq,k=constitutive(case,mi,si,ga);H,e,r,jfac=closure(case,mi,si)
        N.append(ni);psi.append(mi-meq);jh.append(k*jfac/(case.mu*H))
    N=np.asarray(N);psi=np.asarray(psi);jh=np.asarray(jh)
    shear=m*gamma*psi/case.a;drain=m*jh/case.a
    Is=cumulative_simpson(shear,x=ts,initial=0);Id=cumulative_simpson(drain,x=ts,initial=0)
    lhs=np.log((s+case.sigma0)/(case.s_init+case.sigma0));volume=(m-case.m0)/case.a
    residual=lhs-Is-volume
    Delta=case.mcrit-case.m0
    IU=speed_integral(forcing,case.start,case.end)
```

```
Gamma_asy=case.a/(case.m0*Delta)*math.log((case.s_init+case.sigma0)/(case.qcrit*case.E+ca
↪  se.sigma0)) if Delta>0 else
↪  None
Lgoal=math.log((case.s_init+case.sigma0)/(case.qcrit*case.E+case.sigma0)) if
↪  case.initial_stress_ratio>case.qcrit else None
Ghit=None if hit is None else
↪  case.chi_m_inv*speed_integral(forcing,case.start,case.start+hit)
def finite_or_none(value): return float(value) if np.isfinite(value) else None
eps_m=float(np.max(np.abs(m-case.m0))/Delta) if Delta>0 else None
eps_N=float(np.max(case.mcrit*np.sqrt(N)/((1+np.sqrt(N))*Delta))) if Delta>0 else None
record={'case_id':case_id,'regime':regime,'shape_n':case.shape_n,'qcrit':case.qcrit,
        'criterion_origin':'closure_derived','hypothesis_id':'H_SATURATED_MIXED_MATRIX_CH
        ↪  I_ASL',
        'forcing_origin':'post_event_inversion','forcing_window':f'[{case.start:g},{case.
        ↪  end:g}]',
        'q0':case.initial_stress_ratio,'s_init_Pa':case.s_init,'m0':case.m0,'mcrit':case.
        ↪  mcrit,'H0_m':case.H0,'chi_trajectory_m_inv':case.chi_m_inv,
        'status':state['status'],'diagnostic_cut_elapsed_s':float(cut),
        'diagnostic_cut_identity':'first_hit' if hit is not None else ('legal_window_end'
        ↪  if state['status']=='completed_conditional_window' else 'hypothesis_exit'),
        'I_U_window_m':IU,'Gamma_window_prescribed':case.chi_m_inv*IU,
        'Gamma_until_actual_stop':case.chi_m_inv*speed_integral(forcing,case.start,case.s
        ↪  tart+state['end_elapsed_s']),
        'Gamma_at_first_hit':Ghit,'Gamma_c_asy':Gamma_asy,'chi_asy_m_inv':None if
        ↪  Gamma_asy is None else Gamma_asy/IU,
        'Gamma_hit_relative_error_vs_asy':None if Ghit is None or Gamma_asy is None else
        ↪  (Ghit-Gamma_asy)/Gamma_asy,
        'max_N_before_cut':float(np.max(N)),'eps_m':eps_m,'eps_N':eps_N,
        'signed_shear_log_contribution':float(Is[-1]),'signed_volume_log_contribution':fl
        ↪  oat(volume[-1]),
        'direct_drain_log_integral':float(Id[-1]),'absolute_shear_log_integral':float(sim
        ↪  pson(np.abs(shear),x=ts)),
        'absolute_drain_log_integral':float(simpson(np.abs(drain),x=ts)),
        'absolute_gamma_psi_integral':float(simpson(np.abs(gamma*psi),x=ts)),
        'absolute_J_over_H_integral':float(simpson(np.abs(jh),x=ts)),
        'max_identity_residual':float(np.max(np.abs(residual))),
        'final_identity_residual':float(residual[-1]),'drain_quadrature_minus_delta_m_ove
        ↪  r_a':float(Id[-1]-volume[-1]),
        'L_goal':Lgoal,'eps_J':None if Lgoal is None else
        ↪  float(simpson(np.abs(drain),x=ts)/Lgoal),
        'absolute_drain_to_shear_ratio':None if simpson(np.abs(shear),x=ts)==0 else
        ↪  float(simpson(np.abs(drain),x=ts)/simpson(np.abs(shear),x=ts)),
        'diagnostic_samples':len(ts),'diagnostic_max_step_s':float(np.max(np.diff(ts))),
        'approximation_status':'not_applicable_Delta_nonpositive' if Delta<=0 else
        ↪  ('pre_layer_comparison_no_prespecified_success_cutoff' if hit is not None
        ↪  else 'diagnostics_only_no_first_hit'),
        'approximation_reason':'requires Delta>0, weak drainage, nearly fixed m and N
        ↪  correction small relative to Delta; report diagnostics without a pass
        ↪  percentage'}
Gamma=np.asarray([case.chi_m_inv*speed_integral(forcing,case.start,case.start+t) for t in
↪  ts])
sasy=np.full_like(s,np.nan) if Delta<=0 else
↪  (case.s_init+case.sigma0)*np.exp(-case.m0*Delta*Gamma/case.a)-case.sigma0
frame=pd.DataFrame({'case_id':case_id,'regime':regime,'elapsed_s':ts,'m':m,'s_Pa':s,'s_ra
↪  tio':s/case.E,
```

```
                        'gamma_s_inv':gamma,'N':N,'tan_psi':psi,'J_over_H_s_inv':jh,
                        'Gamma_sh':Gamma,'s_asy_ratio':sasy/case.E,'shear_log_integrand':shea
                        ↪  r,
                        'drain_log_integrand':drain,'shear_log_integral':Is,'direct_drain_log
                        ↪  _integral':Id,
                        'volume_log_term':volume,'log_stress_ratio':lhs,'identity_residual':r
                        ↪  esidual})
    return record,frame


def reached_state(state):
    hit=state['onset_elapsed_s'] is not None
    if state['status']=='completed_conditional_window':
        return 'later_first_hit' if hit else 'valid_no_hit_in_window'
    return 'first_hit_then_domain_exit' if hit else 'domain_exit_before_first_hit'


def sol():
    parser=argparse.ArgumentParser(description='Q2 v3 initial-effective-stress /
    ↪  cumulative-shear joint conditional boundary')
    parser.add_argument('--delivery-dir',type=Path,default=Path(__file__).resolve().parent)
    parser.add_argument('--output-dir',type=Path)
    args=parser.parse_args();started=datetime.now(timezone.utc).isoformat()
    delivery=args.delivery_dir.resolve();out=(args.output_dir or delivery/'results').resolve()
    out.mkdir(parents=True,exist_ok=True)
    cfg=json.loads((delivery/'config.json').read_text());en=cfg['initial_state_enhancement'];
    ↪  num=cfg['numerics']
    forcing=Forcing(delivery/cfg['forcing']['source_file']);ref=Case(**cfg['reference'])
    baseline_cfg=json.loads((delivery/'data/v2_reference_config.json').read_text())
    baseline_binding=json.loads((delivery/'data/BASELINE_BINDING.json').read_text())
    baseline_hashes={r['path'].split('/',1)[1]:r['sha256'] for r in
    ↪  baseline_binding['protected_checks']
                    if r['path'].startswith('Q2_active_data_20261029_v2/')}
    if fingerprint(delivery/'data/v2_reference_config.json')!=baseline_hashes['config.json']:
        raise RuntimeError('Inherited reference configuration does not match reviewed v2
        ↪  binding')
    if ref!=Case(**baseline_cfg['reference']):
        raise ValueError('Frozen v3 reference differs from reviewed v2; inherited outputs
        ↪  require re-audit')
    if fingerprint(delivery/cfg['forcing']['source_file'])!=baseline_hashes[cfg['forcing']['s
    ↪  ource_file']]:
        raise ValueError('Forcing differs from reviewed v2; historical results cannot be
        ↪  silently reused')
    import shutil
    reuse=delivery/en['v2_reused_results']

    reused=[]
    for source in sorted(reuse.iterdir()):
        if source.is_file() and source.name not in
        ↪  ('ANALYSIS_RUN_RECEIPT.json','summary.json'):
            if fingerprint(source)!=baseline_hashes['results/'+source.name]:
                raise RuntimeError('Historical result reuse input has changed: '+source.name)
            shutil.copyfile(source,out/source.name);reused.append(source.name)
    oldcum=pd.read_csv(reuse/'cumulative_shear_check.csv')
```

```
oldsum=pd.read_csv(reuse/'enhancement_case_summary.csv')
oldbr=pd.read_csv(reuse/'enhancement_search_brackets.csv')
reference_old=oldsum[(oldsum.shape_n==2)&(oldsum.m0==.54)&(oldsum.mcrit==.60)&(oldsum.T_s
↪  ==73)].iloc[0]
summaries=[];trajectories=[];diagnostics=[];series=[];bounds=[];brackets=[];regressions=[
↪  ];relax=[]
IU=speed_integral(forcing,ref.start,ref.end)
for i,q0 in enumerate(en['q0_values'],1):
    case=replace(ref,initial_stress_ratio=float(q0));cid=f'V3_Q{i:02d}'
    admission=initial_classification(case)
    if admission['initial_status']!='admissible_unreached_initial_state':
        raise ValueError('Frozen main matrix contains non-interior initial state')
    for regime,chi in [('common_design',en['common_chi_design_m_inv']),('common_compariso
    ↪  n',en['common_chi_comparison_m_inv'])]:
        c=replace(case,chi_m_inv=float(chi))
        f,state,sol=run_case(c,forcing,num,keep_solution=True)
        f.insert(0,'case_id',cid);f['regime']=regime;f['q0']=q0;trajectories.append(f)
        rec,ser=shear_diagnostics(c,forcing,sol,state,cid,regime,en['diagnostic_max_step_
        ↪  s'])
        diagnostics.append(rec);series.append(ser)
        H,e,r,jfactor=closure(c,c.m0,c.s_init)
        gamma0=chi*forcing.speed(c,0)
        alpha,N,meq,k=constitutive(c,c.m0,c.s_init,gamma0)
        J=k*jfactor/c.mu
        dm,ds,_=rhs(c,forcing,0,(c.m0,c.s_init,0))
        summaries.append({'case_id':cid,'regime':regime,'shape_n':c.shape_n,
            'closure_id':'no_tension_power_n2','criterion_origin':'closure_derived',
            'hypothesis_id':cfg['hypothesis_id'],'forcing_origin':'post_event_inversion',
            'initial_state_origin':en['initial_state_identity'],'forcing_window':'[27,100
            ↪  ]',
            'q0':q0,'s_init_Pa':c.s_init,'qmax':admission['qmax'],'qcrit':c.qcrit,
            'm0':c.m0,'mcrit':c.mcrit,'H0_m':c.H0,'chi_m_inv':chi,
            'T_s':c.end-c.start,'I_U_m':IU,'initial_basal_excess_Pa':e,'J0_m_s':J,
            'm_dot0_s_inv':dm,'s_dot0_Pa_s':ds,'initial_s_dot_sign':int(np.sign(ds)),
            'gamma0_s_inv':gamma0,'N0':N,'tan_psi0':c.m0-meq,
            'initial_shear_driver_s_inv':gamma0*(c.m0-meq),'initial_J_over_H_s_inv':J/H,
            'initial_driver_sum_s_inv':gamma0*(c.m0-meq)+J/H,
            'top_liquid_supply_required_initially':bool(J<0),
            'Gamma_window_prescribed':chi*IU,'Gamma_at_first_hit':rec['Gamma_at_first_hit
            ↪  '],
            'Gamma_c_asy':rec['Gamma_c_asy'],'chi_asy_m_inv':rec['chi_asy_m_inv'],
            'reached_status':reached_state(state),'origin':'computed_v3; shared
            ↪  hydrostatic cases are regression reruns',**state})
        if q0==.5:
            oldregime='design' if regime=='common_design' else 'critical_upper_neighbor'
            original=oldcum[(oldcum.case_id=='V2_01')&(oldcum.regime==oldregime)].iloc[0]
            onset_error=abs(state['onset_elapsed_s']-float(original.diagnostic_cut_elapse
            ↪  d_s))
            gamma_error=abs(rec['Gamma_at_first_hit']-float(original.Gamma_at_first_hit))
            regressions.append({'case_id':cid,'regime':regime,'source_case':'V2_01',
                'source_regime':oldregime,'onset_difference_s':onset_error,
                'Gamma_hit_difference':gamma_error,'pass':onset_error<1e-6 and
                ↪  gamma_error<1e-7})
        print(cid,q0,regime,reached_state(state),'hit',state['onset_elapsed_s'],flush=Tru
        ↪  e)
```

```
if q0==.5:
    root=float(reference_old.chi_star_m_inv)
    oldroot=oldbr[(oldbr.case_id=='V2_01')&(oldbr.record_type=='valid_root_bracket')]
    ↪  .iloc[0]
    grid=oldbr[(oldbr.case_id=='V2_01')&(oldbr.record_type=='search_grid')].drop(colu
    ↪  mns=['case_id','origin']).to_dict('records')
    eps=max(cfg['threshold_search']['xtol_m_inv']*10,root*1e-5)
    neighbor={}
    for side,chi in [('left',root-eps),('right',root+eps)]:
        _,state=run_case(replace(case,chi_m_inv=chi),forcing,num,False)
        neighbor[f'neighbor_{side}_chi_m_inv']=chi
        neighbor[f'neighbor_{side}_margin']=state['min_mean_effective_ratio']-case.qc
        ↪  rit if state['status']=='completed_conditional_window' else
        ↪  None
        neighbor[f'neighbor_{side}_status']=state['status']
    if not (neighbor['neighbor_left_margin']>0 and
    ↪  neighbor['neighbor_right_margin']<0):
        raise RuntimeError('Reused hydrostatic root lost its valid opposite-sign
        ↪  neighbors')
    grid.append({'record_type':'valid_root_bracket','chi_m_inv':root,
        'bracket_lo_m_inv':float(oldroot.bracket_lo_m_inv),'bracket_hi_m_inv':float(o
        ↪  ldroot.bracket_hi_m_inv),
        'bracket_margin_lo':float(oldroot.bracket_margin_lo),'bracket_margin_hi':floa
        ↪  t(oldroot.bracket_margin_hi),
        'neighbor_check_origin':'v2 exact root reused; two neighbors actually
        ↪  recomputed v3',**neighbor})
    tr={'chi_critical_m_inv':root,'threshold_status':'root_found_in_valid_bracket',
        'critical_identity':'v2 hydrostatic root reused; both actual neighbors
        ↪  rechecked v3',
        'valid_complete_window_grid_points':int(reference_old.valid_complete_window_g
        ↪  rid_points),
        'incomplete_or_failed_grid_points':int(reference_old.incomplete_or_failed_gri
        ↪  d_points)}
    origin='v2_exact_root_reused_with_actual_v3_neighbor_regression'
else:
    tr,grid=threshold(case,forcing,cfg,num);origin='computed_v3_new_initial_state_bou
    ↪  ndary'
for row in grid:brackets.append({'case_id':cid,'q0':q0,'origin':origin,**row})
root=tr['chi_critical_m_inv'];valid=[g for g in grid if
↪  g.get('record_type')=='valid_root_bracket']
Delta=case.mcrit-case.m0
Gasy=case.a/(case.m0*Delta)*math.log((case.s_init+case.sigma0)/(case.qcrit*case.E+cas
↪  e.sigma0))
bound={'case_id':cid,'q0':q0,'s_init_Pa':case.s_init,'qcrit':case.qcrit,'qmax':admiss
↪  ion['qmax'],
    'm0':case.m0,'mcrit':case.mcrit,'H0_m':case.H0,'shape_n':case.shape_n,
    'initial_state_origin':en['initial_state_identity'],'criterion_origin':'closure_d
    ↪  erived',
    'forcing_origin':'post_event_inversion','forcing_window':'[27,100]','T_s':73,'I_U
    ↪  _m':IU,
    'Gamma_c_asy':Gasy,'chi_asy_m_inv':Gasy/IU,'chi_star_m_inv':root,
    'Gamma_star_window':None if root is None else root*IU,
    'chi_star_relative_error_vs_asy':None if root is None else
    ↪  (root-Gasy/IU)/(Gasy/IU),
```

```
        'chi_star_bracket':json.dumps([valid[0]['bracket_lo_m_inv'],valid[0]['bracket_hi_
        ↪ m_inv']]) if valid else
        ↪ None,
        'origin':origin,**tr}
    if root is not None:
        eps=max(cfg['threshold_search']['xtol_m_inv']*10,root*1e-5)
        c=replace(case,chi_m_inv=root+eps)
        f,state,sol=run_case(c,forcing,num,keep_solution=True)
        f.insert(0,'case_id',cid);f['regime']='own_critical_upper_neighbor';f['q0']=q0;tr
        ↪ ajectories.append(f)
        rec,ser=shear_diagnostics(c,forcing,sol,state,cid,'own_critical_upper_neighbor',e
        ↪ n['diagnostic_max_step_s'])
        rec['chi_star_m_inv']=root;rec['chi_eval_minus_star_m_inv']=eps
        diagnostics.append(rec);series.append(ser)
        bound.update({'chi_neighbor_eval_m_inv':root+eps,'neighbor_status':reached_state(
        ↪ state),
            'neighbor_completion_status':state['status'],'neighbor_hit_elapsed_s':state['
            ↪ onset_elapsed_s'],
            'Gamma_neighbor_at_first_hit':rec['Gamma_at_first_hit'],
            'eps_m_at_own_neighbor':rec['eps_m'],'eps_N_at_own_neighbor':rec['eps_N'],
            'drain_to_shear_at_own_neighbor':rec['absolute_drain_to_shear_ratio'],
            'L_goal':rec['L_goal'],'eps_J_at_own_neighbor':rec['eps_J']})
    bounds.append(bound)

    c=replace(case,mode='zero_shear',chi_m_inv=0)
    f,state,sol=run_case(c,forcing,num,keep_solution=True)
    f.insert(0,'case_id',cid);f['regime']='zero_shear_relaxation';f['q0']=q0;trajectories
    ↪ .append(f)
    rec,ser=shear_diagnostics(c,forcing,sol,state,cid,'zero_shear_relaxation',en['diagnos
    ↪ tic_max_step_s'])
    rec['approximation_status']='not_applicable_no_shear_relaxation_control'
    diagnostics.append(rec);series.append(ser)
    dm,ds,J=rhs(c,forcing,0,(c.m0,c.s_init,0))
    consistent=(abs(state['final_m']-case.m0)<1e-12 and
    ↪ abs(f.mean_effective_stress_Pa.iloc[-1]-case.s_init)<1e-8) if q0==.5 else
    ↪ bool((state['final_m']-case.m0)*dm>0 and
    ↪ (f.mean_effective_stress_Pa.iloc[-1]-case.s_init)*ds>0)
    relax.append({'case_id':cid,'q0':q0,'initial_m_dot_s_inv':dm,'initial_s_dot_Pa_s':ds,
    ↪ 'J0_m_s':J,
        'delta_m':state['final_m']-case.m0,'delta_s_Pa':float(f.mean_effective_stress_Pa.
        ↪ iloc[-1]-case.s_init),
        'direction_or_static_check_pass':consistent,
        'identity':'numerical zero-shear pressure-relaxation / budget check; not external
        ↪ validation',**state})
    print(cid,q0,'own boundary',tr['threshold_status'],root,flush=True)
if not all(r['pass'] for r in regressions):raise RuntimeError('Hydrostatic v2 regression
↪ failed')
if not all(r['direction_or_static_check_pass'] for r in relax):raise
↪ RuntimeError('Zero-shear pressure relaxation check failed')
qmax=initial_classification(ref)['qmax']
admissions=[initial_classification(replace(ref,initial_stress_ratio=q)) for q in
↪ [0,.1,ref.qcrit,.17,.30,.5,.75,qmax,qmax+1e-4,-.01]]
expected=['initial_closure_limit','initial_positive_thickness_layer','initial_first_layer
↪ _boundary',
```

```
        *(['admissible_unreached_initial_state']*4),'initial_saturation_boundary',
        'invalid_initial_state','invalid_initial_state']
    if [row['initial_status'] for row in admissions]!=expected:raise RuntimeError('Initial
    ↪  admission classification check failed')
    for row in admissions:
        row['root_search_in_this_classification_record']=False
        row['separate_dynamic_boundary_table']='initial_state_boundary.csv' if row['q0'] in
        ↪  en['q0_values'] else None
        row['Gamma_initial_condition']=0.0 if row['initial_condition_satisfied'] else None
    tables={'initial_state_response.csv':pd.DataFrame(summaries),'initial_state_boundary.csv'
    ↪  :pd.DataFrame(bounds),
        'initial_state_search_brackets.csv':pd.DataFrame(brackets),'initial_state_admission.c
        ↪  sv':pd.DataFrame(admissions),
        'initial_state_trajectories.csv':pd.concat(trajectories,ignore_index=True),
        'initial_state_cumulative_check.csv':pd.DataFrame(diagnostics),
        'initial_state_cumulative_series.csv':pd.concat(series,ignore_index=True),
        'initial_state_relaxation_checks.csv':pd.DataFrame(relax),'v2_hydrostatic_regression.
        ↪  csv':pd.DataFrame(regressions)}
    for name,frame in tables.items():frame.to_csv(out/name,index=False)
    save_json(out/'summary.json',{'version':cfg['version'],'main_response_settings':8,'initia
    ↪  l_state_boundaries':4,
        'new_root_searches':3,'v2_exact_roots_reused':1,'numerical_zero_shear_controls':4,
        'admission_classifications':len(admissions),'diagnostic_trajectories':len(diagnostics
        ↪  ),
        'diagnostic_saved_rows':sum(len(x) for x in series),'qmax':qmax,
        'historical_result_files_reused':reused,'actual_event_unknowns':cfg['actual_event_unk
        ↪  nowns'],
        'scientific_status':'conditional initial-state sensitivity; local initial state
        ↪  remains unobserved'})
    inputs={str(p.relative_to(delivery)):fingerprint(p) for p in
    ↪  sorted((delivery/'data').rglob('*')) if p.is_file()}
    for name in ('config.json',):inputs[name]=fingerprint(delivery/name)
    save_json(out/'ANALYSIS_RUN_RECEIPT.json',{'started_utc':started,'finished_utc':datetime.
    ↪  now(timezone.utc).isoformat(),
        'command':sys.argv,'code_sha256':fingerprint(__file__),'input_sha256':inputs,
        'environment':{'python':platform.python_version(),'numpy':np.__version__,'pandas':pd.
        ↪  __version__,'scipy':scipy.__version__},
        'result_sha256':{p.name:fingerprint(p) for p in sorted(out.iterdir()) if p.is_file()
        ↪  and p.name!='ANALYSIS_RUN_RECEIPT.json'},
        'execution':'completed','checkpoint_B':'pending','independent_review':'pending','loca
        ↪  l_validation':'unresolved'})
    print('Completed frozen v3 initial-state matrix, joint boundaries and saved-state
    ↪  diagnostics',flush=True)


if __name__=='__main__':sol()
```

问题三

q3/draw.py

```
from __future__ import annotations
import sys
sys.dont_write_bytecode = True
import argparse, datetime, hashlib, json, math, os, re
```

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.ticker import MaxNLocator
STYLE = {
    'font_family': 'sans-serif',
    'fonts': ['Arial', 'Noto Sans CJK SC', 'Microsoft YaHei', 'PingFang SC', 'DejaVu Sans'],
    'font_size': 8,
    'tick_size': 7,
    'title_size': 9,
    'legend_size': 7,
    'panel_label_size': 10,
    'status_size': 7,
    'supertitle_size': 9,
    'panel_label_offset_pt': (-22, 8),
    'panel_label_anchor': (0, 1),
    'panel_label_weight': 'bold',
    'title_pad_pt': 10,
    'panel_xycoords': 'axes fraction',
    'panel_textcoords': 'offset points',
    'panel_annotation_clip': False,
    'status_horizontal_alignment': 'center',
    'status_vertical_alignment': 'center',
    'pending_text_anchor': (0.5, 0.5),
    'q4_status_x_fraction': 0.5,
    'supertitle_y': 0.98,
    'sizes': {
        'grid': (7.2, 5.4),
        'row': (7.2, 3.2),
        'validation': (7.2, 3.4),
        'sensitivity_row': (7.6, 3.6),
        'sensitivity_grid': (7.6, 6.0),
    },
    'figures': {
        'response': {
            'shape': (2, 2),
            'size': 'grid',
            'legend_columns': 3,
            'layout': {'left': 0.1, 'right': 0.98, 'bottom': 0.1, 'top': 0.86, 'wspace':
            ↪  0.29, 'hspace': 0.45},
        },
        'learning': {
            'shape': (2, 2),
            'size': 'grid',
            'legend_columns': 4,
            'layout': {'left': 0.1, 'right': 0.98, 'bottom': 0.1, 'top': 0.86, 'wspace':
            ↪  0.29, 'hspace': 0.4},
        },
        'validation': {
            'shape': (1, 2),
```

```
        'size': 'validation',
        'legend_columns': None,
        'layout': {'left': 0.1, 'right': 0.98, 'bottom': 0.2, 'top': 0.78, 'wspace': 0.3},
    },
    'distribution': {
        'shape': (2, 2),
        'size': 'grid',
        'legend_columns': 4,
        'layout': {'left': 0.17, 'right': 0.98, 'bottom': 0.1, 'top': 0.85, 'wspace':
        ↪  0.48, 'hspace': 0.43},
    },
    'lead': {
        'shape': (1, 2),
        'size': 'row',
        'legend_max_columns': 3,
        'layout': {'left': 0.1, 'right': 0.98, 'bottom': 0.21, 'top': 0.76, 'wspace':
        ↪  0.3},
    },
    'sensitivity_phase': {
        'shape': (1, 2),
        'size': 'sensitivity_row',
        'legend_columns': 2,
        'layout': {'left': 0.09, 'right': 0.91, 'bottom': 0.2, 'top': 0.77, 'wspace':
        ↪  0.4},
    },
    'sensitivity_limits': {
        'shape': (2, 2),
        'size': 'sensitivity_grid',
        'legend_columns': 2,
        'layout': {'left': 0.12, 'right': 0.97, 'bottom': 0.1, 'top': 0.86, 'wspace':
        ↪  0.4, 'hspace': 0.55},
    },
    'q4': {
        'shape': (1, 2),
        'size': 'row',
        'layout': {'left': 0.13, 'right': 0.98, 'bottom': 0.2, 'top': 0.76, 'wspace':
        ↪  0.3},
    },
},
'colors': {
    'truth': '#252525',
    'baseline': '#B57A34',
    'passive': '#777777',
    'active': '#246A98',
    'failure': '#A94C47',
    'class_threshold': '#aaaaaa',
    'fallback_legend': '#333333',
    'bound_connector': '#999999',
},
'line_styles': {'truth': '-', 'baseline': ':', 'passive': '--', 'active': '-'},
'mode_styles': {'raw': {'line': '-', 'marker': 'o'}, 'fallback': {'line': '--', 'marker':
↪  's'}},
'query_markers': {'passive': 'o', 'active': 'x'},
'lead_markers': ['o', 's', '^', 'D', 'v'],
```

```
'lead_line_styles': ['-', '--', ':', '-.'],
'lead_color_order': ['truth', 'active', 'baseline', 'passive', 'failure'],
'line_width': 1.2,
'marker_size': 3,
'line_widths': {
    'score': 1.1,
    'class_threshold': 0.7,
    'baseline_reference': 1,
    'validation_target': 1,
    'bound_connector': 0.8,
    'distribution_truth': 1.3,
    'distribution_baseline': 1.4,
    'distribution_gp': 1,
    'interval': 2,
    'lead': 1,
    'local_zero': 1,
    'q4_zero': 0.9,
},
'mark_sizes': {'validation': 4, 'interval_endpoint': 7, 'lead': 3},
'scatter': {
    'query_area': 15,
    'query_edge_width': 0.8,
    'query_zorder': 3,
    'class_display_jitter': 0.025,
    'nohit_area': 30,
},
'interval_marker': '|',
'nohit_marker': 's',
'errorbar_capsize': 2,
'hollow_marker_facecolor': 'none',
'legend_marker_line': '',
'default_rc': {
    'pdf.fonttype': 42,
    'svg.fonttype': 'none',
    'axes.unicode_minus': True,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'text.usetex': False,
    'lines.linewidth': 1.5,
    'lines.markersize': 6,
},
'views': {
    'offset_xlim': (-1.55, 1.55),
    'class_ylim': (-0.13, 1.13),
    'class_yticks': [0, 0.5, 1],
    'miss_bottom': -0.06,
    'miss_minimum_top': 1,
    'miss_headroom': 0.15,
    'validation_xlim': (-0.25, 2.25),
    'validation_lower_floor': 0.9,
    'validation_lower_padding': 0.01,
    'validation_upper': 1.005,
    'cdf_xlim': {'strong': (0.765, 0.825), 'near': (71.3, 73)},
    'cdf_ylim': (0, 1.02),
```

```
    },
    'sensitivity': {
        'phase_cmap': 'cividis',
        'no_hit_color': '#D9D9D9',
        'phase_shading': 'flat',
        'phase_rasterized': False,
        'colorbar_fraction': 0.05,
        'colorbar_pad': 0.025,
        'nominal_marker': 'o',
        'nominal_area': 35,
        'nominal_face': 'white',
        'nominal_edge_width': 0.8,
        'nominal_zorder': 4,
        'unknown_marker': 'x',
        'initial_marker': '^',
        'category_area': 15,
        'category_edge_width': 0.8,
        'bar_width': 0.55,
        'amplification_area': 35,
        'amplification_marker': 'o',
        'amplification_bottom_factor': 1.6,
        'amplification_top_factor': 1.8,
        'amplification_ratio_anchor': (0.06, 0.92),
        'amplification_ratio_ha': 'left',
        'regime_tick_rotation': 0,
        'chi_xlim_padding': 0.01,
        'log_tick_size': 8,
        'nohit_strip_y': 1.02,
        'nohit_strip_area': 23,
        'chi_legend_loc': 'upper right',
        'chi_legend_anchor': (1, 0.96),
        'condition_markers': {'strong': 'o', 'near': 's'},
        'condition_lines': {'strong': '--', 'near': '-'},
        'width_legend_loc': 'upper left',
    },
    'grid': False,
    'dpi': 300,
    'formats': ['.png', '.pdf', '.svg'],
    'alignment_tolerance_pt': 1.5,
    'legend_anchor': (0.54, 0.99),
    'legend': {'frameon': False, 'loc': 'upper center'},
}
REGIMES = ['strong', 'near']
NAMES = {'strong': 'Strong shear', 'near': 'Near boundary'}
BASE_KEYS = {'strong': 'Q2_strong_design', 'near': 'Q2_near_boundary_comparison'}
FAMILIES = ['nominal_triangular', 'uniform_support', 'lower_mode_triangular']
FAMILY_LABELS = ['Nominal', 'Uniform', 'Lower mode']
REQUIRED = ['summary.json', 'baseline_results.json', 'gp_metrics.csv',
↪  'gp_predictions.csv.gz', 'baseline_test_metrics.json', 'validation.json',
↪  'lead_margins.csv', 'gp_training.json', 'test_truth.csv', 'cost_comparison.csv']

def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()
```

```
def save(p, obj):
    Path(p).parent.mkdir(parents=True, exist_ok=True)
    Path(p).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False) + '\n')

def load_json(p):
    try:
        return json.loads(Path(p).read_text())
    except (OSError, ValueError) as e:
        raise ValueError(f'Cannot read JSON input {Path(p).name}: {e}') from e

def need_columns(frame, fields, filename):
    missing = sorted(set(fields) - set(frame.columns))
    if missing:
        raise ValueError(f'{filename}: missing required columns {missing}')

def apply_style():
    plt.rcParams.update({
        'font.family': STYLE['font_family'],
        'font.sans-serif': STYLE['fonts'],
        'font.size': STYLE['font_size'],
        'axes.labelsize': STYLE['font_size'],
        'axes.titlesize': STYLE['title_size'],
        'xtick.labelsize': STYLE['tick_size'],
        'ytick.labelsize': STYLE['tick_size'],
        'legend.fontsize': STYLE['legend_size'],
        **STYLE['default_rc'],
        'axes.grid': STYLE['grid'],
        'savefig.dpi': STYLE['dpi'],
    })

def panel(ax, label, title=None):
    ax.annotate(label, STYLE['panel_label_anchor'], xycoords=STYLE['panel_xycoords'],
    ↪  xytext=STYLE['panel_label_offset_pt'], textcoords=STYLE['panel_textcoords'],
    ↪  weight=STYLE['panel_label_weight'], fontsize=STYLE['panel_label_size'],
    ↪  annotation_clip=STYLE['panel_annotation_clip'])
    if title:
        ax.set_title(title, pad=STYLE['title_pad_pt'])

def new_figure(name):
    config = STYLE['figures'][name]
    fig, axes = plt.subplots(*config['shape'], figsize=STYLE['sizes'][config['size']])
    fig.subplots_adjust(**config['layout'])
    return (fig, axes)

def require_matplotlib_panel_alignment(fig, path, tolerance_pt=1.5):
    fig.canvas.draw()
    w, h = fig.get_size_inches() * 72
    rects = []
    for i, ax in enumerate(fig.axes):
        if getattr(ax, '_q3_auxiliary', False):
            continue
        p = ax.get_position()
        rects.append(dict(id=str(i), left=p.x0 * w, bottom=p.y0 * h, right=p.x1 * w, top=p.y1
        ↪  * h, width=p.width * w, height=p.height * h))
```

```
    checks = []
    for i, a in enumerate(rects):
        for b in rects[i + 1:]:
            same_row = abs(a['bottom'] - b['bottom']) < tolerance_pt and abs(a['top'] -
            ↪  b['top']) < tolerance_pt
            same_col = abs(a['left'] - b['left']) < tolerance_pt and abs(a['right'] -
            ↪  b['right']) < tolerance_pt
            if same_row:
                checks.append(abs(a['height'] - b['height']) <= tolerance_pt and
                ↪  abs(a['width'] - b['width']) <= tolerance_pt)
            if same_col:
                checks.append(abs(a['width'] - b['width']) <= tolerance_pt)
            overlap_x = min(a['right'], b['right']) - max(a['left'], b['left'])
            overlap_y = min(a['top'], b['top']) - max(a['bottom'], b['bottom'])
            checks.append(not (overlap_x > tolerance_pt and overlap_y > tolerance_pt))
    answer = dict(status='pass' if all(checks) else 'fail', tolerance_pt=tolerance_pt,
    ↪  scope='rectangular comparable plot-area edges/size/nonoverlap only',
    ↪  width_pt=float(w), height_pt=float(h), panels=rects, checks=len(checks), exemptions=[{
        'axis': i,
        'reason': 'auxiliary colorbar encodes the same field; excluded from comparable
        ↪  scientific-panel width/edge groups',
    } for i, ax in enumerate(fig.axes) if getattr(ax, '_q3_auxiliary', False)])
    save(path, answer)
    if not all(checks):
        raise ValueError('Final subplot plot-area alignment failed')

def export(fig, name, out, audit):
    require_matplotlib_panel_alignment(fig, audit / f'{name}.alignment.json',
    ↪  STYLE['alignment_tolerance_pt'])
    for ext in STYLE['formats']:
        fig.savefig(out / f'{name}{ext}', dpi=STYLE['dpi'])
    plt.close(fig)

def read_inputs(results, sensitivity_results=None):
    for f in REQUIRED:
        if not (results / f).is_file():
            raise ValueError(f'Missing required saved analysis result: {f}')
    d = {k: load_json(results / f'{k}.json') for k in ['summary', 'baseline_results',
    ↪  'baseline_test_metrics', 'validation', 'gp_training']}
    for key, file in [('metrics', 'gp_metrics.csv'), ('pred', 'gp_predictions.csv.gz'),
    ↪  ('lead', 'lead_margins.csv'), ('truth', 'test_truth.csv'), ('cost',
    ↪  'cost_comparison.csv')]:
        try:
            d[key] = pd.read_csv(results / file)
        except (OSError, ValueError) as e:
            raise ValueError(f'Cannot read {file}: {e}') from e
    need_columns(d['metrics'], ['regime', 'method', 'budget', 'mode', 'subset', 'mae_s',
    ↪  'false_negative', 'unknown_predictions', 'total_nominal_q2_calls'], 'gp_metrics.csv')
    need_columns(d['pred'], ['regime', 'method', 'budget', 'mode', 'd', 'status', 'tau_s',
    ↪  'p_hit', 'true_status', 'true_tau_s'], 'gp_predictions.csv.gz')
    need_columns(d['truth'], ['regime', 'd', 'status', 'tau_s'], 'test_truth.csv')
    if not {'edges_s', 'conditions'} <= set(d['baseline_results']):
        raise ValueError('baseline_results.json: edges_s and conditions required')
    edges = np.asarray(d['baseline_results']['edges_s'], float)
```

```
if not np.all(np.isfinite(edges)) or not np.all(np.diff(edges) > 0):
    raise ValueError('baseline time edges must be finite and strictly increasing')
for key in ['truth', 'pred']:
    frame = d[key]
    if not frame.status.isin(['hit', 'no_hit', 'initial', 'unknown']).all():
        raise ValueError(f'{key}: invalid outcome status')
    if frame.loc[frame.status != 'hit', 'tau_s'].notna().any():
        raise ValueError(f'{key}: non-hit outcome cannot carry future first-passage time')
    times = frame.loc[frame.status == 'hit', 'tau_s']
    if times.isna().any() or ((times <= 0) | (times > 73)).any():
        raise ValueError(f'{key}: invalid hit time')
d['lead'] = normalize_lead(d['lead'])
d['validation_rows'] = normalize_validation(d['validation'])
for regime in REGIMES:
    sp = spec_of(base_condition(d, regime), 'robust_shortest')
    rows = d['lead'][d['lead'].regime == regime]
    if not len(rows):
        raise ValueError(f'lead_margins.csv: missing {regime} delay scenarios')
    interval = sp.get('finite_interval_s')
    if interval is not None:
        shift = rows.publication_delay_s + rows.action_delay_s
        if not np.allclose(rows.finite_lower_s + shift, interval[0], atol=1e-09, rtol=0)
        ↪  or not np.allclose(rows.finite_upper_s + shift, interval[1], atol=1e-09,
        ↪  rtol=0):
            raise ValueError(f'lead_margins.csv: {regime} margins are not the formal-set
            ↪  translation')
    if not (rows.includes_no_hit.astype(bool) ==
    ↪  bool(sp.get('includes_no_hit_by_horizon'))).all():
        raise ValueError(f'lead_margins.csv: {regime} no-hit category changed during
        ↪  translation')
cost_alias = {
    'construction_wall_s': 'build_s',
    'query_wall_s': 'query_s',
    'probability_set_wall_s': 'probability_s',
    'first_use_total_wall_s': 'first_use_total_s',
    'repeat_query_total_wall_s': 'repeat_use_total_s',
}
d['cost'] = d['cost'].rename(columns={a: b for a, b in cost_alias.items() if a in
↪  d['cost'] and b not in d['cost']})
cost = d['cost']
cost_fields = ['build_s', 'query_s', 'probability_s', 'first_use_total_s',
↪  'repeat_use_total_s']
need_columns(cost, ['regime', 'method', 'budget', 'mode', 'construction_q2_calls',
↪  'fallback_q2_calls'] + cost_fields, 'cost_comparison.csv')
if not np.isfinite(cost[cost_fields].to_numpy(float)).all() or (cost[cost_fields] <
↪  0).any().any():
    raise ValueError('cost_comparison.csv: required workload costs must be finite and
    ↪  nonnegative')
if not np.allclose(cost.first_use_total_s, cost.build_s + cost.query_s +
↪  cost.probability_s, atol=1e-08, rtol=1e-09):
    raise ValueError('cost_comparison.csv: first-use total does not equal construction
    ↪  plus same queries plus probability task')
if not np.allclose(cost.repeat_use_total_s, cost.query_s + cost.probability_s,
↪  atol=1e-08, rtol=1e-09):
```

```
        raise ValueError('cost_comparison.csv: repeated workload total is inconsistent')
    if (results / 'q4_spatial_demo.csv').is_file():
        d['q4_demo'] = normalize_q4(pd.read_csv(results / 'q4_spatial_demo.csv'))
        for regime in REGIMES:
            rows = d['q4_demo'][(d['q4_demo'].regime == regime) & (d['q4_demo'].status ==
            ↪  'arrived')]
            sp = spec_of(base_condition(d, regime), 'robust_shortest')
            interval = sp['finite_interval_s']
            if not np.allclose(rows.finite_arrival_lower_s - rows.travel_time_s, interval[0],
            ↪  atol=1e-08, rtol=0) or not np.allclose(rows.finite_arrival_upper_s -
            ↪  rows.travel_time_s, interval[1], atol=1e-08, rtol=0):
                raise ValueError('q4_spatial_demo.csv: assumed zero bridge must translate the
                ↪  formal local candidate exactly')
            if not (rows.includes_no_hit.astype(bool) ==
            ↪  bool(sp.get('includes_no_hit_by_horizon'))).all():
                raise ValueError('q4_spatial_demo.csv: local no-hit category was lost')
    else:
        d['q4_demo'] = None
    d['sensitivity'] = read_sensitivity(results / 'sensitivity' if sensitivity_results is
    ↪  None else sensitivity_results)
    return d

def normalize_lead(frame):
    alias = {
        'd_pub_s': 'publication_delay_s',
        'd_act_s': 'action_delay_s',
        'lower_lead_s': 'finite_lower_s',
        'upper_lead_s': 'finite_upper_s',
        'lead_lower_s': 'finite_lower_s',
        'lead_upper_s': 'finite_upper_s',
        'includes_no_hit_by_horizon': 'includes_no_hit',
    }
    frame = frame.rename(columns={a: b for a, b in alias.items() if a in frame and b not in
    ↪  frame}).copy()
    need_columns(frame, ['regime', 'publication_delay_s', 'action_delay_s', 'finite_lower_s',
    ↪  'finite_upper_s', 'includes_no_hit'], 'lead_margins.csv')
    if frame[['publication_delay_s', 'action_delay_s']].isna().any().any():
        raise ValueError('lead_margins.csv: missing delays')
    if (frame[['publication_delay_s', 'action_delay_s']] < 0).any().any():
        raise ValueError('lead_margins.csv: negative delays')
    if (frame.finite_lower_s > frame.finite_upper_s).any():
        raise ValueError('lead_margins.csv: inverted finite margin interval')
    return frame

def normalize_q4(frame):
    alias = {
        'target_arrival_status': 'status',
        'arrival_lower_s': 'finite_arrival_lower_s',
        'arrival_upper_s': 'finite_arrival_upper_s',
        'lead_lower_s': 'finite_lead_lower_s',
        'lead_upper_s': 'finite_lead_upper_s',
        'd_pub_s': 'publication_delay_s',
        'd_act_s': 'action_delay_s',
        'lower_lead_s': 'finite_lead_lower_s',
```

```
        'upper_lead_s': 'finite_lead_upper_s',
        'effective_lead_lower_s': 'finite_lead_lower_s',
        'effective_lead_upper_s': 'finite_lead_upper_s',
    }
    frame = frame.rename(columns={a: b for a, b in alias.items() if a in frame and b not in
    ↪  frame}).copy()
    need_columns(frame, ['regime', 'scenario_id', 'status', 'publication_delay_s',
    ↪  'action_delay_s', 'finite_lead_lower_s', 'finite_lead_upper_s', 'travel_time_s',
    ↪  'horizon_s', 'position_m', 'finite_arrival_lower_s', 'finite_arrival_upper_s',
    ↪  'terrain_identity', 'stage_bridge', 'includes_no_hit', 'natural_warning_radius_m'],
    ↪  'q4_spatial_demo.csv')
    known = ['arrived', 'valid_no_arrival_by_horizon', 'no_arrival_by_horizon', 'unknown',
    ↪  'failed']
    if not frame.status.isin(known).all():
        raise ValueError('q4_spatial_demo.csv: unsupported propagation outcome')
    if (frame.horizon_s <= 0).any() or (frame.position_m <= 0).any():
        raise ValueError('q4_spatial_demo.csv: positive horizon and target distance required')
    if not np.allclose(frame.position_m, 1000, rtol=0, atol=1e-09) or not
    ↪  np.allclose(frame.horizon_s, 180, rtol=0, atol=1e-09):
        raise ValueError('q4_spatial_demo.csv: this fixed synthetic figure requires the
        ↪  declared 1000m target and 180s propagation horizon')
    if frame.natural_warning_radius_m.notna().any():
        raise ValueError('q4_spatial_demo.csv: synthetic target must not carry a natural
        ↪  warning radius')
    if frame[['terrain_identity', 'stage_bridge']].isna().any().any():
        raise ValueError('q4_spatial_demo.csv: explicit terrain and bridge identities
        ↪  required')
    if frame.loc[frame.status != 'arrived', ['finite_lead_lower_s',
    ↪  'finite_lead_upper_s']].notna().any().any():
        raise ValueError('q4_spatial_demo.csv: no-arrival/unknown must not carry a finite
        ↪  target lead')
    if frame.loc[frame.status != 'arrived', ['travel_time_s', 'finite_arrival_lower_s',
    ↪  'finite_arrival_upper_s']].notna().any().any():
        raise ValueError('q4_spatial_demo.csv: nonarrival must retain empty travel/arrival
        ↪  times')
    arrived = frame[frame.status == 'arrived']
    if ((arrived.travel_time_s <= 0) | (arrived.travel_time_s > arrived.horizon_s)).any():
        raise ValueError('q4_spatial_demo.csv: arrived travel time outside declared horizon')
    shift = arrived.publication_delay_s + arrived.action_delay_s
    if not np.allclose(arrived.finite_arrival_lower_s - shift, arrived.finite_lead_lower_s,
    ↪  atol=1e-08, rtol=0) or not np.allclose(arrived.finite_arrival_upper_s - shift,
    ↪  arrived.finite_lead_upper_s, atol=1e-08, rtol=0):
        raise ValueError('q4_spatial_demo.csv: target lead does not equal same-scenario
        ↪  arrival minus delays')
    if arrived[['finite_lead_lower_s', 'finite_lead_upper_s']].isna().any().any() or
    ↪  (arrived.finite_lead_lower_s > arrived.finite_lead_upper_s).any():
        raise ValueError('q4_spatial_demo.csv: invalid arrived finite interval')
    return frame

def normalize_validation(value):
    rows = value.get('claims', value.get('results', value.get('families', [])))
    if isinstance(rows, dict):
        flat = []
        for regime, v in rows.items():
```

```
            seq = v.get('families', v.get('claims', [])) if isinstance(v, dict) else v
            for row in seq:
                flat.append(dict(regime=regime, **row))
        rows = flat
    if not isinstance(rows, list):
        raise ValueError('validation.json: claims/results must contain rows')
    frame = pd.DataFrame(rows)
    if not len(frame):
        return frame
    aliases = {
        'condition': 'regime',
        'n_samples': 'n',
        'included': 'k',
        'empirical_coverage': 'empirical_mass',
        'one_sided_lower_mass': 'lower_bound',
        'one_sided_lower_bound': 'lower_bound',
        'one_sided_lower': 'lower_bound',
        'certified': 'passed',
        'all_target_pass': 'passed',
    }
    frame = frame.rename(columns={a: b for a, b in aliases.items() if a in frame and b not in
    ↪  frame})
    need_columns(frame, ['regime', 'family', 'n', 'k', 'empirical_mass', 'lower_bound',
    ↪  'passed'], 'validation.json claims')
    if not all((isinstance(x, (bool, np.bool_)) for x in frame.passed)):
        raise ValueError('validation.json: passed must contain Boolean values, not strings')
    if not np.isfinite(frame[['n', 'k', 'empirical_mass',
    ↪  'lower_bound']].to_numpy(float)).all():
        raise ValueError('validation.json: finite counts, proportions and bounds required')
    if ((frame.k < 0) | (frame.n <= 0) | (frame.k > frame.n)).any() or not
    ↪  np.equal(frame[['n', 'k']], np.floor(frame[['n', 'k']])).all().all():
        raise ValueError('validation.json: invalid integer binomial counts')
    target = 1 - float(value.get('alpha', 0.05))
    if not (frame.passed == (frame.lower_bound >= target - 1e-12)).all():
        raise ValueError('validation.json: pass status contradicts the exact lower bound')
    if not np.allclose(frame.k / frame.n, frame.empirical_mass, rtol=0, atol=1e-10):
        raise ValueError('validation empirical proportions do not match k/n')
    if ((frame.lower_bound < 0) | (frame.lower_bound > frame.empirical_mass + 1e-12)).any():
        raise ValueError('validation bounds outside valid range')
    return frame

def training_rows(data, regime, method):
    v = data['gp_training']
    if regime in v and method in v[regime]:
        return pd.DataFrame(v[regime][method])
    if f'{regime}_{method}' in v:
        return pd.DataFrame(v[f'{regime}_{method}'])
    if isinstance(v.get('rows'), list):
        f = pd.DataFrame(v['rows'])
        return f[(f.regime == regime) & (f.method == method)]
    raise ValueError(f'gp_training.json: missing {regime}/{method} training rows')

def base_condition(data, regime):
    conditions = data['baseline_results']['conditions']
```

```
    return conditions[BASE_KEYS[regime]] if BASE_KEYS[regime] in conditions else
    ↪  conditions[regime]

def family_cdf(x, family):
    u = np.clip((np.asarray(x) + 1.5) / 3, 0, 1)
    if family == 'uniform_support':
        return u
    if family == 'lower_mode_triangular':
        return 2 * u - u * u
    if family == 'nominal_triangular':
        return np.where(u <= 0.5, 2 * u * u, 1 - 2 * (1 - u) ** 2)
    raise ValueError(f'Unsupported declared family {family}')

def source_cdf(frame, edges, family='nominal_triangular'):
    f = frame.sort_values('d')
    x = f.d.to_numpy()
    expected = -1.5 + (np.arange(len(f)) + 0.5) * 3 / len(f)
    if not np.allclose(x, expected, rtol=0, atol=1e-09):
        raise ValueError('CDF diagnostic requires the saved equal-cell midpoint input grid')
    weights = np.diff(family_cdf(np.linspace(-1.5, 1.5, len(f) + 1), family))
    hit = f.status.to_numpy() == 'hit'
    t = f.tau_s.to_numpy()
    c = np.array([weights[hit & (t <= e)].sum() for e in edges])
    return (c, float(weights[f.status.to_numpy() == 'no_hit'].sum()),
    ↪  float(weights[f.status.to_numpy() == 'unknown'].sum()))

def spec_of(c, name):
    s = c[name]
    return s.get('set', s) if isinstance(s, dict) else s

def fmt(x, digits=6):
    if x is None or (isinstance(x, (float, np.floating)) and (not np.isfinite(x))):
        return '—'
    if isinstance(x, (bool, np.bool_)):
        return '是' if x else '否'
    if digits == 0:
        return f'{float(x):.0f}'
    if abs(float(x)) < 0.001 and float(x) != 0:
        return f'{float(x):.3g}'
    return f'{float(x):.{digits}f}'.rstrip('0').rstrip('.')

def interval_text(spec, digits=3):
    if spec is None:
        return '无可行集合'
    interval = spec.get('finite_interval_s')
    text = '∅' if interval is None else f'[{fmt(interval[0], digits)}, {fmt(interval[1],
    ↪  digits)}]'
    if spec.get('includes_no_hit_by_horizon', False):
        text += '∪{⊥}'
    return text

def response_figure(data, out, audit):
    fig, axes = new_figure('response')
    C = STYLE['colors']
```

```
    for col, regime in enumerate(REGIMES):
        truth = data['truth'][data['truth'].regime == regime].sort_values('d')
        h = truth.status == 'hit'
        ax = axes[0, col]
        ax.plot(truth.d[h], truth.tau_s[h], color=C['truth'], lw=STYLE['line_width'])
        for method, mark in STYLE['query_markers'].items():
            tr = training_rows(data, regime, method)
            h = tr.status == 'hit'
            ax.scatter(tr.d[h], tr.tau_s[h], s=STYLE['scatter']['query_area'],
            ↪  color=C[method], marker=mark,
            ↪  linewidths=STYLE['scatter']['query_edge_width'],
            ↪  zorder=STYLE['scatter']['query_zorder'])
            known = tr.status.isin(['hit', 'no_hit'])
            offset = STYLE['scatter']['class_display_jitter'] if method == 'active' else
            ↪  -STYLE['scatter']['class_display_jitter']
            axes[1, col].scatter(tr.d[known], (tr.status[known] == 'hit').astype(float) +
            ↪  offset, s=STYLE['scatter']['query_area'], color=C[method], marker=mark,
            ↪  linewidths=STYLE['scatter']['query_edge_width'])
        ax.set(xlabel='Coherent velocity offset (m/s)', ylabel='Elapsed first-hit time (s)',
        ↪  xlim=STYLE['views']['offset_xlim'])
        panel(ax, chr(97 + col), NAMES[regime])
        f = data['pred']
        p = f[(f.regime == regime) & (f.method == 'active') & (f.budget == f.budget.max()) &
        ↪  (f['mode'] == 'raw')].sort_values('d')
        axes[1, col].plot(p.d, p.p_hit, color=C['active'], lw=STYLE['line_widths']['score'])
        axes[1, col].axhline(0.5, color=C['class_threshold'],
        ↪  ls=STYLE['line_styles']['baseline'], lw=STYLE['line_widths']['class_threshold'])
        axes[1, col].set(xlabel='Coherent velocity offset (m/s)', ylabel='Hit class / GP
        ↪  score', xlim=STYLE['views']['offset_xlim'], ylim=STYLE['views']['class_ylim'],
        ↪  yticks=STYLE['views']['class_yticks'])
        panel(axes[1, col], chr(99 + col))
    fig.legend(handles=[Line2D([], [], color=C['truth'], label='Direct Q2'), Line2D([], [],
    ↪  color=C['passive'], marker=STYLE['query_markers']['passive'],
    ↪  ls=STYLE['legend_marker_line'], label='Passive queries'), Line2D([], [],
    ↪  color=C['active'], marker=STYLE['query_markers']['active'],
    ↪  ls=STYLE['legend_marker_line'], label='Active queries')],
    ↪  ncol=STYLE['figures']['response']['legend_columns'],
    ↪  bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
    export(fig, 'fig01_mapping_queries', out, audit)

def learning_figure(data, out, audit):
    fig, axes = new_figure('learning')
    C = STYLE['colors']
    m = data['metrics']
    for col, regime in enumerate(REGIMES):
        for method in ['passive', 'active']:
            for mode, appearance in STYLE['mode_styles'].items():
                ls = appearance['line']
                mark = appearance['marker']
                f = m[(m.regime == regime) & (m.method == method) & (m['mode'] == mode) &
                ↪  (m.subset == 'all')].sort_values('budget')
                axes[0, col].plot(f.budget, f.mae_s, color=C[method], ls=ls, marker=mark,
                ↪  ms=STYLE['marker_size'])
                axes[1, col].plot(f.total_nominal_q2_calls, f.false_negative,
                ↪  color=C[method], ls=ls, marker=mark, ms=STYLE['marker_size'])
```

```
        b = data['baseline_test_metrics'].get(regime, {})
        if b.get('separated_002', {}).get('mae_s') is not None:
            axes[0, col].axhline(b['separated_002']['mae_s'], color=C['baseline'],
            ↪  ls=STYLE['line_styles']['baseline'],
            ↪  lw=STYLE['line_widths']['baseline_reference'])
        axes[0, col].set(xlabel='Training Q2 calls', ylabel='Elapsed-time MAE (s)',
        ↪  xticks=sorted(m.budget.unique()))
        axes[1, col].set(xlabel='Training + fallback Q2 calls', ylabel='Missed true hits
        ↪  (count)', ylim=(STYLE['views']['miss_bottom'],
        ↪  max(STYLE['views']['miss_minimum_top'], m[(m.regime == regime) & (m.subset ==
        ↪  'all')].false_negative.max() + STYLE['views']['miss_headroom'])))
        axes[1, col].yaxis.set_major_locator(MaxNLocator(integer=True))
        for row in [0, 1]:
            panel(axes[row, col], chr(97 + row * 2 + col), NAMES[regime] if row == 0 else
            ↪  None)
    fig.legend(handles=[Line2D([], [], color=C['passive'], label='Passive GP'), Line2D([],
    ↪  [], color=C['active'], label='Active GP'), Line2D([], [], color=C['fallback_legend'],
    ↪  ls=STYLE['mode_styles']['fallback']['line'], label='With fallback'), Line2D([], [],
    ↪  color=C['baseline'], ls=STYLE['line_styles']['baseline'], label='Separated baseline
    ↪  MAE')], ncol=STYLE['figures']['learning']['legend_columns'],
    ↪  bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
    export(fig, 'fig02_gp_accuracy_cost', out, audit)

def validation_figure(data, out, audit):
    fig, axes = new_figure('validation')
    v = data['validation_rows']
    C = STYLE['colors']
    for col, regime in enumerate(REGIMES):
        ax = axes[col]
        ax.axhline(0.95, color=C['failure'], lw=STYLE['line_widths']['validation_target'],
        ↪  ls=STYLE['mode_styles']['fallback']['line'])
        f = v[v.regime == regime].copy() if len(v) else v
        if not len(f):
            ax.text(*STYLE['pending_text_anchor'], 'Random validation pending',
            ↪  ha=STYLE['status_horizontal_alignment'],
            ↪  va=STYLE['status_vertical_alignment'], transform=ax.transAxes)
        else:
            f = f.set_index('family').reindex(FAMILIES)
            if f.empirical_mass.isna().any():
                raise ValueError(f'validation.json: three declared families required for
                ↪  {regime}')
            x = np.arange(3)
            ax.plot(x, f.empirical_mass, color=C['truth'],
            ↪  marker=STYLE['mode_styles']['raw']['marker'],
            ↪  ls=STYLE['mode_styles']['raw']['line'], ms=STYLE['mark_sizes']['validation'])
            ax.plot(x, f.lower_bound, color=C['baseline'],
            ↪  marker=STYLE['mode_styles']['fallback']['marker'],
            ↪  ls=STYLE['mode_styles']['fallback']['line'],
            ↪  ms=STYLE['mark_sizes']['validation'])
            for j, (_, row) in enumerate(f.iterrows()):
                ax.plot([j, j], [row.lower_bound, row.empirical_mass],
                ↪  color=C['bound_connector'], lw=STYLE['line_widths']['bound_connector'])
```

```
        ax.set(xticks=range(3), xticklabels=FAMILY_LABELS,
        ↪  xlim=STYLE['views']['validation_xlim'],
        ↪  ylim=(min(STYLE['views']['validation_lower_floor'], float(f.lower_bound.min()) -
        ↪  STYLE['views']['validation_lower_padding']) if len(f) else
        ↪  STYLE['views']['validation_lower_floor'], STYLE['views']['validation_upper']),
        ↪  ylabel='Frozen-set probability mass')
        panel(ax, chr(97 + col), NAMES[regime])
    handles = [Line2D([], [], color=C['failure'],
    ↪  ls=STYLE['mode_styles']['fallback']['line'], label='Target 0.95')]
    if len(v):
        handles = [Line2D([], [], color=C['truth'],
        ↪  marker=STYLE['mode_styles']['raw']['marker'], label='New random sample mass'),
        ↪  Line2D([], [], color=C['baseline'],
        ↪  marker=STYLE['mode_styles']['fallback']['marker'],
        ↪  ls=STYLE['mode_styles']['fallback']['line'], label='Simultaneous exact lower
        ↪  bound')] + handles
    fig.legend(handles=handles, ncol=STYLE['figures']['validation']['legend_columns'] or
    ↪  len(handles), bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
    export(fig, 'fig03_frozen_validation', out, audit)

def distribution_figure(data, out, audit):
    fig, axes = new_figure('distribution')
    C = STYLE['colors']
    edges = np.asarray(data['baseline_results']['edges_s'])
    for col, regime in enumerate(REGIMES):
        ax = axes[0, col]
        truth = data['truth'][data['truth'].regime == regime]
        c, _, _ = source_cdf(truth, edges)
        ax.plot(edges, c, color=C['truth'], lw=STYLE['line_widths']['distribution_truth'])
        bc = base_condition(data, regime)
        pr = bc['propagation'][0]
        cdfv = np.asarray(pr.get('cdf', np.r_[0, np.cumsum(pr['finite_mass'])]))
        ax.plot(edges, cdfv, color=C['baseline'], ls=STYLE['line_styles']['baseline'],
        ↪  lw=STYLE['line_widths']['distribution_baseline'])
        p = data['pred']
        maxbudget = p.budget.max()
        for method in ['passive', 'active']:
            f = p[(p.regime == regime) & (p.method == method) & (p.budget == maxbudget) &
            ↪  (p['mode'] == 'raw')]
            c, _, _ = source_cdf(f, edges)
            ax.plot(edges, c, color=C[method], ls=STYLE['line_styles'][method],
            ↪  lw=STYLE['line_widths']['distribution_gp'])
        ax.set(xlim=STYLE['views']['cdf_xlim'][regime], ylim=STYLE['views']['cdf_ylim'],
        ↪  xlabel='Elapsed first-hit time (s)', ylabel='Unconditional finite CDF')
        panel(ax, chr(97 + col), NAMES[regime])
        ax = axes[1, col]
        for i, (key, label) in enumerate([('nominal_central', 'Nominal central'),
        ↪  ('nominal_shortest', 'Nominal shortest'), ('robust_shortest', 'Robust
        ↪  shortest')]):
            s = spec_of(bc, key)
            finite = s.get('finite_interval_s') if s else None
            if finite is not None:
                ax.plot(finite, [i, i], color=C['baseline'],
                ↪  lw=STYLE['line_widths']['interval'], marker=STYLE['interval_marker'],
                ↪  ms=STYLE['mark_sizes']['interval_endpoint'])
```

```
                if s.get('includes_no_hit_by_horizon'):
                    ax.scatter(finite[1], i, s=STYLE['scatter']['nohit_area'],
                    ↪  marker=STYLE['nohit_marker'],
                    ↪  facecolors=STYLE['hollow_marker_facecolor'], edgecolors=C['baseline'])
            ax.set(yticks=[0, 1, 2], yticklabels=['Nominal central', 'Nominal shortest', 'Robust
            ↪  shortest'], xlabel='Elapsed interval endpoints (s)')
            ax.invert_yaxis()
            panel(ax, chr(99 + col))
        fig.legend(handles=[Line2D([], [], color=C['truth'], label='Q2 midpoint diagnostic'),
        ↪  Line2D([], [], color=C['baseline'], ls=STYLE['line_styles']['baseline'],
        ↪  label='Integrated 1D mapping'), Line2D([], [], color=C['passive'],
        ↪  ls=STYLE['line_styles']['passive'], label='Passive GP raw'), Line2D([], [],
        ↪  color=C['active'], label='Active GP raw')],
        ↪  ncol=STYLE['figures']['distribution']['legend_columns'],
        ↪  bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
        export(fig, 'fig04_probability_sets', out, audit)

    def lead_figure(data, out, audit):
        lead = data['lead']
        actions = sorted(lead.action_delay_s.unique())
        C = STYLE['colors']
        fig, axes = new_figure('lead')
        marks = STYLE['lead_markers']
        styles = STYLE['lead_line_styles']
        for col, regime in enumerate(REGIMES):
            ax = axes[col]
            f = lead[lead.regime == regime]
            for j, action in enumerate(actions):
                z = f[f.action_delay_s == action].sort_values('publication_delay_s')
                if not len(z):
                    continue
                color = C[STYLE['lead_color_order'][j % len(STYLE['lead_color_order'])]]
                center = (z.finite_lower_s + z.finite_upper_s) / 2
                err = (z.finite_upper_s - z.finite_lower_s) / 2
                ax.errorbar(z.publication_delay_s, center, yerr=err, color=color, ls=styles[j %
                ↪  len(styles)], marker=marks[j % len(marks)], ms=STYLE['mark_sizes']['lead'],
                ↪  capsize=STYLE['errorbar_capsize'], lw=STYLE['line_widths']['lead'],
                ↪  label=f'Action delay {fmt(action, 1)} s')
            ax.axhline(0, color=C['failure'], ls=STYLE['line_styles']['baseline'],
            ↪  lw=STYLE['line_widths']['local_zero'])
            ax.set(xlabel='Publication elapsed time (s)', ylabel='Finite local lead interval (s)')
            panel(ax, chr(97 + col), NAMES[regime] + ('; + no-hit' if
            ↪  f.includes_no_hit.astype(bool).any() else ''))
        handles, labels = axes[0].get_legend_handles_labels()
        fig.legend(handles, labels, ncol=min(STYLE['figures']['lead']['legend_max_columns'],
        ↪  len(actions)), bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
        export(fig, 'fig05_local_lead', out, audit)

    def q4_figure(data, out, audit):
        demo = data['q4_demo']
        if demo is None:
            return False
        pairs = demo[['publication_delay_s',
        ↪  'action_delay_s']].drop_duplicates().sort_values(['publication_delay_s',
        ↪  'action_delay_s'])
```

```
    pub = float(pairs.iloc[-1].publication_delay_s)
    act = float(pairs.iloc[-1].action_delay_s)
    fig, axes = new_figure('q4')
    C = STYLE['colors']
    for col, regime in enumerate(REGIMES):
        f = demo[(demo.regime == regime) & (demo.publication_delay_s == pub) &
        ↪  (demo.action_delay_s == act)].sort_values('scenario_id')
        if not len(f):
            raise ValueError('q4_spatial_demo.csv: selected saved delay pair missing in a
            ↪  regime')
        ax = axes[col]
        labels = []
        for j, (_, r) in enumerate(f.iterrows()):
            labels.append(f'S{j + 1}')
            if r.status == 'arrived':
                ax.plot([r.finite_lead_lower_s, r.finite_lead_upper_s], [j, j],
                ↪  color=C['active'], lw=STYLE['line_widths']['interval'],
                ↪  marker=STYLE['interval_marker'],
                ↪  ms=STYLE['mark_sizes']['interval_endpoint'])
            else:
                label = f'No arrival by {fmt(r.horizon_s, 1)} s' if 'no_arrival' in r.status
                ↪  else str(r.status)
                ax.text(STYLE['q4_status_x_fraction'], j, label,
                ↪  transform=ax.get_yaxis_transform(),
                ↪  va=STYLE['status_vertical_alignment'],
                ↪  ha=STYLE['status_horizontal_alignment'], fontsize=STYLE['status_size'],
                ↪  color=C['passive'])
        ax.axvline(0, color=C['failure'], ls=STYLE['line_styles']['baseline'],
        ↪  lw=STYLE['line_widths']['q4_zero'])
        ax.set(yticks=range(len(f)), yticklabels=labels, xlabel='Conditional target lead
        ↪  interval (s)', ylabel='Fixed synthetic Q4 scenario')
        ax.invert_yaxis()
        panel(ax, chr(97 + col), NAMES[regime])
    fig.suptitle(f'Synthetic 8-degree slope; 1000 m; elapsed publication {fmt(pub, 1)} s,
    ↪  action {fmt(act, 1)} s', y=STYLE['supertitle_y'], fontsize=STYLE['supertitle_size'])
    export(fig, 'fig06_synthetic_target_lead', out, audit)
    return True
SENSITIVITY_FILES = ['phase_map.csv', 'chi_perturbation.csv', 'crossing_amplification.csv',
↪  'local_derivatives.csv', 'derivative_step_stability.csv', 'distribution_sensitivity.csv',
↪  'set_sensitivity.csv', 'summary.json']

def read_sensitivity(folder):
    for name in SENSITIVITY_FILES:
        if not (folder / name).is_file():
            raise ValueError('Missing required saved sensitivity result: sensitivity/' + name)
    d = {name[:-4]: pd.read_csv(folder / name) for name in SENSITIVITY_FILES if
    ↪  name.endswith('.csv')}
    d['summary'] = load_json(folder / 'summary.json')
    d['legacy_numerical_convergence'] = pd.read_csv(folder /
    ↪  'legacy_numerical_convergence.csv') if (folder /
    ↪  'legacy_numerical_convergence.csv').is_file() else None
    fields = {
        'phase_map': ['regime', 'q0', 'offset_m_s', 'status', 'tau_s'],
        'chi_perturbation': ['regime', 'chi_multiplier', 'offset_m_s', 'status', 'tau_s'],
```

```
    'crossing_amplification': ['regime', 'status', 'tau_s', 'qdot_cross_s_inv',
    ↪  'seconds_per_0_001_q', 'horizon_slack_s', 'shear_qdot_s_inv',
    ↪  'drainage_qdot_s_inv', 'drainage_to_abs_shear_ratio'],
    'local_derivatives': ['regime', 'parameter', 'step', 'absolute_parameter_step',
    ↪  'minus_status', 'plus_status', 'minus_tau_s', 'plus_tau_s', 'derivative',
    ↪  'derivative_unit', 'elasticity', 'parameter_reference_scale',
    ↪  'scaled_sensitivity'],
    'derivative_step_stability': ['regime', 'parameter', 'finite_step_count',
    ↪  'total_step_count'],
    'distribution_sensitivity': ['regime', 'support_halfwidth_m_s', 'family',
    ↪  'target_mass', 'hit_mass', 'no_hit_mass', 'unknown_mass', 'total_mass'],
    'set_sensitivity': ['regime', 'support_halfwidth_m_s', 'target_mass', 'method',
    ↪  'finite_lower_s', 'finite_upper_s', 'finite_width_s', 'includes_no_hit',
    ↪  'worst_model_mass'],
}
for name, cols in fields.items():
    need_columns(d[name], cols, 'sensitivity/' + name + '.csv')
for name in ['phase_map', 'chi_perturbation']:
    frame = d[name]
    if not frame.regime.isin(REGIMES).all() or not frame.status.isin(['hit', 'no_hit',
    ↪  'unknown', 'initial']).all():
        raise ValueError('invalid sensitivity condition/status: ' + name)
    if frame.loc[frame.status != 'hit', 'tau_s'].notna().any():
        raise ValueError('non-hit sensitivity members must not carry finite time')
    hit_times = frame.loc[frame.status == 'hit', 'tau_s'].to_numpy(float)
    if not np.isfinite(hit_times).all() or not ((hit_times > 0) & (hit_times <=
    ↪  73)).all():
        raise ValueError('hit sensitivity members require finite time in(0,73]')
phase = d['phase_map']
if phase.duplicated(['regime', 'q0', 'offset_m_s']).any():
    raise ValueError('duplicate sensitivity phase design point')
for regime in REGIMES:
    frame = phase[phase.regime == regime]
    if len(frame) != frame.q0.nunique() * frame.offset_m_s.nunique():
        raise ValueError('incomplete phase design grid')
amp = d['crossing_amplification']
if len(amp) != len(REGIMES) or set(amp.regime) != set(REGIMES):
    raise ValueError('one nominal amplification row per condition required')
if not np.allclose(amp.seconds_per_0_001_q, 0.001 / abs(amp.qdot_cross_s_inv),
↪  rtol=1e-09, atol=1e-12):
    raise ValueError('local amplification does not match saved crossing slope')
if not np.allclose(amp.shear_qdot_s_inv + amp.drainage_qdot_s_inv, amp.qdot_cross_s_inv,
↪  rtol=1e-08, atol=1e-12):
    raise ValueError('saved mechanical terms do not sum to qdot')
dist = d['distribution_sensitivity']
if not np.isfinite(dist[['hit_mass', 'no_hit_mass', 'unknown_mass',
↪  'total_mass']]).all().all() or (dist[['hit_mass', 'no_hit_mass', 'unknown_mass']] <
↪  -1e-10).any().any():
    raise ValueError('invalid sensitivity probability mass')
if not np.allclose(dist.total_mass, dist.hit_mass + dist.no_hit_mass + dist.unknown_mass,
↪  atol=1e-08, rtol=0):
    raise ValueError('saved sensitivity total_mass contradicts component masses')
if not np.allclose(dist.hit_mass + dist.no_hit_mass + dist.unknown_mass, 1, atol=1e-08,
↪  rtol=0):
```

```
    raise ValueError('sensitivity probability mass does not close')
local = d['local_derivatives']
if not local.parameter.isin(['offset_m_s', 'q0', 'chi_relative']).all():
    raise ValueError('unsupported sensitivity derivative parameter')
if not np.isfinite(local.absolute_parameter_step.to_numpy(float)).all() or not
↪  (local.absolute_parameter_step > 0).all():
    raise ValueError('finite-difference absolute step must be positive finite')
for side in ['minus', 'plus']:
    status = local[side + '_status']
    tau = local[side + '_tau_s']
    if not status.isin(['hit', 'no_hit', 'initial', 'unknown']).all():
        raise ValueError('invalid finite-difference member status')
    times = tau[status == 'hit'].to_numpy(float)
    if not np.isfinite(times).all() or not ((times > 0) & (times <= 73)).all():
        raise ValueError('finite-difference hit side needs finite time in(0,73]')
    if tau[status != 'hit'].notna().any():
        raise ValueError('finite-difference non-hit side cannot carry time')
both = local.minus_status.eq('hit') & local.plus_status.eq('hit')
if not np.isfinite(local.loc[both, 'derivative'].to_numpy(float)).all():
    raise ValueError('both-hit finite-difference pair requires its finite derivative, not
    ↪  category-transition NaN')
expected = (local.loc[both, 'plus_tau_s'] - local.loc[both, 'minus_tau_s']) / (2 *
↪  local.loc[both, 'absolute_parameter_step'])
if not np.allclose(local.loc[both, 'derivative'], expected, rtol=1e-07, atol=1e-08):
    raise ValueError('finite difference disagrees with saved two-side times and step')
if local.loc[~both, ['derivative', 'elasticity',
↪  'scaled_sensitivity']].notna().any().any():
    raise ValueError('category-transition pair must not have a finite derivative or
    ↪  normalized derivative')
nominal = d['crossing_amplification'].set_index('regime')
for index, row in local[both].iterrows():
    scale = 1.5 if row.parameter == 'offset_m_s' else 0.5 if row.parameter == 'q0' else
    ↪  float(d['summary']['conditions'][row.regime])
    if not math.isfinite(row.parameter_reference_scale) or not
    ↪  math.isclose(row.parameter_reference_scale, scale, rel_tol=1e-10, abs_tol=1e-14):
        raise ValueError('finite-difference reference scale differs from declared design')
    scaled = row.derivative * scale / float(nominal.loc[row.regime, 'tau_s'])
    if not math.isfinite(row.scaled_sensitivity) or not
    ↪  math.isclose(row.scaled_sensitivity, scaled, rel_tol=1e-07, abs_tol=1e-08):
        raise ValueError('scaled sensitivity disagrees with derivative, reference scale
        ↪  and nominal time')
    if row.parameter == 'offset_m_s':
        if pd.notna(row.elasticity):
            raise ValueError('ordinary elasticity at nominal d=0 must remain undefined')
    elif not math.isfinite(row.elasticity) or not math.isclose(row.elasticity, scaled,
    ↪  rel_tol=1e-07, abs_tol=1e-08):
        raise ValueError('ordinary elasticity disagrees with finite difference')
sets = d['set_sensitivity']

def strict_bool(value):
    if isinstance(value, (bool, np.bool_)):
        return bool(value)
    if isinstance(value, str) and value in ['True', 'False']:
        return value == 'True'
```

```
        raise ValueError('sensitivity set includes_no_hit must contain Boolean True/False,
        ↪  not arbitrary strings')
    sets['includes_no_hit'] = sets.includes_no_hit.map(strict_bool)
    if not np.isfinite(sets[['finite_lower_s', 'finite_upper_s', 'finite_width_s',
    ↪  'worst_model_mass', 'target_mass']].to_numpy(float)).all():
        raise ValueError('finite sensitivity candidate requires finite endpoints and masses')
    if (sets.finite_width_s < 0).any() or not np.allclose(sets.finite_upper_s -
    ↪  sets.finite_lower_s, sets.finite_width_s, atol=1e-09, rtol=0):
        raise ValueError('sensitivity set endpoint/width mismatch')
    if (sets.worst_model_mass < sets.target_mass - 1e-10).any():
        raise ValueError('sensitivity set violates its declared numerical mass target')
    legacy = d['legacy_numerical_convergence']
    if legacy is not None:
        need_columns(legacy, ['base_status', 'tight_status', 'base_tau_s', 'tight_tau_s',
        ↪  'tau_abs_difference_s', 'status_agrees', 'base_rtol', 'tight_rtol',
        ↪  'base_max_step_s', 'tight_max_step_s'],
        ↪  'sensitivity/legacy_numerical_convergence.csv')
        if len(legacy) != 6:
            raise ValueError('historical numerical convergence requires exactly six saved
            ↪  cases')
        if not legacy.status_agrees.map(lambda value: isinstance(value, (bool, np.bool_)) and
        ↪  bool(value)).all():
            raise ValueError('historical convergence status_agrees must be Boolean True for
            ↪  every case')
        if not legacy.base_status.isin(['hit', 'no_hit']).all() or not
        ↪  legacy.base_status.eq(legacy.tight_status).all():
            raise ValueError('historical convergence base/tight statuses must agree')
        for base, tight, ratio in [('base_rtol', 'tight_rtol', 10.0), ('base_max_step_s',
        ↪  'tight_max_step_s', 2.0)]:
            values = legacy[[base, tight]].to_numpy(float)
            if not np.isfinite(values).all() or not (values > 0).all() or (not
            ↪  np.allclose(legacy[base] / legacy[tight], ratio, rtol=1e-12, atol=0)):
                raise ValueError('historical numerical tightening ratios contradict the
                ↪  declared 10x tolerance / 2x step design')
        hit = legacy.base_status.eq('hit')
        columns = ['base_tau_s', 'tight_tau_s', 'tau_abs_difference_s']
        if legacy.loc[~hit, columns].notna().any().any():
            raise ValueError('historical no-hit convergence case must leave all time fields
            ↪  empty')
        values = legacy.loc[hit, columns].to_numpy(float)
        if not np.isfinite(values).all() or not ((values[:, :2] > 0) & (values[:, :2] <=
        ↪  73)).all() or (not (values[:, 2] >= 0).all()):
            raise ValueError('historical hit convergence cases require valid finite times and
            ↪  nonnegative differences')
        expected = abs(legacy.loc[hit, 'base_tau_s'] - legacy.loc[hit, 'tight_tau_s'])
        if not np.allclose(legacy.loc[hit, 'tau_abs_difference_s'], expected, rtol=1e-07,
        ↪  atol=1e-12):
            raise ValueError('historical convergence difference contradicts saved finite
            ↪  times')
    return d

def grid_cell_edges(x):
    x = np.asarray(x, float)
    if len(x) < 2 or not np.all(np.diff(x) > 0):
```

```
        raise ValueError('ordered design coordinates required')
    return np.r_[x[0] - (x[1] - x[0]) / 2, (x[:-1] + x[1:]) / 2, x[-1] + (x[-1] - x[-2]) / 2]

def sensitivity_phase_figure(data, out, audit):
    from matplotlib.colors import Normalize
    from matplotlib.patches import Patch
    d = data['sensitivity']['phase_map']
    st = STYLE['sensitivity']
    fig, axes = new_figure('sensitivity_phase')
    for col, regime in enumerate(REGIMES):
        f = d[d.regime == regime]
        q = np.sort(f.q0.unique())
        x = np.sort(f.offset_m_s.unique())
        pivot = f.pivot(index='q0', columns='offset_m_s', values='tau_s').reindex(index=q,
        ↪  columns=x)
        vals = pivot.to_numpy(float)
        mask = np.ma.masked_invalid(vals)
        cmap = plt.get_cmap(st['phase_cmap']).copy()
        cmap.set_bad(st['no_hit_color'])
        maximum = float(np.nanmax(vals))
        mesh = axes[col].pcolormesh(grid_cell_edges(x), grid_cell_edges(q), mask, cmap=cmap,
        ↪  norm=Normalize(0, maximum), shading=st['phase_shading'],
        ↪  rasterized=st['phase_rasterized'])
        cb = fig.colorbar(mesh, ax=axes[col], fraction=st['colorbar_fraction'],
        ↪  pad=st['colorbar_pad'])
        cb.ax._q3_auxiliary = True
        cb.set_label('Elapsed first-hit time (s)')
        axes[col].scatter([0], [0.5], marker=st['nominal_marker'], s=st['nominal_area'],
        ↪  facecolors=st['nominal_face'], edgecolors=STYLE['colors']['truth'],
        ↪  linewidths=st['nominal_edge_width'], zorder=st['nominal_zorder'])
        for status, marker in [('unknown', st['unknown_marker']), ('initial',
        ↪  st['initial_marker'])]:
            group = f[f.status == status]
            if len(group):
                axes[col].scatter(group.offset_m_s, group.q0, marker=marker,
                ↪  s=st['category_area'], color=STYLE['colors']['failure'],
                ↪  linewidths=st['category_edge_width'])
        axes[col].set(xlabel='Whole-path velocity offset (m/s)', ylabel='Initial stress ratio
        ↪  q0', xlim=(x[0], x[-1]), ylim=(q[0], q[-1]))
        panel(axes[col], chr(97 + col), NAMES[regime])
    handles = [Patch(facecolor=st['no_hit_color'], label='No hit within 73 s'), Line2D([],
    ↪  [], marker=st['nominal_marker'], ls=STYLE['legend_marker_line'],
    ↪  markerfacecolor=st['nominal_face'], markeredgecolor=STYLE['colors']['truth'],
    ↪  label='Nominal q0=0.5, offset=0')]
    fig.legend(handles=handles, ncol=STYLE['figures']['sensitivity_phase']['legend_columns'],
    ↪  bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
    export(fig, 'fig07_initial_velocity_sensitivity', out, audit)

def sensitivity_limits_figure(data, out, audit):
    sens = data['sensitivity']
    amp = sens['crossing_amplification'].set_index('regime')
    sets = sens['set_sensitivity']
    chi = sens['chi_perturbation']
    st = STYLE['sensitivity']
```

```
C = STYLE['colors']
fig, axes = new_figure('sensitivity_limits')
values = [float(amp.loc[r, 'seconds_per_0_001_q']) for r in REGIMES]
axes[0, 0].scatter(range(len(REGIMES)), values, color=[C['truth'], C['active']],
↪  s=st['amplification_area'], marker=st['amplification_marker'])
axes[0, 0].set_ylim(min(values) / st['amplification_bottom_factor'], max(values) *
↪  st['amplification_top_factor'])
axes[0, 0].text(*st['amplification_ratio_anchor'], 'Near / strong = ' + fmt(values[1] /
↪  values[0], 1), transform=axes[0, 0].transAxes, fontsize=STYLE['status_size'],
↪  va=STYLE['status_vertical_alignment'], ha=st['amplification_ratio_ha'])
axes[0, 0].set(xticks=range(len(REGIMES)), xticklabels=[NAMES[r] for r in REGIMES],
↪  ylabel='Time / 0.001 state margin (s)', yscale='log')
axes[0, 0].tick_params(axis='x', labelrotation=st['regime_tick_rotation'])
panel(axes[0, 0], 'a', 'Local transversal amplification')
ax = axes[0, 1]
f = chi[(chi.regime == 'near') & np.isclose(chi.offset_m_s, 0)]
order = f.sort_values('chi_multiplier')
hit = order[order.status == 'hit']
no = order[order.status == 'no_hit']
ax.plot(order.chi_multiplier, order.tau_s, color=C['active'],
↪  ls=STYLE['line_styles']['active'], marker=STYLE['mode_styles']['raw']['marker'],
↪  lw=STYLE['line_widths']['distribution_gp'], ms=STYLE['marker_size'])
if len(no):
    ax.scatter(no.chi_multiplier, np.full(len(no), st['nohit_strip_y']),
    ↪  transform=ax.get_xaxis_transform(), marker=STYLE['nohit_marker'],
    ↪  s=st['nohit_strip_area'], facecolors=STYLE['hollow_marker_facecolor'],
    ↪  edgecolors=C['passive'], clip_on=False, label='No hit (category)')
ax.set(xlabel='χ / nominal χ (near-boundary condition)', ylabel='Elapsed first-hit time
↪  (s)', xlim=(float(order.chi_multiplier.min()) - st['chi_xlim_padding'],
↪  float(order.chi_multiplier.max()) + st['chi_xlim_padding']))
panel(ax, 'b', 'Conditional χ perturbation')
if len(no):
    ax.legend(loc=st['chi_legend_loc'], frameon=STYLE['legend']['frameon'],
    ↪  bbox_to_anchor=st['chi_legend_anchor'])
for regime in REGIMES:
    a = sets[(sets.regime == regime) & (sets.method == 'robust_shortest') &
    ↪  np.isclose(sets.target_mass, 0.95)].sort_values('support_halfwidth_m_s')
    a = a.drop_duplicates('support_halfwidth_m_s')
    color = C['truth'] if regime == 'strong' else C['active']
    marker = st['condition_markers'][regime]
    line = st['condition_lines'][regime]
    axes[1, 0].plot(a.support_halfwidth_m_s, a.finite_width_s, color=color, ls=line,
    ↪  marker=marker, lw=STYLE['line_width'], ms=STYLE['marker_size'],
    ↪  label=NAMES[regime])
    b = sets[(sets.regime == regime) & (sets.method == 'robust_shortest') &
    ↪  np.isclose(sets.support_halfwidth_m_s, 1.5)].sort_values('target_mass')
    axes[1, 1].plot(b.target_mass, b.finite_width_s, color=color, ls=line, marker=marker,
    ↪  lw=STYLE['line_width'], ms=STYLE['marker_size'], label=NAMES[regime])
axes[1, 0].set(xlabel='Offset support half-width (m/s)', ylabel='Robust finite width
↪  (s)', yscale='log')
panel(axes[1, 0], 'c', 'Input-support sensitivity')
axes[1, 1].set(xlabel='Target model mass', ylabel='Robust finite width (s)', yscale='log')
panel(axes[1, 1], 'd', 'Coverage-target sensitivity')
handles, labels = axes[1, 0].get_legend_handles_labels()
```

```
    fig.legend(handles, labels,
    ↪  ncol=STYLE['figures']['sensitivity_limits']['legend_columns'],
    ↪  bbox_to_anchor=STYLE['legend_anchor'], **STYLE['legend'])
    for ax in [axes[0, 0], axes[1, 0], axes[1, 1]]:
        ax.tick_params(axis='y', labelsize=st['log_tick_size'])
    export(fig, 'fig08_boundary_probability_sensitivity', out, audit)

def sol():
    ap = argparse.ArgumentParser(description='Q3 plots')
    base = Path(__file__).resolve().parent
    ap.add_argument('--results', type=Path, default=base / 'results')
    ap.add_argument('--output', type=Path, default=base / 'figures_reproduced')
    ap.add_argument('--sensitivity-results', type=Path, default=None)
    args = ap.parse_args()
    results = args.results.resolve()
    out = args.output.resolve()
    if results == out or out.is_relative_to(results):
        raise ValueError('Figure output must not be inside saved analysis results')
    if out.exists():
        raise FileExistsError('output directory exists: use a new output directory')
    data = read_inputs(results, None if args.sensitivity_results is None else
    ↪  args.sensitivity_results.resolve())
    out.mkdir(parents=True, exist_ok=True)
    audit = out / 'audit'
    audit.mkdir(exist_ok=True)
    apply_style()
    response_figure(data, out, audit)
    learning_figure(data, out, audit)
    validation_figure(data, out, audit)
    distribution_figure(data, out, audit)
    lead_figure(data, out, audit)
    has_q4 = q4_figure(data, out, audit)
    sensitivity_phase_figure(data, out, audit)
    sensitivity_limits_figure(data, out, audit)
    save(audit / 'PLOT_RECEIPT.json',
    ↪  dict(utc=datetime.datetime.now(datetime.timezone.utc).isoformat(),
    ↪  source_sha256=sha(__file__), result_version=data['summary'].get('version',
    ↪  data['summary'].get('run_id')), input_sha256={f: sha(results / f) for f in REQUIRED +
    ↪  (['q4_spatial_demo.csv'] if has_q4 else [])}, sensitivity_input_sha256={name:
    ↪  sha((results / 'sensitivity' if args.sensitivity_results is None else
    ↪  args.sensitivity_results.resolve()) / name) for name in SENSITIVITY_FILES +
    ↪  (['legacy_numerical_convergence.csv'] if ((results / 'sensitivity' if
    ↪  args.sensitivity_results is None else args.sensitivity_results.resolve()) /
    ↪  'legacy_numerical_convergence.csv').is_file() else [])}, output_sha256={p.name:
    ↪  sha(p) for p in out.iterdir() if p.is_file()}, q2_calls=0, training_calls=0,
    ↪  random_draws=0, styles=STYLE, audit_scope='self-contained rectangular axes geometry
    ↪  only; external PDF and manual visual audits recorded separately',
    ↪  independent_scientific_acceptance='pending'))
    print(json.dumps(dict(figure_count=8 if has_q4 else 7, q2_calls=0), ensure_ascii=False))
if __name__ == '__main__':
    sol()
```

q3/sol.py

```
from __future__ import annotations
import os
for _name in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
    os.environ.setdefault(_name, '1')
import sys
sys.dont_write_bytecode = True
import argparse, hashlib, json, math, platform, time, warnings
from pathlib import Path
from datetime import datetime, timezone
from dataclasses import dataclass, asdict
from decimal import Decimal
from types import SimpleNamespace
from concurrent.futures import ProcessPoolExecutor
import numpy as np
import pandas as pd
import scipy, sklearn
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.optimize import brentq
from scipy.stats import beta, binom
from sklearn.gaussian_process import GaussianProcessClassifier, GaussianProcessRegressor
from sklearn.gaussian_process.kernels import ConstantKernel, Matern
from sklearn.exceptions import ConvergenceWarning
PACKAGE_ROOT = Path(__file__).resolve().parent
ROOT = PACKAGE_ROOT
DATA = PACKAGE_ROOT / 'data'
OUT = PACKAGE_ROOT / 'results'
ATOL = (1e-11, 1e-06, 1e-10)
G = 9.81
_DENSE_NODES = np.array([0.0, 1 / 3, 2 / 3, 1.0])
_CUBIC_INVERSE = np.linalg.inv(np.vander(_DENSE_NODES, 4, increasing=True))
FAMILIES = ['nominal_triangular', 'uniform_support', 'lower_mode_triangular']
CONDITIONS = {'Q2_strong_design': 0.02, 'Q2_near_boundary_comparison': 0.0002265378784906}

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def fingerprint(path):
    return sha(path)

def save(path, obj):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False) + '\n')

def stamp():
    return datetime.now(timezone.utc).isoformat()

@dataclass(frozen=True)
class Case:
    m0: float
    H0: float
    mcrit: float
    rho_s: float
```

```
    rho_f: float
    mu: float
    a: float
    sigma0: float
    delta: float
    Kfs_transfer_m_s: float
    water_conversion_mu_Pa_s: float
    theta_deg: float
    chi_m_inv: float
    curve_shift: float = 0.0
    start: float = 27.0
    end: float = 100.0
    mode: str = 'coupled'
    shape_n: float = 2.0
    initial_stress_ratio: float = 0.5

    @property
    def s_init(self):
        return self.initial_stress_ratio * self.E

    @property
    def qcrit(self):
        return (self.shape_n - 1) / (2 * (self.shape_n + 1))

    @property
    def ki(self):
        return self.Kfs_transfer_m_s * self.water_conversion_mu_Pa_s / (self.rho_f * G)

    @property
    def Hs(self):
        return self.m0 * self.H0

    @property
    def E(self):
        return (self.rho_s - self.rho_f) * G * math.cos(math.radians(self.theta_deg)) *
        ↪  self.Hs

class Forcing:

    def __init__(self, path):
        d = pd.read_csv(path)
        self.t = d.time_since_refined_onset_s.to_numpy(float)
        self.u = d.velocity_m_s.to_numpy(float)
        if not (np.all(np.diff(self.t) > 0) and np.all(self.u >= 0)):
            raise ValueError('Forcing requires ordered times and nonnegative velocities')

    def speed(self, case, t):
        if case.start < self.t[0] or case.end > self.t[-1]:
            raise ValueError('Requested forcing window exceeds digitized data; no
            ↪  extrapolation')
        return max(0.0, float(np.interp(t + case.start, self.t, self.u)) + case.curve_shift)

def constitutive(case, m, s, gamma):
    if not 0 < m < 1 or s <= -case.sigma0:
```

```
        raise ValueError('Constitutive state outside domain')
    alpha = case.a / (m * (s + case.sigma0))
    N = 0.0 if gamma == 0 else case.mu * gamma / (case.rho_s * gamma ** 2 * case.delta ** 2 +
    ↪  max(s, 0.0))
    meq = case.mcrit / (1 + math.sqrt(N))
    k = case.ki * math.exp((case.m0 - m) / 0.04)
    return (alpha, N, meq, k)

def closure(case, m, s):
    H = case.Hs / m
    zeta = s / case.E
    if zeta >= case.qcrit:
        e = (case.shape_n + 1) / case.shape_n * (case.E / 2 - s)
        r = 1.0
        Jfactor = case.shape_n * e / H
    else:
        r = math.sqrt(max(0.0, zeta / case.qcrit))
        e = case.E * (1 - (case.shape_n - 1) * r / case.shape_n)
        Jfactor = case.E / H
    return (H, e, r, Jfactor)

def rhs(case, forcing, t, y):
    m, s, _ = y
    U = forcing.speed(case, t)
    gamma = 0.0 if case.mode == 'zero_shear' else case.chi_m_inv * U
    alpha, N, meq, k = constitutive(case, m, s, gamma)
    H, e, r, Jfactor = closure(case, m, s)
    if case.mode == 'drained':
        dm = -m * gamma * (m - meq)
        J = H * dm / m
        ds = 0.0
    else:
        J = k * Jfactor / case.mu
        dm = m * J / H
        ds = (gamma * (m - meq) + J / H) / alpha
    return (dm, ds, J)

def profile(case, m, s, x):
    H, e, r, _ = closure(case, m, s)
    if s / case.E >= case.qcrit:
        pe = e * (1 - x ** case.shape_n)
        effective = case.E * (1 - x) - pe
    else:
        rn = max(r, 1e-100)
        pe = np.where(x <= r, e - case.E * x ** case.shape_n / (case.shape_n * rn **
        ↪  (case.shape_n - 1)), case.E * (1 - x))
        effective = np.where(x <= r, case.E * ((case.shape_n - 1) * r / case.shape_n - x + x
        ↪  ** case.shape_n / (case.shape_n * rn ** (case.shape_n - 1))), 0.0)
    hydro = case.rho_f * G * math.cos(math.radians(case.theta_deg)) * H * (1 - x)
    return (pe, effective, hydro + pe)

def initial_classification(case):
    q0 = case.initial_stress_ratio
    B = case.rho_f * G * math.cos(math.radians(case.theta_deg)) * case.H0
```

```
    qmax = 0.5 + B / ((case.shape_n + 1) * case.E)
    if not math.isfinite(q0) or q0 < 0 or q0 > qmax + 1e-12:
        status = 'invalid_initial_state'
    elif math.isclose(q0, 0, rel_tol=0, abs_tol=1e-12):
        status = 'initial_closure_limit'
    elif math.isclose(q0, qmax, rel_tol=0, abs_tol=1e-12):
        status = 'initial_saturation_boundary'
    elif q0 < case.qcrit - 1e-12:
        status = 'initial_positive_thickness_layer'
    elif math.isclose(q0, case.qcrit, rel_tol=0, abs_tol=1e-12):
        status = 'initial_first_layer_boundary'
    else:
        status = 'admissible_unreached_initial_state'
    f0 = 1 - math.sqrt(q0 / case.qcrit) if 0 <= q0 < case.qcrit else 0.0 if q0 >= case.qcrit
    ↪  else None
    if status == 'invalid_initial_state':
        f0 = None
    return {
        'q0': q0,
        's_init_Pa': case.s_init,
        'qcrit': case.qcrit,
        'qmax': qmax,
        'initial_status': status,
        'initial_fluid_supported_fraction': f0,
        'initial_condition_satisfied': None if status in ('initial_closure_limit',
        ↪  'invalid_initial_state') else status in ('initial_positive_thickness_layer',
        ↪  'initial_first_layer_boundary'),
        'dynamic_first_hit_elapsed_s': None,
        'identity': 'mathematical closure admission only; not local observed or credible
        ↪  range',
    }

def run_case(case, forcing, numerics, dense=True, keep_solution=False):
    if not (0 < case.m0 < 1 and 0 < case.mcrit < 1 and (case.H0 > 0) and (case.ki > 0) and
    ↪  (case.mu > 0) and (case.shape_n > 1)):
        raise ValueError('Invalid physical input')
    admission = initial_classification(case)
    if admission['initial_status'] != 'admissible_unreached_initial_state':
        raise ValueError('Initial state is classified separately: ' +
        ↪  admission['initial_status'])
    if case.mode == 'drained' and case.initial_stress_ratio != 0.5:
        raise ValueError('Historical ideal-drained control is restricted to hydrostatic
        ↪  initial state')

    def zero_stress(t, y):
        return y[1]
    zero_stress.terminal = True
    zero_stress.direction = -1

    def desaturation(t, y):
        H, e, r, _ = closure(case, y[0], y[1])
        return e + case.rho_f * G * math.cos(math.radians(case.theta_deg)) * H / case.shape_n
    desaturation.terminal = True
    desaturation.direction = -1
```

```
def fluid_layer(t, y):
    return y[1] / case.E - case.qcrit
fluid_layer.terminal = False
fluid_layer.direction = -1
duration = case.end - case.start
sol = solve_ivp(lambda t, y: rhs(case, forcing, t, y), (0, duration), (case.m0,
↪  case.s_init, 0.0), method='Radau', rtol=numerics['rtol'], atol=(1e-11, 1e-06, 1e-10),
↪  max_step=numerics['max_step_s'], dense_output=True, events=(zero_stress,
↪  desaturation, fluid_layer))
if not sol.success:
    raise RuntimeError(sol.message)
state = 'completed_conditional_window'
if len(sol.t_events[0]):
    state = 'mean_stress_zero_closure_limit'
if len(sol.t_events[1]):
    state = 'saturation_hypothesis_boundary'
if dense:
    ts = np.unique(np.r_[np.arange(0, sol.t[-1], numerics['output_step_s']), sol.t[-1],
    ↪  sol.t_events[2]])
else:
    ts = np.unique(np.r_[sol.t, sol.t_events[2]])
ys = sol.sol(ts)
rows = []
residual = []
for t, (m, s, budget) in zip(ts, ys.T):
    H, e, r, _ = closure(case, m, s)
    U = forcing.speed(case, t)
    gamma = 0.0 if case.mode == 'zero_shear' else case.chi_m_inv * U
    alpha, N, meq, k = constitutive(case, m, s, gamma)
    dm, ds, J = rhs(case, forcing, t, (m, s, budget))
    if case.mode == 'drained':
        e = 0.0
        r = 1.0
    resid = gamma * (m - meq) - alpha * ds + dm / m
    residual.append(resid)
    rows.append({
        'elapsed_s': t,
        'source_relative_s': t + case.start,
        'U_input_m_s': U,
        'gamma_input_s_inv': gamma,
        'm': m,
        'H_m': H,
        'mean_effective_stress_Pa': s,
        'mean_effective_ratio': s / case.E,
        'basal_excess_Pa': e,
        'basal_excess_ratio': e / case.E,
        'fluid_supported_fraction': 1 - r,
        'drainage_flux_m_s': J,
        'cumulative_outflow_m': budget,
        'skeleton_thickness_m': H * r,
        'skeleton_to_delta_ratio': H * r / case.delta,
        'alpha_Pa_inv': alpha,
        'k_m2': k,
```

```
            'N': N,
            'meq': meq,
            'solid_volume_residual_m': m * H - case.Hs,
            'water_budget_residual_m': H - case.H0 + budget,
            'constitutive_residual_s_inv': resid,
        })
    frame = pd.DataFrame(rows)
    x = np.linspace(0, 1, 1001)
    profile_checks = []
    for j in np.unique(np.linspace(0, len(frame) - 1, min(21, len(frame))).astype(int)):
        row = frame.iloc[j]
        pe, eff, total = profile(case, row.m, row.mean_effective_stress_Pa, x)
        if case.mode == 'drained':
            eff = case.E * (1 - x)
            total = case.rho_f * G * math.cos(math.radians(case.theta_deg)) * row.H_m * (1 -
            ↪  x)
        profile_checks.append((np.min(eff), np.min(total), abs(np.trapezoid(eff, x) -
        ↪  row.mean_effective_stress_Pa) / case.E))
    onset = float(sol.t_events[2][0]) if len(sol.t_events[2]) else None
    summary = {
        'q0': case.initial_stress_ratio,
        's_init_Pa': case.s_init,
        'initial_status': admission['initial_status'],
        'status': state,
        'onset_elapsed_s': onset,
        'onset_source_relative_s': None if onset is None else onset + case.start,
        'max_fluid_fraction': float(frame.fluid_supported_fraction.max()),
        'min_mean_effective_ratio': float(frame.mean_effective_ratio.min()),
        'final_m': float(frame.m.iloc[-1]),
        'final_H_m': float(frame.H_m.iloc[-1]),
        'end_elapsed_s': float(sol.t[-1]),
        'ki_m2': case.ki,
        'E_Pa': case.E,
        'max_solid_residual_m': float(frame.solid_volume_residual_m.abs().max()),
        'max_water_residual_m': float(frame.water_budget_residual_m.abs().max()),
        'max_constitutive_residual_s_inv': float(max((abs(x) for x in residual))),
        'min_profile_effective_stress_Pa': float(min((x[0] for x in profile_checks))),
        'min_profile_total_pore_Pa': float(min((x[1] for x in profile_checks))),
        'max_profile_mean_relative_error': float(max((x[2] for x in profile_checks))),
        'min_skeleton_to_delta_ratio': float(frame.skeleton_to_delta_ratio.min()),
        'continuum_scale_warning': bool(frame.skeleton_to_delta_ratio.min() < 10),
        'continuum_warning_identity': '10 delta is a disclosed resolution diagnostic, not a
        ↪  calibrated formation boundary',
    }
    if keep_solution:
        return (frame, summary, sol)
    return (frame, summary)
Q2_NAMESPACE = SimpleNamespace(Case=Case, Forcing=Forcing, constitutive=constitutive,
↪  closure=closure, rhs=rhs, initial_classification=initial_classification,
↪  run_case=run_case, G=G)
Q2_CFG = None
FINAL = None

def configure(data_dir):
```

```
    global DATA, Q2_CFG, FINAL
    DATA = Path(data_dir)
    for name in ('q2_config.json', 'forcing.csv', 'frozen_validation.json'):
        if not (DATA / name).is_file():
            raise FileNotFoundError('required delivered input missing: data/' + name)
    Q2_CFG = json.loads((DATA / 'q2_config.json').read_text())
    FINAL = json.loads((DATA / 'frozen_validation.json').read_text())
    if Q2_CFG['version'] != 'Q2-ACTIVE-DATA-20261029-v3':
        raise ValueError('wrong frozen Q2 version')
    expected = json.loads((DATA / 'INPUT_HASHES.json').read_text())
    for name, value in expected.items():
        if sha(DATA / name) != value:
            raise ValueError('input fingerprint mismatch: ' + name)
    forcing = pd.read_csv(DATA / 'forcing.csv')
    required = ['time_since_refined_onset_s', 'velocity_m_s']
    if any((c not in forcing for c in required)):
        raise ValueError('forcing requires ' + str(required))
    a = forcing[required].to_numpy(float)
    if not np.isfinite(a).all() or not np.all(np.diff(a[:, 0]) > 0) or np.any(a[:, 1] < 0):
        raise ValueError('invalid forcing values/order/units')
    if a[0, 0] > 27 or a[-1, 0] < 100:
        raise ValueError('forcing does not cover frozen source-relative27-100s')

def load_frozen_q2():
    return Q2_NAMESPACE

def frozen_config():
    if Q2_CFG is None:
        configure(DATA)
    return Q2_CFG

def load_default_forcing():
    f = Forcing(DATA / 'forcing.csv')
    f.identity = 'same_event_retrospective_digitized_Q2_v3'
    f.metadata = {
        'source_file': 'data/forcing.csv',
        'source_sha256': sha(DATA / 'forcing.csv'),
        'not_available_for_pre_event_forecast': True,
    }
    return f

def audit_dense_first_hit(sol, *, stress_scale_E, qcrit, rtol, atol_stress=ATOL[1],
↪  state_index=1, known_hit_time=None):
    answer = {
        'method': 'accepted_step_Radau_cubic_stationary_point_audit',
        'audit_status': 'separated_no_contact',
        'recovered_crossing_tau_s': None,
        'contact_candidates': [],
        'min_interpolant_margin_q': None,
        'max_reconstruction_residual_q': 0.0,
        'max_contact_tolerance_q': 0.0,
        'accepted_steps_checked': 0,
        'tolerance_identity': '10*(atol_s/E + rtol*max|q|) heuristic; no certified global
        ↪  error',
```

```
    'continuous_physical_first_hit_certified': False,
}
if sol.sol is None or len(sol.t) < 2 or stress_scale_E <= 0:
    answer.update(audit_status='audit_unavailable_unknown')
    return answer
interpolants = getattr(sol.sol, 'interpolants', None)
if interpolants is None or any((type(p).__name__ != 'RadauDenseOutput' for p in
↪  interpolants)):
    answer.update(audit_status='unsupported_dense_interpolant_unknown')
    return answer
cutoff = float(sol.t[-1]) if known_hit_time is None else min(float(sol.t[-1]),
↪  float(known_hit_time))
crossings = []
minima = []
bad_reconstruction = False
for a, b0 in zip(sol.t[:-1], sol.t[1:]):
    a = float(a)
    b = min(float(b0), cutoff)
    if a >= cutoff or b <= a:
        continue
    width = b - a
    times = a + width * _DENSE_NODES
    q = np.asarray(sol.sol(times)[state_index], dtype=float) / stress_scale_E
    if not np.isfinite(q).all():
        answer['audit_status'] = 'nonfinite_dense_path_unknown'
        return answer
    values = q - qcrit
    coeff = _CUBIC_INVERSE @ values
    mid_margin = float(sol.sol(a + 0.5 * width)[state_index] / stress_scale_E - qcrit)
    reconstruction = abs(float(np.polynomial.polynomial.polyval(0.5, coeff)) - mid_margin)
    tol = max(128 * np.finfo(float).eps * max(1.0, float(np.max(np.abs(q)))), 10 *
    ↪  (float(atol_stress) / stress_scale_E + float(rtol) * float(np.max(np.abs(q)))))
    answer['max_reconstruction_residual_q'] =
    ↪  max(answer['max_reconstruction_residual_q'], reconstruction)
    answer['max_contact_tolerance_q'] = max(answer['max_contact_tolerance_q'], tol)
    answer['accepted_steps_checked'] += 1
    if reconstruction > tol:
        bad_reconstruction = True
    extrema = []
    for root in np.polynomial.polynomial.polyroots([coeff[1], 2 * coeff[2], 3 *
    ↪  coeff[3]]):
        if abs(root.imag) < 1e-09 and 0 < float(root.real) < 1:
            extrema.append(float(root.real))
    cuts = sorted(set([0.0, *extrema, 1.0]))
    cut_times = [a + width * s for s in cuts]
    margins = [float(sol.sol(t)[state_index] / stress_scale_E - qcrit) for t in cut_times]
    local_min = min(margins)
    minima.append(local_min)
    for j, (s, t, g) in enumerate(zip(cuts, cut_times, margins)):
        if abs(g) <= tol:
            is_interior = 0 < s < 1
            if known_hit_time is None or is_interior:
                answer['contact_candidates'].append({
                    'elapsed_s': t,
```

```
                        'margin_q': g,
                        'tolerance_q': tol,
                        'location': 'interior_stationary' if is_interior else 'step_boundary',
                        'step_start_s': a,
                        'step_end_s': b,
                    })
        for left, right, gl, gr in zip(cut_times[:-1], cut_times[1:], margins[:-1],
        ↪  margins[1:]):
            if gl >= 0 and gr < -tol:
                root = left if gl == 0 else brentq(lambda t: float(sol.sol(t)[state_index] /
                ↪  stress_scale_E - qcrit), left, right, xtol=1e-12, rtol=1e-14)
                crossings.append(float(root))
    answer['min_interpolant_margin_q'] = min(minima) if minima else None
    if bad_reconstruction:
        answer['audit_status'] = 'cubic_reconstruction_unresolved_unknown'
    elif crossings:
        answer.update(audit_status='recovered_crossing',
        ↪  recovered_crossing_tau_s=min(crossings))
    elif answer['contact_candidates']:
        answer['audit_status'] = 'contact_candidate'
    return answer

def build_case(q0=None, chi_m_inv=None, start_s=None, horizon_s=None):
    q2 = load_frozen_q2()
    cfg = frozen_config()
    values = dict(cfg['reference'])
    values['initial_stress_ratio'] = float(values['initial_stress_ratio'] if q0 is None else
    ↪  q0)
    values['chi_m_inv'] = float(values['chi_m_inv'] if chi_m_inv is None else chi_m_inv)
    if not math.isfinite(values['initial_stress_ratio']):
        raise ValueError('q0 must be finite; NaN/Inf cannot form a serializable conditional
        ↪  member')
    if not math.isfinite(values['chi_m_inv']) or values['chi_m_inv'] < 0:
        raise ValueError('chi must be a finite nonnegative conditional input')
    start = float(cfg['forcing']['start_relative_s'] if start_s is None else start_s)
    horizon = float(cfg['forcing']['end_relative_s'] - start if horizon_s is None else
    ↪  horizon_s)
    if not math.isfinite(start) or not math.isfinite(horizon) or horizon <= 0:
        raise ValueError('A finite positive supported horizon is required')
    values.update(start=start, end=start + horizon, curve_shift=0.0, mode='coupled',
    ↪  shape_n=cfg['initial_state_enhancement']['shape_n'])
    return q2.Case(**values)

class VelocityForcing:

    def __init__(self, times_s, velocity_m_s, *, identity='declared_Q3_input_path',
    ↪  metadata=None):
        self.t = np.asarray(times_s, dtype=float).copy()
        self.u = np.asarray(velocity_m_s, dtype=float).copy()
        if self.t.ndim != 1 or self.t.shape != self.u.shape or self.t.size < 2:
            raise ValueError('Forcing requires two or more paired 1-D time/velocity values')
        if not (np.all(np.isfinite(self.t)) and np.all(np.isfinite(self.u)) and
        ↪  np.all(np.diff(self.t) > 0) and np.all(self.u >= 0)):
            raise ValueError('Forcing requires ordered finite times and nonnegative finite
            ↪  speed')
```

```
        self.identity = str(identity)
        self.metadata = dict(metadata or {})
        self.path_sha256 = hashlib.sha256(self.t.astype('<f8').tobytes() +
        ↪  self.u.astype('<f8').tobytes()).hexdigest()

    def speed(self, case, t):
        if case.start < self.t[0] or case.end > self.t[-1]:
            raise ValueError('Requested path window exceeds input support; no extrapolation')
        at = float(t) + case.start
        if at < self.t[0] - 1e-10 or at > self.t[-1] + 1e-10:
            raise ValueError('Solver evaluation exceeds declared forcing support')
        return max(0.0, float(np.interp(at, self.t, self.u)) + case.curve_shift)

def _cut_diagnostics(q2, case, forcing, at, y):
    m, s, water_out = (float(v) for v in y)
    H, _, _, _ = q2.closure(case, m, s)
    dm, ds, flux = q2.rhs(case, forcing, float(at), (m, s, water_out))
    return {
        'elapsed_s': float(at),
        'source_relative_s': float(at + case.start),
        'm': m,
        's_Pa': s,
        'q': s / case.E,
        'H_m': H,
        'qdot_s_inv': float(ds / case.E),
        'drainage_flux_m_s': float(flux),
        'cumulative_outflow_m': water_out,
        'solid_budget_residual_m': float(m * H - case.Hs),
        'water_budget_residual_m': float(H - case.H0 + water_out),
    }

def _terminal_first_hit(q2, case, forcing, numerics):

    def zero(t, y):
        return y[1]
    zero.terminal = True
    zero.direction = -1

    def desaturation(t, y):
        H, e, _, _ = q2.closure(case, y[0], y[1])
        return e + case.rho_f * q2.G * math.cos(math.radians(case.theta_deg)) * H /
        ↪  case.shape_n
    desaturation.terminal = True
    desaturation.direction = -1

    def first_layer(t, y):
        return y[1] / case.E - case.qcrit
    first_layer.terminal = True
    first_layer.direction = -1
    return solve_ivp(lambda t, y: q2.rhs(case, forcing, t, y), (0.0, case.end - case.start),
    ↪  (case.m0, case.s_init, 0.0), method='Radau', rtol=numerics['rtol'], atol=ATOL,
    ↪  max_step=numerics['max_step_s'], dense_output=True, events=(zero, desaturation,
    ↪  first_layer))
```

```
def run_member(q0=None, chi_m_inv=None, forcing=None, *, start_s=None, horizon_s=None,
↪  mode='terminal_first_hit', numerics=None, trajectory_points=5, case=None):
    started = time.perf_counter()
    q2 = load_frozen_q2()
    cfg = frozen_config()
    if case is None:
        case = build_case(q0, chi_m_inv, start_s, horizon_s)
    else:
        if any((x is not None for x in (q0, chi_m_inv, start_s, horizon_s))):
            raise ValueError('Pass a case or conditional input arguments, not both')
        expected = build_case(case.initial_stress_ratio, case.chi_m_inv, case.start, case.end
        ↪  - case.start)
        if asdict(case) != asdict(expected):
            raise ValueError('Case contains an unfrozen material/closure/mode change')
    if mode not in ('terminal_first_hit', 'frozen_full_window'):
        raise ValueError('Unknown numerical calling mode')
    setting = dict(cfg['numerics'])
    if numerics is not None:
        if set(numerics) - {'rtol', 'max_step_s', 'output_step_s'}:
            raise ValueError('Only documented numerical settings may change')
        setting.update(numerics)
    if any((not math.isfinite(float(setting[k])) or float(setting[k]) <= 0 for k in ('rtol',
    ↪  'max_step_s', 'output_step_s'))):
        raise ValueError('Numerical settings must be finite and positive')
    forcing = load_default_forcing() if forcing is None else forcing
    admission = q2.initial_classification(case)
    result = {
        'status': 'unknown',
        'tau_s': None,
        'qdot_cross_s_inv': None,
        'first_hit_valid': False,
        'whole_window_valid': None,
        'initial_classification': admission,
        'case': asdict(case),
        'material_and_closure_identity': 'exact_pinned_Q2_reference_no_fit',
        'chi_identity': 'unidentified_conditional_input_transfer_design_not_local_estimate',
        'forcing_identity': getattr(forcing, 'identity', 'caller_supplied_speed_protocol'),
        'forcing_metadata': getattr(forcing, 'metadata', {}),
        'mode': mode,
        'numerics': {**setting, 'method': 'Radau', 'atol': list(ATOL)},
        'solver_status': None,
        'cut_diagnostics': None,
        'hit_diagnostics': None,
        'samples': [],
        'failure': None,
        'dense_event_audit': None,
        'recorded_solver_hit_tau_s': None,
        'horizon_s': case.end - case.start,
        'probability_identity': 'one conditional member; no probability assigned here',
    }
    if admission['initial_status'] in ('initial_first_layer_boundary',
    ↪  'initial_positive_thickness_layer'):
        result.update(status='initial', solver_status=admission['initial_status'],
        ↪  initial_boundary_time_s=0.0, initial_is_not_future_prediction=True)
```

```
elif admission['initial_status'] != 'admissible_unreached_initial_state':
    result.update(solver_status=admission['initial_status'], failure='Initial
    ↪  closure/saturation limit or illegal state: not a no-hit outcome')
else:
    try:
        if mode == 'frozen_full_window':
            _, summary, sol = q2.run_case(case, forcing, setting, dense=False,
            ↪  keep_solution=True)
            solver_status = summary['status']
            result['frozen_summary'] = summary
        else:
            sol = _terminal_first_hit(q2, case, forcing, setting)
            solver_status = 'completed_conditional_window'
            if len(sol.t_events[0]):
                solver_status = 'mean_stress_zero_closure_limit'
            elif len(sol.t_events[1]):
                solver_status = 'saturation_hypothesis_boundary'
            elif len(sol.t_events[2]):
                solver_status = 'first_layer_boundary_terminated'
            if not sol.success:
                solver_status = 'numerical_failure'
        result['solver_status'] = solver_status
        result['solver_evaluations'] = {'nfev': int(sol.nfev), 'njev': int(sol.njev),
        ↪  'nlu': int(sol.nlu)}
        end = float(sol.t[-1])
        hits = sol.t_events[2]
        exits = [float(t) for events in sol.t_events[:2] for t in events]
        earliest_exit = min(exits) if exits else math.inf
        hit = float(hits[0]) if len(hits) else None
        result['recorded_solver_hit_tau_s'] = hit
        whole_valid = bool(sol.success and (not exits) and math.isclose(end, case.end -
        ↪  case.start, rel_tol=0, abs_tol=1e-09))
        result['whole_window_valid'] = whole_valid
        audit = audit_dense_first_hit(sol, stress_scale_E=case.E, qcrit=case.qcrit,
        ↪  rtol=setting['rtol'], known_hit_time=hit)
        result['dense_event_audit'] = audit
        recovered = audit['recovered_crossing_tau_s']
        if recovered is not None and (hit is None or recovered < hit):
            hit = recovered
            result['event_detection_identity'] =
            ↪  'Q3_Radau_dense_interpolant_recovered_crossing'
        elif hit is not None:
            result['event_detection_identity'] =
            ↪  'original_Q2_directional_event_with_dense_prefix_audit'
        unresolved = 'unknown' in audit['audit_status']
        if hit is None and audit['audit_status'] == 'contact_candidate':
            unresolved = True
        if hit is not None:
            early_contacts = [c for c in audit['contact_candidates'] if c['location'] ==
            ↪  'interior_stationary' and c['elapsed_s'] < hit - 1e-09]
            if early_contacts:
                unresolved = True
                result['earlier_contact_candidate'] = True
            if recovered is None:
```

```
                    at_hit = _cut_diagnostics(q2, case, forcing, hit, sol.sol(hit))
                    last_width = max(1e-12, float(sol.t[-1] - sol.t[-2]))
                    slope_scale = audit['max_contact_tolerance_q'] / last_width
                    if abs(at_hit['qdot_s_inv']) <= slope_scale:
                        unresolved = True
                        result['flat_recorded_event_contact_candidate'] = True
            if unresolved:
                result.update(status='unknown', first_hit_valid=False, failure='Dense
                ↪  numerical path has unresolved near-threshold contact or audit gap; not
                ↪  no_hit')
            elif sol.success and hit is not None and (hit <= earliest_exit + 1e-10):
                result.update(status='hit', tau_s=hit, first_hit_valid=True)
                hit_diag = _cut_diagnostics(q2, case, forcing, hit, sol.sol(hit))
                result['hit_diagnostics'] = hit_diag
                result['qdot_cross_s_inv'] = hit_diag['qdot_s_inv']
                if mode == 'terminal_first_hit' and end < case.end - case.start - 1e-09:
                    result['whole_window_valid'] = None
                    result['post_hit_window'] = 'not_integrated_for_stopped_first_hit_object'
                elif not whole_valid:
                    result['post_hit_window'] =
                    ↪  'later_exit_does_not_erase_earlier_valid_first_hit'
            elif whole_valid:
                result.update(status='no_hit', first_hit_valid=False)
            else:
                result.update(status='unknown', failure='Exited/failed before a valid
                ↪  recorded first hit')
            result['cut_diagnostics'] = _cut_diagnostics(q2, case, forcing, end, sol.sol(end))
            if trajectory_points:
                count = max(2, int(trajectory_points))
                ts = np.unique(np.r_[np.linspace(0, end, count), [] if hit is None else
                ↪  [hit]])
                result['samples'] = [_cut_diagnostics(q2, case, forcing, t, sol.sol(t)) for t
                ↪  in ts]
        except (ValueError, RuntimeError, FloatingPointError, OverflowError) as error:
            result.update(status='unknown',
            ↪  solver_status='input_numerical_or_physical_exception',
            ↪  failure=f'{type(error).__name__}: {error}', whole_window_valid=False)
    result['elapsed_wall_s'] = time.perf_counter() - started
    return result

def convergence_check(q0=0.5, chi_m_inv=0.02, forcing=None, *, start_s=None, horizon_s=None):
    cfg = frozen_config()
    base = dict(cfg['numerics'])
    tight = {**base, 'rtol': base['rtol'] / 10, 'max_step_s': base['max_step_s'] / 2}
    a = run_member(q0, chi_m_inv, forcing, start_s=start_s, horizon_s=horizon_s,
    ↪  numerics=base, trajectory_points=0)
    b = run_member(q0, chi_m_inv, forcing, start_s=start_s, horizon_s=horizon_s,
    ↪  numerics=tight, trajectory_points=0)
    delta = abs(a['tau_s'] - b['tau_s']) if a['tau_s'] is not None and b['tau_s'] is not None
    ↪  else None
    return {
        'base': a,
        'tight': b,
        'status_agrees': a['status'] == b['status'],
```

```
        'absolute_tau_difference_s': delta,
        'identity': 'same_frozen_rhs_and_atol; rtol_tightened_10x_and_max_step_halved',
    }

def cdf(x, family):
    u = np.clip((np.asarray(x) + 1.5) / 3, 0, 1)
    if family == 'uniform_support':
        return u
    if family == 'lower_mode_triangular':
        return 2 * u - u * u
    if family == 'nominal_triangular':
        return np.where(u <= 0.5, 2 * u * u, 1 - 2 * (1 - u) ** 2)
    raise ValueError(family)

class BaselineModel:

    def __init__(self, rows):
        self.rows = sorted(rows, key=lambda r: r['d'])
        self.x = np.array([r['d'] for r in self.rows])
        self.status = np.array([r['status'] for r in self.rows])
        self.tau = np.array([np.nan if r['tau_s'] is None else r['tau_s'] for r in self.rows])
        hits = self.status == 'hit'
        self.hx = self.x[hits]
        self.ht = self.tau[hits]
        if np.any(~np.isin(self.status, ['hit', 'no_hit'])):
            raise ValueError('unknown/initial blocks baseline')
        if not len(self.hx):
            raise ValueError('no hit interpolation support')
        self.maximum_sample_reversal_s = float(max(0, np.max(np.diff(self.ht))))
        if np.any(self.status[np.flatnonzero(hits)[0]:] != 'hit'):
            raise ValueError('nonmonotone classes')
        self.interp = PchipInterpolator(self.hx, self.ht, extrapolate=False)
        self.boundary = None if np.all(hits) else [float(self.x[self.x < self.hx[0]][-1]),
        ↪  float(self.hx[0])]

    @classmethod
    def from_json(cls, path):
        return cls(json.loads(Path(path).read_text())['rows'])

    def predict(self, d):
        d = np.asarray(d, dtype=float)
        s = np.full(d.shape, 'unknown', dtype='<U8')
        t = np.full(d.shape, np.nan)
        valid = np.isfinite(d) & (d >= max(-1.5, self.x[0])) & (d <= min(1.5, self.x[-1]))
        if self.boundary is None:
            hit = valid
        else:
            s[valid & (d <= self.boundary[0])] = 'no_hit'
            hit = valid & (d >= self.boundary[1])
        s[hit] = 'hit'
        t[hit] = self.interp(d[hit])
        bad = (s == 'hit') & ~np.isfinite(t)
        s[bad] = 'unknown'
        t[bad] = np.nan
```

```
    return (s, t)

def propagated(self, edges):
    if self.x[0] > -1.5 or self.x[-1] < 1.5:
        raise ValueError('probability propagation requires complete offset support
        ↪  [-1.5,1.5]; partial supplied map is not a probability model')
    edges = np.asarray(edges, dtype=float)
    if edges.ndim != 1 or len(edges) < 2 or (not np.isfinite(edges).all()) or (edges[0]
    ↪  != 0) or (not np.all(np.diff(edges) > 0)) or (edges[-1] < np.max(self.ht)):
        raise ValueError('probability time grid must start at0, increase, and include
        ↪  every finite mapped first-hit time')
    out = []
    for f in FAMILIES:
        values = []
        lower = []
        upper = []
        for t in edges:
            total = 0.0
            lo_mass = 0.0
            hi_mass = 0.0
            for left, right, yl, yr in zip(self.hx[:-1], self.hx[1:], self.ht[:-1],
            ↪  self.ht[1:]):
                mass = float(cdf(right, f) - cdf(left, f))
                if max(yl, yr) <= t:
                    total += mass
                    lo_mass += mass
                    hi_mass += mass
                elif min(yl, yr) <= t:
                    root = brentq(lambda z: float(self.interp(z)) - t, left, right,
                    ↪  xtol=1e-12)
                    total += float(cdf(right, f) - cdf(root, f)) if yr < yl else
                    ↪  float(cdf(root, f) - cdf(left, f))
                    hi_mass += mass
            values.append(total)
            lower.append(lo_mass)
            upper.append(hi_mass)
        values = np.array(values)
        mass = np.diff(values)
        none = 0 if self.boundary is None else float(cdf(self.boundary[0], f))
        unresolved = 0 if self.boundary is None else float(cdf(self.boundary[1], f) -
        ↪  cdf(self.boundary[0], f))
        out.append({
            'family': f,
            'finite_mass': mass.tolist(),
            'no_hit_mass': none,
            'boundary_unresolved_mass': unresolved,
            'cdf': values.tolist(),
            'sample_segment_cdf_lower': lower,
            'sample_segment_cdf_upper': upper,
        })
    for row in out:
        total = sum(row['finite_mass']) + row['no_hit_mass'] +
        ↪  row['boundary_unresolved_mass']
        if not math.isclose(total, 1.0, rel_tol=0, abs_tol=1e-08):
```

```
                raise ValueError('probability mass does not cover complete declared input
                ↪  support')
        return out

def shortest_set(edges, masses, no_hit, target=0.95):
    edges = np.asarray(edges)
    a = np.atleast_2d(masses)
    n = np.asarray(no_hit)
    K = a.shape[1]
    p = np.c_[np.zeros(len(a)), np.cumsum(a, axis=1)]
    best = None
    operations = 0

    def offer(i, j, inc):
        nonlocal best
        value = (np.zeros(len(a)) if i is None else p[:, j] - p[:, i]) + (n if inc else 0)
        if value.min() + 1e-12 < target:
            return
        spec = {
            'finite_interval_s': None if i is None else [float(edges[i]), float(edges[j])],
            'finite_width_s': 0.0 if i is None else float(edges[j] - edges[i]),
            'includes_no_hit_by_horizon': inc,
            'family_masses': value.tolist(),
            'worst_mass': float(value.min()),
        }
        strict_width = Decimal(0) if i is None else Decimal(str(edges[j])) -
        ↪  Decimal(str(edges[i]))
        key = (strict_width, -round(float(value.min()), 12), int(inc), -1 if i is None else i)
        if best is None or key < best[0]:
            best = (key, spec)
    for inc in (False, True):
        if inc:
            offer(None, None, True)
        j = 0
        for i in range(K):
            j = max(j, i + 1)
            while j <= K and np.min(p[:, j] - p[:, i] + (n if inc else 0)) < target - 1e-12:
                j += 1
                operations += len(a)
            if j > K:
                break
            offer(i, j, inc)
            operations += len(a)
    return {
        'set': None if best is None else best[1],
        'operations_bound_counter': operations,
        'complexity': 'O(RK) nonnegative masses; two category modes',
    }

def central_set(edges, row, alpha=0.05):
    cumulative = np.r_[0, np.cumsum(row['finite_mass'])]
    lo = int(np.searchsorted(cumulative, alpha / 2, side='left'))
    hi = int(np.searchsorted(cumulative, 1 - alpha / 2, side='left'))
    K = len(edges) - 1
```

```
    i = max(0, min(K - 1, lo - 1))
    inc = hi > K
    j = K if inc else min(K, hi)
    mass = float(sum(row['finite_mass'][i:j]) + (row['no_hit_mass'] if inc else 0))
    return {
        'finite_interval_s': [float(edges[i]), float(edges[j])],
        'finite_width_s': float(edges[j] - edges[i]),
        'includes_no_hit_by_horizon': bool(inc),
        'nominal_mass': mass,
    }

def build(regime, chi):
    original = load_default_forcing()
    progress = OUT / f'progress_{regime}.json'
    records = {r['d'].hex(): r for r in json.loads(progress.read_text())} if
    ↪  progress.exists() else {}
    start = time.perf_counter()

    def query(d, role):
        d = float(d)
        key = d.hex()
        if key not in records:
            p = VelocityForcing(original.t, np.maximum(0, original.u + d),
            ↪  identity='Q3_v2_coherent_offset', metadata={'offset_m_s': d})
            tick = time.perf_counter()
            r = run_member(0.5, chi, p, start_s=27, horizon_s=73, trajectory_points=0)
            records[key] = {
                'd': d,
                'status': r['status'],
                'tau_s': r['tau_s'],
                'role': role,
                'wall_s': time.perf_counter() - tick,
                'path_sha256': p.path_sha256,
                'solver_status': r['solver_status'],
                'dense_event_audit': r['dense_event_audit'],
            }
            save(OUT / f'progress_{regime}.json', list(records.values()))
        return records[key]
    for x in np.linspace(-1.5, 1.5, 33):
        query(x, 'initial_monotonicity_grid')
    for x in np.linspace(-1.5, 1.5, 65)[1::2]:
        query(x, 'midpoint_monotonicity_check')
    rows = sorted(records.values(), key=lambda r: r['d'])
    hit = [r for r in rows if r['status'] == 'hit']
    non = [r for r in rows if r['status'] == 'no_hit']
    if non and hit:
        lo = max((r['d'] for r in non))
        hi = min((r['d'] for r in hit))
        while hi - lo > 1e-07:
            mid = (lo + hi) / 2
            r = query(mid, 'class_boundary_bisection')
            if r['status'] == 'hit':
                hi = mid
            elif r['status'] == 'no_hit':
```

```
                lo = mid
            else:
                break
    for _ in range(40):
        rows = sorted(records.values(), key=lambda r: r['d'])
        h = [r for r in rows if r['status'] == 'hit']
        x = np.array([r['d'] for r in h])
        y = np.array([r['tau_s'] for r in h])
        p = PchipInterpolator(x, y)
        mids = (x[:-1] + x[1:]) / 2
        score = np.abs(p(mids) - (y[:-1] + y[1:]) / 2)
        score[np.diff(x) < 0.0001] = -1
        j = int(np.argmax(score))
        if score[j] < 1e-07:
            break
        query(mids[j], 'adaptive_curvature_split')
    rows = sorted(records.values(), key=lambda r: r['d'])
    model = BaselineModel(rows)
    save(OUT / f'baseline_model_{regime}.json', {
        'regime': regime,
        'chi': chi,
        'q0': 0.5,
        'rows': rows,
        'boundary': model.boundary,
        'maximum_sample_reversal_s': model.maximum_sample_reversal_s,
        'global_monotonicity_proved': False,
        'wall_s': time.perf_counter() - start,
    })
    return model

def edges_from(cfg, horizon):
    early = np.arange(0, cfg['early_end_s'] + cfg['early_step_s'] / 2, cfg['early_step_s'])
    late = np.arange(cfg['late_start_s'], horizon + cfg['late_step_s'] / 2,
    ↪  cfg['late_step_s'])
    edges = np.unique(np.round(np.r_[early, cfg['middle_edges_s'], late, horizon], 9))
    assert edges[0] == 0 and edges[-1] == horizon and np.all(np.diff(edges) > 0)
    return edges
CONFIG = dict(version='Q3-GP-v2.0', seed=20261030, budgets=[8, 16, 32], q0=0.5,
↪  conditions={'strong': 0.02, 'near': 0.0002265378784906}, support=[-1.5, 1.5],
↪  horizon_s=73.0, pool_count=601, min_query_distance=0.012, test_count=600,
↪  test_protocol='all 600 midpoint cells; training on 601 edges; min distance .0025 m/s',
↪  boundary_radius=0.08, alpha=0.05, delta=0.05, classifier_length_scale=0.45,
↪  regression_length_scale=0.45, classifier_fallback_probability=[0.35, 0.65],
↪  regression_fallback_std_s={'strong': 0.0005, 'near': 0.05}, final_test_use='once after
↪  freeze; no retuning; numerical scenario test, not iid coverage certificate',
↪  probability_identity='declared input families conditional on fixed Q2, not natural
↪  disaster probability')

class Oracle:

    def __init__(self, out):
        self.original = load_default_forcing()
        self.cache = {}
        self.rows = []
```

```
        self.out = out
        if (out / 'q2_queries.jsonl').exists():
            self.rows = [json.loads(s) for s in (out /
            ↪  'q2_queries.jsonl').read_text().splitlines()]
            self.cache = {(r['regime'], round(r['d'], 12)): r for r in self.rows}

    def query(self, regime, d, role):
        key = (regime, round(float(d), 12))
        if key in self.cache:
            return self.cache[key]
        f = VelocityForcing(self.original.t, np.maximum(0, self.original.u + d),
        ↪  identity='fixed_Q2_coherent_offset_conditional', metadata={'offset_m_s':
        ↪  float(d)})
        t = time.perf_counter()
        r = run_member(0.5, CONFIG['conditions'][regime], f, start_s=27.0, horizon_s=73.0,
        ↪  trajectory_points=0)
        row = dict(regime=regime, d=float(d), role=role, status=r['status'],
        ↪  tau_s=r['tau_s'], wall_s=time.perf_counter() - t,
        ↪  input_path_sha256=f.path_sha256, solver_status=r['solver_status'],
        ↪  first_hit_valid=r['first_hit_valid'], whole_window_valid=r['whole_window_valid'],
        ↪  qdot_cross_s_inv=r['qdot_cross_s_inv'], failure=r['failure'],
        ↪  dense_event_audit=r['dense_event_audit'])
        self.rows.append(row)
        self.cache[key] = row
        with (self.out / 'q2_queries.jsonl').open('a') as h:
            h.write(json.dumps(row, allow_nan=False) + '\n')
        return row

class TwoPartGP:

    def fit(self, rows):
        self.rows = rows
        self.x = np.array([r['d'] for r in rows])
        self.lo = min(self.x)
        self.hi = max(self.x)
        known = [r for r in rows if r['status'] in ('hit', 'no_hit')]
        if not known:
            raise ValueError('no known future outcomes')
        X = np.array([r['d'] for r in known])[:, None]
        y = np.array([r['status'] == 'hit' for r in known], int)
        self.single = int(y[0]) if len(np.unique(y)) == 1 else None
        self.clf = None
        if self.single is None:
            self.clf = GaussianProcessClassifier(1.0 * Matern(0.45, nu=1.5), optimizer=None,
            ↪  random_state=CONFIG['seed'], max_iter_predict=100).fit(X, y)
        hits = [r for r in known if r['status'] == 'hit']
        self.reg = None
        if len(hits) >= 2:
            self.reg = GaussianProcessRegressor(ConstantKernel(1.0, (0.01, 100)) *
            ↪  Matern(0.45, (0.03, 3.0), nu=2.5), alpha=1e-08, normalize_y=True,
            ↪  n_restarts_optimizer=0, random_state=CONFIG['seed'])
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter('always', ConvergenceWarning)
                self.reg.fit(np.array([r['d'] for r in hits])[:, None], [r['tau_s'] for r in
                ↪  hits])
```

```
            self.warnings = [str(w.message) for w in caught]
        else:
            self.warnings = []
        return self

    def predict(self, d):
        d = np.asarray(d, float)
        X = d[:, None]
        p = np.full(len(d), float(self.single)) if self.clf is None else
        ↪  self.clf.predict_proba(X)[:, 1]
        mu, std = (np.full(len(d), np.nan), np.full(len(d), np.inf)) if self.reg is None else
        ↪  self.reg.predict(X, return_std=True)
        status = np.where(p >= 0.5, 'hit', 'no_hit').astype(object)
        bad = (d < self.lo) | (d > self.hi) | ~np.isfinite(d)
        bad |= (status == 'hit') & (~np.isfinite(mu) | (mu <= 0) | (mu > 73))
        status[bad] = 'unknown'
        mu = np.where(status == 'hit', mu, np.nan)
        return (status, mu, p, std)

def train(oracle, regime, method, out):
    construction_started = time.perf_counter()
    pool = np.linspace(-1.5, 1.5, CONFIG['pool_count'])
    indices = np.rint(np.linspace(0, 600, 8)).astype(int).tolist()
    rows = [oracle.query(regime, pool[i], f'{method}_train') for i in indices]
    snapshots = []
    started = time.perf_counter()
    qtime = sum((r['wall_s'] for r in rows))
    fit_time = 0.0
    acq_time = 0.0
    for n in CONFIG['budgets']:
        while len(rows) < n:
            t = time.perf_counter()
            gp = TwoPartGP().fit(rows)
            fit_time += time.perf_counter() - t
            t = time.perf_counter()
            dist = np.min(abs(pool[:, None] - np.array([r['d'] for r in rows])), axis=1)
            if method == 'active':
                _, _, p, s = gp.predict(pool)
                finite = np.isfinite(s)
                scale = np.max(s[finite]) if finite.any() else 1.0
                uncertainty = np.where(finite, s / max(scale, 1e-15), 0)
                score = (4 * np.exp(-((p - 0.5) / 0.15) ** 2) + uncertainty + 0.05) *
                ↪  np.minimum(dist / 0.06, 1)
                score[dist < CONFIG['min_query_distance']] = -np.inf
            else:
                score = dist
            i = int(np.argmax(score))
            acq_time += time.perf_counter() - t
            row = oracle.query(regime, pool[i], f'{method}_train')
            qtime += row['wall_s']
            rows.append(row)
        t = time.perf_counter()
        gp = TwoPartGP().fit(rows)
        fit_time += time.perf_counter() - t
```

```
        snapshots.append((n, gp, dict(regime=regime, method=method, budget=n,
        ↪  q2_nominal_calls=n, q2_nominal_wall_s=qtime, fit_wall_s=fit_time,
        ↪  acquisition_wall_s=acq_time, total_nominal_training_wall_s=time.perf_counter() -
        ↪  construction_started + sum((r['wall_s'] for r in rows if r['role'] !=
        ↪  f'{method}_train')), actual_elapsed_training_wall_s=time.perf_counter() -
        ↪  construction_started, single_class=gp.single,
        ↪  regression_kernel=str(gp.reg.kernel_) if gp.reg else None, warnings=gp.warnings)))
    save(out / f'{regime}_{method}_training.json', rows)
    return snapshots

def metrics(truth, status, tau, mask):
    mask = np.asarray(mask, bool)
    hit = truth.status.to_numpy() == 'hit'
    known = truth.status.isin(['hit', 'no_hit']).to_numpy()
    both = mask & hit & (status == 'hit') & np.isfinite(tau)
    err = tau[both] - truth.tau_s.to_numpy()[both]
    k = mask & known
    return dict(n=int(mask.sum()), n_true_hit=int((mask & hit).sum()),
    ↪  n_both_hit=int(both.sum()), mae_s=float(np.mean(abs(err))) if len(err) else None,
    ↪  max_abs_error_s=float(max(abs(err))) if len(err) else None,
    ↪  rmse_s=float(np.sqrt(np.mean(err ** 2))) if len(err) else None,
    ↪  false_negative=int((mask & hit & (status != 'hit')).sum()), false_positive=int((mask
    ↪  & (truth.status.to_numpy() == 'no_hit') & (status == 'hit')).sum()),
    ↪  unknown_predictions=int((mask & (status == 'unknown')).sum()),
    ↪  class_error_rate=float(np.mean(status[k] != truth.status.to_numpy()[k])) if k.any()
    ↪  else None)

def distribution_metrics(truth, status, tau):
    d = truth.d.to_numpy()
    z = (d + 1.5) / 3
    weights = {
        'nominal_triangular': np.where(z <= 0.5, 4 * z, 4 * (1 - z)),
        'uniform_support': np.ones(len(d)),
        'lower_mode_triangular': 2 * (1 - z),
    }
    edges = np.r_[np.linspace(0, 1.5, 301), np.linspace(60, 73, 261)]
    rows = []
    for family, w in weights.items():
        w = w / w.sum()
        truehit = truth.status.to_numpy() == 'hit'
        predhit = status == 'hit'
        actual = np.array([w[truehit & (truth.tau_s.to_numpy() <= t)].sum() for t in edges])
        pred = np.array([w[predhit & (tau <= t)].sum() for t in edges])

        def end(s, t):
            sel = s == 'hit'
            order = np.argsort(t[sel])
            times = t[sel][order]
            m = w[sel][order]
            cumulative = np.cumsum(m)
            return [float(times[min(len(times) - 1, np.searchsorted(cumulative, q))]) if
            ↪  len(times) and cumulative[-1] >= q - 1e-14 else None for q in [0.025, 0.5,
            ↪  0.975]]
        a = end(truth.status.to_numpy(), truth.tau_s.to_numpy())
```

```
        b = end(status, tau)
        rows.append(dict(family=family, no_hit_mass_true=float(w[truth.status.to_numpy() ==
        ↪  'no_hit'].sum()), no_hit_mass_pred=float(w[status == 'no_hit'].sum()),
        ↪  unknown_mass_pred=float(w[status == 'unknown'].sum()),
        ↪  no_hit_mass_abs_error=float(abs(w[status == 'no_hit'].sum() -
        ↪  w[truth.status.to_numpy() == 'no_hit'].sum())),
        ↪  cdf_sup_error=float(max(abs(actual - pred))), finite_subcdf_quantiles_true=a,
        ↪  finite_subcdf_quantiles_pred=b, quantile_endpoint_abs_error_s=[abs(x - y) if x is
        ↪  not None and y is not None else None for x, y in zip(a, b)]))
    return rows

def finite(v, name, nonnegative=False):
    if isinstance(v, bool) or not isinstance(v, (float, int)) or (not math.isfinite(v)):
        raise ValueError(f'{name}: finite number required')
    if nonnegative and v < 0:
        raise ValueError(f'{name}: must be >=0')
    return float(v)

def text(v, name):
    if not isinstance(v, str) or not v.strip():
        raise ValueError(f'{name}: nonempty text required')
    return v

def connect(q3, q4):
    for document in (q3, q4):
        if document.get('time_unit') != 's' or document.get('distance_unit') != 'm':
            raise ValueError('explicit SI time/distance units required')
        text(document.get('time_origin_id'), 'time_origin_id')
    if q3['time_origin_id'] != q4['time_origin_id']:
        raise ValueError('time origins differ')
    entries = {}
    for x in q3['entries']:
        sid = text(x.get('scenario_id'), 'scenario_id')
        if sid in entries:
            raise ValueError('duplicate Q3 scenario')
        for k in ('scenario_hash', 'entry_id', 'entry_state_hash', 'bridge_evidence_id'):
            text(x.get(k), k)
        if x.get('stage') != 'propagation_entry':
            raise ValueError('Q3 must provide explicit propagation_entry, not local
            ↪  threshold')
        if x.get('status') not in ('entry_reached', 'not_entered', 'unknown'):
            raise ValueError('invalid entry status')
        if x['status'] == 'entry_reached':
            finite(x.get('entry_time_s'), 'entry_time_s', True)
        elif x.get('entry_time_s') is not None:
            raise ValueError('unknown/not-entered entry must not carry invented time')
        entries[sid] = x
    out, seen = ([], set())
    for y in q4['arrivals']:
        sid = text(y.get('scenario_id'), 'scenario_id')
        target = text(y.get('target_id'), 'target_id')
        key = (sid, target)
        if key in seen:
            raise ValueError('duplicate scenario-target')
```

```
        seen.add(key)
        if sid not in entries:
            raise ValueError('unmatched Q4 scenario')
        x = entries[sid]
        for k in ('scenario_hash', 'entry_id', 'entry_state_hash'):
            if x[k] != y.get(k):
                raise ValueError(f'{k} differs')
        if y.get('stage') != 'target_arrival':
            raise ValueError('Q4 must label target_arrival stage')
        for k in ('terrain_version', 'propagation_version', 'distance_definition',
        ↪  'arrival_criterion'):
            text(y.get(k), k)
        distance = finite(y.get('distance_m'), 'distance_m', True)
        pub = finite(y.get('publication_time_s'), 'publication_time_s', True)
        act = finite(y.get('action_delay_s'), 'action_delay_s', True)
        if y.get('status') not in ('arrived', 'not_arrived', 'out_of_domain', 'failed',
        ↪  'unknown'):
            raise ValueError('invalid arrival status')
        if y['status'] == 'arrived' and x['status'] == 'not_entered':
            raise ValueError('arrived contradicts not_entered')
        arrival, lead = (None, None)
        if y['status'] == 'arrived':
            travel = finite(y.get('travel_time_s'), 'travel_time_s', True)
            if x['status'] == 'entry_reached':
                arrival = x['entry_time_s'] + travel
                lead = arrival - pub - act
        elif y.get('travel_time_s') is not None:
            raise ValueError('non-arrival must not carry a finite travel time')
        out.append({
            **y,
            'entry_status': x['status'],
            'entry_time_s': x.get('entry_time_s'),
            'bridge_evidence_id': x['bridge_evidence_id'],
            'distance_m': distance,
            'arrival_time_s': arrival,
            'effective_lead_s': lead,
            'positive_effective_lead': None if lead is None else lead > 0,
            'connection_status': 'computed_conditional' if lead is not None else
            ↪  'pending_or_nonarrival',
        })
    missing = sorted(set(entries) - {r['scenario_id'] for r in out})
    return {
        'time_origin_id': q3['time_origin_id'],
        'time_unit': 's',
        'distance_unit': 'm',
        'rows': out,
        'scenarios_without_q4': missing,
        'scope': 'conditional scenario calculation; no probability or warning radius
        ↪  inferred',
    }

def record_query(regime, d, role='numerical_test'):
    f0 = load_default_forcing()
    f = VelocityForcing(f0.t, np.maximum(0, f0.u + float(d)),
    ↪  identity='delivered_fixed_Q2_offset', metadata={'offset_m_s': float(d)})
```

```
    r = run_member(0.5, CONFIG['conditions'][regime], f, start_s=27.0, horizon_s=73.0,
    ↪  trajectory_points=0)
    return {
        'regime': regime,
        'd': float(d),
        'role': role,
        'status': r['status'],
        'tau_s': r['tau_s'],
        'qdot_cross_s_inv': r['qdot_cross_s_inv'],
        'wall_s': r['elapsed_wall_s'],
        'input_path_sha256': f.path_sha256,
        'solver_status': r['solver_status'],
        'failure': r['failure'],
        'whole_window_valid': r['whole_window_valid'],
    }

def grid_edges():
    return edges_from({
        'early_end_s': 1.5,
        'early_step_s': 0.005,
        'middle_edges_s': [2, 5, 10, 20, 30, 40, 50, 60],
        'late_start_s': 60.0,
        'late_step_s': 0.05,
    }, 73.0)

def categorical_masses(status, tau, d):
    edges = grid_edges()
    bins = []
    none = []
    unknown = []
    h = np.asarray(status) == 'hit'
    nn = np.asarray(status) == 'no_hit'
    uu = ~(h | nn)
    xedges = np.linspace(-1.5, 1.5, len(d) + 1)
    for family in FAMILIES:
        w = np.diff(cdf(xedges, family))
        b = np.histogram(np.asarray(tau)[h], bins=edges, weights=w[h])[0]
        bins.append(b)
        none.append(float(w[nn].sum()))
        unknown.append(float(w[uu].sum()))
    return (np.array(bins), np.array(none), unknown)

def common_probability_task(status, tau, d):
    start = time.perf_counter()
    a, n, u = categorical_masses(status, tau, d)
    edges = grid_edges()
    result = {
        'nominal_central': central_set(edges, {'finite_mass': a[0].tolist(), 'no_hit_mass':
        ↪  n[0]}),
        'nominal_shortest': shortest_set(edges, a[:1], n[:1]),
        'robust_shortest': shortest_set(edges, a, n),
        'cdf': np.cumsum(a, axis=1).tolist(),
        'no_hit_mass': n.tolist(),
        'unknown_mass': u,
```

```
    }
    return (result, time.perf_counter() - start)

def numerical_pipeline(results):
    global OUT
    OUT = results / 'baseline'
    OUT.mkdir()
    edges = grid_edges()
    baseline = {}
    basemodel = {}
    baselinecost = {}
    for name, chi in CONDITIONS.items():
        reg = {'Q2_strong_design': 'strong', 'Q2_near_boundary_comparison': 'near'}[name]
        t = time.perf_counter()
        model = build(name, chi)
        construction = time.perf_counter() - t
        t = time.perf_counter()
        prop = model.propagated(edges)
        a = np.array([r['finite_mass'] for r in prop])
        n = np.array([r['no_hit_mass'] for r in prop])
        baseline[name] = {
            'propagation': prop,
            'nominal_central': central_set(edges, prop[0]),
            'nominal_shortest': shortest_set(edges, a[:1], n[:1]),
            'robust_shortest': shortest_set(edges, a, n),
            'sample_count': len(model.rows),
            'boundary': model.boundary,
            'maximum_sample_reversal_s': model.maximum_sample_reversal_s,
        }
        integralwall = time.perf_counter() - t
        baselinecost[reg] = {
            'construction_wall_s': construction,
            'continuous_integral_set_wall_s': integralwall,
            'q2_calls': len(model.rows),
        }
        basemodel[reg] = model
        chosen = baseline[name]['robust_shortest']['set']
        candidate = next((x for x in FINAL['candidates'] if x['condition'] == reg))
        if not np.allclose(chosen['finite_interval_s'], candidate['finite_interval_s'],
        ↪  atol=1e-12, rtol=0) or chosen['includes_no_hit_by_horizon'] !=
        ↪  candidate['includes_no_hit']:
            raise RuntimeError('frozen candidate differs from recomputed baseline; final
            ↪  validation blocked')
    save(results / 'baseline_results.json', {'edges_s': edges.tolist(), 'conditions':
    ↪  baseline})
    gpout = results / 'gp'
    gpout.mkdir()
    oracle = Oracle(gpout)
    models = {}
    training = []
    nested = {}
    for reg in CONFIG['conditions']:
        nested[reg] = {}
        for method in ['passive', 'active']:
```

```
        models[reg, method] = train(oracle, reg, method, gpout)
        nested[reg][method] = models[reg, method][-1][1].rows
        training.extend((x[2] for x in models[reg, method]))
        print('training', reg, method, 'complete', flush=True)
save(results / 'gp_training.json', nested)
save(results / 'gp_training_summary.json', training)
save(results / 'FROZEN_GP_BEFORE_DIAGNOSTIC.json', {
    'utc': stamp(),
    'analysis_sha256': sha(__file__),
    'config': CONFIG,
    'training_sha256': sha(results / 'gp_training.json'),
    'test_identity': '600 fixed cell midpoints; not binomial certification',
})
alltruth = []
allpred = []
allmetrics = []
basechecks = {}
distributions = []
costs = []
setcomparisons = {}
for reg in CONFIG['conditions']:
    d = -1.5 + (np.arange(600) + 0.5) * 3 / 600
    tt = time.perf_counter()
    truth = pd.DataFrame([oracle.query(reg, x, 'fixed_midpoint_diagnostic') for x in d])
    directwall = time.perf_counter() - tt
    truth['regime'] = reg
    alltruth.append(truth.drop(columns=['dense_event_audit'], errors='ignore'))
    trueprob, probwall = common_probability_task(truth.status.to_numpy(),
    ↪  truth.tau_s.to_numpy(), d)
    setcomparisons[reg] = {'direct': trueprob}
    costs.append({
        'regime': reg,
        'method': 'Direct Q2',
        'budget': 600,
        'mode': 'direct',
        'construction_wall_s': 0.0,
        'query_wall_s': directwall,
        'probability_set_wall_s': probwall,
        'first_use_total_wall_s': directwall + probwall,
        'repeat_query_total_wall_s': directwall + probwall,
        'construction_q2_calls': 0,
        'fallback_q2_calls': 600,
    })
    t = time.perf_counter()
    bs, bt = basemodel[reg].predict(d)
    querywall = time.perf_counter() - t
    basechecks[reg] = {'all': metrics(truth, bs, bt, np.ones(600, bool))}
    dist = np.min(abs(d[:, None] - basemodel[reg].x), axis=1)
    basechecks[reg]['separated_002'] = metrics(truth, bs, bt, dist >= 0.002)
    basechecks[reg]['exact_overlap_count'] = int((dist < 1e-12).sum())
    basechecks[reg]['within_002_count'] = int((dist < 0.002).sum())
    bp, bpwall = common_probability_task(bs, bt, d)
    setcomparisons[reg]['baseline'] = bp
    bcost = baselinecost[reg]
```

```
costs.append({
    'regime': reg,
    'method': '1D baseline',
    'budget': bcost['q2_calls'],
    'mode': 'interpolation',
    'construction_wall_s': bcost['construction_wall_s'],
    'query_wall_s': querywall,
    'probability_set_wall_s': bpwall,
    'first_use_total_wall_s': bcost['construction_wall_s'] + querywall + bpwall,
    'repeat_query_total_wall_s': querywall + bpwall,
    'construction_q2_calls': bcost['q2_calls'],
    'fallback_q2_calls': 0,
    'continuous_integral_extra_wall_s': bcost['continuous_integral_set_wall_s'],
    'primary_continuous_first_use_total_wall_s': bcost['construction_wall_s'] +
    ↪  bcost['continuous_integral_set_wall_s'],
    'primary_continuous_repeat_use_wall_s': bcost['continuous_integral_set_wall_s'],
})
for method in ['passive', 'active']:
    for n, gp, timing in models[reg, method]:
        t = time.perf_counter()
        status, tau, p, std = gp.predict(d)
        querywall = time.perf_counter() - t
        trainx = np.array([r['d'] for r in gp.rows])
        minimum = float(np.min(abs(d[:, None] - trainx)))
        if minimum < 0.002 - 1e-12:
            raise RuntimeError('GP near-duplicate split guard failed')
        center = None if gp.single is not None else float(np.linspace(-1.5, 1.5,
        ↪  601)[np.argmin(abs(gp.predict(np.linspace(-1.5, 1.5, 601))[2] - 0.5))])
        boundary = np.zeros(600, bool) if center is None else abs(d - center) <= 0.08
        for mode in ['raw', 'fallback']:
            fallbackwall = 0.0
            calls = 0
            if mode == 'fallback':
                fallback_loop_started = time.perf_counter()
                mask = (status == 'unknown') | (p > 0.35) & (p < 0.65) | (status ==
                ↪  'hit') & (std > CONFIG['regression_fallback_std_s'][reg])
                for j in np.flatnonzero(mask):
                    r = record_query(reg, d[j], 'GP_fallback')
                    fallbackwall += r['wall_s']
                    calls += 1
                    status[j] = r['status']
                    tau[j] = np.nan if r['tau_s'] is None else r['tau_s']
                    with (results / 'fallback_queries.jsonl').open('a') as h:
                        h.write(json.dumps({**r, 'method': method, 'budget': n}) +
                        ↪  '\n')
                fallback_solver_wall_s = fallbackwall
                fallbackwall = time.perf_counter() - fallback_loop_started
            t = time.perf_counter()
            distribution = distribution_metrics(truth, status, tau)
            sets, pwall = common_probability_task(status, tau, d)
            pwalltotal = time.perf_counter() - t
            commonwall = pwall
            total = timing['total_nominal_training_wall_s'] + querywall +
            ↪  fallbackwall + commonwall
```

```
                fields = {
                    'regime': reg,
                    'method': method,
                    'budget': n,
                    'mode': mode,
                    'fallback_calls': calls,
                    'fallback_wall_s': fallbackwall,
                    'total_nominal_q2_calls': n + calls,
                    'total_nominal_wall_s': total,
                }
                for subset, mask in [('all', np.ones(600, bool)), ('boundary', boundary)]:
                    allmetrics.append({**fields, 'subset': subset, **metrics(truth,
                    ↪  status, tau, mask)})
                distributions.extend(({**fields, **r} for r in distribution))
                costs.append({
                    'regime': reg,
                    'method': method + ' GP',
                    'budget': n,
                    'mode': mode,
                    'construction_wall_s': timing['total_nominal_training_wall_s'],
                    'query_wall_s': querywall + fallbackwall,
                    'probability_set_wall_s': commonwall,
                    'first_use_total_wall_s': total,
                    'repeat_query_total_wall_s': querywall + fallbackwall + commonwall,
                    'construction_q2_calls': n,
                    'fallback_q2_calls': calls,
                    'metric_diagnostic_extra_wall_s': pwalltotal - commonwall,
                })
                if n == 32:
                    setcomparisons[reg][method + '_' + mode] = sets
                for j, x in enumerate(d):
                    allpred.append({
                        'regime': reg,
                        'method': method,
                        'budget': n,
                        'mode': mode,
                        'd': x,
                        'true_status': truth.status.iloc[j],
                        'true_tau_s': truth.tau_s.iloc[j],
                        'status': status[j],
                        'tau_s': tau[j],
                        'p_hit': p[j],
                        'std_s': std[j],
                        'boundary': bool(boundary[j]),
                    })
    print('numerical diagnostic', reg, 'complete', flush=True)
pd.concat(alltruth, ignore_index=True).to_csv(results / 'test_truth.csv', index=False)
pd.DataFrame(allpred).to_csv(results / 'gp_predictions.csv.gz', index=False)
pd.DataFrame(allmetrics).to_csv(results / 'gp_metrics.csv', index=False)
pd.DataFrame(costs).to_csv(results / 'cost_comparison.csv', index=False)
save(results / 'distribution_metrics.json', distributions)
save(results / 'baseline_test_metrics.json', basechecks)
save(results / 'set_comparison.json', setcomparisons)
return {
```

```
        'baseline_cost': baselinecost,
        'gp_unique_train_and_diagnostic_calls': len(oracle.rows),
        'baseline_calls': sum((x['q2_calls'] for x in baselinecost.values())),
        'gp_fallback_calls': sum((r['fallback_calls'] for r in allmetrics if r['subset'] ==
        ↪  'all')),
    }

def contains_candidate(row, candidate):
    if row['status'] == 'no_hit':
        return bool(candidate['includes_no_hit'])
    if row['status'] != 'hit' or row['tau_s'] is None:
        return False
    lo, hi = candidate['finite_interval_s']
    return lo <= float(row['tau_s']) <= hi

def validate_final(results, workers):
    frozen = json.loads((DATA / 'frozen_validation.json').read_text())
    children = np.random.SeedSequence(frozen['seed']).spawn(6)
    inputs = []
    k = 0
    for c in frozen['candidates']:
        for family in frozen['families']:
            rng = np.random.default_rng(children[k])
            k += 1
            n = frozen['n_per_candidate_family']
            if family == 'nominal_triangular':
                d = rng.triangular(-1.5, 0.0, 1.5, n)
            elif family == 'uniform_support':
                d = rng.uniform(-1.5, 1.5, n)
            else:
                d = rng.triangular(-1.5, -1.5, 1.5, n)
            inputs.extend(({'candidate': c['name'], 'regime': c['condition'], 'family':
            ↪  family, 'member': j, 'd': float(x)} for j, x in enumerate(d)))
    inputfile = results / 'validation_inputs.csv'
    pd.DataFrame(inputs).to_csv(inputfile, index=False)
    bindings = {
        'utc_before_labels': stamp(),
        'analysis_sha256': sha(__file__),
        'frozen_config_sha256': sha(DATA / 'frozen_validation.json'),
        'validation_inputs_sha256': sha(inputfile),
        'source_binding_sha256': sha(DATA / 'SOURCE_BINDING.json'),
        'forcing_sha256': sha(DATA / 'forcing.csv'),
        'q2_config_sha256': sha(DATA / 'q2_config.json'),
        'label_generator': 'delivered sol.py; equivalence to original Q2 required before
        ↪  certification release',
        'criterion': 'each one-sided CP lower>=.95, all fixed n denominator; any unknown or
        ↪  initial blocks its claim',
        'optional_stopping': False,
        'distribution_parameters': {
            'nominal_triangular': [-1.5, 0.0, 1.5],
            'uniform_support': [-1.5, 1.5],
            'lower_mode_triangular': [-1.5, -1.5, 1.5],
        },
        'input_construction': 'max(0,U_ref_node+d) applied to every provided node, then
        ↪  original piecewise linear interpolation; d constant over complete path',
```

```
}
save(results / 'FROZEN_VALIDATION_BINDINGS.json', bindings)
jobs = [(r['regime'], r['d']) for r in inputs]
start = time.perf_counter()
rows = []
with ProcessPoolExecutor(max_workers=workers, initializer=configure,
↪  initargs=(str(DATA),)) as pool:
    for i, row in enumerate(pool.map(validation_worker, jobs, chunksize=32)):
        r = {**inputs[i], **row}
        rows.append(r)
        with (results / 'validation_members.jsonl').open('a') as h:
            h.write(json.dumps(r, allow_nan=False) + '\n')
        if (i + 1) % 1024 == 0:
            print('random validation', i + 1, '/', len(jobs), flush=True)
pd.DataFrame(rows).to_csv(results / 'validation_members.csv.gz', index=False)
claims = []
for c in frozen['candidates']:
    for family in frozen['families']:
        selected = [r for r in rows if r['candidate'] == c['name'] and r['family'] ==
        ↪  family]
        n = len(selected)
        if n != frozen['n_per_candidate_family']:
            raise RuntimeError('incomplete fixed validation')
        accepted = sum((contains_candidate(r, c) for r in selected))
        unknown = sum((r['status'] == 'unknown' for r in selected))
        initial = sum((r['status'] == 'initial' for r in selected))
        lower = 0.0 if accepted == 0 else float(beta.ppf(frozen['per_claim_delta'],
        ↪  accepted, n - accepted + 1))
        claims.append({
            'regime': c['condition'],
            'candidate': c['name'],
            'family': family,
            'n': n,
            'k': accepted,
            'empirical_mass': accepted / n,
            'lower_bound': lower,
            'unknown': unknown,
            'initial': initial,
            'passed': bool(lower >= 1 - frozen['alpha'] and unknown == 0 and (initial ==
            ↪  0)),
        })
answer = {
    'version': FINAL['version'],
    'status': 'completed_fixed_n',
    'alpha': frozen['alpha'],
    'familywise_delta': frozen['familywise_delta'],
    'per_claim_delta': frozen['per_claim_delta'],
    'claims': claims,
    'all_passed': all((x['passed'] for x in claims)),
    'wall_s': time.perf_counter() - start,
    'bindings_sha256': sha(results / 'FROZEN_VALIDATION_BINDINGS.json'),
    'identity': 'new independent declared-input iid Q2 simulation validation; not
    ↪  natural-event coverage',
}
```

```
    save(results / 'validation.json', answer)
    return answer

def validation_worker(job):
    return record_query(job[0], job[1], 'fixed_n_random_validation')

def lead_outputs(results, validation):
    baseline = json.loads((results / 'baseline_results.json').read_text())['conditions']
    rows = []
    for c in FINAL['candidates']:
        reg = c['condition']
        key = 'Q2_strong_design' if reg == 'strong' else 'Q2_near_boundary_comparison'
        unknown = max((x['boundary_unresolved_mass'] for x in baseline[key]['propagation']))
        claims = [x for x in validation.get('claims', []) if x['regime'] == reg]
        certified = None if not claims else all((x['passed'] for x in claims))
        for pub in FINAL['delays']['publication_s']:
            for act in FINAL['delays']['action_s']:
                lo, hi = c['finite_interval_s']
                a = lo - pub - act
                b = hi - pub - act
                rows.append({
                    'regime': reg,
                    'candidate': c['name'],
                    'publication_delay_s': pub,
                    'action_delay_s': act,
                    'finite_lower_s': a,
                    'finite_upper_s': b,
                    'includes_no_hit': c['includes_no_hit'],
                    'unknown_mass': unknown,
                    'finite_width_s': hi - lo,
                    'finite_all_positive': a > 0,
                    'finite_all_nonpositive': b <= 0,
                    'certification_passed': certified,
                    'identity': 'translated frozen local set; delays hypothetical, no
                    ↪  infinite lead assigned to no_hit',
                })
    pd.DataFrame(rows).to_csv(results / 'lead_margins.csv', index=False)

def execution_controls(results):
    checks = {}
    m = BaselineModel([{'d': -1.0, 'status': 'hit', 'tau_s': 2.0}, {'d': 1.0, 'status':
    ↪  'hit', 'tau_s': 1.0}])
    s, t = m.predict(np.array([-1.5, 1.5]))
    checks['ALG01_incomplete_support_unknown'] = bool(np.all(s == 'unknown') and
    ↪  np.isnan(t).all())
    tiny = shortest_set([0, 1e-11, 2e-11], [[0.5, 0.5]], [0], target=0.5)['set']
    checks['ALG02_strict_tiny_width_primary'] = tiny['finite_width_s'] == 1e-11 and
    ↪  tiny['worst_mass'] == 0.5
    checks['no_benefit_no_hit_category'] = not shortest_set([0, 1, 2], [[0.5, 0.5]], [0],
    ↪  target=0.5)['set']['includes_no_hit_by_horizon']
    candidate = {'finite_interval_s': [0.775, 0.815], 'includes_no_hit': False}
    checks['closed_lower_and_upper'] = all((contains_candidate({'status': 'hit', 'tau_s': t},
    ↪  candidate) for t in [0.775, 0.815]))
```

```
    checks['closed_interval_outside_excluded'] = all((not contains_candidate({'status':
    ↪  'hit', 'tau_s': t}, candidate) for t in [np.nextafter(0.775, -np.inf),
    ↪  np.nextafter(0.815, np.inf)]))
    checks['nonhit_categories_not_future_hits'] = all((not contains_candidate({'status': s,
    ↪  'tau_s': None}, candidate) for s in ['no_hit', 'initial', 'unknown']))
    for name, q0, chi, d, expect in [('strong_hit', 0.5, 0.02, 0, 'hit'), ('near_hit', 0.5,
    ↪  0.0002265378784906, 0.5, 'hit'), ('no_hit', 0.5, 0.0002265378784906, -1, 'no_hit'),
    ↪  ('initial', 0.1, 0.02, 0, 'initial'), ('invalid_initial', -0.1, 0.02, 0, 'unknown')]:
        f = load_default_forcing()
        vf = VelocityForcing(f.t, np.maximum(0, f.u + d))
        r = run_member(q0, chi, vf, trajectory_points=0)
        checks[name] = r['status'] == expect
        checks[name + '_nonhit_no_time'] = r['status'] == 'hit' or r['tau_s'] is None
    try:
        build_case(float('nan'), 0.02)
        checks['nan_initial_rejected'] = False
    except ValueError:
        checks['nan_initial_rejected'] = True

    def tangent(t, y):
        return np.array([2 * (t - 1) / 3])
    sol = solve_ivp(tangent, (0, 2), [1 / 6 + 1 / 3], method='Radau', dense_output=True,
    ↪  first_step=2, max_step=2, rtol=1e-08, atol=1e-12)
    audit = audit_dense_first_hit(sol, stress_scale_E=1.0, qcrit=1 / 6, rtol=1e-08,
    ↪  atol_stress=1e-12, state_index=0)
    checks['tangent_not_no_hit'] = audit['audit_status'] == 'contact_candidate'
    save(results / 'execution_controls.json', {'checks': checks, 'all_passed':
    ↪  all(checks.values()), 'independent_review': 'pending'})
    if not all(checks.values()):
        raise RuntimeError('execution controls failed')

def join_q4_same_scenarios(q3_entries, q4_responses, expected_targets):
    import math
    if not isinstance(expected_targets, dict) or not expected_targets or any((not v for v in
    ↪  expected_targets.values())):
        raise ValueError('expected_targets must map target id to frozen geometry hash')
    targets = list(expected_targets)
    entries = {}
    for row in q3_entries:
        sid = row['scenario_id']
        if sid in entries:
            raise ValueError('duplicate Q3 scenario')
        if row['entry_stage'] != 'propagation_entry':
            raise ValueError('local first hit is not automatically propagation entry')
        if not row.get('bridge_evidence_id'):
            raise ValueError('explicit stage bridge evidence required')
        if row['status'] not in ('entry_reached', 'not_entered', 'unknown'):
            raise ValueError('unknown Q3 entry status')
        entries[sid] = row
    response = {}
    identities = ['scenario_hash', 'joint_input_hash', 'entry_id', 'entry_geometry_hash',
    ↪  'entry_state_hash', 'time_origin_id']
    for row in q4_responses:
        key = (row['scenario_id'], row['target_id'])
```

```
    if key in response:
        raise ValueError('duplicate Q4 scenario/target')
    if key[0] not in entries or key[1] not in targets:
        raise ValueError('unexpected scenario/target')
    q = entries[key[0]]
    if any((q.get(k) != row.get(k) or q.get(k) is None for k in identities)):
        raise ValueError('joint scenario, entry state or clock mismatch')
    if row.get('target_geometry_hash') != expected_targets[key[1]]:
        raise ValueError('target geometry mismatch')
    if row['time_unit'] != 's' or row['distance_unit'] != 'm':
        raise ValueError('unit mismatch')
    if row['target_arrival_status'] not in ('arrived', 'valid_no_arrival_by_horizon',
    ↪  'unknown', 'not_run'):
        raise ValueError('invalid target arrival status')
    travel = row.get('travel_time_s')
    if row['target_arrival_status'] == 'arrived':
        if travel is None or not math.isfinite(travel) or travel < 0:
            raise ValueError('invalid travel time')
        if q['status'] == 'not_entered':
            raise ValueError('not-entered member cannot have arrived')
        if row.get('prefix_review_status') != 'pass':
            raise ValueError('arrived prefix needs explicit validity review')
        valid_until = row.get('valid_until_s')
        if valid_until is None or not math.isfinite(valid_until) or valid_until < travel:
            raise ValueError('valid prefix does not include arrival')
    elif travel is not None:
        raise ValueError('non-arrived travel time must remain null')
    if row['target_arrival_status'] == 'valid_no_arrival_by_horizon':
        horizon = row.get('horizon_s', row.get('propagation_horizon_s'))
        if horizon is None or not math.isfinite(horizon) or horizon <= 0:
            raise ValueError('valid-noarrival needs a positive finite propagation
            ↪  horizon')
        if 'horizon_s' in row and 'propagation_horizon_s' in row and (row['horizon_s'] !=
        ↪  row['propagation_horizon_s']):
            raise ValueError('conflicting propagation horizon aliases')
        valid_until = row.get('valid_until_s')
        if row.get('whole_window_valid') is not True or valid_until is None or (not
        ↪  math.isfinite(valid_until)) or (valid_until < horizon):
            raise ValueError('valid-noarrival requires a valid complete propagation
            ↪  window')
    response[key] = row
out = []
for sid, q in entries.items():
    for target in targets:
        r = response.get((sid, target))
        row = {
            'scenario_id': sid,
            'target_id': target,
            'travel_time_s': None,
            'absolute_arrival_s': None,
            'effective_lead_s': None,
            'target_arrival_status': 'unknown',
            'reason': 'missing scenario-target response',
        }
```

```
            if r is None:
                out.append(row)
                continue
            row.update(travel_time_s=r.get('travel_time_s'),
            ↪  target_arrival_status=r['target_arrival_status'],
            ↪  run_termination_status=r.get('run_termination_status'),
            ↪  whole_window_valid=bool(r.get('whole_window_valid', False)),
            ↪  prefix_review_status=r.get('prefix_review_status'),
            ↪  horizon_s=r.get('horizon_s', r.get('propagation_horizon_s')),
            ↪  reason='relative Q4 response only')
            t_in = q.get('T_in_s')
            pub = q.get('t_pub_s')
            action = q.get('d_act_s')
            if r['target_arrival_status'] == 'arrived' and q['status'] == 'entry_reached' and
            ↪  (t_in is not None):
                if not math.isfinite(t_in):
                    raise ValueError('nonfinite entry time')
                row['absolute_arrival_s'] = t_in + r['travel_time_s']
                row['reason'] = 'same-scenario arrival; bridge source must still be
                ↪  scientifically reviewed'
                if pub is not None and action is not None:
                    if not math.isfinite(pub) or not math.isfinite(action) or action < 0:
                        raise ValueError('invalid publication/action time')
                    row['effective_lead_s'] = row['absolute_arrival_s'] - pub - action
            out.append(row)
    return out

def q4_five_point_conditional_demo(context, candidates, publication_delays_s,
↪  action_delays_s):
    import math
    demo = context['existing_five_point_ODE_demo']
    points = demo['points']
    bridge = float(demo['synthetic_bridge']['T_in_equals_T_L_plus_s'])
    if bridge != 0:
        raise ValueError('this fixed demonstration declares T_in=T_L+0 only')
    if len(points) != 5:
        raise ValueError('expected five fixed pre-existing Q4 ODE checks')
    out = []
    for spec in candidates:
        interval = spec.get('finite_interval_s')
        if interval is not None and (len(interval) != 2 or not all((math.isfinite(x) for x in
        ↪  interval)) or interval[0] > interval[1]):
            raise ValueError('invalid local candidate interval')
        for p in points:
            status = p['target_arrival_status']
            travel = p['travel_time_s']
            if status == 'arrived' and (travel is None or not math.isfinite(travel) or travel
            ↪  < 0 or (travel > p['horizon_s'])):
                raise ValueError('invalid saved Q4 finite arrival')
            if status != 'arrived' and travel is not None:
                raise ValueError('Q4 nonarrival cannot have finite travel time')
            for pub in publication_delays_s:
                for action in action_delays_s:
                    if not math.isfinite(pub) or not math.isfinite(action) or min(pub,
                    ↪  action) < 0:
```

```
                    raise ValueError('invalid hypothetical delay')
                arrival = None if interval is None or travel is None else [v + bridge +
                ↪  travel for v in interval]
                lead = None if arrival is None else [v - pub - action for v in arrival]
                out.append({
                    'candidate': spec['name'],
                    'condition': spec['condition'],
                    'q4_scenario_id': p['scenario_id'],
                    'target_id': p['target_id'],
                    'target_distance_m': p['target_distance_m'],
                    'distance_definition': p['distance_definition'],
                    'source_local_finite_interval_s': interval,
                    'hypothetical_bridge_T_in_minus_T_L_s': bridge,
                    'publication_delay_s': pub,
                    'action_delay_s': action,
                    'q4_target_arrival_status': status,
                    'q4_horizon_from_T_in_s': p['horizon_s'],
                    'q4_run_termination_status': p['run_termination_status'],
                    'target_finite_arrival_interval_from_shear_s': arrival,
                    'target_finite_lead_interval_s': lead,
                    'includes_local_no_hit': bool(spec['includes_no_hit']),
                    'includes_local_unknown': bool(spec.get('includes_unknown', False)),
                    'local_no_hit_lead_s': None,
                    'unknown_lead_s': None,
                    'conditional_finite_censored_arrival_lower_bound_from_shear_s':
                    ↪  interval[0] + bridge + p['horizon_s'] if interval is not None and
                    ↪  status == 'valid_no_arrival_by_horizon' else None,
                    'negative_finite_lead_interpretation': 'publication/action cannot
                    ↪  finish before target arrival under this hypothesis',
                    'category_interpretation': 'Q3 no-hit and unknown remain separate; Q4
                    ↪  noarrival only by180s from entry, not never',
                    'joint_probability_weight': None,
                    'statistical_identity': 'translation of fixed local candidate within
                    ↪  each unweighted synthetic Q4 scenario; not new joint95% or
                    ↪  natural-event coverage',
                    'geographic_warning_radius_m': None,
                    'scope': 'explicit conditional scalar-slope demonstration with
                    ↪  assumed stage bridge; not actual Gyirong propagation',
                })
    return out

def connect(q3, q4):
    for doc in (q3, q4):
        if doc.get('time_unit') != 's' or doc.get('distance_unit') != 'm':
            raise ValueError('Q4 explicit SI units required')
    if q3.get('expected_targets') != q4.get('expected_targets'):
        raise ValueError('expected target geometry contracts differ')
    for entry in q3['entries']:
        if entry['status'] == 'entry_reached':
            finite(entry.get('T_in_s'), 'T_in_s')
        elif entry.get('T_in_s') is not None:
            raise ValueError('unknown/not-entered must not carry invented entry time')
        for field in ('t_pub_s', 'd_act_s'):
            if entry.get(field) is not None:
```

```
                finite(entry[field], field, field == 'd_act_s')
    for response in q4['arrivals']:
        horizon = finite(response.get('propagation_horizon_s'), 'propagation_horizon_s', True)
        if horizon <= 0:
            raise ValueError('positive propagation horizon required')
        if type(response.get('whole_window_valid')) is not bool:
            raise ValueError('whole_window_valid must be Boolean')
        if response['target_arrival_status'] == 'valid_no_arrival_by_horizon' and (not
        ↪  response['whole_window_valid']):
            raise ValueError('failed/incomplete window cannot certify no arrival')
        if response['target_arrival_status'] == 'arrived':
            travel = finite(response.get('travel_time_s'), 'travel_time_s', True)
            if travel > horizon:
                raise ValueError('arrival exceeds supported horizon')
    rows = join_q4_same_scenarios(q3['entries'], q4['arrivals'], q3['expected_targets'])
    return {'rows': rows, 'scope': 'strict scenario-target connection; stage bridge still
    ↪  needs scientific evidence'}

def q4_outputs(results, context):
    rows = q4_five_point_conditional_demo(context, FINAL['candidates'],
    ↪  FINAL['delays']['publication_s'], FINAL['delays']['action_s'])
    save(results / 'q4_conditional_demo.json', rows)
    table = []
    for r in rows:
        a = r['target_finite_arrival_interval_from_shear_s']
        l = r['target_finite_lead_interval_s']
        table.append({
            'regime': r['condition'],
            'scenario_id': r['q4_scenario_id'],
            'target_id': r['target_id'],
            'position_m': r['target_distance_m'],
            'travel_time_s': None if a is None else a[0] -
            ↪  r['source_local_finite_interval_s'][0],
            'arrival_lower_s': None if a is None else a[0],
            'arrival_upper_s': None if a is None else a[1],
            'publication_delay_s': r['publication_delay_s'],
            'action_delay_s': r['action_delay_s'],
            'effective_lead_lower_s': None if l is None else l[0],
            'effective_lead_upper_s': None if l is None else l[1],
            'status': r['q4_target_arrival_status'],
            'horizon_s': r['q4_horizon_from_T_in_s'],
            'includes_no_hit': r['includes_local_no_hit'],
            'unknown_lead_s': None,
            'terrain_identity': 'synthetic constant8degree slope h=2m, target1000m',
            'stage_bridge': 'hypothetical T_in=T_L+0; not observed Gyirong bridge',
            'tested_target_actionable_for_entire_finite_set': None if l is None else l[0] > 0,
            'natural_warning_radius_m': None,
        })
    pd.DataFrame(table).to_csv(results / 'q4_spatial_demo.csv', index=False)
SENSITIVITY_DESIGN = {
    'version': 'Q3-SENSITIVITY-v4.0',
    'q0_design': [0.17, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.48, 0.49, 0.495, 0.5, 0.505, 0.51,
    ↪  0.52, 0.55, 0.6, 0.65, 0.7, 0.75],
    'offset_grid_count': 17,
```

```
    'offset_support_m_s': [-1.5, 1.5],
    'conditions': {'strong': 0.02, 'near': 0.0002265378784906},
    'chi_multipliers': [0.9, 0.95, 0.99, 1.0, 1.01, 1.05, 1.1],
    'chi_offset_design_m_s': [-1.5, 0.0, 1.5],
    'derivative_steps': {'offset_m_s': [0.0005, 0.001, 0.002], 'q0': [1e-06, 1e-05, 0.0001],
    ↪  'chi_relative': [0.0001, 0.001, 0.01]},
    'support_halfwidths_m_s': [0.5, 1.0, 1.5],
    'probability_targets': [0.9, 0.95, 0.99],
    'families': ['nominal_triangular', 'uniform_support', 'lower_mode_triangular'],
    'q0_chi_identity': 'unweighted mathematical design conditions, not empirical credible
    ↪  ranges or random parameters',
    'scope': 'fixed Q2 material/equations/threshold/time window; no new natural-event
    ↪  coverage or changed final candidates',
    'state_rule': 'finite tau only for hit; initial/unknown separate; no_hit never filled
    ↪  with horizon',
    'certification_rule': 'new support/target/initial/chi sensitivities do not inherit frozen
    ↪  final CP certificate',
}

def sens_cdf(x, family, halfwidth):
    a = float(halfwidth)
    if not 0 < a <= 1.5:
        raise ValueError('sensitivity support must remain inside original[-1.5,1.5]')
    u = np.clip((np.asarray(x, dtype=float) + a) / (2 * a), 0.0, 1.0)
    if family == 'uniform_support':
        return u
    if family == 'lower_mode_triangular':
        return 2 * u - u * u
    if family == 'nominal_triangular':
        return np.where(u <= 0.5, 2 * u * u, 1 - 2 * (1 - u) ** 2)
    raise ValueError('unknown declared sensitivity family')

def sens_propagate(model, edges, halfwidth):
    from scipy.optimize import brentq
    edges = np.asarray(edges, dtype=float)
    a = float(halfwidth)
    if not 0 < a <= 1.5:
        raise ValueError('sensitivity support must remain inside original[-1.5,1.5]')
    if edges.ndim != 1 or len(edges) < 2 or (not np.isfinite(edges).all()) or (edges[0] != 0)
    ↪  or (not np.all(np.diff(edges) > 0)):
        raise ValueError('sensitivity probability grid requires finite1D increasing edges
        ↪  starting at0')
    if model.x[0] > -a or model.x[-1] < a:
        raise ValueError('incomplete support for sensitivity propagation')
    finite_left = max(float(model.hx[0]), -a)
    finite_right = min(float(model.hx[-1]), a)
    supported_max = None
    if finite_right >= finite_left:
        times = [float(model.interp(finite_left)), float(model.interp(finite_right))]
        times.extend((float(v) for v in model.ht[(model.hx > finite_left) & (model.hx <
        ↪  finite_right)]))
        supported_max = max(times)
        if edges[-1] < supported_max - 1e-12:
            raise ValueError('sensitivity grid omits finite hits inside its current input
            ↪  support')
```

```
out = []
for family in SENSITIVITY_DESIGN['families']:
    values = []
    for target in edges:
        total = 0.0
        for rawl, rawr in zip(model.hx[:-1], model.hx[1:]):
            left = max(float(rawl), -a)
            right = min(float(rawr), a)
            if right <= left:
                continue
            yl = float(model.interp(left))
            yr = float(model.interp(right))
            mass = float(sens_cdf(right, family, a) - sens_cdf(left, family, a))
            if max(yl, yr) <= target:
                total += mass
            elif min(yl, yr) <= target:
                if yl == target:
                    root = left
                elif yr == target:
                    root = right
                else:
                    root = float(brentq(lambda z: float(model.interp(z)) - target, left,
                    ↪  right, xtol=1e-12))
                total += float(sens_cdf(right, family, a) - sens_cdf(root, family, a)) if
                ↪  yr < yl else float(sens_cdf(root, family, a) - sens_cdf(left, family,
                ↪  a))
        values.append(total)
    values = np.asarray(values)
    masses = np.diff(values)
    if np.min(masses) < -1e-10:
        raise ValueError('nonmonotone finite CDF')
    roundoff = float(max(0.0, -masses.min()))
    masses = np.maximum(0.0, masses)
    if model.boundary is None:
        none = unknown = 0.0
    else:
        lo, hi = model.boundary
        none = float(sens_cdf(lo, family, a))
        unknown = float(sens_cdf(hi, family, a) - sens_cdf(lo, family, a))
    total = float(masses.sum() + none + unknown)
    if abs(total - 1) > 1e-08:
        raise ValueError('sensitivity probability mass not closed')
    out.append({
        'family': family,
        'support_halfwidth_m_s': a,
        'finite_mass': masses.tolist(),
        'cdf': values.tolist(),
        'no_hit_mass': none,
        'unknown_mass': unknown,
        'total_mass': total,
        'negative_roundoff_removed': roundoff,
        'finite_support_max_tau_s': supported_max,
        'probability_identity': 'declared narrowed-support distribution propagated
        ↪  through fixed numerical map; no new certificate',
```

```
        })
    return out

def sens_certificate_identity(regime, halfwidth, target, method, spec, q0, chi_m_inv,
↪  families):
    import math
    expected = {'strong': (0.02, [0.775, 0.815], False), 'near': (0.0002265378784906, [71.4,
    ↪  72.95], True)}
    if regime not in expected or spec is None:
        return 'diagnostic_no_new_certificate'
    chi, interval, none = expected[regime]
    same = method == 'robust_shortest' and halfwidth == 1.5 and (target == 0.95) and (q0 ==
    ↪  0.5) and (chi_m_inv == chi) and (set(families) == {'nominal_triangular',
    ↪  'uniform_support', 'lower_mode_triangular'}) and (spec.get('finite_interval_s') is
    ↪  not None) and (len(spec['finite_interval_s']) == 2) and all((x == y for x, y in
    ↪  zip(spec['finite_interval_s'], interval))) and
    ↪  (spec.get('includes_no_hit_by_horizon') is none)
    return 'prior_frozen_robust_only' if same else 'diagnostic_no_new_certificate'

def run_sensitivity(data_dir, core_results_dir, out_dir):
    import json, time, hashlib, platform, sys, inspect
    from pathlib import Path
    from datetime import datetime, timezone
    import numpy as np
    import pandas as pd
    data_dir = Path(data_dir)
    core_results_dir = Path(core_results_dir)
    out_dir = Path(out_dir)
    if out_dir.exists():
        raise FileExistsError('sensitivity output exists; use a new directory')
    out_dir.mkdir(parents=True)

    def digest(p):
        return hashlib.sha256(Path(p).read_bytes()).hexdigest()

    def dump(p, o):
        Path(p).write_text(json.dumps(o, ensure_ascii=False, indent=2, allow_nan=False) +
        ↪  '\n')
    start = time.perf_counter()
    started = datetime.now(timezone.utc).isoformat()
    design = json.loads(json.dumps(SENSITIVITY_DESIGN))
    source_path = Path(__file__)
    engine_source_path = Path(inspect.getsourcefile(run_member))
    source_start_sha = digest(source_path)
    engine_start_sha = digest(engine_source_path)
    bound_inputs = {n: digest(data_dir / n) for n in ['q2_config.json', 'forcing.csv',
    ↪  'frozen_validation.json']}
    core_paths = {reg: core_results_dir / 'baseline' / ('baseline_model_' +
    ↪  ('Q2_strong_design' if reg == 'strong' else 'Q2_near_boundary_comparison') + '.json')
    ↪  for reg in design['conditions']}
    if any((not p.is_file() for p in core_paths.values())):
        raise FileNotFoundError('complete delivered baseline models required for density-only
        ↪  sensitivity')
    freeze = {
```

```
    'utc_before_queries': started,
    'design': design,
    'analysis_source_sha256': source_start_sha,
    'engine_source_sha256': engine_start_sha,
    'engine_source_file': engine_source_path.name,
    'input_sha256': bound_inputs,
    'core_baseline_sha256': {r: digest(p) for r, p in core_paths.items()},
    'raw_scenario_draws': 'none; fixed design grid, no Monte Carlo probability assigned',
    'scientific_boundary': 'certified main candidate endpoints/seed/n/material/threshold
    ↪  unchanged',
}
dump(out_dir / 'FROZEN_SENSITIVITY_DESIGN.json', freeze)
original = load_default_forcing()
cache = {}
queries = []
cache_requests = 0

def query(q0, chi, d, role):
    nonlocal cache_requests
    q0 = float(q0)
    chi = float(chi)
    d = float(d)
    key = (q0.hex(), chi.hex(), d.hex())
    if key in cache:
        cache_requests += 1
        return cache[key]
    forcing = VelocityForcing(original.t, np.maximum(0.0, original.u + d),
    ↪  identity='fixed_Q2_sensitivity_design', metadata={'offset_m_s': d})
    result = run_member(q0, chi, forcing, start_s=27.0, horizon_s=73.0,
    ↪  trajectory_points=0)
    row = {
        'query_id': len(queries),
        'q0': q0,
        'chi_m_inv': chi,
        'offset_m_s': d,
        'role': role,
        'status': result['status'],
        'tau_s': result['tau_s'],
        'qdot_cross_s_inv': result['qdot_cross_s_inv'],
        'first_hit_valid': result['first_hit_valid'],
        'whole_window_valid': result['whole_window_valid'],
        'solver_status': result['solver_status'],
        'wall_s': result['elapsed_wall_s'],
        'input_path_sha256': forcing.path_sha256,
        'failure': result['failure'],
    }
    hit = result.get('hit_diagnostics')
    if hit is not None and row['status'] == 'hit':
        physics = load_frozen_q2()
        case = physics.Case(**result['case'])
        gamma = case.chi_m_inv * forcing.speed(case, hit['elapsed_s'])
        alpha, N, meq, k = physics.constitutive(case, hit['m'], hit['s_Pa'], gamma)
        H, e, _, Jfactor = physics.closure(case, hit['m'], hit['s_Pa'])
        J = k * Jfactor / case.mu
```

```
        shear = gamma * (hit['m'] - meq) / (alpha * case.E)
        drainage = J / H / (alpha * case.E)
        row.update(hit_m=hit['m'], hit_s_Pa=hit['s_Pa'], hit_gamma_s_inv=gamma,
        ↪  shear_qdot_s_inv=shear, drainage_qdot_s_inv=drainage,
        ↪  mechanism_sum_qdot_s_inv=shear + drainage,
        ↪  mechanism_sum_abs_difference=abs(shear + drainage - row['qdot_cross_s_inv']))
    if row['status'] != 'hit' and row['tau_s'] is not None:
        raise RuntimeError('non-hit received finite time')
    cache[key] = row
    queries.append(row)
    with (out_dir / 'q2_queries.jsonl').open('a') as h:
        h.write(json.dumps(row, allow_nan=False) + '\n')
    return row
phase = []
for reg, chi in design['conditions'].items():
    for q0 in design['q0_design']:
        for d in np.linspace(-1.5, 1.5, design['offset_grid_count']):
            phase.append({'regime': reg, **query(q0, chi, d, 'q0_offset_phase_grid')})
    print('sensitivity phase map', reg, 'complete', flush=True)
pd.DataFrame(phase).to_csv(out_dir / 'phase_map.csv', index=False)
chi_rows = []
for reg, chi in design['conditions'].items():
    for mult in design['chi_multipliers']:
        for d in design['chi_offset_design_m_s']:
            chi_rows.append({'regime': reg, 'chi_multiplier': mult, **query(0.5, chi *
            ↪  mult, d, 'chi_local_design')})
pd.DataFrame(chi_rows).to_csv(out_dir / 'chi_perturbation.csv', index=False)
crossing = []
derivatives = []
for reg, chi in design['conditions'].items():
    base = query(0.5, chi, 0.0, 'nominal_crossing')
    slope = base['qdot_cross_s_inv']
    crossing.append({
        'regime': reg,
        'status': base['status'],
        'tau_s': base['tau_s'],
        'qdot_cross_s_inv': slope,
        'seconds_per_0_001_q': None if slope is None or slope == 0 else 0.001 /
        ↪  abs(slope),
        'horizon_slack_s': None if base['status'] != 'hit' else 73.0 - base['tau_s'],
        'positive_0_001_q_linear_delay_exceeds_horizon': None if slope is None or
        ↪  base['status'] != 'hit' else bool(0.001 / abs(slope) > 73.0 - base['tau_s']),
        'shear_qdot_s_inv': base.get('shear_qdot_s_inv'),
        'drainage_qdot_s_inv': base.get('drainage_qdot_s_inv'),
        'mechanism_sum_qdot_s_inv': base.get('mechanism_sum_qdot_s_inv'),
        'mechanism_sum_abs_difference': base.get('mechanism_sum_abs_difference'),
        'drainage_to_abs_shear_ratio': None if base.get('shear_qdot_s_inv') in (None, 0)
        ↪  else base['drainage_qdot_s_inv'] / abs(base['shear_qdot_s_inv']),
        'hit_gamma_s_inv': base.get('hit_gamma_s_inv'),
        'identity': 'first-order transversal response to a hypothetical state error; no
        ↪  state-error distribution assigned',
    })
    for parameter, steps in design['derivative_steps'].items():
        for h in steps:
```

```
            if parameter == 'offset_m_s':
                minus = query(0.5, chi, -h, 'local_derivative')
                plus = query(0.5, chi, h, 'local_derivative')
                delta = h
                center = 0.0
                unit = 's/(m/s)'
            elif parameter == 'q0':
                minus = query(0.5 - h, chi, 0.0, 'local_derivative')
                plus = query(0.5 + h, chi, 0.0, 'local_derivative')
                delta = h
                center = 0.5
                unit = 's per unit q0'
            else:
                minus = query(0.5, chi * (1 - h), 0.0, 'local_derivative')
                plus = query(0.5, chi * (1 + h), 0.0, 'local_derivative')
                delta = chi * h
                center = chi
                unit = 's/(m^-1)'
            finite = minus['status'] == 'hit' and plus['status'] == 'hit'
            derivative = (plus['tau_s'] - minus['tau_s']) / (2 * delta) if finite else
            ↪  None
            elasticity = derivative * center / base['tau_s'] if finite and base['status']
            ↪  == 'hit' and (parameter != 'offset_m_s') else None
            derivatives.append({
                'regime': reg,
                'parameter': parameter,
                'step': h,
                'absolute_parameter_step': delta,
                'minus_status': minus['status'],
                'plus_status': plus['status'],
                'minus_tau_s': minus['tau_s'],
                'plus_tau_s': plus['tau_s'],
                'derivative': derivative,
                'derivative_unit': unit,
                'elasticity': elasticity,
                'parameter_reference_scale': 1.5 if parameter == 'offset_m_s' else center,
                'scaled_sensitivity': None if not finite or base['status'] != 'hit' else
                ↪  derivative * (1.5 if parameter == 'offset_m_s' else center) /
                ↪  base['tau_s'],
                'scale_identity': 'offset uses declared1.5m/s halfwidth, q0 uses.5, chi
                ↪  uses nominalchi; not a probability-based global ranking',
                'method': 'central finite difference, two finite hits only' if finite
                ↪  else 'category transition/unknown; finite derivative not defined',
                'minus_query_id': minus['query_id'],
                'plus_query_id': plus['query_id'],
            })
pd.DataFrame(crossing).to_csv(out_dir / 'crossing_amplification.csv', index=False)
pd.DataFrame(derivatives).to_csv(out_dir / 'local_derivatives.csv', index=False)
stability = []
for reg in design['conditions']:
    for parameter in design['derivative_steps']:
        ds = [r['derivative'] for r in derivatives if r['regime'] == reg and
        ↪  r['parameter'] == parameter and (r['derivative'] is not None)]
        stability.append({
```

```
            'regime': reg,
            'parameter': parameter,
            'finite_step_count': len(ds),
            'total_step_count': 3,
            'derivative_min': None if not ds else min(ds),
            'derivative_max': None if not ds else max(ds),
            'relative_step_spread': None if len(ds) < 2 or np.mean(np.abs(ds)) == 0 else
            ↪  (max(ds) - min(ds)) / float(np.mean(np.abs(ds))),
            'identity': 'step variation diagnostic, not a rigorous derivative error
            ↪  bound',
        })
pd.DataFrame(stability).to_csv(out_dir / 'derivative_step_stability.csv', index=False)
edges = grid_edges()
probabilities = {}
dist_rows = []
set_rows = []
reweight_checks = []
for reg, path in core_paths.items():
    model = BaselineModel.from_json(path)
    probabilities[reg] = {}
    for a in design['support_halfwidths_m_s']:
        propagated = sens_propagate(model, edges, a)
        probabilities[reg][str(a)] = propagated
        masses = np.array([r['finite_mass'] for r in propagated])
        none = np.array([r['no_hit_mass'] for r in propagated])
        targets = design['probability_targets'] if a == 1.5 else [0.95]
        for target in targets:
            nominal = shortest_set(edges, masses[:1], none[:1], target=target)['set']
            robust = shortest_set(edges, masses, none, target=target)['set']
            for method, spec in [('nominal_shortest', nominal), ('robust_shortest',
            ↪  robust)]:
                set_rows.append({
                    'regime': reg,
                    'support_halfwidth_m_s': a,
                    'target_mass': target,
                    'method': method,
                    'finite_lower_s': None if spec is None or spec['finite_interval_s']
                    ↪  is None else spec['finite_interval_s'][0],
                    'finite_upper_s': None if spec is None or spec['finite_interval_s']
                    ↪  is None else spec['finite_interval_s'][1],
                    'finite_width_s': None if spec is None else spec['finite_width_s'],
                    'includes_no_hit': None if spec is None else
                    ↪  spec['includes_no_hit_by_horizon'],
                    'worst_model_mass': None if spec is None else spec['worst_mass'],
                    'independent_certification': sens_certificate_identity(reg, a,
                    ↪  target, method, spec, 0.5, design['conditions'][reg],
                    ↪  design['families']),
                })
            for row in propagated:
                dist_rows.append({
                    'regime': reg,
                    'support_halfwidth_m_s': a,
                    'family': row['family'],
                    'target_mass': target,
```

```
                    'hit_mass': float(sum(row['finite_mass'])),
                    'no_hit_mass': row['no_hit_mass'],
                    'unknown_mass': row['unknown_mass'],
                    'total_mass': row['total_mass'],
                })
        if a == 1.5:
            orig = model.propagated(edges)
            delta = max((float(np.max(np.abs(np.array(r['cdf']) - np.array(s['cdf']))))
            ↪  for r, s in zip(propagated, orig)))
            reweight_checks.append({'regime': reg,
            ↪  'original_fullsupport_CDF_max_abs_difference': delta, 'pass': delta <
            ↪  1e-10})
pd.DataFrame(dist_rows).to_csv(out_dir / 'distribution_sensitivity.csv', index=False)
pd.DataFrame(set_rows).to_csv(out_dir / 'set_sensitivity.csv', index=False)
dump(out_dir / 'probability_sensitivity.json', {'edges_s': edges.tolist(), 'conditions':
↪  probabilities})
reuse_checks = []
for reg, path in core_paths.items():
    prior = json.loads(path.read_text())['rows']
    for d in [-1.5, 0.0, 1.5]:
        exact = [r for r in prior if abs(float(r['d']) - d) < 1e-14]
        new = query(0.5, design['conditions'][reg], d, 'reuse_check')
        if exact:
            p = exact[0]
            delta = None if new['tau_s'] is None or p['tau_s'] is None else
            ↪  abs(new['tau_s'] - p['tau_s'])
            ok = new['status'] == p['status'] and (new['tau_s'] is None and p['tau_s'] is
            ↪  None or (delta is not None and delta < 1e-05))
            reuse_checks.append({
                'regime': reg,
                'offset_m_s': d,
                'prior_status': p['status'],
                'current_status': new['status'],
                'tau_abs_difference_s': delta,
                'pass': ok,
            })
checks = {
    'no_nonhit_finite_times': all((r['tau_s'] is None for r in queries if r['status'] !=
    ↪  'hit')),
    'phase_grid_complete': len(phase) == 646,
    'chi_design_complete': len(chi_rows) == 42,
    'mass_closure': all((abs(r['total_mass'] - 1) < 1e-08 for r in dist_rows)),
    'original_fullsupport_reweight_equal': all((r['pass'] for r in reweight_checks)),
    'reused_nominal_reference_equal': all((r['pass'] for r in reuse_checks)),
    'mechanism_qdot_sum_matches': all((r.get('mechanism_sum_abs_difference', 0.0) < 1e-12
    ↪  for r in queries)),
    'source_unchanged_during_sensitivity': digest(source_path) == source_start_sha and
    ↪  digest(engine_source_path) == engine_start_sha,
}
if not all(checks.values()):
    raise RuntimeError('sensitivity self-check failure')
q = pd.DataFrame(queries)
q.to_csv(out_dir / 'query_summary.csv', index=False)
strong = crossing[0]['seconds_per_0_001_q']
```

```
near = crossing[1]['seconds_per_0_001_q']
phase_states = {reg: {str(k): int(v) for k, v in
↪  pd.DataFrame(phase).query('regime==@reg').status.value_counts().items()} for reg in
↪  design['conditions']}
summary = {
    'version': design['version'],
    'conditions': design['conditions'],
    'unique_new_Q2_calls': len(queries),
    'cache_reuse_requests': cache_requests,
    'phase_rows': len(phase),
    'phase_states': phase_states,
    'chi_rows': len(chi_rows),
    'derivative_rows': len(derivatives),
    'crossing_amplification': crossing,
    'near_to_strong_amplification_ratio': near / strong,
    'nominal_horizon_slack_s': {r['regime']: r['horizon_slack_s'] for r in crossing},
    'linearization_boundary': 'state-error time amplification applies to transverse local
    ↪  roots only; if delay exceeds horizon slack retain category transition, not a
    ↪  finite in-window confidence band',
    'derivative_step_stability': stability,
    'checks': checks,
    'reweight_checks': reweight_checks,
    'nominal_reference_checks': reuse_checks,
    'independent_review': 'pending; execution self-check only',
    'probability_boundary': design['certification_rule'],
    'interpretation': 'q0/chi/offset grid is unweighted, not event probability; finite
    ↪  derivatives only where two sides hit',
}
dump(out_dir / 'summary.json', summary)
receipt = {
    'version': design['version'],
    'started_utc': started,
    'completed_utc': datetime.now(timezone.utc).isoformat(),
    'wall_s': time.perf_counter() - start,
    'python': platform.python_version(),
    'executable': sys.executable,
    'analysis_source_sha256': source_start_sha,
    'engine_source_sha256': engine_start_sha,
    'input_sha256': bound_inputs,
    'core_baseline_sha256': freeze['core_baseline_sha256'],
    'unique_new_Q2_calls': len(queries),
    'cache_reuse_requests': cache_requests,
    'Q2_solver_wall_sum_s': float(q.wall_s.sum()),
    'density_reweight_extra_Q2_calls': 0,
    'outputs': {p.name: digest(p) for p in out_dir.iterdir() if p.is_file()},
    'checks': checks,
    'independent_review': 'pending',
}
dump(out_dir / 'SENSITIVITY_RECEIPT.json', receipt)
print(json.dumps({
    'sensitivity_output': str(out_dir),
    'unique_Q2_calls': len(queries),
    'wall_s': receipt['wall_s'],
    'all_execution_checks_pass': True,
```

```
    }), flush=True)
    return summary
RESULT_VERSION = 'Q3-STRENGTHENED-v4.0'

def integrated_sensitivity(core_results, out_dir):
    answer = run_sensitivity(DATA, core_results, out_dir)
    required = ['legacy_unweighted_reference.csv', 'legacy_numerical_convergence.csv',
    ↪  'LEGACY_SENSITIVITY_SOURCES.json']
    for name in required:
        if not (DATA / name).is_file():
            raise FileNotFoundError('required sensitivity reference missing: data/' + name)
        (out_dir / name).write_bytes((DATA / name).read_bytes())
    legacy = pd.read_csv(DATA / 'legacy_unweighted_reference.csv')
    phase = pd.read_csv(out_dir / 'phase_map.csv')
    aliases = {'Q2_strong_design': 'strong', 'Q2_near_boundary_comparison': 'near'}
    checks = []
    for _, row in legacy.iterrows():
        if row.regime not in aliases:
            continue
        sub = phase[(phase.regime == aliases[row.regime]) & np.isclose(phase.q0,
        ↪  float(row.q0), rtol=0, atol=1e-14) & np.isclose(phase.offset_m_s,
        ↪  float(row.offset_m_s), rtol=0, atol=1e-14)]
        if len(sub) != 1:
            raise RuntimeError('old reference cannot be matched to unique new phase input')
        new = sub.iloc[0]
        delta = None if row.status != 'hit' or new.status != 'hit' else
        ↪  abs(float(row.first_hit_s) - float(new.tau_s))
        ok = bool(row.status == new.status and (row.status != 'hit' or (delta is not None and
        ↪  delta < 1e-05)))
        checks.append({
            'regime': aliases[row.regime],
            'q0': float(row.q0),
            'offset_m_s': float(row.offset_m_s),
            'old_status': row.status,
            'new_status': new.status,
            'tau_abs_difference_s': delta,
            'pass': ok,
        })
    if len(checks) != 24 or not all((x['pass'] for x in checks)):
        raise RuntimeError('legacy24 scenario replay mismatch')
    oldnum = pd.read_csv(DATA / 'legacy_numerical_convergence.csv')
    if len(oldnum) != 6 or not oldnum.status_agrees.all():
        raise ValueError('legacy numerical reference malformed')
    oldchecks = {
        'role': 'historical numerical tolerance tests, not new physical sensitivity or new
        ↪  certification',
        'case_count': len(oldnum),
        'max_finite_tau_difference_s': float(oldnum.tau_abs_difference_s.max()),
        'all_statuses_equal': bool(oldnum.status_agrees.all()),
    }
    save(out_dir / 'LEGACY_REUSE_CHECKS.json', {
        'matched24_scenarios': checks,
        'old4_initial_admission_controls': 'preserved in source table, outside this phase
        ↪  grid',
```

```
        'old_numerical_controls': oldchecks,
    })
    result = {
        'version': RESULT_VERSION,
        'actual_sensitivity_receipt_sha256': sha(out_dir / 'SENSITIVITY_RECEIPT.json'),
        'analysis_sha256': sha(__file__),
        'legacy_input_sha256': {n: sha(DATA / n) for n in required},
        'new_unique_Q2_calls': answer['unique_new_Q2_calls'],
        'legacy_reference_only_new_Q2_calls': 0,
        'legacy_24_matching_checks_passed': True,
        'metadata_identity': 'new phase/chi/derivative sensitivity plus explicit historical
        ↪  numerical references; only original frozen robust objects retain old statistical
        ↪  evidence',
        'output_sha256': {str(p.relative_to(out_dir)): sha(p) for p in out_dir.rglob('*') if
        ↪  p.is_file()},
    }
    save(out_dir / 'INTEGRATED_SENSITIVITY_RECEIPT.json', result)
    return answer

def sensitivity_only_run(results, core_results, started, wall, source_sha):
    if core_results is None:
        raise ValueError('--stage sensitivity requires an explicit delivered --core-results
        ↪  directory')
    core_results = core_results.resolve()
    answer = integrated_sensitivity(core_results, results / 'sensitivity')
    summary = {
        'version': RESULT_VERSION,
        'execution_stage': 'sensitivity',
        'sensitivity': answer,
        'core_results_supplied_explicitly': str(core_results),
        'base_certificate': 'not rerun by this supplementary stage; no new scenarios inherit
        ↪  it',
        'independent_review': 'pending',
        'analysis_source_sha256': source_sha,
    }
    save(results / 'summary.json', summary)
    if sha(__file__) != source_sha:
        raise RuntimeError('source changed during supplementary run')
    save(results / 'RUN_RECEIPT.json', {
        'version': RESULT_VERSION,
        'stage': 'sensitivity',
        'started_utc': started,
        'completed_utc': stamp(),
        'wall_s': time.perf_counter() - wall,
        'command_arguments': sys.argv[1:],
        'python': platform.python_version(),
        'executable': sys.executable,
        'source_sha256': source_sha,
        'versions': {
            'numpy': np.__version__,
            'scipy': scipy.__version__,
            'pandas': pd.__version__,
            'sklearn': sklearn.__version__,
        },
```

```
        'input_sha256': {p.name: sha(p) for p in DATA.iterdir() if p.is_file()},
        'output_sha256': {str(p.relative_to(results)): sha(p) for p in results.rglob('*') if
        ↪  p.is_file()},
        'scientific_acceptance': 'pending independent review; supplementary stage is not a
        ↪  complete base workflow reproduction',
    })
    print(json.dumps({'results': str(results), 'stage': 'sensitivity', 'unique_new_Q2_calls':
    ↪  answer['unique_new_Q2_calls']}), flush=True)

def sol():
    global DATA, OUT
    ap = argparse.ArgumentParser(description='Q3 analysis')
    ap.add_argument('--data', type=Path, default=PACKAGE_ROOT / 'data')
    ap.add_argument('--results', type=Path, default=PACKAGE_ROOT / 'results')
    ap.add_argument('--stage', choices=['all', 'develop', 'sensitivity'], default='all')
    ap.add_argument('--workers', type=int, default=4)
    ap.add_argument('--q3-entry', type=Path)
    ap.add_argument('--q4-arrivals', type=Path)
    ap.add_argument('--core-results', type=Path)
    args = ap.parse_args()
    if not 1 <= args.workers <= 4:
        raise ValueError('workers must be1..4')
    configure(args.data.resolve())
    results = args.results.resolve()
    if results.exists():
        raise FileExistsError('results directory exists: use a new output directory')
    results.mkdir(parents=True)
    started = stamp()
    wall = time.perf_counter()
    before = sha(__file__)
    save(results / 'RUN_CONFIG.json', {'final': FINAL, 'gp': CONFIG, 'stage': args.stage,
    ↪  'workers': args.workers})
    if args.stage == 'sensitivity':
        sensitivity_only_run(results, args.core_results, started, wall, before)
        return
    if args.core_results is not None:
        raise ValueError('--core-results is only for the supplementary sensitivity stage')
    execution_controls(results)
    cost = numerical_pipeline(results)
    validation = validate_final(results, args.workers) if args.stage == 'all' else {'status':
    ↪  'pending_development_only', 'claims': [], 'all_passed': False}
    if args.stage != 'all':
        save(results / 'validation.json', validation)
    lead_outputs(results, validation)
    q4 = json.loads((DATA / 'q4_context.json').read_text()) if (DATA /
    ↪  'q4_context.json').exists() else {'status': 'pending', 'reason': 'q4_context.json not
    ↪  delivered'}
    if bool(args.q3_entry) != bool(args.q4_arrivals):
        raise ValueError('both q3-entry and q4-arrivals are required for connection')
    if 'existing_five_point_ODE_demo' in q4:
        q4_outputs(results, q4)
    if args.q3_entry:
        linked = connect(json.loads(args.q3_entry.read_text()),
        ↪  json.loads(args.q4_arrivals.read_text()))
```

```
    save(results / 'q4_connected.json', linked)
sensitivity = integrated_sensitivity(results, results / 'sensitivity')
summary = {
    'version': RESULT_VERSION,
    'base_certification_version': FINAL['version'],
    'sensitivity': sensitivity,
    'started_utc': started,
    'completed_utc': stamp(),
    'wall_s': time.perf_counter() - wall,
    'execution_stage': args.stage,
    'random_validation': validation,
    'conditions': CONFIG['conditions'],
    'candidate_sets': FINAL['candidates'],
    'cost': cost,
    'q4_summary': q4,
    'method_choice': 'adaptive_piecewise_integration_primary; GP actual comparison and
    ↪  optional accelerator',
    'independent_review': 'pending',
    'natural_event_coverage': 'not identified',
    'self_contained_source_sha256': before,
}
save(results / 'summary.json', summary)
if sha(__file__) != before:
    raise RuntimeError('source changed during run')
receipt = {
    'version': RESULT_VERSION,
    'base_certification_version': FINAL['version'],
    'started_utc': started,
    'completed_utc': stamp(),
    'wall_s': time.perf_counter() - wall,
    'command_arguments': sys.argv[1:],
    'python': platform.python_version(),
    'executable': sys.executable,
    'versions': {
        'numpy': np.__version__,
        'scipy': scipy.__version__,
        'pandas': pd.__version__,
        'sklearn': sklearn.__version__,
    },
    'source_sha256': before,
    'input_sha256': {p.name: sha(p) for p in DATA.iterdir() if p.is_file()},
    'output_sha256': {str(p.relative_to(results)): sha(p) for p in results.rglob('*') if
    ↪  p.is_file()},
    'scientific_acceptance': 'pending independent review; execution receipt is not
    ↪  acceptance',
}
save(results / 'RUN_RECEIPT.json', receipt)
print(json.dumps({
    'results': str(results),
    'validation': validation['status'],
    'all_claims_pass': validation.get('all_passed'),
    'wall_s': receipt['wall_s'],
}), flush=True)
```

```
if __name__ == '__main__':
    sol()
```

问题四

q4/base/code/draw.py

```
from pathlib import Path
import argparse
import csv
import hashlib
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

COLORS = ['#777777', '#4477AA', '#66AABB', '#AA3377']
METHODS = ['A_single_fill', 'B_MF_fill', 'Bm_MF_count_matched', 'C_MF_active']
LABELS = ['Single GP', 'MF passive', 'MF matched', 'MF active']


def fingerprint(path):
    return {'path': str(path), 'sha256': hashlib.sha256(path.read_bytes()).hexdigest(),
    ↪  'bytes': path.stat().st_size}


def sol():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--root', type=Path, required=True)
    ap.add_argument('--out', type=Path, required=True)
    args = ap.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    spec_path = args.root / 'results/PLOT_INPUTS.json'
    spec = json.loads(spec_path.read_text())
    inputs = [fingerprint(spec_path)]
    figures = []
    plt.rcParams.update({'font.family': 'sans-serif', 'font.sans-serif': ['Arial',
    ↪  'Helvetica', 'DejaVu Sans'], 'font.size': 9,
                         'axes.labelsize': 10, 'pdf.fonttype': 42, 'svg.fonttype': 'none',
                         'axes.spines.top': False, 'axes.spines.right': False,
                         'legend.frameon': False, 'axes.linewidth': .8})

    def asset(key):
        path = args.root / spec[key]
        if not path.resolve().is_relative_to(args.root.resolve()):
            raise ValueError('plot input must be inside delivery root')
        inputs.append(fingerprint(path))
        return path

    def canvas():
        return plt.subplots(figsize=(7.2, 4.2), layout='constrained')

    def save(fig, name, note):
        fig.canvas.draw()
```

```
    (args.out / (name + '.alignment.json')).write_text(json.dumps({
        'verdict': 'NOT APPLICABLE', 'reason': 'one scientific plot-area', 'plot_areas':
        ↪  1}))
    fig.savefig(args.out / (name + '.pdf'), bbox_inches='tight')
    fig.savefig(args.out / (name + '.svg'), bbox_inches='tight')
    fig.savefig(args.out / (name + '.png'), dpi=600, bbox_inches='tight')
    figures.append({'name': name, 'interpretation': note,
                    'files': [fingerprint(args.out / (name + '.' + e)) for e in ('pdf',
                    ↪  'svg', 'png')]})
    plt.close(fig)

if 'morris' in spec:
    d = json.loads(asset('morris').read_text())
    fig, ax = canvas()
    ax.scatter(d['mu_star'], d['sigma'], color=COLORS[1], s=40)
    for x, y, label in zip(d['mu_star'], d['sigma'], ['V', 'T', 'μ', 'ξ']):
        ax.annotate(label, (x, y), xytext=(6, 5), textcoords='offset points')
    ax.set(xlabel='Mean absolute elementary effect (m)', ylabel='Elementary-effect SD
    ↪  (m)',
           xlim=(0, max(d['mu_star']) * 1.2), ylim=(0, max(d['sigma']) * 1.2))
    save(fig, '01_morris', 'Eight complete trajectories; domain-dependent screening of
    ↪  raw depth margin (m).')

if 'hydrograph' in spec:
    d = np.load(asset('hydrograph'))
    series = d['series']
    support_rows = []
    for coordinate in (348660., 348720., 348780.):
        x, y = d['x'], d['y']
        j = int(np.searchsorted(x, coordinate))
        a = (coordinate-x[j-1])/(x[j]-x[j-1])
        selected = (y >= 3134880) & (y <= 3135600)
        h = (1-a)*d['snapshot_U'][:, :, j-1, 0] + a*d['snapshot_U'][:, :, j, 0]
        peak = h[:, selected].max(axis=1)
        hits = np.flatnonzero(peak >= .1)
        support_rows.append({'target_x_m': coordinate,
            'snapshot_peak_h_lower_bound_m': float(peak.max()),
            'first_recorded_exceedance_time_s': float(d['snapshot_t'][hits[0]]) if
            ↪  len(hits) else None})
    (args.out/'TARGET_SUPPORT_SENSITIVITY.json').write_text(json.dumps({
        'results': support_rows, 'nominal_snapshot_interval_s': 30,
        'interpretation': 'Saved-field spatial support comparison; peak lower bounds and
        ↪  first observed snapshot exceedances, not stepwise first-arrival brackets.'},
        ↪  indent=2))
    for column, name, ylabel in [(1, '02_target_depth', 'Target peak depth (m)'),
                                 (4, '03_target_discharge', 'Signed downstream discharge
                                 ↪  (m³/s)')]:
        fig, ax = canvas()
        ax.plot(series[:, 0], series[:, column], color=COLORS[1], lw=1.5)
        if column == 1:
            ax.axhline(.1, color=COLORS[0], ls='--', lw=.8, label='Arrival threshold: 0.1
            ↪  m')
            ax.legend(loc='upper right')
        ax.set(xlabel='Time after prescribed entry (s)', ylabel=ylabel)
```

```
        save(fig, name, 'Stored same-time cross-section series; signed discharge and peak
        ↪  depth have distinct meanings.')

if 'metrics' in spec:
    rows = list(csv.DictReader(asset('metrics').open()))
    for stratum, suffix in [('global', 'global'), ('development_predicted_boundary',
    ↪  'band')]:
        fig, ax = canvas()
        for seed in sorted({int(r['seed']) for r in rows}):
            selected = [next(r for r in rows if r['method'] == method and int(r['seed'])
            ↪  == seed and r['stratum'] == stratum) for method in METHODS]
            values = [float(r['disagreement_rate']) if r['disagreement_rate'] else np.nan
            ↪  for r in selected]
            ax.plot(range(4), values, color='#BBBBBB', lw=.8, zorder=1)
            ax.scatter(range(4), values, c=COLORS, s=35, zorder=2,
            ↪  marker={137:'o',271:'s',503:'^'}[seed], label=f'Seed {seed}')
        ax.set_xticks(range(4), LABELS)
        ax.set(ylabel='Classification disagreement / valid HF cases', ylim=(-.025, 1.025))
        ax.legend(loc='upper right')
        save(fig, '04_comparison_' + suffix, 'Single matched seed; no cross-seed
        ↪  stability estimate; all valid high-layer cases retained, unknowns excluded
        ↪  only from valid-case denominator.')

if 'costs' in spec:
    rows = list(csv.DictReader(asset('costs').open()))
    fig, ax = canvas()
    for j, method in enumerate(METHODS):
        v = [float(r['physical_child_wall_sum_s']) / 60 for r in rows if r['method'] ==
        ↪  method]
        ax.scatter(np.full(len(v), j), v, color=COLORS[j], s=35)
    ax.set_xticks(range(4), LABELS)
    ax.set(ylabel='Required physical child wall-time sum (min)')
    save(fig, '05_cost', 'Sum of measured required physical child wall times only;
    ↪  GP/acquisition timings missing, so this is not full cost, CPU time, or parallel
    ↪  elapsed time.')

if 'surface' in spec:
    d = np.load(asset('surface'))
    a = d['axis']; v = 2e5 * 4**a; mu = .03 + .12*a
    fig, ax = canvas()
    img = ax.pcolormesh(v, mu, d['mean_g'], shading='auto', cmap='RdBu_r')
    if d['mean_g'].min() <= 0 <= d['mean_g'].max():
        ax.contour(v, mu, d['mean_g'], levels=[0], colors='black', linewidths=1)
    fig.colorbar(img, ax=ax, label='Frozen GP mean log margin g')
    ax.set(xlabel='Prescribed volume (m³)', ylabel='Coulomb coefficient mu')
    save(fig, '06_surface', 'One frozen active model at T=960 s and xi=500 m/s²; zero
    ↪  contour is a surrogate boundary, not a natural probability.')

if 'full_field' in spec:
    d = np.load(asset('full_field'))
    fig, ax = canvas()
    xx, yy = np.meshgrid(d['x'], d['y'])
    wet = np.ma.masked_less(d['max_h'], .1)
    img = ax.pcolormesh(xx/1000, yy/1000, wet, shading='auto', cmap='Blues',
    ↪  rasterized=True)
```

```
        if 'full_route' in spec:
            route = np.loadtxt(asset('full_route'), delimiter=',', skiprows=1)
            ax.plot(route[:,0]/1000, route[:,1]/1000, color='#666666', lw=.6,
            ↪  label='Reference axis')
        ax.scatter([350.640], [3135.040], marker='>', s=35, color='#AA3377',
        ↪  label='Prescribed entry', zorder=5, clip_on=False)
        ax.plot([340.500, 341.100], [3129.103736725446]*2, color='#AA3377', lw=2, label='Port
        ↪  target')
        ax.legend(loc='lower right', fontsize=8)
        ax.set(xlabel='UTM easting (km)', ylabel='UTM northing (km)', aspect='equal',
               xlim=((d['x'].min()-20)/1000, (d['x'].max()+20)/1000),
               ylim=((d['y'].min()-20)/1000, (d['y'].max()+20)/1000))
        fig.colorbar(img, ax=ax, label='Space-time maximum depth ≥ 0.1 m (m)')
        save(fig, '07_full_footprint', 'Ever-threshold depth footprint differs from final
        ↪  instantaneous inundation; axis line is a geometric reference.')

    if 'inverse' in spec:
        d = np.load(asset('inverse'))
        fig, ax = canvas()
        ax.scatter(d['normalized_arrival_error'], d['normalized_depth_error'], s=4,
        ↪  color='#BBBBBB', rasterized=True)
        indices = d['pareto_indices'].astype(int)
        ax.plot(d['normalized_arrival_error'][indices], d['normalized_depth_error'][indices],
        ↪  color=COLORS[3], lw=1.2)
        ax.axvline(1, ls='--', color=COLORS[0], lw=.8)
        ax.axhline(1, ls='--', color=COLORS[0], lw=.8)
        ax.set(xlabel='Absolute arrival error / 60 s', ylabel='Absolute peak-depth error /
        ↪  0.75 m')
        save(fig, '08_inverse', 'Synthetic truth at independent 15 m discretization;
        ↪  candidate surface from 30 m library, with separate physical representative
        ↪  rechecks.')

    (args.out / 'PLOT_RECEIPT.json').write_text(json.dumps({'source':
    ↪  fingerprint(Path(__file__)), 'inputs': inputs,
          'figures': figures, 'solver_executed': False, 'GP_fitted': False,
          'visual_and_collision_review': 'pending'}, ensure_ascii=False, indent=2))


if __name__ == '__main__':
    sol()
```

q4/code/draw.py

```
from pathlib import Path
import argparse,csv,json,hashlib
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def sol():
 ap=argparse.ArgumentParser();ap.add_argument('--root',type=Path,required=True);ap.add_argume⌟
 ↪  nt('--out',type=Path,required=True);a=ap.parse_args();a.out.mkdir(parents=True,exist_ok=⌟
 ↪  True)
 p=a.root/'results/PUBLISH_COMPARISON.csv';rows=list(csv.DictReader(p.open()))
```

```
 plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':['Arial','Helvetica','Deja
 ↪  Vu
 ↪  Sans'],'font.size':9,'pdf.fonttype':42,'svg.fonttype':'none'})
 names={'arrived':'Arrived','valid_no_arrival_by_horizon':'No arrival\nby 1800
 ↪  s','unknown':'Unknown'}
 cells=[]
 for r in rows:
  cost=f"{float(r['new_PDE_child_wall_s']):.2f}" if r['new_PDE_child_wall_s'] else 'Cached
  ↪  history'
  cells.append([r['request'],f"{float(r['mu']):.5f}",names[r['GP_estimate']],names[r['physica
  ↪  l_status']],names[r['published_status']],cost])
 fig,ax=plt.subplots(figsize=(9,3.2));ax.axis('off')
 table=ax.table(cellText=cells,colLabels=['Request','μ','GP estimate','Physical
 ↪  model','Published','Fresh PDE (s)'],cellLoc='center',colWidths=[.23,.1,.15,.17,.17,.18],
 ↪  bbox=[0,.16,1,.74]);table.auto_set_font_size(False);table.set_fontsize(8)
 for (r,c),cell in table.get_celld().items():
  cell.set_linewidth(.4);cell.set_edgecolor('#BBBBBB')
  if r==0:cell.set_facecolor('#EAF0F5');cell.set_text_props(weight='bold')
 ax.set_title('Physical verification controls publication; GP estimates remain
 ↪  unchanged',fontsize=10,pad=8)
 fig.text(.09,.065,'Four historical replays + two new PDE queries. This is interface
 ↪  consistency, not GP accuracy or natural-event validation.',fontsize=8)
 outputs=[]
 for ext in ['pdf','svg','png']:
  dest=a.out/('publication_guard.'+ext);fig.savefig(dest,dpi=600,bbox_inches='tight');outputs
  ↪  .append({'path':dest.name,'sha256':hashlib.sha256(dest.read_bytes()).hexdigest()})
 plt.close(fig)
 (a.out/'PLOT_RECEIPT.json').write_text(json.dumps({'source_sha256':hashlib.sha256(Path(__fil
 ↪  e__).read_bytes()).hexdigest(),'input_sha256':hashlib.sha256(p.read_bytes()).hexdigest()
 ↪  ,'outputs':outputs,'PDE_ran':False,'GP_fitted':False,'alignment':'single table
 ↪  plot-area'},indent=2))
if __name__=='__main__':sol()
```

q4/code/sol.py

```
from pathlib import Path
import json, math, time
import numpy as np
from scipy.integrate import solve_ivp

G = 9.81
DRY = 1e-4
ORDER = 1
FAST = None


import ctypes,hashlib,subprocess,sys,json,re
from pathlib import Path
import numpy as np

SOURCE=r'''
#include <math.h>
#include <stdlib.h>
#include <string.h>
static const double G=9.81;
```

```
static double wallp(double h,double w){
  if(h<=0)return 0.;double c=sqrt(G*h);
  if(w<=0){double cs=fmax(0.,c+.5*w),hs=cs*cs/G;return .5*G*hs*hs;}
  double f=w/c,lo=1.,hi=1.+2.*f+f*f;
  for(int j=0;j<60;j++){double
  ↪  r=.5*(lo+hi),value=(r-1.)*sqrt((r+1.)/(2.*r));if(value<f)lo=r;else hi=r;}
  double hs=h*.5*(lo+hi);return .5*G*hs*hs;
}
static double mm(double a,double b,double c){return
↪  a*b>0&&a*c>0?copysign(fmin(fmin(fabs(a),fabs(b)),fabs(c)),a):0.;}
static void state(const double *w,const double *s,double sign,double *u,double *z){
  double h=fmax(0.,w[0]+sign*s[0]-(w[3]+sign*s[3]));
  *z=w[3]+sign*s[3];u[0]=h;u[1]=h*(w[1]+sign*s[1]);u[2]=h*(w[2]+sign*s[2]);
}
static void hll(const double *L,const double *R,double zl,double zr,int axis,double
↪  dry,double *F,double *CL,double *CR){
  double vl[2]={0,0},vr[2]={0,0},UL[3],UR[3],FL[3],FR[3];
  for(int k=0;k<2;k++){if(L[0]>dry)vl[k]=L[k+1]/L[0];if(R[0]>dry)vr[k]=R[k+1]/R[0];}
  double zs=fmax(zl,zr),hl=fmax(0.,L[0]+zl-zs),hr=fmax(0.,R[0]+zr-zs);
  double nl=vl[axis],nr=vr[axis],cl=sqrt(G*hl),cr=sqrt(G*hr);
  double sl=fmin(nl-cl,nr-cr),sr=fmax(nl+cl,nr+cr);
  if(hr<=0.){sl=nl-cl;sr=nl+2*cl;}if(hl<=0.){sl=nr-2*cr;sr=nr+cr;}
  UL[0]=hl;UR[0]=hr;for(int k=0;k<2;k++){UL[k+1]=hl*vl[k];UR[k+1]=hr*vr[k];}
  for(int k=0;k<3;k++){FL[k]=UL[k]*nl;FR[k]=UR[k]*nr;}
  FL[axis+1]+=.5*G*hl*hl;FR[axis+1]+=.5*G*hr*hr;
  for(int k=0;k<3;k++){double mid=sr-sl>dry?(sr*FL[k]-sl*FR[k]+sl*sr*(UR[k]-UL[k]))/(sr-sl):0
  ↪  .;F[k]=sl>=0?FL[k]:(sr<=0?FR[k]:mid);}
  *CL=.5*G*(L[0]*L[0]-hl*hl);*CR=.5*G*(R[0]*R[0]-hr*hr);
}
int fluxes(int ny,int nx,int order,int bc,double dry,const double *V,const double *Z,
 double *FX,double *FY,double *XL,double *XR,double *YL,double *YR,double *SX,double
 ↪  *SY,double *entryH,double *entryZ){
  int nyp=ny+2,nxp=nx+2,n=nyp*nxp;double *W=calloc(n*4,sizeof(double)),*sx=calloc(n*4,sizeof(
  ↪  double)),*sy=calloc(n*4,sizeof(double));
  if(!W||!sx||!sy){free(W);free(sx);free(sy);return 1;}
  for(int p=0;p<n;p++){double h=V[3*p];W[4*p]=h+Z[p];W[4*p+3]=Z[p];if(h>dry){W[4*p+1]=V[3*p+1
  ↪  ]/h;W[4*p+2]=V[3*p+2]/h;}}
  if(order==2){
    for(int i=0;i<nyp;i++)for(int j=1;j<nxp-1;j++){
      int p=i*nxp+j;for(int k=0;k<4;k++)sx[4*p+k]=mm(2*(W[4*p+k]-W[4*(p-1)+k]),.5*(W[4*(p+1)+
      ↪  k]-W[4*(p-1)+k]),2*(W[4*(p+1)+k]-W[4*p+k]));
      double h=V[3*p];if(h<=dry||V[3*(p-1)]<=dry||V[3*(p+1)]<=dry||h<.5*fabs(sx[4*p]-sx[4*p+3
      ↪  ]))for(int
      ↪  k=0;k<4;k++)sx[4*p+k]=0.;
    }
    for(int i=1;i<nyp-1;i++)for(int j=0;j<nxp;j++){
      int p=i*nxp+j;for(int k=0;k<4;k++)sy[4*p+k]=mm(2*(W[4*p+k]-W[4*(p-nxp)+k]),.5*(W[4*(p+n
      ↪  xp)+k]-W[4*(p-nxp)+k]),2*(W[4*(p+nxp)+k]-W[4*p+k]));
      double h=V[3*p];if(h<=dry||V[3*(p-nxp)]<=dry||V[3*(p+nxp)]<=dry||h<.5*fabs(sy[4*p]-sy[4
      ↪  *p+3]))for(int
      ↪  k=0;k<4;k++)sy[4*p+k]=0.;
    }
    if(bc==1||bc==2){
      for(int i=0;i<nyp;i++)for(int k=0;k<4;k++){
```

```
        sx[4*(i*nxp)+k]=sx[4*(i*nxp+(bc==1?1:nx))+k];
        sx[4*(i*nxp+nx+1)+k]=sx[4*(i*nxp+(bc==1?nx:1))+k];
      }
      for(int j=0;j<nxp;j++)for(int k=0;k<4;k++){
        sy[4*j+k]=sy[4*((bc==1?1:ny)*nxp+j)+k];
        sy[4*((ny+1)*nxp+j)+k]=sy[4*((bc==1?ny:1)*nxp+j)+k];
      }
    }
  }
  for(int i=0;i<ny;i++)for(int j=0;j<=nx;j++){
    int p=(i+1)*nxp+j,q=p+1,f=i*(nx+1)+j;double L[3],R[3],zl,zr;
    if(order==2){state(W+4*p,sx+4*p,.5,L,&zl);state(W+4*q,sx+4*q,-.5,R,&zr);}
    else{memcpy(L,V+3*p,3*sizeof(double));memcpy(R,V+3*q,3*sizeof(double));zl=Z[p];zr=Z[q];}
    if(j==0&&bc==3){memcpy(L,R,3*sizeof(double));L[1]*=-1;zl=zr;}
    if(j==nx&&(bc==3||bc==4)){memcpy(R,L,3*sizeof(double));R[1]*=-1;zr=zl;}
    hll(L,R,zl,zr,0,dry,FX+3*f,XL+f,XR+f);
    if(j==0&&bc==3){FX[3*f]=0;FX[3*f+2]=0;FX[3*f+1]=wallp(R[0],R[0]>dry?-R[1]/R[0]:0.);}
    if(j==nx&&(bc==3||bc==4)){FX[3*f]=0;FX[3*f+2]=0;FX[3*f+1]=wallp(L[0],L[0]>dry?L[1]/L[0]:0
    ↪  .);}
    if(j==nx){for(int k=0;k<3;k++)entryH[3*i+k]=L[k];entryZ[i]=zl;}
  }
  for(int i=0;i<=ny;i++)for(int j=0;j<nx;j++){
    int p=i*nxp+j+1,q=p+nxp,f=i*nx+j;double L[3],R[3],zl,zr;
    if(order==2){state(W+4*p,sy+4*p,.5,L,&zl);state(W+4*q,sy+4*q,-.5,R,&zr);}
    else{memcpy(L,V+3*p,3*sizeof(double));memcpy(R,V+3*q,3*sizeof(double));zl=Z[p];zr=Z[q];}
    if(i==0&&bc==3){memcpy(L,R,3*sizeof(double));L[2]*=-1;zl=zr;}
    if(i==ny&&bc==3){memcpy(R,L,3*sizeof(double));R[2]*=-1;zr=zl;}
    hll(L,R,zl,zr,1,dry,FY+3*f,YL+f,YR+f);
    if(i==0&&bc==3){FY[3*f]=0;FY[3*f+1]=0;FY[3*f+2]=wallp(R[0],R[0]>dry?-R[2]/R[0]:0.);}
    if(i==ny&&bc==3){FY[3*f]=0;FY[3*f+1]=0;FY[3*f+2]=wallp(L[0],L[0]>dry?L[2]/L[0]:0.);}
  }
  if(order==2)for(int i=0;i<ny;i++)for(int j=0;j<nx;j++){
    int p=(i+1)*nxp+j+1,f=i*nx+j;double L[3],R[3],zl,zr;
    state(W+4*p,sx+4*p,-.5,L,&zl);state(W+4*p,sx+4*p,.5,R,&zr);SX[f]=-.5*G*(L[0]+R[0])*(zr-zl
    ↪  );
    state(W+4*p,sy+4*p,-.5,L,&zl);state(W+4*p,sy+4*p,.5,R,&zr);SY[f]=-.5*G*(L[0]+R[0])*(zr-zl
    ↪  );
  }
  free(W);free(sx);free(sy);return 0;
}
'''


class FastFlux:
    def __init__(self,build_dir):
        self.sha=hashlib.sha256(SOURCE.encode()).hexdigest();out=Path(build_dir)/self.sha[:16
        ↪  ];out.mkdir(parents=True,exist_ok=True)
        src=out/'flux.c';lib=out/'flux.so'

        if True:
            src.write_text(SOURCE)
            command=['cc','-O3','-std=c99','-shared','-fPIC',str(src),'-o',str(lib),'-lm']
            if sys.platform=='darwin':
                probe=subprocess.run(['xcrun','--show-sdk-path'],capture_output=True,text=Tru
                ↪  e,check=True)
```

```
                default=Path(probe.stdout.strip());candidates=[]
                for candidate in default.parent.glob('MacOSX*.sdk'):
                    match=re.fullmatch(r'MacOSX(\d+)\.(\d+)\.sdk',candidate.name)
                    if match:candidates.append(((int(match[1]),int(match[2])),candidate))


                sdk=min(candidates,key=lambda x:x[0])[1] if candidates else default
                command+=['-isysroot',str(sdk)]
            run=subprocess.run(command,capture_output=True,text=True)
            (out/'build_command.json').write_text(json.dumps(command))
            (out/'build.stdout').write_text(run.stdout);(out/'build.stderr').write_text(run.s
            ↪  tderr)
            if run.returncode:raise RuntimeError('C99 flux compilation failed: '+run.stderr)
        self.command=json.loads((out/'build_command.json').read_text())
        self.library_sha256=hashlib.sha256(lib.read_bytes()).hexdigest()
        self.library=ctypes.CDLL(str(lib.resolve()));self.fn=self.library.fluxes
        ptr=ctypes.POINTER(ctypes.c_double)
        self.fn.argtypes=[ctypes.c_int]*4+[ctypes.c_double]+[ptr]*12
        self.fn.restype=ctypes.c_int

    def fluxes(self,V,Z,order,boundary,east_wall=False):
        if np.iscomplexobj(V) or np.iscomplexobj(Z):raise TypeError('real-valued states
        ↪  required')
        if np.ma.isMaskedArray(V) or np.ma.isMaskedArray(Z):raise TypeError('masked arrays
        ↪  must be resolved before the native boundary')
        V=np.asarray(V,dtype=np.float64,order='C');Z=np.asarray(Z,dtype=np.float64,order='C')
        if V.ndim!=3 or V.shape[-1]!=3 or Z.ndim!=2 or V.shape[:2]!=Z.shape or
        ↪  min(Z.shape)<3:raise ValueError('padded V shape (ny+2,nx+2,3) and matching bed
        ↪  required')
        if not np.isfinite(V).all() or not np.isfinite(Z).all() or np.any(V[...,0]<0):raise
        ↪  ValueError('finite nonnegative-thickness states and finite bed required')
        if order not in (1,2) or boundary not in
        ↪  ('wall','open','outflow','slope','periodic'):raise ValueError('unsupported flux
        ↪  scheme/boundary')
        if not np.isfinite(self.dry) or self.dry<=0:raise ValueError('positive finite dry
        ↪  threshold required')
        if east_wall and boundary in ('slope','periodic'):raise ValueError('inlet combined
        ↪  with slope/periodic is not supported by the native kernel')
        ny,nx=Z.shape[0]-2,Z.shape[1]-2
        fx=np.zeros((ny,nx+1,3));fy=np.zeros((ny+1,nx,3));xl=np.zeros((ny,nx+1));xr=xl.copy()
        ↪  ;yl=np.zeros((ny+1,nx));yr=yl.copy()
        sx=np.zeros((ny,nx));sy=sx.copy();eh=np.zeros((ny,3));ez=np.zeros(ny)
        arrays=[np.ascontiguousarray(V),np.ascontiguousarray(Z),fx,fy,xl,xr,yl,yr,sx,sy,eh,ez]
        bc=3 if boundary=='wall' else (4 if east_wall else (1 if boundary=='slope' else (2 if
        ↪  boundary=='periodic' else 0)))
        status=self.fn(ny,nx,order,bc,self.dry,*[a.ctypes.data_as(ctypes.POINTER(ctypes.c_dou
        ↪  ble)) for a in
        ↪  arrays])
        if status:raise MemoryError('C flux allocation failed')
        left=eh[:,None,:]
        return fx,fy,xl,xr,yl,yr,sx,sy,left,ez[:,None]

import numpy as np
from scipy.linalg import cho_factor,cho_solve
```

```
from scipy.optimize import minimize


def matern52(x,y,amplitude,length):
    r=np.sqrt(np.sum(((np.asarray(x)[:,None,:]-np.asarray(y)[None,:,:])/length)**2,axis=-1))
    a=np.sqrt(5)*r
    return amplitude**2*(1+a+a*a/3)*np.exp(-a)


def morris_design(dimension=4,trajectories=8,levels=6,seed=83031):
    rng=np.random.default_rng(seed);delta=levels/(2*(levels-1));points=[];orders=[]
    for _ in range(trajectories):
        direction=rng.choice([-1,1],dimension)
        base=rng.integers(0,levels//2,size=dimension)/(levels-1)
        x=np.where(direction>0,base,1-base)
        order=rng.permutation(dimension);track=[x.copy()]
        for j in order:
            x=x.copy();x[j]+=delta*direction[j];track.append(x.copy())
        points.append(track);orders.append(order)
    return np.array(points),np.array(orders)


def morris_effects(points,responses):
    points=np.asarray(points);responses=np.asarray(responses);ee=np.full((len(points),points.
    ↪  shape[2]),np.nan)
    for r,trajectory in enumerate(points):
        for j in range(len(trajectory)-1):
            difference=trajectory[j+1]-trajectory[j];changed=np.flatnonzero(abs(difference)>1
            ↪  e-10)
            if len(changed)!=1:raise ValueError('Morris step must change exactly one
            ↪  coordinate')
            d=changed[0];ee[r,d]=(responses[r,j+1]-responses[r,j])/difference[d]
    return ee,{'mu':np.mean(ee,axis=0).tolist(),'mu_star':np.mean(abs(ee),axis=0).tolist(),'s
    ↪  igma':np.std(ee,axis=0,ddof=1).tolist()}


class ARGP:
    def fit(self,xlow,ylow,xhigh,yhigh):
        ylow=np.asarray(ylow);yhigh=np.asarray(yhigh)
        self.xl=np.asarray(xlow);self.xh=np.asarray(xhigh)
        if self.xl.ndim!=2 or self.xh.ndim!=2 or self.xl.shape[1]!=self.xh.shape[1] or
        ↪  len(ylow)!=len(self.xl) or len(yhigh)!=len(self.xh) or not
        ↪  all(np.isfinite(v).all() for v in [self.xl,self.xh,ylow,yhigh]):raise
        ↪  ValueError('finite matching low/high GP data required')
        self.ml=float(np.mean(ylow));self.mh=float(np.mean(yhigh))
        self.scale=max(float(np.std(np.r_[ylow,yhigh])),1e-3)
        self.y=np.r_[(ylow-self.ml)/self.scale,(yhigh-self.mh)/self.scale]
        self.nL=len(xlow);self.dim=self.xl.shape[1];self.noise=1e-8
        def unpack(theta):
            k=self.dim
            return np.exp(theta[0]),np.exp(theta[1:k+1]),np.exp(theta[k+1]),np.exp(theta[k+2:
            ↪  2*k+2]),theta[-1]
        self.unpack=unpack
        def matrix(theta):
```

```
        al,ll,ad,ld,rho=unpack(theta)
        Lll=matern52(self.xl,self.xl,al,ll)
        Llh=rho*matern52(self.xl,self.xh,al,ll)
        Hhh=rho*rho*matern52(self.xh,self.xh,al,ll)+matern52(self.xh,self.xh,ad,ld)
        return np.block([[Lll,Llh],[Llh.T,Hhh]])+np.eye(len(self.y))*self.noise
    self.matrix=matrix
    def loss(theta):
        try:
            cf=cho_factor(matrix(theta),lower=True)
            return .5*self.y@cho_solve(cf,self.y)+np.log(np.diag(cf[0])).sum()+.5*len(sel
            ↪  f.y)*np.log(2*np.pi)
        except np.linalg.LinAlgError:return 1e30
    bounds=[(-3,3)]+[(-3,1)]*self.dim+[(-4,3)]+[(-3,1)]*self.dim+[(-3,3)]
    starts=[np.r_[0,np.full(self.dim,np.log(l)),np.log(.5),np.full(self.dim,np.log(l)),1.
    ↪  ] for l in
    ↪  (.15,.4,.8)]
    attempts=[minimize(loss,s,method='L-BFGS-B',bounds=bounds,options={'maxiter':100,'fto
    ↪  l':1e-8}) for s in
    ↪  starts]
    valid=[]
    for attempt in attempts:
        if not np.isfinite(attempt.x).all() or not np.isfinite(attempt.fun) or
        ↪  attempt.fun>=1e29:continue
        try:
            K=matrix(attempt.x)
            if np.isfinite(K).all():cho_factor(K,lower=True);valid.append(attempt)
        except np.linalg.LinAlgError:pass
    if not valid:raise ArithmeticError('no finite SPD multi-fidelity GP optimizer
    ↪  iterate')
    converged=[a for a in valid if a.success];best=min(converged or valid,key=lambda
    ↪  a:a.fun);self.theta=best.x
    self.cf=cho_factor(matrix(best.x),lower=True);self.alpha=cho_solve(self.cf,self.y)
    self.fit_record={'theta':self.theta.tolist(),'rho':float(self.theta[-1]),'jitter_norm
    ↪  alized_variance':self.noise,'covariance_condition_number':float(np.linalg.cond(ma
    ↪  trix(best.x))),
        'mean_L':self.ml,'mean_H':self.mh,'output_scale':self.scale,'negative_log_likelih
        ↪  ood':float(best.fun),
        'optimizer_attempts':[{'success':bool(a.success),'fun':float(a.fun) if
        ↪  np.isfinite(a.fun) else None,'message':str(a.message)} for a in attempts],
        'fit_status':'converged_MLE' if converged else 'finite_nonconverged_fallback',
        'fallback_policy':'prefer finite SPD converged starts; if none, lowest finite SPD
        ↪  iterate flagged, no test-based retries',
        'uncertainty_scope':'latent output conditional on fitted hyperparameters; no
        ↪  hyperparameter or natural-event uncertainty',
        'latent_models':'independent L and delta prior; full joint posterior
        ↪  cross-covariance retained'}
    return self

def cross(self,x,level):
    al,ll,ad,ld,rho=self.unpack(self.theta)
    if level=='L':return np.c_[matern52(x,self.xl,al,ll),rho*matern52(x,self.xh,al,ll)]
    return np.c_[rho*matern52(x,self.xl,al,ll),rho*rho*matern52(x,self.xh,al,ll)+matern52
    ↪  (x,self.xh,ad,ld)]
```

```
    def prior(self,x,y,level_x='H',level_y='H'):
        al,ll,ad,ld,rho=self.unpack(self.theta)
        k=matern52(x,y,al,ll)
        if level_x==level_y=='L':return k
        if level_x!=level_y:return rho*k
        return rho*rho*k+matern52(x,y,ad,ld)

    def predict(self,x,level='H'):
        c=self.cross(x,level);mean=c@self.alpha
        var=np.diag(self.prior(x,x,level,level))-np.sum(c*cho_solve(self.cf,c.T).T,axis=1)
        if var.min() < -1e-7:raise ArithmeticError('negative GP posterior variance')
        return (self.ml if level=='L' else
        ↪  self.mh)+self.scale*mean,self.scale*np.sqrt(np.maximum(0,var))

    def posterior_cross(self,x,y,level_x='H',level_y='H'):
        return self.scale**2*(self.prior(x,y,level_x,level_y)-self.cross(x,level_x)@cho_solve
        ↪  (self.cf,self.cross(y,level_y).T))

    def acquisition(self,candidates,integration_grid,cost_low,cost_high,explore=False):
        m,s=self.predict(integration_grid)
        w=np.ones_like(m) if explore else np.exp(-.5*(m/np.maximum(s,1e-8))**2)
        w=w/max(w.sum(),1e-30)
        result={}
        for level,cost in [('L',cost_low),('H',cost_high)]:
            _,sq=self.predict(candidates,level)
            cov=self.posterior_cross(integration_grid,candidates,'H',level)
            result[level]=np.sum(w[:,None]*cov**2,axis=0)/(sq**2+self.noise*self.scale**2)/co
            ↪  st
        return result


def minmod3(a,b,c):
    return
    ↪  np.where((a*b>0)&(a*c>0),np.sign(a)*np.minimum(np.minimum(abs(a),abs(b)),abs(c)),0.)


def exact_wall_pressure(h,w):


    h=np.asarray(h,dtype=float);w=np.broadcast_to(np.asarray(w,dtype=float),h.shape)
    c=np.sqrt(G*h);cs=np.maximum(0,c+.5*w);hs=np.array(cs*cs/G,copy=True)
    positive=(w>0)&(h>0)
    if np.any(positive):
        f=w[positive]/c[positive];lo=np.ones_like(f);hi=1+2*f+f*f
        for _ in range(60):
            r=.5*(lo+hi);v=(r-1)*np.sqrt((r+1)/(2*r));lo=np.where(v<f,r,lo);hi=np.where(v>=f,
            ↪  r,hi)
        hs[positive]=h[positive]*.5*(lo+hi)
    return np.where(h>0,.5*G*hs*hs,0.)


def muscl_faces(V,Z,boundary):
    h=V[...,0];vel=np.divide(V[...,1:],h[...,None],out=np.zeros_like(V[...,1:]),where=h[...,N
    ↪  one]>DRY)
```

```
    W=np.concatenate([(h+Z)[...,None],vel,Z[...,None]],axis=-1)
    sx=np.zeros_like(W);sy=np.zeros_like(W)
    sx[:,1:-1]=minmod3(2*(W[:,1:-1]-W[:,:-2]),.5*(W[:,2:]-W[:,:-2]),2*(W[:,2:]-W[:,1:-1]))
    sy[1:-1]=minmod3(2*(W[1:-1]-W[:-2]),.5*(W[2:]-W[:-2]),2*(W[2:]-W[1:-1]))

    badx=np.zeros_like(h,dtype=bool);bady=np.zeros_like(h,dtype=bool)
    badx[:,1:-1]=(h[:,1:-1]<=DRY)|(h[:,:-2]<=DRY)|(h[:,2:]<=DRY)
    bady[1:-1]=(h[1:-1]<=DRY)|(h[:-2]<=DRY)|(h[2:]<=DRY)
    badx|=h<.5*abs(sx[...,0]-sx[...,3]);bady|=h<.5*abs(sy[...,0]-sy[...,3])
    sx[badx]=0.;sy[bady]=0.
    if boundary=='slope':
        sx[:,0]=sx[:,1];sx[:,-1]=sx[:,-2];sy[0]=sy[1];sy[-1]=sy[-2]
    if boundary=='periodic':
        sx[:,0]=sx[:,-2];sx[:,-1]=sx[:,1];sy[0]=sy[-2];sy[-1]=sy[1]
    def state(A):
        hh=np.maximum(0,A[...,0]-A[...,3])
        return np.concatenate([hh[...,None],hh[...,None]*A[...,1:3]],axis=-1),A[...,3]
    left,zleft=state(W[1:-1,:-1]+.5*sx[1:-1,:-1]);right,zright=state(W[1:-1,1:]-.5*sx[1:-1,1:
    ↪  ])
    top,ztop=state(W[:-1,1:-1]+.5*sy[:-1,1:-1]);bottom,zbottom=state(W[1:,1:-1]-.5*sy[1:,1:-1
    ↪  ])
    wp=W[1:-1,1:-1];spx=sx[1:-1,1:-1];spy=sy[1:-1,1:-1]
    um,zm=state(wp-.5*spx);up,zp=state(wp+.5*spx)
    vm,ym=state(wp-.5*spy);vp,yp=state(wp+.5*spy)
    source_x=-.5*G*(um[...,0]+up[...,0])*(zp-zm)
    source_y=-.5*G*(vm[...,0]+vp[...,0])*(yp-ym)
    return left,right,zleft,zright,top,bottom,ztop,zbottom,source_x,source_y


def hll_pair(left, right, zl, zr, axis):

    hL, hR = left[..., 0], right[..., 0]
    uL = np.divide(left[..., 1:], hL[..., None], out=np.zeros_like(left[..., 1:]),
    ↪  where=hL[..., None] > DRY)
    uR = np.divide(right[..., 1:], hR[..., None], out=np.zeros_like(right[..., 1:]),
    ↪  where=hR[..., None] > DRY)
    zs = np.maximum(zl, zr)
    hl, hr = np.maximum(0, hL + zl - zs), np.maximum(0, hR + zr - zs)
    UL = np.concatenate([hl[..., None], hl[..., None]*uL], axis=-1)
    UR = np.concatenate([hr[..., None], hr[..., None]*uR], axis=-1)
    nl, nr = uL[..., axis], uR[..., axis]
    cl, cr = np.sqrt(G*hl), np.sqrt(G*hr)
    sl = np.minimum(nl-cl, nr-cr)
    sr = np.maximum(nl+cl, nr+cr)

    sl = np.where(hr <= 0., nl-cl, sl)
    sr = np.where(hr <= 0., nl+2*cl, sr)
    sl = np.where(hl <= 0., nr-2*cr, sl)
    sr = np.where(hl <= 0., nr+cr, sr)
    FL, FR = UL*nl[..., None], UR*nr[..., None]
    FL[..., axis+1] += .5*G*hl**2
    FR[..., axis+1] += .5*G*hr**2
    den = sr-sl
    middle = np.divide(sr[..., None]*FL-sl[..., None]*FR+(sl*sr)[..., None]*(UR-UL),
```

```
                      den[..., None], out=np.zeros_like(UL), where=den[..., None] > DRY)
    flux = np.where((sl >= 0)[..., None], FL, np.where((sr <= 0)[..., None], FR, middle))
    return flux, .5*G*(hL*hL-hl*hl), .5*G*(hR*hR-hr*hr)


def triangle_cdf(t, duration):
    r=np.clip(np.asarray(t)/duration,0,1)
    return np.where(r<=.5,2*r*r,1-2*(1-r)**2)


def pulse_volume(t0,t1,volume,duration,shape='triangle'):
    if shape=='double':

        half=duration/2
        return volume*.5*(triangle_cdf(t1,half)-triangle_cdf(t0,half)+
                          triangle_cdf(t1-half,half)-triangle_cdf(t0-half,half))
    return volume*(triangle_cdf(t1,duration)-triangle_cdf(t0,duration))


def euler_step(U,z,dx,dy,dt,mu=0.,xi=1e30,boundary='wall',inlet=None,t=0.):
    V=np.pad(U,((1,1),(1,1),(0,0)),mode='edge')
    Z=np.pad(z,((1,1),(1,1)),mode='edge')
    if boundary=='wall':
        V[:,0,1]*=-1;V[:,-1,1]*=-1;V[0,:,2]*=-1;V[-1,:,2]*=-1
    elif boundary=='slope':
        Z[:,0]=2*Z[:,1]-Z[:,2];Z[:,-1]=2*Z[:,-2]-Z[:,-3]
        Z[0]=2*Z[1]-Z[2];Z[-1]=2*Z[-2]-Z[-3]
    elif boundary=='periodic':
        V=np.pad(U,((1,1),(1,1),(0,0)),mode='wrap');Z=np.pad(z,((1,1),(1,1)),mode='wrap')
    elif boundary=='outflow':


        V[:,0]=0.;V[:,-1]=0.;V[0]=0.;V[-1]=0.
    elif boundary!='open':raise ValueError(boundary)
    if inlet is not None:


        V[1:-1,-1]=U[:,-1];V[1:-1,-1,1]*=-1
    if FAST is not None:
        fx,fy,xl,xr,yl,yr,source_x,source_y,left,zleft=FAST.fluxes(V,Z,ORDER,boundary,inlet
        ↪  is not None)
    else:
        source_x=np.zeros_like(z);source_y=np.zeros_like(z)
        if ORDER==2:
            left,right,zleft,zright,top,bottom,ztop,zbottom,source_x,source_y=muscl_faces(V,Z」
            ↪  ,boundary)
        else:
            left,right,zleft,zright=V[1:-1,:-1],V[1:-1,1:],Z[1:-1,:-1],Z[1:-1,1:]
            top,bottom,ztop,zbottom=V[:-1,1:-1],V[1:,1:-1],Z[:-1,1:-1],Z[1:,1:-1]


        if boundary=='wall':
            left[:,0]=right[:,0];left[:,0,1]*=-1;zleft[:,0]=zright[:,0]
            top[0]=bottom[0];top[0,:,2]*=-1;ztop[0]=zbottom[0]
```

```
        bottom[-1]=top[-1];bottom[-1,:,2]*=-1;zbottom[-1]=ztop[-1]
    if boundary=='wall' or inlet is not None:
        right[:,-1]=left[:,-1];right[:,-1,1]*=-1;zright[:,-1]=zleft[:,-1]
    fx,xl,xr=hll_pair(left,right,zleft,zright,0)
    fy,yl,yr=hll_pair(top,bottom,ztop,zbottom,1)
    if boundary=='wall':
        vx=np.divide(right[:,0,1],right[:,0,0],out=np.zeros(len(z)),where=right[:,0,0]>DR
        ↪ Y)
        fx[:,0]=0.;fx[:,0,1]=exact_wall_pressure(right[:,0,0],-vx)
        vy=np.divide(bottom[0,:,2],bottom[0,:,0],out=np.zeros(z.shape[1]),where=bottom[0,
        ↪ :,0]>DRY)
        fy[0]=0.;fy[0,:,2]=exact_wall_pressure(bottom[0,:,0],-vy)
        vy=np.divide(top[-1,:,2],top[-1,:,0],out=np.zeros(z.shape[1]),where=top[-1,:,0]>D
        ↪ RY)
        fy[-1]=0.;fy[-1,:,2]=exact_wall_pressure(top[-1,:,0],vy)
    if boundary=='wall' or inlet is not None:
        vx=np.divide(left[:,-1,1],left[:,-1,0],out=np.zeros(len(z)),where=left[:,-1,0]>DR
        ↪ Y)
        fx[:,-1]=0.;fx[:,-1,1]=exact_wall_pressure(left[:,-1,0],vx)
vin=0.;fr=None;energy_in=0.;pressure_in=0.;inlet_old_flux=None
if inlet is not None:
    weights=np.asarray(inlet.get('weights',inlet['rows']),dtype=float)
    mask=weights>0;speed=inlet['speed'];width=float(weights.sum()*dy)
    vin=float(pulse_volume(t,t+dt,inlet['volume'],inlet['duration'],inlet.get('shape','tr
    ↪ iangle')))
    q=vin/(dt*width);h=q/speed
    fr=speed/math.sqrt(G*h) if h>0 else None
    if h>0 and fr<=1:raise ArithmeticError('inlet not supercritical')

    inlet_old_flux=fx[mask,-1].copy()
    incoming=np.zeros_like(inlet_old_flux);incoming[:,0]=-q;incoming[:,1]=q*speed+.5*G*h*h
    hface2=[];wall_pressure=[]
    for j in np.flatnonzero(mask):
        bednodes=inlet.get('face_bed_samples',{}).get(int(j),np.array([zleft[j,-1]]))
        hh=np.maximum(0,left[j,-1,0]+zleft[j,-1]-bednodes)
        hface2.append(float(np.mean(hh**2)))
        if vin==0:
            vv=left[j,-1,1:]/left[j,-1,0] if left[j,-1,0]>DRY else np.zeros(2)
            wall_pressure.append(float(exact_wall_pressure(hh,vv[0]).mean()))
    hface2=np.array(hface2)
    if vin==0:


        incoming[:,1]=wall_pressure
    fx[mask,-1]=inlet_old_flux*(1-weights[mask,None])+incoming*weights[mask,None]
    pressure_in=dt*dy*float(np.sum((incoming[:,1]-q*speed)*weights[mask]))


    xl[mask,-1]=.5*G*(left[mask,-1,0]**2-hface2)*weights[mask]
    xr[mask,-1]=0.
    zin=inlet.get('bed_mean',float(np.average(z[mask,-1],weights=weights[mask])))
    energy_in=vin*(.5*speed**2+G*(zin-float(z.min())+h))
bflux=dy*(fx[:,-1].sum(0)-fx[:,0].sum(0))+dx*(fy[-1].sum(0)-fy[0].sum(0))
```

```
    def head(A,B):
        h=A[...,0];v=np.divide(A[...,1:],h[...,None],out=np.zeros_like(A[...,1:]),where=h[...
        ↪  ,None]>DRY)
        return .5*(v*v).sum(-1)+G*(B+h)
    H=head(U,z-float(z.min()))
    eb=dy*(np.sum(fx[:,-1,0]*H[:,-1])-np.sum(fx[:,0,0]*H[:,0]))+dx*(np.sum(fy[-1,:,0]*H[-1])-
    ↪  np.sum(fy[0,:,0]*H[0]))
    if inlet is not None:


        eb-=dy*float(np.sum(-q*weights[mask]*H[mask,-1]))
        eb-=energy_in/dt
    slope=np.zeros_like(U)
    slope[...,1]=-(xl[:,1:]-xr[:,:-1])/dx+source_x/dx
    slope[...,2]=-(yl[1:]-yr[:-1])/dy+source_y/dy
    result=U-dt/dx*(fx[:,1:]-fx[:,:-1])-dt/dy*(fy[1:]-fy[:-1])+dt*slope
    if result[...,0].min() < 0 or not np.isfinite(result).all():
        raise ArithmeticError('negative thickness or nonfinite state; no clipping')
    pre=result[...,1:].copy();h=result[...,0]
    vel=np.divide(pre,h[...,None],out=np.zeros_like(pre),where=h[...,None]>DRY)
    speed=np.linalg.norm(vel,axis=-1)
    r=np.maximum(0,speed-dt*mu*G)
    a=np.divide(dt*G,xi*h,out=np.zeros_like(h),where=h>DRY)
    newspeed=2*r/(1+np.sqrt(1+4*a*r))
    ratio=np.ones_like(speed)
    ratio[h>DRY]=np.divide(newspeed,speed,out=np.zeros_like(speed),where=speed>0)[h>DRY]
    result[...,1:]*=ratio[...,None]
    friction_impulse=(result[...,1:]-pre).sum((0,1))*dx*dy
    ke_before=.5*np.divide((pre*pre).sum(-1),h,out=np.zeros_like(h),where=h>0).sum()*dx*dy
    ke_after=.5*np.divide((result[...,1:]*result[...,1:]).sum(-1),h,out=np.zeros_like(h),wher
    ↪  e=h>0).sum()*dx*dy
    before_clear=result[...,1:].copy()
    result[...,1:][h<=DRY]=0
    clear=(result[...,1:]-before_clear).sum((0,1))*dx*dy
    ke_after_clear=.5*np.divide((result[...,1:]*result[...,1:]).sum(-1),h,out=np.zeros_like(h
    ↪  ),where=h>0).sum()*dx*dy
    return result,{'boundary':dt*bflux,'bed':dt*slope[...,1:].sum((0,1))*dx*dy,
                   'friction':friction_impulse,'dry_clear':clear,'vin':vin,'Fr':fr,'energy_in
                   ↪  ':energy_in,
                   'energy_boundary':dt*eb,'friction_energy_loss':float(ke_before-ke_after),
                   'dry_clear_energy_loss':float(ke_after-ke_after_clear),
                   'boundary_faces':dt*np.concatenate([-dy*fx[:,0],dy*fx[:,-1],-dx*fy[0],dx*f
                   ↪  y[-1]],axis=0),
                   'inlet_advective_impulse':vin*inlet['speed'] if inlet is not None else 0.,
                   'prescribed_inflow_pressure_impulse':pressure_in if vin>0 else 0.,
                   'closed_gate_pressure_impulse':pressure_in if vin==0 else 0.,
                   'inlet_pressure_impulse':pressure_in}


def step(U,z,dx,dy,dt,mu=0.,xi=1e30,boundary='wall',inlet=None,t=0.):
    one,b1=euler_step(U,z,dx,dy,dt,mu,xi,boundary,inlet,t)
    if ORDER==1:return one,b1
    two,b2=euler_step(one,z,dx,dy,dt,mu,xi,boundary,inlet,t)
```

```
    result=.5*U+.5*two
    budget={k:.5*(b1[k]+b2[k]) for k in b1 if k!='Fr'}


    h=result[...,0];dry=h<=DRY;before=result[...,1:].copy()
    extra_energy=.5*np.divide((before*before).sum(-1),h,out=np.zeros_like(h),where=(h>0)&dry)
    .sum()*dx*dy
    result[...,1:][dry]=0.
    budget['dry_clear']+=(result[...,1:]-before).sum((0,1))*dx*dy
    budget['dry_clear_energy_loss']+=float(extra_energy)
    fs=[v for v in [b1['Fr'],b2['Fr']] if v is not None];budget['Fr']=min(fs) if fs else None
    return result,budget


def energy(U,z,area):
    h=U[...,0];v=np.divide(U[...,1:],h[...,None],out=np.zeros_like(U[...,1:]),where=h[...,Non
    e]>DRY)
    return float((.5*h*(v*v).sum(-1)+.5*G*h*h+G*h*z).sum()*area)


def evolve(U,z,dx,dy,end,mu=0.,xi=1e30,boundary='wall',cfl=.35,maxdt=1.,inlet=None,target=Non
e,save_every=30.,trace_path=None,legal_exit_mask=None,exit_threshold_m3=None):
    started=time.perf_counter();t=0.;steps=0;area=dx*dy
    initial=U.sum((0,1))*area;boundary_sum=np.zeros(3);bed=np.zeros(2);friction=np.zeros(2);c
    lear=np.zeros(2)
    zenergy=z-float(z.min())
    vin=0.;ein=0.;ebound=0.;edrag=0.;edry=0.;pinadv=0.;pinpressure=0.;einitial=energy(U,zener
    gy,area);frmin=math.inf;masserr=0.;momerr=0.;minh=float(U[...,0].min())
    ledgers=[];faces=[];vin_all=0.;vout_all=0.;max_energy=einitial;pin_flow=0.;pin_gate=0.
    side_out=np.zeros(4);illegal_out=0.;first_illegal=None
    if exit_threshold_m3 is None:exit_threshold_m3=1e-6*max(initial[0]+(inlet['volume'] if
    inlet is not None else 0),1)
    hmax=U[...,0].copy();vmax=np.zeros_like(z);event_vmax=np.zeros_like(z);snap=[];series=[];
    nextsave=0.
    while t<end-1e-10:
        h=U[...,0];vel=np.divide(U[...,1:],h[...,None],out=np.zeros_like(U[...,1:]),where=h[.
        ..,None]>DRY)
        c=np.sqrt(G*h);rate=float(np.max((abs(vel[...,0])+c)/dx+(abs(vel[...,1])+c)/dy))
        if inlet is not None:
            w=np.asarray(inlet.get('weights',inlet['rows']),dtype=float).sum()*dy
            hpeak=2*inlet['volume']/inlet['duration']/w/inlet['speed']
            cp=math.sqrt(G*hpeak)
            rate=max(rate,(inlet['speed']+cp)/dx+cp/dy)
        dt=min(maxdt,end-t,cfl/rate if rate>0 else maxdt)
        U,b=step(U,z,dx,dy,dt,mu,xi,boundary,inlet,t)
        t+=dt;steps+=1;boundary_sum+=b['boundary'];bed+=b['bed'];friction+=b['friction'];clea
        r+=b['dry_clear']
        vin+=b['vin'];ein+=b['energy_in']
        ebound+=b['energy_boundary'];edrag+=b['friction_energy_loss']
        edry+=b['dry_clear_energy_loss']
        pinadv+=b['inlet_advective_impulse'];pinpressure+=b['inlet_pressure_impulse']
        pin_flow+=b['prescribed_inflow_pressure_impulse'];pin_gate+=b['closed_gate_pressure_i
        mpulse']
```

```
    vin_all+=float(-np.minimum(b['boundary_faces'][:,0],0).sum());vout_all+=float(np.maxi
    ↪  mum(b['boundary_faces'][:,0],0).sum())
    if b['Fr'] is not None:frmin=min(frmin,b['Fr'])
    total=U.sum((0,1))*area
    bf=b['boundary_faces'];cuts=[0,len(z),2*len(z),2*len(z)+z.shape[1],2*len(z)+2*z.shape
    ↪  [1]]
    for k in range(4):side_out[k]+=float(np.maximum(bf[cuts[k]:cuts[k+1],0],0).sum())
    if legal_exit_mask is not None:
        illegal_out+=float(np.maximum(bf[~legal_exit_mask,0],0).sum())
        if first_illegal is None and illegal_out>exit_threshold_m3:first_illegal=t
    et=energy(U,zenergy,area);max_energy=max(max_energy,et)
    if trace_path is not None:
        faces.append(b['boundary_faces'])
        ledgers.append([t,dt,*total,*b['bed'],*b['friction'],*b['dry_clear'],et,b['energy
        ↪  _boundary'],b['friction_energy_loss'],b['dry_clear_energy_loss'],b['vin'],b['
        ↪  Fr'] or
        ↪  0.,b['inlet_advective_impulse'],b['prescribed_inflow_pressure_impulse'],b['cl
        ↪  osed_gate_pressure_impulse'],b['energy_in']])
    masserr=max(masserr,abs(total[0]-initial[0]+boundary_sum[0]))
    momerr=max(momerr,float(np.max(abs(total[1:]-initial[1:]+boundary_sum[1:]-bed-frictio
    ↪  n-clear))))
    minh=min(minh,float(U[...,0].min()));hmax=np.maximum(hmax,U[...,0])
    v=np.divide(U[...,1:],U[...,0,None],out=np.zeros_like(U[...,1:]),where=U[...,0,None]>
    ↪  DRY)
    vmax=np.maximum(vmax,np.linalg.norm(v,axis=-1))
    event_vmax=np.maximum(event_vmax,np.where(U[...,0]>=.1,np.linalg.norm(v,axis=-1),0.))
    target_q=0.;peak_east=0.;peak_north=0.;peak_tangent=0.
    if isinstance(target,dict):
        line=sum(w*(U[j,:,:] if target.get('axis')=='y' else U[:,j,:]) for w,j in
        ↪  zip(target['weights'],target['columns']))
        vals=line[:,0]
        target_h=float(vals[target['rows']].max())
        active=np.flatnonzero(target['rows']);k=int(active[np.argmax(vals[active])])
        if vals[k]>DRY:peak_east=float(line[k,1]/vals[k]);peak_north=float(-line[k,2]/val
        ↪  s[k])
        lengths=target.get('quadrature_lengths',target['rows']*(dx if
        ↪  target.get('axis')=='y' else dy))
        normal=2 if target.get('axis')=='y' else 1
        target_q=float(np.sum(line[:,normal]*lengths)*target.get('normal_sign',1))
        peak_tangent=float(target.get('tangent_coordinates',np.arange(len(line)))[k])
    else:target_h=float(U[...,0][target].max()) if target is not None else 0.
    series.append((t,target_h,float(total[0]),float(np.linalg.norm(U[...,1:],axis=-1).sum
    ↪  ()*area),target_q,peak_east,peak_north,peak_tangent))
    if t>=nextsave or t>=end-1e-10:
        snap.append((t,U.copy()));nextsave+=save_every
    if steps>100000:raise RuntimeError('step budget exceeded')
total=U.sum((0,1))*area
diag={'steps':steps,'wall_s':time.perf_counter()-started,'min_h':minh,'mass_error_abs_m3'
↪  :masserr,
      'mass_error_rel':masserr/max(initial[0]+vin,1),'momentum_error_abs':momerr,
      'momentum_error_rel':momerr/max(float(np.linalg.norm(bed))+float(np.linalg.norm(bou
      ↪  ndary_sum[1:])),1),
      'initial':initial.tolist(),'final':total.tolist(),'boundary_integral':boundary_sum.
      ↪  tolist(),
```

```
            'bed_impulse':bed.tolist(),'friction_impulse':friction.tolist(),'dry_clear_impulse'
            ↪  :clear.tolist(),
            'inlet_volume_m3':vin,'min_inlet_Fr':frmin if math.isfinite(frmin) else None,
            'total_inflow_m3':vin_all,'total_outflow_m3':vout_all,'dry_threshold_m':DRY,
            'outflow_by_side_m3':dict(zip(['west','east','north','south'],side_out.tolist())),
            'illegal_outflow_m3':illegal_out,'first_significant_illegal_outflow_s':first_illega
            ↪  l,'illegal_outflow_threshold_m3':exit_threshold_m3,
            'whole_window_valid':first_illegal is None,'valid_until_s':first_illegal if
            ↪  first_illegal is not None else end,
            'inlet_advective_impulse_west_per_density':pinadv,'inlet_pressure_impulse_west_per_
            ↪  density':pinpressure,
            'prescribed_inflow_pressure_impulse_west_per_density':pin_flow,'closed_gate_pressur
            ↪  e_impulse_west_per_density':pin_gate,
            'legacy_inlet_pressure_definition':'total inlet-strip pressure including post-pulse
            ↪  closed gate; not solely prescribed inflow',
            'energy_in_per_density':ein,'energy_final_per_density':energy(U,zenergy,area),'ener
            ↪  gy_reference_elevation_m':float(z.min()),
            'energy_initial_per_density':einitial,'energy_boundary_outward_per_density':ebound,
            'friction_energy_loss_per_density':edrag,
            'dry_clear_energy_loss_per_density':edry,'maximum_energy_per_density':max_energy,
            'energy_residual_per_density':energy(U,zenergy,area)-einitial+ebound+edrag+edry,
            'energy_status':'first-order boundary head flux; residual combines transport
            ↪  dissipation, bed discretization and source splitting; no exact entropy claim',
            'independent_review':'pending'}
        if trace_path is not None:
            np.savez_compressed(trace_path,ledger=ledgers,boundary_faces=faces,initial=initial,in
            ↪  itial_energy=einitial,
                columns='t,dt,V,Px,Psouth,bed_x,bed_south,drag_x,drag_south,clear_x,clear_south,E
                ↪  ,Eboundary,Eloss_drag,Eloss_clear,Vin,Fr,Pin_adv,Pin_flow_pressure,Pin_gate_p
                ↪  ressure,Ein',
                face_order='west,east,north,south; each outward and time-integrated; momentum per
                ↪  density',dx=dx,dy=dy)
        return U,diag,np.array(series),snap,hmax,vmax,event_vmax


    def fp(p):
        import hashlib
        p=Path(p);return {'path':str(p),'bytes':p.stat().st_size,'sha256':hashlib.sha256(p.read_b
        ↪  ytes()).hexdigest()}


    def write_json(p,data):
        Path(p).write_text(json.dumps(data,ensure_ascii=False,indent=2,allow_nan=False)+'\n')


    def runtime_versions():
        from importlib.metadata import version,PackageNotFoundError
        result={}
        for name in ('numpy','scipy','matplotlib','pillow','rasterio','pyproj'):
            try:result[name]=version(name)
            except PackageNotFoundError:result[name]=None
        return result
```

```
def benchmarks(out):
    checks=[]
    def add(name,ok,**data):
        row={'test':name,'passed':bool(ok),**data};checks.append(row);print(json.dumps(row),f
        ↪  lush=True)
    time_grid=np.linspace(0,960,20001);dt=time_grid[1]-time_grid[0]
    single=np.asarray(pulse_volume(time_grid[:-1],time_grid[1:],8e5,960,'triangle'))/dt
    double=np.asarray(pulse_volume(time_grid[:-1],time_grid[1:],8e5,960,'double'))/dt
    volume_error=float(abs(single.sum()-double.sum())*dt/8e5)
    pressure_error=float(abs(np.sum(single**2)-np.sum(double**2))/np.sum(single**2))
    add('equal_volume_support_peak_double_pulse',volume_error<1e-10 and pressure_error<1e-6
    ↪  and abs(single.max()-double.max())/single.max()<1e-3,
        volume_relative_difference=volume_error,squared_flux_integral_relative_difference=pre
        ↪  ssure_error,total_support_s=960,design='two V/2,T/2 triangles adjacent in
        ↪  [0,T]')
    x,y=np.meshgrid(np.linspace(-2,2,41),np.linspace(-2,2,37));z=.4*np.cos(x)*np.cos(y)+.08*x
    ↪  +.1*y
    U=np.zeros((*z.shape,3));U[...,0]=np.maximum(0,.25-z);old=U.copy()
    U,d,*_=evolve(U,z,1,1,10,maxdt=.1)
    dh=float(abs(U[...,0]-old[...,0]).max());speed=float(np.linalg.norm(np.divide(U[...,1:],U
    ↪  [...,0,None],out=np.zeros_like(U[...,1:]),where=U[...,0,None]>DRY),axis=-1).max())
    add('lake_wet_dry',dh<1e-9 and speed<1e-8,depth_error=dh,speed=speed,diagnostics=d)
    errs=[]
    for dx in (10.,5.,2.5):
        x=np.arange(-1000+dx/2,1000,dx);z=np.zeros((1,len(x)));U=np.zeros((*z.shape,3));U[0,:
        ↪  ,0]=np.where(x<0,10,0)
        U,d,*_=evolve(U,z,dx,1e8,20,boundary='open')
        c=math.sqrt(G*10);r=x/20;truth=np.where(r<-c,10,np.where(r<=2*c,(2*c-r)**2/(9*G),0))
        err=float(abs(U[0,:,0]-truth).sum()/truth.sum());errs.append(err)
        np.savez(out/f'ritter_{dx:g}.npz',x=x,h=U[0,:,0],exact=truth)
        add(f'ritter_{dx:g}',err<.15,error=err,diagnostics=d)
    add('ritter_convergence',all(a>b for a,b in zip(errs,errs[1:])) and
    ↪  errs[-1]<.08,errors=errs)
    xx,yy=np.meshgrid(np.arange(-20,21),np.arange(-20,21));z=np.zeros(xx.shape);U=np.zeros((*
    ↪  z.shape,3));U[...,0]=np.where(xx*xx+yy*yy<25,2,0)
    U,d,*_=evolve(U,z,1,1,3,trace_path=out/'dry_spread_budget.npz')
    closed_mass_error=abs(d['final'][0]-d['initial'][0])/d['initial'][0]
    add('dry_spread',d['mass_error_rel']<1e-9 and d['min_h']>=0 and closed_mass_error<1e-10
    ↪  and d['total_inflow_m3']<1e-10 and d['total_outflow_m3']<1e-10,
        diagnostics=d,closed_storage_relative_error=closed_mass_error,symmetry=float(abs(U[..
        ↪  .,0]-U[...,0].T).max()))
    add('closed_flat_no_energy_creation',d['maximum_energy_per_density']<=d['energy_initial_p
    ↪  er_density']*(1+1e-8),initial_energy=d['energy_initial_per_density'],maximum_energy=d
    ↪  ['maximum_energy_per_density'])

    slope=.08;mu=.03;xi=500.;depth=2.;T=4.;A=G*(slope-mu);B=G/(xi*depth)
    exact=math.sqrt(A/B)*math.tanh(math.sqrt(A*B)*T);errs=[]
    for dx in (.5,.25,.125):
        z=np.broadcast_to(-slope*np.arange(16)*dx,(5,16)).copy();U=np.zeros((*z.shape,3));U[.
        ↪  ..,0]=depth
        U,d,*_=evolve(U,z,dx,dx,T,mu,xi,'slope',maxdt=dx*.02)
        v=float((U[...,1]/U[...,0]).mean());err=abs(v-exact)/exact;errs.append(err)
        add(f'slope_integrated_{dx:g}',err<.1,speed=v,exact=exact,error=err,diagnostics=d)
    add('slope_convergence',all(a>b for a,b in zip(errs,errs[1:])) and
    ↪  errs[-1]<.03,errors=errs)
```

```
from scipy.integrate import solve_ivp
for label,v0 in [('terminal',math.sqrt(A/B)),('superterminal',15.)]:
    dx=.125;z=np.broadcast_to(-slope*np.arange(16)*dx,(5,16)).copy();U=np.zeros((*z.shape
    ↪  ,3));U[...,0]=depth;U[...,1]=depth*v0
    U,d,*_=evolve(U,z,dx,dx,T,mu,xi,'slope',maxdt=.0025)
    reference=float(solve_ivp(lambda
    ↪  t,v:A-B*v*v,[0,T],[v0],rtol=1e-11,atol=1e-12,method='DOP853').y[0,-1])
    observed=float((U[...,1]/U[...,0]).mean());err=abs(observed-reference)/reference
    add('slope_'+label,err<.01,reference=reference,observed=observed,error=err,diagnostic
    ↪  s=d)
z=np.broadcast_to(-.01*np.arange(20),(5,20)).copy();U=np.zeros((*z.shape,3));U[...,0]=2
U,d,*_=evolve(U,z,1,1,2,.1,500,'slope',maxdt=.02)
add('subthreshold_no_start',float(abs(U[...,1:]).max())<1e-12,max_momentum=float(abs(U[..
↪  .,1:]).max()))
z=np.zeros((8,40));U=np.zeros((*z.shape,3));U[...,0]=2;U[...,1]=.02
U,d,*_=evolve(U,z,1,1,2,.1,500,'open',maxdt=.02)
add('stop_no_reverse',float(U[...,1].min())>=0 and float(U[...,1].max())<1e-12,min_moment
↪  um=float(U[...,1].min()),max_momentum=float(U[...,1].max()))
z=np.zeros((10,80));U=np.zeros((*z.shape,3));mask=np.ones(10,dtype=bool)
inlet={'rows':mask,'speed':15.,'volume':1000.,'duration':20.}
U,d,*_=evolve(U,z,5,5,60,.01,500,'outflow',inlet=inlet,trace_path=out/'inlet_budget.npz')
add('inlet_open_budget',abs(d['inlet_volume_m3']-1000)<1e-3 and
↪  abs(d['total_inflow_m3']-1000)<1e-3 and d['mass_error_rel']<1e-8 and
↪  d['momentum_error_rel']<1e-8,diagnostics=d)

z=np.zeros((12,40));U=np.zeros((*z.shape,3));U[:]=[2.,4.,1.5]
old=U.copy();U,d,*_=evolve(U,z,3,3,2,boundary='open',maxdt=.05)
add('oblique_open_uniform',float(abs(U-old).max())<1e-10 and
↪  d['momentum_error_rel']<1e-8,diagnostics=d)

z=np.zeros((12,30));U=np.zeros((*z.shape,3));U[3:9,18:26,0]=1.;U[3:9,18:26,1]=15.;U[3:9,1
↪  8:26,2]=2.
U,d,*_=evolve(U,z,2,2,10,boundary='outflow',maxdt=.01,trace_path=out/'dry_exterior_budget
↪  .npz')
add('dry_exterior_actual_exit',d['total_outflow_m3']>.8*d['initial'][0] and
↪  d['total_inflow_m3']<1e-9 and d['mass_error_rel']<1e-8 and
↪  d['momentum_error_rel']<1e-8,diagnostics=d,
    analytic_bound='initial 192m3, advective edge clears east boundary in <1.6s at15m/s;
    ↪  gravity spreading allowed, require >80% exits in10s')

z=np.zeros((1,100));U=np.zeros((*z.shape,3));U[...,0]=1.
U,d,*_=evolve(U,z,1,1e8,.5,boundary='outflow',maxdt=.01,trace_path=out/'subcritical_drain
↪  _budget.npz')
add('subcritical_dry_exterior_drain',d['total_outflow_m3']>0 and
↪  d['total_inflow_m3']<1e-8 and d['mass_error_rel']<1e-8,diagnostics=d,
    restriction='wet interior at rest, dry exterior dam-break; no imposed downstream
    ↪  stage or backwater claim')
U=np.zeros((5,10,3));U[...,0]=1.;z=np.zeros((5,10));A,_=step(U,z,1,1,.01,boundary='outflo
↪  w');U[...,1]=1e-12;B,_=step(U,z,1,1,.01,boundary='outflow')
jump=float(abs(A-B).max());add('boundary_small_perturbation_continuity',jump<1e-8,max_sta
↪  te_difference=jump,perturbation=1e-12)
U=np.zeros((4,4,3));U[...,0]=DRY*.5;U[...,1]=1e-4;z=np.zeros((4,4));A,b=step(U,z,1,1,.01,
↪  boundary='open')
add('dry_clear_separate_from_drag',float(abs(b['friction']).max())<1e-15 and
↪  abs(b['dry_clear'][0]+.0016)<1e-14,
```

```
    friction_impulse=b['friction'].tolist(),dry_clear_impulse=b['dry_clear'].tolist(),dry
    ↪  _clear_energy=b['dry_clear_energy_loss'])
errs=[];T=.5;mu=.02;xi=300.;depth=2.;v0=20.;aa=G*mu;bb=G/(xi*depth)
ref=math.sqrt(aa/bb)*math.tan(math.atan(v0*math.sqrt(bb/aa))-T*math.sqrt(aa*bb))
for dt in (.05,.025,.0125):
    z=np.zeros((5,10));U=np.zeros((*z.shape,3));U[...,0]=depth;U[...,1]=depth*v0*.6;U[...
    ↪  ,2]=depth*v0*.8
    U,d,*_=evolve(U,z,100,100,T,mu,xi,'open',maxdt=dt)
    obs=float(np.linalg.norm(U[...,1:]/U[...,0,None],axis=-1).mean());errs.append(abs(obs
    ↪  -ref)/ref)
add('oblique_pure_drag_convergence',errs[-1]<.005 and all(a>b for a,b in
↪  zip(errs,errs[1:])),errors=errs,reference=ref)
if ORDER==2:
    errors=[]
    for dx in (60.,30.,15.):
        ss=17/30;depth=1.85;dt=1e-4;z=np.broadcast_to(-ss*np.arange(16)*dx,(5,16)).copy()
        ↪  ;U=np.zeros((*z.shape,3));U[...,0]=depth
        A,b=step(U,z,dx,dx,dt,.05,500,'slope');observed=float((A[...,1]/A[...,0]).mean())
        ↪  /dt;reference=G*(ss-.05)
        err=abs(observed-reference)/reference;errors.append(err)
        add('thin_steep_start_'+str(dx),err<1e-4,acceleration=observed,reference=referenc
        ↪  e,relative_error=err)
    U=np.zeros((3,8,3));U[...,0]=1.;U[...,1]=np.array([12,12,12,12,12,12,11,10])[None,:];
    ↪  z=np.zeros((3,8))
    A,b=step(U,z,1,1,.001,boundary='wall')
    add('reconstructed_wall_gradient_no_water',abs(float(A[...,0].sum()-U[...,0].sum()))<
    ↪  1e-12 and
    ↪  float(abs(b['boundary_faces'][:,0]).max())<1e-12,
        volume_change=float(A[...,0].sum()-U[...,0].sum()),maximum_wall_mass_flux_integra
        ↪  l=float(abs(b['boundary_faces'][:,0]).max()),wall_energy=b['energy_boundary'])
    weights=np.array([.2,.3,.5]);inlet={'rows':weights>0,'weights':weights,'speed':15.,'v
    ↪  olume':100.,'duration':20.}
    A,b=step(U,z,1,1,.001,boundary='wall',inlet=inlet,t=30.)
    add('partial_gate_gradient_no_water',abs(float(A[...,0].sum()-U[...,0].sum()))<1e-12
    ↪  and float(abs(b['boundary_faces'][:,0]).max())<1e-12,
        volume_change=float(A[...,0].sum()-U[...,0].sum()),maximum_wall_mass_flux_integra
        ↪  l=float(abs(b['boundary_faces'][:,0]).max()))
    from scipy.optimize import brentq
    pressures=[];references=[]
    for w in (-15.,-math.sqrt(G),0.,2.,15.):
        p=float(exact_wall_pressure(1.,w));pressures.append(p)
        if w<=0:ref=.5/G*max(0,math.sqrt(G)+w/2)**4
        else:
            rr=brentq(lambda r:(r-1)*math.sqrt((r+1)/(2*r))-w/math.sqrt(G),1,1+2*w/math.s
            ↪  qrt(G)+(w/math.sqrt(G))**2,xtol=1e-13)
            ref=.5*G*rr*rr
        references.append(ref)
    add('exact_wall_rarefaction_vacuum_shock',max(abs(a-b) for a,b in
    ↪  zip(pressures,references))<1e-9 and pressures[0]==0,
        pressure=pressures,reference=references)
    dry_pressures=exact_wall_pressure(np.zeros(3),np.array([-15.,0.,15.]))
    add('exact_wall_dry_zero_pressure',float(abs(dry_pressures).max())==0,pressure=dry_pr
    ↪  essures.tolist())
    U=np.zeros((3,8,3));U[...,0]=1.;U[...,1]=15.;z=np.zeros((3,8))
```

```
        inlet={'rows':weights>0,'weights':weights,'speed':15.,'volume':100.,'duration':20.,'f
        ↪  ace_bed_samples':{j:np.array([0.,2.]) for j in
        ↪  range(3)}}
        _,bb=euler_step(U,z,1,1,1e-6,boundary='wall',inlet=inlet,t=30.)
        expected=.5*float(exact_wall_pressure(1.,15.))*weights.sum()*1e-6
        add('partial_closed_dry_face_nodes',abs(bb['inlet_pressure_impulse']-expected)<1e-12,
        ↪  pressure_impulse=bb['inlet_pressure_impulse'],reference=expected)
    write_json(out/'CHECKS.json',{'checks':checks,'all_passed':all(c['passed'] for c in
    ↪  checks),'independent_review':'pending'})


def smooth_benchmark(out):
    errors=[]
    for n in (100,200,400):
        length=1000.;dx=length/n;x=(np.arange(n)+.5)*dx;c0=math.sqrt(2*G);amp=.1*c0;k=2*np.pi
        ↪  /length;T=10.
        c=c0+amp*np.sin(k*x);h=c*c/G;u=2*(c-c0)
        U=np.zeros((1,n,3));U[0,:,0]=h;U[0,:,1]=h*u;z=np.zeros((1,n))
        U,d,*_=evolve(U,z,dx,1e8,T,boundary='periodic',maxdt=10.)
        exact_c=np.full_like(x,c0)
        for _ in range(100):exact_c=c0+amp*np.sin(k*(x-(3*exact_c-2*c0)*T))
        exact_h=exact_c**2/G;err=float(abs(U[0,:,0]-exact_h).mean());errors.append(err)
        np.savez(out/f'smooth_{n}.npz',x=x,h=U[0,:,0],exact_h=exact_h)
        print(json.dumps({'n':n,'L1_h':err,'steps':d['steps']}),flush=True)
    orders=np.log2(np.array(errors[:-1])/np.array(errors[1:]))
    write_json(out/'SMOOTH_CHECK.json',{'errors':errors,'observed_orders':orders.tolist(),'pa
    ↪  ssed':bool(all(a>b for a,b in zip(errors,errors[1:])) and
    ↪  orders[-1]>1.5),'scope':'wet smooth friction-free simple wave only; full friction
    ↪  integration is mixed order','reconstruction_order':ORDER})


def terrain(out,root,full=False):
    import csv,rasterio
    from rasterio.warp import reproject,Resampling
    from rasterio.transform import from_origin
    from pyproj import Transformer
    srcdir=root/'补充数据/吉隆数据 2/09_ 补充 DEM'
    sources={'copernicus':srcdir/'Copernicus_DSM_COG_10_N28_00_E085_00_DEM.tif','aster':srcdi
    ↪  r/'ASTGTMV003_N28E085/ASTGTMV003_N28E085_dem.tif'}

    bounds=[339840,3128160,350640,3136080] if full else [348000,3134520,350640,3135840]
    res=15.;transform=from_origin(bounds[0],bounds[3],res,res)
    records={}
    for name,p in sources.items():
        a=np.full((int((bounds[3]-bounds[1])/res),int((bounds[2]-bounds[0])/res)),np.nan)
        with rasterio.open(p) as s:
            reproject(rasterio.band(s,1),a,src_transform=s.transform,src_crs=s.crs,dst_transf
            ↪  orm=transform,dst_crs='EPSG:32645',resampling=Resampling.bilinear,dst_nodata=
            ↪  np.nan)
            records[name]={'source':fp(p),'original_crs':str(s.crs),'original_transform':list
            ↪  (s.transform),'nodata':s.nodata,'vertical_datum':'product documentation
            ↪  pending; not from geographic CRS','missing':int((~np.isfinite(a)).sum())}
        if not np.isfinite(a).all():raise ValueError('terrain missing values')
        np.savez_compressed(out/f'{name}_support.npz',z=a,bounds=bounds,resolution=res,crs='E
        ↪  PSG:32645')
```

```
    rows=list(csv.DictReader((root/'outputs/Q3_implementation_20261030/data_diagnostics/gyiro
    ↪  ng_author_mapped_path.csv').open()))
    tr=Transformer.from_crs(4326,32645,always_xy=True)
    coords=np.array([tr.transform(float(r['longitude']),float(r['latitude'])) for r in rows])
    np.savetxt(out/'mapped_path_utm.csv',coords[239:721] if full else
    ↪  coords[239:401],delimiter=',',header='easting_m,northing_m',comments='')
    write_json(out/'TERRAIN.json',{'sources':records,'bounds':bounds,'support_resolution_m':1
    ↪  5,'flow_grid_candidates_m':[60,30],
        'entry':{'x_m':350640,'y_interval_m':[3135025,3135055],'width_m':30,'speed_m_s':30,'d
        ↪  irection':'west','identity':'D_ONLY boundary near author node270, ~928m after C;
        ↪  no measured state'},
        'target':{'y_m':3129103.736725446,'x_range_m':[340500,341100],'identity':'port-refere
        ↪  nce cross-section near author699; no observed hydrograph'} if full else
        ↪  {'x_m':348720,'y_range_m':[3134880,3135600],'identity':'nested cross-section; not
        ↪  port'},
        'no_filling_or_elevation_correction':True,'hydraulic_qualification':'pending','full_e
        ↪  vent_reconstruction':False})


def sample_terrain(p,dx):
    from scipy.ndimage import map_coordinates
    f=np.load(p);a=f['z'];bounds=f['bounds'];res=float(f['resolution'])
    if str(f['crs'])!='EPSG:32645' or res<=0 or not np.isfinite(a).all() or a.ndim!=2:raise
    ↪  ValueError('finite EPSG32645 metre terrain support required')
    expected=((bounds[3]-bounds[1])/res,(bounds[2]-bounds[0])/res)
    if not np.allclose(expected,a.shape,rtol=0,atol=1e-9):raise ValueError('terrain
    ↪  bounds/resolution/shape mismatch')
    if not np.isfinite(dx) or dx<=0 or not np.allclose(np.array(expected)*res/dx,np.round(np.
    ↪  array(expected)*res/dx),rtol=0,atol=1e-9):raise ValueError('flow resolution must tile
    ↪  the fixed physical domain exactly')
    nx=int(round((bounds[2]-bounds[0])/dx));ny=int(round((bounds[3]-bounds[1])/dx))

    xx=bounds[0]+(np.arange(nx)+.5)*dx;yy=bounds[3]-(np.arange(ny)+.5)*dx
    cc,rr=np.meshgrid((xx-bounds[0])/res-.5,(bounds[3]-yy)/res-.5)
    return map_coordinates(a,[rr,cc],order=1,mode='nearest'),xx,yy


def pilot(out,inputs,dx,args):
    z,x,y=sample_terrain(inputs/f'{args.terrain_name}_support.npz',dx);U=np.zeros((*z.shape,3
    ↪  ))
    width=float(args.entry_width);entry_lo=3135040-width/2;entry_hi=3135040+width/2
    if not np.isfinite(width) or width<=0 or entry_lo<y.min()-dx/2 or
    ↪  entry_hi>y.max()+dx/2:raise ValueError('entry width outside domain')
    weights=np.maximum(0,np.minimum(y+dx/2,entry_hi)-np.maximum(y-dx/2,entry_lo))/dx
    mask=weights>0
    from scipy.ndimage import map_coordinates
    f=np.load(inputs/f'{args.terrain_name}_support.npz');ys=np.linspace(entry_lo,entry_hi,121
    ↪  );bounds=f['bounds']
    zs=map_coordinates(f['z'],[(bounds[3]-ys)/15-.5,np.full(len(ys),f['z'].shape[1]-.5)],orde
    ↪  r=1,mode='nearest')
    zin=float(np.trapezoid(zs,ys)/width)
    inlet={'rows':mask,'weights':weights,'speed':args.speed,'volume':args.volume,'duration':a
    ↪  rgs.duration,'shape':args.shape,'bed_mean':zin}
```

```
inlet['face_bed_samples']={int(j):np.interp(np.linspace(max(y[j]-dx/2,entry_lo),min(y[j]+dx/2,entry_hi),61),ys,zs) for j in np.flatnonzero(mask)}
write_json(out/'ENTRY_FACE.json',{'x_m':350640,'y_interval_m':[entry_lo,entry_hi],'continuous_bed_y_m':ys.tolist(),'continuous_bed_z_m':zs.tolist(),'bed_mean_m':zin,
    'grid_faces':[{'row':int(j),'center_y_m':float(y[j]),'fraction':float(weights[j]),'mapped_cell_bed_m':float(z[j,-1]),'face_mean_bed_m':float(inlet['face_bed_samples'][int(j)].mean()),'bed_mapping_difference_m':float(z[j,-1]-inlet['face_bed_samples'][int(j)].mean())} for j in np.flatnonzero(mask)],
    'normal_in_utm':[-1,0],'width_m':width,'speed_m_s':args.speed,'peak_physical_h_m':2*args.volume/args.duration/width/args.speed,'subcell_pressure':'fraction*g*h_physical^2/2, complementary face wall; half-cell bed pressure correction',
    'boundary_restriction':'dry exterior except upstream east gate; inlet closed outside pulse support','vector_convention':'internal [east,south]; exported north velocity=-internal south'})

if args.full:
    yt=3129103.736725446;row=int(np.searchsorted(-y,-yt));alpha=(yt-y[row-1])/(y[row]-y[row-1])
    target={'axis':'y','columns':[row-1,row],'weights':[1-alpha,alpha],'rows':(x>=340500)&(x<=341100)}
    target.update({'quadrature_lengths':np.maximum(0,np.minimum(x+dx/2,341100)-np.maximum(x-dx/2,340500)),'normal_sign':1,'tangent_coordinates':x})
else:
    col=int(np.searchsorted(x,348720));alpha=(348720-x[col-1])/(x[col]-x[col-1])
    target={'axis':'x','columns':[col-1,col],'weights':[1-alpha,alpha],'rows':(y>=3134880)&(y<=3135600)}
    target.update({'quadrature_lengths':np.maximum(0,np.minimum(y+dx/2,3135600)-np.maximum(y-dx/2,3134880)),'normal_sign':-1,'tangent_coordinates':y})
legal=np.zeros(2*len(y)+2*len(x),dtype=bool)
if args.full:legal[2*len(y)+len(x):]=(x>=340500)&(x<=341100)
else:legal[:len(y)]=(y>=3134880)&(y<=3135600)
U,d,series,snaps,hm,vm,event_vm=evolve(U,z,dx,dx,args.horizon,args.mu,args.xi,'outflow',maxdt=args.maxdt,cfl=args.cfl,inlet=inlet,target=target,trace_path=out/f'pilot_{dx:g}_budget.npz',legal_exit_mask=legal)
np.savez_compressed(out/f'pilot_{dx:g}.npz',z=z,x=x,y=y,final=U,max_h=hm,max_speed=vm,max_speed_when_h_ge_0p1=event_vm,series=series,
    series_columns='time_s,target_max_h_m,volume_m3,total_momentum_norm_per_density,Q_downstream_m3_s,u_east_at_hpeak_m_s,u_north_at_hpeak_m_s,tangent_coordinate_at_hpeak_m',snapshot_t=[s[0] for s in snaps],snapshot_U=[s[1] for s in snaps])
arrivals={}
for threshold in (.05,.1,.2):
    hits=np.flatnonzero(series[:,1]>=threshold)
    prefix_valid=bool(len(hits) and series[hits[0],0]<=d['valid_until_s'])
    arrivals[str(threshold)]={'status':'arrived' if prefix_valid else ('valid_no_arrival_by_horizon' if d['whole_window_valid'] else 'unknown'),
        'time_bracket_s':[float(series[hits[0]-1,0]) if hits[0]>0 else 0.,float(series[hits[0],0])] if prefix_valid else None}
finalspeed=np.linalg.norm(np.divide(U[...,1:],U[...,0,None],out=np.zeros_like(U[...,1:]),where=U[...,0,None]>DRY),axis=-1)
d.update({'dx_m':dx,'shape':list(z.shape),'target_max_h':float(series[:,1].max()),'threshold_arrivals':arrivals,'status':arrivals['0.1']['status'],
```

```
        'run_termination':'out_of_domain' if not d['whole_window_valid'] else
        ↪  'time_horizon','final_motion_state':'stopped' if finalspeed.max()<1e-3 else
        ↪  'moving',
        'terrain_qualification':'pending','full_reach':args.full,'legal_exit_geometry':{'side
        ↪  ':'south' if args.full else 'west','interval_m':[340500,341100] if args.full else
        ↪  [3134880,3135600]},
        'arrival_support':{'axis':target['axis'],'coordinate_m':3129103.736725446 if
        ↪  args.full else 348720,'interpolation_columns':target['columns'],'interpolation_we
        ↪  ights':target['weights']},
        'field_convention':'saved U=(h,hu_east,hu_south); north momentum=-hu_south; lengths
        ↪  m, times s, velocity m/s'})
    d.update({'peak_h_anywhere_m':float(hm.max()),'maximum_speed_m_s':float(vm.max()),'inunda
    ↪  ted_area_m2':float((hm>=.1).sum()*dx*dx),
              'pulse_definition':'two adjacent half-volume/half-duration triangles within
              ↪  [0,T]' if args.shape=='double' else 'single triangle in [0,T]'})
    peak=int(np.argmax(series[:,1]));d.update({'target_Q_downstream_max_m3_s':float(series[:,
    ↪  4].max()),'target_Q_downstream_min_m3_s':float(series[:,4].min()),
        'target_peak_h_paired':dict(zip(['time_s','h_m','u_east_m_s','u_north_m_s','tangent_c
        ↪  oordinate_m'],[float(series[peak,j]) for j in
        ↪  [0,1,5,6,7]])),
        'speed_envelope_for_figures':'max speed computed only at steps with local event
        ↪  h>=0.1m; raw max_speed separately retained','loads_not_computed':True})
    if series[peak,1]<.1:
        d['target_peak_h_paired']={'time_s':None,'h_m':float(series[peak,1]),'u_east_m_s':Non
        ↪  e,'u_north_m_s':None,'tangent_coordinate_m':None}
    d['ever_threshold_coverage_area_m2']=d['inundated_area_m2'];d['final_threshold_inundated_
    ↪  area_m2']=float((U[...,0]>=.1).sum()*dx*dx)
    d['arrival_support'].update({'crs':'EPSG:32645','tangent_interval_m':[340500,341100] if
    ↪  args.full else [3134880,3135600],'criterion':'instantaneous maximum vertical event
    ↪  depth >= threshold on fixed interpolated line'})
    write_json(out/f'pilot_{dx:g}.json',d);print(json.dumps(d),flush=True)


def run_morris(out,args):

    import csv,subprocess,sys,os,hashlib
    from concurrent.futures import ThreadPoolExecutor,as_completed
    names=['volume_m3','duration_s','mu','xi_m_s2']
    limits=np.array([[2e5,8e5],[720.,1200.],[.03,.15],[250.,1000.]])
    points,orders=morris_design(4,8,6,83031);points=np.round(points,12)
    def physical(x):
        return np.array([np.exp(np.log(limits[j,0])+x[j]*np.log(limits[j,1]/limits[j,0])) if
        ↪  j!=2 else limits[j,0]+x[j]*(limits[j,1]-limits[j,0]) for j in range(4)])
    configs=[];ids=[];unique={}
    for tr in range(8):
        row=[]
        for j in range(5):
            values=physical(points[tr,j]);config=dict(zip(names,values.tolist()))
            key=hashlib.sha256(json.dumps(config,sort_keys=True).encode()).hexdigest();unique
            ↪  [key]=config;row.append(key)
        ids.append(row)
    policy={'purpose':'S20 development Morris; not final GP
    ↪  test','names':names,'limits':limits.tolist(),'scales':['log','log','linear','log'],
        'trajectories':8,'levels':6,'delta':.6,'seed':83031,'points':points.tolist(),'orders'
        ↪  :orders.tolist(),'query_ids':ids,
```

```
    'dx_m':args.dx,'horizon_s':args.horizon,'response':'target max event depth minus0.1m;
    ↪  peak depth and area also retained',
    'unknown_policy':'failure/out_of_domain is missing, never negative label; report
    ↪  incomplete trajectories',
    'workers':args.workers,'backend':args.backend,'scheme_order':args.order,'data_scope':
    ↪  'D_ONLY fixed-bed conditional
    ↪  propagation'}
write_json(out/'MORRIS_POLICY.json',policy)
physics=out/'physics';physics.mkdir();runs={}
def query(key,config):
    folder=physics/key[:16];argv=[sys.executable,'-B',str(Path(__file__).resolve()),'pilo
    ↪  t','--out',str(folder),
        '--inputs',str(args.inputs),'--dx',str(args.dx),'--order',str(args.order),'--back
        ↪  end',args.backend,
        '--horizon',str(args.horizon),'--cfl',str(args.cfl),'--dry',str(args.dry),'--maxd
        ↪  t',str(args.maxdt),
        '--volume',repr(config['volume_m3']),'--duration',repr(config['duration_s']),'--m
        ↪  u',repr(config['mu']),'--xi',repr(config['xi_m_s2']),
        '--speed',str(args.speed),'--terrain-name',args.terrain_name]
    start=time.perf_counter()
    with (physics/(key[:16]+'.stdout')).open('w') as
    ↪  stdout,(physics/(key[:16]+'.stderr')).open('w') as stderr:
        process=subprocess.run(argv,stdout=stdout,stderr=stderr,env={**os.environ,'OPENBL
        ↪  AS_NUM_THREADS':'1','OMP_NUM_THREADS':'1'})
    row={'query_id':key,'config':config,'argv':argv,'exit_code':process.returncode,'total
    ↪  _wall_s':time.perf_counter()-start,'folder':str(folder)}
    result=folder/f'pilot_{args.dx:g}.json'
    if process.returncode==0 and result.exists():
        d=json.loads(result.read_text());valid=d['whole_window_valid']
        row.update({'valid':valid,'status':d['status'],'margin_m':d['target_max_h']-.1 if
        ↪  valid else
        ↪  None,'peak_h_m':d['peak_h_anywhere_m'],'area_m2':d['inundated_area_m2'],
            'actual_volume_in_m3':d['total_inflow_m3'],'declared_volume_m3':config['volum
            ↪  e_m3'],'solver_wall_s':d['wall_s'],'mass_error_rel':d['mass_error_rel'],'
            ↪  momentum_error_rel':d['momentum_error_rel'],
            'receipt':fp(folder/'RUN_RECEIPT.json')})
    else:row.update({'valid':False,'status':'numerical_failure','margin_m':None})
    return row
with ThreadPoolExecutor(max_workers=args.workers) as pool:
    futures={pool.submit(query,key,config):key for key,config in unique.items()}
    for done in as_completed(futures):
        row=done.result();runs[row['query_id']]=row
        print(json.dumps({'completed':len(runs),'unique_queries':len(unique),'query':row[
        ↪  'query_id'][:16],'status':row['status'],'margin_m':row['margin_m'],'wall_s':r
        ↪  ow['total_wall_s']}),flush=True)
write_json(out/'MORRIS_QUERIES.json',list(runs.values()))
response=np.array([[runs[key]['margin_m'] if runs[key]['valid'] else np.nan for key in
↪  row] for row in ids])
complete=np.isfinite(response).all(axis=1)
if complete.any():
    ee,summary=morris_effects(points[complete],response[complete]);np.savez(out/'MORRIS_E
    ↪  FFECTS.npz',points=points,orders=orders,response=response,complete=complete,ee=ee)
    write_json(out/'MORRIS_SUMMARY.json',{'names':names,**summary,'complete_trajectories'
    ↪  :int(complete.sum()),'total_trajectories':8,'role':'exploratory domain-dependent
    ↪  screening, not Sobol','independent_review':'pending'})
```

```
    else:write_json(out/'MORRIS_SUMMARY.json',{'complete_trajectories':0,'status':'pending','
    ↪  reason':'no complete valid trajectory; no
    ↪  imputation'})


class SingleGP:

    def fit(self,x,y):
        self.x=np.asarray(x);self.mean=float(np.mean(y));self.scale=max(float(np.std(y)),1e-3
        ↪  );self.y=(np.asarray(y)-self.mean)/self.scale
        if self.x.ndim!=2 or len(self.x)!=len(self.y) or not np.isfinite(self.x).all() or not
        ↪  np.isfinite(self.y).all():raise ValueError('finite matching single GP data
        ↪  required')
        self.noise=1e-8;dim=self.x.shape[1]
        def matrix(theta):return matern52(self.x,self.x,np.exp(theta[0]),np.exp(theta[1:]))+n
        ↪  p.eye(len(y))*self.noise
        def loss(theta):
            try:
                c=cho_factor(matrix(theta),lower=True);return .5*self.y@cho_solve(c,self.y)+n
                ↪  p.log(np.diag(c[0])).sum()+.5*len(y)*np.log(2*np.pi)
            except np.linalg.LinAlgError:return 1e30
        attempts=[minimize(loss,np.r_[0,np.full(dim,np.log(l))],method='L-BFGS-B',bounds=[(-3
        ↪  ,3)]+[(-3,1)]*dim,options={'maxiter':100,'ftol':1e-8}) for l in
        ↪  (.15,.4,.8)]
        valid=[]
        for a in attempts:
            if not np.isfinite(a.x).all() or not np.isfinite(a.fun) or a.fun>=1e29:continue
            try:
                K=matrix(a.x)
                if np.isfinite(K).all():cho_factor(K,lower=True);valid.append(a)
            except np.linalg.LinAlgError:pass
        if not valid:raise ArithmeticError('no finite SPD single GP optimizer iterate')
        converged=[a for a in valid if a.success];best=min(converged or valid,key=lambda
        ↪  a:a.fun);self.theta=best.x;self.cf=cho_factor(matrix(best.x),lower=True);self.alp
        ↪  ha=cho_solve(self.cf,self.y)
        self.fit_record={'theta':self.theta.tolist(),'mean':self.mean,'scale':self.scale,'jit
        ↪  ter_normalized_variance':self.noise,
            'NLL':float(best.fun),'covariance_condition_number':float(np.linalg.cond(matrix(b
            ↪  est.x))),
            'fit_status':'converged_MLE' if converged else 'finite_nonconverged_fallback',
            'fallback_policy':'prefer finite SPD converged starts; otherwise flag lowest
            ↪  finite SPD iterate',
            'optimizer_attempts':[{'success':bool(a.success),'fun':float(a.fun) if
            ↪  np.isfinite(a.fun) else None,'message':str(a.message)} for a in attempts]}
        return self
    def predict(self,x):
        k=matern52(np.asarray(x),self.x,np.exp(self.theta[0]),np.exp(self.theta[1:]))
        var=np.exp(2*self.theta[0])-np.sum(k*cho_solve(self.cf,k.T).T,axis=1)
        if var.min()<-1e-7:raise ArithmeticError('single GP negative variance')
        return self.mean+self.scale*k@self.alpha,self.scale*np.sqrt(np.maximum(0,var))


def gp_response(depth,harr=.1):
    h=np.asarray(depth,dtype=float)
```

```
    if not np.isfinite(h).all() or np.any(h<0) or not np.isfinite(harr) or harr<=0:raise
    ↪  ValueError('finite nonnegative depth and positive threshold required')
    return np.log1p((h-harr)/(2*harr))


def solver_identity():

    import inspect,ast,hashlib
    names=['hll_pair','minmod3','muscl_faces','exact_wall_pressure','triangle_cdf','pulse_vol
    ↪  ume','euler_step','step','energy','evolve','sample_terrain','pilot']
    functions={name:hashlib.sha256(ast.dump(ast.parse(inspect.getsource(globals()[name])),inc
    ↪  lude_attributes=False).encode()).hexdigest() for name in
    ↪  names}
    binding={'G':G,'functions':functions,'C_source_sha256':hashlib.sha256(SOURCE.encode()).he
    ↪  xdigest()}
    return {'sha256':hashlib.sha256(json.dumps(binding,sort_keys=True).encode()).hexdigest(),
    ↪  'details':binding}


def algorithm_identity():
    import inspect,ast,hashlib
    names=['matern52','ARGP','SingleGP','gp_response','config_from_plane','execute_physics','
    ↪  PhysicsRegistry','save_gp_state','method_predictions','validate_execution','validate_
    ↪  prediction_models','run_formal_training','run_formal_test','seal_cached_seed','load_f
    ↪  ixed_gp','cache_logical_id','FollowupRegistry','import_seed137','run_followup_validat
    ↪  ion','evaluate_followup','seal_followup_subset']
    details={name:hashlib.sha256(ast.dump(ast.parse(inspect.getsource(globals()[name])),inclu
    ↪  de_attributes=False).encode()).hexdigest() for name in
    ↪  names}
    return {'sha256':hashlib.sha256(json.dumps(details,sort_keys=True).encode()).hexdigest(),
    ↪  'details':details}


def validate_execution(policy,args,policy_path):
    expected_methods={'A_single_fill','B_MF_fill','Bm_MF_count_matched','C_MF_active'}
    if set(policy['methods'])!=expected_methods or len(policy['methods'])!=4 or
    ↪  policy['seeds'] not in ([137],[137,271,503]):raise ValueError('exact declared
    ↪  method/seed set required')
    if policy['solver_identity']['sha256']!=solver_identity()['sha256'] or
    ↪  policy['algorithm_identity']['sha256']!=algorithm_identity()['sha256']:raise
    ↪  ValueError('frozen solver/algorithm changed')
    for name,expected in policy['execution'].items():
        if getattr(args,name)!=expected:raise ValueError('frozen numerical setting changed:
        ↪  '+name)
    for asset in policy['input_assets']:
        p=Path(args.inputs)/asset['logical_id']
        if not p.is_file() or fp(p)['sha256']!=asset['sha256']:raise ValueError('frozen input
        ↪  asset changed: '+asset['logical_id'])
    if policy['h_arr_m']!=.1 or policy['horizon_s']!=args.horizon:raise
    ↪  ValueError('arrival/window semantics changed')
    expected_geometry={'entry_x_m':350640,'entry_y_interval_m':[3135025,3135055],'target_axis
    ↪  ':'x','target_coordinate_m':348720,'target_tangent_interval_m':[3134880,3135600]}
    if policy['geometry']!=expected_geometry or policy['low_dx_m']!=40 or
    ↪  policy['high_dx_m']!=30:raise ValueError('frozen geometry/fidelity semantics changed')
```

```
    for dependency in policy['dependencies']:bound_file(policy_path,dependency)
    td=json.loads(bound_file(policy_path,policy['test_design']).read_text())
    if policy['test_global_count']!=16 or policy['test_boundary_count']!=516:raise
    ↪  ValueError('new16 global and516 static potential points required')
    global_points=np.asarray(td['global']);critical=np.asarray(td['development_predicted_boun」
    ↪  dary'])
    expected=np.array([(z,j/128) for z in (.125,.375,.625,.875) for j in range(129)])
    if global_points.shape!=(16,2) or not np.isfinite(global_points).all() or
    ↪  np.any((global_points<0)|(global_points>1)):raise ValueError('invalid global
    ↪  coordinates')
    if not np.array_equal(critical,expected):raise ValueError('static critical tree changed')
    return True


def validate_prediction_models(methods,registry_rows,test,policy,training_root):
    expected={(int(seed),tag) for seed in policy['seeds'] for tag in policy['methods']}
    actual=[(int(m['seed']),m['method']) for m in methods]
    if len(actual)!=len(set(actual)) or set(actual)!=expected:raise
    ↪  ValueError('duplicate/missing/unexpected seed-method ID')
    records={r['key']:r for r in registry_rows};by_id={(int(m['seed']),m['method']):m for m
    ↪  in methods}
    for meta in methods:
        logical=Path(meta['folder']);folder=Path(training_root)/logical
        if logical.is_absolute() or not
        ↪  folder.resolve().is_relative_to(Path(training_root).resolve()):raise
        ↪  ValueError('method folder must be relative inside training root')
        state=json.loads((folder/'GP_STATE.json').read_text())
        if state.get('seed')!=meta['seed'] or state.get('method')!=meta['method']:raise
        ↪  ValueError('model seed/method mismatch')
        for kind,label in [('low','L'),('high','H')]:
            keys=meta[kind+'_keys']
            if state[kind+'_keys']!=keys or len(keys)!=len(set(keys)):raise ValueError('model
            ↪  training ID mismatch/duplicate')
            if any(k not in records or not records[k]['valid'] or records[k]['level']!=label
            ↪  for k in keys):raise ValueError('invalid or wrong-layer training ID')
            if keys:
                x=np.array([records[k]['x'] for k in
                ↪  keys]);y=np.array([records[k]['response_g'] for k in keys])
                if not np.array_equal(np.array(state['x'+label]),x) or not
                ↪  np.array_equal(np.array(state['y'+label]),y):raise ValueError('model
                ↪  training data not bound to physical registry')
        fit=state['fit'];theta=np.array(fit['theta']);nll=fit.get('negative_log_likelihood',f」
        ↪  it.get('NLL'))
        if not np.isfinite(theta).all() or not np.isfinite(nll) or fit['fit_status'] not in
        ↪  ('converged_MLE','finite_nonconverged_fallback'):raise ValueError('invalid frozen
        ↪  GP fit')
        f=np.load(folder/'TEST_PREDICTIONS_FROZEN.npz')
        if f['x'].shape!=test.shape or not np.array_equal(f['x'],test):raise
        ↪  ValueError('prediction coordinate order mismatch')
        if f['mean_g'].shape!=(len(test),) or f['std_g'].shape!=(len(test),) or not
        ↪  np.isfinite(f['mean_g']).all() or not np.isfinite(f['std_g']).all() or
        ↪  np.any(f['std_g']<0):raise ValueError('invalid prediction shape/finite/std')
    for seed in policy['seeds']:
        A=by_id[(seed,'A_single_fill')];B=by_id[(seed,'B_MF_fill')];Bm=by_id[(seed,'Bm_MF_cou」
        ↪  nt_matched')];C=by_id[(seed,'C_MF_active')]
```

```
        if A['high_keys']!=B['high_keys'] or B['high_keys']!=Bm['high_keys'] or
        ↪  len(A['high_keys'])!=16 or len(C['high_keys'])!=16:raise ValueError('HF matching
        ↪  budget/design mismatch')
        if len(A['low_keys']) or len(B['low_keys'])!=24 or
        ↪  len(Bm['low_keys'])!=len(C['low_keys']) or not 24<=len(C['low_keys'])<=32:raise
        ↪  ValueError('LF count matching mismatch')
    return True


def config_from_plane(point,policy):
    config=dict(policy['fixed_parameters'])
    for x,item in zip(point,policy['axes']):
        lo,hi=item['bounds'];config[item['name']]=float(np.exp(np.log(lo)+x*np.log(hi/lo)))
        ↪  if item['scale']=='log' else float(lo+x*(hi-lo))
    return config


def execute_physics(folder,config,args,dx=None):

    import subprocess,sys,os
    folder=Path(folder);folder.parent.mkdir(parents=True,exist_ok=True)
    dx=float(dx if dx is not None else config.get('dx_m',args.dx))
    inputs=Path(config.get('input_dir',args.inputs))
    argv=[sys.executable,'-B',str(Path(__file__).resolve()),'pilot','--out',str(folder),'--in
    ↪  puts',str(inputs),
        '--dx',repr(dx),'--order',str(args.order),'--backend',args.backend,'--horizon',str(co
        ↪  nfig.get('horizon_s',args.horizon)),
        '--cfl',str(config.get('cfl',args.cfl)),'--dry',str(config.get('dry_m',args.dry)),'--
        ↪  maxdt',str(args.maxdt),
        '--volume',repr(config['volume_m3']),'--duration',repr(config['duration_s']),'--mu',r
        ↪  epr(config['mu']),'--xi',repr(config['xi_m_s2']),
        '--entry-width',str(config.get('entry_width_m',args.entry_width)),'--speed',str(confi
        ↪  g.get('speed_m_s',args.speed)),'--shape',config.get('shape','triangle'),'--terrai
        ↪  n-name',config.get('terrain_name','copernicus')]
    if config.get('full',False):argv+=['--full']
    started=time.perf_counter()
    with folder.with_suffix('.stdout').open('w') as
    ↪  stdout,folder.with_suffix('.stderr').open('w') as stderr:
        proc=subprocess.run(argv,stdout=stdout,stderr=stderr,env={**os.environ,'OPENBLAS_NUM_
        ↪  THREADS':'1','OMP_NUM_THREADS':'1'})
    row={'config':config,'dx_m':dx,'argv':argv,'exit_code':proc.returncode,'total_wall_s':tim
    ↪  e.perf_counter()-started,'folder':str(folder)}
    result=folder/f'pilot_{dx:g}.json'
    if proc.returncode or not result.exists():
        row.update({'valid':False,'status':'numerical_failure'});return row
    d=json.loads(result.read_text());arr=d['threshold_arrivals']['0.1'];valid=d['whole_window
    ↪  _valid']
    row.update({'valid':valid,'status':d['status'],'target_max_h_m':d['target_max_h'],'respon
    ↪  se_g':float(gp_response(d['target_max_h'])) if valid else
    ↪  None,
        'raw_margin_m':d['target_max_h']-.1 if valid else
        ↪  None,'response_g_unit':'dimensionless_log_margin',
        'time_s':float(np.mean(arr['time_bracket_s'])) if arr['status']=='arrived' else
        ↪  None,'time_bracket_s':arr['time_bracket_s'],
```

```
        'solver_wall_s':d['wall_s'],'mass_error_rel':d['mass_error_rel'],'momentum_error_rel'
        ↪  :d['momentum_error_rel'],
        'receipt':fp(folder/'RUN_RECEIPT.json'),'result':fp(result)})
    receipt=json.loads((folder/'RUN_RECEIPT.json').read_text());row['compile_and_load_s']=flo
    ↪  at((receipt.get('acceleration') or
    ↪  {}).get('compile_and_load_s',0.))
    return row


def run_near_boundary(out,args):
    config={'volume_m3':4e5,'duration_s':960.,'mu':.03,'xi_m_s2':500.,'speed_m_s':30.}
    history=[]
    def low(mu,index):
        c={**config,'mu':float(mu)};row=execute_physics(out/f'coarse_{index:02d}',c,args,40.)
        history.append(row);write_json(out/'COARSE_HISTORY.json',history)
        if not row['valid']:raise RuntimeError('invalid coarse bracket query; not a negative
        ↪  sample')
        return row
    a,b=.03,.15;left=low(a,0);right=low(b,1)
    if not (left['response_g']>=0 and right['response_g']<0):
        write_json(out/'NEAR_POINT_FROZEN.json',{'status':'no_bracket','history':history,'no_
        ↪  fine_queries':True});return
    for i in range(8):
        mid=.5*(a+b);row=low(mid,i+2)
        if row['response_g']>=0:a=mid;left=row
        else:b=mid;right=row
    c={**config,'mu':.5*(a+b)};frozen={'config':c,'mu_bracket':[a,b],'left':left,'right':righ
    ↪  t,'selection':'8 fixed40m bisections, no fine label
    ↪  accessed','fine_status':'not_run'}
    write_json(out/'NEAR_POINT_FROZEN.json',frozen);binding=fp(out/'NEAR_POINT_FROZEN.json');
    ↪  print(json.dumps({'frozen':binding,'config':c}),flush=True)
    fine30=execute_physics(out/'fine30',c,args,30.);fine15=execute_physics(out/'fine15',c,arg
    ↪  s,15.)
    write_json(out/'NEAR_COMPARISON.json',{'frozen_point':binding,'coarse_bracket':[left,righ
    ↪  t],'fine30':fine30,'fine15':fine15,
        'scope':'one development near-LF-boundary; numerical ambiguity
        ↪  retained','independent_review':'pending'})


def run_perturbations(out,args):
    from concurrent.futures import ThreadPoolExecutor,as_completed
    base={'volume_m3':args.volume,'duration_s':args.duration,'mu':args.mu,'xi_m_s2':args.xi,'
    ↪  speed_m_s':args.speed}
    cases={'base':base,'T720':{**base,'duration_s':720.},'T1200':{**base,'duration_s':1200.},
    ↪  'xi250':{**base,'xi_m_s2':250.},'xi1000':{**base,'xi_m_s2':1000.}}
    write_json(out/'PERTURBATION_DESIGN.json',{'cases':cases,'dx_m':30.,'role':'development-o
    ↪  nly confirmation of fixed
    ↪  parameters','test_truth_accessed':False})
    result={}
    with ThreadPoolExecutor(max_workers=args.workers) as pool:
        futures={pool.submit(execute_physics,out/name,c,args,30.):name for name,c in
        ↪  cases.items()}
        for f in as_completed(futures):
            name=futures[f];result[name]=f.result();print(json.dumps({'case':name,'status':re
            ↪  sult[name]['status'],'g':result[name].get('response_g')}),flush=True)
```

```
    write_json(out/'PERTURBATION_RESULTS.json',result)


def run_global_cases(out,args):

    from concurrent.futures import ThreadPoolExecutor,as_completed
    common={'volume_m3':8e5,'duration_s':960.,'mu':.05,'xi_m_s2':500.,'speed_m_s':30.,'full':
    ↪  True,'horizon_s':3600.,'input_dir':str(args.full_inputs)}
    cases={}
    for terrain_name in ('copernicus','aster'):
        for shape,duration in [('triangle',960.),('wide',1200.),('double',960.)]:
            cases[terrain_name+'_'+shape+'_40']={**common,'terrain_name':terrain_name,'shape'
            ↪  :'triangle' if shape=='wide' else
            ↪  shape,'duration_s':duration,'dx_m':40.}
    cases['copernicus_triangle_30']={**common,'terrain_name':'copernicus','shape':'triangle',
    ↪  'dx_m':30.}
    cases['copernicus_high_mobility_40']={**common,'volume_m3':8e5,'mu':.03,'xi_m_s2':1000.,'
    ↪  terrain_name':'copernicus','shape':'triangle','dx_m':40.}
    cases['copernicus_low_mobility_40']={**common,'volume_m3':2e5,'mu':.15,'xi_m_s2':250.,'te
    ↪  rrain_name':'copernicus','shape':'triangle','dx_m':40.}
    write_json(out/'GLOBAL_CASE_POLICY.json',{'cases':cases,'same_volume_m3':8e5,'global_hori
    ↪  zon_s':3600,
        'two_pulses':'two adjacent V/2,T/2 triangles inside [0,T], same total support/peak as
        ↪  single triangle',
        'wide_pulse':'sameV but longerT; pressure/energy input not assumed
        ↪  identical','scope':'full ~14.188km conditional reach, not local GP accuracy or
        ↪  event reconstruction',
        'unknown_observed_entry_time':None,'independent_review':'pending'})
    result={}
    with ThreadPoolExecutor(max_workers=min(args.workers,2)) as pool:
        futures={pool.submit(execute_physics,out/name,c,args,c['dx_m']):name for name,c in
        ↪  cases.items()}
        for done in as_completed(futures):
            name=futures[done];result[name]=done.result();print(json.dumps({'full_case':name,
            ↪  'status':result[name]['status'],'target_max_h_m':result[name].get('target_max
            ↪  _h_m'),'wall_s':result[name]['total_wall_s']}),flush=True)
            write_json(out/'GLOBAL_RESULTS.json',result)
    table=[]
    for name,row in result.items():
        detail=json.loads(Path(row['result']['path']).read_text()) if row['valid'] else {}
        table.append({'case_id':name,**{k:row['config'][k] for k in
        ↪  ['volume_m3','duration_s','mu','xi_m_s2','terrain_name','shape']},
            'H_s':3600,'dx_m':row['dx_m'],'status':row['status'],'travel_time_s':row.get('tim
            ↪  e_s'),'target_max_h_m':row.get('target_max_h_m'),
            'domain_space_time_peak_h_m':detail.get('peak_h_anywhere_m'),'ever_h_ge0p1_covera
            ↪  ge_area_m2':detail.get('ever_threshold_coverage_area_m2'),
            'actual_inflow_m3':detail.get('total_inflow_m3'),'mass_error_rel':row.get('mass_e
            ↪  rror_rel'),'query_wall_s':row['total_wall_s']})
    import csv
    with (out/'GLOBAL_TABLE.csv').open('w') as f:
        w=csv.DictWriter(f,fieldnames=list(table[0]));w.writeheader();w.writerows(table)


def along_axis_baseline(inputs,full,mu,xi,h_vertical=2.,u0=30.,horizon=3600.):
```

```
    from scipy.integrate import quad
    from scipy.optimize import brentq
    from scipy.ndimage import map_coordinates
    coordinates=np.loadtxt(Path(inputs)/'mapped_path_utm.csv',delimiter=',',skiprows=1)
    terrain=np.load(Path(inputs)/'copernicus_support.npz');bounds=terrain['bounds'];res=float(terrain['resolution']);bed=terrain['z']
    distance=np.r_[0,np.cumsum(np.linalg.norm(np.diff(coordinates,axis=0),axis=1))]
    def crossing(axis,value,start=0):
        for j in range(start,len(coordinates)-1):
            a,b=coordinates[j,axis],coordinates[j+1,axis]
            if a!=b and (a-value)*(b-value)<=0:
                f=(value-a)/(b-a);return j,float(distance[j]+f*(distance[j+1]-distance[j]))
        raise ValueError('mapped axis does not intersect the declared boundary/target')
    i0,s0=crossing(0,350640);it,st=crossing(1,3129103.736725446,i0) if full else crossing(0,348720,i0)
    s=np.r_[s0,distance[(distance>s0)&(distance<st)],st]-s0
    x=np.interp(s+s0,distance,coordinates[:,0]);y=np.interp(s+s0,distance,coordinates[:,1])
    z=map_coordinates(bed,[(bounds[3]-y)/res-.5,(x-bounds[0])/res-.5],order=1,mode='nearest')
    energy=.5*u0*u0;t=0.;position=0.;stopped=False;trajectory=[[0.,0.,u0]]
    for j,ds in enumerate(np.diff(s)):
        slope=(z[j]-z[j+1])/ds;a=G*(slope-mu);b=2*G/(xi*h_vertical);eq=a/b
        if energy<=0 and a<=0:stopped=True;break
        e0=energy
        def e(l):return eq+(e0-eq)*np.exp(-b*l)
        reach=float(ds)
        if e(reach)<=0 and eq<0:reach=float(-np.log(-eq/(e0-eq))/b);stopped=True
        def travel(l):return quad(lambda u:1/np.sqrt(max(2*e(u),1e-20)),0,l,epsabs=1e-7,epsrel=1e-7,limit=100)[0]
        elapsed=travel(reach)
        if t+elapsed>horizon:
            reach=brentq(lambda l:travel(l)-(horizon-t),0,reach);position+=reach;t=horizon;energy=max(0,float(e(reach)));trajectory.append([t,position,math.sqrt(2*energy)]);break
        position+=reach;t+=elapsed;energy=max(0,float(e(reach)));trajectory.append([t,position,math.sqrt(2*energy)])
        if stopped:break
    arrived=position>=s[-1]-1e-6
    return {'status':'arrived' if arrived else 'valid_no_arrival_by_horizon','travel_time_s':t if arrived else None,'axis_position_at_cutoff_or_stop_m':position,
        'target_axis_distance_m':float(s[-1]),'baseline_stopped':stopped,'trajectory':trajectory,'h_vertical_closure_m':h_vertical,
        'scope':'horizontal along-axis energy ODE, constant assumed thickness, no mass hydrograph/lateral inundation; not low fidelity or event validation'}


def run_synthetic_inverse(out,args):

    from concurrent.futures import ThreadPoolExecutor,as_completed
    truth={'volume_m3':7e5,'duration_s':960.,'mu':.04,'xi_m_s2':500.,'speed_m_s':30.,'cfl':.175}
    vv=np.linspace(6e5,8e5,4);mm=np.linspace(.03,.05,4)
```

```
configurations=[{'volume_m3':float(v),'duration_s':960.,'mu':float(mu),'xi_m_s2':500.,'sp
↪  eed_m_s':30.} for v in vv for mu in
↪  mm]
policy={'truth':truth,'observation_generator_dx_m':15.,'candidate_model_dx_m':30.,'candid
↪  ate_library':configurations,
    'arrival_error_scale_s':60.,'target_depth_error_scale_m':.75,'candidate_parameter_gri
    ↪  d_per_axis':41,
    'calibration_outputs':['arrival_time','target_peak_depth'],'held_out_output':'target_
    ↪  peak_signed_discharge',
    'scope':'synthetic observation with grid/time mismatch, not local calibration or
    ↪  natural error distribution',
    'representatives':'min arrival error, min depth error, min max(normalized errors),
    ↪  unique points returned to PDE'}
write_json(out/'INVERSE_POLICY.json',policy)

observed=execute_physics(out/'synthetic_truth_15',truth,args,15.)
if not observed['valid'] or observed['time_s'] is None:raise RuntimeError('registered
↪  synthetic truth did not arrive; keep failure, do not choose another truth')
rows=[]
with ThreadPoolExecutor(max_workers=args.workers) as pool:
    futures={pool.submit(execute_physics,out/f'library_{j:02d}',c,args,30.):j for j,c in
    ↪  enumerate(configurations)}
    by_index={}
    for done in as_completed(futures):
        j=futures[done];by_index[j]=done.result();print(json.dumps({'inverse_training':j,
        ↪  'status':by_index[j]['status']}),flush=True)
rows=[by_index[j] for j in range(len(configurations))]
write_json(out/'INVERSE_PHYSICAL_LIBRARY.json',{'observed':observed,'training':rows})
if any(not r['valid'] or r['time_s'] is None for r in rows):raise RuntimeError('inverse
↪  predeclared library contains nonarrival/unknown; do not impute finite time')
x=np.array([[(r['config']['volume_m3']-6e5)/2e5,(r['config']['mu']-.03)/.02] for r in
↪  rows])
times=np.array([r['time_s'] for r in rows]);depths=np.array([r['target_max_h_m'] for r in
↪  rows])
gp_t=SingleGP().fit(x,times);gp_h=SingleGP().fit(x,depths)
grid=np.array([(a,b) for a in np.linspace(0,1,41) for b in np.linspace(0,1,41)])
pred_t,_=gp_t.predict(grid);pred_h,_=gp_h.predict(grid)
e1=abs(pred_t-observed['time_s'])/60.;e2=abs(pred_h-observed['target_max_h_m'])/.75
order=np.argsort(e1,kind='stable');front=[];best=np.inf
for j in order:
    if e2[j]<best:front.append(int(j));best=e2[j]
representatives=list(dict.fromkeys([int(np.argmin(e1)),int(np.argmin(e2)),int(np.argmin(n
↪  p.maximum(e1,e2)))]))
checked=[];observed_detail=json.loads(Path(observed['result']['path']).read_text())
for i in representatives:
    c={'volume_m3':float(6e5+2e5*grid[i,0]),'duration_s':960.,'mu':float(.03+.02*grid[i,1
    ↪  ]),'xi_m_s2':500.,'speed_m_s':30.}
    actual=execute_physics(out/f'pareto_recheck_{i:04d}',c,args,30.)
    detail=json.loads(Path(actual['result']['path']).read_text()) if actual['valid'] else
    ↪  {}
    checked.append({'grid_index':i,'normalized_parameter':grid[i].tolist(),'surrogate_err
    ↪  ors':[float(e1[i]),float(e2[i])],'physical':actual,
        'held_out_Q_error_m3_s':detail.get('target_Q_downstream_max_m3_s',np.nan)-observe
        ↪  d_detail['target_Q_downstream_max_m3_s'] if detail else
        ↪  None,
```

```
            'truth_parameter_relative_errors':{'volume':(c['volume_m3']/truth['volume_m3']-1)
            ↪  ,'mu':(c['mu']/truth['mu']-1)}})
    np.savez_compressed(out/'INVERSE_SEARCH.npz',training_x=x,training_time_s=times,training_
    ↪  depth_m=depths,grid=grid,predicted_time_s=pred_t,predicted_depth_m=pred_h,
        normalized_arrival_error=e1,normalized_depth_error=e2,pareto_indices=front,single_com
        ↪  patible=e1<=1,multi_compatible=(e1<=1)&(e2<=1))

    slope=.10;h=2.;mu0=.05;xi0=500.;terminal=math.sqrt(xi0*h*(slope-mu0));tobs=10.
    transient=terminal*math.tanh(G*(slope-mu0)*tobs/terminal)
    family=[{'mu':mu,'xi':terminal**2/(h*(slope-mu)),'terminal_m_s':terminal,
        'u_at10s_m_s':terminal*math.tanh(G*(slope-mu)*tobs/terminal)} for mu in
        ↪  (.02,.035,.05,.065,.075)]
    recovered_mu=slope-terminal/(G*tobs)*math.atanh(transient/terminal);recovered_xi=terminal
    ↪  **2/(h*(slope-recovered_mu))
    write_json(out/'INVERSE_RESULTS.json',{'policy':fp(out/'INVERSE_POLICY.json'),'observed':
    ↪  observed,'arrival_GP':gp_t.fit_record,'depth_GP':gp_h.fit_record,
        'single_constraint_candidates':int((e1<=1).sum()),'two_constraint_candidates':int(((e
        ↪  1<=1)&(e2<=1)).sum()),
        'candidate_grid_count':len(grid),'Pareto_indices':front,'representatives_recomputed':
        ↪  checked,
        'nonidentifiable_terminal_velocity_family':family,'two_observable_exact_recovery':{'t
        ↪  rue_mu':mu0,'recovered_mu':recovered_mu,'true_xi':xi0,'recovered_xi':recovered_xi
        ↪  },
        'not_a_natural_calibration':True,'independent_review':'pending'})


def strict_q3_q4_join(q3,q4,entity_root,allow_synthetic=False):


    entity_root=Path(entity_root).resolve()
    if q3.get('expected_targets')!=q4.get('expected_targets'):raise ValueError('expected
    ↪  target geometries differ')
    expected=q4.get('expected_targets',{})
    if not expected:raise ValueError('expected targets required')
    def verify(record):
        p=(entity_root/record['path']).resolve()
        if not p.is_relative_to(entity_root) or not p.is_file() or
        ↪  fp(p)['sha256']!=record['sha256']:raise ValueError('entity missing, outside root
        ↪  or fingerprint mismatch')
        return p
    entries={}
    for entry in q3.get('entries',[]):
        sid=entry['scenario_id']
        if sid in entries:raise ValueError('duplicate Q3 scenario')
        if entry.get('entry_stage')!='propagation_entry':raise ValueError('local first hit is
        ↪  not automatically propagation entry')
        bridge=entry.get('bridge_evidence')
        if not bridge or not entry.get('bridge_evidence_id'):raise ValueError('bound
        ↪  stage-bridge evidence required, not only a label')
        b=json.loads(verify(bridge).read_text())
        if b.get('source_kind')=='synthetic_assumption' and not allow_synthetic:raise
        ↪  ValueError('synthetic stage assumption cannot enter a natural join')
        if b.get('source_kind') not in
        ↪  ('synthetic_assumption','independently_reviewed_stage_bridge'):raise
        ↪  ValueError('unreviewed stage bridge')
```

```
    if entry.get('status') not in ('entry_reached','not_entered','unknown'):raise
    ↪  ValueError('invalid Q3 entry status')
    entries[sid]=entry
responses={};identities=['scenario_hash','joint_input_hash','entry_id','entry_geometry_ha⌋
↪  sh','entry_state_hash','time_origin_id']
entity_fields={'parameters':'parameter_hash','terrain':'terrain_hash','code':'code_hash',⌋
↪  'entry_geometry':'entry_geometry_hash','entry_state':'entry_state_hash',
    'target_geometry':'target_geometry_hash','criterion':'arrival_criterion_hash','distan⌋
    ↪  ce':'distance_definition_hash'}
for response in q4.get('arrivals',[]):
    key=(response['scenario_id'],response['target_id'])
    if key in responses or key[0] not in entries or key[1] not in expected:raise
    ↪  ValueError('duplicate or unexpected Q4 scenario-target')
    entry=entries[key[0]]
    if any(entry.get(k) is None or entry.get(k)!=response.get(k) for k in
    ↪  identities):raise ValueError('joint member, entry/state or clock mismatch')
    if response['target_geometry_hash']!=expected[key[1]] or
    ↪  response.get('time_unit')!='s' or response.get('distance_unit')!='m':raise
    ↪  ValueError('target geometry or SI-unit mismatch')
    entities=response.get('entities',{})
    for kind,field in entity_fields.items():
        if kind not in entities:raise ValueError('missing bound entity: '+kind)
        verify(entities[kind])
        if response.get(field)!=entities[kind]['sha256']:raise ValueError('declared
        ↪  entity identity differs: '+kind)
    criterion=json.loads(verify(entities['criterion']).read_text());distance=json.loads(v⌋
    ↪  erify(entities['distance']).read_text())
    if criterion.get('definition')!=response.get('arrival_criterion') or
    ↪  distance.get('definition')!=response.get('distance_definition'):raise
    ↪  ValueError('distance/arrival semantics mismatch')
    status=response.get('target_arrival_status');travel=response.get('travel_time_s');val⌋
    ↪  id_until=response.get('valid_until_s')
    horizon=response.get('propagation_horizon_s')
    if response.get('horizon_s',horizon)!=horizon:raise ValueError('conflicting horizon
    ↪  aliases')
    if status=='arrived':
        if entry['status']=='not_entered':raise ValueError('not-entered scenario cannot
        ↪  have a propagated arrival')
        if travel is None or not math.isfinite(travel) or travel<0 or
        ↪  response.get('prefix_review_status')!='pass' or valid_until is None or not
        ↪  math.isfinite(valid_until) or valid_until<travel:raise ValueError('arrival
        ↪  lacks finite valid independently reviewed prefix')
    elif status=='valid_no_arrival_by_horizon':
        if travel is not None or horizon is None or not math.isfinite(horizon) or
        ↪  horizon<=0 or response.get('whole_window_valid') is not True or valid_until
        ↪  is None or not math.isfinite(valid_until) or valid_until<horizon:raise
        ↪  ValueError('nonarrival requires complete valid window')
    elif status not in ('unknown','not_run') or travel is not None:raise
    ↪  ValueError('invalid unknown/nonarrival time')
    responses[key]=response
joined=[]
for sid,entry in entries.items():
    for target in expected:
```

```
            response=responses.get((sid,target));row={'scenario_id':sid,'target_id':target,'a
            ↪  bsolute_arrival_s':None,'effective_lead_s':None,'travel_time_s':None,'target_
            ↪  arrival_status':'unknown'}
            if response is None:row['reason']='missing expected scenario-target
            ↪  response';joined.append(row);continue
            row.update(travel_time_s=response['travel_time_s'],target_arrival_status=response
            ↪  ['target_arrival_status'],run_termination_status=response.get('run_terminatio
            ↪  n_status'),
                whole_window_valid=response.get('whole_window_valid'),valid_until_s=response.
                ↪  get('valid_until_s'),reason='relative propagation response
                ↪  only')
            if response['target_arrival_status']=='arrived' and
            ↪  entry['status']=='entry_reached' and entry.get('T_in_s') is not None:
                if not math.isfinite(entry['T_in_s']):raise ValueError('nonfinite entry time')
                row['absolute_arrival_s']=entry['T_in_s']+response['travel_time_s']
                pub,action=entry.get('t_pub_s'),entry.get('d_act_s')
                if pub is not None and action is not None:
                    if not math.isfinite(pub) or not math.isfinite(action) or action<0:raise
                    ↪  ValueError('invalid publication/action delay')
                    row['effective_lead_s']=row['absolute_arrival_s']-pub-action
            joined.append(row)
    return {'joined':joined,'scientific_bridge_truth':'must be independently established;
    ↪  entity consistency alone does not establish it'}


def freeze_formal_policy(out,args):

    import shutil
    from scipy.stats import qmc
    prior=Path(args.previous_design);policy=json.loads((prior/'FORMAL_POLICY.json').read_text
    ↪  ());designs=json.loads((prior/'TRAINING_DESIGNS.json').read_text())
    if set(designs)!={'137','271','503'}:raise ValueError('original three-seed designs
    ↪  required')
    (out/'TRAINING_DESIGNS.json').write_bytes((prior/'TRAINING_DESIGNS.json').read_bytes())
    old_path=prior/'TEST_DESIGN_ORIGINAL96.json';original=json.loads(old_path.read_text());fo
    ↪  rbidden=[*original['global'],*original['development_predicted_boundary']]
    for d in designs.values():forbidden.extend(d['low']);forbidden.extend(d['active_candidate
    ↪  s']);forbidden.extend(d['matched_passive_low_pool'])
    pool=qmc.Sobol(2,scramble=True,seed=93071).random_base2(6);f=np.asarray(forbidden);keep=[
    ↪  i for i,x in enumerate(pool) if
    ↪  np.min(np.linalg.norm(x-f,axis=1))>1e-12][:16]
    if len(keep)!=16:raise ValueError('not enough independent global coordinates')
    critical=np.array([(z,j/128) for z in (.125,.375,.625,.875) for j in range(129)])
    if np.min(np.linalg.norm(critical[:,None,:]-f[None,:,:],axis=-1))<=1e-12:raise
    ↪  ValueError('critical tree coincides with prior training/test coordinate')
    write_json(out/'TEST_DESIGN.json',{'global':pool[keep].tolist(),'development_predicted_bo
    ↪  undary':critical.tolist(),'global_seed':93071,'kept_pool_indices':keep,'critical_labe
    ↪  l':'potential static tree, only selected HF queried; not development-predicted
    ↪  band','historical_test_coordinates':fp(old_path),'labels':'not_evaluated'})
    policy.update({'artifact_id':'Q4-FOLLOWUP-v1','seeds':[137,271,503],'test_global_count':1
    ↪  6,'test_boundary_count':516,'test_global_seed':93071,
        'test_design':fp(out/'TEST_DESIGN.json'),'training_design':fp(out/'TRAINING_DESIGNS.j
        ↪  son'),'solver_identity':solver_identity(),'algorithm_identity':algorithm_identity
        ↪  (),
```

```
        'design_generator':fp(__file__),'parent_policy':fp(prior/'FORMAL_POLICY.json'),'test_
        ↪  access':'12 models and all532 static predictions sealed before any new validation
        ↪  truth',
        'boundary_definition':'4 fixed zV*129 mu nodes; actual slices zV3/8,7/8; initial
        ↪  j0,64,128; leftmost adjacent opposite classes; at most4 integer bisections',
        'critical_max_H':14,'critical_max_L':4,'old24_role':'historical only, never imported
        ↪  as new test','independent_review':'pending'})
    for dep in policy['dependencies']:
        source=Path(dep['path']);shutil.copy2(source,out/source.name)
    write_json(out/'FORMAL_POLICY.json',policy)


def bound_file(policy_path,record):
    candidates=[Path(policy_path).parent/Path(record['path']).name,Path(record['path'])]
    for p in candidates:
        if p.is_file() and fp(p)['sha256']==record['sha256']:return p
    raise ValueError('bound policy dependency missing or changed: '+record['path'])


class PhysicsRegistry:
    def __init__(self,out,args,policy):
        self.out=Path(out);self.args=args;self.policy=policy;self.records={};self.requests=[]
        self.out.mkdir();self.source_hash=fp(__file__)['sha256'];self.terrain=fp(Path(args.in
        ↪  puts)/'copernicus_support.npz')['sha256']
    def key(self,x,level):
        import hashlib
        config=config_from_plane(x,self.policy);dx=self.policy['low_dx_m'] if level=='L' else
        ↪  self.policy['high_dx_m']
        body={'config':config,'dx':dx,'horizon':self.args.horizon,'dry':self.args.dry,'cfl':s
        ↪  elf.args.cfl,'backend':self.args.backend,'source':self.source_hash,'terrain':self
        ↪  .terrain}
        return hashlib.sha256(json.dumps(body,sort_keys=True).encode()).hexdigest(),config,dx
    def request(self,x,level,role):
        key,config,dx=self.key(x,level);cached=key in self.records
        if not cached:
            row=execute_physics(self.out/key[:20],config,self.args,dx);row.update({'key':key,
            ↪  'x':np.asarray(x).tolist(),'level':level});self.records[key]=row
            write_json(self.out/'REGISTRY.json',list(self.records.values()))
        self.requests.append({'key':key,'role':role,'cache_hit':cached});return key
    def batch(self,x,level,role):
        from concurrent.futures import ThreadPoolExecutor,as_completed
        planned={};keys=[]
        for point in x:
            key,config,dx=self.key(point,level);keys.append(key)
            if key in
            ↪  self.records:self.requests.append({'key':key,'role':role,'cache_hit':True})
            else:planned[key]=(point,config,dx)
        with ThreadPoolExecutor(max_workers=self.args.workers) as pool:
            futures={pool.submit(execute_physics,self.out/key[:20],c,self.args,dx):key for
            ↪  key,(point,c,dx) in planned.items()}
            for future in as_completed(futures):
                key=futures[future];point,_,_=planned[key];row=future.result();row.update({'k
                ↪  ey':key,'x':np.asarray(point).tolist(),'level':level});self.records[key]=
                ↪  row
```

```
                self.requests.append({'key':key,'role':role,'cache_hit':False});write_json(se
                ↪  lf.out/'REGISTRY.json',list(self.records.values()))
                print(json.dumps({'role':role,'level':level,'new_completed':len(self.records)
                ↪  ,'status':row['status'],'g':row.get('response_g'),'wall_s':row['total_wal
                ↪  l_s']}),flush=True)
        return keys
    def data(self,keys):
        rows=[self.records[k] for k in keys]
        if any(not r['valid'] for r in rows):raise RuntimeError('invalid physical training
        ↪  query; unknown is not a negative label')
        return np.array([r['x'] for r in rows]),np.array([r['response_g'] for r in rows])


def save_gp_state(path,model,keys_low,keys_high,registry,seed,method):
    state={'seed':seed,'method':method,'fit':model.fit_record,'low_keys':keys_low,'high_keys'
    ↪  :keys_high,'scope':'conditional latent GP; no natural probability/95%
    ↪  claim'}
    if keys_low:
        x,y=registry.data(keys_low);state.update({'xL':x.tolist(),'yL':y.tolist()})
    x,y=registry.data(keys_high);state.update({'xH':x.tolist(),'yH':y.tolist()});write_json(p
    ↪  ath,state)


def method_predictions(folder,model,test,high_keys,registry):
    started=time.perf_counter()
    mean,std=model.predict(test)
    np.savez(folder/'TEST_PREDICTIONS_FROZEN.npz',x=test,mean_g=mean,std_g=std)

    axis=np.linspace(0,1,61);surface=np.array([(a,b) for b in axis for a in axis]);ms=[];ss=[]
    for start in range(0,len(surface),512):
        a,b=model.predict(surface[start:start+512]);ms.extend(a);ss.extend(b)
    np.savez_compressed(folder/'SURFACE_PREDICTIONS.npz',axis=axis,mean_g=np.array(ms).reshap
    ↪  e(61,61),std_g=np.array(ss).reshape(61,61))
    write_json(folder/'PREDICTION_SCOPE.json',{'model':'arrival margin g
    ↪  only','time_proxy':'not_modelled; physical first-arrival brackets retained in query
    ↪  results','not_a_natural_probability':True})
    return time.perf_counter()-started


def run_formal_training(out,args):

    import csv
    policy_path=Path(args.policy);policy=json.loads(policy_path.read_text())
    validate_execution(policy,args,policy_path)
    design_path=bound_file(policy_path,policy['training_design']);test_path=bound_file(policy
    ↪  _path,policy['test_design'])
    designs=json.loads(design_path.read_text());test_design=json.loads(test_path.read_text())
    test=np.r_[np.array(test_design['global']),np.array(test_design['development_predicted_bo
    ↪  undary'])]
    args.horizon=policy['horizon_s'];registry=FollowupRegistry(out/'physics',args,policy);reg
    ↪  istry.import_cache(args.cache_registry);methods=[]
    if args.training_dir is not None:methods=list(import_seed137(out,args,registry,test))
    write_json(out/'METHODS.json',methods)
```

```
write_json(out/'EXECUTED_POLICY_BINDING.json',{'policy':fp(policy_path),'training_design'
↪  :fp(design_path),'test_design':fp(test_path),'source':fp(__file__),'solver':solver_id
↪  entity(),'test_labels':'not_accessed'})
for seed in policy['seeds']:
    if seed==137 and args.training_dir is not None:continue
    d=designs[str(seed)];xl=np.array(d['low']);hi0=np.array(d['high_initial_indices']);hi
    ↪  p=np.array(d['high_passive_added_indices'])

    pool=np.array(d['active_candidates']);integration=np.array(d['integration_grid'])
    matched_pool=np.array(d['matched_passive_low_pool'])
    if min(np.linalg.norm(np.r_[xl,pool,matched_pool][:,None,:]-test[None,:,:],axis=-1).r
    ↪  avel())<1e-12:raise ValueError('test/train or acquisition coordinate
    ↪  collision')
    low=registry.batch(xl,'L',f'seed{seed}_shared_L24');initial=registry.batch(xl[hi0],'H
    ↪  ',f'seed{seed}_shared_H8')
    passive_added=registry.batch(xl[hip],'H',f'seed{seed}_passive_H8');passive=initial+pa
    ↪  ssive_added
    xh,yh=registry.data(passive);xlow,ylow=registry.data(low)
    for tag in ('A_single_fill','B_MF_fill'):
        folder=out/f'seed{seed}'/tag;folder.mkdir(parents=True);started=time.perf_counter
        ↪  ()
        model=SingleGP().fit(xh,yh) if tag.startswith('A') else
        ↪  ARGP().fit(xlow,ylow,xh,yh)
        fit_s=time.perf_counter()-started;save_gp_state(folder/'GP_STATE.json',model,[]
        ↪  if tag.startswith('A') else low,passive,registry,seed,tag)
        aux_s=method_predictions(folder,model,test,passive,registry)
        methods.append({'seed':seed,'method':tag,'folder':str(folder.relative_to(out)),'l
        ↪  ow_keys':[] if tag.startswith('A') else
        ↪  low,'high_keys':passive,'fit_wall_s':fit_s,'acquisition_wall_s':0.,'auxiliary
        ↪  _prediction_wall_s':aux_s})
        write_json(out/'METHODS.json',methods)
    folder=out/f'seed{seed}'/'C_MF_active';folder.mkdir(parents=True);lkeys=list(low);hke
    ↪  ys=list(initial);history=[];nlow=0;nhigh=0;iteration=0;fit_s=0.;acq_s=0.
    while nhigh<policy['active_high_added']:
        iteration+=1;xlow,ylow=registry.data(lkeys);xh,yh=registry.data(hkeys)
        started=time.perf_counter();model=ARGP().fit(xlow,ylow,xh,yh);fit_s+=time.perf_co
        ↪  unter()-started
        save_gp_state(folder/f'action{iteration:02d}_MODEL.json',model,lkeys,hkeys,regist
        ↪  ry,seed,'C_MF_active')
        started=time.perf_counter();explore=iteration%4==0
        scores=model.acquisition(pool,integration,policy['estimated_cost_L_s'],policy['es
        ↪  timated_cost_H_s'],explore=explore)

        fallback=not any(np.any(v>0) for v in scores.values())
        if fallback:scores=model.acquisition(pool,integration,policy['estimated_cost_L_s'
        ↪  ],policy['estimated_cost_H_s'],explore=True)
        eligibleL=np.min(np.linalg.norm(pool[:,None,:]-xlow[None,:,:],axis=-1),axis=1)>1e
        ↪  -12
        eligibleH=np.min(np.linalg.norm(pool[:,None,:]-xh[None,:,:],axis=-1),axis=1)>1e-12
        if nlow>=policy['active_low_cap']:eligibleL[:]=False
        scoreL=np.where(eligibleL,scores['L'],-np.inf);scoreH=np.where(eligibleH,scores['
        ↪  H'],-np.inf)
        chosen=int(np.argmax(np.r_[scoreL,scoreH]));level='L' if chosen<len(pool) else
        ↪  'H';index=chosen%len(pool);point=pool[index]
```

```
        np.savez(folder/f'action{iteration:02d}_ACQUISITION.npz',candidates=pool,integrat
        ↪  ion=integration,scoreL=scores['L'],scoreH=scores['H'],eligibleL=eligibleL,eli
        ↪  gibleH=eligibleH)
        predicted_mean,predicted_std=model.predict(point[None,:]);acq_s+=time.perf_counte
        ↪  r()-started
        key=registry.request(point,level,f'seed{seed}_active_action{iteration}');record=r
        ↪  egistry.records[key]
        if not record['valid']:raise RuntimeError('active physical query invalid; method
        ↪  aborted, not relabeled')
        if level=='L':lkeys.append(key);nlow+=1
        else:hkeys.append(key);nhigh+=1
        history.append({'iteration':iteration,'x':point.tolist(),'level':level,'key':key,
        ↪  'exploration':explore,'underflow_global_fallback':fallback,
            'predicted_H_mean':float(predicted_mean[0]),'predicted_H_std':float(predicted
            ↪  _std[0]),'selected_score':float(scores[level][index]),
            'cost_estimate_s':policy['estimated_cost_'+level+'_s'],'actual_total_wall_s':
            ↪  record['total_wall_s'],'actual_g':record['response_g']})
        write_json(folder/'ACTIVE_HISTORY.json',history);write_json(folder/'ACTIVE_TIMING
        ↪  .json',{'fit_wall_s':fit_s,'acquisition_wall_s':acq_s,'high_added':nhigh,'low
        ↪  _added':nlow});print(json.dumps({'seed':seed,'active_action':iteration,'level
        ↪  ':level,'H_added':nhigh,'L_added':nlow}),flush=True)
    xlow,ylow=registry.data(lkeys);xh,yh=registry.data(hkeys);started=time.perf_counter()
    ↪  ;model=ARGP().fit(xlow,ylow,xh,yh);fit_s+=time.perf_counter()-started
    save_gp_state(folder/'GP_STATE.json',model,lkeys,hkeys,registry,seed,'C_MF_active');a
    ↪  ux_s=method_predictions(folder,model,test,hkeys,registry)
    methods.append({'seed':seed,'method':'C_MF_active','folder':str(folder.relative_to(ou
    ↪  t)),'low_keys':lkeys,'high_keys':hkeys,'fit_wall_s':fit_s,'acquisition_wall_s':ac
    ↪  q_s,'auxiliary_prediction_wall_s':aux_s})
    write_json(out/'METHODS.json',methods)


    extra=registry.batch(matched_pool[:nlow],'L',f'seed{seed}_Bm_matched_passive_L{nlow}'
    ↪  ) if nlow else
    ↪  []
    bm_low=low+extra;xlow,ylow=registry.data(bm_low);xh,yh=registry.data(passive)
    bm_folder=out/f'seed{seed}'/'Bm_MF_count_matched';bm_folder.mkdir();started=time.perf
    ↪  _counter();bm=ARGP().fit(xlow,ylow,xh,yh);bm_fit=time.perf_counter()-started
    save_gp_state(bm_folder/'GP_STATE.json',bm,bm_low,passive,registry,seed,'Bm_MF_count_
    ↪  matched');bm_aux=method_predictions(bm_folder,bm,test,passive,registry)
    methods.append({'seed':seed,'method':'Bm_MF_count_matched','folder':str(bm_folder.rel
    ↪  ative_to(out)),'low_keys':bm_low,'high_keys':passive,'fit_wall_s':bm_fit,'acquisi
    ↪  tion_wall_s':0.,'auxiliary_prediction_wall_s':bm_aux})
    write_json(out/'METHODS.json',methods)
def local_fp(path):
    p=Path(path);record=fp(p);record['path']=str(p.relative_to(out));return record
prediction_manifest=[local_fp(out/m['folder']/'TEST_PREDICTIONS_FROZEN.npz') for m in
↪  methods]
expected=len(policy['seeds'])*len(policy['methods'])
if len(prediction_manifest)!=expected:raise RuntimeError('all method predictions must be
↪  frozen before test truth')
validate_prediction_models(methods,list(registry.records.values()),test,policy,out)
if any(np.min(np.linalg.norm(np.array(r['x'])[None,:]-test,axis=1))<1e-12 for r in
↪  registry.records.values()):raise ValueError('test coordinate already present in
↪  training physical library')
```

```
    write_json(out/'METHODS.json',methods);write_json(out/'QUERY_REQUESTS.json',registry.requ
    ↪  ests)
    seal={'phase':'training_complete_test_not_run','predictions':prediction_manifest,'model_s
    ↪  tates':[local_fp(out/m['folder']/'GP_STATE.json') for m in
    ↪  methods],
        'policy':fp(policy_path),'test_design':fp(test_path),'source':fp(__file__),'solver':s
        ↪  olver_identity(),'methods':local_fp(out/'METHODS.json'),
        'registry':local_fp(out/'physics/REGISTRY.json'),'requests':local_fp(out/'QUERY_REQUE
        ↪  STS.json'),'terrain':fp(Path(args.inputs)/'copernicus_support.npz'),
        'numerics':{'order':args.order,'backend':args.backend,'dry':args.dry,'cfl':args.cfl,'
        ↪  maxdt':args.maxdt},'no_test_label_coordinates_in_registry':True}
    write_json(out/'PHASE1_SEAL.json',seal);print(json.dumps({'phase1_seal':fp(out/'PHASE1_SE
    ↪  AL.json'),'test_queries':0}),flush=True)


def seal_cached_seed(out,args):


    import shutil,importlib.util,sys
    parent=Path(args.training_dir);policy_path=Path(args.policy);policy=json.loads(policy_pat
    ↪  h.read_text())
    validate_execution(policy,args,policy_path)
    binding=json.loads((parent/'EXECUTED_POLICY_BINDING.json').read_text())
    snapshot=parent/'SOURCE_SNAPSHOT.py'
    if fp(snapshot)['sha256']!=binding['source']['sha256']:raise ValueError('parent source
    ↪  snapshot changed')
    spec=importlib.util.spec_from_file_location('q4_prior_cache_source',snapshot);prior=impor
    ↪  tlib.util.module_from_spec(spec);sys.modules[spec.name]=prior;spec.loader.exec_module
    ↪  (prior)
    if prior.solver_identity()['sha256']!=solver_identity()['sha256'] or
    ↪  binding['solver']['sha256']!=solver_identity()['sha256']:raise ValueError('physical
    ↪  solver differs from cached labels')
    if (parent/'TEST_ACCESS_STARTED.json').exists():raise ValueError('parent test already
    ↪  accessed')
    td=json.loads(bound_file(policy_path,policy['test_design']).read_text());test=np.r_[td['g
    ↪  lobal'],td['development_predicted_boundary']]
    all_rows=json.loads((parent/'physics/REGISTRY.json').read_text());records={r['key']:r for
    ↪  r in all_rows};methods=[];used=set();lineage=[]
    for tag in policy['methods']:
        relative=Path('seed137')/tag;old=parent/relative;folder=out/relative
        shutil.copytree(old,folder)
        state=json.loads((folder/'GP_STATE.json').read_text())
        oldpred=folder/'TEST_PREDICTIONS_FROZEN.npz';saved=np.load(oldpred);ox=saved['x'].cop
        ↪  y();om=saved['mean_g'].copy();os=saved['std_g'].copy();saved.close()
        indices=[]
        for point in test:
            found=np.flatnonzero(np.all(ox==point,axis=1))
            if len(found)!=1:raise ValueError('new coordinate not uniquely in original frozen
            ↪  predictions')
            indices.append(int(found[0]))
        oldpred.rename(folder/'TEST_PREDICTIONS_PARENT48.npz')
        np.savez(oldpred,x=ox[indices],mean_g=om[indices],std_g=os[indices])
        keys=state['low_keys']+state['high_keys'];used.update(keys)
        methods.append({'seed':137,'method':tag,'folder':str(relative),'low_keys':state['low_
        ↪  keys'],'high_keys':state['high_keys'],
```

```
            'fit_wall_s':None,'acquisition_wall_s':None,'auxiliary_prediction_wall_s':None,
            'cost_missing_reason':'original interrupted parent saved fitted states but did
            ↪  not persist per-method timing; unknown, never zero-filled'})
        lineage.append({'method':tag,'original_state':fp(old/'GP_STATE.json'),'copied_state':
        ↪  fp(folder/'GP_STATE.json'),
            'original_prediction':fp(old/'TEST_PREDICTIONS_FROZEN.npz'),'selected_indices':in
            ↪  dices,'new_prediction':fp(oldpred),'GP_refitted':False})
    rows=[r for r in all_rows if r['key'] in used]
    for row in rows:
        if not row['valid']:raise ValueError('invalid cache label')
        for field in ('result','receipt'):
            path=Path(row[field]['path'])
            if fp(path)['sha256']!=row[field]['sha256']:raise ValueError('actual query
            ↪  evidence changed')
        receipt=json.loads(Path(row['receipt']['path']).read_text())
        if receipt['failure'] is not None or
        ↪  receipt['code']['sha256']!=binding['source']['sha256']:raise ValueError('cache
        ↪  source/run receipt mismatch')
    validate_prediction_models(methods,rows,test,policy,out)
    (out/'physics').mkdir();write_json(out/'physics/REGISTRY.json',rows);write_json(out/'METH
    ↪  ODS.json',methods)
    write_json(out/'QUERY_REQUESTS.json',[{'key':k,'role':'cache_import_model_membership_not_
    ↪  original_request_history','cache_hit':True} for k in
    ↪  sorted(used)])
    write_json(out/'CACHE_MIGRATION.json',{'parent_binding':binding,'parent_source':fp(snapsh
    ↪  ot),'new_source':fp(__file__),
        'same_physical_solver':solver_identity(),'lineage':lineage,'queries_retained':len(row
        ↪  s),'original_parent_completed':False,
        'test_labels_accessed':False,'new_PDE_queries':0,'per_method_timing':'not
        ↪  recoverable; null, not imputed'})
    def local_fp(p):
        p=Path(p);a=fp(p);a['path']=str(p.relative_to(out));return a
    seal={'phase':'training_complete_test_not_run','predictions':[local_fp(out/m['folder']/'T
    ↪  EST_PREDICTIONS_FROZEN.npz') for m in
    ↪  methods],
        'model_states':[local_fp(out/m['folder']/'GP_STATE.json') for m in
        ↪  methods],'policy':fp(policy_path),
        'test_design':fp(bound_file(policy_path,policy['test_design'])),'source':fp(__file__)
        ↪  ,'solver':solver_identity(),
        'methods':local_fp(out/'METHODS.json'),'registry':local_fp(out/'physics/REGISTRY.json
        ↪  '),'requests':local_fp(out/'QUERY_REQUESTS.json'),
        'terrain':fp(Path(args.inputs)/'copernicus_support.npz'),'numerics':{k:getattr(args,k
        ↪  ) for k in
        ↪  ['order','backend','dry','cfl','maxdt']},
        'no_test_label_coordinates_in_registry':True,'migration':local_fp(out/'CACHE_MIGRATIO
        ↪  N.json')}
    if any(np.min(np.linalg.norm(np.asarray(r['x'])[None,:]-test,axis=1))<1e-12 for r in
    ↪  rows):raise ValueError('test/training collision')
    write_json(out/'PHASE1_SEAL.json',seal)


def run_formal_test(out,args):
```

```
    training=Path(args.training_dir);seal_path=training/'PHASE1_SEAL.json';seal=json.loads(se
    ↪  al_path.read_text());policy_path=Path(args.policy);policy=json.loads(policy_path.read
    ↪  _text())
    expected=len(policy['seeds'])*len(policy['methods'])
    if seal['phase']!='training_complete_test_not_run' or len(seal['predictions'])!=expected
    ↪  or len(seal['model_states'])!=expected:raise ValueError('complete all-model phase1
    ↪  seal required')
    if fp(__file__)['sha256']!=seal['source']['sha256'] or
    ↪  fp(policy_path)['sha256']!=seal['policy']['sha256']:raise ValueError('test
    ↪  source/policy changed since training seal')
    if solver_identity()['sha256']!=seal['solver']['sha256']:raise ValueError('test solver
    ↪  identity differs')
    validate_execution(policy,args,policy_path)
    if fp(Path(args.inputs)/'copernicus_support.npz')['sha256']!=seal['terrain']['sha256']:ra
    ↪  ise ValueError('test terrain
    ↪  differs')
    for key,value in seal['numerics'].items():
        if getattr(args,key)!=value:raise ValueError('test numerical settings differ: '+key)
    for record in seal['predictions']+seal['model_states']+[seal['methods'],seal['registry'],
    ↪  seal['requests']]:
        p=training/record['path']
        if not p.is_file() or fp(p)['sha256']!=record['sha256']:raise ValueError('sealed
        ↪  training artifact changed: '+str(p))
    test_path=bound_file(policy_path,policy['test_design']);td=json.loads(test_path.read_text
    ↪  ());test=np.r_[np.array(td['global']),np.array(td['development_predicted_boundary'])]
    methods=json.loads((training/'METHODS.json').read_text());old_records=json.loads((trainin
    ↪  g/'physics/REGISTRY.json').read_text())
    validate_prediction_models(methods,old_records,test,policy,training)
    if any(np.min(np.linalg.norm(np.array(r['x'])[None,:]-test,axis=1))<1e-12 for r in
    ↪  old_records):raise ValueError('test truth already present in sealed training library')
    for p in training.rglob('TEST_KEYS.json'):raise ValueError('training directory already
    ↪  contains test labels')
    access=training/'TEST_ACCESS_STARTED.json'
    with access.open('x') as f:
        json.dump({'phase1_seal':fp(seal_path),'source':fp(__file__),'test_output':str(out),'
        ↪  purpose':'first independent test access; not
        ↪  training/tuning'},f,ensure_ascii=False,indent=2)
    write_json(out/'PHASE2_GATE.json',{'phase1_seal':fp(seal_path),'source':fp(__file__),'pol
    ↪  icy':fp(policy_path),'test_design':fp(test_path),'all_checks':'pass','test_queries_be
    ↪  fore_gate':0})
    args.horizon=policy['horizon_s'];registry=PhysicsRegistry(out/'physics',args,policy);regi
    ↪  stry.records={r['key']:r for r in
    ↪  old_records}
    test_high=registry.batch(test,'H','shared_independent_test_H24');test_low=registry.batch(
    ↪  test,'L','shared_independent_test_L24')
    write_json(out/'METHODS.json',methods);write_json(out/'TEST_KEYS.json',{'H':test_high,'L'
    ↪  :test_low});write_json(out/'QUERY_REQUESTS.json',registry.requests)
    evaluate_formal_results(out,policy,methods,registry,test_high,test_low,training)


def evaluate_formal_results(out,policy,methods,registry,test_high,test_low,training_root):
    import csv
    rowsH=[registry.records[k] for k in test_high];rowsL=[registry.records[k] for k in
    ↪  test_low]
```

```
truth=np.array([r.get('response_g') if r['valid'] else np.nan for r in rowsH],dtype=float)
low=np.array([r.get('response_g') if r['valid'] else np.nan for r in rowsL],dtype=float)
numeric_gray=np.isfinite(truth)&np.isfinite(low)&(abs(truth)<=abs(truth-low))
rows=[];costs=[]
for method in methods:
    folder=Path(training_root)/method['folder'];f=np.load(folder/'TEST_PREDICTIONS_FROZEN
    ↪  .npz');pred=f['mean_g']
    keys=list(dict.fromkeys(method['low_keys']+method['high_keys']));q=[registry.records[
    ↪  k] for k in
    ↪  keys]
    costs.append({'seed':method['seed'],'method':method['method'],'LF_queries':len(method
    ↪  ['low_keys']),'HF_queries':len(method['high_keys']),
        'physical_child_wall_sum_s':sum(r['total_wall_s'] for r in
        ↪  q),'solver_wall_sum_s':sum(r['solver_wall_s'] for r in q),
        'compile_load_sum_s':sum(r['compile_and_load_s'] for r in q),'GP_fit_wall_s':meth
        ↪  od['fit_wall_s'],'acquisition_wall_s':method['acquisition_wall_s'],'auxiliary
        ↪  _prediction_wall_s':method['auxiliary_prediction_wall_s'],
        'logical_cold_start_total_s':sum(r['total_wall_s'] for r in q)+sum(method[k] for
        ↪  k in ['fit_wall_s','acquisition_wall_s','auxiliary_prediction_wall_s']) if
        ↪  all(method[k] is not None for k in
        ↪  ['fit_wall_s','acquisition_wall_s','auxiliary_prediction_wall_s']) else None})
    for label,index in [('global',np.arange(policy['test_global_count'])),('development_p
    ↪  redicted_boundary',np.arange(policy['test_global_count'],len(truth)))]:
        valid=index[np.isfinite(truth[index])];y=truth[valid];p=pred[valid];actual=np.arr
        ↪  ay([rowsH[i]['status']=='arrived' for i in
        ↪  valid]);positive=p>=0
        TP=int((actual&positive).sum());FN=int((actual&~positive).sum());FP=int((~actual&
        ↪  positive).sum());TN=int((~actual&~positive).sum())
        margin_true=.1*(2*np.exp(y)-1)-.1;margin_pred=np.maximum(0,.1*(2*np.exp(p)-1))-.1
        rows.append({'seed':method['seed'],'method':method['method'],'stratum':label,'n':
        ↪  len(index),'valid_n':len(valid),'unknown_n':len(index)-len(valid),
            'TP':TP,'FN':FN,'FP':FP,'TN':TN,'disagreement_rate':float((FN+FP)/len(valid))
            ↪  if len(valid) else None,
            'arrival_miss_rate':FN/(TP+FN) if TP+FN else
            ↪  None,'RMSE_g':float(np.sqrt(np.mean((p-y)**2))) if len(valid) else None,
            'MAE_g':float(np.mean(abs(p-y))) if len(valid) else None,'RMSE_raw_margin_m':
            ↪  float(np.sqrt(np.mean((margin_true-margin_pred)**2))) if len(valid) else
            ↪  None,
            'numerically_unresolved_n':int(numeric_gray[index].sum()),'true_arrival_n':TP
            ↪  +FN,'time_proxy':'not_modelled; physical arrival brackets separately
            ↪  retained'})
for filename,data in [('COMPARISON_METRICS.csv',rows),('METHOD_COSTS.csv',costs)]:
    with (out/filename).open('w') as f:
        writer=csv.DictWriter(f,fieldnames=list(data[0]));writer.writeheader();writer.wri
        ↪  terows(data)
write_json(out/'TEST_NUMERICAL_STATUS.json',{'gH':[float(x) if np.isfinite(x) else None
↪  for x in truth],'gL':[float(x) if np.isfinite(x) else None for x in low],
    'numerically_unresolved':numeric_gray.tolist(),'rule':policy['numerically_unresolved_
    ↪  rule'],'meaning':'empirical discrepancy, not a certified continuum error
    ↪  bound'})
write_json(out/'EXPERIMENT_STATUS.json',{'generated':True,'methods':len(methods),'test_po
↪  ints':len(test_high),'test_query_unique_H':len(set(test_high)),'test_query_unique_L':
↪  len(set(test_low)),
    'physical_unique_queries':len(registry.records),'independent_review':'pending','test_
    ↪  used_for_tuning':False,'natural_validation':False})
```

```
def load_fixed_gp(state):

    fit=state['fit'];theta=np.asarray(fit['theta']);noise=fit['jitter_normalized_variance']
    if state['method']=='A_single_fill':
        m=SingleGP();m.x=np.asarray(state['xH']);m.mean=fit['mean'];m.scale=fit['scale'];m.y=
        ↪  (np.asarray(state['yH'])-m.mean)/m.scale
        K=matern52(m.x,m.x,np.exp(theta[0]),np.exp(theta[1:]))+noise*np.eye(len(m.x))
    else:
        m=ARGP();m.xl=np.asarray(state['xL']);m.xh=np.asarray(state['xH']);m.nL=len(m.xl);m.d
        ↪  im=m.xl.shape[1]
        m.ml=fit['mean_L'];m.mh=fit['mean_H'];m.scale=fit['output_scale'];m.y=np.r_[(np.asarr
        ↪  ay(state['yL'])-m.ml)/m.scale,(np.asarray(state['yH'])-m.mh)/m.scale]
        def unpack(t):
            k=m.dim;return
            ↪  np.exp(t[0]),np.exp(t[1:k+1]),np.exp(t[k+1]),np.exp(t[k+2:2*k+2]),t[-1]
        m.unpack=unpack;al,ll,ad,ld,rho=unpack(theta)
        A=matern52(m.xl,m.xl,al,ll);B=rho*matern52(m.xl,m.xh,al,ll)
        D=rho*rho*matern52(m.xh,m.xh,al,ll)+matern52(m.xh,m.xh,ad,ld)
        K=np.block([[A,B],[B.T,D]])+noise*np.eye(len(m.y))
    m.theta=theta;m.noise=noise;m.fit_record=fit;m.cf=cho_factor(K,lower=True);m.alpha=cho_so
    ↪  lve(m.cf,m.y)
    return m


def cache_logical_id(point,level,args,policy):
    config=config_from_plane(point,policy);dx=policy['low_dx_m'] if level=='L' else
    ↪  policy['high_dx_m']
    body={'config':config,'dx':dx,'horizon':args.horizon,'dry':args.dry,'cfl':args.cfl,'backe
    ↪  nd':args.backend,
          'order':args.order,'maxdt':args.maxdt,'entry_width':args.entry_width,'solver':solve
          ↪  r_identity()['sha256'],
          'assets':[(r['logical_id'],r['sha256']) for r in policy['input_assets']]}
    return hashlib.sha256(json.dumps(body,sort_keys=True).encode()).hexdigest()


class FollowupRegistry(PhysicsRegistry):
    def __init__(self,out,args,policy):
        super().__init__(out,args,policy);self.aliases={};self.imported=[]
    def key(self,x,level):
        logical=cache_logical_id(x,level,self.args,self.policy)
        return self.aliases.get(logical,logical),config_from_plane(x,self.policy),self.policy
        ↪  ['low_dx_m'] if level=='L' else
        ↪  self.policy['high_dx_m']
    def import_cache(self,path):
        if path is None:return
        parent=Path(path);rows=json.loads(parent.read_text());snapshot=parent.parent.parent/'
        ↪  SOURCE_SNAPSHOT.py'
        import importlib.util,sys
        spec=importlib.util.spec_from_file_location('followup_cache_source',snapshot);prior=i
        ↪  mportlib.util.module_from_spec(spec);sys.modules[spec.name]=prior;spec.loader.exe
        ↪  c_module(prior)
        if prior.solver_identity()!=solver_identity():raise ValueError('cache physical solver
        ↪  changed')
```

```
        for row in rows:
            if not row['valid']:raise ValueError('invalid cached label')
            expected=config_from_plane(row['x'],self.policy)
            if row['config']!=expected or row['dx_m']!=(40 if row['level']=='L' else
            ↪  30):raise ValueError('cached coordinate/config/fidelity mismatch')
            for field in ('receipt','result'):
                if fp(Path(row[field]['path']))['sha256']!=row[field]['sha256']:raise
                ↪  ValueError('cached evidence hash mismatch')
            receipt=json.loads(Path(row['receipt']['path']).read_text())
            if receipt['failure'] is not None or not receipt['source_unchanged_during_run']
            ↪  or receipt['code']['sha256']!=fp(snapshot)['sha256']:raise ValueError('cache
            ↪  source or actual run mismatch')
            argv=receipt['argv']
            for flag,value in [('--horizon',self.args.horizon),('--dry',self.args.dry),('--cf
            ↪  l',self.args.cfl),('--order',self.args.order),('--maxdt',self.args.maxdt),('-
            ↪  -entry-width',self.args.entry_width)]:
                if flag not in argv or float(argv[argv.index(flag)+1])!=value:raise
                ↪  ValueError('cache numerics mismatch: '+flag)
            if argv[argv.index('--backend')+1]!=self.args.backend or '--full' in argv:raise
            ↪  ValueError('cache execution scope mismatch')
            assets={Path(a['path']).name:a['sha256'] for a in receipt['inputs']}
            if any(assets.get(a['logical_id'])!=a['sha256'] for a in
            ↪  self.policy['input_assets']):raise ValueError('cache terrain/input assets
            ↪  changed')
            logical=cache_logical_id(row['x'],row['level'],self.args,self.policy)
            if logical in self.aliases and self.aliases[logical]!=row['key']:raise
            ↪  ValueError('conflicting logical cache entries')
            self.aliases[logical]=row['key'];self.records[row['key']]=row
            self.imported.append({'logical_id':logical,'original_key':row['key'],'original_re
            ↪  ceipt':row['receipt'],'original_result':row['result'],'origin_source':receipt
            ↪  ['code'],'reused_without_PDE':True})
        write_json(self.out/'REGISTRY.json',list(self.records.values()));write_json(self.out/
        ↪  'CACHE_IMPORT.json',{'cache_registry':fp(parent),'source':fp(snapshot),'same_solv
        ↪  er':solver_identity(),'entries':self.imported})


def import_seed137(out,args,registry,test):
    import shutil
    parent=Path(args.training_dir);seal=json.loads((parent/'PHASE1_SEAL.json').read_text());m
    ↪  etas=json.loads((parent/'METHODS.json').read_text());lineage=[]
    if seal['solver']!=solver_identity():raise ValueError('seed137 model science differs')
    for meta in metas:
        if meta['seed']!=137:raise ValueError('only seed137 imported')
        relative=Path(meta['folder']);old=parent/relative;folder=out/relative;shutil.copytree
        ↪  (old,folder)
        for record in seal['model_states']:
            if Path(record['path'])==relative/'GP_STATE.json' and
            ↪  fp(old/'GP_STATE.json')['sha256']!=record['sha256']:raise ValueError('seed137
            ↪  sealed state changed')
        state=json.loads((folder/'GP_STATE.json').read_text());model=load_fixed_gp(state);sav
        ↪  ed=np.load(old/'TEST_PREDICTIONS_FROZEN.npz');mean,std=model.predict(saved['x'])
        if not np.allclose(mean,saved['mean_g'],rtol=0,atol=1e-10) or not
        ↪  np.allclose(std,saved['std_g'],rtol=0,atol=1e-10):raise
        ↪  ValueError('fixed-parameter model restore differs')
```

```
        (folder/'TEST_PREDICTIONS_FROZEN.npz').rename(folder/'HISTORICAL24_PREDICTIONS.npz')
        aux=method_predictions(folder,model,test,meta['high_keys'],registry)
        meta={**meta,'fit_wall_s':None,'acquisition_wall_s':None,'auxiliary_prediction_wall_s
        ↪  ':aux,'cost_missing_reason':'historical137 fit/acquisition missing; new548
        ↪  prediction wall measured'}
        lineage.append({'method':meta['method'],'original_state':fp(old/'GP_STATE.json'),'new
        ↪  _state':fp(folder/'GP_STATE.json'),'GP_refitted':False,'historical_prediction_max
        ↪  _abs_difference':float(abs(mean-saved['mean_g']).max())})
        yield meta
    write_json(out/'SEED137_REUSE.json',{'parent_seal':fp(parent/'PHASE1_SEAL.json'),'entries
    ↪  ':lineage,'new_PDE':0,'new_fit':0})


def run_followup_validation(out,args):

    training=Path(args.training_dir);policy_path=Path(args.policy);policy=json.loads(policy_p
    ↪  ath.read_text());seal=json.loads((training/'PHASE1_SEAL.json').read_text())
    validate_execution(policy,args,policy_path)
    if seal['source']['sha256']!=fp(__file__)['sha256'] or
    ↪  seal['policy']['sha256']!=fp(policy_path)['sha256']:raise ValueError('sealed
    ↪  source/policy changed')
    for rec in seal['predictions']+seal['model_states']+[seal['methods'],seal['registry'],sea
    ↪  l['requests']]:
        if fp(training/rec['path'])['sha256']!=rec['sha256']:raise ValueError('sealed
        ↪  model/library changed')
    td=json.loads(bound_file(policy_path,policy['test_design']).read_text());test=np.r_[td['g
    ↪  lobal'],td['development_predicted_boundary']]
    methods=json.loads((training/'METHODS.json').read_text());old=json.loads((training/'physi
    ↪  cs/REGISTRY.json').read_text());validate_prediction_models(methods,old,test,policy,tr
    ↪  aining)
    with (training/'FOLLOWUP_TEST_ACCESS_STARTED.json').open('x') as f:json.dump({'source':fp
    ↪  (__file__),'seal':fp(training/'PHASE1_SEAL.json'),'labels_before_access':0},f)
    write_json(out/'PHASE2_GATE.json',{'source':fp(__file__),'seal':fp(training/'PHASE1_SEAL.
    ↪  json'),'all_declared_models_and532_predictions_immutable':True})
    registry=FollowupRegistry(out/'physics',args,policy)

    globalH=registry.batch(test[:16],'H','new_global16_H') if
    ↪  policy.get('run_new_global_truth',False) else []
    globalL=registry.batch(test[:16],'L','new_global16_L') if
    ↪  policy.get('run_new_global_truth',False) else []
    states={cut:{'evaluated':{},'sequence':[],'status':'initial'} for cut in (1,3)}
    def save_row(cut,j,key):
        r=registry.records[key];states[cut]['evaluated'][j]=key;states[cut]['sequence'].appen
        ↪  d({'j':j,'prediction_index':16+cut*129+j,'key':key,'status':r['status'],'h_m':r.g
        ↪  et('target_max_h_m'),'valid':r['valid']})
    initial=[(cut,j) for cut in (1,3) for j in (0,64,128)]
    keys=registry.batch(np.array([test[16+c*129+j] for c,j in
    ↪  initial]),'H','critical_registered_initial6')
    for (cut,j),key in zip(initial,keys):save_row(cut,j,key)
    for cut,st in states.items():
        valid=all(registry.records[k]['valid'] for k in
        ↪  st['evaluated'].values());js=sorted(st['evaluated'])
```

```
        transitions=[(a,b) for a,b in zip(js[:-1],js[1:]) if
        ↪  (registry.records[st['evaluated'][a]]['status']=='arrived')!=(registry.records[st
        ↪  ['evaluated'][b]]['status']=='arrived')] if valid else
        ↪  []
        st['initial_transitions']=transitions;st['status']='unknown_initial' if not valid
        ↪  else 'no_bracket' if not transitions else 'bracket_found'
        if transitions:st['lo'],st['hi']=transitions[0];st['left_positive']=registry.records[
        ↪  st['evaluated'][st['lo']]]['status']=='arrived'
    for iteration in range(4):
        planned=[(cut,(st['lo']+st['hi'])//2) for cut,st in states.items() if
        ↪  st['status']=='bracket_found']
        if not planned:break
        keys=registry.batch(np.array([test[16+c*129+j] for c,j in
        ↪  planned]),'H',f'critical_bisection_round{iteration+1}')
        for (cut,j),key in zip(planned,keys):
            save_row(cut,j,key);r=registry.records[key];st=states[cut]
            if not r['valid']:st['status']='unknown_bisection'
            elif (r['status']=='arrived')==st['left_positive']:st['lo']=j
            else:st['hi']=j
    searches=[];endpointH=[];endpointL=[];endpoint_indices=[]
    for cut,st in states.items():
        final=[st['lo'],st['hi']] if st['status']=='bracket_found' else None
        if final:
            st['status']='multiple_initial_transitions' if len(st['initial_transitions'])>1
            ↪  else 'final_bracket'
            for j in final:endpointH.append(st['evaluated'][j]);endpoint_indices.append(16+cu
            ↪  t*129+j)
        zv=(.125,.375,.625,.875)[cut]
        searches.append({'slice':cut,'zV':zv,'volume_m3':2e5*4**zv,'initial_transitions':st['
        ↪  initial_transitions'],'final_j':final,'status':st['status'],'HF_queries':len(st['
        ↪  sequence']),'sequence':st['sequence'],'final_mu_interval':[.03+.12*j/128 for j in
        ↪  final] if final else None,'discontinuity_warning':'labels bracket a discrete
        ↪  change; not a smooth h crossing'})
    if endpoint_indices:endpointL=registry.batch(test[endpoint_indices],'L','critical_final_e
    ↪  ndpoints_L') if policy.get('run_endpoint_low',False) else
    ↪  [None]*len(endpoint_indices)
    write_json(out/'CRITICAL_SEARCH.json',searches)
    write_json(out/'QUERY_REQUESTS.json',registry.requests);write_json(out/'TEST_KEYS.json',{
    ↪  'global_H':globalH,'global_L':globalL,'endpoint_H':endpointH,'endpoint_L':endpointL,'
    ↪  endpoint_prediction_indices':endpoint_indices})
    evaluate_followup(out,policy,methods,registry,training,globalH,globalL,endpointH,endpoint
    ↪  L,endpoint_indices,searches)


def evaluate_followup(out,policy,methods,registry,training,gH,gL,eH,eL,indices,searches):
    import csv
    rows=[];boundary=[]
    for meta in methods:
        f=np.load(training/meta['folder']/'TEST_PREDICTIONS_FROZEN.npz')
        for label,H,L,ix in
        ↪  [('new_global',gH,gL,list(range(16))),('real_critical_endpoints',eH,eL,indices)]:
```

```
            good=[(h,l,i) for h,l,i in zip(H,L,ix) if
            ↪  registry.records[h]['valid']];y=np.array([registry.records[h]['response_g']
            ↪  for h,l,i in good]);p=np.array([f['mean_g'][i] for h,l,i in
            ↪  good]);actual=np.array([registry.records[h]['status']=='arrived' for h,l,i in
            ↪  good],dtype=bool);pred=p>=0
            TP=int((actual&pred).sum());FN=int((actual&~pred).sum());FP=int((~actual&pred).su
            ↪  m());TN=int((~actual&~pred).sum())
            rows.append({'seed':meta['seed'],'method':meta['method'],'stratum':label,'n':len(
            ↪  H),'valid_n':len(good),'unknown_n':len(H)-len(good),'TP':TP,'FN':FN,'FP':FP,'
            ↪  TN':TN,'disagreement_rate':float(np.mean(actual!=pred)) if len(good) else
            ↪  None,'arrival_miss_rate':FN/(TP+FN) if TP+FN else
            ↪  None,'RMSE_g':float(np.sqrt(np.mean((p-y)**2))) if len(good) else
            ↪  None,'RMSE_raw_margin_m':float(np.sqrt(np.mean((.2*np.exp(y)-.1-np.maximum(0,
            ↪  .2*np.exp(p)-.1))**2))) if len(good) else
            ↪  None})
        for search in searches:
            cut=search['slice'];means=f['mean_g'][16+cut*129:16+(cut+1)*129];js=np.flatnonzer
            ↪  o((means[:-1]>=0)!=(means[1:]>=0));roots=[]
            for j in
            ↪  js:roots.append(.03+.12*(j+abs(means[j])/(abs(means[j])+abs(means[j+1]))))/128)
            interval=search['final_mu_interval'];distance=max(interval[0]-roots[0],0,roots[0]
            ↪  -interval[1]) if roots and interval else
            ↪  None
            boundary.append({'seed':meta['seed'],'method':meta['method'],'slice':cut,'true_br
            ↪  acket':interval,'predicted_zero_crossings':roots,'mu_boundary_distance_to_tru
            ↪  e_bracket':distance,'prediction_has_no_crossing':not
            ↪  roots,'search_status':search['status']})
    with (out/'COMPARISON_METRICS.csv').open('w') as
    ↪  f:w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
    write_json(out/'BOUNDARY_LOCATION.json',boundary)
    write_json(out/'VALIDATION_COSTS.json',{'global_H':len(gH),'global_L':len(gL),'all_critic
    ↪  al_search_H':sum(s['HF_queries'] for s in searches),'critical_endpoint_L':sum(k is
    ↪  not None for k in eL),'actual_unique_queries':len(registry.records),'child_wall_sum_s
    ↪  ':sum(r['total_wall_s'] for r in
    ↪  registry.records.values()),'search_selection_cost_included':True})
    write_json(out/'EXPERIMENT_STATUS.json',{'methods':len(methods),'seeds':policy['seeds'],'
    ↪  new_global_coordinates':16,'new_global_points_actually_queried':len(gH),'critical_pot
    ↪  ential_coordinates':516,'actual_critical_H':sum(s['HF_queries'] for s in
    ↪  searches),'actual_endpoint_H':len(eH),'actual_endpoint_pairs':sum(k is not None for k
    ↪  in eL),'historical24_used_as_new_test':False,'test_returned_to_training':False,'indep
    ↪  endent_review':'pending'})


def seal_followup_subset(out,args):

    import shutil
    parent=Path(args.training_dir);policy_path=Path(args.policy);policy=json.loads(policy_pat
    ↪  h.read_text());validate_execution(policy,args,policy_path)
    if policy['seeds']!=[137]:raise ValueError('deadline137 subset only')
    methods=[m for m in json.loads((parent/'METHODS.json').read_text()) if m['seed']==137]
    if len(methods)!=4:raise ValueError('all four complete137 methods required')
    td=json.loads(bound_file(policy_path,policy['test_design']).read_text());test=np.r_[td['g
    ↪  lobal'],td['development_predicted_boundary']];keys=set();lineage=[]
    for meta in methods:
```

```
        old=parent/meta['folder'];folder=out/meta['folder'];shutil.copytree(old,folder);keys.update(meta['low_keys']+meta['high_keys'])
        lineage.append({'method':meta['method'],'origin_state':fp(old/'GP_STATE.json'),'state':fp(folder/'GP_STATE.json'),'origin_prediction':fp(old/'TEST_PREDICTIONS_FROZEN.npz'),'prediction':fp(folder/'TEST_PREDICTIONS_FROZEN.npz'),'refit':False})
    rows=[r for r in json.loads((parent/'physics/REGISTRY.json').read_text()) if r['key'] in keys]
    validate_prediction_models(methods,rows,test,policy,out);(out/'physics').mkdir();write_json(out/'physics/REGISTRY.json',rows);write_json(out/'METHODS.json',methods);write_json(out/'QUERY_REQUESTS.json',[])
    write_json(out/'SUBSET_MIGRATION.json',{'parent_source':fp(parent/'SOURCE_SNAPSHOT.py'),'source':fp(__file__),'model_seeds':[137],'excluded_partial_seeds':[271,503],'lineage':lineage,'new_PDE':0,'refit':0,'reason':'18:40 deadline, scope selected before critical truth'})
    def local(p):
        p=Path(p);r=fp(p);r['path']=str(p.relative_to(out));return r
    write_json(out/'PHASE1_SEAL.json',{'phase':'training_complete_test_not_run','source':fp(__file__),'policy':fp(policy_path),'solver':solver_identity(),
        'predictions':[local(out/m['folder']/'TEST_PREDICTIONS_FROZEN.npz') for m in methods],'model_states':[local(out/m['folder']/'GP_STATE.json') for m in methods],
        'methods':local(out/'METHODS.json'),'registry':local(out/'physics/REGISTRY.json'),'requests':local(out/'QUERY_REQUESTS.json'),'migration':local(out/'SUBSET_MIGRATION.json')})


def decision_request(args):

    if args.full or args.dx!=30:raise ValueError('decision publication supports local30m only')
    config={'volume_m3':float(args.volume),'duration_s':float(args.duration),'mu':float(args.mu),'xi_m_s2':float(args.xi),
            'speed_m_s':float(args.speed),'entry_width_m':float(args.entry_width),'shape':args.shape,'terrain_name':args.terrain_name}
    if not all(np.isfinite(config[k]) for k in ['volume_m3','duration_s','mu','xi_m_s2','speed_m_s','entry_width_m']):raise ValueError('finite request required')
    body={'config':config,'numerics':{**{k:float(getattr(args,k)) for k in ['dx','horizon','cfl','dry','maxdt']},'order':int(args.order),'backend':str(args.backend)},
          'inputs':[{'logical_id':n,'sha256':fp(Path(args.inputs)/n)['sha256']} for n in ['copernicus_support.npz','aster_support.npz','TERRAIN.json','mapped_path_utm.csv']],
          'science':solver_identity(),'target':{'axis':'x','coordinate_m':348720,'tangent_interval_m':[3134880,3135600],
          'criterion':'instantaneous max vertical event depth >= threshold on fixed interpolated line','threshold_m':.1,'horizon_s':float(args.horizon),'full_reach':False}}
    return {'request_id':hashlib.sha256(json.dumps(body,sort_keys=True).encode()).hexdigest(),**body}


def physical_evidence_check(request,result_path,receipt_path,source_certificate):

    d=json.loads(Path(result_path).read_text());receipt=json.loads(Path(receipt_path).read_text())
```

```
    if receipt['failure'] is not None or not receipt['source_unchanged_during_run']:raise
    ↪  ValueError('failed or changed-source physical run')
    if receipt['code']['sha256']!=source_certificate['source']['sha256'] or
    ↪  source_certificate['science']!=request['science']:raise ValueError('physical
    ↪  scientific identity mismatch')
    if not any(Path(r['path']).name==Path(result_path).name and
    ↪  r['sha256']==fp(Path(result_path))['sha256'] for r in receipt['outputs']):raise
    ↪  ValueError('result not bound by actual run receipt')
    inputs={Path(i['path']).name:i['sha256'] for i in receipt['inputs']}
    if any(inputs.get(i['logical_id'])!=i['sha256'] for i in request['inputs']):raise
    ↪  ValueError('physical input asset mismatch')
    argv=receipt['argv'];c=request['config'];n=request['numerics']
    numeric={'--volume':c['volume_m3'],'--duration':c['duration_s'],'--mu':c['mu'],'--xi':c['
    ↪  xi_m_s2'],'--speed':c['speed_m_s'],
          '--entry-width':c['entry_width_m'],'--dx':n['dx'],'--horizon':n['horizon'],'--cf
          ↪  l':n['cfl'],'--dry':n['dry'],'--order':n['order'],'--maxdt':n['maxdt']}
    for flag,value in numeric.items():
        if flag not in argv or float(argv[argv.index(flag)+1])!=value:raise
        ↪  ValueError('physical config/numerics mismatch: '+flag)
    for flag,value in [('--backend',n['backend']),('--shape',c['shape']),('--terrain-name',c[
    ↪  'terrain_name'])]:
        if flag not in argv or argv[argv.index(flag)+1]!=value:raise ValueError('physical
        ↪  mode mismatch: '+flag)
    if '--full' in argv or d['full_reach']:raise ValueError('physical domain scope mismatch')
    support=d['arrival_support'];target=request['target']
    if support['axis']!=target['axis'] or support['coordinate_m']!=target['coordinate_m'] or
    ↪  support['tangent_interval_m']!=target['tangent_interval_m']:raise
    ↪  ValueError('physical target support mismatch')
    if not d['whole_window_valid'] or not np.isfinite(d['valid_until_s']) or
    ↪  d['valid_until_s']<n['horizon']:raise ValueError('domain/time window invalid;
    ↪  unknown')
    for field in ['mass_error_rel','momentum_error_rel']:
        if not np.isfinite(d[field]) or abs(d[field])>1e-8:raise ValueError('physical budget
        ↪  check failed: '+field)
    if not np.isfinite(d['target_max_h']) or d['target_max_h']<0 or not
    ↪  np.isfinite(d['min_h']) or d['min_h']<0:raise ValueError('invalid physical thickness')
    arrival=d['threshold_arrivals']['0.1'];status=arrival['status'];bracket=arrival['time_bra
    ↪  cket_s']
    if status=='arrived':
        if d['target_max_h']<.1 or bracket is None or not
        ↪  0<=bracket[0]<=bracket[1]<=n['horizon']:raise ValueError('arrival
        ↪  criterion/bracket inconsistent')
    elif status=='valid_no_arrival_by_horizon':
        if d['target_max_h']>=.1 or bracket is not None:raise ValueError('no-arrival
        ↪  criterion inconsistent')
    else:raise ValueError('no valid physical class; unknown')
    return {'status':status,'time_bracket_s':bracket,'target_max_h_m':d['target_max_h'],'resu
    ↪  lt':fp(Path(result_path)),'receipt':fp(Path(receipt_path))}


def bind_decision_cache(out,args):

    import importlib.util,sys,shutil
```

```
    request=decision_request(args);spec=importlib.util.spec_from_file_location('guard_evidenc
    ↪  e_source',args.evidence_source);prior=importlib.util.module_from_spec(spec);sys.modul
    ↪  es[spec.name]=prior;spec.loader.exec_module(prior)
    certificate={'source':fp(args.evidence_source),'science':prior.solver_identity(),'binding
    ↪  _source':fp(__file__)}
    checked=physical_evidence_check(request,args.evidence_result,args.evidence_receipt,certif
    ↪  icate)
    evidence=out/'evidence';evidence.mkdir();shutil.copy2(args.evidence_result,evidence/'pilo
    ↪  t_30.json');shutil.copy2(args.evidence_receipt,evidence/'RUN_RECEIPT.json')
    entry={'schema':'strict-physical-verification-cache-v1','request':request,'source_certifi
    ↪  cate':certificate,
           'result':{'package_path':'evidence/pilot_30.json','sha256':fp(evidence/'pilot_30.j
           ↪  son')['sha256']},
           'receipt':{'package_path':'evidence/RUN_RECEIPT.json','sha256':fp(evidence/'RUN_RE
           ↪  CEIPT.json')['sha256']},'bound_physical_status':checked['status']}
    write_json(out/'CACHE_ENTRY.json',entry)


def load_decision_cache(path,request):
    path=Path(path);entry=json.loads(path.read_text())
    if entry['schema']!='strict-physical-verification-cache-v1' or
    ↪  entry['request']!=request:raise ValueError('cache request identity mismatch')
    resolved=[]
    for field in ('result','receipt'):
        relative=Path(entry[field]['package_path']);p=path.parent/relative
        if relative.is_absolute() or not
        ↪  p.resolve().is_relative_to(path.parent.resolve()):raise ValueError('cache package
        ↪  path unsafe')
        if not p.is_file() or fp(p)['sha256']!=entry[field]['sha256']:raise ValueError('cache
        ↪  file absent or changed: '+field)
        resolved.append(p)
    return physical_evidence_check(request,*resolved,entry['source_certificate'])


def run_decision(out,args):
    import datetime
    started=time.perf_counter();request=decision_request(args);estimate={'status':'not_availa
    ↪  ble','authoritative':False};reason=None;verified=None;kind='none';fresh_cost=None
    if args.gp_state is not None:
        state=json.loads(args.gp_state.read_text());c=request['config']
        in_scope=(2e5<=c['volume_m3']<=8e5 and .03<=c['mu']<=.15 and c['duration_s']==960 and
        ↪  c['xi_m_s2']==500 and c['speed_m_s']==30 and c['entry_width_m']==30 and
        ↪  c['terrain_name']=='copernicus' and c['shape']=='triangle' and args.horizon==1800
        ↪  and args.cfl==.35 and args.dry==1e-4 and args.order==2 and args.maxdt==2)
        if in_scope:
            model=load_fixed_gp(state);x=np.array([[np.log(c['volume_m3']/2e5)/np.log(4),(c['
            ↪  mu']-.03)/.12]]);m,s=model.predict(x)
            estimate={'status':'arrived' if m[0]>=0 else
            ↪  'valid_no_arrival_by_horizon','mean_g':float(m[0]),'std_g':float(s[0]),'model
            ↪  _state':fp(args.gp_state),'authoritative':False}
        else:estimate={'status':'outside_frozen_GP_scope','authoritative':False,'model_state'
        ↪  :fp(args.gp_state)}
    write_json(out/'REQUEST_AND_GP_FROZEN.json',{'frozen_utc':datetime.datetime.now(datetime.
    ↪  timezone.utc).isoformat(),'source':fp(__file__),'request':request,'GP_estimate':estim
    ↪  ate,'physical_truth_accessed_yet':False})
```

```
    if args.cache_entry is not None:
        try:verified=load_decision_cache(args.cache_entry,request);kind='exact_fingerprint_ca
        ↪  che'
        except Exception as exc:reason=type(exc).__name__+": "+str(exc)
    if verified is None and not args.cache_only:
        try:
            row=execute_physics(out/'physical_verification',request['config'],args,30.)
            write_json(out/'PHYSICAL_CALL.json',row);fresh_cost={'child_wall_s':row['total_wa
            ↪  ll_s'],'solver_wall_s':row.get('solver_wall_s'),'compile_and_load_s':row.get(
            ↪  'compile_and_load_s')}
            if row.get('valid'):
                try:verified=physical_evidence_check(request,row['result']['path'],row['recei
                ↪  pt']['path'],{'source':fp(__file__),'science':solver_identity()});kind='f
                ↪  resh_30m_PDE'
                except (ValueError,KeyError,OSError,TypeError) as exc:reason=str(exc)
            else:reason='physical query failed or invalid domain/window; no class published'
        except Exception as exc:
            reason="physical verification exception: "+type(exc).__name__+": "+str(exc)
            write_json(out/"VERIFICATION_EXCEPTION.json",{"error_type":type(exc).__name__,"me
            ↪  ssage":str(exc),"GP_fallback_forbidden":True})
    final={'request':request,'GP_estimate':estimate,'final_status':verified['status'] if
    ↪  verified else 'unknown','needs_verification':verified is None,
           'arrival_time_bracket_s':verified['time_bracket_s'] if verified else
           ↪  None,'verification_source':kind,'physical_evidence':verified,
           'unverified_reason':reason if not verified else None,'fresh_physical_cost':fresh_c
           ↪  ost,'decision_wall_s':time.perf_counter()-started,
           'published_from_GP':False,'scope':'D_ONLY conditional physical-model verification;
           ↪  not natural-event zero-error guarantee'}
    write_json(out/'DECISION.json',final)


def sol():
    import argparse,sys,platform,datetime,importlib.metadata
    ap=argparse.ArgumentParser();ap.add_argument('mode',choices=['bench','terrain','pilot','s
    ↪  mooth','morris','freeze','near','perturb','train','test','global','inverse','baseline
    ↪  ','join','seal-cache','validate-new','seal-subset','decide','bind-cache']);ap.add_arg
    ↪  ument('--out',type=Path,required=True);ap.add_argument('--inputs',type=Path);ap.add_a
    ↪  rgument('--dx',type=float,default=30);ap.add_argument('--dry',type=float,default=1e-4
    ↪  );ap.add_argument('--order',type=int,choices=[1,2],default=2);ap.add_argument('--back
    ↪  end',choices=['numpy','c'],default='numpy');ap.add_argument('--workers',type=int,choi
    ↪  ces=[1,2,3],default=2);ap.add_argument('--morris-dir',type=Path);ap.add_argument('--p
    ↪  olicy',type=Path);ap.add_argument('--training-dir',type=Path);ap.add_argument('--prev
    ↪  ious-design',type=Path);ap.add_argument('--full-inputs',type=Path);ap.add_argument('-
    ↪  -q3-doc',type=Path);ap.add_argument('--q4-doc',type=Path);ap.add_argument('--entity-r
    ↪  oot',type=Path);ap.add_argument('--allow-synthetic',action='store_true')
```

```
ap.add_argument('--gp-state',type=Path);ap.add_argument('--cache-entry',type=Path);ap.add
↪  _argument('--cache-only',action='store_true');ap.add_argument('--evidence-result',typ
↪  e=Path);ap.add_argument('--evidence-receipt',type=Path);ap.add_argument('--evidence-s
↪  ource',type=Path);ap.add_argument('--cache-registry',type=Path);ap.add_argument('--en
↪  try-width',type=float,default=30.);ap.add_argument('--full',action='store_true');ap.a
↪  dd_argument('--volume',type=float,default=8e5);ap.add_argument('--duration',type=floa
↪  t,default=960);ap.add_argument('--speed',type=float,default=30);ap.add_argument('--mu
↪  ',type=float,default=.05);ap.add_argument('--xi',type=float,default=500);ap.add_argum
↪  ent('--horizon',type=float,default=1800);ap.add_argument('--cfl',type=float,default=.
↪  35);ap.add_argument('--maxdt',type=float,default=2);ap.add_argument('--terrain-name',
↪  default='copernicus',choices=['copernicus','aster']);ap.add_argument('--shape',defaul
↪  t='triangle',choices=['triangle','double']);args=ap.parse_args()
global DRY,ORDER,FAST
DRY=args.dry;ORDER=args.order
args.out.mkdir(parents=True,exist_ok=False);start=datetime.datetime.now(datetime.timezone
↪  .utc).isoformat();code_before=fp(__file__)
(args.out/'SOURCE_SNAPSHOT.py').write_bytes(Path(__file__).read_bytes())
root=Path(__file__).resolve().parents[3] if 'work/q4/' in str(Path(__file__).resolve())
↪  else Path.cwd()

compile_metadata=None
inputs=[];error=None
try:
    if args.inputs:inputs=[fp(p) for p in sorted(args.inputs.iterdir()) if p.is_file()]
    if args.backend=='c' and args.mode not in
    ↪  ('morris','freeze','near','perturb','train','test','global','inverse','baseline',
    ↪  'join','seal-cache','validate-new','seal-subset','decide','bind-cache','terrain'):
        tic=time.perf_counter();FAST=FastFlux(args.out/'cbuild');FAST.dry=DRY
        compile_metadata={'C_source_sha256':FAST.sha,'library_sha256':FAST.library_sha256
        ↪  ,'compile_and_load_s':time.perf_counter()-tic,'actual_compile_command':FAST.c
        ↪  ommand,'fast_math':False}
    if args.mode=='bench':benchmarks(args.out)
    elif args.mode=='smooth':smooth_benchmark(args.out)
    elif args.mode=='morris':run_morris(args.out,args)
    elif args.mode=='freeze':freeze_formal_policy(args.out,args)
    elif args.mode=='near':run_near_boundary(args.out,args)
    elif args.mode=='perturb':run_perturbations(args.out,args)
    elif args.mode=='train':run_formal_training(args.out,args)
    elif args.mode=='test':run_formal_test(args.out,args)
    elif args.mode=='decide':run_decision(args.out,args)
    elif args.mode=='bind-cache':bind_decision_cache(args.out,args)
    elif args.mode=='seal-subset':seal_followup_subset(args.out,args)
    elif args.mode=='validate-new':run_followup_validation(args.out,args)
    elif args.mode=='seal-cache':seal_cached_seed(args.out,args)
    elif args.mode=='global':run_global_cases(args.out,args)
    elif args.mode=='inverse':run_synthetic_inverse(args.out,args)
    elif args.mode=='baseline':write_json(args.out/'AXIS_BASELINE.json',along_axis_baseli
    ↪  ne(args.inputs,args.full,args.mu,args.xi,horizon=args.horizon))
    elif args.mode=='join':write_json(args.out/'Q3_Q4_JOIN.json',strict_q3_q4_join(json.l
    ↪  oads(args.q3_doc.read_text()),json.loads(args.q4_doc.read_text()),args.entity_roo
    ↪  t,args.allow_synthetic))
    elif args.mode=='terrain':terrain(args.out,root,args.full)
    else:pilot(args.out,args.inputs,args.dx,args)
except Exception as exc:
```

```
        error={'type':type(exc).__name__,'message':str(exc)};raise
    finally:
        write_json(args.out/'RUN_RECEIPT.json',{'started_at':start,'finished_at':datetime.dat
        ↪  etime.now(datetime.timezone.utc).isoformat(),'argv':sys.argv,'code':code_before,'
        ↪  code_after':fp(__file__),'acceleration':compile_metadata,'source_unchanged_during
        ↪  _run':code_before==fp(__file__),'inputs':inputs,'environment':{'python':sys.versi
        ↪  on,'platform':platform.platform(),'versions':runtime_versions()},'failure':error,
        ↪  'outputs':[fp(p) for p in sorted(args.out.iterdir()) if p.is_file() and
        ↪  p.name!='RUN_RECEIPT.json'],'scientific_approval':'pending'})


if __name__=='__main__':sol()
```

q4/followup/code/draw.py

```
from pathlib import Path
import argparse,json,hashlib
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def sol():
 ap=argparse.ArgumentParser();ap.add_argument('--root',type=Path,required=True);ap.add_argume
 ↪  nt('--out',type=Path,required=True);args=ap.parse_args();args.out.mkdir(parents=True,exi
 ↪  st_ok=True)
 plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':['Arial','Helvetica','Deja
 ↪  Vu
 ↪  Sans'],'font.size':9,'pdf.fonttype':42,'svg.fonttype':'none','axes.spines.top':False,'ax
 ↪  es.spines.right':False,'legend.frameon':False})
 inputs=[];outputs=[]
 def read(path):
  p=args.root/path;inputs.append({'path':path,'sha256':hashlib.sha256(p.read_bytes()).hexdige
  ↪  st()});return
  ↪  p
 def save(fig,name):
  for ext in ['pdf','svg','png']:
   p=args.out/(name+'.'+ext);fig.savefig(p,dpi=600,bbox_inches='tight');outputs.append({'path
   ↪  ':p.name,'sha256':hashlib.sha256(p.read_bytes()).hexdigest()})
  plt.close(fig)
 searches=json.loads(read('results/critical/CRITICAL_SEARCH.json').read_text());models=json.l
 ↪  oads(read('results/model_seal/METHODS.json').read_text())
 colors=['#777777','#4477AA','#66AABB','#AA3377'];labels={'A_single_fill':'Single
 ↪  GP','B_MF_fill':'MF passive','Bm_MF_count_matched':'MF matched','C_MF_active':'MF
 ↪  active'}
 for search in searches:
  fig,ax=plt.subplots(figsize=(7.2,4.2),layout='constrained');cut=search['slice'];mu=.03+.12*
  ↪  np.arange(129)/128
  for meta,color in zip(models,colors):
   f=np.load(read('results/model_seal/'+meta['folder']+'/TEST_PREDICTIONS_FROZEN.npz'));ax.pl
   ↪  ot(mu,f['mean_g'][16+cut*129:16+(cut+1)*129],color=color,lw=1.2,label=labels[meta['met
   ↪  hod']])
```

```
  valid=[r for r in search['sequence'] if r['valid']];xs=[.03+.12*r['j']/128 for r in
  ↪  valid];ys=[np.log1p((r['h_m']-.1)/.2) for r in valid]
  ax.scatter(xs,ys,s=26,color='black',zorder=5,label='Queried 30 m PDE')
  interval=search['final_mu_interval']
  if interval:
   for value in interval:ax.axvline(value,color='#999999',ls=':',lw=.8)
  ax.axhline(0,color='#999999',ls='--',lw=.8);ax.set(xlabel='Coulomb coefficient
  ↪  μ',ylabel='Log depth margin g',xlim=(.025,.155));ax.legend(loc='upper right',fontsize=8)
  save(fig,'critical_slice'+str(cut))
 f=np.load(read('results/full_diagnostics/FULL_COMMON_ENERGY_CURVES.npz'));fig,ax=plt.subplot」
 ↪  s(figsize=(7.2,4.2),layout='constrained')
 reference=8825015023.410414
 for name,color in zip(['copernicus_30','copernicus_40','aster_40','double_40'],colors):
  a=f[name];ax.plot(a[:,0],100*a[:,3]/reference,lw=1.2,label=name.replace('_','
  ↪  '),color=color)
 ax.axvline(960,color='#999999',ls=':',lw=.8);ax.set(xlabel='Time after prescribed entry
 ↪  (s)',ylabel='Energy ledger residual / fixed input (%)');ax.legend(loc='upper
 ↪  left',fontsize=8)
 save(fig,'full_energy_normalized')
 (args.out/'PLOT_RECEIPT.json').write_text(json.dumps({'source_sha256':hashlib.sha256(Path(__」
 ↪  file__).read_bytes()).hexdigest(),'inputs':inputs,'outputs':outputs,'PDE_ran':False,'GP_」
 ↪  fitted':False,'alignment':'single plot-area per figure; not
 ↪  applicable'},indent=2))
if __name__=='__main__':sol()
```